

PDI&VC

Processamento Digital de Imagens e Visão Computacional

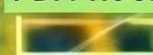
SUNFLOWER | B 86 CONF. →



SUNFLOWER | 0.98 CONF. (PDI->VC)



PDI PROCESS | EDGE DET.



GIRASSOL | 95% (VC USE)



Fundamentos, Algoritmos
e Aplicações Integradas

AUTOR: FRANCISCO DE ASSIS ZAMPIROLI



Processamento Digital de Imagens e Visão Computacional

Livro interativo com Python

Francisco de Assis Zampiroli

Sumário

PDI e VC — Python	1
Organização	1
Ficha Catalográfica	2
Prefácio	3
Contexto desta edição	3
Como este livro é produzido	3
Idiomas e Conversão de Código	4
Uso de ferramentas de Inteligência Artificial	4
A biblioteca <code>morph.py</code>	5
Exercícios de Programação (EPs) e Validação Automática	6
Código aberto	6
Guia do Professor: Publicação de EPs no Moodle	7
Integração com o Moodle (VPL)	7
Como Publicar no Moodle	7
Antes de começar: <i>Notebooks</i> em Python	8
I Parte I — Fundamentos de PDI	1
1 Fundamentos e Primeiros Passos	2
1.1 Objetivos	2
1.2 Antes de começar: <i>Notebooks</i> em Python	2
1.3 Fundamentos	2
1.3.1  Visão Computacional	3
1.3.2  Processamento Digital de Imagens (PDI)	3
1.4 Etapas do PDI	4
1.5 Formação da Imagem e o Espectro	4
1.6 O que é uma Imagem Digital?	7
Representação Matemática	7
1.7 Configuração do Ambiente	8
1.8 Fundamentos de Matrizes - atenção à cópia de referências	9
1.9 Lendo e Exibindo Imagens	10
Alternativa: <i>Download</i> da Imagem para Armazenamento Local	11
1.10 Conversão de Tipos e Limiarização	12
Conversão para Tons de Cinza	12
Limiarização (<i>Thresholding</i>)	12
Exemplo prático de conversão e limiarização	12
1.11 Limiarização pelo método de Otsu	13
1.12 Acesso a Pixels	14
1.13 Resumo	16
1.14  Uso do Gemini Notebook como Tutor Complementar	16
Funcionalidades Disponíveis na Plataforma	16
1.15 Lista de Exercícios	17
Referências do Capítulo	18

1.16		Parte Prática com Exercícios de Programação	18
		Objetivo deste Caderno	18
	§	Instruções Passo a Passo	18
		Importante: Regras e Boas Práticas	20
1.16.1	EP01_01 	Três métricas de distância em PDI	20
1.16.2	EP01_02 	Desempenho Preditivo — Métricas de ML em VC	27
1.16.3	EP01_03 	<i>Mean Average Precision</i> (mAP) — Curva Precisão-Sensibilidade	29
1.16.4	EP01_04 	Leitura e Informações de uma Imagem em Matriz	33
1.16.5	EP01_05 	Negativo de uma Imagem em Tons de Cinza	34
1.16.6	EP01_06 	Conversão RGB → Tons de Cinza (ITU-R BT.601)	36
1.16.7	EP01_07 	Limiarização Manual: Imagem Binária	38
1.16.8	EP01_08 	Remapeamento por Faixa de Intensidade	39
1.16.9	EP01_09 	Padrão de Tabuleiro: Matriz de Xadrez	42
1.16.10	EP01_10 	Metadados: Leitura de Arquivo PGM	43
1.16.11	EP01_11 	Análise de Vizinhaça: Filtro de Máximo 1D	45
2		Da Captura ao Pixel - Amostragem, Quantização e Conectividade	48
2.1		Objetivos	48
2.2		O Olho e a Câmera - Elementos da Percepção Visual	48
	2.2.1	Visão humana	48
	2.2.2	Sensores digitais	48
2.3		Ilusões de Ótica: Os Desafios da Percepção Visual	49
	2.3.1	Ambiguidade e Contexto	49
	2.3.2	Brilho e Contraste Local	50
	2.3.3	Geometria e Perspectiva	50
2.4		O Modelo Matemático da Formação da Imagem	50
	2.4.1	Representação Digital e Canais Espectrais	51
	2.4.2	Preparando o Ambiente Prático	52
	2.4.3	Implementação da Leitura de Imagem em <code>morph.py</code>	52
	2.4.4	RGB e RGBA: Exemplo Prático com a Imagem Mandrill	53
2.5		Digitalização: Amostragem e Quantização	54
	2.5.1	Amostragem - Discretização do espaço	54
	2.5.2	Quantização - Discretização da intensidade	55
	2.5.3	Efeitos da amostragem e quantização	55
	2.5.4	Análise Técnica	57
2.6		Relações entre Pixels - Topologia da Imagem	58
	2.6.1	Vizinhaça	58
	2.6.2	Adjacência, conectividade e caminhos	58
	2.6.3	Distâncias entre pixels	59
2.7		Armazenamento de Imagens	59
	2.7.1	Exemplo: Extração de Metadados e Localização GPS	60
	2.7.2	Por que esta separação é importante?	62
2.8		Transformações Geométricas Básicas	62
	2.8.1	Translação	63
	2.8.2	Rotação	64
	2.8.3	Escala (Redimensionamento)	65
	2.8.4	Cisalhamento (<i>Shear</i>)	66
2.9		Resumo	67
2.10		Uso do Gemini Notebook como Tutor Complementar	67
2.11		Lista de Exercícios	68
		Referências do Capítulo	68
2.12		Parte Prática com Exercícios de Programação	68
		Objetivo deste Caderno	68
	2.12.1	EP02_01  Ajuste de Brilho e Contraste	69

2.12.2	EP02_02	 Subamostragem Espacial	71
2.12.3	EP02_03	 Quantização de Níveis de Cinza	73
2.12.4	EP02_04	 Transformada de Distância em Imagem Binária	75
2.12.5	EP02_05	Translação de Imagem	77
2.12.6	EP02_06	 Rotação de Imagem	78
2.12.7	EP02_07	 Redimensionamento (Escala)	80
2.12.8	EP02_08	 Cisalhamento (Shear)	84
2.12.9	EP02_09	 Transformação Afim Genérica	85
2.12.10	EP02_10	 Correção de Perspectiva (Homografia)	89
2.12.11	EP02_11	 Correção de Perspectiva (Homografia) em Imagem Real	91
3	Operações Espaciais: Intensidade, Histograma e Filtragem		96
3.1	Objetivos		96
3.2	Operações em Nível de Intensidade		96
3.2.1	Preparando o Ambiente Prático		97
3.2.2	Operações Aritméticas		98
3.2.3	Mistura Ponderada (<i>Alpha Blending</i>)		99
3.2.4	Operações Lógicas e Máscaras Bit a Bit		101
3.3	Histograma de Imagens		102
3.3.1	Equalização de Histograma		103
3.3.2	Especificação de Histograma		107
3.4	Fundamentos Espaciais: Vizinhança, Convolução e <i>Kernels</i>		108
3.4.1	Vizinhança		108
3.4.2	Tratamento de Bordas		109
3.4.3	Correlação e Convolução		111
3.4.4	Correlação vs. Convolução no OpenCV		111
3.4.5	O Papel do <i>Kernel</i>		113
3.4.6	Exemplo Numérico: Correlação Passo a Passo		115
3.5	Filtragem Espacial de Suavização		116
3.5.1	Filtro de Média (<i>Box Filter</i>)		116
3.5.2	Filtro Gaussiano		117
3.6	Filtragem Espacial de Realce		119
3.6.1	Laplaciano		120
3.6.2	Operador de Sobel		123
3.6.3	Operador de Prewitt		125
3.6.4	<i>Unsharp Masking</i> (USM)		126
3.6.5	Detector de Canny		128
3.7	Filtros de Ordem: Filtro da Mediana		129
3.7.1	Ruído Impulsivo: Sal e Pimenta		129
3.7.2	Filtro da Mediana		130
3.8	Aplicação Prática: Pré-processamento para Segmentação		133
3.9	Resumo		134
3.10	 Uso do Gemini Notebook como Tutor Complementar		134
3.11	Lista de Exercícios		135
	Referências do Capítulo		136
3.12	 Parte Prática com Exercícios de Programação		136
	 Objetivo deste Caderno		136
3.12.1	EP03_01	 Adição Saturada de Constante	137
3.12.2	EP03_02	 <i>Alpha Blending</i> de Duas Imagens	139
3.12.3	EP03_03	 Inversão de Imagem (Negativo Fotográfico)	140
3.12.4	EP03_04	 Equalização de Histograma (L bits)	142
3.12.5	EP03_05	 Aplicação de Máscara AND Binária	143
3.12.6	EP03_06	Filtro de Média com <i>Kernel</i> $N \times N$	146
3.12.7	EP03_07	 Operador Laplaciano (w4) para Realce de Bordas	149

3.12.8	EP03_08		Gradiente de Sobel: Gx e Gy	150
3.12.9	EP03_09		Filtro da Mediana 3×3	154
3.12.10	EP03_10		<i>Unsharp Masking</i> (USM)	155
4	Morfologia Matemática e Segmentação de Imagens			159
4.1	Objetivos			159
4.2	Limiarização			160
4.2.1	Imagem de Moedas			161
4.2.2	Escolha do Pré-processamento para o Otsu			162
4.2.3	Resultado: CLAHE como Melhor Pré-processamento			164
4.3	Morfologia Matemática			165
4.3.1	Erosão e Dilatação			165
4.3.2	Abertura e Fechamento			172
4.3.3	Operadores Geodésicos			175
4.3.4	Reconstrução Morfológica			179
4.3.5	Preenchimento de Buracos e Remoção de Bordas			181
4.3.6	<i>Pipeline</i> de Limpeza Binária com CLAHE			184
4.3.7	Morfologia em Tons de Cinza			185
4.4	Segmentação de Imagens: Fundamentação e Taxonomia			188
4.4.1	Rotulação			189
4.4.2	Transformada de Distância			193
4.4.3	Transformada de Distância Euclidiana em quatro passos			196
4.4.4	Segmentação por <i>Watershed</i>			199
4.5	Extração de Componentes e Descritores de Forma			204
4.5.1	Rotulação e Estatísticas de Componentes			205
4.5.2	Descritores de Forma			206
4.5.3	Conexão com Detecção de Objetos Moderna			208
4.6	Resumo			210
4.7	 Uso do Gemini Notebook como Tutor Complementar			211
4.8	Lista de Exercícios			211
	Referências do Capítulo			212
4.9	 Parte Prática com Exercícios de Programação			212
	 Objetivo deste Caderno			213
4.9.1	EP04_01		Limiarização Global por Limiar Fixo	213
4.9.2	EP04_02		Limiarização Automática de Otsu	215
4.9.3	EP04_03		Dilatação Binária Plana (mm.dil0)	218
4.9.4	EP04_04		Erosão Binária Plana (mm.ero0)	219
4.9.5	EP04_05		Abertura Morfológica (Remoção de Ruído)	223
4.9.6	EP04_06		Fechamento Morfológico (Preenchimento de Falhas)	225
4.9.7	EP04_07		Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)	227
4.9.8	EP04_08		Gradiente Morfológico, Top-hat e Black-hat	229
4.9.9	EP04_09		Transformada de Distância e o “Miolo” do Objeto	231
4.9.10	EP04_10		Separação de <i>Blobs</i> , Rotulação e Descritores	232
II	Parte II — Visão Computacional			236
5	Análise de Documentos e Inspeção Industrial			237
5.1	Objetivos do Capítulo			237
5.2	Configuração do Ambiente			238
5.3	Bases de Imagens para Experimentação			239
5.3.1	Imagens Públicas com <code>skimage.data</code>			239
5.4	Normalização de Fundo e Equalização Local de Contraste			241
5.5	Reconhecimento Óptico de Caracteres (OCR)			243

5.6	Tradução Automática do Texto Reconhecido	246
5.7	Fundamentos de OMR e Inspeção Industrial	248
5.8	Projetos Práticos: Construção de um <i>Pipeline</i> de Análise Documental	248
5.8.1	Alinhamento Automático de Documentos (<i>OCR/OMR Pre-processing</i>)	249
5.8.2	Algoritmo de Retificação de Inclinação (<i>Deskew</i>)	250
5.8.3	Modelagem Matemática	250
5.8.4	Limitações Práticas	254
5.8.5	Detecção de Bordas e Contornos	255
5.8.6	Isolamento, Segmentação e Decodificação do <i>QRCode</i>	257
5.8.7	Decodificação de Códigos de Barras	262
5.9	O MCTest como Estudo de Caso: do Protótipo ao Sistema em Produção	263
5.9.1	Obtenção e Preparação do Módulo	263
5.9.2	Extração da Área de Respostas	264
5.9.3	Segmentação do <i>QRCode</i>	266
5.9.4	Localização dos Quadros de Respostas	268
5.9.5	Leitura Automática das Respostas	268
5.10	Inspeção Industrial Automatizada	270
5.10.1	Referências e <i>Datasets</i> Públicos	271
5.10.2	Detecção de Defeitos por Análise de Textura	272
5.11	Resumo	275
	Próximos Passos	276
5.12	 Uso do Gemini Notebook como Tutor Complementar	276
5.13	Lista de Exercícios	277
	Referências do Capítulo	277
5.14	 Parte Prática com Exercícios de Programação	278
	Legenda de Dificuldade	278
	 Objetivo deste Caderno	279
5.14.1	EP06_01  Avaliação de Segmentação por IoU (<i>Intersection over Union</i>)	280
5.14.2	EP06_02  Filtro de Marcadores por Circularidade	283
5.14.3	EP06_03  Classificação de Marcações em Folhas de Resposta (OMR)	285
5.14.4	EP06_04  Estimador de Inclinação por Mediana Angular (<i>Deskew</i>)	287
5.14.5	EP06_05  Normalização de Fundo por Divisão (Correção de Iluminação)	288
5.14.6	EP06_06  Mapa de Variância Local para Detecção de Textura	290
5.14.7	EP06_07  <i>Pipeline</i> de Inspeção Industrial: Registro por Translação e Subtração	292
5.14.8	EP06_08  Segmentação e Decodificação Real de <i>QRCode</i> com OpenCV	294
6	Classificação de Imagens e Reconhecimento de Padrões	299
6.1	Objetivos do Capítulo	299
6.2	Configuração do Ambiente	299
6.3	Um Problema Concreto: Classificação de Frutas	301
6.4	Panorama do Capítulo: O <i>Pipeline</i> Clássico de Classificação de Imagens	303
6.5	Fundamentos de Reconhecimento de Padrões	303
6.6	Extração de Descritores Clássicos	304
6.6.1	<i>Local Binary Patterns</i> (LBP)	305
6.6.2	<i>Histogram of Oriented Gradients</i> (HOG)	307
6.6.3	O Impacto da Escala e a Normalização de Características	309
6.7	 Mapa Conceitual	311
6.8	Descritores na Prática	311
6.9	Um Problema Concreto: Simulando Descritores de Frutas	316
6.10	Classificador k-NN: Como Funciona por Dentro	319
6.10.1	Métrica de Distância	320
6.10.2	Regra de Decisão	320
6.10.3	O Papel do Parâmetro k	322
6.11	Projeto Prático 1: Classificação de Dígitos Manuscritos com k-NN	323

6.11.1	Classificação com Vetores de Intensidade	324
6.11.2	Classificação com Descritores HOG	325
6.12	Avaliação de Classificadores	327
6.12.1	Escolha de k por Validação Cruzada	327
6.12.2	O Compromisso entre Viés e Variância: Diagnóstico de <i>Overfitting</i> e <i>Underfitting</i>	328
6.13	Projeto Prático 2: Comparação de Descritores para Classificação de Texturas	330
6.13.1	<i>Pipeline</i> de Extração de Características e Avaliação Comparativa	332
6.13.2	Diagnóstico Fino do Classificador: Precisão, Revocação e F1-Score	335
6.14	Limitações dos Descritores Artesanais	337
6.15	Resumo	338
6.16	 Uso do Gemini Notebook como Tutor	338
6.17	Lista de Exercícios	338
	Referências do Capítulo	339
6.18	 Parte Prática com Exercícios de Programação	339
	 Objetivo deste Caderno	340
	<i>Download</i>	340
	Resumo dos Métodos do <code>morph.py</code> (Cap. 7)	341
6.18.1	EP07_01  Classificador k-NN Passo a Passo	341
6.18.2	EP07_02  Normalização <i>Z-score</i> e Robustez do k-NN a Escalas Distintas	343
6.18.3	EP07_03  Avaliação por Matriz de Confusão	346
6.18.4	EP07_04  Codificação Manual do Descritor LBP	348
6.18.5	EP07_05  Histograma de Orientações de uma Célula HOG	350
6.18.6	EP07_06  <i>Pipeline</i> Completo: Descritores + k-NN + Avaliação Multi-Classe	353
6.18.7	EP07_07  Classificação Real de um Mosaico de Texturas via LBP + k-NN	356
7	Compreendendo Cenas: Correspondência de Características, Detecção e Segmentação 362	
7.1	Objetivos do Capítulo	362
7.2	Configuração do Ambiente	363
7.3	Detectores e Descritores Locais de Características	364
7.3.1	O Algoritmo ORB (<i>Oriented FAST and Rotated BRIEF</i>)	364
7.3.2	Explorando o Simulador do ORB	365
7.4	Correspondência de Características e Homografia com ORB	366
7.4.1	Detectando e Correspondendo Características com ORB	368
7.5	Modelagem Matemática: Homografia e RANSAC	370
7.5.1	Homografia	370
7.5.2	RANSAC	370
7.5.3	Estimando a Homografia com RANSAC	372
7.5.4	Registrando a Imagem	373
7.6	Detecção de Objetos: <i>Haar Cascade</i> (Viola-Jones)	375
7.7	Detecção de Objetos: Caixas Delimitadoras, IoU e NMS	377
7.7.1	<i>Intersection over Union</i> (IoU)	378
7.7.2	Supressão de Não-Máximos (NMS)	378
7.8	Segmentação Visual: Semântica, de Instâncias e Panóptica	380
7.8.1	Segmentação Semântica por Limiarização	380
7.8.2	Segmentação de Instâncias por Rotulação	380
7.8.3	Alternativa: Transformada de Distância + <i>Watershed</i>	382
7.8.4	Medição e Extração de Atributos com <code>mm.measure</code>	384
7.8.5	Exportação de Anotações: de <code>mm.measure</code> ao Formato YOLO	386
7.8.6	Validação de Anotações por IoU: <code>mm.verifyBoundingBox</code>	387
7.8.7	Visualização de Anotações Salvas: <code>mm.showBoundingBox</code>	387
7.9	Visão Geral dos Modelos Modernos	388
7.10	Limitações das Abordagens Clássicas e Motivação para o <i>Deep Learning</i>	390
7.11	Resumo	390
	Próximos Passos	391

7.12	 Uso do Gemini Notebook como Apoio ao Estudo	391
7.13	Lista de Exercícios	391
	Referências do Capítulo	392
7.14	 Parte Prática com Exercícios de Programação	392
	 Objetivo deste Caderno	393
	Download	393
	Resumo dos Métodos do <code>morph.py</code> (Cap. 8)	393
7.14.1	EP08_01  Distância de Hamming e Correspondência de Descritores Binários	394
7.14.2	EP08_02  Homografia e RANSAC: A Votação por <i>Inliers</i>	395
7.14.3	EP08_03  Imagem Integral: Somas Retangulares em Tempo Constante	398
7.14.4	EP08_04  IoU e Supressão de Não-Máximos (NMS)	401
7.14.5	EP08_05  Rotulação de Componentes Conectados: Segmentação de Instâncias	403
7.14.6	EP08_06  Segmentação de Instâncias em Imagem Real: Objetos Geométricos	405
8	Aprendizado Profundo para Visão Computacional	410
8.1	Objetivos do Capítulo	410
8.2	Panorama: Classificação, Detecção e Segmentação	411
8.3	Configuração do Ambiente	412
8.4	Da Convolução Fixa à Convolução Aprendida	413
8.4.1	Neurônio Artificial	414
8.4.2	Camada Convolutiva	416
8.4.3	Função de Ativação	416
8.4.4	<i>Pooling</i>	418
8.4.5	Treinamento de Redes Neurais: Como as <i>CNNs</i> Aprendem	418
8.4.6	Arquitetura de uma <i>CNN</i>	421
8.4.7	Como o Gradiente Ajusta os <i>Kernels</i> da Convolução	421
8.5	Projetos Práticos: Do Conceito à Implementação	426
8.6	Projeto Prático 1: Uma <i>CNN</i> do Zero para os Dígitos	428
8.6.1	Detalhamento do Bloco 1: Preparação e Estruturação dos Dados	428
8.6.2	Detalhamento do Bloco 2: Definição da Arquitetura Convolutiva	430
8.6.3	Detalhamento do Bloco 3: Instanciação e Parâmetros de Otimização	431
8.6.4	Detalhamento do Bloco 4: Laço de Treinamento e Avaliação	432
8.6.5	Detalhamento do Bloco 5: Visualização do Fluxo de Ativações	434
8.6.6	Inspecionando o Grafo Computacional com <code>torchviz</code>	435
8.6.7	Inspecionando o Grafo Computacional Detalhado com <code>torchviz</code>	438
8.6.8	Comparando com o Capítulo 7	439
8.7	Transferência de Aprendizado	441
8.7.1	Detalhamento do Bloco 1: Divisão dos Domínios, Escassez e Exibição das Amostras	441
8.7.2	Detalhamento do Bloco 2: Arquitetura Modular e Rotinas Genéricas	443
8.7.3	Detalhamento do Bloco 3: Pré-treinamento, Transferência e Análise Comparativa	444
8.7.4	Detalhamento do Bloco 4: Visualização do Fluxo de Ativações com <code>mm.showNet</code>	447
8.8	Aplicações em Larga Escala com Modelos Pré-treinados	448
8.8.1	Classificação de Imagens com Arquiteturas Consolidadas (<i>ResNet-18</i>)	449
8.8.2	Detecção de Objetos com Redes de Região (<i>Faster R-CNN</i>)	451
8.8.3	Segmentação Semântica com <i>DeepLabV3</i>	453
8.9	Projeto Prático 2: Detecção de Objetos com YOLO e Transferência de Aprendizado	455
8.9.1	Bloco 1: Definição das Formas, Injeção de Ruído e Exibição das Amostras do Dataset	455
8.9.2	Bloco 2: Montagem da Estrutura de Pastas e do Manifesto <code>data.yaml</code>	458
8.9.3	Bloco 3: Pré-processamento com Filtro de Mediana para Restauração da Imagem	459
8.9.4	Bloco 4: Restauração do <i>Dataset</i> e Treinamento do YOLOv8 (mAP)	460
8.9.5	Bloco 5: Comparativo de Inferência: Imagem Ruidosa vs. Restaurada com Filtro de Mediana	464
8.9.6	Usando um Conjunto de Dados Real	466
8.10	Projeto Prático 3: Segmentação Semântica com U-Net	466

8.10.1	Bloco 1: Gerador do Conjunto Sintético de Nódulos e Exibição Inicial	466
8.10.2	Bloco 2: Linha de Base Clássica (Filtragem, Otsu e Morfologia)	468
8.10.3	Bloco 3: Construção da Arquitetura U-Net e Funções de Perda	469
8.10.4	Bloco 4: Treinamento da U-Net e Avaliação de Desempenho	471
8.10.5	Bloco 5: Comparativo Quantitativo e Qualitativo (Clássico vs. U-Net)	472
8.10.6	Bloco 6: Integração com Datasets Reais via Roboflow	475
8.11	Integrando Geometria Computacional e Aprendizado Profundo	476
8.11.1	Realidade Aumentada com Marcadores e Homografia	476
8.11.2	Fotogrametria e Referência de Escala	477
8.11.3	Visão Estereoscópica e Estimativa de Profundidade	479
8.12	Resumo	480
8.13	 Uso do Gemini Notebook como Tutor Complementar	481
8.14	Lista de Exercícios	481
8.15	Encerramento da Parte II	482
	Referências do Capítulo	483
8.16	 Parte Prática com Exercícios de Programação	483
	 Objetivo deste Caderno	484
8.16.1	EP09_01  Convolução 2D Manual (Forward de uma Camada Aprendida)	485
8.16.2	EP09_02  <i>Pooling</i> Manual (Máximo e Média)	487
8.16.3	EP09_03  Contagem de Parâmetros Treináveis de uma CNN	489
8.16.4	EP09_04  Interseção sobre União (IoU) e Supressão de Não-Máximos (NMS)	491
8.16.5	EP09_05  <i>Pipeline</i> Integrado: Da Detecção à Medição do Mundo Real	493
	Referências	496
	Appendices	499
A	Sobre o MCTest	499
A.1	Instalação do MCTest	499
A.2	Criando um exame no MCTest	499
A.3	Criando uma questão paramétrica	500
A.4	Exemplo real: uma questão do Simulado 4	501
A.5	Considerações finais	502
B	Sobre o Moodle (VPL)	504
B.1	Configurando uma atividade VPL	504
B.2	Publicando os EPs como atividades VPL	504
B.3	Considerações finais	504
C	Uso do SEB nos simulados quinzenais	506
D	Geração de material de aula com o Gemini Notebook	507
D.1	Vídeos conceituais e de resolução de EPs	507
D.2	Slides de apoio	507
D.3	Considerações finais	507

Lista de Figuras

1	Botão clicável Executar Colab , disponível no início das partes Teórica e Prática de cada capítulo, permitindo a execução interativa dos exemplos e exercícios.	4
1.1	Diagrama relacional detalhando as distinções fundamentais, sinergias e interconexões entre PDI e VC no contexto de sistemas baseados em imagens.	3
1.2	Representação do fluxo sequencial de processamento: a saída de cada nível torna-se a entrada do nível subsequente.	5
1.3	Fluxo detalhado do PDI: da aquisição sensorial à extração de atributos e reconhecimento automatizado, incluindo em processamento colorido e compressão.	5
1.4	Representação do espectro visível e sua posição em relação às demais radiações eletromagnéticas, destacando a variação dos comprimentos de onda de 380 nm a 750 nm.	6
1.5	(A) Espectro eletromagnético completo em escala logarítmica, com destaque para a faixa visível. (B) Detalhe da luz visível (380-750 nm) e suas cores. (C) Decomposição da luz branca pelo prisma: menor comprimento de onda sofre maior refração, separando UV, visível e infravermelho.	6
1.6	Exemplos de imagens sintéticas representadas matricialmente.	10
1.7	Mandrill (Mandrillus sphinx) em ambiente natural. Crédito: Julien Renoult (CC BY 4.0). . .	11
1.8	Processamento básico de imagens: (a) imagem original, (b) imagem em tons de cinza, (c) imagem binarizada por limiar ($T=128$).	13
1.9	Comparação entre limiarização manual ($T=128$) e automática (Otsu) sobre a imagem em tons de cinza.	14
1.10	Acesso a um pixel específico (r, c) e a representação de sua vizinhança na imagem.	15
1.11	Estúdio do Gemini Notebook.	17
1.12	Simulador: Distâncias Euclidiana, <i>City-block</i> e <i>Chessboard</i>	22
1.13	Simulador: Desempenho Preditivo — Métricas de ML	29
1.14	Simulador: <i>Mean Average Precision</i> (mAP) - Curva P.S	32
1.15	Simulador: Estatísticas de Pixels	34
1.16	Simulador: Transformação de Negativo	36
1.17	Simulador: Percepção de Cor e Pesos ITU-R	37
1.18	Simulador: Limiarização Interativa	39
1.19	Simulador: Ajuste de Brilho e Contraste	41
1.20	Simulador: Gerador de Xadrez	43
1.21	Simulador: Estrutura do Arquivo PGM	45
1.22	Simulador: Filtro de Máximo Local 1D	47
2.1	Comparativo didático entre o sistema visual biológico e o eletrônico: no topo, a anatomia do olho humano destacando a retina e os fotorreceptores (cones e bastonetes); abaixo, a estrutura de uma câmera digital detalhando o sensor CMOS, a matriz de filtros de Bayer (RGGB) e o processo de quantização digital realizado pelo ADC.	49
2.2	Coletânea de desafios perceptivos: (topo esquerdo) Vaso de Rubin — ambiguidade figura-fundo; (topo central) Elefante de Shepard — incongruência geométrica; (topo direita) Sombra de Adelson — constância de brilho baseada no contexto; (inferior esquerdo) Grade de pontos — inibição lateral; (inferior direito) Escada de Schröder.	50
2.3	Imagem RGB (3 canais, $M \times N \times 3$) e versão RGBA (4 canais, $M \times N \times 4$) com transparência alfa gradual da esquerda para a direita. Imagem Mandrill — USC SIPI Image Database (domínio público).	54

2.4	Efeito da subamostragem. Os títulos exibem as dimensões (W x H) e o tamanho da matriz em memória (KB).	56
2.5	Efeito da redução da profundidade de bits. Os títulos exibem a quantidade de bits, níveis e o tamanho em memória (KB).	57
2.6	Ilustração de vizinhanças 4 em uma matriz 3x3. No centro (1,1), o pixel de interesse.	58
2.7	Area de Proteção Ambiental Quilombos do Médio Ribeira - Barbudo-rajado (<i>Malacoptila striata</i>). Crédito: Thomas Fuhrmann (CC BY-SA 4.0).	61
2.8	Exemplo de translação da imagem do pássaro com deslocamentos (50,50) e (100,50).	63
2.9	Exemplo de rotação da imagem do pássaro em 30° e 45° usando interpolação bilinear.	64
2.10	Comparação de interpolação com zoom no detalhe do olho (recorte 60x60, ampliado 4x). Note o efeito de blocos no vizinho mais próximo vs. a suavização na bilinear.	66
2.11	Exemplo de cisalhamento da imagem do pássaro: horizontal (shx=0.3), vertical (shy=0.3) e combinado (shx=0.2, shy=0.2).	67
2.12	Simulador: Ajuste de Brilho e Contraste	71
2.13	Simulador: Subamostragem	73
2.14	Simulador: Quantização	75
2.15	Simulador: Ajuste de Brilho e Contraste	77
2.16	Simulador: Translação (tx, ty)	79
2.17	Simulador: Rotação (ângulo)	81
2.18	Simulador: Redimensionamento (Nearest vs Bilinear)	83
2.19	Simulador: Cisalhamento (shx, shy)	86
2.20	Simulador: Transformação Afim (matriz 2x3)	88
2.21	Simulador: Correção de Perspectiva (Homografia)	90
2.22	Aquisição da imagem de um Sudoku à esquerda. À direita, conversão para tons de cinza e redimensionamento. Crédito: Héctor Rodríguez de Guardamar, Espanha (CC BY 2.0).	92
2.23	Correção de perspectiva: original e vista frontal retificada.	93
2.24	Simulador: correção de perspectiva do Sudoku.	94
3.1	<i>Mandrill</i> (<i>Mandrillus sphinx</i>) fotografado em ambiente natural na África do Sul. A imagem possui resolução de 500 px e contém metadados EXIF com coordenadas GPS aproximadas (-26.20473, 28.22662). Crédito: Carlos Guilherme Rodrigues (CC BY-SA 3.0).	98
3.2	Operações aritméticas saturadas: adição de constante (clareamento) e subtração de constante (escurecimento com saturação em 0).	99
3.3	Retrato de um leopardo (<i>Panthera pardus</i>) em ambiente natural. Crédito: C. Brück (CC BY-SA 4.0).	100
3.4	<i>Alpha blending</i> entre recortes alinhados de mandrill e do leopardo (Figure 3.3) para diferentes valores de α . Em $\alpha=1$ vê-se apenas mandrill; em $\alpha=0$, apenas o leopardo; valores intermediários fundem os olhares das duas imagens proporcionalmente.	101
3.5	Operações lógicas bit a bit com máscara circular: AND (isolamento da ROI), OR (iluminação da ROI) e NOT (negativo).	102
3.6	Histogramas da imagem original, de uma versão escurecida e de uma versão clareada. Os valores na segunda linha indicam a quantidade de pixels saturados em 0 (preto) e 255 (branco).	103
3.7	Equalização de histograma em imagem 5x5 com 3 bits (L=8): imagem original com tons concentrados em {2,3,4}, imagem equalizada com tons redistribuídos para {1,5,7}, e os respectivos histogramas evidenciando o espalhamento das frequências.	105
3.8	Equalização de histograma: mm.equalize (global via CDF) vs. CLAHE (adaptativa com limitação de contraste). Os histogramas revelam o espalhamento progressivo das intensidades.	107
3.9	Especificação de histograma: mandril mapeada para o perfil tonal do leopardo. A CDF da saída aproxima a CDF de referência.	108
3.10	Correlação com <i>kernel</i> de média 3x3: versão didática (mm.conv0) vs. vetorizada (mm.conv via cv2.filter2D). A diferença máxima de 39 ocorre nas bordas.	115
3.11	<i>Patch</i> 5x5 extraído da imagem do leopardo (posição [250:255, 250:255]). A janela amarela destaca a vizinhança 3x3 centrada no pixel [1,1] onde a correlação será calculada.	116

3.12	Filtro de média com <i>kernels</i> de tamanho crescente (3×3 , 7×7 , 15×15). O borramento das bordas aumenta com o tamanho do <i>kernel</i>	117
3.13	<i>Kernel</i> Gaussiano 5×5 ($=1$): pesos normalizados, maiores no centro e decrescentes radialmente. A grade facilita a leitura de cada coeficiente.	118
3.14	Comparação entre filtro de média e Gaussiano (<i>kernel</i> 9×9 , $=0$). O Gaussiano preserva melhor as bordas, visível no detalhe do rosto.	119
3.15	Simulador: Filtragem Espacial de Realce 1D.	120
3.16	<i>Kernel</i> Laplaciano w4 aplicado ao <i>patch</i> 5×5 : a janela amarela destaca a vizinhança 3×3 onde o operador de segunda derivada é calculado.	122
3.17	Laplaciano aplicado à imagem do leopardo: resposta bruta (bordas detectadas) com w4 e w8, e imagens realçadas pela subtração do Laplaciano. w8 é mais sensível às diagonais.	123
3.18	Operador de Sobel na imagem do leopardo: Gx detecta bordas verticais, Gy detecta bordas horizontais, e a magnitude $ f $ combina ambos, revelando todas as bordas independente de direção.	125
3.19	Operador de Prewitt: Gx, Gy e magnitude — comparável ao Sobel, mas sem ponderação Gaussiana perpendicular.	125
3.20	Realce de nitidez por <i>Unsharp Masking</i> na imagem do leopardo: a imagem suavizada é subtraída da original para gerar a máscara de alta frequência, que é então reintroduzida para realçar bordas e detalhes.	127
3.21	<i>Unsharp Masking</i> na imagem do leopardo com $=1$ e k variando de 0.5 a 8.0. Para $k>2$ surgem artefatos nas bordas (halos) e o ruído de fundo começa a ser visível.	128
3.22	Detector de Canny com diferentes pares de limiar: limiares baixos capturam mais bordas (inclusive ruído); limiares altos retêm apenas as bordas mais fortes.	129
3.23	Ruído sal e pimenta com densidades crescentes (2%, 5%, 10%). O parâmetro prob indica a fração total de pixels corrompidos, metade sal (255) e metade pimenta (0).	130
3.24	Comparação de filtros para remoção de ruído sal e pimenta (10%): Gaussiano, Média, Mediana, Bilateral e Morfológico. Linha superior: imagens completas; linha inferior: crop da região de interesse.	133
3.25	<i>Pipeline</i> de pré-processamento: CLAHE $\rightarrow$ Gaussiano $\rightarrow$ Canny. Cada etapa prepara a imagem para a seguinte, resultando em bordas limpas e bem definidas.	134
3.26	Simulador: Adição Saturada de Constante	138
3.27	Simulador: Alpha <i>Blending</i> de Duas Imagens	140
3.28	Simulador: Inversão de Imagem (Negativo Fotográfico)	142
3.29	Simulador: Equalização de Histograma (L bits)	144
3.30	Simulador: Aplicação de Máscara AND Binária	146
3.31	Simulador: Filtro de Média com <i>Kernel</i> $N\times N$	148
3.32	Simulador: Operador Laplaciano (w4) para Realce de Bordas	151
3.33	Simulador: Gradiente de Sobel: Gx e Gy	153
3.34	Simulador: Filtro da Mediana 3×3	155
3.35	Simulador: <i>Unsharp Masking</i> (USM)	158
4.1	Imagem com moedas de vários tipos. Crédito: GAZI.MD.AHAD (CC BY-SA 4.0).	161
4.2	Comparação dos pré-processamentos: imagem histograma+T* $^2B(T)$ Otsu. A melhor separação bimodal indica o limiar mais confiável.	164
4.3	Elemento estruturante em formato de cruz (B_{cruz}) utilizado para conectividade-4.	166
4.4	Erosão e dilatação em imagem binária 10×10 com elemento estruturante 'L' assimétrico 3×3 . Validação da dualidade erosão–dilatação.	172
4.5	Simulador interativo de erosão morfológica: visualização do critério de inclusão do elemento estruturante assimétrico B_L sobre a imagem binária A	173
4.6	Abertura, fechamento, composição e filtro sequencial alternado aplicados à binarização Otsu das moedas com CLAHE. Elemento estruturante: disco 13×13	175
4.7	Simulador interativo de dilatação geodésica: o marcador f (verde) propaga-se passo a passo pelos caminhos livres da máscara g , sem atravessar paredes — ilustrando como $\text{delta}_g^{(n)}(f)$ encontra a saída do labirinto.	176

4.8	Resolução de labirinto via Operações Complementares: Invertendo a máscara e o marcador no início do <i>pipeline</i> , o fluxo é resolvido diretamente no domínio complementar através de mm.cero e mm.suprec, eliminando re-inversões redundantes.	179
4.9	<i>Pipeline</i> de Reconstrução Morfológica por Dilatação Condicionada: a máscara contém dois objetos, o marcador isola apenas o núcleo do objeto principal, e as iterações subsequentes em laço reconstroem sua forma exata até a convergência.	181
4.10	<i>Pipeline</i> de preenchimento de buracos (<i>clohole</i>): o marcador é obtido a partir da borda da imagem e restringido ao complemento f^c . As iterações de dilatação geodésica reconstruem o fundo externo; após a complementação final, os buracos internos ficam preenchidos.	183
4.11	<i>Pipeline</i> de eliminação de estruturas de borda (<i>edgeoff</i>): o marcador captura as raízes conectadas às extremidades da matriz, a reconstrução delimita a extensão desses elementos e a subtração booleana preserva unicamente os objetos totalmente internos.	184
4.12	<i>Pipeline</i> completo de segmentação com CLAHE: Otsu → abertura (r=9) → clohole → abertura (r=33) → edgeoff.	185
4.13	Simulador interativo avançado de morfologia com elemento estruturante editável e perfil 1D. .	186
4.14	Morfologia em tons de cinza e seus respectivos histogramas. A erosão reduz intensidades locais, a dilatação as amplia, o gradiente destaca bordas, e os operadores Top-hat e Black-hat evidenciam detalhes locais brilhantes e escuros. Elemento estruturante: disco de diâmetro 19. .	188
4.15	Algoritmo de rotulação por <i>flood-fill</i> com pilha: passo a passo, cards, linha do tempo, código Python e fluxograma.	191
4.16	Simulador interativo de rotulação de componentes conexas (<i>connected component labeling</i>): visualização da expansão flood-fill, conectividade-4 e conectividade-8.	192
4.17	Efeito da conectividade na rotulação: a imagem binária 10×10 contém pixels diagonalmente adjacentes. Com conectividade-4, esses pixels formam componentes distintas; com conectividade-8, fundem-se em uma única componente.	192
4.18	Algoritmo da Transformada de Distância: passo a passo, cards, linha do tempo, código Python e fluxograma.	194
4.19	Simulador interativo da Transformada de Distância (TD) iterativa via erosão em tons de cinza. Pixels fora da imagem agora assumem o valor máximo (144), propagando os custos apenas a partir do fundo interno.	195
4.20	Transformada de Distância em imagem binária 10×10. Esquerda: imagem original (<i>foreground</i> = 255). Centro: mm.dist1 iterativa (erosões com elemento cruz). Direita: mm.dist (L2 Euclidiana).	196
4.21	Simulador interativo 2D (4x4) com sincronização estrita de filas de propagação horizontal (b) para obter a convergência exata descrita no artigo.	197
4.22	Menor caminho geodésico em um labirinto. As distâncias geodésicas são calculadas a partir da entrada e da saída usando mm.gdist. Os pixels cujo somatório das duas distâncias é igual à distância mínima entre os marcadores pertencem a um caminho ótimo.	199
4.23	Algoritmo Didático de <i>Watershed</i> por Crescimento de Regiões Limitado por Máscara: passo a passo, cards, linha do tempo, código Python e fluxograma.	200
4.24	Simulador interativo do Algoritmo <i>Watershed</i> por propagação morfológica. Desenhe a máscara, posicione os marcadores e ajuste o Elemento Estruturante para observar a inundação. Quando as bacias se encontram simultaneamente, o empate é resolvido assumindo uma das regiões de forma aleatória.	201
4.25	<i>Pipeline watershed</i> delimitado por máscara em imagem binária 20×20.	203
4.26	<i>Pipeline Watershed</i> para separação de moedas sobrepostas: da máscara binária dilatada até os contornos finais anotados sobre a imagem original.	204
4.27	Componentes conexas extraídos após o <i>pipeline</i> CLAHE → Otsu → limpeza morfológica. Cada objeto é colorido com cor distinta e anotado com sua área em pixels.	206
4.28	Contornos extraídos com <i>findContours</i> . Cada moeda é anotada com sua circularidade — valores próximos de 1 confirmam forma circular.	208

4.29	<i>Bounding boxes</i> derivadas dos componentes conexos sobrepostas à imagem original. As anotações são exportadas no formato YOLO (<i>classe cx cy w h</i>), com coordenadas normalizadas para o intervalo $[0,1]$	209
4.30	Simulador: Limiarização Global por Limiar Fixo	215
4.31	Simulador: Limiarização Automática de Otsu	217
4.32	Simulador: Dilatação Binária Plana (mm.dil0)	220
4.33	Simulador: Erosão Binária Plana (mm.ero0)	222
4.34	Simulador: Abertura Morfológica (erosão + dilatação)	224
4.35	Simulador: Fechamento Morfológico (dilatação + erosão)	226
4.36	Simulador: Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)	228
4.37	Simulador: Gradiente Morfológico, Top-hat e Black-hat	230
4.38	Simulador: Transformada de Distância	233
4.39	Simulador: Separação de Blobs, Rotulação e Descritores	235
5.1	Amostra de imagens públicas do <i>skimage.data</i> , agrupadas por área de aplicação.	241
5.2	Comparação entre estratégias de pré-processamento para binarização da imagem de texto: (a) imagem original; (b) limiarização direta pelo método de Otsu; (c) fundo estimado por filtragem Gaussiana; (d) imagem após normalização de fundo (divisão pela imagem suavizada); (e) CLAHE seguido de limiarização por Otsu; e (f) normalização de fundo seguida de limiarização por Otsu. As imagens ilustram o efeito de cada técnica na compensação de variações de iluminação e na qualidade da segmentação.	243
5.3	Comparação do texto extraído pelo Tesseract a partir de quatro versões da mesma imagem: (a) imagem original; (b) CLAHE seguido de limiarização por Otsu; (c) normalização de fundo seguida de limiarização por Otsu; e (d) normalização de fundo em tons de cinza, sem limiarização. A comparação evidencia que a binarização externa nem sempre favorece o reconhecimento, uma vez que o Tesseract já realiza sua própria binarização adaptativa internamente.	246
5.4	Fluxo de reconhecimento e tradução automática. A imagem pré-processada por normalização de fundo, sem limiarização, é utilizada como entrada para o Tesseract OCR, e o texto reconhecido é traduzido do inglês para o português por meio da biblioteca <i>deep-translator</i>	247
5.5	<i>Pipeline</i> de ingestão de documentos: rasterização adaptativa de páginas PDF para matrizes discretas em formato PNG, exibindo a página dois.	250
5.6	Simulador interativo de correção de inclinação (<i>deskew</i>): mova o <i>slider</i> para inclinar o documento e observe as três etapas do <i>pipeline</i> — imagem inclinada, bordas Canny e resultado corrigido.	253
5.7	<i>Pipeline</i> de retificação axial: exibição comparativa entre a entrada rotacionada original, o mapa de gradientes estruturais de Canny e o resultado final alinhado com fundo normalizado em branco.	254
5.8	Deteção dos discos marcadores, extração de contornos e retificação por transformação de perspectiva.	257
5.9	Simulador interativo de filtragem por circularidade: mova o <i>slider</i> para ajustar o limiar C e observe quais componentes são aceitos (verde) ou rejeitados (vermelho).	258
5.10	<i>Pipeline</i> de processamento do <i>QRCode</i> : limiarização, abertura morfológica, inversão e recorte final para decodificação.	260
5.11	Simulador interativo do <i>pipeline</i> de isolamento do <i>QRCode</i> : navegue pelas etapas de filtragem, ajuste o elemento estruturante do fechamento morfológica e veja a deteção do contorno quadrado sobre a imagem original do gabarito.	261
5.12	Decodificação de código de barras linear com <i>pyzbar</i> : imagem de entrada e dados extraídos.	263
5.13	Imagem da folha de respostas em escala de cinza carregada a partir do PDF rasterizado.	265
5.14	Área de respostas extraída por <i>getAnswerArea</i> : região retificada contendo os quadros de marcação.	266
5.15	Região do <i>QRCode</i> isolada por <i>segmentQRcode</i> dentro da área de respostas retificada.	267
5.16	Recorte inferior da área de respostas, concentrando os quadros de bolhas a serem segmentados.	268

5.17	Respostas lidas automaticamente pelo MCTest após segmentação e classificação de todas as bolhas.	270
5.18	Detecção de defeito por subtração de imagem: produto de referência, imagem com defeito simulado e máscara de anomalia detectada.	272
5.19	Simulador interativo de inspeção industrial por subtração de imagens: controle as distorções geométricas de desalinhamento (registro) e o limiar de detecção para observar o impacto nos falsos positivos.	273
5.20	Detecção de heterogeneidade de textura: mapa de variância local e máscara de anomalia.	275
5.21	Simulador: IoU entre máscara de referência e máscara predita	282
5.22	Simulador: Filtro de Marcadores por Circularidade	284
5.23	Simulador: Classificação de Marcações OMR	286
5.24	Simulador: Estimador de Inclinação por Mediana Angular	288
5.25	Simulador: Normalização de Fundo por Divisão	290
5.26	Simulador: Mapa de Variância Local para Detecção de Textura	292
5.27	Simulador: <i>Pipeline</i> de Inspeção — Registro por Translação e Subtração	295
5.28	Simulador: Segmentação Geométrica + Verificação por Decodificação de QRCode	298
6.1	Exemplo motivacional: diferentes frutas formando agrupamentos distintos em um espaço de características.	302
6.2	Panorama do <i>pipeline</i> clássico de classificação de imagens: extração de descritores (LBP, HOG), formação do espaço de características e classificação via k-NN. Não se aplica a modelos de <i>deep learning</i> (CNNs, YOLO), que aprendem <i>features end-to-end</i> diretamente dos pixels. Fonte: elaborado com auxílio do Gemini Notebook (Google, 2025).	303
6.3	Comparação visual de diferentes descritores aplicados à mesma imagem. Cada descritor revela aspectos distintos da cena.	305
6.4	Histograma dos códigos LBP da imagem do cameramen. Cada barra representa a frequência relativa de um código LBP, formando o vetor de características utilizado para descrever a textura.	306
6.5	Primeiros 100 componentes do vetor de características HOG.	308
6.6	Importância da normalização das características para o classificador k-NN.	311
6.7	Mapa conceitual do processo de classificação de imagens utilizando descritores clássicos (LBP, HOG, pixels brutos) e classificador tradicional (k-NN). Não se aplica a modelos de <i>deep learning</i> (CNNs, YOLO), que aprendem <i>features end-to-end</i> diretamente dos pixels.	312
6.8	Matrizes de confusão obtidas pelo classificador k-NN utilizando três descritores diferentes. As linhas representam a classe real (Maçã, Banana e Laranja) e as colunas a classe predita. Quanto maior a concentração de valores na diagonal principal, melhor o desempenho do descritor.	315
6.9	Comparação detalhada de acurácia global em cenário realista. Observe como os descritores extraídos estruturalmente (LBP e HOG) superam o uso de intensidades puras de pixels brutos.	316
6.10	Ilustração do processo de geração de dados sintéticos com <i>make_classification</i> . À esquerda, visualização das três classes em um espaço bidimensional formado por duas características informativas. À direita, importância relativa de cada característica para a classificação, evidenciando que apenas 7 das 10 características são efetivamente discriminativas, enquanto as demais são redundantes (2) ou repetidas (1).	319
6.11	Classificação de uma nova amostra pelo algoritmo k-NN. O ponto de teste (estrela) é classificado a partir dos três vizinhos mais próximos, destacados por círculos.	322
6.12	Simulador interativo da fronteira de decisão do k-NN: adicione pontos de treinamento e ajuste o valor de k para observar o efeito sobre a região de decisão.	323
6.13	Amostra de dígitos manuscritos da base <i>load_digits</i> , utilizada como estudo de caso de classificação.	324
6.14	Matriz de confusão do classificador k-NN treinado com vetores de intensidade brutos (pixels).	325
6.15	Comparação de acurácia entre descritores de pixels brutos e HOG para o classificador k-NN (k=3) na base de dígitos.	326
6.16	Acurácia média por validação cruzada (5 partições) em função do parâmetro k, para a base de dígitos com vetores de intensidade brutos.	328

6.17	Acurácia nos conjuntos de treinamento e teste para diferentes valores de k . As regiões coloridas representam, de forma conceitual, tendências de comportamento do classificador: maior risco de sobreajuste (vermelho), compromisso entre viés e variância (verde) e maior risco de subajuste (azul).	330
6.18	Amostras sintéticas das três classes de textura utilizadas no experimento de classificação, geradas com ruído gaussiano e variabilidade intra-classe.	332
6.19	Acurácia média obtida via validação cruzada (5-fold) para os descritores de Pixels Brutos, LBP e HOG aplicados à base de texturas sintéticas.	334
6.20	Métricas de avaliação detalhadas para o classificador k-NN com descritores LBP	337
6.21	Simulador: Classificador k-NN Passo a Passo	343
6.22	Simulador: Efeito da Normalização Z-score na Distância k-NN	345
6.23	Simulador: Precisão x Revocação	347
6.24	Simulador: Código LBP	349
6.25	Simulador Didático: Histograma HOG de uma Celula (mapeamento de ângulos para bins)	352
6.26	Simulador: Pipeline k-NN Multi-Classe (votacao, desempate e matriz de confusao)	355
6.27	Mosaicos de referência (formato PGM ASCII) utilizados no EP07_07. Caso 1: grade 2x2 com uma amostra de cada classe. Caso 2: grade 3x3 com classes repetidas e maior variabilidade.	360
6.28	Simulador: Classificação de um Mosaico de Texturas via LBP + k-NN	361
7.1	Visão geral dos principais conceitos abordados neste capítulo, incluindo correspondência de características, detecção de objetos, segmentação de imagens e suas relações com abordagens modernas baseadas em <i>Deep Learning</i> . Fonte: elaborado com auxílio do Gemini Notebook (Google, 2025).	363
7.2	Comparação do desempenho de diferentes implementações da distância de Hamming.	366
7.3	Simulador interativo do descritor ORB: explore a lógica de rotação e construção do descritor binário. Altere a rotação para observar como o padrão de amostragem de testes binários do BRIEF (linhas verdes e vermelhas) se orienta dinamicamente para garantir a invariância angular.	367
7.4	Imagem original gerada pelo Gemini e versão rotacionada (20°), simulando uma mudança de ponto de vista.	368
7.5	As 5 melhores correspondências de características ORB entre a imagem original e a cena sintética, antes da filtragem por RANSAC.	370
7.6	Simulador interativo do algoritmo RANSAC: ajuste o limiar de distância e execute o algoritmo para observar a separação entre <i>inliers</i> e <i>outliers</i>	371
7.7	Simulador interativo do RANSAC aplicado ao registro de imagens: pontos-chave de duas imagens são casados por um descritor, algumas correspondências são espúrias (<i>outliers</i>), e o RANSAC estima a transformação de similaridade que alinha a maioria delas.	372
7.8	Correspondências classificadas como <i>inliers</i> (verde) pelo RANSAC ao estimar a homografia entre as duas imagens.	373
7.9	Registro da imagem da cena utilizando a homografia inversa estimada pelo RANSAC.	374
7.10	Detecção de faces e olhos com <i>Haar Cascade</i>	377
7.11	Efeito da Supressão de Não-Máximos (NMS): múltiplas detecções redundantes (esquerda) são reduzidas a uma detecção por objeto (direita).	379
7.12	Segmentação clássica (não baseada em aprendizado profundo): máscara semântica (moeda x fundo) via limiarização de Otsu, e segmentação de instâncias via rotulação de componentes conectados.	382
7.13	Separação de moedas que se tocam via transformada de distância + <i>watershed</i> , sem erosão do contorno das moedas.	384
7.14	<i>Bounding boxes</i> reconstruídas a partir do arquivo de anotações YOLO exportado por <i>mm.save-Measures</i> , sobrepostas à imagem original das moedas.	388
7.15	Simulador: Distância de Hamming entre Dois Descritores Binários	396
7.16	Simulador: RANSAC — Votação por Inliers entre Modelos Candidatos	398
7.17	Simulador: Soma Retangular em O(1) — Múltiplas Situações de Borda	400
7.18	Simulador: IoU e Supressão de Não-Máximos	402
7.19	Simulador: Rotulação de Componentes Conectados — Conectividade 4 vs. 8	405

7.20	Imagem utilizada no EP08_06. À esquerda, a imagem original. À direita, a imagem com as <i>bounding boxes</i> do arquivo de gabarito.	407
7.21	Simulador: Acurácia de Segmentação em Múltiplos Objetos (IoU)	408
8.1	Visão geral dos principais conceitos abordados neste capítulo. Fonte: elaborado com auxílio do <i>Gemini Notebook</i> (Google, 2025).	411
8.2	Simulador interativo de convolução 2D clássica: escolha entre as três imagens sintéticas de entrada 12×12 (casa, rosto feliz ou triste) e um filtro 3×3 (Sobel V, Sobel H, Nitidez ou Identidade) e avance passo a passo para observar como os produtos internos locais do campo receptivo constroem o mapa de características célula a célula.	414
8.3	Simulador interativo do Neurônio Artificial em contexto de CNN: alterne entre um neurônio de camada convolucional (onde x_i são intensidades de pixel em um campo receptivo e w_i são pesos do <i>kernel</i>) e um neurônio de camada totalmente conectada, ajustando entradas, pesos, viés e função de ativação para visualizar o cálculo de z e da saída y em tempo real.	415
8.4	Simulador interativo da Camada Convolucional: escolha entre três imagens de entrada 12×12 (casa, rosto feliz ou rosto triste) para observar como os mesmos <i>kernels</i> fixos reagem a diferentes bordas e formas. Navegue pelo campo receptivo com os botões ou reprodução automática, ajuste a função de ativação e alterne o zero-padding, acompanhando o mapa de características sendo revelado célula a célula, com o cálculo detalhado termo a termo e a fórmula da dimensão de saída em tempo real.	417
8.5	Simulador interativo da função de ativação ReLU: arraste o controle para ver como valores negativos são zerados e valores positivos são preservados, tanto na curva quanto em um mapa de características real.	418
8.6	Simulador interativo de <i>pooling</i> : escolha entre <i>max-pooling</i> e <i>average-pooling</i> e avance passo a passo para observar a redução da resolução espacial do mapa de características.	419
8.7	Simulador interativo da Função de Perda (<i>Cross-Entropy</i>): selecione a classe real da imagem (Casa, Feliz ou Triste) e ajuste as probabilidades estimadas pela Softmax para visualizar o cálculo da penalização escalar e o gráfico do logaritmo negativo em tempo real.	420
8.8	Simulador interativo de <i>Backpropagation</i> : avance os passos da Regra da Cadeia para acompanhar o fluxo do sinal de erro no sentido inverso da rede, observando a computação das derivadas parciais do gradiente em cada camada.	421
8.9	Simulador interativo dos Algoritmos de Otimização: compare a trajetória do SGD e do Adam sobre uma superfície de perda não convexa com mapa de calor e curvas de nível. A linha sólida mostra o otimizador selecionado avançando passo a passo; a linha tracejada mostra, para comparação instantânea, o caminho completo que o outro otimizador percorreria a partir do mesmo ponto inicial. Observe como o SGD fica retido no Mínimo Local à direita, enquanto o Adam pode ou não transpor a barreira central dependendo do impulso acumulado e da taxa de aprendizado. Clique em qualquer ponto do mapa para redefinir o ponto inicial dos pesos.	422
8.10	Arquitetura simplificada de uma <i>CNN</i> para classificação de imagens, destacando o <i>forward pass</i> e as etapas de treinamento por meio da função de perda, da retropropagação e do algoritmo de otimização.	423
8.11	Simulador interativo da arquitetura de uma <i>CNN</i> : escolha uma das imagens de entrada 12×12 (casa, rosto feliz ou triste), clique em cada bloco do <i>pipeline</i> — Entrada, <i>Conv+ReLU</i> , <i>Pooling</i> , <i>Flatten</i> , <i>FC</i> e <i>Softmax</i> — e execute passos de treinamento reais (<i>forward pass</i> + retropropagação) para observar a perda e a acurácia evoluindo, os <i>kernels</i> sendo ajustados e o <i>Softmax</i> passando a apontar a classe correta.	424
8.12	Simulador interativo do cálculo do gradiente de um peso do <i>kernel</i> convolucional com explicação detalhada da soma do gradiente inicial e alinhamento à esquerda.	427
8.13	Amostras de dígitos do conjunto de treinamento após conversão para <i>tensors</i> PyTorch e normalização.	429
8.14	Representação do fluxo de transformações dimensionais dos <i>tensors</i> ao longo da arquitetura <i>CNNDígitos</i>	430
8.15	Curvas de treinamento e avaliação da <i>CNN</i> na base de dígitos: evolução da perda de entropia cruzada no conjunto de treinamento e da acurácia no conjunto de teste ao longo de 50 épocas.	434

8.16	Fluxo de ativações da CNN treinada ao processar um exemplo real do dígito 3, do <i>dataset load_digits</i> : dimensões dos tensores camada a camada, da entrada ao <i>logit</i> de saída.	435
8.17	Grafo computacional da <i>CNNDigitos</i> gerado via <i>torchviz</i> , mostrando as operações de <i>forward</i> e os nós de gradiente (<i>backward</i>) associados a cada parâmetro treinável.	437
8.18	Grafo detalhado da classe <i>CNNDigitos</i> gerado via <i>torchviz</i>	439
8.19	Comparação de acurácia entre os classificadores clássicos do Capítulo 7 (pixels brutos e HOG com k-NN) e a CNN treinada neste capítulo, na mesma base de dígitos.	440
8.20	Amostras dos conjuntos de treinamento após pré-processamento e adequação dimensional aos tensores PyTorch: Domínio A (dígitos 0 a 4, tarefa de origem) e Domínio B (dígitos 5 a 9 reindexados para 0 a 4, tarefa de destino).	443
8.21	Comparação de acurácia no conjunto de teste do Domínio B (dígitos 5 a 9) sob restrição de dados (20 exemplos de treino): demonstração do impacto do congelamento rígido e da transferência negativa em redes de baixa capacidade.	447
8.22	Fluxo de ativações e transformações dimensionais dos tensores no modelo de transferência de aprendizado ao processar uma amostra de teste do Domínio B (dígito 7, reindexado para classe 2).	448
8.23	Imagem real de teste carregada via <i>mm.read</i> e exibida com <i>mm.show</i> antes da conversão para <i>tensor</i> normalizado e alimentação da ResNet-18.	450
8.24	Resultado da inferência com o modelo Faster R-CNN pré-treinado no <i>dataset COCO</i> : identificação da classe ‘cão’ (<i>dog</i>), caixa delimitadora e pontuação de confiança sobrepostas.	453
8.25	Amostras iniciais do dataset sintético ruidoso de objetos geométricos com as caixas delimitadoras do formato YOLO sobrepostas.	458
8.26	Comparativo entre a imagem ruidosa original do dataset sintético e a versão restaurada via Filtro de Mediana com as anotações YOLO sobrepostas.	459
8.27	Comparativo de inferência com a YOLOv8: detecção na imagem ruidosa original e na imagem restaurada via Filtro de Mediana com as predições exibidas via <i>mm.show</i>	465
8.28	Amostras do conjunto de dados sintético de nódulos: imagem em escala de cinza sob ruído e respectiva máscara binária de referência exibidas via <i>mm.show</i>	468
8.29	Linha de base clássica de segmentação: suavização, limiarização de Otsu e abertura morfológica exibidas.	469
8.30	Evolução do treinamento da U-Net compacta: redução da perda e elevação do índice IoU médio no conjunto de validação ao longo de 35 épocas.	472
8.31		474
8.32	Realidade aumentada baseada em marcador ArUco: detecção de canto e sobreposição de imagem virtual por matriz de homografia exibidas via <i>mm.show</i>	477
8.33	Medição de dimensões físicas reais utilizando um cartão de referência de escala conhecida (8,56 cm) renderizada via <i>mm.show</i>	479
8.34	Par estéreo sintético e mapa de disparidade correspondente: regiões mais claras indicam maior disparidade (objetos mais próximos da câmera) exibidos via <i>mm.show</i>	480
8.35	Simulador: Convolução 2D Manual (correlação cruzada + viés + ReLU)	487
8.36	Simulador: Pooling Manual (máximo vs. média)	489
8.37	Simulador: Contagem de Parâmetros — Convolução vs. Camada Totalmente Conectada	491
8.38	Simulador: IoU e Supressão de Não-Máximos	493
8.39	Simulador: Pipeline Integrado — NMS + Medição por Referência de Escala	495
A.1	Questão real gerada pelo MCTest para o Simulado 4 — tópico “Operadores Morfológicos”.	503

Lista de Tabelas

1	Diferentes formatos de publicação gerados automaticamente a partir do mesmo código-fonte.	4
1.1	Conexão entre PDI, VC e outras áreas da ciência.	4
1.2	Principais tipos de imagem digital e seus respectivos conjuntos de valores possíveis para cada pixel.	7
1.3	Principais bibliotecas utilizadas ao longo deste livro para manipulação matricial, PDI e visualização de resultados.	8
2.1	Convenções de escala e ordem de canais nas principais bibliotecas de PDI.	51
2.2	Comparativo entre estratégias de compactação e eficiência de processamento.	58
2.3	Comparativo de métricas de distância aplicadas à malha de pixels.	59
2.4	Principais formatos de armazenamento de imagens digitais e suas aplicações em PDI.	59
2.5	Métodos de interpolação disponíveis em <code>mm.resize</code> e seus respectivos números de vizinhos utilizados no cálculo.	65
3.1	Algoritmo de equalização de histograma.	103
3.2	Interpretação típica dos coeficientes do <i>kernel</i> .	113
3.3	Etapas do <i>Unsharp Masking</i> .	126
3.4	Média vs. mediana com um pixel corrompido. A mediana ignora o outlier; a média é deslocada ~40 níveis.	129
4.1	Técnicas de pré-processamento avaliadas para melhorar a separação entre moedas e fundo antes da aplicação do método de Otsu.	162
4.2	Propriedades de abertura e fechamento.	172
4.3	Sequências do filtro sequencial alternado <code>mm.asf</code> .	174
4.4	Comparação entre os operadores <i>clohole</i> e <i>edgeoff</i> .	182
4.5	Taxonomia simplificada das principais abordagens de segmentação e refinamento estudadas neste capítulo.	189
4.6	<i>Pipeline</i> completo do <i>watershed</i> baseado em marcadores para imagens reais.	202
4.7	<i>Pipeline</i> simplificado utilizado no exemplo didático da Figure 4.25.	202
4.8	Comparação entre as abordagens baseadas em componentes conexos e em contornos.	205
5.1	Imagens do <code>skimage.data</code> utilizadas nos experimentos deste capítulo.	240
5.2	Seleção de imagens públicas representativas disponíveis no módulo <i>skimage.data</i> , organizadas por área de aplicação.	240
6.1	Conjunto de descritores cromáticos, texturais e geométricos utilizados para representar imagens de frutas.	316
6.2	Exemplo simplificado de classificação de frutas utilizando duas características normalizadas.	319
7.1	Comparação entre os principais paradigmas de segmentação visual.	380
7.2	Arquiteturas representativas de Aprendizado Profundo para detecção e segmentação de imagens.	389
8.1	Comparativo entre as principais tarefas de Visão Computacional segundo a granularidade da informação produzida.	411

PDI e VC — Python

Bem-vindo ao livro de PDI e Visão Computacional — versão Python / Português.

Organização

- **Parte I — Fundamentos de PDI** — Representação, histogramas, filtragem, morfologia
 - **Parte II — Visão Computacional** — Segmentação, descritores, detecção, deep learning
-

Ficha Catalográfica

© 2026 Francisco de Assis Zampirolli, Universidade Federal do ABC (UFABC).
Todos os direitos reservados.

Este livro está sob a licença:

Creative Commons Attribution-ShareAlike 4.0 International License

Detalhes em: creativecommons.org/licenses/by-sa/4.0

Projeto gráfico: Francisco de Assis Zampirolli

CATALOGAÇÃO NA FONTE

SISTEMA DE BIBLIOTECAS DA UNIVERSIDADE FEDERAL DO ABC

[CÓDIGO CUTTER] Zampirolli, Francisco de Assis
Processamento Digital de Imagens e Visão Computacional
/ Francisco de Assis Zampirolli – Santo André, SP :
Edição do Autor, 2026.
[xx], [NNN] p. : il.

ISBN: [ISBN A DEFINIR]

1. Processamento Digital de Imagens. 2. Visão Computacional. 3. Morfologia
Matemática. 4. Python. 5. Correção Automática. 6. Moodle. I. Título.

CDD [XX] ed. — [CÓDIGO CDD]

Prefácio

Este livro constitui uma **obra em permanente atualização** nas áreas de **Processamento Digital de Imagens (PDI)** e **Visão Computacional (VC)**, concebida como material didático interativo para cursos de graduação e pós-graduação em Computação, Engenharia e áreas afins.

! Destaque

A obra fundamenta-se na metodologia descrita em Zampirolli et al. (2025) — extensão de Zampirolli et al. (2024), trabalho premiado na trilha **Recursos e Ambientes Educacionais** da **EduComp 2024**. O conteúdo integra a biblioteca `morph.py`, desenvolvida pelo autor.

Este não é um livro estático. Seu conteúdo evolui continuamente à medida que exemplos são aprimorados, novas seções são incorporadas e as abordagens pedagógicas são refinadas com base na experiência de uso e no retorno de estudantes e docentes. Dessa forma, a obra é tratada como um *projeto em constante evolução*, buscando acompanhar tanto os avanços tecnológicos quanto as melhores práticas de ensino em PDI e VC.

Contexto desta edição

O conteúdo desta edição foi elaborado entre maio e julho de 2026, durante a oferta da disciplina de Processamento Digital de Imagens em duas turmas de graduação — majoritariamente de estudantes de Ciência da Computação, no período matutino — e em uma turma de pós-graduação em Ciência da Computação, todas na UFABC.

A metodologia de ensino adotada seguiu uma sequência fixa em cada aula. No início, era exibido um vídeo curto (em média 7 minutos), gerado no **Gemini Notebook**, alternando entre a parte conceitual do capítulo e a resolução dos Exercícios de Programação (EPs) — nesse segundo caso, quase todos os EPs eram resolvidos durante a própria aula. Em seguida, apresentava-se um conjunto de cerca de 12 *slides*, também gerados no Gemini Notebook, cujas fontes eram o PDF completo do livro e o arquivo `morph.py`, a partir de um *prompt* específico para gerar o conteúdo teórico/prático de cada capítulo. Após os *slides*, abriam-se os dois *notebooks* Colab do capítulo (teórico e prático), com ênfase nos simuladores interativos e na execução dos blocos de código alterando parâmetros; nas aulas em laboratório, os alunos colavam as respostas dos EPs diretamente na atividade VPL do Moodle para avaliação automática.

A avaliação contínua incluiu simulados quinzenais, aplicados com o **SEB (Safe Exam Browser)** no Moodle, com EPs semelhantes aos das listas (cada simulado concedia um bônus de 5% sobre a nota final). As duas provas da disciplina também utilizaram o SEB, porém com questões parametrizadas geradas no **MCTest**, cuja descrição combina LaTeX e parâmetros no formato `[[code:variavel]]`, definidos em trechos de Python delimitados por `[[def: ... ]]` na própria questão. Esse processo gerou 110 variações de prova, sorteadas individualmente entre os estudantes. O **Apêndice A** detalha a criação dessas questões no MCTest e sua exportação para o Moodle; o **Apêndice B** descreve a configuração da atividade VPL; e o **Apêndice C** detalha a configuração do SEB, tanto para os simulados quinzenais quanto para as provas.

Como este livro é produzido

O conteúdo deste livro é desenvolvido em **Quarto** e armazenado na pasta `all` do repositório github.com/fzampirolli/pdi-vc. A partir de um único código-fonte, o livro é automaticamente gerado e publicado em diferentes formatos, apresentados na Table 1.

Tabela 1: Diferentes formatos de publicação gerados automaticamente a partir do mesmo código-fonte.

Formato	Descrição
HTML	Versão para <i>web</i> , com simuladores interativos e navegação entre capítulos: fzampirolli.github.io/pdi-vc/
PDF	Versão adequada para impressão ou leitura <i>offline</i> : livro.pt.py.pdf
Notebooks (.ipynb)	Compatíveis com Jupyter e Google Colab , permitindo executar, modificar e experimentar os exemplos de código e os Exercícios de Programação (EPs). Um filtro personalizado mantém referências cruzadas, numeração de figuras e tabelas, além de formatar automaticamente as citações conforme o padrão ABNT.

Cada capítulo disponibiliza dois botões **Executar Colab** (Figure 1), que direcionam para ambientes complementares:

1. **Parte Teórica**, localizada na abertura de cada capítulo, contendo os conceitos apresentados e exemplos de código executáveis;
2. **Parte Prática**, na seção **Parte Prática com Exercícios de Programação** (EPs), dedicada aos exercícios e aos seus simuladores interativos.



Figura 1: Botão clicável **Executar Colab**, disponível no início das partes **Teórica** e **Prática** de cada capítulo, permitindo a execução interativa dos exemplos e exercícios.

A geração dos *notebooks* é totalmente automatizada pelo *script* `gerar_notebooks_alunos.py`, que adapta o conteúdo para ambientes interativos, permitindo sua execução sem a necessidade de instalação do Quarto.

Idiomas e Conversão de Código

O **núcleo de referência** é escrito em **Português** com exemplos em **Python**. Versões em outros idiomas e linguagens de programação são geradas via **APIs de IA**, por meio de um fluxo automatizado que realiza a tradução textual e a conversão de sintaxe do código.

i Nota

As versões traduzidas via IA servem como **ponto de partida** e podem conter imprecisões; recomenda-se revisão antes da aplicação em sala de aula.

Uso de ferramentas de Inteligência Artificial

A elaboração deste livro contou com o apoio de **ferramentas gratuitas de Inteligência Artificial (IA)** para aprimorar a qualidade do texto, dos exemplos de código e dos materiais de apoio. Ao longo do desenvolvimento, diferentes assistentes de IA, como **ChatGPT**, **Claude**, **DeepSeek** e **Gemini**, foram utilizados de forma iterativa em atividades como:

- aprimoramento da clareza, da precisão e da fluidez das explicações;
- identificação e correção de erros de sintaxe, lógica e digitação em exemplos de código;
- sugestão de melhorias na organização, na estrutura e na apresentação do conteúdo.

Além disso, **algumas ilustrações foram produzidas com o auxílio do Gemini**, com o objetivo de facilitar a visualização de conceitos abstratos e complementar as explicações apresentadas ao longo do livro.

Embora essas ferramentas tenham desempenhado um papel importante no processo de produção, **a seleção, a validação técnica, a adaptação, a organização e a responsabilidade pelo conteúdo final são integralmente do autor.**

Mais do que utilizar IA para gerar textos, códigos ou imagens, o principal desafio consiste em selecionar criticamente, verificar, adaptar e integrar esse material de forma tecnicamente correta e pedagogicamente consistente. Nesse contexto, merece destaque o conjunto de **simuladores interativos**, desenvolvido em JavaScript e incorporado às versões HTML e IPYNB do livro, que permite ao leitor experimentar conceitos em tempo real e reforçar a compreensão dos conteúdos apresentados.

A biblioteca `morph.py`

A biblioteca `morph.py` (Zampirolli et al., 2025) acompanha toda a obra como uma ferramenta de apoio ao ensino. Seu objetivo não é substituir bibliotecas consolidadas, como **OpenCV**, **scikit-image**, **scikit-learn**, **NumPy** ou **Matplotlib**, nem competir com elas em desempenho ou abrangência. Em vez disso, procura **tornar transparentes os algoritmos fundamentais de Processamento Digital de Imagens e Visão Computacional (PDI-VC)**, permitindo que o estudante compreenda sua implementação, modifique o código e desenvolva novas funcionalidades.

Sempre que possível, a biblioteca oferece duas versões para um mesmo algoritmo: uma **implementação didática**, escrita passo a passo para refletir diretamente as definições matemáticas apresentadas no livro, e outra **implementação otimizada**, baseada em bibliotecas especializadas para aplicações práticas.

i Herança Técnica

A estrutura da `morph.py` tem como base ferramentas de Morfologia Matemática desenvolvidas no Brasil, como **MMachLib** (Lotufo et al., 1997) e **MMach** (Barrera et al., 1998), além da **mmorph**, utilizada na obra de Dougherty; Lotufo (2003). Essas bibliotecas serviram de referência para o ensino da área durante décadas.

Essa filosofia pode ser observada, por exemplo, nos operadores morfológicos de dilatação:

- `mm.dil`: implementação otimizada, indicada para aplicações práticas;
- `mm.dil0`: implementação didática da dilatação para **elementos estruturantes planares**, seguindo diretamente a definição clássica da Morfologia Matemática;
- `mm.dil1`: implementação didática da dilatação para **funções estruturantes (*kernels* não planares)**, seguindo rigorosamente sua formulação matemática.

A biblioteca também oferece funções para leitura, exibição, conversão de cores, filtragem e morfologia matemática por meio de uma interface simples:

```
from morph import mm

img = mm.gray(mm.read("lena.jpg")) # leitura e conversão para níveis de cinza
grad = mm.gradm(img)               # gradiente morfológico
mm.show(grad)                       # exibição
```

Ao longo do livro, novos métodos são introduzidos na biblioteca para simplificar a realização dos Exercícios de Programação (EPs). Entre eles estão `mm.readTrain()` e `mm.readTest()`, que automatizam a leitura das bases de treinamento e teste utilizadas nos capítulos dedicados ao reconhecimento de padrões.

Além disso, todos os projetos do **Gemini Notebook** incluem o arquivo `morph.py`, permitindo consultar e discutir diretamente a implementação dos algoritmos apresentados no livro. Dessa forma, o estudante pode utilizar o próprio Gemini Notebook como um assistente de estudos, fazendo perguntas como:

Explique como foi implementado o algoritmo `mm.knn0` do `morph.py`, mostrando o código e comentando cada etapa de forma didática. Além disso, explique as diferenças entre `mm.knn0()` e `mm.knn()`.

! Implementações didáticas e implementações otimizadas

Em vários capítulos, um mesmo algoritmo é apresentado em duas versões. As funções com sufixo 0, 1, etc. (por exemplo, `mm.knn0()`, `mm.dil0()` e `mm.dil1()`) são implementações didáticas, desenvolvidas para facilitar o entendimento dos algoritmos. Já as funções sem sufixo (como `mm.knn()` e `mm.dil()`) utilizam implementações otimizadas, frequentemente baseadas em bibliotecas especializadas, como OpenCV, scikit-image e scikit-learn, sendo mais adequadas para aplicações práticas.

Nas atividades avaliadas pelo **VPL do Moodle**, bibliotecas como **scikit-learn** e, em alguns casos, **scikit-image**, podem não ser utilizadas devido às limitações de memória do ambiente de execução. Por esse motivo, os Exercícios de Programação (EPs) empregam preferencialmente as implementações didáticas da `morph.py`, que produzem os mesmos resultados, embora com menor desempenho computacional. Essa restrição não existe quando os programas são executados localmente, por exemplo, no **VS Code**, **Jupyter Notebook** ou **Google Colab**.

O código-fonte da biblioteca está disponível em:

<https://github.com/fzampirolli/pdi-vc/tree/master/morph>

Exercícios de Programação (EPs) e Validação Automática

Cada unidade do livro inclui **Exercícios de Programação (EPs)** práticos e de complexidade crescente, projetados para consolidar os conceitos apresentados ao longo do capítulo. Cada EP é acompanhado por um **simulador interativo**, disponível nas versões HTML e IPYNB, que permite ao estudante manipular parâmetros, visualizar o comportamento dos algoritmos e desenvolver uma compreensão intuitiva do problema antes de iniciar sua implementação. Dessa forma, o processo de aprendizagem combina experimentação, programação e validação automática.

A validação das soluções é feita localmente pela classe `TestSuite` (`testsuite.py`), que compara a saída do programa com os arquivos de casos de teste (`.cases`):

```
TestSuite("EP01_01.py").run()
```

O sistema suporta múltiplas linguagens (Python, Java, C++, C, JavaScript e R) e pode ser integrado diretamente ao Moodle. Para o passo a passo completo de integração com o **VPL do Moodle**, consulte o Guia do Professor: Publicação de EPs no Moodle.

Código aberto

O projeto é regido por princípios de código aberto. O repositório público reúne o texto, os códigos, as imagens e os *scripts*: github.com/fzampirolli/pdi-vc

Cada versão do livro é arquivada permanentemente no **Zenodo** com um *DOI citável*. Para citar este material em trabalhos acadêmicos:

ZAMPIROLI, Francisco de Assis. **PDI+VC — Processamento Digital de Imagens e Visão Computacional**. UFABC, 2026. DOI: [10.5281/zenodo.20784606](https://doi.org/10.5281/zenodo.20784606)

Guia do Professor: Publicação de EPs no Moodle

Esta seção destina-se a docentes que desejam integrar os Exercícios de Programação (EPs) às atividades VPL (*Virtual Programming Lab*) do Moodle.

Integração com o Moodle (VPL)

Os EPs dos *notebooks* podem ser convertidos em atividades **VPL** no Moodle. O *script* `ep_tools.py` (executado via `make build`) automatiza a exportação dos EPs do final de cada capítulo em duas etapas:

1. **Extração** (`make eps`): gera um HTML interativo independente por EP, com enunciado, exemplos e simulador, em `gen/book/eps/<versao>/`. Veja um exemplo em: https://fzampirolli.github.io/pdi-vc/eps/py.pt/EP01_01.html
2. **Conversão** (`make moodle`): transforma o HTML em um fragmento autocontido, sem dependência de CSS externo, pronto para ser colado no editor do Moodle, em `gen/book/eps/<versao>_moodle/`. Veja um exemplo em: https://fzampirolli.github.io/pdi-vc/eps/py.pt/_moodle/EP01_01.html

Ao publicar com `make publish`, essas **duas versões** de cada EP ficam disponíveis nos *links* indicados. O professor pode combinar as duas estratégias: colar a versão Moodle no editor VPL (com simulador funcional) e incluir no enunciado um *link* para a versão completa, onde as fórmulas são renderizadas corretamente pelo MathJax.

⚠ Revisão obrigatória antes de publicar no Moodle

Embora automatizada, a conversão exige revisão do professor em razão das limitações do editor TinyMCE/HTML Purifier do Moodle:

- **Fórmulas matemáticas** — O TinyMCE descarta barras invertidas (`\`), corrompendo equações LaTeX. Por isso, a versão Moodle converte fórmulas simples para HTML puro (entidades como `≥`, `×`). Fórmulas complexas (`\begin{cases}`, matrizes, integrais) podem sair incompletas — revise e, se necessário, reescreva-as em texto ou indique a versão completa via *link*.
- **Figuras externas** — Imagens complementares devem ser inseridas manualmente na atividade VPL.
- **Simuladores** — Funcionam perfeitamente desde que utilizem JavaScript padrão com estilos *inline* e manipulação direta do DOM (`getElementById`). **Atenção:** se incluir fórmulas em LaTeX que contenham o caractere `\` no Moodle, os simuladores podem parar de funcionar.

Como Publicar no Moodle

1. Acesse a versão Moodle do EP desejado:

https://fzampirolli.github.io/pdi-vc/eps/py.pt_moodle/EPXX_YY.html

2. Copie o código-fonte completo da página (`Ctrl+U` → `Ctrl+A` → `Ctrl+C`).
3. No Moodle, crie a atividade VPL, acesse o editor HTML, cole o conteúdo (`Ctrl+V`) e salve.
4. Importe os arquivos `.cases` da pasta `all/capXX/casos/` nas configurações de testes do VPL.

💡 Reaproveitamento dos Casos de Teste

Os arquivos `.cases` são compatíveis com o VPL, permitindo usar o mesmo conjunto de testes tanto no desenvolvimento local quanto na correção automatizada no Moodle.

Antes de começar: *Notebooks* em Python

O conceito de *Literate Programming* (Programação Literária), proposto por Donald Knuth (Knuth, 1984), fundamenta a estrutura deste material. A lógica inverte o paradigma tradicional: o programa é escrito para a leitura humana, assemelhando-se a um ensaio, enquanto o código é extraído separadamente para execução computacional.

O conteúdo é estruturado em *notebooks* — documentos que intercalam **células de texto** (em [Markdown](#)) e **células de código** (em Python).

- **Execução:** células de código são identificadas por []. A execução pode ser realizada com **Shift + Enter** ou pelo botão ▶ da interface.
- **Ambientes:** os *notebooks* podem ser executados localmente, por meio do [Jupyter](#) ou do [Visual Studio Code \(VS Code\)](#), bem como em ambientes de nuvem, como o [Google Colab](#).

💡 Nota sobre o formato

Em ambientes interativos, o código pode ser modificado e executado. Nas versões estáticas (HTML ou PDF), os blocos de código têm finalidade de leitura e referência, sem prejuízo à integridade das explicações.

Parte I

Parte I — Fundamentos de PDI

Capítulo 1

Fundamentos e Primeiros Passos



Este capítulo inaugura a **Parte 1** do livro, dedicada aos fundamentos do Processamento Digital de Imagens (PDI). São apresentados a representação matemática de imagens digitais e os principais métodos para sua manipulação, utilizando a linguagem Python e a biblioteca `morph.py` (Zampirolli et al., 2025).

1.1 Objetivos

Ao final deste capítulo, você será capaz de:

- Compreender a natureza física e matemática da imagem digital $f(x, y)$.
- Identificar as faixas do espectro eletromagnético relevantes para PDI.
- Configurar o ambiente de desenvolvimento em Python.
- Realizar operações básicas: leitura, exibição e salvamento de imagens.
- Manipular estruturas de matrizes (NumPy) sem cair em armadilhas de memória.
- Acessar e modificar intensidades de pixels individualmente.
- Aplicar limiarização manual.

1.2 Antes de começar: *Notebooks* em Python

Este material foi construído sob o conceito de **Literate Programming** (Programação Literária), idealizado por Donald Knuth na década de 1980 (Knuth, 1984). Knuth — também criador do sistema **TeX** para tipografia digital — propôs que os programas fossem escritos como uma narrativa lógica para seres humanos, intercalando código e documentação.

Para executar uma célula, pressione Shift + Enter ou clique no botão ▷.

i Nota sobre o formato

Nas versões renderizadas (PDF ou HTML), o código é apresentado em blocos estáticos para fins de leitura e referência. A execução interativa requer o acesso via Google Colab (disponível no topo da página) ou em ambiente local via VSCode ou Jupyter Notebook.

1.3 Fundamentos

O estudo de sistemas baseados em imagens compreende um ecossistema de disciplinas integradas que transformam dados visuais brutos em conhecimento estruturado. Enquanto algumas áreas focam na geração de representações, outras dedicam-se ao tratamento e à análise desses dados para dar suporte a aplicações tecnológicas complexas.

O diagrama apresentado no Figure 1.1 estabelece a distinção e complementaridade entre o Processamento Digital de Imagens (PDI) e a Visão Computacional (VC). O PDI, destacado em **verde**, tem como foco

a transformação de imagem para imagem, visando melhoria de qualidade ou pré-processamento, como a remoção de ruídos e realce de contraste.

Em contrapartida, a VC, assinalada em **azul**, foca na interpretação do conteúdo visual para extrair modelos ou informações, como o reconhecimento de objetos e gestos. A região de intersecção ilustra a sinergia entre as áreas, onde o PDI prepara os dados visuais para a interpretação pela VC. O mapa também demonstra as interconexões de ambas as disciplinas com áreas como Robótica, Computação Gráfica, Inteligência Artificial (IA) e Neurociência.

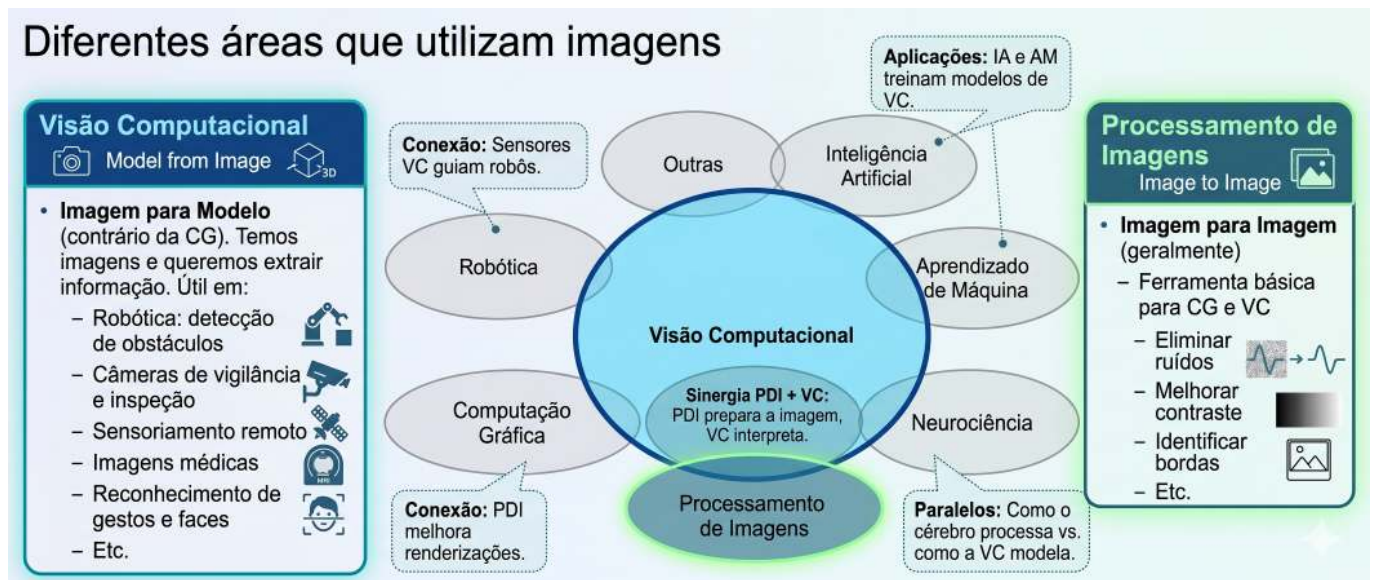


Figura 1.1: Diagrama relacional detalhando as distinções fundamentais, sinergias e interconexões entre PDI e VC no contexto de sistemas baseados em imagens.

1.3.1 Visão Computacional

- **Foco:** *Imagem* → *Modelo* (caminho inverso da Computação Gráfica).
- **Objetivo:** Extrair informação de alto nível a partir de imagens ou vídeos.
- **Aplicações típicas:**
 - Robótica - detecção de obstáculos, localização e navegação autônoma.
 - Vigilância e inspeção - reconhecimento de eventos, leitura de placas, controle de qualidade.
 - Sensoriamento remoto - análise de imagens de satélite, mapeamento ambiental.
 - Imagens médicas - detecção de tumores, segmentação de órgãos, auxílio ao diagnóstico.
 - Interação humano-computador - reconhecimento de gestos, expressões faciais, rastreamento ocular.
- **Relação com outras áreas:** utiliza técnicas de Aprendizado de Máquina e IA para classificar e interpretar cenas; serve como “olhos” da Robótica.

1.3.2 Processamento Digital de Imagens (PDI)

- **Foco:** *Imagem* → *Imagem* (geralmente - transformação de uma imagem em outra).
- **Objetivo:** Melhorar a qualidade visual ou extrair características de baixo nível.
- **Aplicações comuns:**
 - Eliminação de ruídos (filtros de média, mediana, gaussiano).
 - Melhoria de contraste (equalização de histograma, ajuste gamma).
 - Detecção de bordas (Sobel, Canny, Laplaciano).
 - Segmentação (limiarização, crescimento de regiões, *watershed*).

- Transformações geométricas (redimensionamento, rotação, correção de perspectiva).
- **Relação com outras áreas (ver Table 1.1):**
 - É a **base** para a maioria dos sistemas de VC (pré-processamento).
 - A Computação Gráfica frequentemente aplica PDI para pós-processamento (ex.: suavização, realce).
 - Técnicas de IA podem otimizar parâmetros de processamento (ex.: aprendizado de filtros).

Tabela 1.1: Conexão entre PDI, VC e outras áreas da ciência.

Área	Relação com PDI e VC
Inteligência Artificial	Fornecer modelos (redes neurais, SVM) que interpretam saídas da VC.
Robótica	Consome dados de VC para tomar decisões (navegação, manipulação).
Aprendizado de Máquina	Usa descritores extraídos pelo PDI/VC para treinar classificadores.
Computação Gráfica	Caminho inverso: <i>modelo</i> $\rightarrow$ <i>imagem</i> ; muitas vezes aplica PDI para renderização realista.
Neurociência	Inspira modelos de PDI (ex.: filtros semelhantes a células ganglionares da retina).

1.4 Etapas do PDI

As etapas do PDI são apresentadas na Figure 1.2, que podem ser compreendidas como uma cadeia de transformações que reduz a redundância dos dados em busca de significado:

- **Baixo Nível:** Atua diretamente sobre os pixels da imagem ruidosa para realizar melhorias e filtrações, gerando como saída uma imagem limpa ou realçada.
- **Médio Nível:** Recebe a imagem tratada e realiza a segmentação e descrição, transformando a matriz de pixels em atributos estruturados (forma, tamanho e textura).
- **Alto Nível:** Utiliza a tabela de atributos para alimentar processos de lógica e inteligência artificial, resultando na decisão ou reconhecimento final (como o diagnóstico médico).

A Figure 1.3 detalha a sequência completa do PDI, desde a captura até a interpretação. O fluxo inicia-se na Aquisição da Imagem (1) e prossegue com o Melhoramento (2) e a Restauração (3). Em seguida, o conteúdo é isolado pela Segmentação (4) e refinado pela Morfologia (5). A transição crucial ocorre na Representação e Descrição (6), onde objetos visuais são convertidos em dados matemáticos (área, perímetro etc.), permitindo o Reconhecimento (7). Processos auxiliares incluem o Processamento de Imagem Colorida e a Compressão, que contribuem para a eficiência do armazenamento e da análise.

1.5 Formação da Imagem e o Espectro

O processo de formação de uma imagem é fundamentado na interação entre a matéria e a energia radiante. Essencialmente, uma imagem é concebida quando um **sensor registra a radiação** resultante da interação com um objeto físico. No contexto da visão humana e da fotografia convencional, este fenômeno depende de uma fonte de luz que ilumine a cena; as características dos objetos são então codificadas através das **variações de intensidade e cor** da luz que atinge o sensor, conforme ilustrado na Figure 1.4.

A **luz visível** ocupa apenas uma pequena faixa do espectro eletromagnético — entre **380 nm (violeta)** e **750 nm (vermelho)** — conforme ilustrado na Figure 1.5. Sensores digitais convencionais operam nessa mesma janela, mas equipamentos especializados podem captar radiações invisíveis ao olho humano, como o infravermelho e os raios X. Em PDI, a imagem formada depende diretamente da sensibilidade espectral do sensor utilizado.

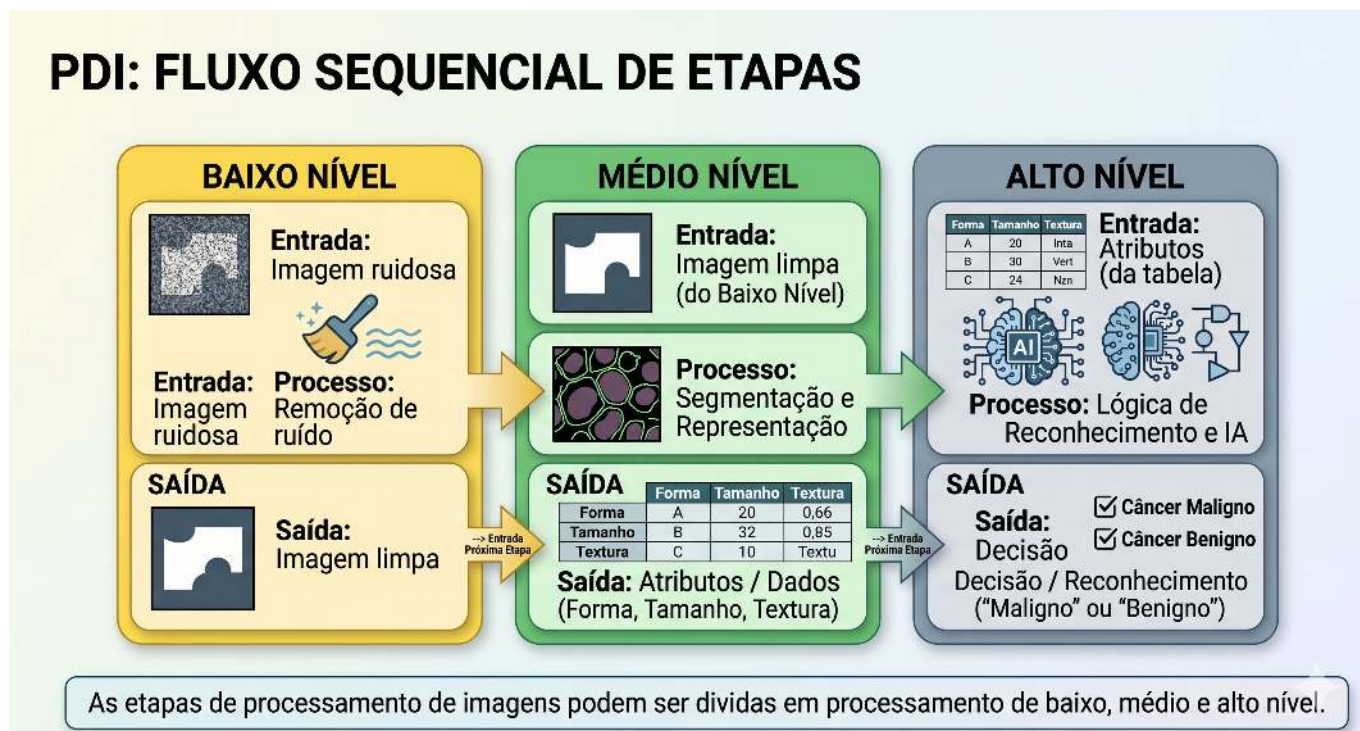


Figura 1.2: Representação do fluxo sequencial de processamento: a saída de cada nível torna-se a entrada do nível subsequente.

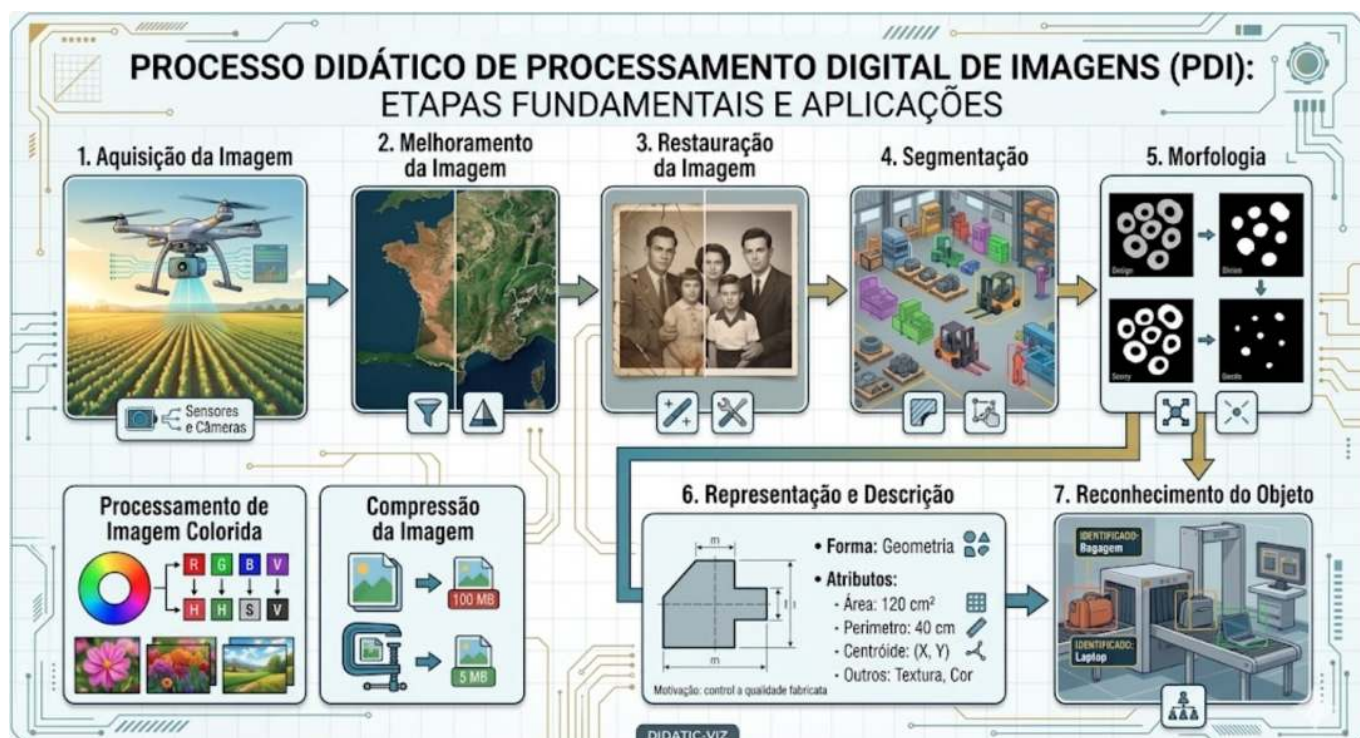


Figura 1.3: Fluxo detalhado do PDI: da aquisição sensorial à extração de atributos e reconhecimento automatizado, incluindo em processamento colorido e compressão.

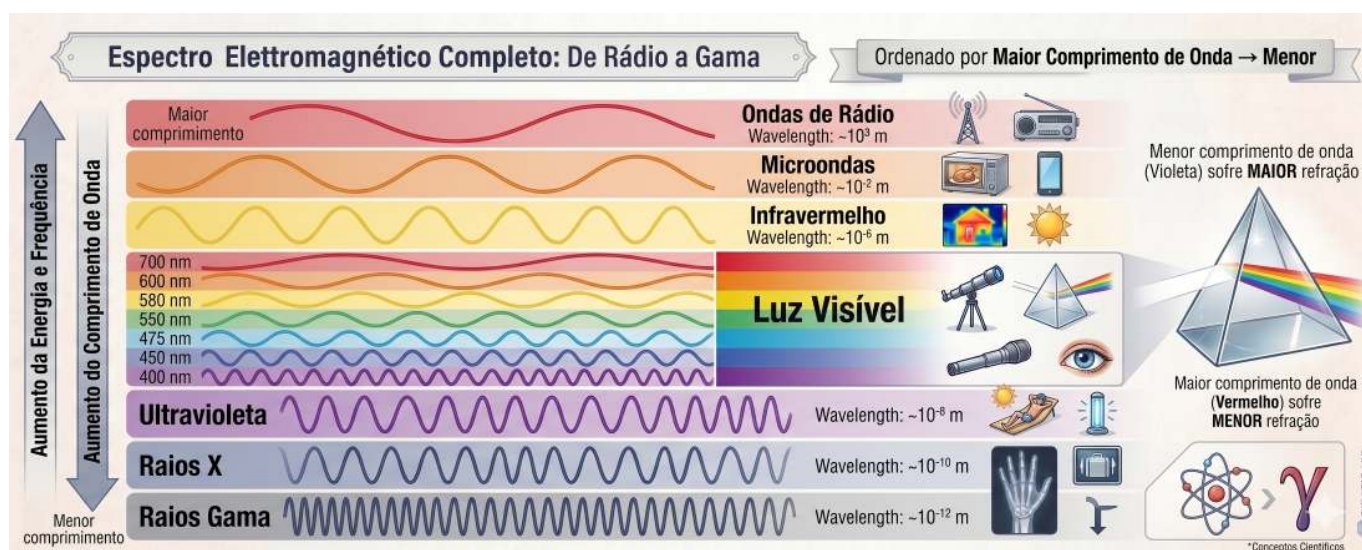


Figura 1.4: Representação do espectro visível e sua posição em relação às demais radiações eletromagnéticas, destacando a variação dos comprimentos de onda de 380 nm a 750 nm.

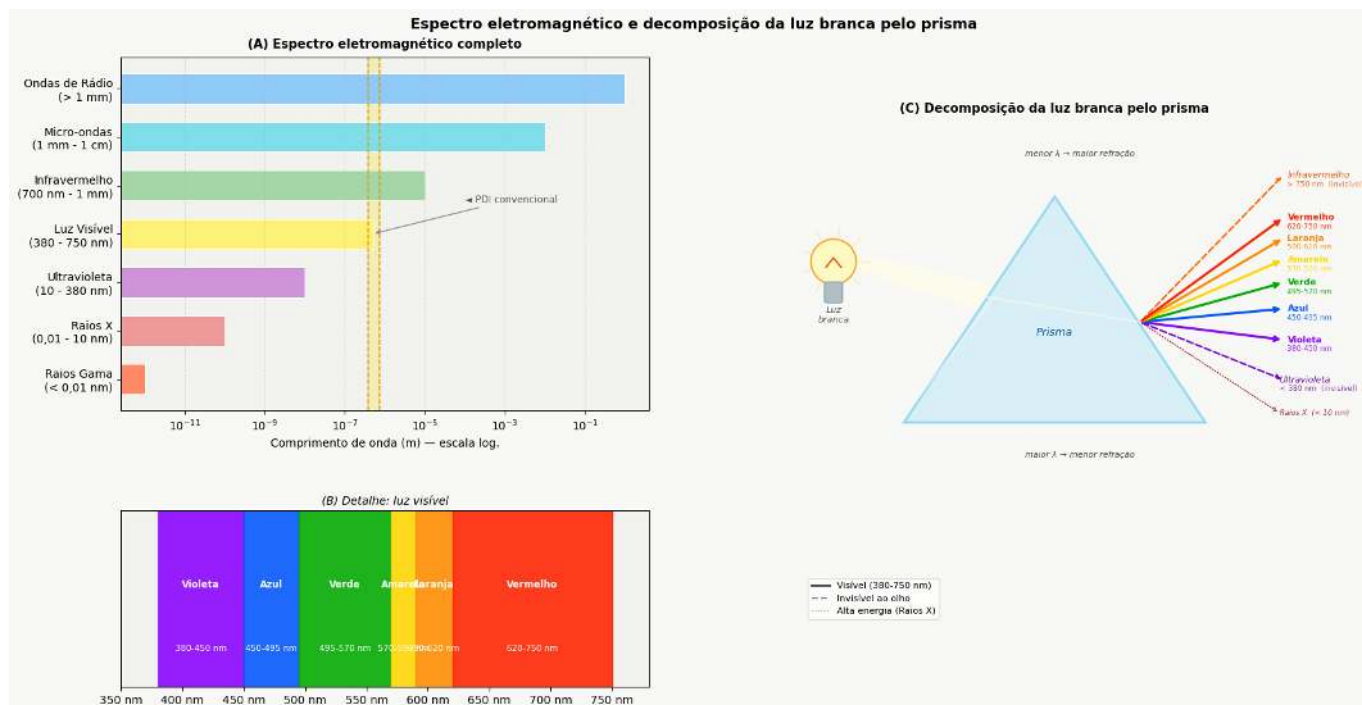


Figura 1.5: (A) Espectro eletromagnético completo em escala logarítmica, com destaque para a faixa visível. (B) Detalhe da luz visível (380-750 nm) e suas cores. (C) Decomposição da luz branca pelo prisma: menor comprimento de onda sofre maior refração, separando UV, visível e infravermelho.

A partir desse processo físico de aquisição, torna-se possível modelar matematicamente a imagem digital como uma função bidimensional discreta, na qual cada ponto da cena é representado por amostras numéricas de intensidade luminosa, formalizando assim os conceitos de pixel e de imagem digital apresentados na próxima seção.

1.6 O que é uma Imagem Digital?

Uma **imagem digital** é formada por uma grade de **pixels** (*Picture Elements*), onde cada pixel é a menor unidade elementar da imagem.

💡 O que é um Pixel?

Um **pixel** é a menor unidade endereçável que compõe uma imagem digital. Cada pixel ocupa uma posição única na grade e armazena um ou mais valores numéricos que representam sua intensidade ou cor.

Representação Matemática

Diferente de uma função contínua, o domínio de uma imagem digital é um plano retangular finito $\mathbb{E} \subset \mathbb{Z}^2$, que representa a grade de amostragem. Este domínio é indexado por coordenadas inteiras:

$$\mathbb{E} = \{(x, y) \in \mathbb{Z}^2 \mid 0 \leq x < L, 0 \leq y < H\} \tag{1.1}$$

Onde:

- L : representa a largura da imagem (número de colunas).
- H : representa a altura da imagem (número de linhas).

A imagem digital é uma função que associa cada par de coordenadas (x, y) a um ou mais valores que descrevem a aparência do pixel.

$$f : \mathbb{E} \rightarrow \mathcal{V} \tag{1.2}$$

O conjunto $\mathcal{V}$ define os valores possíveis para o pixel (codomínio), variando conforme o tipo de imagem, como demonstrado na Table 1.2.

Tabela 1.2: Principais tipos de imagem digital e seus respectivos conjuntos de valores possíveis para cada pixel.

Tipo de imagem	$\mathcal{V}$ (valores do pixel)	Representação
Binária	$\{0, 1\}$ ou $\{0, 255\}$	■ □
Tons de cinza	$\{0, 1, \dots, 255\}$	[] [=] [#] ■
Colorida (RGB)	$\{0, \dots, 255\}^3$ (triplas ordenadas de valores)	■ ■ ■

Exemplo prático: Uma imagem colorida no modelo RGB pode ser representada matematicamente por uma função que associa três valores de intensidade a cada pixel. Computacionalmente, essa representação corresponde a três matrizes sobrepostas — os canais vermelho (*Red*), verde (*Green*) e azul (*Blue*) — nas quais cada elemento armazena a intensidade luminosa do respectivo canal em uma determinada posição da imagem.

Para viabilizar os experimentos práticos de PDI e VC, este livro utiliza o ecossistema científico do Python, integrando bibliotecas voltadas à manipulação matricial, ao processamento de imagens e à visualização de resultados, apresentadas a seguir.

1.7 Configuração do Ambiente

Este material fundamenta-se no ecossistema científico do Python, com destaque para o **NumPy** (matemática matricial), **OpenCV** (VC padrão de mercado) e a biblioteca **morph.py** (Zampirolli et al., 2025), desenvolvida especificamente para fins didáticos neste livro, ver Table 1.3.

Atualmente, o projeto disponibiliza a versão em **Python** e **Português**. A arquitetura do sistema, no entanto, foi projetada para expansão futura, permitindo a adaptação automatizada para outras linguagens de programação (como C++ e Java) e idiomas, por meio de APIs de tradução.

Os **Exercícios de Programação (EPs)** do final de cada capítulo integram o módulo `testsuite.py`, já disponível para práticas em Python, C, C++, Java, JavaScript e R, garantindo a validação imediata das soluções propostas nos *notebooks* de estudo.

Tabela 1.3: Principais bibliotecas utilizadas ao longo deste livro para manipulação matricial, PDI e visualização de resultados.

Biblioteca	Função principal
<code>numpy</code>	Representação matricial de imagens
<code>opencv-python</code>	Leitura, escrita e operações de VC
<code>matplotlib</code>	Visualização de imagens e gráficos
<code>morph.py</code>	Abstração didática das operações de PDI

Versões do morph.py

A biblioteca `morph.py` possui duas versões públicas:

- **Versão 1.0** — versão original, publicada junto ao artigo apresentado no EduComp 2024: <https://github.com/fzampirolli/morph>
- **Versão 1.1** — versão compacta, utilizada nas atividades deste livro: <https://github.com/fzampirolli/pdi-vc/blob/master/morph/morph.py>

A versão 1.1 foi necessária para uso nas atividades do **Moodle/VPL** (Laboratório Virtual de Programação). Na versão 1.0, bibliotecas como `matplotlib`, `requests` e `skimage` eram carregadas no momento do `import morph`, causando erro de memória (Jail: out of memory, 128MiB) no ambiente restrito do VPL.

Na versão 1.1, esses *imports* foram convertidos para **carregamento lazy**: cada biblioteca é importada apenas dentro do método que a utiliza, e somente quando esse método é de fato chamado. Assim, um simples `import morph` carrega apenas `numpy` e `cv2`, que são suficientes para a maioria dos EPs.

```
import os, importlib, urllib.request, numpy as np, matplotlib.pyplot as plt

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

# Verifica se o atributo existe antes de imprimir
version = getattr(morph, "__version__", "local_file")

print(f" Ambiente pronto. Módulo 'morph' carregado com sucesso (Versão: {version}).")
print(f" Funções disponíveis no mm: \n{dir(mm)[-5:]}...")
```

Ambiente pronto. Módulo 'morph' carregado com sucesso (Versão: 1.1.5).
 Funções disponíveis no mm:
 ['verifyBoundingBox_old', 'watershed', 'watershed0', 'watershedB', 'write']...

1.8 Fundamentos de Matrizes - atenção à cópia de referências

Como uma imagem digital é uma matriz, precisamos saber criá-las corretamente. Em Python, existe uma armadilha comum ao usar o operador `*` em listas:

⚠ Atenção à cópia de referências

Ao fazer `m = [[0]*2]*3`, você não cria 3 linhas independentes, mas sim 3 referências para a **mesma** linha. Alterar um valor em uma linha alterará todas as outras!

Para visualizar esse comportamento, você pode testar o código no [Python Tutor](#) e comparar com a forma correta de criar matriz com listas: `m = [[0]*2 for _ in range(3)]`.

A forma recomendada e padrão em PDI é usar **NumPy**, que cria matrizes com dados independentes e oferece eficiência computacional. O código a seguir mostra diferentes formas de criação de imagens sintéticas — o resultado é exibido na Figure 1.6.

```
# Criando uma imagem preta (zeros) de 4, 6 pixels
img_preta = np.zeros((4, 6), dtype='uint8')
img_preta[0,0] = 255 # pixel branco no canto superior esquerdo

# Criando uma imagem branca (255) de 4, 6 pixels
img_branca = np.ones((4, 6), dtype='uint8') * 255
img_branca[3,5] = 0 # pixel preto no canto inferior direito

# Criando uma imagem aleatória para testes (ruído)
img_random = mm.randomImage(4, 6, maxValue=255)

print("Matriz aleatória gerada:")
print(mm.drawImg(img_random))

mm.show(
    [img_preta, img_branca, img_random],
    titles=[
        "Predominantemente preta\n(com 1 pixel branco em (0,0))",
        "Predominantemente branca\n(com 1 pixel preto em (3,5))",
        "Imagem aleatória\n(simulação de ruído)"
    ],
    cols=3,
    figsize=(9, 3),
    axis=True
)
```

Matriz aleatória gerada:

39	121	181	42	98	138
123	145	115	115	177	105
211	12	186	67	254	64
243	162	229	133	251	50

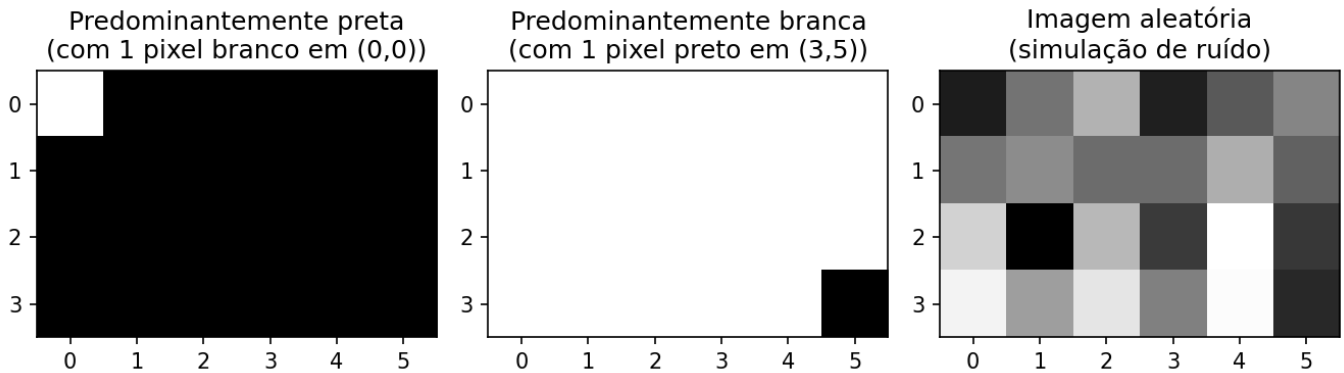


Figura 1.6: Exemplos de imagens sintéticas representadas matricialmente.

1.9 Lendo e Exibindo Imagens

Em bibliotecas como o [NumPy](#), [OpenCV](#) e [scikit-image](#), imagens digitais são frequentemente representadas computacionalmente como *arrays* do NumPy. Dessa forma, operações matriciais e conceitos de álgebra linear podem ser aplicados diretamente ao PDI.

Uma das operações fundamentais em PDI é a leitura de imagens. Na biblioteca `morph.py`, a função `mm.read()` permite carregar imagens tanto de arquivos locais quanto de URLs, como ilustrado na Figure 2.7.

```
import os
import numpy as np

url = "https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg"
caminho = "imagens/mandrill.png"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img = np.array(img_obj)

print(f"Dimensões (H, W, Canais): {img.shape}")
print(f"Tipo de dado: {img.dtype}")

mm.show(img, title="Exemplo: Captura RGB")
```

```
Dimensões (H, W, Canais): (1365, 2048, 3)
Tipo de dado: uint8
```


Exemplo: Captura RGB



Figura 1.7: Mandrill (*Mandrillus sphinx*) em ambiente natural. Crédito: Julien Renoult (CC BY 4.0).

Alternativa: *Download* da Imagem para Armazenamento Local

Em ambientes nos quais a leitura direta de URLs esteja indisponível — devido a restrições de rede, políticas de *firewall* ou ausência de conectividade — uma alternativa consiste em realizar previamente o *download* da imagem para o sistema de arquivos local. Após o armazenamento, a imagem pode ser carregada normalmente pela função `mm.read()`. No exemplo a seguir, o arquivo é salvo localmente com o nome `mandrill.png`.

```
!wget -O mandrill.png \
    https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg
```

```
img = mm.read('mandrill.png')
mm.show(img, title="Exemplo: Captura RGB (arquivo local)")
```

Explicação

- `wget -O mandrill.png <URL>` realiza o *download* da imagem e a armazena localmente com o nome especificado;
- `mm.read('mandrill.png')` efetua a leitura do arquivo diretamente do sistema de arquivos, sem necessidade de requisições HTTP adicionais;
- essa estratégia reduz a dependência de conectividade durante a execução dos experimentos e evita *downloads* repetidos da mesma imagem.

Nota

O prefixo `!` é utilizado em ambientes baseados em *notebooks*, como [Jupyter Notebook](#), [JupyterLab](#) e [Google Colab](#), para executar comandos do sistema operacional diretamente em células de código. Em terminais convencionais, o comando deve ser utilizado sem o prefixo `!`.

Caso o utilitário `wget` não esteja instalado, pode-se utilizar alternativamente:

```
!curl -o mandrill.png \
    https://upload.wikimedia.org/wikipedia/commons/8/87/Mandrillus_sphinx_339428057.jpg
```

1.10 Conversão de Tipos e Limiarização

Conforme vimos, uma imagem colorida no espaço RGB é representada pela função:

$$f : \mathbb{E} \rightarrow \{0, 1, \dots, 255\}^3$$

Ou seja, para cada pixel (x, y) , temos três valores (R, G, B) que definem sua cor.

Conversão para Tons de Cinza

Para converter uma imagem RGB em tons de cinza (*grayscale*), é necessário combinar os três canais em um único valor de intensidade g , que representa o brilho percebido. Como o olho humano não é igualmente sensível ao vermelho, verde e azul, utiliza-se uma **média ponderada**. O padrão [ITU-R BT.601](#) (ITU-R, 2011) define os seguintes pesos:

$$g = 0.299 R + 0.587 G + 0.114 B \quad (1.3)$$

Após o cálculo, o valor g é arredondado para o inteiro mais próximo e ajustado ao intervalo $[0, 255]$. O resultado é uma nova imagem, agora em tons de cinza, representada por:

$$f_{\text{cinza}} : \mathbb{E} \rightarrow \{0, 1, \dots, 255\}$$

Limiarização (*Thresholding*)

A partir da imagem em tons de cinza $f_{\text{cinza}}(x, y)$, uma operação fundamental é a **limiarização**, que produz uma imagem **binária** (apenas preto e branco). Para isso, escolhe-se um valor de corte T (geralmente no intervalo $[0, 255]$) e define-se:

$$f_{\text{bin}}(x, y) = \begin{cases} 255 & \text{se } f_{\text{cinza}}(x, y) > T \\ 0 & \text{caso contrário} \end{cases} \quad (1.4)$$

Exemplo: Com $T = 128$, pixels com intensidade acima de 128 tornam-se brancos (255); os demais tornam-se pretos (0).

A limiarização é amplamente usada para **segmentar objetos do fundo**, extrair bordas ou criar máscaras binárias para processamento posterior.

Nota: O valor 255 representa o branco máximo em imagens de 8 bits, enquanto 0 representa o preto absoluto.

Exemplo prático de conversão e limiarização

A Figure 1.8 ilustra os principais passos para transformar uma imagem colorida em tons de cinza e, em seguida, convertê-la em uma imagem binária por limiarização. O código a seguir implementa essas etapas:

```
# 1. Converter para Tons de Cinza
img_gray = mm.gray(img)

# 2. Aplicar limiar (Pixels > 128 tornam-se 255, outros 0)
limiar = 128
img_binaria = mm.threshold(img_gray, limiar)

# Uso da nova função
mm.show(
    [img, img_gray, img_binaria],
```

```
titles=["Original", "Tons de Cinza", f"Binária (T={limiar})"],
cols=3
)
```



Figura 1.8: Processamento básico de imagens: (a) imagem original, (b) imagem em tons de cinza, (c) imagem binarizada por limiar ($T=128$).

1.11 Limiarização pelo método de Otsu

Conforme apresentado na Equation 1.4, a limiarização converte uma imagem em tons de cinza para binária usando um valor de corte T . Até agora fixamos $T = 128$ manualmente.

```
img_bin_fixo = mm.threshold(img_gray, T=128)
```

Entretanto, a escolha manual de T nem sempre é trivial. A biblioteca `mm` oferece uma alternativa automática: quando o parâmetro `limiar` **não é fornecido**, a função `mm.threshold(img_gray)` calcula o valor de T pelo **método de Otsu** (Otsu, 1979). Esse método, que será detalhado em capítulos futuros, maximiza a variância entre classes do histograma (frequência de cada tom de cinza), separando automaticamente os pixels de objeto e fundo.

O código abaixo compara a limiarização manual ($T = 128$) com a automática (Otsu), mostrando também o valor de T calculado:

```
import cv2
# Limiarização com T fixo (manual)
T_fixo = 128
img_bin_fixo = mm.threshold(img_gray, T_fixo)

# Limiarização pelo método de Otsu (T automático)
T_otso, img_bin_otso = cv2.threshold(img_gray, 0, 255,
                                     cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f'Limiar calculado por Otsu: T = {T_otso}')
# ou simplesmente:
# img_bin_otso = mm.threshold(img_gray)

# Exibição lado a lado
mm.show(
    [img_gray, img_bin_fixo, img_bin_otso],
    titles=["Tons de Cinza", f"Binária (T={T_fixo})", f"Binária (Otsu, T={T_otso})"],
    cols=3
)
```

```
Limiar calculado por Otsu: T = 95.0
```



Figura 1.9: Comparação entre limiarização manual ($T=128$) e automática (Otsu) sobre a imagem em tons de cinza.

A Figure 1.9 mostra que o limiar obtido por Otsu se adapta automaticamente à imagem, resultando em uma binarização mais eficiente do que um valor fixo, especialmente quando as intensidades do objeto e do fundo são bem separadas no histograma. Essa técnica é amplamente utilizada em sistemas de VC para binarização de documentos, detecção de objetos e pré-processamento de imagens.

💡 Simplicidade da biblioteca `morph.py`

Enquanto o OpenCV exige a chamada completa:

```
T_otsu, img_bin = cv2.threshold(img_gray, 0, 255,
                                cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

a biblioteca `mm` abstrai toda essa complexidade: basta chamar `mm.threshold(img_gray)`. O limiar de Otsu é calculado automaticamente e a imagem binária é retornada diretamente. Essa abordagem permite concentrar-se no conceito, não nos detalhes de implementação.

1.12 Acesso a Pixels

Em Python com NumPy, uma imagem é estruturada como um **array multidimensional**. O acesso a um pixel específico utiliza a convenção matricial **linha** (eixo Y) e **coluna** (eixo X): `img[linha, coluna]`.

O código da Figure 1.10 demonstra como extrair esses valores em imagens coloridas (RGB) e em tons de cinza, além de isolar a vizinhança imediata do ponto de interesse.

```
import matplotlib.pyplot as plt

# 1. Definição das coordenadas do pixel alvo
r, c = 600, 800

# 2. Acesso direto aos valores dos pixels
pixel_cinza = img_gray[r, c]
pixel_rgb = img[r, c]

print(f" VALORES NO PIXEL ALVO ({r}, {c}):")
print(f"   • Tons de cinza (Escalar) : {pixel_cinza}")
print(f"   • Colorido (Vetor RGB)      : R={pixel_rgb[0]}, G={pixel_rgb[1]}, B={pixel_rgb[2]}")
print(f"   -" * 50)

# 3. Exibição didática da vizinhança 3x3 ao redor do pixel (r, c)
# Recorta da linha r-1 até r+1, e da coluna c-1 até c+1
vizinhanca_gray = img_gray[r-1:r+2, c-1:c+2]

print(f" MATRIZ DE VIZINHANÇA 3x3 EM TONS DE CINZA:")
```



```

print(f"    (0 pixel alvo central está destacado com entre colchetes)\n")

# Impressão formatada para destacar o pixel central
for i, linha in enumerate(vizinhanca_gray):
    linhas_str = []
    for j, valor in enumerate(linha):
        if i == 1 and j == 1:
            linhas_str.append(f"[{valor:3d}]") # Destaca o centro (100, 100)
        else:
            linhas_str.append(f"{valor:3d} ")
    print("    " + " ".join(linhas_str))

# 4. Demonstração visual usando o mm.show (opcional, mas excelente para o livro)
# Mostra a imagem inteira com um ponto demarcando a região
plt.figure(figsize=(5, 5))
plt.imshow(img)
plt.plot(c, r, 'ro', markersize=8, label=f'Pixel ({r},{c})') # Plota (x, y) -> (c, r)
plt.title(f"Localização do Pixel ({r}, {c}) na Imagem")
plt.legend()
plt.axis('on') # Mantém os eixos para o estudante ver as coordenadas 100, 100
plt.show()

```

VALORES NO PIXEL ALVO (600, 800):

- Tons de cinza (Escalar) : 81
- Colorido (Vetor RGB) : R=92, G=78, B=67

MATRIZ DE VIZINHANÇA 3x3 EM TONS DE CINZA:

(0 pixel alvo central está destacado com entre colchetes)

```

83   96  105
85  [ 81]  96
83   77   84

```

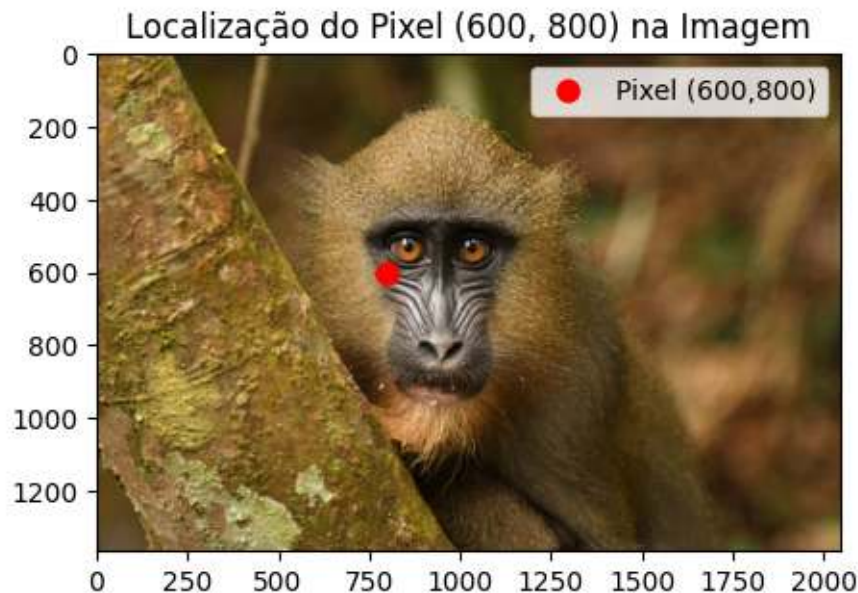


Figura 1.10: Acesso a um pixel específico (r, c) e a representação de sua vizinhança na imagem.

Explicação linha a linha:

1. `img_gray[r, c]` — Retorna um único valor inteiro (escalar) entre 0 e 255, que representa a intensidade de brilho do cinza.

2. `img[r, c]` — Retorna um vetor com três valores `[R, G, B]`, correspondentes às intensidades dos canais Vermelho, Verde e Azul.
3. `img_gray[r-1:r+2, c-1:c+2]` — Realiza um fatiamento (*slicing*) para extrair a submatriz 3×3 ao redor do pixel. Esta operação é a base para a implementação de filtros espaciais e convoluções.
4. O laço `for` subsequente apenas formata essa submatriz no console, exibindo o pixel central entre colchetes `[ ]` para fins didáticos.
5. A função `plt.plot(c, r, 'ro')` plota um ponto vermelho sobre a imagem para correlacionar a coordenada numérica com sua posição visual. Note que o Matplotlib utiliza a ordem padrão de tela `(X, Y)`, invertendo para `(c, r)`.

Como a indexação em Python começa em zero (*zero-based*), o canto superior esquerdo da imagem é a coordenada `(0,0)`. Guarde sempre esta regra: a primeira dimensão da matriz controla a **altura** (linhas/Y) e a segunda controla a **largura** (colunas/X).

1.13 Resumo

Neste capítulo, apresentaram-se os fundamentos de representação de imagens digitais: a definição de pixel, a estruturação de imagens em matrizes e o impacto da amostragem e da quantização na qualidade final:

- **Imagem digital** = função $f(x, y)$ que mapeia coordenadas para intensidades (escalares ou vetoriais).
- **Domínio**: conjunto finito $\mathbb{E} = \{(x, y) \in \mathbb{Z}^2 \mid 0 \leq x < L, 0 \leq y < H\}$.
- **Tipos principais**: binária ($\mathcal{V} = \{0, 255\}$), tons de cinza ($\mathcal{V} = [0, 255]$) e RGB ($\mathcal{V} = [0, 255]^3$).
- A biblioteca `morph.py` (ou `mm`) oferece funções didáticas para operações básicas de PDI, como `mm.gray()`, `mm.threshold()`, `mm.show_multiple()`.
- **Armadilha NumPy**: nunca use `[[0]*n]*m` para criar matrizes — use sempre `np.zeros()` ou `np.ones()`.
- **Limiarização** converte tons de cinza em binária; o **método de Otsu** determina o valor de corte automaticamente maximizando a variância entre classes.
- Acesso a pixels via `img[linha, coluna]`, com indexação *zero-based*.

O Capítulo 2 abordará **histogramas e equalização de contraste**.

1.14 Uso do Gemini Notebook como Tutor Complementar

Nesta edição, além dos *notebooks* interativos no Google Colab, o **Gemini Notebook** está disponível como ferramenta complementar de estudo. A plataforma utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro.

Estude com o Tutor Inteligente

Acesse o ambiente do capítulo pelo *link* abaixo e explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR Gemini Notebook: CAPÍTULO 01](#)

Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

Funcionalidades Disponíveis na Plataforma

O **Gemini Notebook** oferece uma suíte avançada de ferramentas baseadas em IA para transformar o conteúdo estático do livro em uma experiência de aprendizado dinâmica e multimídia. Conforme ilustrado

na Figure 1.11, a plataforma utiliza técnicas de **RAG** (*Retrieval-Augmented Generation*), fundamentadas no trabalho de Lewis et al. (2020), para basear as respostas estritamente nos documentos fornecidos e minimizar a ocorrência de alucinações.

As principais funcionalidades incluem:

- **Resumos Multimodais (Áudio e Vídeo):** Geração de conversas naturais entre especialistas no formato de **Resumo em Áudio** (estilo *podcast*) e **Resumo em Vídeo**, discutindo os temas centrais do capítulo, como as diferenças entre PDI e VC, ou a interpretação de transformações como a limiarização e o método de Otsu.
- **Visualização de Estruturas (Mapa Mental e Infográfico):** Criação automática de diagramas que conectam visualmente os conceitos, por exemplo, o fluxo de processamento desde a captura da imagem digital, passando pela conversão para tons de cinza, limiarização e segmentação binária.
- **Ferramentas de Avaliação (Teste e Cartões Didáticos):** Geração de **Testes** de múltipla escolha e **Cartões Didáticos** (*flashcards*) para fixação de conhecimento, baseados no texto autoral (ex.: perguntas sobre a fórmula de conversão RGB→cinza ou sobre o funcionamento do limiar global e de Otsu).
- **Apoio à Apresentação (Slides e Relatórios):** Auxílio na estruturação de **Apresentações de Slides** e na redação de **Relatórios** técnicos, facilitando a comunicação de resultados de experimentos com imagens.
- **Análise de Dados (Tabela de Dados):** Organização de dados extraídos do texto em tabelas estruturadas, auxiliando na compreensão de exemplos práticos, como a comparação entre diferentes valores de limiar.
- **Chat Contextualizado:** Permite o questionamento direto sobre o código e a teoria, como: “*Como implementar a conversão de RGB para tons de cinza usando os pesos do padrão ITU-R BT.601?*” ou “*O que acontece com a imagem binária se eu escolher um limiar $T=200$ em vez de $T=128$?*”.



Figura 1.11: Estúdio do Gemini Notebook.

1.15 Lista de Exercícios

1. (15%) Com suas próprias palavras, defina **imagem digital** e **pixel**. Dê um exemplo concreto de como uma imagem colorida (RGB) é representada matricialmente no computador.
2. (15%) Explique as diferenças entre **imagem binária**, **tons de cinza (8 bits)** e **colorida RGB**, indicando a faixa de valores possíveis para cada pixel em cada tipo.
3. (20%) Considerando a fórmula de conversão RGB → tons de cinza do padrão ITU-R BT.601:

$$g = 0.299 R + 0.587 G + 0.114 B$$

Calcule o valor do pixel em tons de cinza para $(R, G, B) = (80, 180, 30)$. Arredonde para o inteiro mais próximo.

4. (20%) O que é **limiarização** (*thresholding*)? Explique a diferença entre escolher um limiar T fixo (ex.: $T = 128$) e utilizar o **método de Otsu** para determinação automática do limiar. Em poucas palavras, como o método de Otsu escolhe o limiar?
5. (15%) No contexto da biblioteca didática `mm` discutida no capítulo, responda:
 - a) (7,5%) Como se acessa o valor do pixel na posição (linha=50, coluna=60) de uma imagem em tons de cinza `img_gray`?
 - b) (7,5%) Qual é a vantagem de usar `mm.threshold(img_gray)` sem passar o limiar? Compare com a chamada equivalente no OpenCV.
6. (15%) O que a propriedade `img.shape` retorna para uma imagem NumPy no formato RGB? Dê um exemplo concreto com uma imagem de 640×480 pixels.

Referências do Capítulo

A fundamentação teórica deste capítulo compreende as seguintes obras de PDI e VC:

- Gonzalez; Woods (2018) para os fundamentos de **Processamento Digital de Imagens** (PDI).
- Singh (2019) para a implementação prática de métodos de **processamento e análise de imagens**.
- Szeliski (2022) para o estudo de VC e algoritmos fundamentais.
- Bradski; Kaehler (2008) para a aplicação da biblioteca **OpenCV** em ambiente Python.
- Lewis et al. (2020) para o conceito de **geração aumentada por recuperação (RAG)**, utilizado no suporte ao processamento de informações deste material.



1.16 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

Os **Exercícios de Programação (EPs)** apresentados a seguir podem ser submetidos também em atividades do Moodle (atividades VPL) que fornecem *feedback* automático.

Este caderno foi desenvolvido para superar limitações de uso do Moodle. Com ele, deve-se:

1. **Desenvolver:** Escrever e editar sua solução diretamente no ambiente Colab.
2. **Validar:** Testar seu código localmente utilizando os **mesmos casos de teste** do Moodle.
3. **Organizar:** Salvar seus códigos das atividades VPL de forma segura.
4. **Avaliar:** Quando estiver conectado ao Moodle, basta copiar sua solução e clicar em **Avaliar** no Moodle (se estiver na rede da UFABC) para registrar sua nota oficial.

§ Instruções Passo a Passo

Em um ambiente de execução (como VSCode, Jupyter ou Colab), siga a ordem abaixo para configurar o ambiente e validar seus exercícios:

Preparação do Ambiente

Execute a célula de código abaixo para baixar `morph.py` e `testsuite.py` do repositório da disciplina — apenas se ainda não existirem no diretório local. Com ambos em `./`, o *notebook* e os subprocessos do `TestSuite` encontram o módulo sem configurações extras de caminho.

Nota: O script `testsuite.py` buscará automaticamente os casos de teste em `all/{cap}/cases` no [GitHub](#).

Escrevendo o Código

Salve sua solução em uma célula de código usando o comando mágico `%%writefile`. O nome do arquivo deve seguir o padrão `EPX_Y.*`, onde `X` é o capítulo, `Y` é o exercício e `*` é a extensão da linguagem.

Exemplo: `%%writefile EP01_01.py`

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.5 | TestSuite: 1.1.2

Executando os Testes

Após salvar o arquivo com sua solução, execute o comando abaixo (em uma nova célula) para avaliar os testes automáticos:

```
TestSuite("EP01_01.extensão").run()
```

Substitua a extensão conforme a linguagem usada:

Linguagem	Extensão
Python	.py
Java	.java
C	.c
C++	.cpp
JavaScript	.js
R	.r

Como funciona: O `TestSuite` baixa os casos de teste do GitHub, executa seu programa com cada entrada e compara a saída com a esperada – calculando automaticamente sua nota.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""

TestSuite("EP04_01").run_code(codigo)
```

⚠ Importante: Regras e Boas Práticas

♦ Sobre a Entrada de Dados

O seu programa deve ler a entrada padrão (teclado).

- **Python:** Utilize `input()`.
- **Outras linguagens:** Utilize o comando de leitura padrão equivalente (`cin`, `Scanner`, etc.).

♦ Configuração de IA no Colab

Para um melhor aprendizado, recomenda-se desativar o preenchimento automático de código por IA, pois não estará disponível durante as avaliações. Por exemplo no navegador Chrome:

- Vá em: **Ferramentas > Configurações > IA generativa**
- Desmarque: **Habilitar geração de código**

♦ Integridade Acadêmica (Plágio)

Este recurso de testes locais aplica-se a EPs **sem variações**. No entanto:

- **Individualidade:** Cada estudante deve desenvolver sua própria solução.
- **Deteção de Similaridade:** O professor utiliza ferramentas que detectam cópias, **mesmo com alteração de nomes de variáveis ou espaços em branco**.

1.16.1 EP01_01 📏 Três métricas de distância em PDI

Nesta atividade, você deve escrever um programa que calcule as três distâncias clássicas em PDI: **Euclidiana (L2)**, **City-Block (L1)** e **Chessboard (L_∞)**.

- Leia **4 números reais** que representam as coordenadas: A_x, A_y, B_x, B_y .
- Calcule as três distâncias utilizando as fórmulas:

$$d_{\text{Euclidiana}} = \sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}$$

$$d_{\text{City-block}} = |B_x - A_x| + |B_y - A_y|$$

$$d_{\text{Chessboard}} = \max(|B_x - A_x|, |B_y - A_y|)$$

- Imprima os três resultados, cada um em uma linha, formatados com **duas casas decimais**, na ordem: **Euclidiana**, **City-block**, **Chessboard**.

📌 Importante:

- Utilize as funções matemáticas padrão da sua linguagem: `math.sqrt`, `abs` (ou `fabs`) e `max`.
- A saída deve conter **apenas os números** (um por linha), sem textos adicionais.
- Ver um simulador interativo para esta questão na **Figure 1.12** (gráfico com arrasto dos pontos e visualização das três métricas).

1.16.1.1 🖼 Por que isso importa? – Custo computacional

Em uma imagem **1000×1000 pixels** (1 milhão de pixels), calcular a distância de cada pixel a um ponto de referência exige **1 milhão de operações**. A escolha da métrica afeta o desempenho:

Métrica	Operações por pixel	Custo relativo (1M pixels)	Quando usar
Euclidiana (L2)	2 subtrações, 2 multiplicações, 1 soma, 1 sqrt	 Mais custosa – sqrt é cara	Distância “real” no espaço contínuo
City-block (L1)	2 subtrações, 2 abs , 1 soma	 Moderada – sem raiz quadrada	Grids, robótica, imagens binárias
Chessboard (L∞)	2 subtrações, 2 abs , 1 max	 Mais eficiente	Movimentos de peças, morfologia

A função **sqrt** é computacionalmente mais cara que operações como adição, subtração, multiplicação e valor absoluto. Em CPUs modernas, a diferença pode ser pequena (cerca de $1,5\times$ a $3\times$), mas em sistemas embarcados ou em laços de milhões de iterações, qualquer ganho importa. Por isso, **quando o objetivo é apenas comparar distâncias** (ex.: encontrar o ponto mais próximo), use a **distância euclidiana ao quadrado**.

1.16.1.2 Tarefa (especificação para VPL)

Entrada:

Uma única linha com quatro números reais: A_x A_y B_x B_y

Saída:

Três linhas, cada uma com um número real de duas casas decimais (Euclidiana, City-block, Chessboard).

1.16.1.3 Exemplos

Entrada	Saída	Observação
0	5.00	Triângulo 3-4-5
0	7.00	
3	4.00	
4		
0	1.41	Diagonal unitária
0	2.00	
1	1.00	
1		

Exemplo de teste de **sqrt** em Python, com **timeit** isolando cada operação:

```
import math
import timeit

N = 50_000_000

def apenas_soma():
    a, b = 3.0, 4.0
    return a + b

def soma_e_sqrt():
    a, b = 3.0, 4.0
    return math.sqrt(a*a + b*b)
```

```
t_soma = timeit.timeit(apenas_soma, number=N)
t_sqrt = timeit.timeit(soma_e_sqrt, number=N)

print(f"Soma simples      : {t_soma:.3f} s")
print(f"Soma + sqrt       : {t_sqrt:.3f} s")
print(f"Razão (sqrt/soma)   : {t_sqrt/t_soma:.2f}x")
```

```
Soma simples      : 2.836 s
Soma + sqrt       : 5.602 s
Razão (sqrt/soma) : 1.97x
```

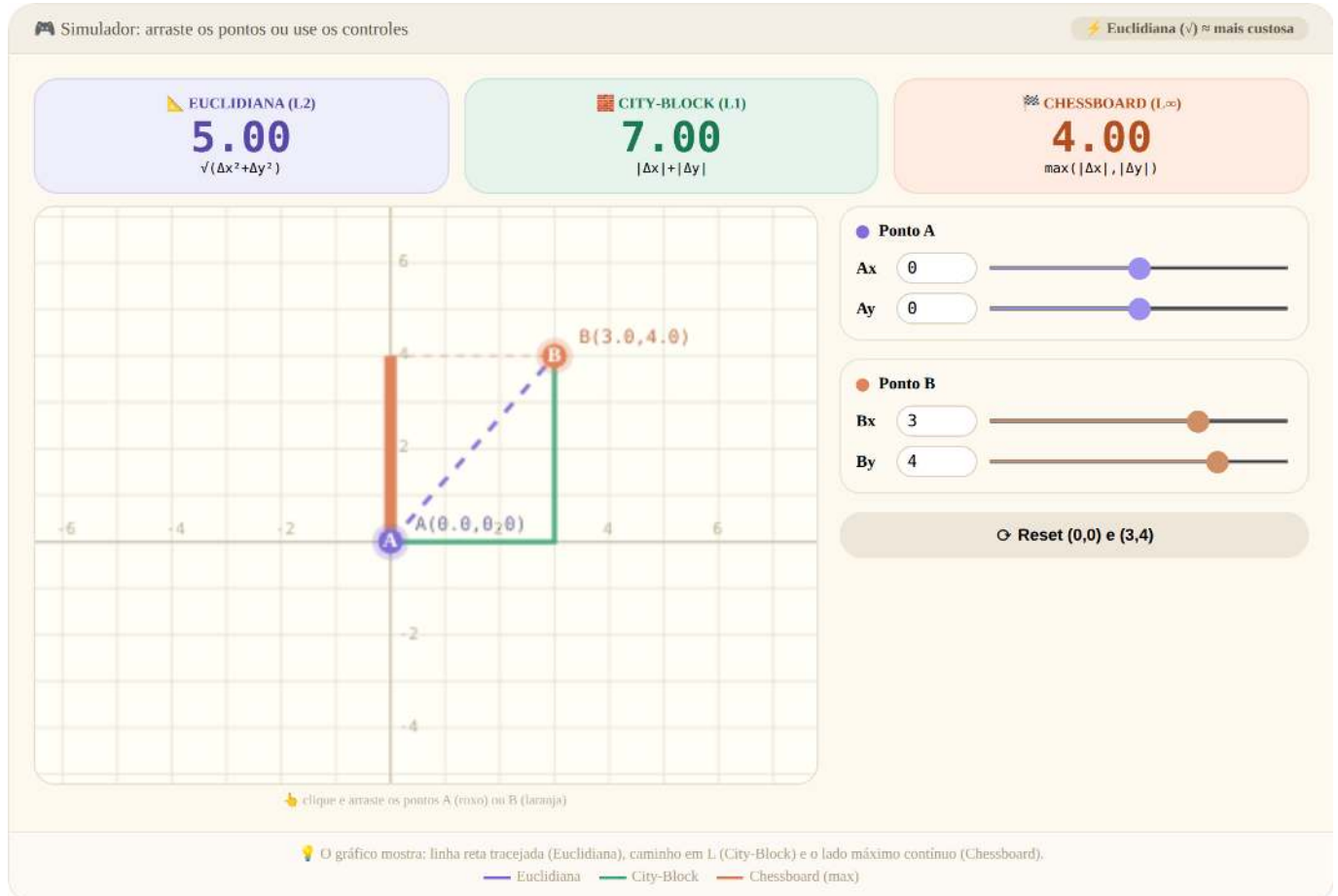


Figura 1.12: Simulador: Distâncias Euclidiana, *City-block* e *Chessboard*

1.16.1.4 🐍 Python

Basta criar uma célula de código normal e inserir o código Python. A entrada pode ser simulada usando `input()`, que funciona normalmente.

Exemplo de célula:

```
%%writefile EP01_01.py
# Código Python
x1,y1,x2,y2 = int(input()), int(input()), int(input()), int(input())
# Cálculo das diferenças
dx = abs(x2 - x1)
dy = abs(y2 - y1)

# 1. Distância Euclidiana (L2)
dist_euclidiana = (dx**2 + dy**2)**0.5
```

```
# 2. Distância City-block / Manhattan (L1)
dist_city_block = dx + dy

# 3. Distância Chessboard / Chebyshev (Linf)
dist_chessboard = max(dx, dy)

# Saída formatada conforme os casos de teste
print(f"{dist_euclidiana:.2f}")
print(f"{dist_city_block:.2f}")
print(f"{dist_chessboard:.2f}")
```

Writing EP01_01.py

```
# Espera que você digite 4 números inteiros ao executar esta célula.
# No Jupyter ou Google Colab, a mágica %run -i permite que o script leia do teclado.
# No terminal comum, você usaria: python3 EP01_01.py (sem o '!' e sem '%run').

# %run -i EP01_01.py
```

```
# Envia 4 inteiros como entrada padrão (stdin) para o script EP01_01.py usando um pipe
!echo -e "0\n0\n4\n4" | python3 EP01_01.py
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.py").run()
```

1.16.1.5 Java

Para executar Java no Colab, você precisa usar uma célula com o prefixo `%%writefile` para salvar o código em arquivo, compilar e executar.

```
%%writefile EP01_01.java
import java.util.Scanner;

class EP01_01 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        double x1 = s.nextDouble(), y1 = s.nextDouble();
        double x2 = s.nextDouble(), y2 = s.nextDouble();

        double dx = Math.abs(x2 - x1);
        double dy = Math.abs(y2 - y1);

        System.out.printf("%.2f\n", Math.sqrt(dx*dx + dy*dy));
        System.out.printf("%.2f\n", dx + dy);
        System.out.printf("%.2f\n", Math.max(dx, dy));
    }
}
```

Writing EP01_01.java

```
!javac EP01_01.java
!echo -e "0\n0\n4\n4" | java EP01_01
```

```
5,66
8,00
4,00
```

```
TestSuite("EP01_01.java").run()
```

1.16.1.6 C

De forma similar, use `%%writefile` para salvar o código, depois compile e execute. Para C, utilizamos o compilador **GCC**.

```
# Instalar o compilador GCC e ferramentas de build
# O build-essential inclui gcc, g++, make, etc.
import platform, shutil, subprocess

def instalar_gcc():
    if shutil.which("gcc"):
        print(" GCC já disponível."); return
    if platform.system() != "Linux":
        print(" Mac: xcode-select --install | Windows: WSL ou MinGW"); return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "build-essential"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "build-essential"]
    subprocess.run(cmd, check=True)
    print(" build-essential instalado!")

instalar_gcc()
```

GCC já disponível.

```
%%writefile EP01_01.c
#include <stdio.h>
#include <math.h>

int main() {
    double x1, y1, x2, y2;
    if (scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2) != 4) return 0;

    double dx = fabs(x2 - x1);
    double dy = fabs(y2 - y1);

    // Euclidiana, City-block e Chessboard
    printf("%.2f\n", sqrt(dx * dx + dy * dy));
    printf("%.2f\n", dx + dy);
    printf("%.2f\n", fmax(dx, dy));

    return 0;
}
```

Writing EP01_01.c

```
# Compila o arquivo .c gerando o executável EP01_01
# -lm é usado para linkar a biblioteca matemática (math.h) se necessário

!gcc EP01_01.c -o EP01_01 -lm
!echo -e "0\n0\n4\n4" | ./EP01_01
```

5.66
8.00
4.00

```
TestSuite("EP01_01.c").run()
```

1.16.1.7 C++

De forma similar ao Java, use `%%writefile` para salvar o código, depois compile e execute. Lembre-se de que, no Colab, também é necessário instalar o seguinte:

```
# Instalar o compilador G++ para C++
# O build-essential inclui o g++, make, etc.
import platform, shutil, subprocess

def instalar_gpp():
    if shutil.which("g++"):
        print(" G++ já disponível."); return
    if platform.system() != "Linux":
        if platform.system() == "Darwin":
            print(" Mac: xcode-select --install")
        else:
            print(" Windows: use WSL ou MinGW (https://www.mingw-w64.org)")
        return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "build-essential"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "build-essential"]
    subprocess.run(cmd, check=True)
    print(" Compilador C++ pronto.")

instalar_gpp()
```

G++ já disponível.

```
%%writefile EP01_01.cpp
#include <iostream>
#include <iomanip>
#include <cmath>
#include <algorithm>

int main() {
    double x1, y1, x2, y2;
    if (!(std::cin >> x1 >> y1 >> x2 >> y2)) return 0;

    double dx = std::abs(x2 - x1);
    double dy = std::abs(y2 - y1);

    std::cout << std::fixed << std::setprecision(2);

    // Euclidiana, City-block e Chessboard
    std::cout << std::sqrt(dx*dx + dy*dy) << std::endl;
    std::cout << (dx + dy) << std::endl;
    std::cout << std::max(dx, dy) << std::endl;

    return 0;
}
```

Writing EP01_01.cpp


```
!g++ EP01_01.cpp -o EP01_01
!echo -e "0\n0\n4\n4" | ./EP01_01
```

```
5.66
8.00
4.00
```

```
TestSuite("EP01_01.cpp").run()
```

1.16.1.8 JavaScript (Node.js)

Para JavaScript, use `%%writefile` para criar o arquivo e execute com Node:

```
%%writefile EP01_01.js
function escreva(s) { try { document.write(s + "<br>"); } catch(e) { console.log(s); } }

process.stdin.once('data', data => {
  const valores = data.toString().trim().split(/\s+/);
  const x1 = parseFloat(valores[0]);
  const y1 = parseFloat(valores[1]);
  const x2 = parseFloat(valores[2]);
  const y2 = parseFloat(valores[3]);

  const dx = Math.abs(x2 - x1);
  const dy = Math.abs(y2 - y1);

  // Euclidiana, City-block e Chessboard
  escreva(Math.sqrt(dx * dx + dy * dy).toFixed(2));
  escreva((dx + dy).toFixed(2));
  escreva(Math.max(dx, dy).toFixed(2));
});
```

```
Writing EP01_01.js
```

```
TestSuite("EP01_01.js").run()
```

1.16.1.9 R

No Colab, pode-se executar código R diretamente utilizando a mágica `%%R`.

O programa deve ler **quatro números** (x1, y1, x2, y2) e exibir a distância euclidiana com **duas casas decimais**.

Exemplo de célula:

```
%%R
dados <- scan(file = "stdin", n = 4, quiet = TRUE)
x1 <- dados[1]; y1 <- dados[2]; x2 <- dados[3]; y2 <- dados[4]
dist <- sqrt((x2 - x1)^2 + (y2 - y1)^2)
cat(sprintf("%.2f", dist))
```

```
# Instalar o R e o Rscript
# O r-base instala o ambiente R completo, incluindo o Rscript
import platform, shutil, subprocess
```

```
def instalar_r():
    if shutil.which("R"):
        print(" R já disponível."); return
    if platform.system() != "Linux":
        if platform.system() == "Darwin":
            print(" Mac: https://cran.r-project.org/bin/macosx/")
        else:
            print(" Windows: https://cran.r-project.org/bin/windows/base/")
        return
    try: import google.colab; cmd = ["apt-get", "install", "-y", "r-base"]
    except ImportError: cmd = ["sudo", "apt-get", "install", "-y", "r-base"]
    subprocess.run(cmd, check=True)
    print(" Ambiente R pronto.")

instalar_r()
```

R já disponível.

Para testar da mesma forma que nos exemplos anteriores, deve-se usar a entrada padrão (stdin) no terminal ou adaptar o código conforme abaixo:

```
%%writefile EP01_01.r
dados <- scan(file = "stdin", n = 4, quiet = TRUE)
dx <- abs(dados[3] - dados[1])
dy <- abs(dados[4] - dados[2])

# Saídas: Euclidiana, City-block e Chessboard
cat(sprintf("%.2f\n%.2f\n%.2f\n",
    sqrt(dx^2 + dy^2),
    dx + dy,
    max(dx, dy)))
```

Writing EP01_01.r

```
!echo -e "0\n0\n4\n4" | Rscript EP01_01.r
```

5.66
8.00
4.00

```
TestSuite("EP01_01.r").run()
```

1.16.2 EP01_02 Desempenho Preditivo — Métricas de ML em VC

Nesta atividade, você entrará no mundo do **Aprendizado de Máquina** (*Machine Learning*). Seu objetivo é avaliar o desempenho de um classificador binário calculando métricas a partir de uma **Matriz de Confusão**.

1.16.2.1 Por que a métrica certa importa?

Imagine um detector de **moedas de R\$ 1,00**. O impacto do erro define a métrica prioritária:

Métrica	Exemplo Prático	Importância em PDI/VC
Acurácia	Contagem de Grãos	Útil quando as classes são equilibradas (ex: metade dos grãos com defeito, metade saudáveis).

Métrica	Exemplo Prático	Importância em PDI/VC
Precisão	Segurança/Biometria	Crucial para evitar Falsos Positivos (ex: não permitir que um impostor acesse um sistema por erro de reconhecimento).
Sensibilidade	Saúde (Tumores)	Crucial para evitar Falsos Negativos (ex: não deixar um tumor passar despercebido em um exame de Raio-X).
F1-score	Cédulas de Dinheiro	Ideal para um equilíbrio entre não rejeitar notas verdadeiras e não aceitar notas falsas.

1.16.2.2 A Matriz de Confusão

	Predita Positiva	Predita Negativa
Real Positiva	VP (Verdadeiro Positivo)	FN (Falso Negativo)
Real Negativa	FP (Falso Positivo)	VN (Verdadeiro Negativo)

Tarefa:

1. Leia **4 valores inteiros** na ordem: VP, FN, FP, VN.
2. Calcule as métricas utilizando as fórmulas:

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN}$$

$$\text{Precisão} = \frac{VP}{VP + FP}$$

$$\text{Sensibilidade (Recall)} = \frac{VP}{VP + FN}$$

$$\text{F1-score} = \frac{2 \times \text{Precisão} \times \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}}$$

3. Imprima os resultados formatados com **duas casas decimais**, um por linha.

Importante:

- Use divisão em ponto flutuante para evitar resultados truncados.
- A ordem de saída deve ser: **Acurácia**, **Precisão**, **Sensibilidade** e **F1-score**.
- Veja a simulação interativa deste EP na Figure 1.13.

1.16.2.3 Exemplo de Execução

Entrada	Saída	Observação
40	0.75	Acurácia
10	0.73	Precisão
15	0.80	Sensibilidade

Entrada	Saída	Observação
35	0.76	F1-score

(Nota: $VP=40$, $FN=10$, $FP=15$, $VN=35$. Total de casos = 100)

Simulador: desempenho de classificadores

Ajuste os valores da matriz de confusão e veja as métricas calculadas em tempo real.

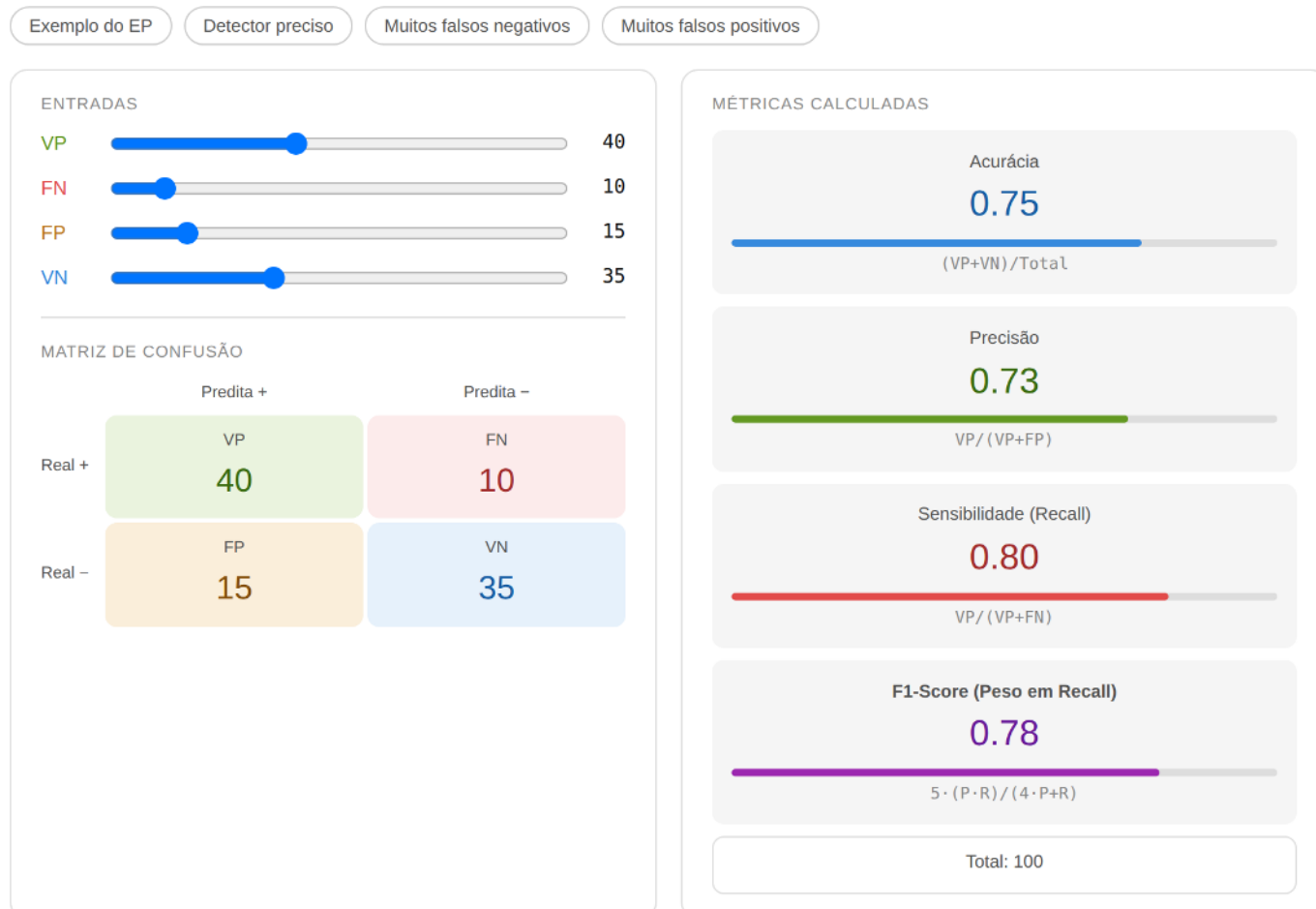


Figura 1.13: Simulador: Desempenho Preditivo — Métricas de ML

```
%%writefile EP01_02.py
# sua solução
```

Writing EP01_02.py

```
TestSuite("EP01_02.py").run()
```

1.16.3 EP01_03 ↗ *Mean Average Precision* (mAP) — Curva Precisão-Sensibilidade

Nesta atividade, você avaliará um classificador binário (ex.: detecção de desmatamento em imagens de satélite, ver dgi.inpe.br) através da **curva Precisão-Sensibilidade** e da métrica **mAP** (*Mean Average Precision*). O mAP é padrão em competições como **COCO** (*Common Objects in Context*) e **PASCAL VOC** (*Visual Object Classes*) e dos modelos **YOLO** (*You Only Look Once*).

1.16.3.1 🧠 Por que o mAP é a métrica-padrão?

Na EP01_02 você viu que a escolha do limiar altera significativamente a Precisão e a Sensibilidade. O **mAP** (*Mean Average Precision*) resolve isso: avalia o modelo em **vários limiares** (cada limiar deve gerar uma matriz de confusão diferente) e resume o desempenho pela **área sob a curva Precisão-Sensibilidade (P-S)**.

Enquanto o *F1-Score* olha para um único ponto de equilíbrio, o mAP considera a curva inteira. Quanto mais próximo de **1,0**, melhor o detector em todos os limiares e classes (ex.: moedas de 25, 50 e 1 real).

Métrica	O que resume	Limitação
F1-Score	Equilíbrio $P \times S$ num único limiar	Depende do limiar escolhido
AP	Área sob a curva P-S de uma classe	Válida apenas para uma classe
mAP	Média das APs sobre todas as classes	Mais complexo de implementar

Referências: [Roboflow — mAP](#) · [Vídeo explicativo](#)

1.16.3.2 Como o mAP é calculado — passo a passo

1. **Limiares fixos** (use sempre esta lista):

```
limiares = [0.00, 0.09, 0.21, 0.31, 0.39, 0.52, 0.60, 0.71, 0.81, 0.89, 1.00]
```

2. Para cada limiar (t), classifique as amostras: **predito = 1 se confiança $\geq t$, senão 0**. Calcule VP, FP, FN, VN e obtenha $Precis\tilde{a}o(t)$ e $Sensibilidade(t)$.
3. Monte a curva P-S: pares ($Sensibilidade(t)$, $Precis\tilde{a}o(t)$), ordenados por Sensibilidade crescente.
4. **Monotonize a Precisão:**

$$P_{mono}[i] = \max_{j \geq i} P[j]$$

5. Calcule a **AP** (área sob a curva monotônica) usando a **regra do trapézio** (aproximação mais precisa que a simples soma de Riemann):

$$AP = \sum_{i=1}^{m-1} \frac{P_{mono}[i-1] + P_{mono}[i]}{2} \cdot (S[i] - S[i-1])$$

6. **mAP** = média das APs de todas as classes. Neste EP há apenas **1 classe**, portanto $mAP = AP$.

Note

Diferença resumida:

A *soma de Riemann* aproxima a área por retângulos, podendo subestimar ou superestimar. A **regra do trapézio** usa trapézios, reduzindo o erro ao considerar a média entre os valores nos extremos do intervalo, sendo geralmente mais precisa para funções suaves por partes, como a curva Precisão-Sensibilidade.

1.16.3.3 📋 Tarefa

Leia um inteiro n (quantidade de amostras). Em seguida leia n linhas, cada uma com: **verdade** (0 ou 1) e **confiança** (float 0.0–1.0).

Calcule e imprima, para o **limiar 0.85** (índice 9 da lista):

- Matriz de Confusão (VP, FN, FP, VN)
- Acurácia, Precisão, Sensibilidade e F1-Score

Em seguida, para **todos os limiares**, imprima:

- Precisões cruas, Precisões monotônicas e Sensibilidades, separadas por ,
- mAP final

1.16.3.4 Importante

- Limiar fixo para as métricas individuais: **0.85**
- Divisão segura: se denominador for zero, use 0
- Formatação: duas casas decimais
- Monotonize de trás para frente
- A Figure 1.14 apresenta uma simulação desta questão

1.16.3.5 Exemplo de Execução

Entrada	Saída Esperada
7	# MÉTRICAS PARA O LIMAR 0.85 #
0 0.94	Matriz de Confusão:
1 0.80	VP = 0, FN = 5
1 0.69	FP = 1, VN = 1
0 0.67	
1 0.30	Métricas de Avaliação:
1 0.15	Acurácia: 0.14
1 0.15	Precisão: 0.00
	Sensibilidade: 0.00
	F1-Score: 0.00
	# MÉTRICAS PARA TODOS OS LIMIARES #
	Precisões: 0.00, 0.00, 0.00, 0.50, 0.50, 0.50, 0.50, 0.50, 0.60, 0.71, 0.71
	Precisões mon.: 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71, 0.71
	Sensibilidades: 0.00, 0.00, 0.00, 0.20, 0.40, 0.40, 0.40, 0.40, 0.60, 1.00, 1.00
	mAP: 0.71

1.16.3.6 Dica para calcular o AP (com regra do trapézio)

```
def calcular_AP(verdades, confiancas, limiares):
    m = len(limiares)
    precisoes = [0.0] * m
    sensibilidades = [0.0] * m
    for i in range(m):
        p, s = calcular_metricas(verdades, confiancas, limiares[i])
        precisoes[m-1-i] = p
        sensibilidades[m-1-i] = s
    prec_mono = precisoes.copy()
    for i in range(m-2, -1, -1):
        if prec_mono[i] < prec_mono[i+1]:
            prec_mono[i] = prec_mono[i+1]
    AP = 0.0
    for i in range(1, m):
        # Regra do trapézio: média das alturas vezes a base
        area_trapezio = (prec_mono[i-1] + prec_mono[i]) / 2.0
        AP += area_trapezio * (sensibilidades[i] - sensibilidades[i-1])
```

```
return precisoes, prec_mono, sensibilidades, AP
```

Simulador: Curva P-S e mAP

Edite as amostras (verdade e confiança) e veja a curva Precisão-Sensibilidade e o mAP calculados em tempo real.

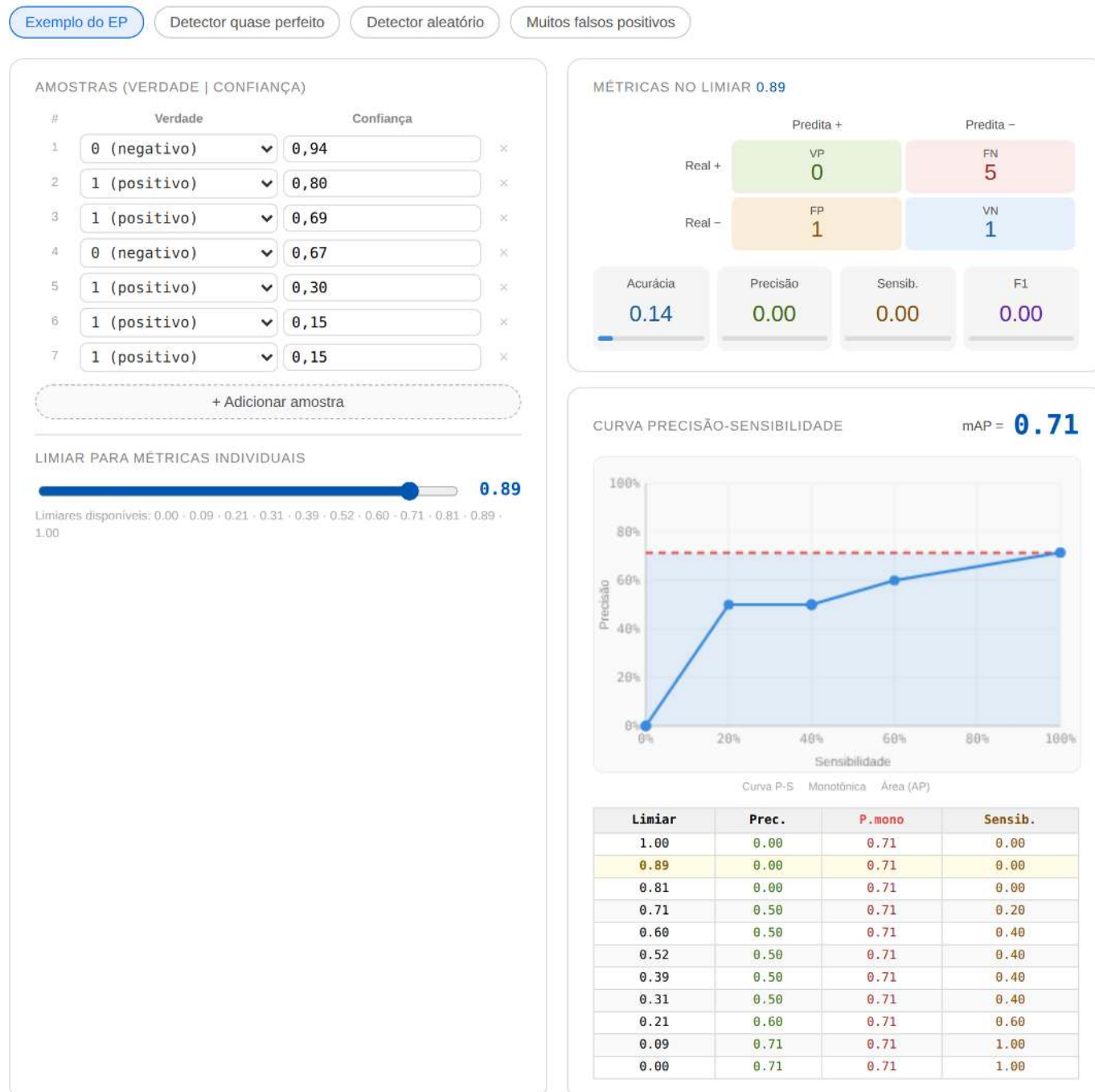


Figura 1.14: Simulador: *Mean Average Precision* (mAP) - Curva P.S

```
%%writefile EP01_03.py
# sua solução
```

```
Writing EP01_03.py
```

```
TestSuite("EP01_03.py").run()
```

1.16.4 EP01_04 Leitura e Informações de uma Imagem em Matriz

Nesta atividade, você deve escrever um programa que processe uma imagem digital representada como uma matriz de pixels em tons de cinza.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia os **L * C** valores inteiros que compõem a matriz da imagem (cada valor entre 0 e 255).
- Calcule e imprima as seguintes informações:
 1. O número de linhas.
 2. O número de colunas.
 3. O valor do maior pixel (**Máximo**).
 4. O valor do menor pixel (**Mínimo**).
 5. A **Média** aritmética de todos os pixels.

Importante:

- A saída deve seguir exatamente o formato rotulado (ex: **Linhas: X**).
- O valor da média deve ser formatado com **duas casas decimais**.
- Ver um simulador interativo para esta questão na **Figure 1.15** (grade interativa para visualização de intensidades e cálculos em tempo real).

1.16.4.1 Por que isso importa? – A Imagem como Dados

Toda imagem digital é, no fundo, uma estrutura de dados. Em tons de cinza de 8 bits, cada pixel é um valor escalar. Extrair estatísticas básicas é o primeiro passo para:

Operação	Utilidade Prática
Máximo/Mínimo	Identificar se a imagem está “lavada” (baixo contraste) ou saturada.
Média	Calcular o brilho global da cena para ajustes de exposição.
Normalização	Redimensionar os valores para intervalos como [0, 1] em redes neurais.

1.16.4.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro L (linhas).

A segunda linha contém o inteiro C (colunas).

As linhas seguintes contêm os elementos da matriz.

Saída:

Cinco linhas formatadas conforme o exemplo:

Linhas: L

Colunas: C

Max: V

Min: V

Media: V.VV

1.16.4.3 📌 Exemplos

Entrada	Saída	Observação
2	Linhas: 2	Imagem pequena de alto contraste
3	Colunas: 3	
0 128 255	Max: 255	
50 100 200	Min: 0	
	Media: 122.17	

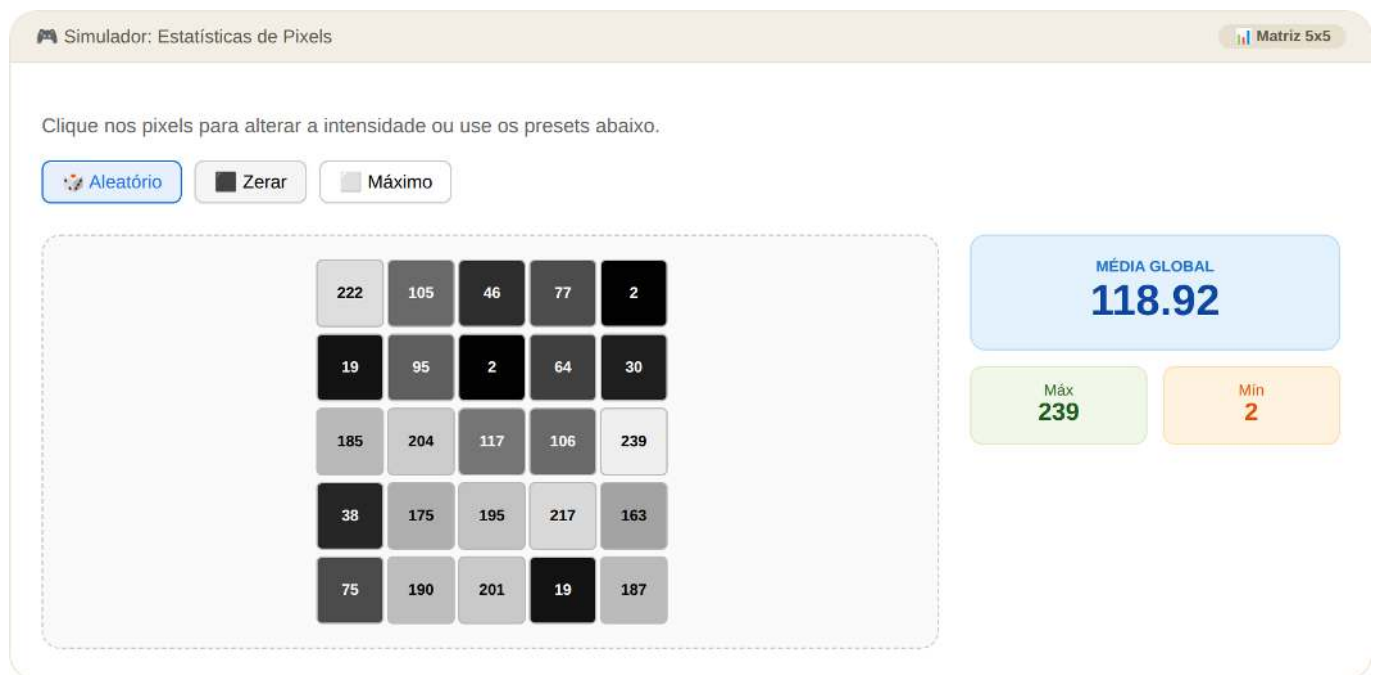


Figura 1.15: Simulador: Estatísticas de Pixels

```
%%writefile EP01_04.py
# sua solução
```

Writing EP01_04.py

```
TestSuite("EP01_04.py").run()
```

1.16.5 EP01_05 ↺ Negativo de uma Imagem em Tons de Cinza

Nesta atividade, deve-se escrever um programa que calcule o negativo de uma imagem digital.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia os valores inteiros que compõem a matriz da imagem.
- Para cada pixel, aplique a transformação de inversão:

$$pixel_{negativo} = 255 - pixel_{original}$$

- Imprima a matriz resultante, mantendo o formato original (L linhas e C colunas).

 Importante:

- Os valores de cada linha na saída devem ser separados por um **espaço em branco**.
- A saída deve conter **apenas os números** da matriz resultante.
- Ver um simulador interativo para esta questão na **Figure 1.16** (comparação em tempo real entre a matriz original e seu negativo).

1.16.5.1 Por que isso importa? – Inversão de Intensidade

O **negativo** é uma transformação linear básica que inverte a escala de brilho. É uma ferramenta essencial para o olho humano identificar detalhes claros que estão “escondidos” em fundos mais escuros, sendo amplamente utilizada em:

Aplicação	Utilidade
Imagens Médicas	Melhora a visualização de anomalias em tecidos densos (ex: Raios-X).
Astronomia	Destacar galáxias e nebulosas tênues contra o vazio do espaço.
Artes Digitais	Efeitos estéticos e preparação de máscaras de seleção.

1.16.5.2 Tarefa (especificação para VPL)**Entrada:**

A primeira linha contém o inteiro **L**.

A segunda linha contém o inteiro **C**.

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz invertida com **L** linhas e **C** colunas.

1.16.5.3 Exemplos

Entrada	Saída	Observação
2	255 127 0	Onde era 0 (preto) vira 255 (branco)
3	205 155 55	
0 128 255		
50 100 255		

```
%%writefile EP01_05.py
# sua solução
```

```
Writing EP01_05.py
```

```
TestSuite("EP01_05.py").run()
```

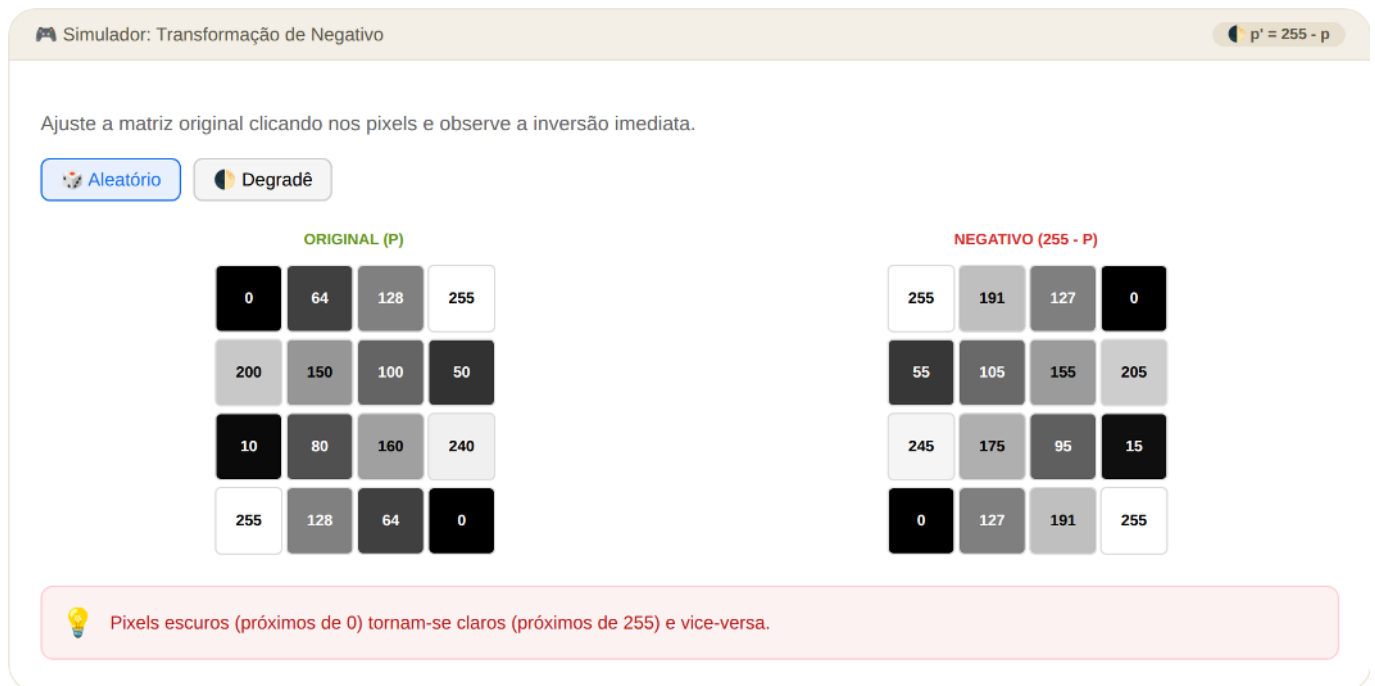


Figura 1.16: Simulador: Transformação de Negativo

1.16.6 EP01_06 🧠 Conversão RGB → Tons de Cinza (ITU-R BT.601)

Nesta atividade, você deve escrever um programa que converta pixels coloridos (RGB) para tons de cinza utilizando a ponderação fisiológica do padrão ITU-R BT.601.

- Leia dois inteiros **L** e **C**, representando o número de linhas e colunas.
- Leia $L \times C$ triplas de inteiros, onde cada tripla representa os canais **R** (Red), **G** (Green) e **B** (Blue) de um pixel.
- Para cada pixel, calcule o valor de cinza (g) usando a fórmula:

$$g = \text{round}(0.299 \times R + 0.587 \times G + 0.114 \times B)$$

- Imprima a matriz resultante (L linhas e C colunas) contendo os valores inteiros convertidos.

📌 Importante:

- Utilize a função `round()` da sua linguagem para garantir o arredondamento correto para o inteiro mais próximo.
- A saída deve conter apenas os valores de cinza, mantendo a estrutura de matriz (separados por espaço na linha).
- Ver um simulador interativo para esta questão na **Figure 1.17** (ajuste os sliders para ver como cada cor contribui para o brilho final).

1.16.6.1 🧠 Por que não usar apenas a média?

O olho humano não percebe todas as cores com a mesma intensidade. Somos muito mais sensíveis ao **Verde** do que ao **Azul** devido à nossa evolução biológica. O padrão ITU-R BT.601 utiliza pesos específicos para criar uma imagem em cinza que pareça naturalmente correta para nossa visão:

Canal	Peso	Percepção Humana
● Verde	58.7%	Máxima sensibilidade (distinção de folhagens).
● Vermelho	29.9%	Sensibilidade média.
● Azul	11.4%	Baixa sensibilidade (tons mais escuros).

1.16.6.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro L.

A segunda linha contém o inteiro C.

As linhas seguintes contêm triplas de inteiros R G B para cada pixel.

Saída:

A matriz de cinzas com L linhas e C colunas.

1.16.6.3 Exemplos

Entrada	Saída	Observação
1 3 255 0 0 0 255 0 0 0 255	76 150 29	Note como o Verde (150) é mais brilhante que o Azul (29)

 Simulador: Percepção de Cor
 ITU-R BT.601

Observe o peso de cada canal colorido no brilho final da imagem.

AJUSTE DOS CANAIS

R

80

G

180

B

30

$0.299 \times 80 = 23.9$
 $0.587 \times 180 = 105.7$
 $0.114 \times 30 = 3.4$
 Soma: 133.0 → 133

Original

Cinza

133

Dica: O Verde tem o maior peso (0.587) porque nossos olhos distinguem melhor tons de folhagens!

Figura 1.17: Simulador: Percepção de Cor e Pesos ITU-R

```
%%writefile EP01_06.py
# sua solução
```

```
Writing EP01_06.py
```

```
TestSuite("EP01_06.py").run()
```

1.16.7 EP01_07 ● Limiarização Manual: Imagem Binária

Nesta atividade, você deve escrever um programa que realize a segmentação de uma imagem através da limiarização (*thresholding*).

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia um inteiro **T**, que será o valor do limiar (corte).
- Leia os valores inteiros da matriz.
- Para cada pixel p , aplique a seguinte regra de binarização:

$$\text{resultado} = \begin{cases} 255 & \text{se } p > T \\ 0 & \text{se } p \leq T \end{cases}$$

- Imprima a matriz resultante contendo apenas os valores 0 ou 255.

📌 Importante:

- Preste atenção ao operador: o pixel só vira branco (255) se for **estritamente maior** que T .
- A saída deve manter a estrutura de matriz (L linhas e C colunas).
- Ver um simulador interativo para esta questão na **Figure 1.18** (ajuste o slider de T para observar como os objetos são isolados do fundo).

1.16.7.1 🧠 O que é Segmentação?

A limiarização é o método mais simples de separar objetos de interesse do fundo da imagem. Ao transformar tons de cinza em preto e branco puro, criamos um mapa binário que facilita a contagem de objetos ou a identificação de formas:

Valor do Pixel (p)	Condição	Resultado Final
Escuro ($p \leq T$)	Fundo/Ruído	0 (Preto)
Claro ($p > T$)	Objeto/Destaque	255 (Branco)

1.16.7.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém o inteiro L.

A segunda linha contém o inteiro C.

A terceira linha contém o inteiro T (limiar).

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz binarizada (0 ou 255) com L linhas e C colunas.

1.16.7.3 📌 Exemplos

Entrada	Saída	Observação
2	0 0 0 255	Note que o valor 128 virou 0 (pois $128 \leq 128$)
4	0 255 255 0	
128		
0 100 128 200		
50 129 255 64		

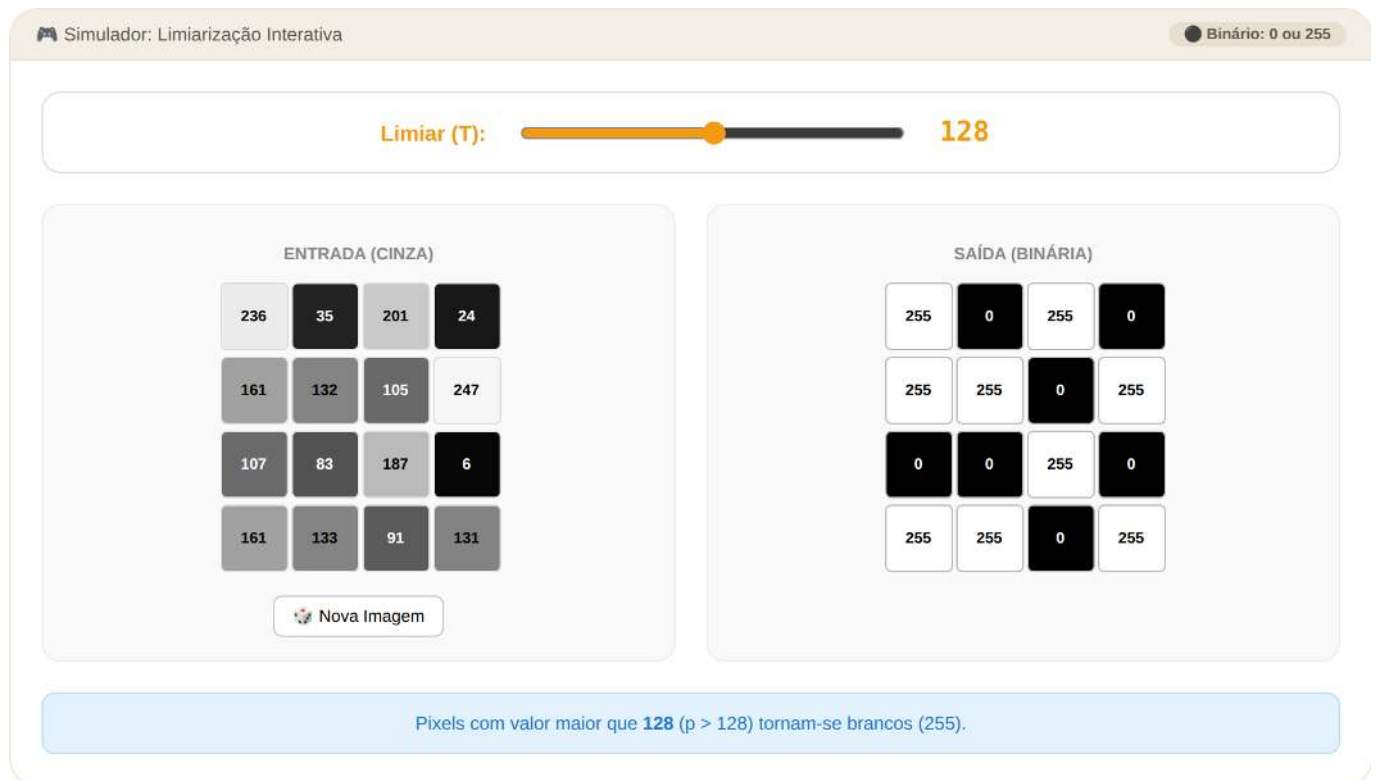


Figura 1.18: Simulador: Limiarização Interativa

```
%%writefile EP01_07.py
# sua solução
```

Writing EP01_07.py

```
TestSuite("EP01_07.py").run()
```

1.16.8 EP01_08 🧠 Remapeamento por Faixa de Intensidade

Nesta atividade, você deve escrever um programa que aplique transformações lineares distintas a diferentes regiões de intensidade da imagem.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia os inteiros **T** (limiar), $\Delta 1$ e $\Delta 2$.
- Leia os valores inteiros da matriz.
- Para cada pixel p , aplique a regra de remapeamento condicional:

$$\text{resultado} = \begin{cases} p + \delta_1 & \text{se } p < T \\ p + \delta_2 & \text{se } p \geq T \end{cases}$$

- Imprima a matriz resultante com os novos valores de intensidade.

Importante:

- δ_1 é o offset aplicado aos pixels escuros (abaixo do limiar).
- δ_2 é o offset aplicado aos pixels claros (maior ou igual ao limiar).
- Os casos de teste garantem que o resultado estará sempre no intervalo válido de **0 a 255**, portanto, não é necessário tratar saturação ou arredondamentos.
- A saída deve manter a estrutura de matriz (**L** linhas e **C** colunas).

1.16.8.1 Transformação Condicional de Pixels

Em Processamento Digital de Imagens (PDI), frequentemente precisamos tratar regiões de forma independente. Esta técnica permite, por exemplo, clarear apenas as sombras de uma fotografia (aumentando os pixels escuros) sem estourar o brilho das áreas já claras, ou vice-versa.

Faixa de Intensidade	Condição	Operação
Pixels Escuros	$p < T$	$p + \delta_1$
Pixels Claros	$p \geq T$	$p + \delta_2$

1.16.8.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém os inteiros L e C.

A segunda linha contém os inteiros T, δ_1 e δ_2 .

As linhas seguintes contêm os elementos da matriz.

Saída:

A matriz transformada com L linhas e C colunas, com valores separados por espaço.

1.16.8.3 Exemplos

Entrada	Saída	Observação
2 4 128 60 -40 0 100 150 255 80 128 200 30	60 160 110 215 140 88 160 90	Pixels < 128 somam 60. Pixels 128 subtraem 40.

```
%%writefile EP01_08.py
# sua solução
```

Writing EP01_08.py

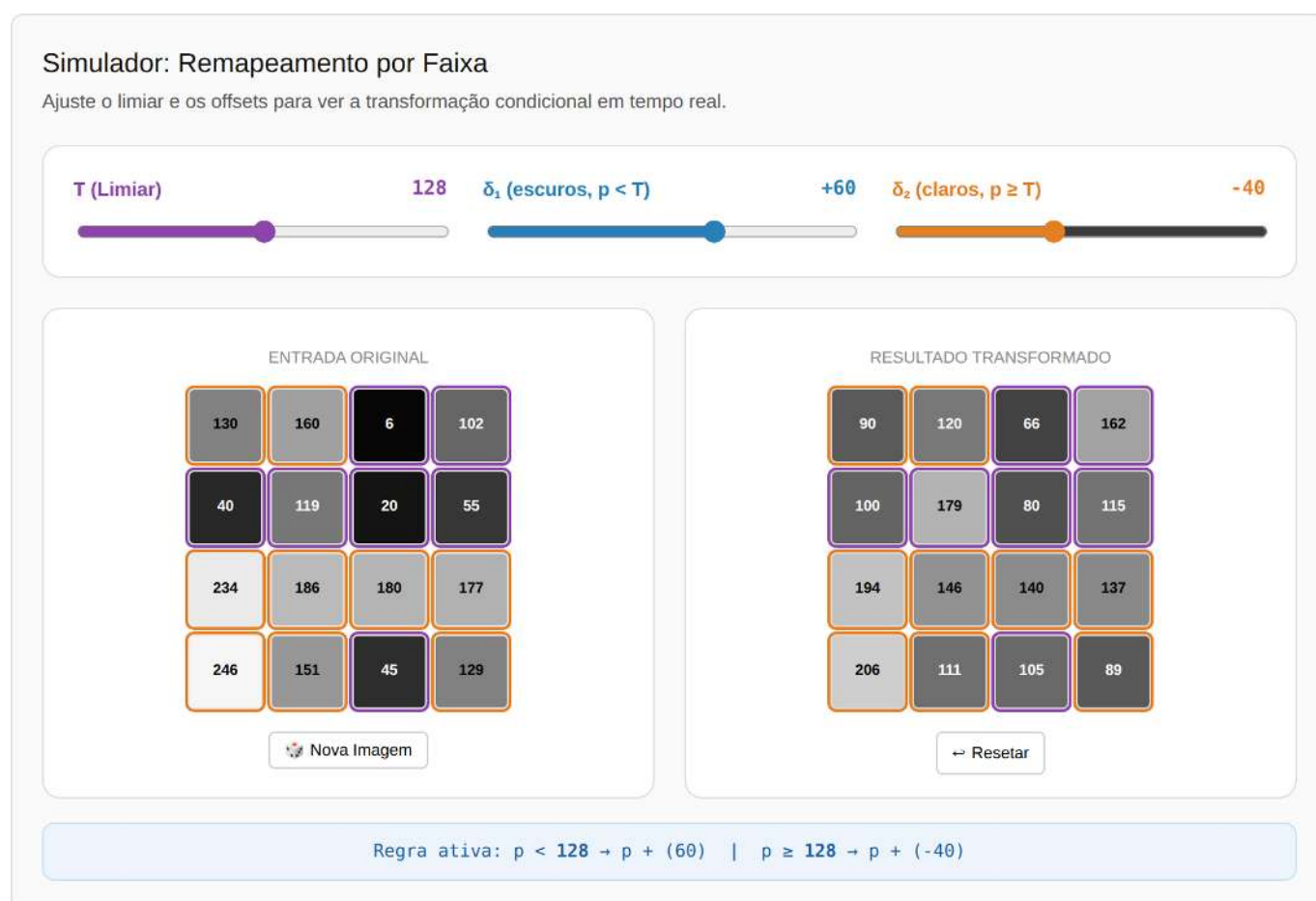


Figura 1.19: Simulador: Ajuste de Brilho e Contraste

```
TestSuite("EP01_08.py").run()
```

1.16.9 EP01_09 Padrão de Tabuleiro: Matriz de Xadrez

Nesta atividade, você deve escrever um programa que gere uma imagem sintética no padrão de um tabuleiro de xadrez.

- Leia dois inteiros **L** (linhas) e **C** (colunas).
- Gere uma matriz onde os valores alternam entre **0** (preto) e **1** (branco).
- A lógica de preenchimento deve seguir a regra da paridade:
- O elemento na posição $(0, 0)$ é sempre **0**.
- Um pixel na posição (i, j) será **1** se a soma dos índices $(i + j)$ for **ímpar**.
- Um pixel na posição (i, j) será **0** se a soma dos índices $(i + j)$ for **par**.

Importante:

- As cores devem alternar corretamente tanto horizontalmente quanto verticalmente.
- A saída deve ser a matriz impressa linha por linha, com os elementos separados por um espaço.
- Ver um simulador interativo para esta questão na **Figure 1.20** (ajuste as dimensões para visualizar a construção da malha e a saída textual correspondente).

1.16.9.1 Padrões Sintéticos

Criar padrões geométricos é um exercício fundamental para dominar a **lógica de índices** em matrizes. Em Processamento Digital de Imagens, o padrão de xadrez não é apenas estético; ele é amplamente utilizado para:

Aplicação	Utilidade
Calibração de Câmera	Estimar parâmetros intrínsecos e extrínsecos da lente.
Correção de Distorção	Identificar e corrigir o efeito de “barril” ou “almofada” em lentes grande-angulares.
Mapeamento 3D	Projetar padrões conhecidos para reconstruir superfícies em sistemas de luz estruturada.

1.16.9.2 Tarefa (especificação para VPL)

Entrada:

Uma linha contendo o inteiro **L** (linhas).

Uma linha contendo o inteiro **C** (colunas).

Saída:

A matriz de xadrez com **L** linhas e **C** colunas, impressa com espaços entre os elementos.

1.16.9.3 Exemplos

Entrada	Saída	Observação
3	0 1 0 1	Note que cada linha começa com o inverso da anterior
4	1 0 1 0	
	0 1 0 1	

Simulador: Gerador de Xadrez
Lógica de Paridade

Altere as dimensões para ver como a lógica de índices constrói o tabuleiro.

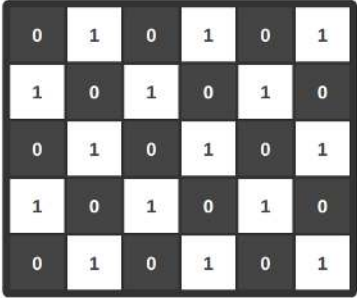
Linhas (L)

5

×

Colunas (C)

6



SAÍDA ESPERADA:

```

0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1

```

Figura 1.20: Simulador: Gerador de Xadrez

```
%%writefile EP01_09.py
# sua solução
```

Writing EP01_09.py

```
TestSuite("EP01_09.py").run()
```

1.16.10 EP01_10 Metadados: Leitura de Arquivo PGM

Nesta atividade, você deve ler um arquivo de imagem no formato **PGM (Portable Gray Map)** e extrair suas dimensões a partir do cabeçalho.

- O formato **PGM (P2)** é um arquivo de texto simples (ASCII) que armazena imagens em tons de cinza.
- O arquivo possui um **cabeçalho** estruturado da seguinte forma:
 - Versão:** O identificador P2.
 - Comentários:** Linhas opcionais que começam com # (devem ser ignoradas).
 - Dimensões:** Dois inteiros representando **Largura** e **Altura**.

4. **Máximo:** Um inteiro representando a intensidade máxima (geralmente 255).

- Após o cabeçalho, seguem-se os dados dos pixels.

Importante:

- **Leitura de Arquivo:** Você deve abrir o arquivo indicado no exemplo utilizando a função `open()` do Python.
- **Ordem de Saída:** Ao contrário da ordem presente no arquivo, a saída esperada deve estar no formato de tupla: (Altura, Largura, Canais).
- Como arquivos PGM são em tons de cinza, o número de **Canais** é sempre **1**.
- Ver um simulador interativo para esta questão na **Figure 1.21** (ajuste as dimensões para ver como o cabeçalho ASCII é gerado).

1.16.10.1 Entendendo o formato PGM

O formato PGM é um dos mais simples para processamento de imagens. Por ser em texto puro, ele permite visualizar os metadados e até os valores dos pixels abrindo o arquivo em um bloco de notas:

Componente	Exemplo	Significado
Magic Number	P2	Identifica que é um PGM em formato de texto (ASCII).
Comentário	# CREATOR...	Linha informativa ignorada pelo processador.
Dimensões	397 343	397 colunas (Largura) e 343 linhas (Altura).
Intensidade	255	Define o valor do branco puro (escala de 0 a 255).

1.16.10.2 Tarefa (especificação para VPL)

Entrada:

Nenhuma entrada via teclado. O programa deve ler o arquivo "aula01fig03b.pgm" presente no diretório de execução.

Saída:

Uma tupla contendo (Altura, Largura, 1).

1.16.10.3 Exemplos

Nome do Arquivo	Saída Esperada	Observação
"aula01fig03b.pgm"	(343, 397, 1)	Note a inversão da ordem: Altura primeiro

Nota

O arquivo necessário para este EP será baixado automaticamente do repositório por meio do seguinte código:



Figura 1.21: Simulador: Estrutura do Arquivo PGM

```
import os, urllib.request

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/all/cap01/imagens"
file = "aula01fig03b.pgm"

opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
urllib.request.install_opener(opener)

for f in [file]:
    if not os.path.exists(f):
        url = f"{BASE_URL}/{f}"
        try:
            urllib.request.urlretrieve(url, f)
            print(f" Arquivo baixado: {f}")
        except urllib.error.HTTPError as e:
            print(f" Erro {e.code}: Não encontrado em {url}")
```

```
%%writefile EP01_10.py
# sua solução
```

Writing EP01_10.py

```
TestSuite("EP01_10.py").run()
```

1.16.11 EP01_11 ↗ Análise de Vizinhança: Filtro de Máximo 1D

Nesta atividade, você deve implementar um filtro morfológico simples de máximo operando em um sinal unidimensional (vetor).

- Leia um inteiro **n**, representando o tamanho do vetor.
- Leia os **n** elementos inteiros que compõem o vetor original **v1**.
- Crie um novo vetor **v2**, onde cada posição *i* é o resultado da comparação entre o elemento atual e seus vizinhos imediatos:

$$v2[i] = \max(v1[i - 1], v1[i], v1[i + 1])$$

Importante:

- **Bordas:** Nas extremidades do vetor (índices 0 e $n - 1$), a vizinhança possui apenas dois elementos (o próprio e o único vizinho disponível). No índice 0, compare apenas $v1[0]$ e $v1[1]$. No último índice, compare apenas $v1[n - 2]$ e $v1[n - 1]$.
- **Saída:** Imprima o cabeçalho “v2:” seguido pelos valores do vetor resultante, um por linha.
- Ver um simulador interativo para esta questão na **Figure 1.22** (passe o mouse sobre os resultados para visualizar a janela de vizinhança utilizada no cálculo).

1.16.11.1 Por que analisar vizinhos?

Em processamento de imagens, o valor de um pixel raramente é isolado; ele depende do contexto ao seu redor. O **Filtro de Máximo** é a base da operação de **Dilatação** em morfologia matemática, servindo para:

Função	Efeito Visual
Realce	Expande estruturas brilhantes e “engorda” objetos claros.
Remoção de Ruído	Elimina pequenos pontos pretos (ruído de “sal e pimenta” escuro).
Preenchimento	Fecha pequenos buracos ou lacunas em formas binárias.

1.16.11.2 Tarefa (especificação para VPL)

Entrada:

Um inteiro n .

Nas linhas seguintes, os n elementos inteiros do vetor.

Saída:

A *string* `v2`: na primeira linha.

Nas linhas seguintes, cada elemento de `v2` (um por linha).

1.16.11.3 Exemplos

Entrada	Saída	Observação
5	v2:	No índice 1: $\max(10, 20, 5) = 20$
10	20	
20	20	
5	30	
30	30	
15	30	

 Simulador: Filtro de Máximo Local

 Janela 1x3

Clique nos valores de **v1** para alterá-los ou passe o mouse em **v2** para ver a vizinhança.

VETOR V1 (ENTRADA)

2	40	21	15	16	3	46	47
---	----	----	----	----	---	----	----

↓

VETOR V2 (RESULTADO)

40	40	40	21	16	46	47	47
----	----	----	----	----	----	----	----

 Aleatório

Passe o mouse sobre v2 para analisar o cálculo.

Figura 1.22: Simulador: Filtro de Máximo Local 1D

```
%%writefile EP01_11.py  
# sua solução
```

```
Writing EP01_11.py
```

```
TestSuite("EP01_11.py").run()
```

Capítulo 2

Da Captura ao Pixel - Amostragem, Quantização e Conectividade



Este capítulo aprofunda a compreensão da imagem digital, transitando da natureza física da captura para a sua representação matemática discreta. Investigamos como a luz se torna dado e como a organização espacial dos pixels define as relações de vizinhança e conectividade essenciais para algoritmos avançados de Visão Computacional (VC).

2.1 Objetivos

Ao final deste capítulo, você será capaz de:

- Explicar o modelo físico de formação da imagem baseado em iluminação e refletância.
- Diferenciar os mecanismos da visão humana e dos sensores digitais.
- Compreender os processos de **amostragem** (discretização do espaço) e **quantização** (discretização da intensidade).
- Descrever as relações topológicas entre pixels: vizinhança, adjacência, conectividade e distâncias.
- Realizar transformações geométricas básicas (translação, rotação, escala) preservando a qualidade.
- Visualizar na prática os efeitos da variação da resolução espacial e da profundidade de bits.

2.2 O Olho e a Câmera - Elementos da Percepção Visual

A formação de uma imagem digital começa com a captura da luz refletida pelos objetos. Como ilustrado na Figure 2.1, tanto o olho humano quanto as câmeras digitais seguem princípios ópticos semelhantes para focar a luz em uma superfície sensível, embora utilizem mecanismos biológicos e eletrônicos distintos para a transdução do sinal.

2.2.1 Visão humana

O olho funciona como um sistema óptico complexo: a luz atravessa a córnea, o humor aquoso, a pupila (controlada pela íris) e o cristalino — que ajusta o foco dinamicamente — até atingir a retina. Na retina, encontram-se os fotorreceptores: os **cones** (6 milhões), concentrados na fóvea, são responsáveis pela visão de cores e detalhes, enquanto os **bastonetes** (120 milhões) garantem a visão em baixa luminosidade (visão escotópica), detectando apenas intensidades de cinza. O ponto cego é a região de onde parte o nervo óptico, carecendo de receptores.

2.2.2 Sensores digitais

Nas câmeras, os sensores de imagem desempenham o papel da retina. Os dois tipos mais comuns são o **CCD** (*Charge-Coupled Device* - Dispositivo de Carga Acoplada) e o **CMOS** (*Complementary Metal-Oxide-Semiconductor* - Semicondutor de Óxido Metálico Complementar). O sensor é composto por uma matriz de fotossítios (pixels) que acumulam carga elétrica proporcional à luz incidente.

Para a reconstrução de cores, utiliza-se o **Filtro de Bayer**, uma matriz de filtros coloridos que permite que cada pixel capture apenas uma componente de cor: vermelho, verde ou azul (RGGB - *Red, Green, Green, Blue*). Posteriormente, um **ADC** (*Analog-to-Digital Converter* - Conversor Analógico-Digital) quantiza essa carga em valores numéricos, definidos por uma profundidade de bits (ex.: 8 bits, resultando em 256 níveis de intensidade).

i Curiosidade

Embora o olho humano tenha milhões de receptores, a resolução de alta definição é restrita à fóvea (visão central), equivalente a aproximadamente 120×120 pixels. A percepção de uma cena completa em alta resolução é resultado de intenso pós-processamento realizado pelo cérebro.

O Olho e a Câmera - Elementos da Percepção Visual

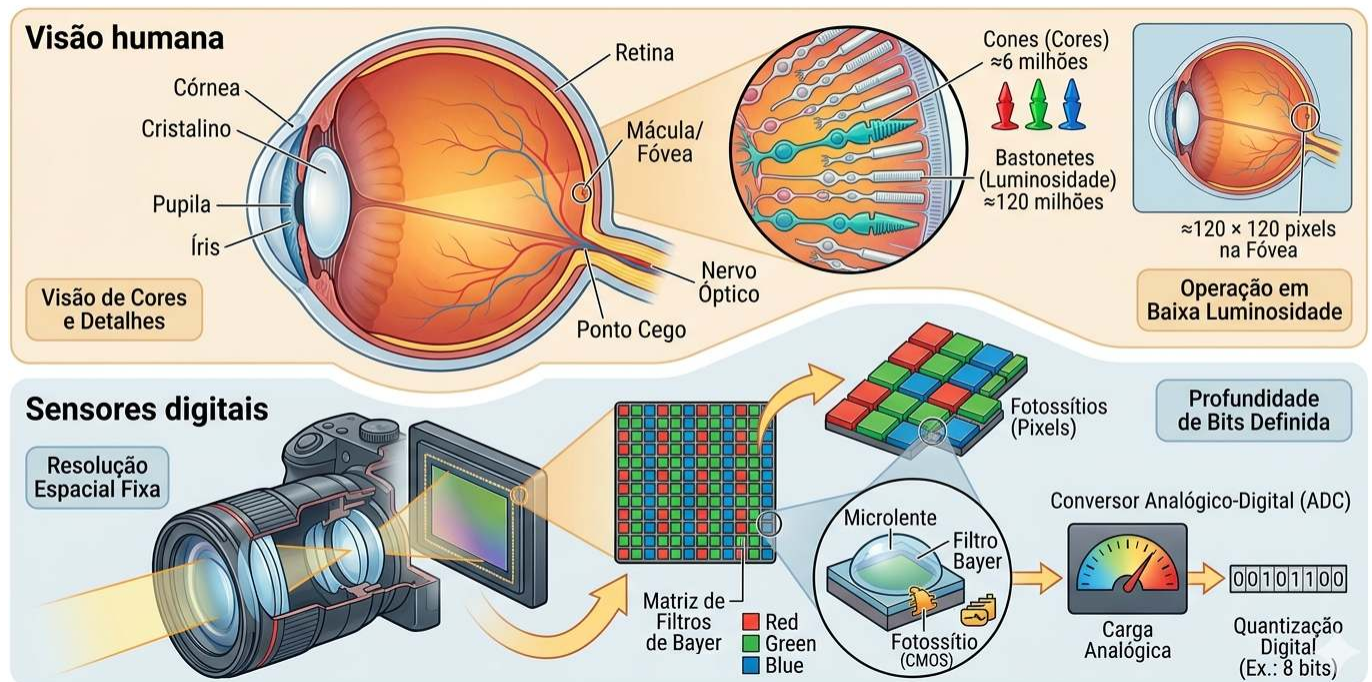


Figura 2.1: Comparativo didático entre o sistema visual biológico e o eletrônico: no topo, a anatomia do olho humano destacando a retina e os fotorreceptores (cones e bastonetes); abaixo, a estrutura de uma câmera digital detalhando o sensor CMOS, a matriz de filtros de Bayer (RGGB) e o processo de quantização digital realizado pelo ADC.

2.3 Ilusões de Ótica: Os Desafios da Percepção Visual

Enquanto os sensores digitais capturam a intensidade da luz de forma linear e objetiva, o sistema visual humano interpreta a cena com base em contexto, experiências prévias e mecanismos biológicos de sobrevivência. As ilusões de ótica não são “erros” do olho, mas evidências do intenso **pós-processamento cerebral** realizado no córtex visual.

2.3.1 Ambiguidade e Contexto

O cérebro busca constantemente dar sentido a padrões ambíguos. No exemplo do **Vaso de Rubin** (veja a Figure 2.2), a percepção alterna entre a figura (vaso) e o fundo (dois rostos), demonstrando que não conseguimos processar ambas as interpretações simultaneamente. Já a ilusão do **Elefante de Shepard** brinca com a nossa incapacidade de reconciliar linhas de contorno que sugerem volume em posições logicamente impossíveis.

2.3.2 Brilho e Contraste Local

Muitas ilusões decorrem da **inibição lateral**, mecanismo pelo qual neurônios vizinhos na retina competem entre si para realçar bordas. Na **Grade de Scintillating**, pontos escuros “fantasmas” parecem surgir nas interseções brancas devido a esse processamento local de contraste.

A ilusão da **Sombra no Tabuleiro de Adelson** é talvez a mais impactante para a VC: o quadrado “A” e o quadrado “B” possuem exatamente o mesmo valor de cinza no sensor (ou arquivo digital), mas o cérebro “corrige” o brilho de “B” por entender que ele está sob uma sombra projetada, percebendo-o como mais claro.

2.3.3 Geometria e Perspectiva

A percepção de profundidade pode ser enganada por construções geométricas que desafiam a lógica tridimensional a partir de um ângulo de visão específico. A **Escada de Schröder** utiliza a ambiguidade da perspectiva para criar um objeto que parece subir ou descer dependendo de como é observado, evidenciando como a nossa interpretação de “cima” e “baixo” depende do ponto de fuga.

Ilusões de Ótica: Os Desafios da Percepção Visual

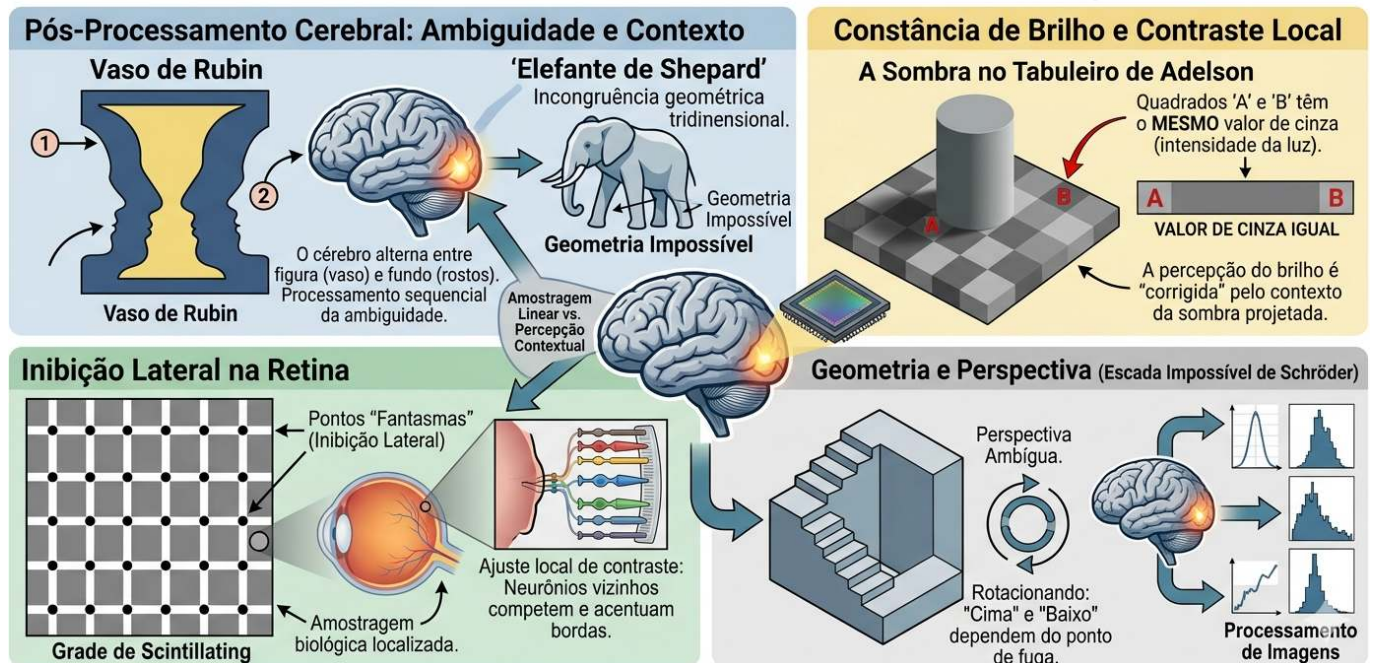


Figura 2.2: Coletânea de desafios perceptivos: (topo esquerdo) Vaso de Rubin — ambiguidade figura-fundo; (topo central) Elefante de Shepard — incongruência geométrica; (topo direita) Sombra de Adelson — constância de brilho baseada no contexto; (inferior esquerdo) Grade de pontos — inibição lateral; (inferior direito) Escada de Schröder.

2.4 O Modelo Matemático da Formação da Imagem

Uma imagem pode ser modelada como o produto de duas funções:

$$f(x, y) = i(x, y) \cdot r(x, y) \quad (2.1)$$

onde:

- $i(x, y)$ é a **iluminação** incidente sobre a cena (energia luminosa por unidade de área), determinada pela fonte de luz;

- $r(x, y)$ é a **refletância** do objeto (fração da luz refletida), determinada pelas propriedades ópticas da superfície, com $0 < r(x, y) < 1$.

Na prática, as duas componentes variam em escalas espaciais distintas: $i(x, y)$ tende a variar lentamente no espaço, enquanto $r(x, y)$ pode apresentar variações abruptas associadas a texturas, bordas e detalhes finos. Técnicas de PDI frequentemente procuram separar ou compensar essas componentes, como em métodos de correção de iluminação não uniforme.

A Figure 2.3 ilustra como esse modelo se manifesta em imagens digitais coloridas, representadas por múltiplos canais espectrais (RGB) e, opcionalmente, por um canal adicional de transparência (RGBA).

2.4.1 Representação Digital e Canais Espectrais

Em imagens digitais coloridas, a função $f(x, y)$ da Equation 2.1 é representada por múltiplos canais espectrais. No padrão **RGB**, cada pixel armazena três amostras independentes:

$$\mathbf{f}(x, y) = [R(x, y), G(x, y), B(x, y)]$$

correspondentes às intensidades das componentes vermelha (*Red*), verde (*Green*) e azul (*Blue*). Em imagens do tipo **RGBA**, adiciona-se um quarto canal:

$$\mathbf{f}(x, y) = [R(x, y), G(x, y), B(x, y), A(x, y)]$$

onde $A(x, y)$ representa o canal **alfa** (*Alpha*), responsável por codificar a opacidade do pixel. Por convenção, o valor $A = 0$ indica um pixel totalmente transparente, enquanto $A = 255$ (ou 1) representa um pixel totalmente opaco.

Assim, uma imagem digital colorida é estruturada como uma matriz multidimensional de dimensões:

- **RGB:** $M \times N \times 3$
- **RGBA:** $M \times N \times 4$

em que cada posição (x, y) armazena as amostras associadas às propriedades ópticas daquela coordenada espacial.

Convenção de escala entre bibliotecas: Diferentes ecossistemas de programação adotam faixas de valores e ordenações distintas para representar os canais digitais, conforme sintetizado na Table 2.1.

Tabela 2.1: Convenções de escala e ordem de canais nas principais bibliotecas de PDI.

Biblioteca	Tipo padrão	Faixa de valores	Ordem das bandas	Forma do array/tensor
OpenCV	uint8	[0, 255]	BGR	$H \times W \times C$
Pillow	uint8	[0, 255]	RGB	$H \times W \times C$
NumPy	uint8/float32	[0, 255] ou [0, 1]	—	$H \times W \times C$
scikit-image	float64	[0, 1]	RGB	$H \times W \times C$
PyTorch	float32	[0, 1]	RGB	$C \times H \times W$
Tensor-Flow/Keras	float32	[0, 1]	RGB	$H \times W \times C$

A conversão entre essas faixas de intensidade é direta e dada por:

$$f_{\text{norm}}(x, y) = \frac{f(x, y)}{255}$$

onde $f(x, y) \in [0, 255]$ e $f_{\text{norm}}(x, y) \in [0, 1]$.

Negligenciar essas discrepâncias é uma fonte frequente de erros em fluxos de trabalho de VC — como, por exemplo, injetar uma imagem lida via OpenCV (padrão BGR, tipo `uint8`) diretamente em um modelo profundo do PyTorch que pressupõe o formato RGB normalizado.

2.4.2 Preparando o Ambiente Prático

O bloco a seguir carrega o módulo `morph.py` do repositório e demonstra, para um pixel sintético, as diferentes convenções de escala e ordem de canais adotadas pelas principais bibliotecas de PDI.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão: {version}).")

# Demonstração das convenções de escala
img_exemplo = np.array([[200, 100, 50]], dtype=np.uint8) # 1 pixel RGB

print("\n Convenções de escala por biblioteca ")
print(f" NumPy / Pillow / OpenCV (uint8): {img_exemplo[0,0]} → faixa [0, 255]")
print(f" scikit-image / PyTorch (float): {img_exemplo[0,0] / 255.0} → faixa [0.0, 1.0]")
print(f" OpenCV: atenção - lê em BGR: {img_exemplo[0,0,:-1]} (canais invertidos)")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.5).

```
Convenções de escala por biblioteca
NumPy / Pillow / OpenCV (uint8): [200 100 50] → faixa [0, 255]
scikit-image / PyTorch (float): [0.78431373 0.39215686 0.19607843] → faixa [0.0, 1.0]
OpenCV: atenção - lê em BGR: [ 50 100 200] (canais invertidos)
```

2.4.3 Implementação da Leitura de Imagem em `morph.py`

O trecho a seguir exibe o código-fonte da função `mm.read`, permitindo verificar diretamente como a biblioteca `morph.py` implementa a leitura de imagens.

```
import inspect
print(inspect.getsource(mm.read))
```

```
@staticmethod
def read(file, pil=False, grayscale=False):
    """Lê imagem (local, URL ou Google Drive) → PIL.Image, ndarray 2D ou RGB 3D."""
    import re, requests
    from PIL import Image
    from io import BytesIO
    from urllib.request import urlopen, Request
    np = mm._get_np()

    # - fonte: URL / Google Drive -
```

```

if isinstance(file, str) and file.startswith(("http://", "https://", "id=")):
    m = re.search(r"id=(\[w-\]+\)", file) or re.search(r"/d/(\[w-\]+\)", file)
    url = f"https://drive.google.com/uc?export=view&id={m.group(1)}" \
        if m and ("id=" in file or "drive.google.com" in file) else file
    hdr = {"User-Agent": "Mozilla/5.0 AppleWebKit/537.36 Chrome/124 Safari/537.36"}
    try:
        r = requests.get(url, headers=hdr, timeout=20)
        if r.status_code == 429: raise requests.exceptions.HTTPError()
        r.raise_for_status()
        file = BytesIO(r.content)
    except:
        file = BytesIO(urlopen(Request(url, headers=hdr), timeout=20).read())

img = Image.open(file)
img.load()
if pil:
    return img
if grayscale:
    return np.array(img.convert("L")) # (H,W)
if img.mode == "L":
    return np.array(img) # (H,W)
if img.mode == "RGBA":
    # fundo branco
    bg = Image.new("RGB", img.size, (255, 255, 255))
    bg.paste(img, mask=img.split()[3])
    return np.array(bg)
return np.array(img.convert("RGB")) # (H,W,3)

```

2.4.4 RGB e RGBA: Exemplo Prático com a Imagem Mandrill

O exemplo a seguir carrega a imagem Mandrill em formato RGB, constrói artificialmente um canal alfa com gradiente horizontal e exibe ambas as representações, ilustrando concretamente as estruturas matriciais $M \times N \times 3$ e $M \times N \times 4$.

```

import os
import numpy as np

# https://commons.wikimedia.org/wiki/File:Mandrill-k-means.png
url = "https://upload.wikimedia.org/wikipedia/commons/a/ab/Mandrill-k-means.png"
caminho = "imagens/mandrill.png"

# Leitura RGB
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_pil = mm.read(url, pil=True)
    mm.write(img_pil, caminho)
else:
    img_pil = mm.read(caminho, pil=True)

img_rgb = np.array(img_pil.convert("RGB")) # shape: (M, N, 3), uint8

# Criação do canal alfa (gradiente horizontal)
h, w, _ = img_rgb.shape
alpha = np.tile(np.linspace(0, 255, w, dtype=np.uint8), (h, 1))
img_rgba = np.dstack([img_rgb, alpha]) # shape: (M, N, 4), uint8

# Diagnóstico
print(f"RGB → shape={img_rgb.shape}, dtype={img_rgb.dtype}, "
      f"faixa=[{img_rgb.min()}, {img_rgb.max()}]")
print(f"RGBA → shape={img_rgba.shape}, dtype={img_rgba.dtype}, "
      f"faixa=[{img_rgba.min()}, {img_rgba.max()}]")
print(f"Pixel (0,0): RGB={img_rgb[0,0]} | RGBA={img_rgba[0,0]}")

# Normalização float32 (padrão scikit-image / PyTorch)

```

```
img_rgb_norm = img_rgb.astype(np.float32) / 255.0
print(f"\nNormalizado (float32): faixa=[{img_rgb_norm.min():.2f}, {img_rgb_norm.max():.2f}]")
print(f"Pixel (0,0) normalizado: {img_rgb_norm[0,0]}")

# Visualização
mm.show(
    [img_rgb, img_rgba],
    title=["RGB - $M \\times N \\times 3$", "RGBA - $M \\times N \\times 4$"],
    cols=2, axis=True
)
```

```
RGB → shape=(512, 512, 3), dtype=uint8, faixa=[14, 248]
RGBA → shape=(512, 512, 4), dtype=uint8, faixa=[0, 255]
Pixel (0,0): RGB=[125 110 58] | RGBA=[125 110 58 0]
```

```
Normalizado (float32): faixa=[0.05, 0.97]
Pixel (0,0) normalizado: [0.49019608 0.43137255 0.22745098]
```

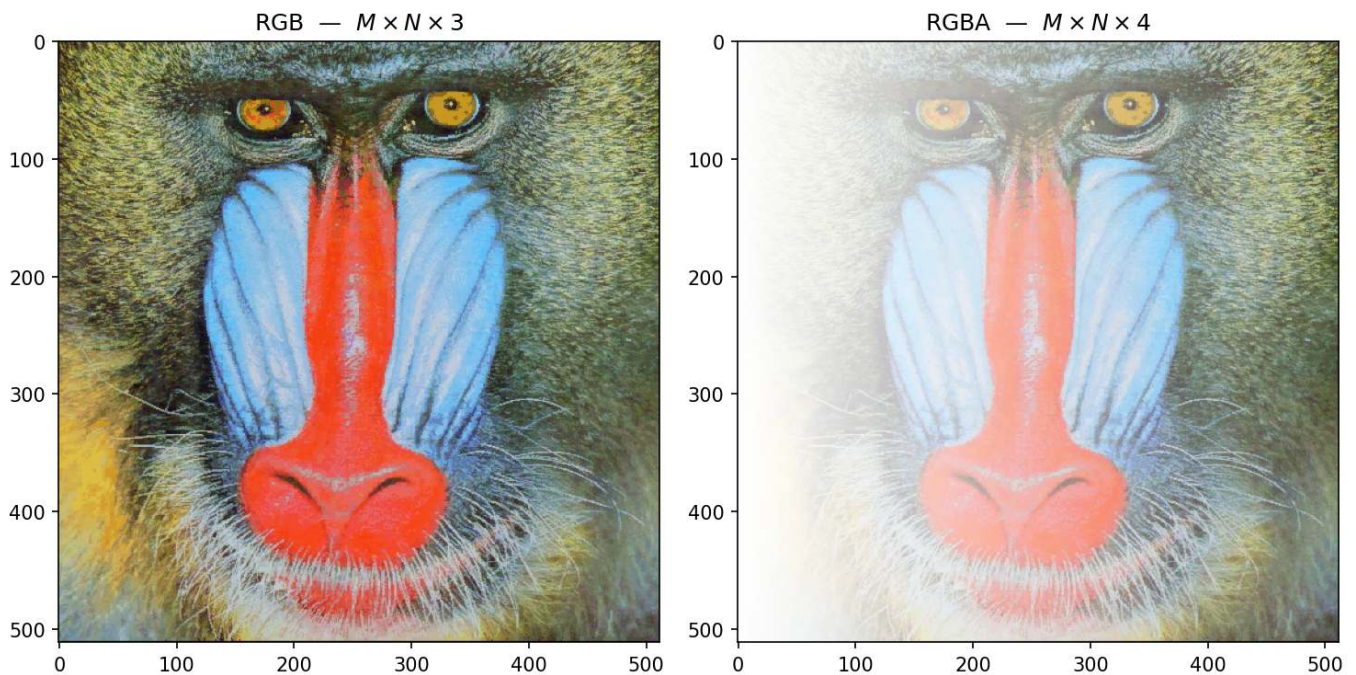


Figura 2.3: Imagem RGB (3 canais, $M \times N \times 3$) e versão RGBA (4 canais, $M \times N \times 4$) com transparência alfa gradual da esquerda para a direita. Imagem Mandrill — [USC SIPI Image Database](#) (domínio público).

2.5 Digitalização: Amostragem e Quantização

Para transformar uma cena contínua em uma imagem digital, dois processos são necessários: amostragem e quantização.

2.5.1 Amostragem - Discretização do espaço

A amostragem consiste em medir o valor da função $f(x, y)$ em pontos igualmente espaçados, formando uma matriz de M linhas (altura) e N colunas (largura). Cada elemento dessa matriz é um **pixel**. A **resolução espacial** é dada por $M \times N$. Quanto maior a resolução, mais detalhes espaciais são preservados, mas maior também o custo computacional e de armazenamento.

2.5.2 Quantização - Discretização da intensidade

A quantização associa a cada pixel um valor numérico discreto, geralmente representado por um inteiro de b bits. A **profundidade de bits** define o número de níveis de intensidade: 2^b . Imagens em tons de cinza costumam usar 8 bits (256 níveis). Imagens coloridas usam três canais de 8 bits (24 bits no total).

Ilustração: Se usarmos apenas 1 bit por pixel (preto e branco), perdemos todos os tons intermediários. Com 2 bits (4 níveis) já se percebe degradês grosseiros. Com 8 bits, o olho humano dificilmente percebe a discretização (visão contínua).

⚠ Erro de quantização

Erro de quantização é a diferença entre o valor analógico real e o valor discreto atribuído. Ele se manifesta como ruído de quantização, visível em regiões com gradiente suave quando se usa poucos bits.

2.5.3 Efeitos da amostragem e quantização

Os experimentos a seguir mostram como a redução da resolução espacial (subamostragem) e da profundidade de bits degradam a qualidade visual. Use o código para explorar diferentes fatores e níveis de cinza.

```
# Carregar imagem de exemplo
base = "https://upload.wikimedia.org/wikipedia/commons"

arquivo = (
    "Area_de_Prote%C3%A7%C3%A3o_Ambiental_"
    "Quilombos_do_M%C3%A9dio_Ribeira_-_Thomas-"
    "Fuhrmann_%282023-02%29_Malacoptila_striata.jpg"
)

url = f"{base}/c/c5/{arquivo}"
caminho = "imagens/barbudo-rajado.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_pil = mm.read(url, pil=True) # retorna PIL Image com EXIF
    mm.write(img_pil, caminho)       # salva preservando EXIF
else:
    img_pil = mm.read(caminho, pil=True)

img_numpy = np.array(img_pil)

exif = img_pil._getexif() # agora funciona - img_pil é PIL Image

img_color = mm.read(caminho)
img_gray0 = mm.gray(img_color)
print(f"Imagem original: {img_color.shape}")
print(f"Tipo da imagem: {type(img_color)}")
```

```
Imagem original: (3067, 2047, 3)
Tipo da imagem: <class 'numpy.ndarray'>
```

O experimento apresentado na Figure 2.4 ilustra o compromisso entre a **resolução espacial** e o **custo de armazenamento** em memória. O código utiliza a técnica de subamostragem por fatiamento (*slicing*) para reduzir a matriz original de pixels conforme um fator f , resultando em uma economia de memória — por exemplo, um fator $f = 8$ reduz o tamanho do dado em 64 vezes (8^2). Para fins de comparação visual, as imagens reduzidas são restauradas às dimensões originais (512×512) através da interpolação por **vizinho mais próximo** (*nearest*). Este processo não recupera a informação perdida, mas torna evidente o efeito de **aliasing** e a estrutura de blocos (pixelização) gerada pela baixa densidade de dados da matriz amostrada.

```
def subsample_simple(image, f):
    # Subamostragem via fatiamento (slicing)
    reduced = image[:, ::f, ::f]

    # Cálculo de memória em KB
    mem_kb = reduced.nbytes / 1024
    label = f"{reduced.shape[1]}x{reduced.shape[0]}, {int(mem_kb)} KB\n(Fator {f})"

    # Restaura o tamanho para visualização (H, W originais)
    res = mm.resize(reduced, (image.shape[1], image.shape[0]), method='nearest')
    return res, label

factors = [1, 4, 8, 12]
img_gray = img_gray0[820:1850, 890:1550] # Recorte para melhor visualização dos detalhes

# Gera os resultados e separa em listas para o mm.show
results = [subsample_simple(img_gray, f) for f in factors]
imgs_list = [r[0] for r in results]
titles_list = [r[1] for r in results]

mm.show(imgs_list, titles=titles_list, cols=4, figsize=(16, 12))
```

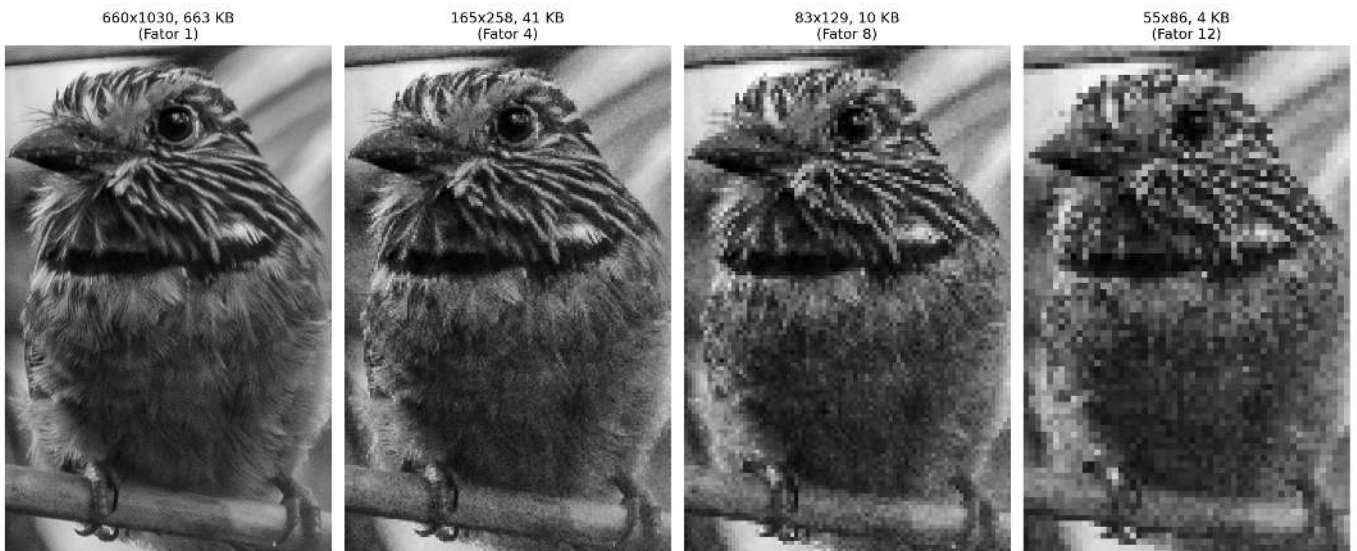


Figura 2.4: Efeito da subamostragem. Os títulos exibem as dimensões (W x H) e o tamanho da matriz em memória (KB).

O experimento na Figure 2.5 foca na **quantização de intensidade**, o processo de discretização da amplitude da função $f(x, y)$. Enquanto a subamostragem afeta a grade espacial, a redução da profundidade de bits limita a quantidade de tons de cinza disponíveis para representar o brilho.

Ao reduzir a profundidade de 8 bits (256 níveis) para valores menores, surge o efeito de **posterização**, onde os gradientes suaves de uma cena são substituídos por transições abruptas. No limite de 1 bit, a imagem torna-se estritamente binária, preservando apenas a silhueta e perdendo detalhes de textura e volume.

```
def quantize_simple(image, bits):
    """Reduz a profundidade de bits e calcula metadados de memória."""
    levels = 2 ** bits
    # Normalização e quantização uniforme
    quantized = (np.floor(image / 256 * levels) / levels * 255).astype(np.uint8)

    # Cálculo de memória em KB
```

```

mem_kb = quantized.nbytes / 1024
label = f"{bits} bits ({levels} níveis)\n{int(mem_kb)} KB"

return quantized, label

# Lista de bits para teste (8 é o padrão, 1 é o binário)
bits_test = [8, 4, 2, 1]

# Gera os resultados e separa em listas para o mm.show
results_q = [quantize_simple(img_gray, b) for b in bits_test]
imgs_q = [r[0] for r in results_q]
titles_q = [r[1] for r in results_q]

mm.show(imgs_q, titles=titles_q, cols=4, figsize=(16, 12))

```

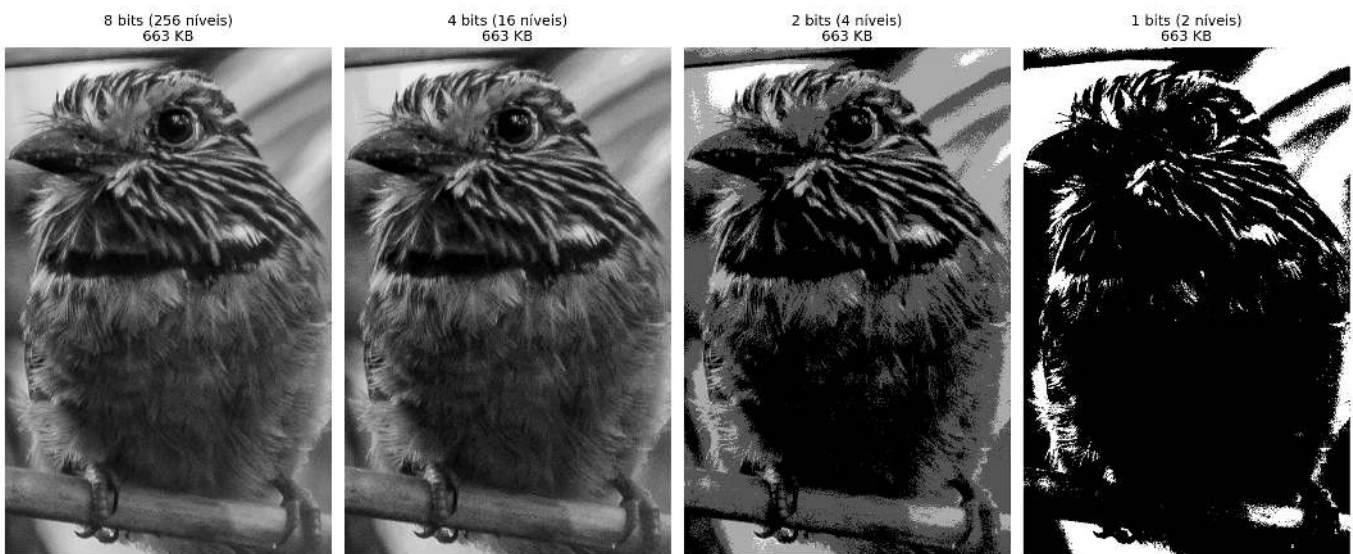


Figura 2.5: Efeito da redução da profundidade de bits. Os títulos exibem a quantidade de bits, níveis e o tamanho em memória (KB).

2.5.4 Análise Técnica

- **Domínio vs. Codomínio:** Note que a resolução espacial (dimensões da matriz) permanece constante em 512x512; o que se altera é apenas o codomínio da função da imagem.
- **Constância de Memória:** Observe nos títulos que o tamanho em KB não diminui. Isso ocorre porque o NumPy armazena cada pixel quantizado em um contêiner de 8 bits (`uint8`), independentemente de o valor real ser apenas 0 ou 1.
- **Percepção:** A degradação visual torna-se crítica abaixo de 4 bits, onde o olho humano começa a perceber as “fronteiras” artificiais criadas pela falta de tons intermediários.

A limitação de tipos de dados menores que um byte no ecossistema Python/NumPy deve-se à arquitetura de hardware, que endereça a memória em blocos de 8 bits (Bytes). Para que se mantenha a compatibilidade com o OpenCV e se garanta a eficiência, mapeiam-se até mesmo elementos binários para contêineres de 1 byte (`uint8` ou `bool8`).

Embora linguagens como ANSI C permitam a compactação de 8 pixels por byte (*bit-packing*), tal abordagem exige a descompactação constante para cálculos e impõe alta complexidade na manipulação de ponteiros. Conforme a Table 2.2, privilegia-se o uso de `uint8` pela facilidade no acesso a vizinhos e pela versatilidade em transformações geométricas. Além disso, métodos nativos do NumPy e OpenCV executam o processamento internamente em baixo nível (C/C++), o que torna operações vetorizadas mais rápidas do que implementações manuais com laços aninhados em Python.

Tabela 2.2: Comparativo entre estratégias de compactação e eficiência de processamento.

Característica	Python (NumPy/OpenCV)	ANSI C (Bit-packing)
Menor Unidade	1 Byte (8 bits)	1 Bit
Memória (Binária)	256 KB (para 512x512)	32 KB (para 512x512)
Velocidade	Alta (Vetorização em C)	Variável (Lenta se houver bit-shift)
Complexidade	Baixa: Métodos prontos	Alta: Ponteiros e Máscaras

2.6 Relações entre Pixels - Topologia da Imagem

Os pixels não são elementos isolados; suas posições relativas definem importantes conceitos para processamento.

2.6.1 Vizinhança

Dado um pixel de coordenadas (x, y) , definem-se dois tipos principais de vizinhança (para imagens em *grid* retangular):

- **Vizinhança-4** (von Neumann): inclui os pixels nas posições $(x-1, y)$, $(x+1, y)$, $(x, y-1)$, $(x, y+1)$.
- **Vizinhança-8** (Moore): inclui todos os oito pixels adjacentes (acrescenta os quatro diagonais).

A escolha da vizinhança influencia operações como detecção de bordas, cálculo de gradientes e conectividade.

```
# # Criação de uma matriz 3x3 para exemplo topológico
# viz = np.zeros((3, 3), dtype='uint8')

# # Definindo Vizinhança-4 (N4) com valor diferente para destaque
# viz[0, 1] = viz[2, 1] = viz[1, 0] = viz[1, 2] = viz[1, 1] = 255

# ou simplesmente (teste com números como argumentos):
viz = mm.secross()

# Exibição da matriz para análise de coordenadas
mm.drawImgPlt(viz, scale=40)
```

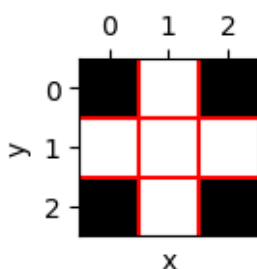


Figura 2.6: Ilustração de vizinhanças 4 em uma matriz 3x3. No centro (1,1), o pixel de interesse.

```
import morph
importlib.reload(morph)
from morph import mm
```

2.6.2 Adjacência, conectividade e caminhos

Dois pixels são **adjacentes** se estão em contato segundo uma vizinhança definida e satisfazem um critério de valor (ex.: mesmo nível de intensidade). Uma **conectividade** define uma relação de equivalência entre pixels que formam uma região conexa. Um **caminho** é uma sequência de pixels adjacentes.

A **conectividade-4** (N4) e **conectividade-8** (N8) podem produzir resultados diferentes na segmentação e no cálculo de componentes conexas (etiquetagem). Por exemplo, um padrão de tabuleiro de xadrez pode ser completamente desconexo em N4, mas totalmente conexo em N8.

2.6.3 Distâncias entre pixels

As métricas de distância são fundamentais para quantificar a proximidade física e a conectividade entre os elementos que compõem a grade digital. Conforme demonstrado na Table 2.3, a escolha da métrica define o custo de deslocamento entre pixels e altera o comportamento de algoritmos de segmentação e análise morfológica.

Para medir a distância entre dois pixels $p(x_1, y_1)$ e $q(x_2, y_2)$, utilizam-se diferentes funções métricas que impõem restrições de movimento distintas sobre o *grid*:

Tabela 2.3: Comparativo de métricas de distância aplicadas à malha de pixels.

Métrica	Definição	Interpretação
Euclidiana	$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	Distância exata em linha reta (contínua)
Manhattan (<i>City block</i>)	$ x_1 - x_2 + y_1 - y_2 $	Movimentos horizontais + verticais
Chebyshev (<i>Tabuleiro</i>)	$\max(x_1 - x_2 , y_1 - y_2)$	Maior deslocamento entre os eixos

Essas distâncias são aplicadas em diversos contextos de PDI, incluindo algoritmos de interpolação geométrica, transformadas de distância, crescimento de regiões e análise de formas.

i Exemplo prático

Considerando dois pixels com deslocamentos relativos $\Delta x = 3$ e $\Delta y = 4$:

- **Euclidiana**: $\sqrt{3^2 + 4^2} = 5$ (hipotenusa do triângulo retângulo).
- **Manhattan**: $3 + 4 = 7$ (soma dos catetos).
- **Chebyshev**: $\max(3, 4) = 4$ (predomínio do maior deslocamento).

2.7 Armazenamento de Imagens

A escolha do formato de arquivo é um passo decisivo no fluxo de processamento, pois determina como os dados de amostragem e quantização serão preservados ou descartados. Conforme se apresenta na Table 2.4, cada extensão equilibra de forma distinta a fidelidade dos dados e a eficiência de armazenamento.

Tabela 2.4: Principais formatos de armazenamento de imagens digitais e suas aplicações em PDI.

Formato	Características	Uso típico
PGM	Formato simples de mapa de cinzas (texto ou binário).	Pesquisa acadêmica e ferramentas Unix.
BMP	Não comprimido (ou compressão simples).	Windows, aplicações legado.
PNG	Compressão sem perdas (<i>lossless</i>).	Web, imagens com transparência.
JPEG	Compressão com perdas (<i>lossy</i>), ideal para fotografias.	Fotos, câmeras digitais.
TIFF	Suporta múltiplas camadas e compressão variada.	Editoração, arquivamento.
RAW	Dados brutos do sensor, sem processamento.	Fotografia profissional.

Formato	Características	Uso típico
DICOM	Padrão médico com metadados clínicos embutidos (paciente, equipamento, protocolo).	Radiologia, tomografia, ressonância magnética.

Os metadados de uma imagem incluem parâmetros como largura, altura, profundidade de bits e codificação de cor. Em formatos científicos, preservam-se também informações de calibração e detalhes da captura. Ao utilizar-se a função `mm.read()`, a biblioteca `morph` preserva automaticamente esses dados para que se respeitem as propriedades originais da imagem.

Em contextos científicos e hospitalares, privilegia-se o padrão **DICOM** (*Digital Imaging and Communications in Medicine*) para garantir que não haja perda de precisão diagnóstica. Repositórios públicos como o [The Cancer Imaging Archive \(TCIA\)](#), o [Alzheimer’s Disease Neuroimaging Initiative \(ADNI\)](#) e o [PhysioNet](#) disponibilizam vastos conjuntos de dados neste formato, incluindo metadados clínicos anonimizados que são essenciais para a investigação científica.

2.7.1 Exemplo: Extração de Metadados e Localização GPS

Diferente da matriz de pixels pura obtida pela leitura convencional — como na imagem do pássaro apresentada no início deste capítulo —, o uso do argumento `pil=True` no método `mm.read()` altera a natureza do objeto retornado (ver Figure 2.7). Enquanto o padrão (`pil=False`) retorna um `numpy.ndarray` RGB, a leitura com `pil=True` retorna um objeto especializado da biblioteca Pillow, capaz de interpretar o cabeçalho **EXIF**.

O cabeçalho **EXIF** (*Exchangeable Image File Format*) funciona como um repositório técnico da captura, permitindo que o objeto Pillow interprete uma vasta gama de informações que vão muito além das coordenadas de GPS. Ao utilizar `pil=True`, o sistema ganha acesso ao “DNA” da imagem, incluindo metadados de hardware (marca e modelo da câmera), configurações ópticas (abertura, distância focal e tempo de exposição) e parâmetros de iluminação (uso de flash e balanço de branco).

Essa distinção é importante no PDI, pois transforma a matriz de amostragem em um conjunto de dados contextualizado, onde as características físicas do sensor e da lente podem ser utilizadas para normalizar brilhos ou corrigir distorções geométricas.

```
from PIL.ExifTags import TAGS
import os, numpy as np

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = (
    "Area_de_Prote%C3%A7%C3%A3o_Ambiental_"
    "Quilombos_do_M%C3%A9dio_Ribeira_-_Thomas-"
    "Fuhrmann_%282023-02%29_Malacoptila_striata.jpg"
)
url = f"{base}/c/c5/{arquivo}"
caminho = "imagens/barbudo-rajado.jpg"

# 1. Leitura - preserva objeto PIL com EXIF
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho) # salva preservando EXIF
else:
    img_obj = mm.read(caminho, pil=True)

img_numpy = np.array(img_obj) # conversão para NumPy

# 2. Extração e conversão de GPS (Tag 34853)
exif = img_obj._getexif()
```



```

if exif and (gps := exif.get(34853)):
    to_dec = lambda dms, ref: float(-(dms[0]+dms[1]/60+dms[2]/3600) if ref in 'SW'
                                   else (dms[0]+dms[1]/60+dms[2]/3600))
    lat, lon = to_dec(gps[2], gps[1]), to_dec(gps[4], gps[3])
    print(f"GPS Decimal: {lat:.6f}, {lon:.6f}")
    print(f"Maps: https://www.google.com/maps/search/?api=1&query={lat},{lon}")

# 3. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Pillow (0,0): {img_obj.getpixel((0, 0))}")
print(f"Tipo NumPy: {type(img_numpy)} | Dimensões [y,x,c]: {img_numpy.shape}")
print(f"NumPy [0,0]: {img_numpy[0, 0]}")

# 4. Exibição
mm.show(img_numpy, scale=30)

```

```

GPS Decimal: -24.587955, -48.629758
Maps: https://www.google.com/maps/search/?api=1&query=-24.587955,-48.629758333333335

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (2047, 3067)
Pillow (0,0): (58, 96, 0)
Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (3067, 2047, 3)
NumPy [0,0]: [58 96  0]

```

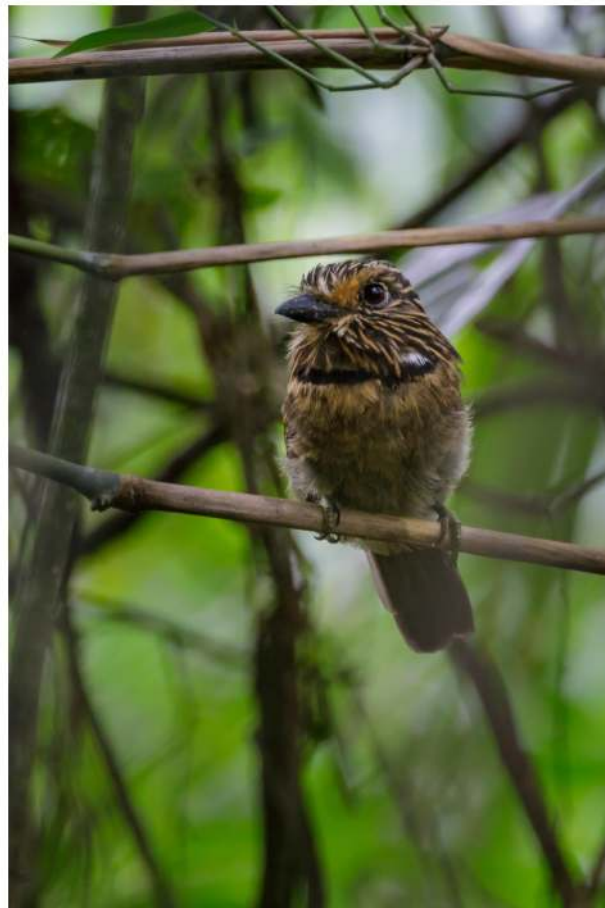


Figura 2.7: Área de Proteção Ambiental Quilombos do Médio Ribeira - Barbudo-rajado (*Malacoptila striata*). Crédito: Thomas Fuhrmann (CC BY-SA 4.0).

i Nota Pedagógica: A sutil diferença das dimensões

Repare que a representação das dimensões muda conforme a estrutura de dados utilizada:

- **No Pillow (.size):** Retorna (Largura, Altura) — no exemplo: (2047, 3067). É uma visão orientada ao arquivo de imagem.
- **No NumPy (.shape):** Segue a convenção matemática de matrizes: (Linhas/Altura, Colunas/Largura, Canais) — no exemplo: (3067, 2047, 3).

Esta distinção é fundamental para evitar erros de indexação ao implementar filtros manuais. Enquanto o objeto Pillow carrega o “**onde**” e o “**quando**” (contexto), o array NumPy carrega o “**quanto**” de luz (intensidade) existe em cada ponto da imagem.

2.7.2 Por que esta separação é importante?

Ao carregar uma imagem via `cv2.imread()`, o resultado é um `<class 'numpy.ndarray'>`, que contém estritamente os valores numéricos resultantes da **quantização** e **amostragem**. No entanto, ao utilizar `pil=True`, o `mm.read()` retorna um objeto da classe `PIL.JpegImagePlugin.JpegImageFile`.

Esta classe mantém o arquivo “aberto” para permitir o acesso ao contexto da captura antes que os dados sejam convertidos em uma matriz bruta. Note que o pixel (0, 0) é idêntico nas duas representações — (58, 96, 0) no Pillow e [58, 96, 0] no NumPy —, confirmando que ambos descrevem os mesmos dados, apenas com interfaces distintas. Essa separação é vital: pixels servem para algoritmos; metadados servem para georreferenciamento, catalogação científica e correções baseadas no hardware de aquisição.

Para inspecionar todos os metadados EXIF de um objeto Pillow:

```
from PIL import ExifTags

exif_raw = img_obj._getexif()
if exif_raw:
    for tag_id, valor in sorted(exif_raw.items()):
        tag_nome = ExifTags.TAGS.get(tag_id, f"TAG_{tag_id}")
        print(f" {tag_nome:40s} : {valor}")
```

2.8 Transformações Geométricas Básicas

As transformações geométricas alteram a posição dos pixels, mantendo os valores de intensidade. São fundamentais para alinhamento, correção de distorções e aumento de dados (*data augmentation*) em aprendizado de máquina.

Uma **transformação afim** é qualquer mapeamento que preserve colinearidade (pontos sobre uma reta permanecem sobre uma reta) e razões de distâncias entre pontos colineares. Em 2D, toda transformação afim pode ser expressa em **coordenadas homogêneas** por uma matriz 3×3 :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}}_{\mathbf{T}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2.2)$$

A sub-matriz 2×2 superior esquerda $\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$ controla rotação, escala e cisalhamento; o vetor $(t_x, t_y)^\top$ controla a translação. As transformações geométricas mais usadas em PDI — translação, rotação e escala — são casos particulares de $\mathbf{T}$, e podem ser **compostas** por multiplicação de matrizes, na ordem $\mathbf{T} = \mathbf{T}_n \cdots \mathbf{T}_2 \mathbf{T}_1$.

i Transformação inversa e interpolação

Na implementação prática (`cv2.warpAffine`), aplica-se a **transformação inversa**: para cada pixel (x', y') da imagem destino, calcula-se a posição de origem $(x, y) = \mathbf{T}^{-1}(x', y')$ e interpola-se o valor. Isso evita buracos na imagem resultante causados pelo mapeamento direto de pixels inteiros para posições não inteiras.

2.8.1 Translação

A translação é a transformação afim mais simples: desloca todos os pixels por um vetor (t_x, t_y) . Em coordenadas homogêneas, é expressa pela matriz:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow \begin{cases} x' = x + t_x \\ y' = y + t_y \end{cases} \quad (2.3)$$

A terceira linha da matriz garante que a operação permaneça no espaço afim, permitindo que translação, rotação e escala sejam combinadas por simples multiplicação de matrizes. Na prática, `cv2.warpAffine` usa apenas as duas primeiras linhas (matriz 2×3), pois a terceira é sempre $[0, 0, 1]$.

Pixels deslocados para além da área original são descartados; áreas descobertas são preenchidas com 0 (preto). Ver um exemplo na Figure 2.8.

```
img_tx1 = mm.translate(img_gray, 50, 50)
img_tx2 = mm.translate(img_gray, 100, 50)
mm.show([img_gray, img_tx1, img_tx2],
        titles=["Original", "Translação (50,50)", "Translação (100,50)"],
        cols=3, figsize=(16, 12))
```



Figura 2.8: Exemplo de translação da imagem do pássaro com deslocamentos (50,50) e (100,50).

2.8.2 Rotação

A rotação por ângulo θ em torno de um ponto central (c_x, c_y) é composta por três transformações afins: translação para a origem, rotação pura e translação de volta. A matriz resultante é:

$$\mathbf{T}_{\text{rot}} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) + c_y \sin \theta \\ \sin \theta & \cos \theta & c_y(1 - \cos \theta) - c_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \quad (2.4)$$

Na implementação, `cv2.getRotationMatrix2D` gera diretamente as duas primeiras linhas de $\mathbf{T}_{\text{rot}}$ (matriz 2×3 para `warpAffine`), aceitando também um fator de escala s que multiplica $\cos \theta$ e $\sin \theta$. Ver exemplo na Figure 2.9.

Como a rotação desloca pixels para novas posições não-inteiras, `cv2.warpAffine` precisa estimar a cor de cada pixel de destino a partir dos vizinhos — processo chamado **interpolação**. O parâmetro `interp` controla esse comportamento:

- **nearest** (`INTER_NEAREST`): atribui o valor do pixel mais próximo. Rápido, mas produz serrilhado (*aliasing*) em bordas diagonais.
- **bilinear** (`INTER_LINEAR`, padrão): média ponderada dos 4 vizinhos mais próximos. Equilibra qualidade e desempenho — adequado para a maioria dos casos.
- **bicubic** (`INTER_CUBIC`): considera os 16 vizinhos em uma superfície cúbica. Produz bordas mais suaves, ao custo de maior processamento.

```
img_rot30 = mm.rotate(img_gray, 30, interp='bilinear')
img_rot45 = mm.rotate(img_gray, 45, interp='bilinear')

mm.show([img_gray, img_rot30, img_rot45],
        titles=["Original", "Rotação 30°", "Rotação 45°"],
        cols=3, figsize=(16, 12))
```



Figura 2.9: Exemplo de rotação da imagem do pássaro em 30° e 45° usando interpolação bilinear.

2.8.3 Escala (Redimensionamento)

O redimensionamento por fatores (s_x, s_y) é uma transformação afim com matriz:

$$\mathbf{T}_{\text{escala}} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.5)$$

Quando $s > 1$ (ampliação), pixels da imagem destino mapeiam para posições não inteiras na origem — exigindo interpolação para estimar o valor. Quando $s < 1$ (redução), múltiplos pixels de origem contribuem para um único pixel destino — exigindo **decimação**. Os mesmos três métodos descritos na rotação estão disponíveis em `mm.resize`, com a diferença de que aqui o impacto visual é mais perceptível: na ampliação, **nearest** produz efeito de blocos (*pixelation*), enquanto **bicubic** preserva melhor a nitidez das bordas, conforme a Table 2.5:

Tabela 2.5: Métodos de interpolação disponíveis em `mm.resize` e seus respectivos números de vizinhos utilizados no cálculo.

Método	Vizinhos usados	Característica
'nearest'	1	Rápido; produz efeito de blocos (<i>pixelation</i>)
'bilinear'	4	Bom compromisso qualidade/custo; bordas suaves
'bicubic'	16	Maior nitidez; preferido em softwares profissionais

O parâmetro `size_or_factor` aceita tanto um escalar (fator uniforme, e.g. 0.5 para reduzir à metade) quanto uma tupla (largura, altura) para dimensões absolutas.

```
# 1. Recorte da região do bico
y, x, offset = 210, 40, 40
crop = img_gray[y-offset:y+offset, x-offset:x+offset]
print(f"Imagem: {img_gray.shape} | Crop: {crop.shape}")

# 2. Ampliar 4× com mm.resize
crop_nearest = mm.resize(crop, size_or_factor=4, method='nearest')
crop_bilinear = mm.resize(crop, size_or_factor=4, method='bilinear')

# 3. Exibição comparativa
mm.show(
    [crop, crop_nearest, crop_bilinear],
    titles=["Original (recorte)", "Vizinho mais próximo (4×)", "Bilinear (4×)"],
    cols=3, figsize=(16, 12), dpi=200
)
```

Imagem: (1030, 660) | Crop: (80, 80)



Figura 2.10: Comparação de interpolação com zoom no detalhe do olho (recorte 60×60, ampliado 4×). Note o efeito de blocos no vizinho mais próximo vs. a suavização na bilinear.

2.8.4 Cisalhamento (*Shear*)

O cisalhamento é uma transformação afim que distorce a imagem ao deslocar cada pixel proporcionalmente à sua posição em um eixo. A matriz geral combina cisalhamento horizontal (sh_x) e vertical (sh_y):

$$\mathbf{T}_{\text{cisalh}} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.6)$$

Para $sh_x \neq 0$ e $sh_y = 0$, cada linha é deslocada horizontalmente proporcionalmente à sua posição vertical — produzindo o efeito de “inclinação” característico. Ver exemplo na Figure 2.11.

```
img_shx = mm.shear(img_gray, shx=0.3)
img_shy = mm.shear(img_gray, shy=0.3)
img_shc = mm.shear(img_gray, shx=0.2, shy=0.2)

mm.show(
    [img_gray, img_shx, img_shy, img_shc],
    titles=["Original", "Horiz. (shx=0.3)", "Vert. (shy=0.3)", "Combinado (0.2, 0.2)"],
    cols=4, figsize=(16, 12)
)
```

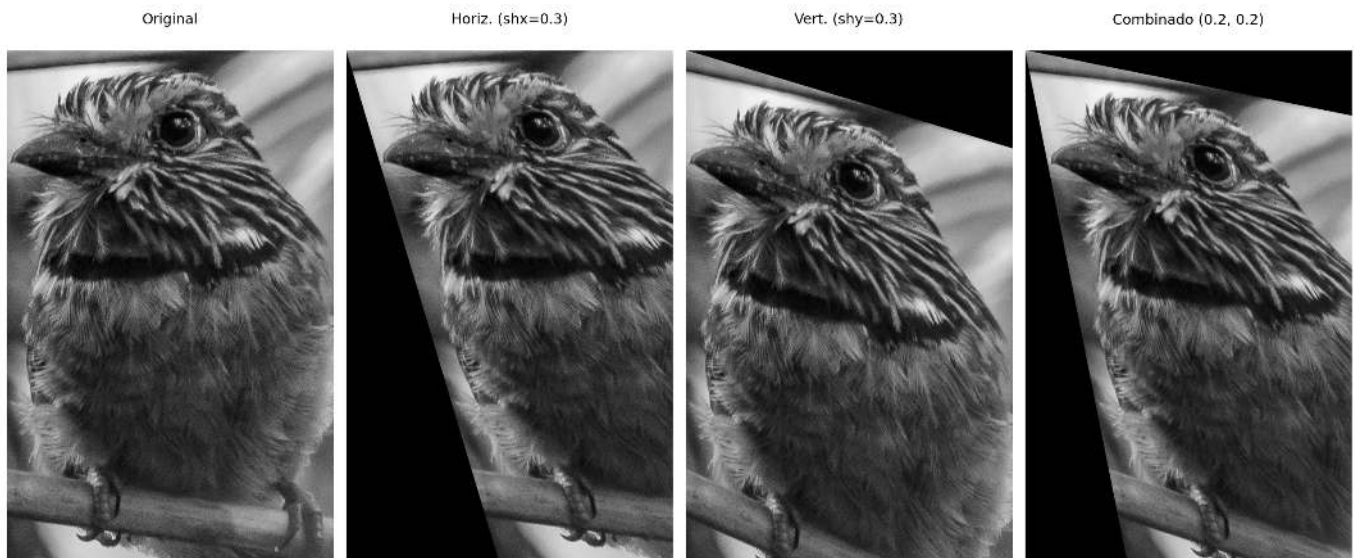



Figura 2.11: Exemplo de cisalhamento da imagem do pássaro: horizontal ($shx=0.3$), vertical ($shy=0.3$) e combinado ($shx=0.2$, $shy=0.2$).

2.9 Resumo

Neste capítulo foram apresentados os fundamentos de digitalização e topologia de imagens:

- **Formação da imagem:** $f(x, y) = i(x, y) \cdot r(x, y)$.
- **Amostragem:** discretização do espaço $\rightarrow$ resolução espacial $M \times N$.
- **Quantização:** discretização da intensidade $\rightarrow$ profundidade de bits b .
- **Relações topológicas:** vizinhança-4, vizinhança-8, conectividade, distâncias (Euclidiana, Manhattan, Chebyshev).
- **Transformações geométricas:** translação, rotação, escala (com interpolação bilinear ou vizinho mais próximo).
- **Formatos de arquivo:** BMP, PNG, JPEG, TIFF, RAW; cada um com diferentes compromissos entre qualidade e tamanho.

O Capítulo 3 abordará **operações espaciais** como convolução, filtragem e morfologia matemática (erosão, dilatação).

2.10 Uso do Gemini Notebook como Tutor Complementar

Nesta edição, incentivamos o uso do **Gemini Notebook** como ferramenta complementar de aprendizagem. Essa ferramenta de IA utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro.

Para cada capítulo, preparamos um projeto específico na plataforma. Para uma experiência de estudo ampliada, utilize o acesso abaixo:

Estude com o Tutor Inteligente

Para interagir com o conteúdo deste capítulo, acesse o link a seguir. O ambiente contém materiais didáticos em diferentes formatos, gerados a partir do **PDF** do capítulo. Na plataforma, explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 [ACESSAR Gemini Notebook: CAPÍTULO 02](#)

⚠ Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

2.11 Lista de Exercícios

1. (15%) Explique, com suas próprias palavras, a diferença entre **amostragem** e **quantização**. Dê um exemplo concreto de cada uma no contexto de uma imagem digital.
2. (15%) Considere uma imagem com resolução espacial de 1024×768 pixels e profundidade de 24 bits (8 bits por canal RGB). Calcule o tamanho total não comprimido da imagem em bytes e em megabytes.
3. (20%) Utilizando o código do laboratório, modifique o fator de subamostragem para 3 e para 6. Descreva visualmente o que ocorre com as bordas dos objetos. O que é o efeito de *aliasing*?
4. (20%) Para a imagem em tons de cinza, aplique quantização com 3 bits (8 níveis) e 5 bits (32 níveis). Compare os resultados e explique por que 5 bits já pode ser considerado suficiente para muitas aplicações.
5. (15%) Dados dois pixels $A = (10, 20)$ e $B = (15, 25)$, calcule as distâncias Euclidiana, Manhattan e Chebyshev entre eles.
6. (15%) Usando a função `mm.rotate`, gire a imagem do pássaro em ângulos de 90° , 180° e 270° com interpolação bilinear. Compare com a rotação usando `method='nearest'`. Em quais situações a interpolação vizinho mais próximo ainda é útil?

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de amostragem, quantização e relações entre pixels.
- Szeliski (2022) para transformações geométricas e conectividade.
- Bradski; Kaehler (2008) para a implementação prática com OpenCV e `morph.py`.



2.12 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Para avaliar os testes, execute `TestSuite("EP04_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (.py, .java, .c, .cpp, .js ou .r). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""
TestSuite("EP04_01").run_code(codigo)
```

2.12.1 EP02_01 ☉ Ajuste de Brilho e Contraste

Nesta atividade, o objetivo é implementar um operador de ponto para **transformação linear de intensidade**, aplicando o ajuste dinâmico de brilho e contraste em uma imagem digital.

2.12.1.1 📋 Diretrizes de Implementação

O algoritmo deve seguir o fluxo de execução abaixo:

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas) da matriz.
2. **Parâmetros:** Ler o valor real α (fator de contraste) e o inteiro β (fator de brilho).
3. **Dados:** Ler os valores inteiros da matriz original.
4. **Mapeamento:** Para cada pixel p , calcular o novo valor p' pela equação:

$$p' = \text{clip}(\text{round}(\alpha \cdot p + \beta))$$

5. **Saída:** Exibir a matriz resultante com as dimensões $L \times C$.

2.12.1.2 📌 Restrições Computacionais

- **Arredondamento (*Round*):** Aplica-se o arredondamento matemático para o inteiro mais próximo antes da conversão de tipo.
- **Saturação (*Clipping*):** Os valores devem ser confinados ao intervalo $[0, 255]$ para preservar o padrão de 8 bits:

$$\text{clip}(x) = \max(0, \min(255, x))$$

- **Simulação:** O efeito dos parâmetros α e β na correção histogramática pode ser observado na Figure 2.12.

2.12.1.3 Fundamentação Teórica

As alterações modificam o histograma da imagem para ajustar o perfil de iluminação e a distinção tonal.

Parâmetro	Função	Impacto Visual
α (Alpha)	Escalar	Modula o Contraste . Se $\alpha > 1$, expande o histograma; se $0 \leq \alpha < 1$, comprime.
β (Beta)	Aditiva	Modula o Brilho . Se positivo, translada o histograma para a direita; se negativo, para a esquerda.
clip	Limitador	Restringe a faixa dinâmica, impedindo erros de <i>underflow</i> e <i>overflow</i> .

2.12.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Valores de **alpha** (α) e **beta** (β).
- Linhas seguintes: Elementos numéricos da matriz original.

Saída:

- Matriz transformada estruturada em L linhas e C colunas.

2.12.1.5 Exemplos

A tabela a seguir apresenta um exemplo prático do comportamento esperado do algoritmo, destacando a atuação dos operadores de arredondamento e saturação.

Entrada	Saída	Observação
1 4 1.5 -30 0 100 180 255	0 120 240 255	Note o efeito de saturação no último pixel

Executando os Testes

Para avaliar os testes, execute `TestSuite("EP02_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

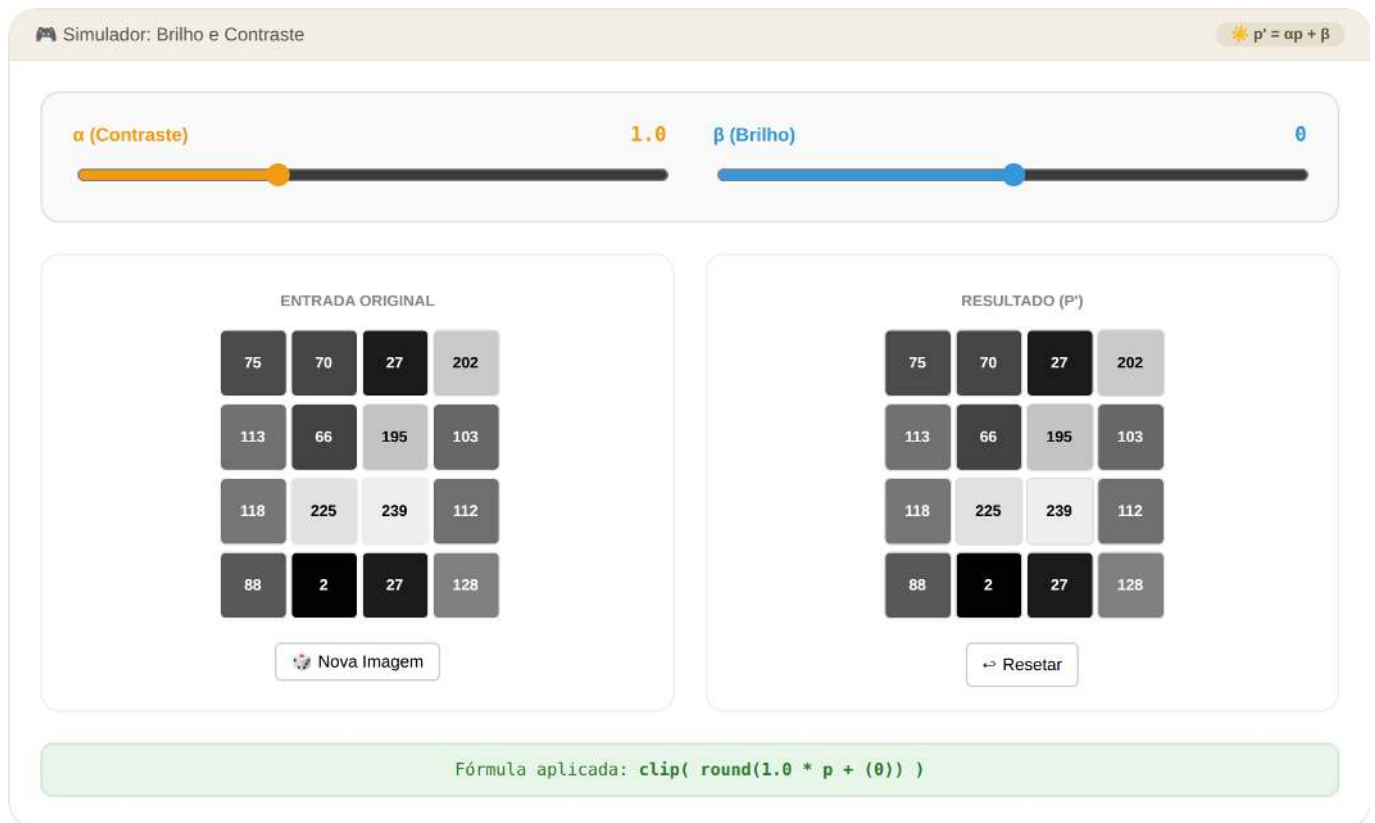


Figura 2.12: Simulador: Ajuste de Brilho e Contraste

```
%%writefile EP02_01.py
# sua solução
```

Writing EP02_01.py

```
TestSuite("EP02_01.py").run()
```

2.12.2 EP02_02 Subamostragem Espacial

Nesta atividade, você deve implementar a redução da resolução espacial de uma imagem através do processo de subamostragem.

- Leia dois inteiros L e C , representando as dimensões da matriz original.
- Leia um valor inteiro f ($f \geq 1$), que representa o fator de amostragem.
- Leia os valores inteiros da matriz original.
- A nova imagem deve ser construída selecionando o pixel da posição $(f \cdot i, f \cdot j)$ da imagem original.
- Imprima a matriz resultante com as novas dimensões.
- Ver na Figure 2.13 uma simulação deste EP.

Importante:

- **Dimensões Finais:** A imagem amostrada terá dimensões $\lceil L/f \rceil \times \lceil C/f \rceil$. No contexto de programação, isso equivale ao tamanho resultante de um fatiamento (slicing) com passo f .
- **Implementação:** Não utilize funções prontas de bibliotecas de processamento de imagens (como OpenCV ou PIL) para o redimensionamento. Implemente a lógica de seleção de pixels manualmente ou via fatiamento de matrizes.

- **Aliasing:** Note que este processo pode causar o efeito de *aliasing* (serrilhamento), onde detalhes finos são perdidos ou padrões indesejados aparecem.

2.12.2.1 🧠 Discretização do Espaço

A subamostragem reduz a resolução espacial de uma imagem, selecionando apenas um pixel a cada f pixels em cada direção. É o processo inverso da interpolação:

Parâmetro	Função	Efeito
Fator f	Salto de amostragem	Define o intervalo de seleção. Um fator 2 reduz a largura e altura pela metade.
Resolução	Densidade de pixels	Diminui a quantidade total de informação espacial da imagem.
Aliasing	Efeito colateral	Surgimento de padrões em escada ou blocos devido à perda de detalhes finos.

2.12.2.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o fator f .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz reduzida com as dimensões correspondentes ao fatiamento por f .

2.12.2.3 📌 Exemplos

Entrada	Saída	Observação
2 4 2 10 20 30 40 50 60 70 80	10 30	O fator 2 seleciona os pixels (0,0) e (0,2) da primeira linha. A segunda linha é ignorada.

```
%%writefile EP02_02.py
# Código Python
```

```
Writing EP02_02.py
```

```
TestSuite("EP02_02.py").run()
```

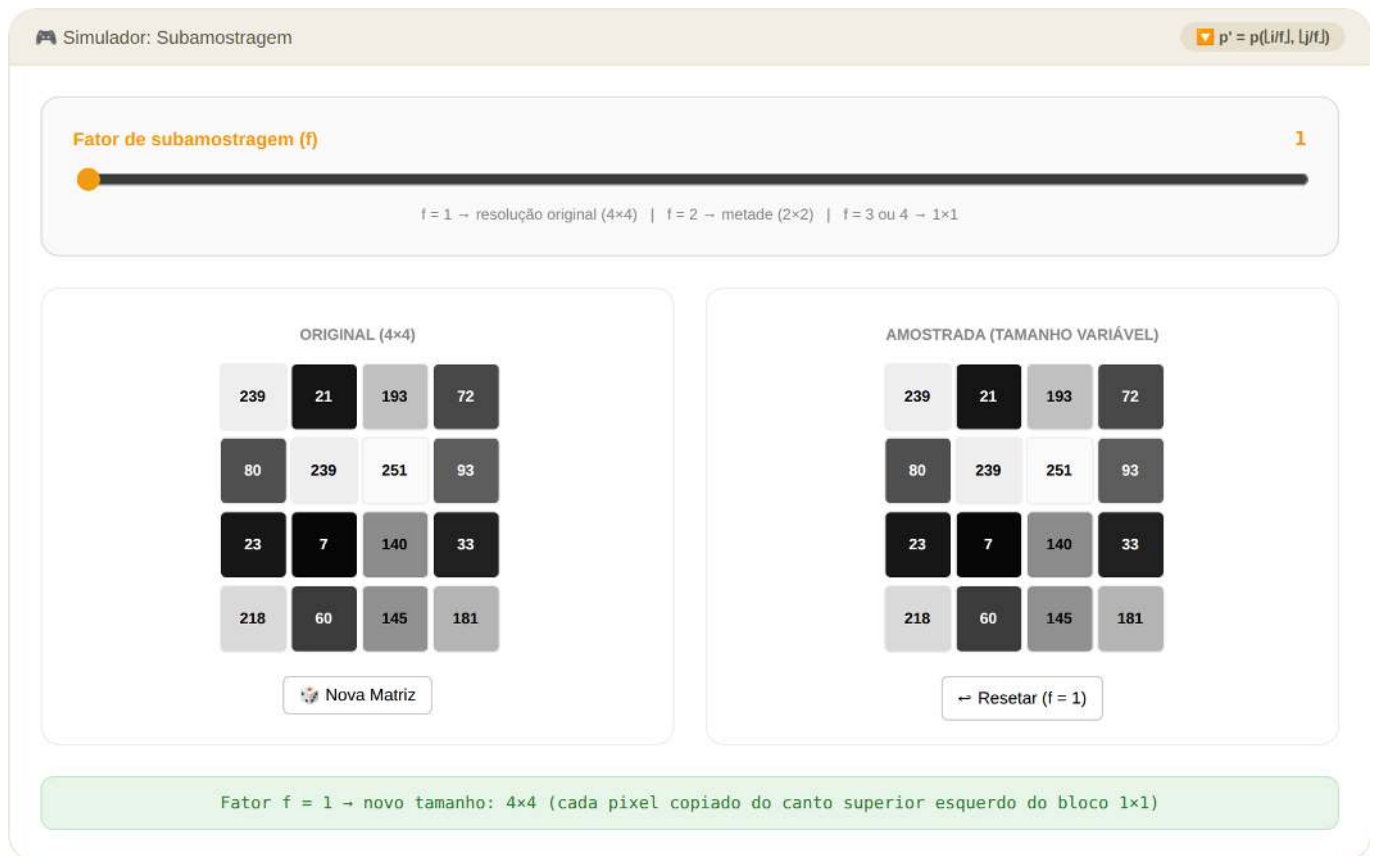



Figura 2.13: Simulador: Subamostragem

2.12.3 EP02_03 🐼 Quantização de Níveis de Cinza

Nesta atividade, você deve implementar a quantização uniforme de uma imagem, reduzindo a quantidade de níveis de intensidade de cinza originais para uma nova escala baseada em um número menor de bits.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia um inteiro k ($1 \leq k \leq 8$), representando o novo número de bits da imagem.
- Calcule o número de níveis ($N = 2^k$) e o tamanho do intervalo (passo).
- Para cada pixel p , calcule o novo valor p' mapeando-o para o índice do nível discretizado correspondente (variando de 0 a $2^k - 1$).
- Imprima a matriz resultante com os mesmos valores de dimensões originais.
- Ver na Figure 2.14 uma simulação deste EP.

📌 Importante:

- **Posterização:** Ao reduzir drasticamente os níveis (ex: $k = 2$), você notará que degradês suaves se transformam em faixas abruptas de cor devido à perda de resolução de amplitude.
- **Cálculo do Passo:** O intervalo entre cada nível é definido por $passo = 256/2^k$.
- **Mapeamento:** O método de quantização uniforme por truncamento que mapeia o pixel para o índice do seu respectivo nível discretizado é dado por:

$$p' = \left\lfloor \frac{p}{passo} \right\rfloor$$

Em termos de implementação (como em Python), isso equivale à divisão inteira: $p' = p // passo$.

2.12.3.1 🧠 Discretização da Amplitude

Enquanto a subamostragem lida com a resolução espacial, a quantização foca na precisão da cor (amplitude). Reduzir os bits significa simplificar a informação cromática:

Parâmetro	Função	Efeito
Bits (k)	Profundidade de cor	Define quantos tons diferentes a imagem pode ter (2^k).
Passo	Intervalo de tom	Espaçamento entre os níveis de cinza permitidos.
Posterização	Fenômeno visual	Transformação de variações contínuas em blocos de cor sólida.

2.12.3.2 📋 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o número de bits k .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com os índices dos níveis quantizados, mantendo o tamanho original $L \times C$.

2.12.3.3 📌 Exemplos

Entrada	Saída	Observação
1 4 2 0 80 170 255	0 1 2 3	Com $k = 2$, temos $2^2 = 4$ níveis discretos disponíveis (0, 1, 2, 3). O passo é $256/4 = 64$. Aplicando a divisão inteira por elemento: $0//64 = 0$, $80//64 = 1$, $170//64 = 2$, $255//64 = 3$.
1 5 1 10 50 120 200 250	0 0 0 1 1	Com $k = 1$, temos $2^1 = 2$ níveis (0 e 1). Passo = $256/2 = 128$. Pixels menores que 128 resultam em 0, e pixels maiores ou iguais a 128 resultam em 1.

```
%%writefile EP02_03.py
# Código Python
```

```
Writing EP02_03.py
```

```
TestSuite("EP02_03.py").run()
```

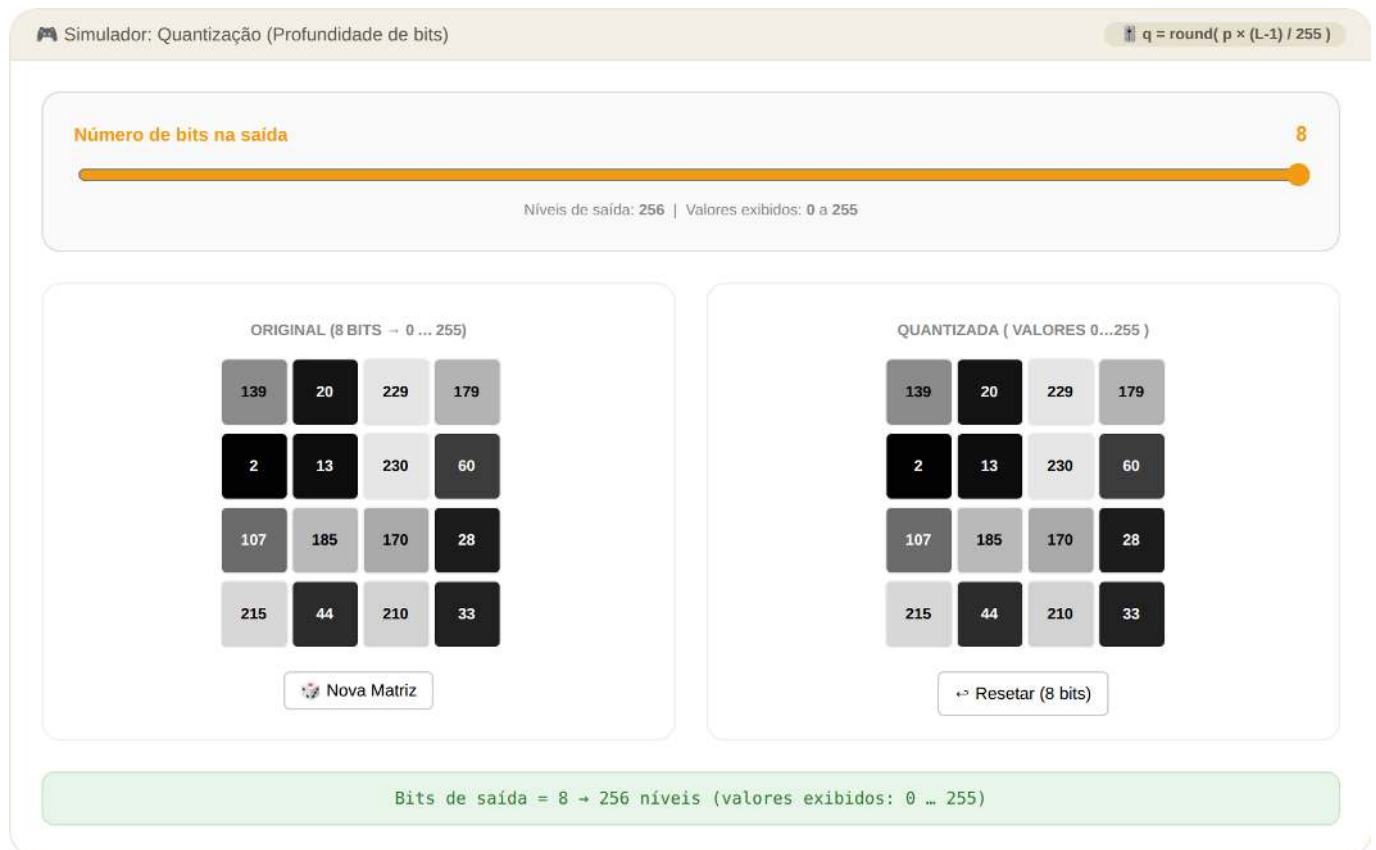


Figura 2.14: Simulador: Quantização

2.12.4 EP02_04 Transformada de Distância em Imagem Binária

Dada uma imagem binária onde pixels de valor **1** representam o *objeto* e pixels **0** representam o *fundo*, a distância de um pixel de fundo recebe a **menor distância ao pixel de objeto mais próximo**. Pixels de objeto recebem distância **0**. Para simplificar, considere que a imagem tem **apenas um único objeto com um pixel de valor 1**.

Problema: Leia uma imagem binária $L \times C$ e uma métrica, e calcule essa distância simplificada aplicando uma das três fórmulas:

$$d_{\text{Euclidiana}} = \sqrt{(\Delta r)^2 + (\Delta c)^2}$$

$$d_{\text{City-block}} = |\Delta r| + |\Delta c|$$

$$d_{\text{Chessboard}} = \max(|\Delta r|, |\Delta c|)$$

onde Δr é a diferença de linhas e Δc a diferença de colunas entre dois pixels.

2.12.4.1 Por que isso importa? - Aplicações da DT

A Transformada de Distância (DT) aparece em dezenas de pipelines de visão computacional:

Métrica	Complexidade	Aplicação típica
Euclidiana	 $O(n^2)$ ingênuo	Esqueletização, matching de formas
City-block	 $O(n)$ com 2 passes	Morfologia, dilatação/erosão
Chessboard	 $O(n)$ com 2 passes	Morfologia, dilatação/erosão

2.12.4.2 Requisitos Técnicos

- **Entrada:** * Primeira linha: L e C (inteiros).
 - Segunda linha: nome da métrica (`euclidean`, `cityblock` ou `chessboard`).
 - Em seguida, a matriz binária $L \times C$ (valores 0 ou 1).
- **Pixels de objeto (1):** distância = 0 (ou 0.00 para euclidiana).
- **Pixels de fundo (0):** distância ao único pixel de objeto na imagem.
- **Arredondamento (euclidiana):** imprimir com **2 casas decimais** (formato `:.2f`). City-block e Chessboard produzem inteiros — imprimir sem decimais.
- **Saída:** valores separados por espaço, uma linha por linha da matriz.
- Ver na Figure 2.15 uma simulação deste EP.

2.12.4.3 Exemplos

Entrada	Saída	Observação
4	2 2 2 2	A distância Chessboard é $\max(\ dx\ , \ dy\)$. O único pixel objeto é $(2, 2) = 0$; os demais armazenam sua distância mínima até ele.
4	2 1 1 1	
chessboard	2 1 0 1	
0 0 0 0	2 1 1 1	
0 0 0 0		
0 0 1 0		
0 0 0 0		

2.12.4.4 Observações finais

- Como a imagem tem **apenas um objeto de um pixel**, a distância de cada pixel de fundo é simplesmente a distância desse pixel ao único ponto objeto.
- A implementação pode usar força bruta (percorrer todos os pixels da imagem e calcular a distância diretamente), pois L e C são pequenos nos casos de teste.
- Este problema é um aquecimento para a Transformada de Distância geral, que será trabalhada em capítulos posteriores com múltiplos objetos e algoritmos otimizados.

```
%%writefile EP02_04.py
# Código Python
```

```
Writing EP02_04.py
```

```
TestSuite("EP02_04.py").run()
```

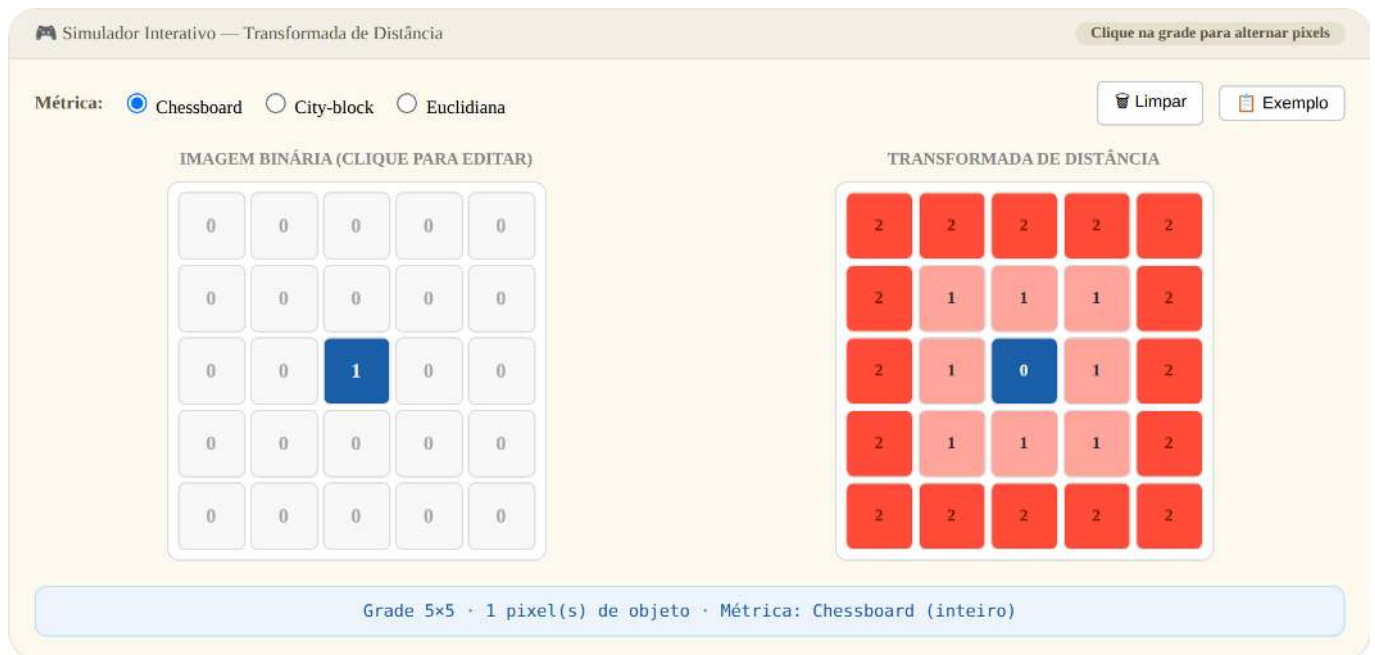


Figura 2.15: Simulador: Ajuste de Brilho e Contraste

2.12.5 EP02_05 Translação de Imagem

Nesta atividade, você deve implementar o deslocamento espacial de uma imagem. A translação move cada pixel da imagem original para uma nova posição com base em um vetor de deslocamento.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia dois inteiros t_x (deslocamento horizontal) e t_y (deslocamento vertical).
- Leia os valores inteiros da matriz original.
- Calcule a nova posição (x', y') para cada pixel (x, y) original.
- Imprima a matriz resultante com as mesmas dimensões da original.
- Ver na Figure 2.16 uma simulação deste EP.

Importante:

- **Preenchimento:** Pixels que “entram” na imagem devido ao deslocamento e não possuem correspondente na original devem ser preenchidos com **0** (preto).
- **Descarte:** Pixels que, após a translação, ficarem fora dos limites da matriz ($0 \dots L - 1$ ou $0 \dots C - 1$) devem ser ignorados.
- **Coordenadas:** Considere x como o índice da linha e y como o índice da coluna.

2.12.5.1 Deslocamento Espacial

Transladar uma imagem significa mover todos os seus pontos por uma distância fixa em direções especificadas. Matematicamente, usando coordenadas homogêneas, a operação é descrita como:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Que resulta nas equações simples:

- $x' = x + t_x$
- $y' = y + t_y$

2.12.5.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os inteiros t_x e t_y .

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz resultante com as mesmas dimensões $L \times C$ após o deslocamento.

2.12.5.3 Exemplos

Entrada	Saída	Observação
2 2 1 1 10 20 30 40	0 0 0 10	Deslocamento ($t_x = 1, t_y = 1$): Cada pixel move uma posição para a direita (horizontal) e uma para baixo (vertical). O pixel $(0, 0) = 10$ vai para o destino $(1, 1)$ (canto inferior direito). As posições vazias são preenchidas com 0.
3 3 -1 0 1 2 3 4 5 6 7 8 9	2 3 0 5 6 0 8 9 0	Deslocamento ($t_x = -1, t_y = 0$): Cada pixel move uma posição para a esquerda (horizontal). A primeira coluna original (1, 4, 7) é descartada, as demais colunas movem-se para a esquerda, e a última coluna resultante é preenchida com zeros (0).

```
%%writefile EP02_05.py
# Código Python
```

```
Writing EP02_05.py
```

```
TestSuite("EP02_05.py").run()
```

2.12.6 EP02_06 Rotação de Imagem

Nesta atividade, você deve implementar a rotação de uma imagem em torno do seu centro geométrico. Esta operação requer o mapeamento de coordenadas e o uso de técnicas de interpolação para determinar os novos valores dos pixels.

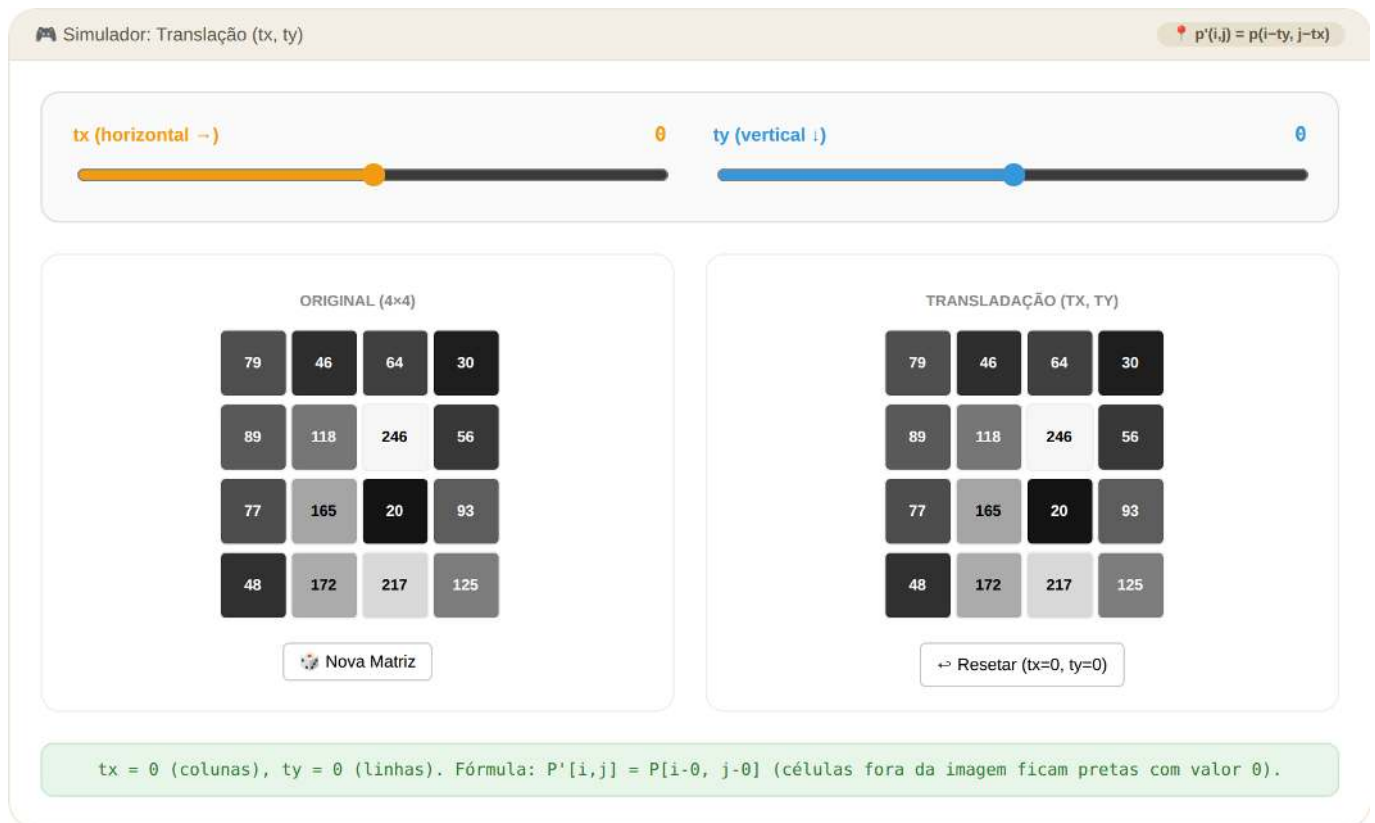


Figura 2.16: Simulador: Translação (tx, ty)

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia um valor real θ (ângulo em graus) e uma *string* representando o **método** de interpolação (**nearest** ou **bilinear**).
- Leia os valores inteiros da matriz original.
- Realize a rotação em torno do centro da imagem ($L/2, C/2$).
- Imprima a matriz resultante com as mesmas dimensões da original.
- Ver na Figure 2.17 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para evitar “buracos” na imagem final, percorra cada pixel (x', y') da imagem de destino e calcule sua posição correspondente (x, y) na imagem original usando a matriz de rotação inversa.
- **Interpolação:**
 - **nearest:** Atribui o valor do pixel mais próximo da coordenada calculada.
 - **bilinear:** Calcula uma média ponderada baseada nos 4 vizinhos mais próximos.
- **Bordas:** Pixels cuja origem (x, y) caia fora dos limites da imagem original devem ser preenchidos com 0.

2.12.6.1 🧠 Transformação por Ângulo

A rotação de um ponto (x, y) em relação à origem por um ângulo θ é dada pela matriz de transformação. Para rotacionar em torno de um centro (x_c, y_c) , primeiro transladamos o centro para a origem, rotacionamos e transladamos de volta:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & x_c \\ \sin \theta & \cos \theta & y_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x - x_c \\ y - y_c \\ 1 \end{bmatrix}$$

Dica: Use o mapeamento inverso para garantir que todos os pixels da imagem de saída sejam preenchidos corretamente.

2.12.6.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém o ângulo `theta` (em graus) e o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz rotacionada com L linhas e C colunas.

2.12.6.3 Exemplos

Entrada	Saída	Observação
2	3 1	Rotação de 90° horário: a coluna 0 vira a linha 0 (de baixo para cima). $(0, 0) = 1 \rightarrow (1, 0)$, $(1, 0) = 3 \rightarrow (0, 0)$, $(0, 1) = 2 \rightarrow (1, 1)$, $(1, 1) = 4 \rightarrow (0, 1)$.
2	4 2	
90 nearest		
1 2		
3 4		Rotação de 45°: o pixel central permanece 255; os vizinhos diretos recebem valor interpolado ≈ 180 por bilinear; os cantos permanecem 0.
3	0 180 0	
3	180 255 180	
45 bilinear	0 180 0	
0 0 0		
0 255 0		
0 0 0		

```
%%writefile EP02_06.py
# Código Python
```

```
Writing EP02_06.py
```

```
TestSuite("EP02_06.py").run()
```

2.12.7 EP02_07 Redimensionamento (Escala)

Nesta atividade, você deve implementar o redimensionamento de uma imagem utilizando fatores de escala. Diferente da subamostragem simples, aqui utilizaremos técnicas de interpolação para permitir tanto a ampliação quanto a redução da imagem.

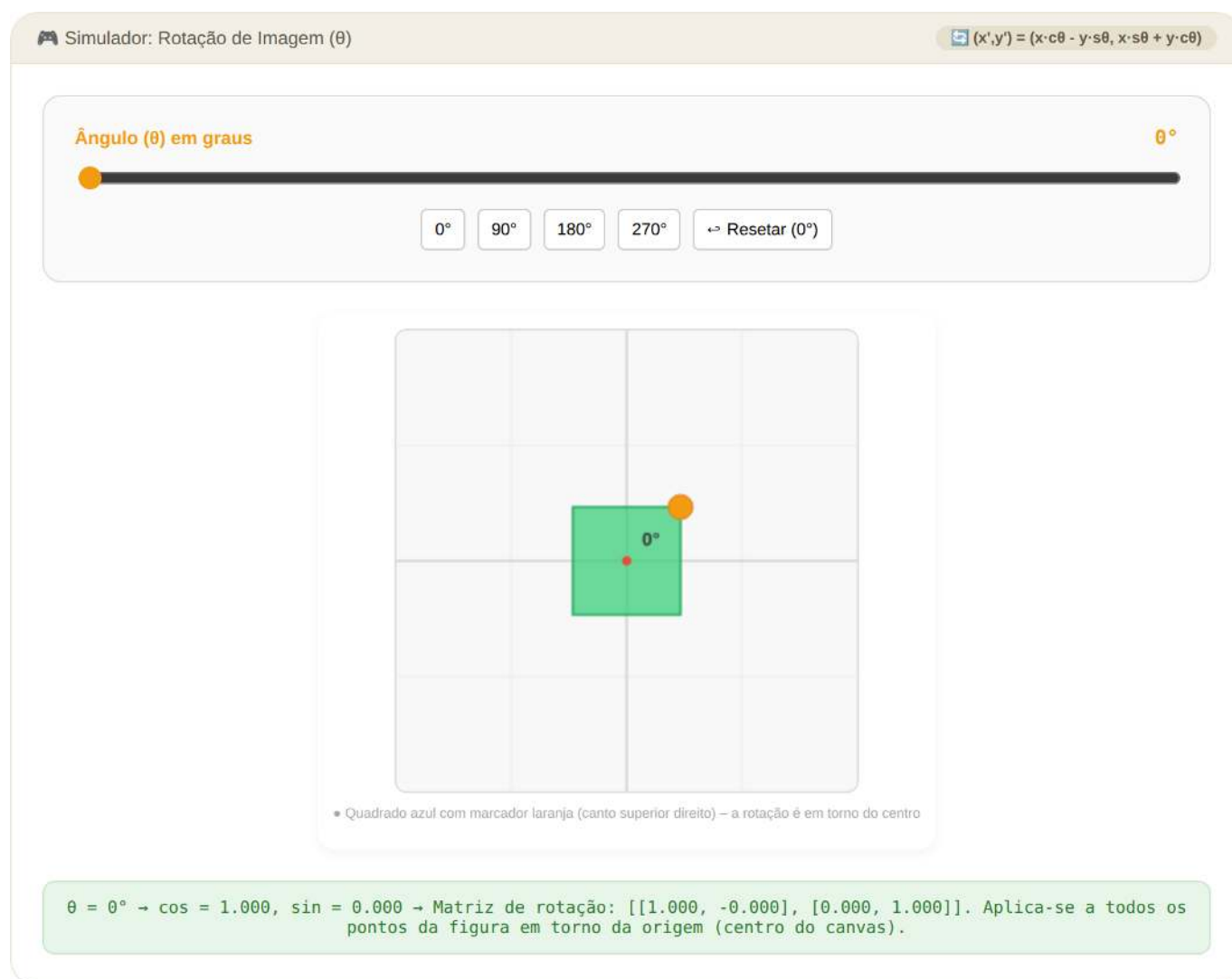


Figura 2.17: Simulador: Rotação (ângulo)

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz original.
- Leia dois valores reais s_x (escala nas linhas) e s_y (escala nas colunas).
- Leia uma *string* representando o **método** de interpolação (**nearest** ou **bilinear**).
- Leia os valores inteiros da matriz original.
- Calcule as novas dimensões: $L' = \text{round}(L \times s_x)$ e $C' = \text{round}(C \times s_y)$.
- Imprima a matriz resultante com as novas dimensões.
- Ver na Figure 2.18 uma simulação deste EP.

Importante:

- **Mapeamento Inverso:** Para cada pixel (x', y') da imagem de destino, encontre a posição correspondente na origem usando $(x, y) = (x'/s_x, y'/s_y)$.
- **Interpolação:**
 - **nearest:** Seleciona o valor do pixel mais próximo (arredondamento das coordenadas).
 - **bilinear:** Realiza uma interpolação linear dupla entre os quatro pixels vizinhos mais próximos na imagem original.
- **Bordas:** Certifique-se de que o mapeamento não tente acessar índices fora do intervalo $[0, L - 1]$ e $[0, C - 1]$.

2.12.7.1 Interpolação para Ampliação/Redução

Redimensionar uma imagem por fatores (s_x, s_y) exige o preenchimento de vazios (na ampliação) ou a fusão de informações (na redução). O método de interpolação define a qualidade visual do resultado:

Método	Funcionamento	Efeito Visual
Nearest	Pega o valor do vizinho mais próximo.	Rápido, mas gera efeito “pixelado” ou blocos.
Bilinear	Média ponderada dos 4 vizinhos (2×2) .	Suaviza a imagem, reduzindo o serrilhamento.

2.12.7.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém **L**.

A segunda linha contém **C**.

A terceira linha contém os fatores **sx** e **sy**.

A quarta linha contém o método **interp** (**nearest** ou **bilinear**).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz redimensionada com dimensões $L' \times C'$.

2.12.7.3 Exemplos

Entrada	Saída	Observação
2	1 1 2 2	Ampliação 2×: cada pixel original é replicado em um bloco 2×2. A imagem 2 × 2 vira 4 × 4.
2	1 1 2 2	
2.0 2.0	3 3 4 4	
nearest	3 3 4 4	
1 2	10	Redução 0.5×: a imagem 2 × 2 vira 1 × 1. Com nearest, o único pixel de saída amostra a posição (0, 0) = 10.
3 4		
2		
2		
0.5 0.5		
nearest		
10 20		
30 40		

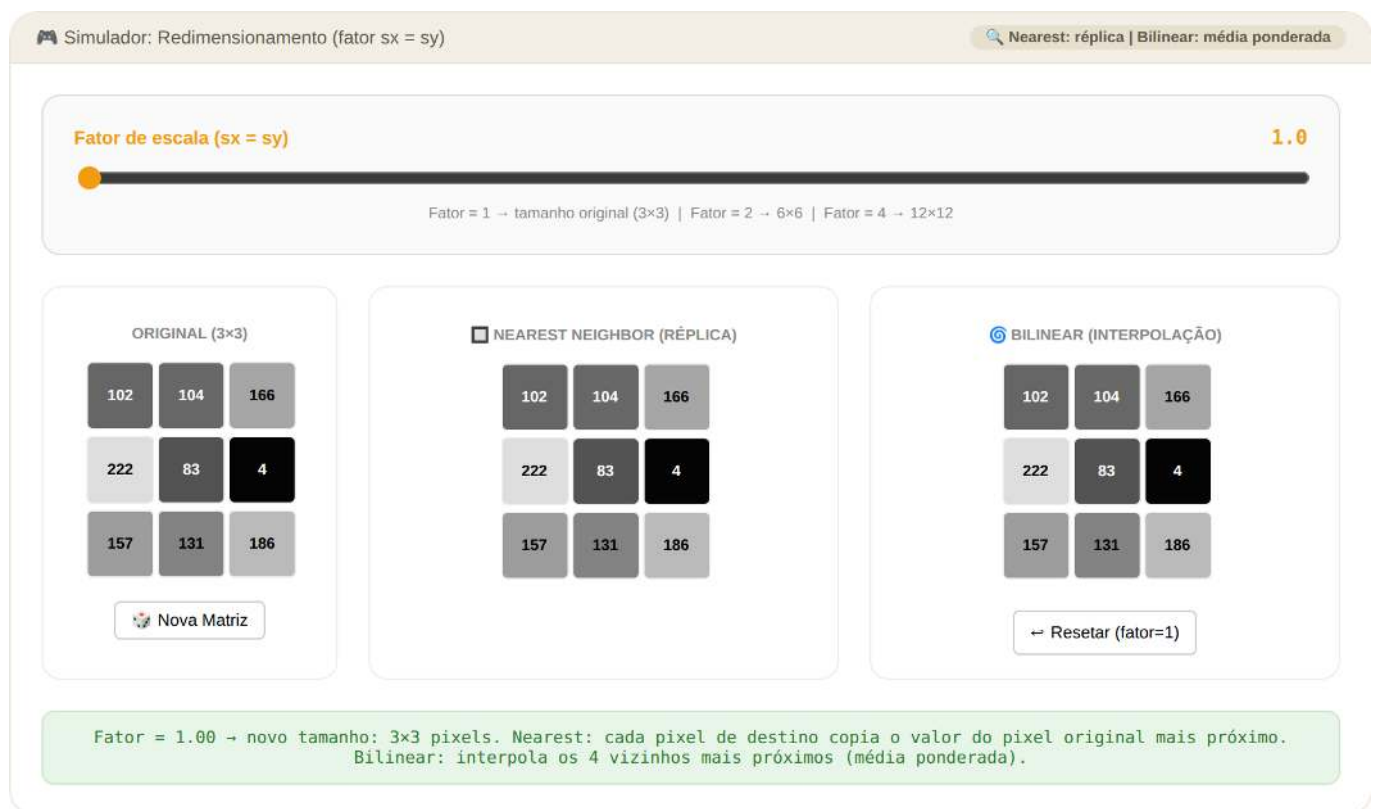


Figura 2.18: Simulador: Redimensionamento (Nearest vs Bilinear)

```
%%writefile EP02_07.py
# Código Python
import numpy as np
from morph import mm

# 1. Leitura das dimensões, fatores e método
l = int(input())
c = int(input())
sx, sy = map(float, input().split())
interp = input().strip()

# 2. Leitura da imagem original
img = mm.readImg(l, c)

# 3. Novas dimensões
```

```

l_new = round(l * sx)
c_new = round(c * sy)

# 4. Redimensionamento usando mm.resize
# cv2.resize usa (largura, altura) = (colunas, linhas)
resultado = mm.resize(img, (c_new, l_new), method=interp)

# 5. Exibição
print(mm.drawImg(resultado))

```

Writing EP02_07.py

TestSuite("EP02_07.py").run()

2.12.8 EP02_08 ✂ Cisalhamento (Shear)

Nesta atividade, você deve implementar a transformação de cisalhamento em uma imagem. O cisalhamento é uma transformação afim que desloca cada ponto em uma direção fixada, por um valor proporcional à sua distância de uma reta paralela a essa direção, resultando em um efeito de inclinação.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz.
- Leia dois valores reais sh_x (cisalhamento horizontal) e sh_y (cisalhamento vertical).
- Leia uma *string* representando o **método** de interpolação (**nearest** ou **bilinear**).
- Leia os valores inteiros da matriz original.
- Aplique a transformação mantendo o tamanho original da imagem (cortando o que ultrapassar os limites).
- Imprima a matriz resultante com as dimensões $L \times C$.
- Ver na Figure 2.19 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para cada pixel (x', y') da imagem de destino, calcule a posição correspondente na origem (x, y) utilizando a matriz de cisalhamento inversa.
- **Preenchimento:** Coordenadas que resultarem em posições fora da matriz original devem ser preenchidas com 0.
- **Coordenadas:** Para fins desta implementação, considere x como o índice da linha e y como o índice da coluna.

2.12.8.1 🧠 Distorção Afim

O cisalhamento altera a geometria da imagem inclinando seus eixos. A relação entre as coordenadas originais (x, y) e as transformadas (x', y') é dada por:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Isso resulta nas equações:

- $x' = x + sh_x \cdot y$
- $y' = y + sh_y \cdot x$

2.12.8.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém os fatores `shx` e `shy`.

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com as mesmas dimensões $L \times C$.

2.12.8.3 Exemplos

Entrada	Saída	Observação
3	10 20 30	Cisalhamento horizontal: linha i desloca $\lfloor i \cdot 0.5 \rfloor$ pixels. Linha $0 \rightarrow 0\text{px}$, linha $1 \rightarrow 0\text{px}$, linha $2 \rightarrow 1\text{px}$. Os pixels deslocados para fora são descartados e as posições vazias preenchidas com 0.
3	0 40 50	
0.5 0.0	0 0 70	
nearest		
10 20 30		
40 50 60		Cisalhamento vertical: coluna j desloca $\lfloor j \cdot 1.0 \rfloor$ pixels para baixo. Coluna $0 \rightarrow 0\text{px}$ (inalterada), coluna $1 \rightarrow 1\text{px}$: 20 desce para $(1, 1)$ e $(0, 1)$ fica 0.
70 80 90		
2	10 0	
2	30 20	
0.0 1.0		
nearest		
10 20		
30 40		

```
%%writefile EP02_08.py
# Código Python
```

```
Writing EP02_08.py
```

```
TestSuite("EP02_08.py").run()
```

2.12.9 EP02_09 Transformação Afim Genérica

Nesta atividade, você deve implementar uma transformação afim arbitrária em uma imagem. Esta operação é a generalização de todas as transformações lineares (escala, rotação, cisalhamento) combinadas com a translação, permitindo manipulações geométricas complexas através de uma única matriz.

- Leia dois inteiros L e C , representando as dimensões da matriz.
- Leia **seis valores reais** (a, b, t_x, c, d, t_y) que compõem a matriz de transformação afim 2×3 .
- Leia uma *string* representando o **método** de interpolação (`nearest` ou `bilinear`).
- Leia os valores inteiros da matriz original.
- Aplique a transformação mantendo o tamanho original $L \times C$.

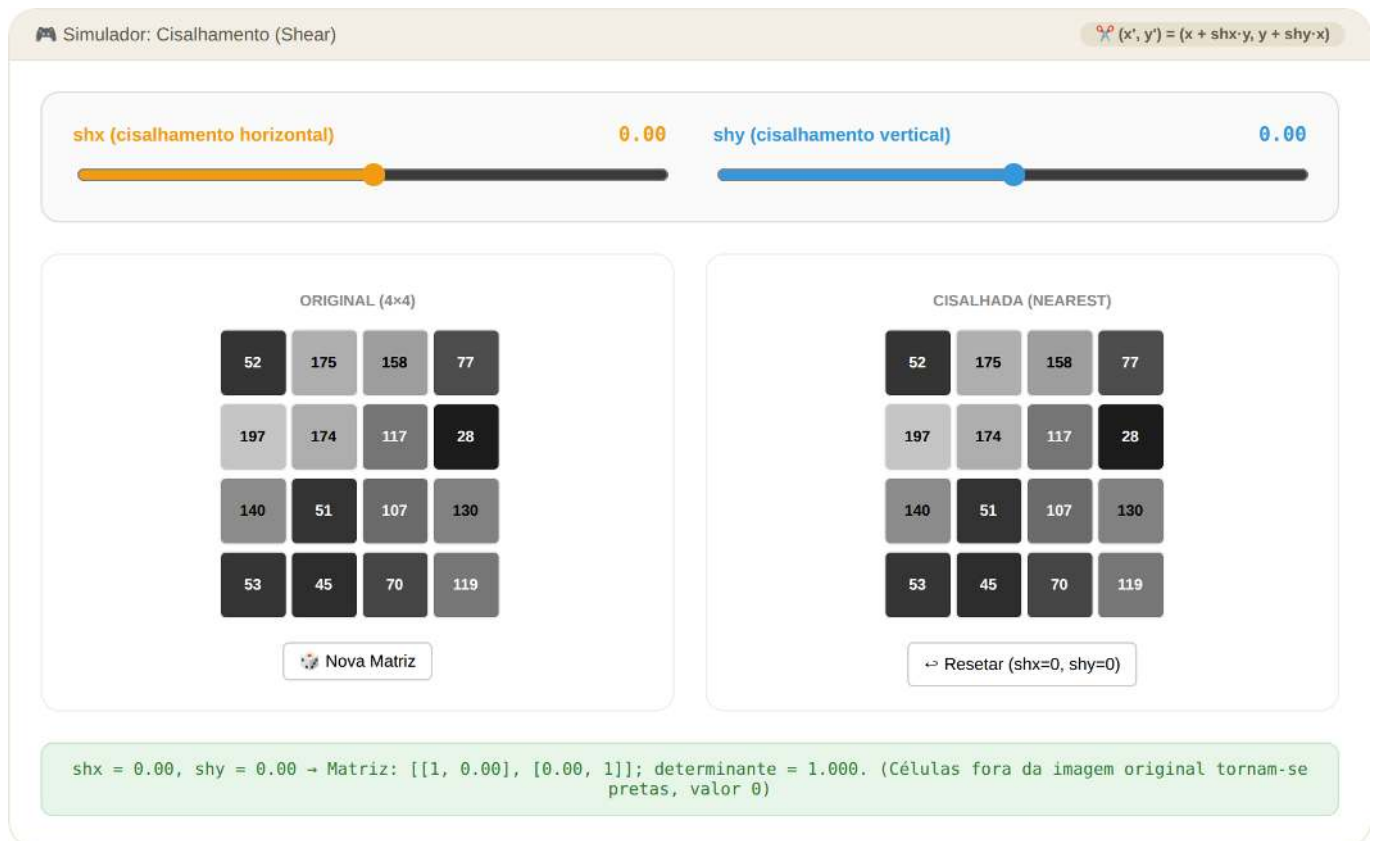


Figura 2.19: Simulador: Cisalhamento (shx, shy)

- Imprima a matriz resultante.
- Ver na Figure 2.20 uma simulação deste EP.

📌 Importante:

- **Mapeamento Inverso:** Para calcular o valor de cada pixel na imagem de destino, você deve utilizar a **inversa** da matriz de transformação afim fornecida para encontrar a coordenada correspondente na imagem original.
- **Preenchimento:** Coordenadas calculadas que caiam fora dos limites $[0, L - 1]$ e $[0, C - 1]$ da imagem original devem resultar em um pixel de valor **0**.
- **Flexibilidade:** Esta implementação deve ser capaz de realizar qualquer uma das tarefas anteriores (translação, rotação, etc.) bastando alterar os parâmetros da matriz.

Dica:

```
flags = cv2.INTER_NEAREST if interp == 'nearest' else \
        cv2.INTER_CUBIC   if interp == 'bicubic'   else \
        cv2.INTER_LANCZOS4 if interp == 'lanczos'  else \
        cv2.INTER_LINEAR

r = cv2.warpAffine(img, M, (C, L), flags=flags)
```

2.12.9.1 🧠 Combinação de Operações

A transformação afim preserva pontos, retas e planos. No processamento de imagens, ela mapeia a posição (x, y) para (x', y') seguindo o sistema:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

Ou, de forma compacta em coordenadas homogêneas:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2.12.9.2 Tarefa (especificação para VPL)

Entrada:

A primeira linha contém L .

A segunda linha contém C .

A terceira linha contém seis floats: a b t_x c d t_y .

A quarta linha contém o método `interp` (`nearest` ou `bilinear`).

As linhas seguintes contêm os elementos da matriz $L \times C$.

Saída:

A matriz transformada com as dimensões originais $L \times C$.

2.12.9.3 Exemplos

Entrada	Saída	Observação
2	15 20	Translação fracionária ($t_x = 0.5, t_y = 0.5$): cada pixel de saída (i, j) amostra a posição $(i + 0.5, j + 0.5)$ da entrada via bilinear. Ex: $(0, 0)$ interpola os quatro vizinhos $\rightarrow 15$.
2	25 30	
1.0 0.0 0.5 0.0 1.0 0.5		
bilinear		
10 20		Escala $2\times$ via matriz afim ($a = 2, d = 2$): cada pixel de saída (i, j) amostra a posição $(2i, 2j)$ da entrada com nearest. Ex: $(0, 2) \rightarrow (0, 4)$ fora da imagem $\rightarrow$ nearest clipa para $(0, 2) = 3...$ aguarda confirmação da lógica de borda.
30 40		
3	1 1 2	
3	1 1 2	
2.0 0.0 0.0 0.0 2.0 0.0	4 4 5	
nearest		
1 2 3		
4 5 6		
7 8 9		

```
%%writefile EP02_09.py
# Código Python
```

```
Writing EP02_09.py
```

```
TestSuite("EP02_09.py").run()
```

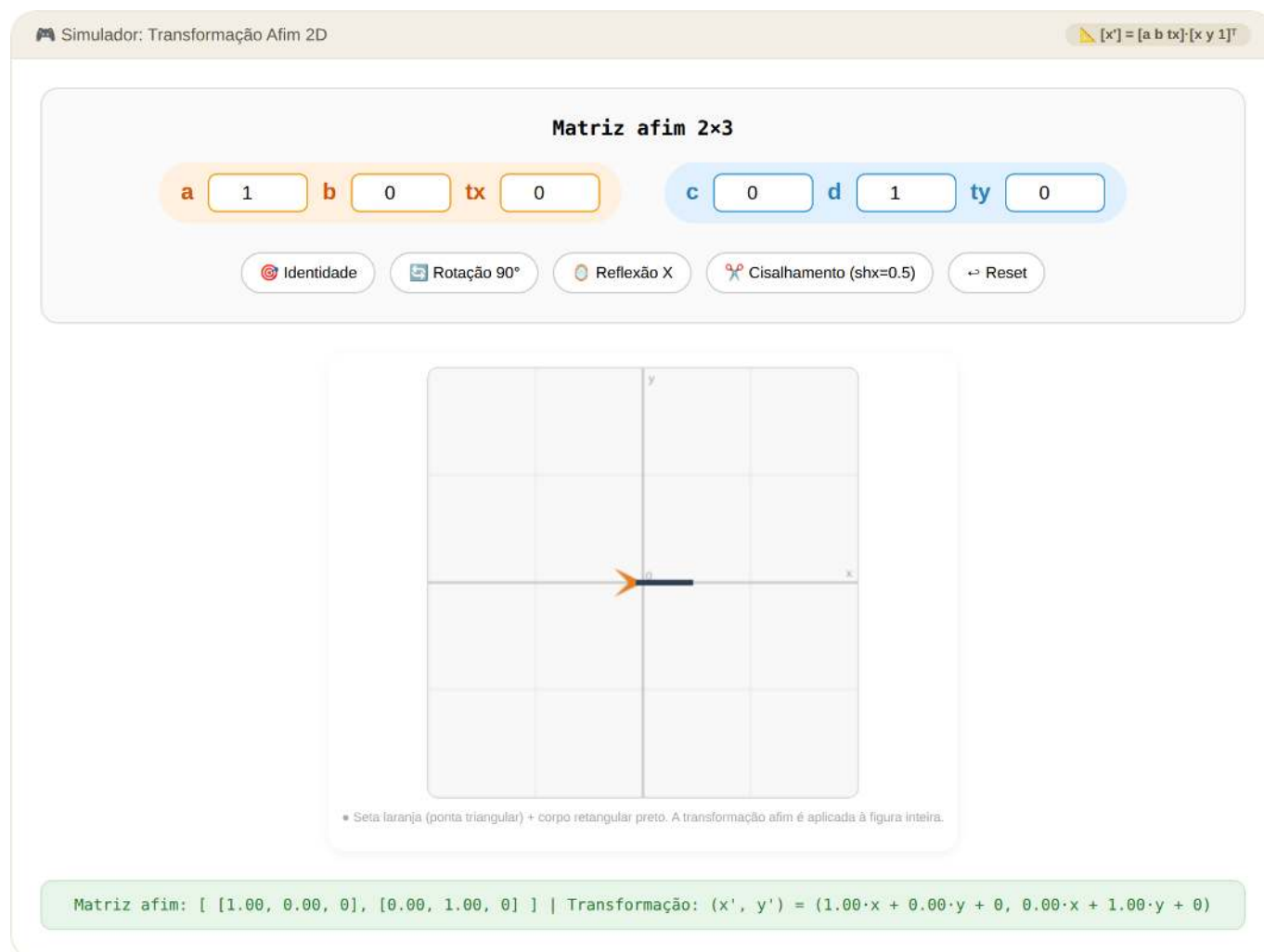


Figura 2.20: Simulador: Transformação Afim (matriz 2x3)

2.12.10 EP02_10 🎯 Correção de Perspectiva (Homografia)

Nesta atividade, você deve implementar a transformação de perspectiva, também conhecida como homografia. Diferente das transformações afins, a perspectiva não preserva o paralelismo, permitindo “retificar” objetos inclinados, como documentos ou placas capturados em ângulos oblíquos.

- Leia dois inteiros **L** e **C**, representando as dimensões da matriz original.
- Leia **quatro pares de coordenadas** (x, y) representando os cantos do quadrilátero de origem (objeto distorcido).
- Leia **quatro pares de coordenadas** (x, y) representando os cantos do quadrilátero de destino (onde o objeto deve ser mapeado).
- Leia os valores da matriz original.
- Calcule a matriz de homografia 3×3 e aplique a transformação.
- Imprima a matriz resultante com as dimensões de saída especificadas.
- Ver na Figure 2.21 uma simulação deste EP.

📌 Importante:

- **Graus de Liberdade:** A homografia possui 8 graus de liberdade (o nono elemento da matriz 3×3 é uma constante de normalização, geralmente 1), exigindo no mínimo 4 pontos correspondentes para ser calculada.
- **Projeção:** Após multiplicar as coordenadas pela matriz, é necessário dividir os resultados x' e y' pela componente homogênea w para retornar ao plano 2D.
- **Uso de Bibliotecas:** Para esta tarefa, você pode utilizar as funções `cv2.getPerspectiveTransform` para obter a matriz e `cv2.warpPerspective` para aplicar a transformação, ou implementar o sistema linear e o mapeamento inverso manualmente para um desafio extra.

```
# Dimensões de saída: bounding box dos pontos destino + 1
w = int(max(pts2[:, 0])) + 1; h = int(max(pts2[:, 1])) + 1
# M = cv2.getPerspectiveTransform(pts1, pts2)
# dst = cv2.warpPerspective(img, M, (w, h))
# ou
dst = mm.perspective_transform(img, pts1, pts2, size=(w, h))
```

2.12.10.1 🧠 Deformação não-afim

Enquanto transformações afins mapeiam paralelogramos em paralelogramos, a homografia mapeia qualquer quadrilátero em outro quadrilátero. Isso é essencial para visão computacional:

Operação	Característica	Aplicação Típica
Homografia	Projeção em plano	Correção de documentos, escaneamento de placas.
Ponto de Fuga	Convergência de linhas	Reconstrução 3D a partir de imagens 2D.
Warping	Deformação de malha	Estabilização de vídeo e panoramas (stitching).

2.12.10.2 📌 Exemplos

Entrada	Saída	Observação
4 4	10 20 30 40	As 4 primeiras linhas após as dimensões são os pontos de origem; as 4 seguintes são os destinos. Com pontos idênticos, a transformação de perspectiva é a identidade e a imagem é preservada.
0 0	50 60 70 80	
3 0	90 100 110 120	
0 3	130 140 150 160	
3 3		
0 0		
3 0		
0 3		
3 3		
10 20 30 40		
50 60 70 80		
90 100 110 120		
130 140 150 160		

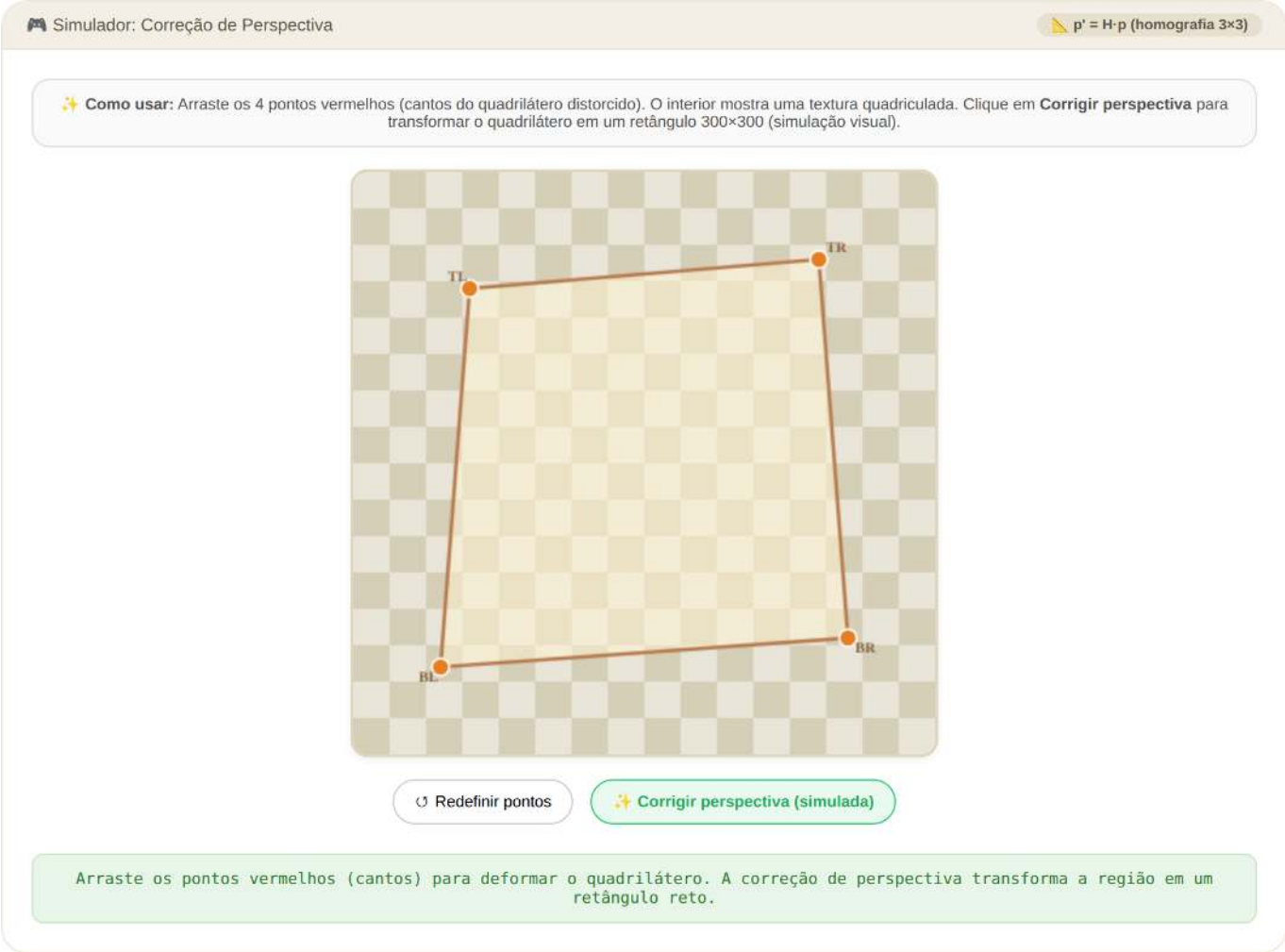


Figura 2.21: Simulador: Correção de Perspectiva (Homografia)

```
%%writefile EP02_10.py
# Código Python

Writing EP02_10.py

TestSuite("EP02_10.py").run()
```


2.12.11 EP02_11 🏆 Correção de Perspectiva (Homografia) em Imagem Real

Nesta atividade, o objetivo é aplicar a transformação de perspectiva (homografia) para “retificar” um objeto inclinado em uma fotografia real. Você lidará com a imagem de um jornal, onde a grade de um jogo de Sudoku está distorcida devido ao ângulo em que a foto foi capturada.

Seu programa deve ler os parâmetros de entrada do terminal, carregar a imagem, calcular a matriz de homografia 3×3 , aplicar a transformação geométrica e exibir um indicador global de validação.

- Leia dois inteiros **L** e **C**, representando as dimensões de linhas e colunas (altura e largura) que a imagem retificada de saída deve ter.
- Leia **quatro pares de coordenadas** (x, y) via terminal, representando os quatro cantos do quadrilátero de origem (o Sudoku distorcido na imagem original).
- Calcule automaticamente os **quatro pares de coordenadas de destino** utilizando as dimensões L e C fornecidas, mapeando os cantos para as extremidades da nova imagem: $(0, 0)$, $(C - 1, 0)$, $(0, L - 1)$ e $(C - 1, L - 1)$.
- Carregue a imagem local **sudoku.png** e converta-a para tons de cinza (grayscale).
- Calcule a matriz de homografia e aplique a transformação espacial na imagem.
- **Saída:** Calcule e imprima a **soma de todos os pixels** da imagem resultante.

📌 Importante:

- **Arquivo de entrada:** A imagem **sudoku.png** deve estar na mesma pasta do script. O programa deve lê-la diretamente do disco (ex: usando `mm.read("sudoku.png")` ou `cv2.imread`).
- **Ordem dos Pontos:** Garanta que a leitura dos 4 pontos de origem e a geração dos 4 pontos de destino sigam rigorosamente a mesma ordem dos cantos: Superior-Esquerdo (TL), Superior-Direito (TR), Inferior-Esquerdo (BL) e Inferior-Direito (BR).
- **Dimensões no OpenCV:** Lembre-se que funções como `cv2.warpPerspective` esperam o tamanho da imagem de saída no formato `(largura, altura)`, o que equivale a `(C, L)`.
- **Interpolação:** Para garantir a consistência matemática da soma dos pixels com o corretor automático, utilize a interpolação bilinear padrão (`flags=cv2.INTER_LINEAR`).
- **Créditos:** A imagem utilizada é “Sudoku en periódico” de Héctor Rodríguez, sob licença **CC BY 2.0**.

2.12.11.1 🧠 Contexto do Problema

A homografia possui 8 graus de liberdade, exigindo no mínimo 4 correspondências de pontos para ser calculada. Ao contrário de transformações afins, ela mapeia qualquer quadrilátero em outro quadrilátero, permitindo que linhas que convergem para pontos de fuga voltem a ser paralelas:

Operação	Característica	Aplicação Típica
Homografia	Projeção entre planos	Retificação de documentos, escaneamento de placas e QR Codes.
Mapeamento Inverso	Varredura do destino para a origem	Evita “buracos” ou pixels vazios na imagem final retificada.
Warping	Reamostragem espacial	Correção de distorção de lentes e montagem de panoramas (<i>stitching</i>).

2.12.11.2 📌 Exemplos

Entrada	Saída	Observação
500 500 100 120 420 95 80 440 450 460	32982820	As duas primeiras entradas são as dimensões de saída (L e C). As 4 linhas seguintes são as coordenadas (x, y) dos cantos do Sudoku na imagem original + PAD. A saída é a soma total dos pixels da imagem retificada.
200 200 100 120 420 95 80 440 450 460	5277150	

2.12.11.3 Aquisição da imagem do sudoku e conversão para níveis de cinza

A Figure 2.22 mostra a leitura da imagem original seguida da conversão para tons de cinza e redimensionamento para uma matriz de 500×500 pixels, preparando os dados para a etapa seguinte. A correção de perspectiva, aplicada na Figure 2.23 via matriz de homografia, elimina as deformações causadas pelo ângulo da câmera e produz uma visão frontal e regular da grade do Sudoku.



Figura 2.22: Aquisição da imagem de um Sudoku à esquerda. À direita, conversão para tons de cinza e redimensionamento. Crédito: Héctor Rodríguez de Guardamar, Espanha (CC BY 2.0).

```
import cv2
import numpy as np

# --- 1. Carrega a imagem salva (sudoku.png) ---
img = mm.read("sudoku.png")          # BGR, 500x500

# --- 2. Padding para não cortar vértices ---
```

```

PAD = 60
img_pad = cv2.copyMakeBorder(
    img, PAD, PAD, PAD, PAD,
    cv2.BORDER_CONSTANT, value=[255, 255, 255]
)

# --- 3. Pontos de origem (cantos da grade na imagem expandida) ---
pts1 = np.float32([
    [100, 160], # TL
    [390, 45],  # TR
    [200, 580], # BL
    [570, 420], # BR
])
#      W      H

# --- 4. Pontos de destino (vista frontal 500x500) ---
SIZE = 500
pts2 = np.float32([
    [0, 0],
    [SIZE, 0],
    [0, SIZE],
    [SIZE, SIZE],
])

# --- 5. Homografia e retificação ---
img_rect = mm.perspective_transform(img_pad, pts1, pts2, size=(SIZE, SIZE))

# --- 6. Exibição ---
mm.show(
    [img_pad, img_rect],
    titles=["Original (com padding)", "Vista frontal retificada"],
    cols=2, figsize=(10, 6), axis=True
)

```

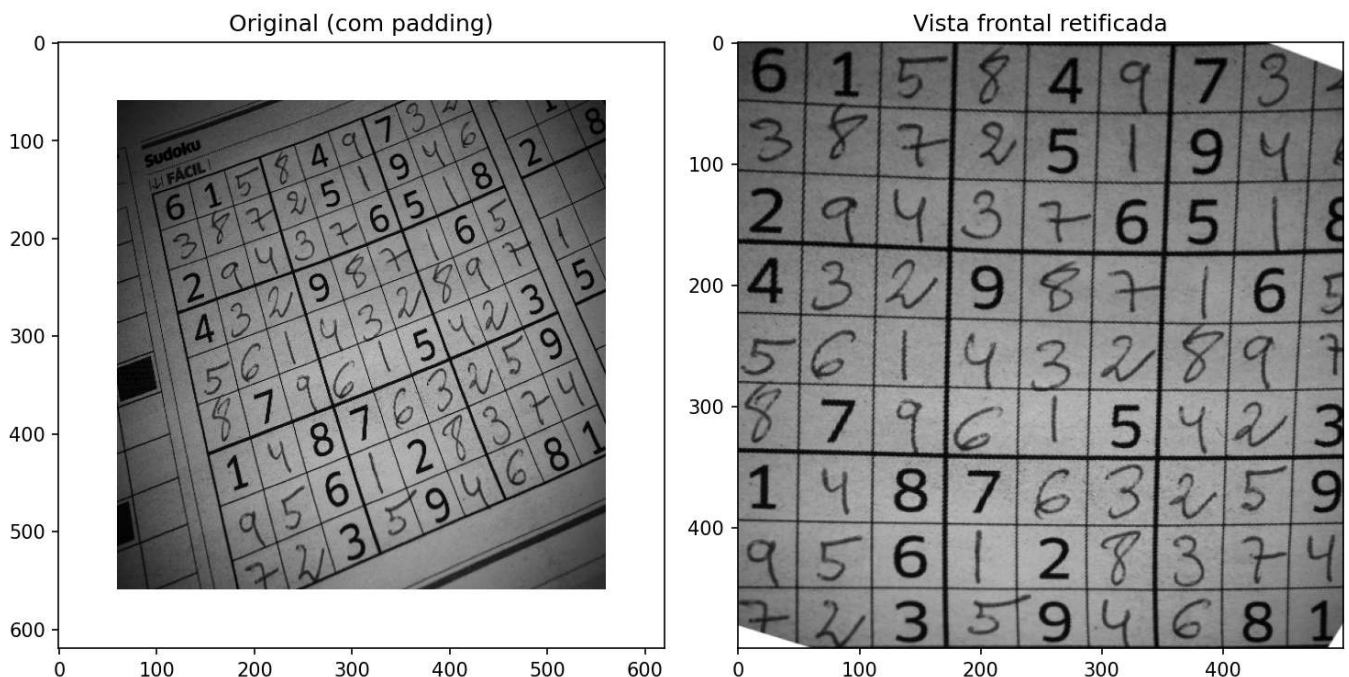


Figura 2.23: Correção de perspectiva: original e vista frontal retificada.

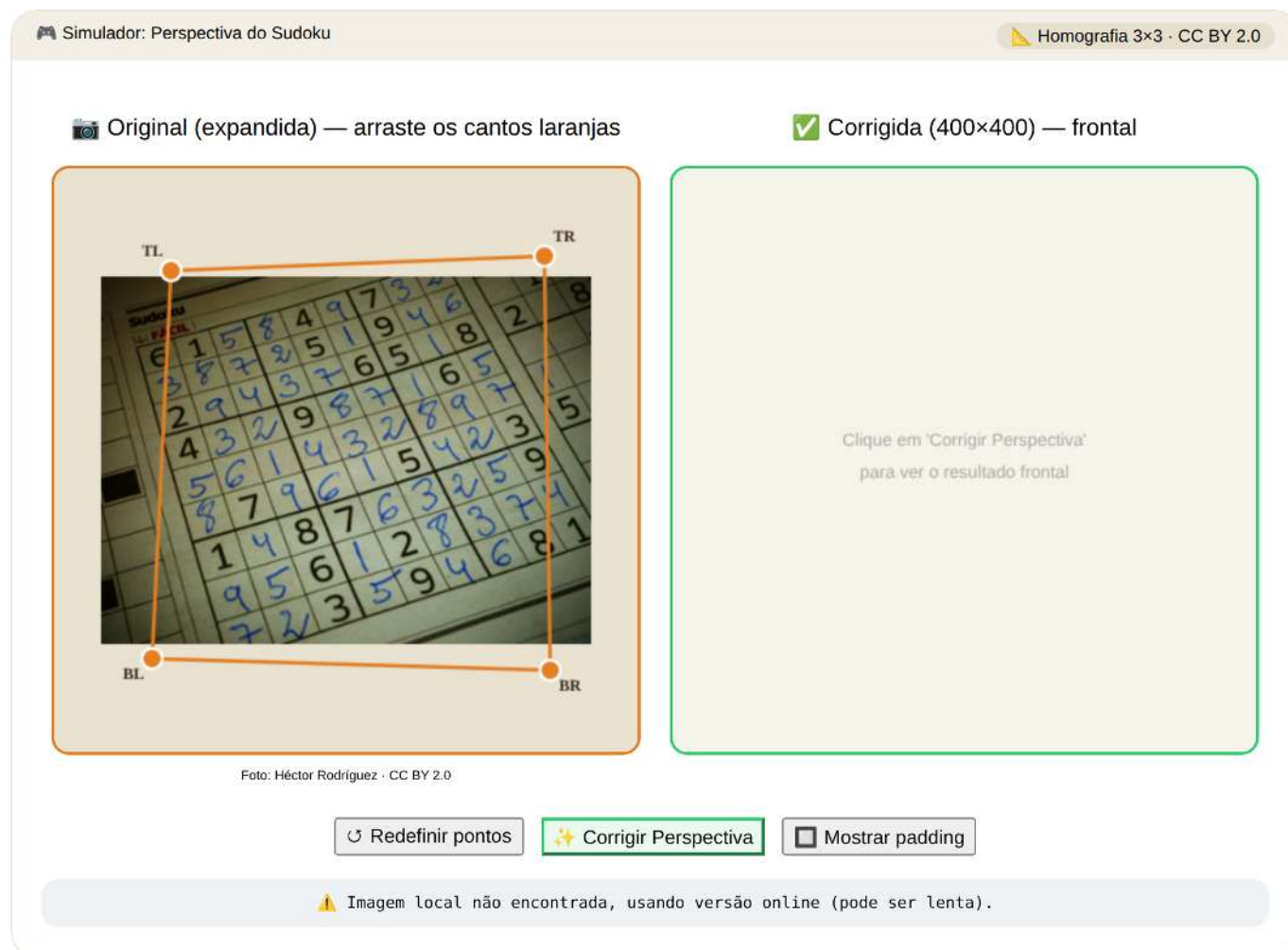


Figura 2.24: Simulador: correção de perspectiva do Sudoku.

```
%%writefile EP02_11.py  
# Código Python
```

```
Writing EP02_11.py
```

```
TestSuite("EP02_11.py").run()
```


Capítulo 3

Operações Espaciais: Intensidade, Histograma e Filtragem



Este capítulo aprofunda o processamento de imagens no domínio espacial, partindo da manipulação direta de pixels e histogramas para o realce de contraste, até a aplicação de filtros locais por convolução para suavização, redução de ruído e detecção de bordas. O objetivo é desenvolver a intuição matemática e computacional que sustenta grande parte dos algoritmos modernos de Visão Computacional.

3.1 Objetivos

Ao final deste capítulo, você será capaz de:

- **Manipular intensidade e pixels:** Executar operações aritméticas saturadas (`mm.addm`, `mm.subm`) e lógicas bit a bit (`mm.band`, `mm.bor`, `mm.bnot`) para combinação e seleção de regiões de interesse (ROI), e aplicar *alpha blending* (`mm.blend`) para fusão ponderada de imagens;
- **Processar histogramas:** Interpretar o histograma como diagnóstico tonal, aplicar equalização global (`mm.equalize`) e adaptativa (CLAHE), e realizar especificação de histograma para transferência de perfil tonal entre imagens;
- **Compreender fundamentos espaciais:** Entender vizinhança, *padding* de borda, e a diferença entre correlação cruzada (`mm.conv`) e convolução — incluindo por que kernels assimétricos como Sobel produzem resultados distintos nas duas operações;
- **Aplicar filtragem de suavização:** Utilizar o filtro de média (`mm.conv` com kernel uniforme) e o filtro Gaussiano (`cv2.GaussianBlur`) para redução de ruído, compreendendo a vantagem da ponderação radial e da separabilidade Gaussiana;
- **Aplicar filtragem de realce:** Usar o Laplaciano (w_4 e w_8) para realce isotrópico de bordas, o operador de Sobel para gradiente direcional (G_x , G_y , magnitude e direção), e o *Unsharp Masking* para amplificação controlada de alta frequência pelo parâmetro k ;
- **Utilizar filtros de ordem:** Aplicar o filtro da mediana (`cv2.medianBlur`) para remoção eficaz de ruído sal e pimenta, compreendendo por que sua natureza não linear e robustez a outliers o tornam superior aos filtros lineares nesse cenário;
- **Resolver problemas práticos:** Encadear técnicas em pipelines de pré-processamento (ex.: CLAHE $\rightarrow$ Gaussiano $\rightarrow$ Canny) e utilizar as funções da biblioteca `morph.py` (`mm.conv`, `mm.histImg`, `mm.equalize`, `mm.drawImageKernel`) para análise e visualização didática de cada etapa.

3.2 Operações em Nível de Intensidade

O nível mais elementar de processamento de imagens atua diretamente sobre os valores dos pixels, sem considerar vizinhança. Essas operações — chamadas de **transformações de ponto** (*point operations*) — são as mais rápidas computacionalmente e formam a base para técnicas mais complexas.

Formalmente, uma transformação de ponto pode ser descrita como:

$$g(x, y) = T[f(x, y)] \quad (3.1)$$

onde $f(x, y)$ é a imagem de entrada, $g(x, y)$ é a saída e T é uma função aplicada a cada pixel individualmente.

3.2.1 Preparando o Ambiente Prático

O bloco a seguir carrega o módulo `morph.py` do repositório.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão:{version}).")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.5).

Como objeto de estudo ao longo deste capítulo, utilizaremos as imagens de vida selvagem apresentadas nas Figure 3.1 e Figure 3.3. A partir delas, exploraremos operações espaciais sobre intensidade, histogramas e filtragem, analisando seus efeitos no realce, na suavização, na redução de ruído e na detecção de bordas, de modo a compreender os fundamentos matemáticos e computacionais do PDI.

```
import os

# Fonte : https://commons.wikimedia.org/wiki/File:Carlos_Guilherme_Rodrigues_(76515283).jpeg
# Mapa : https://www.google.com/maps?q=-26.204734,28.226624

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = "Carlos_Guilherme_Rodrigues_%2876515283%29.jpeg"
url = f"{base}/9/9b/{arquivo}"
caminho = "imagens/mandrill-exif.jpg"

# 1. Leitura com cache local
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_color = np.array(img_obj)
img_gray = mm.gray(img_color)

# 2. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Tipo NumPy: {type(img_color)} | Dimensões [y,x,c]: {img_color.shape}")

# 3. Exibição
mm.show(img_color, scale=90)
```

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (960, 540)
 Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (540, 960, 3)



Figura 3.1: *Mandrill* (*Mandrillus sphinx*) fotografado em ambiente natural na África do Sul. A imagem possui resolução de 500 px e contém metadados EXIF com coordenadas GPS aproximadas (-26.20473, 28.22662). Crédito: Carlos Guilherme Rodrigues (CC BY-SA 3.0).

3.2.2 Operações Aritméticas

Operações aritméticas entre imagens são amplamente usadas em PDI para combinar, comparar ou realçar informações. A **subtração de imagens** é especialmente poderosa para detectar diferenças entre dois quadros — por exemplo, na remoção de fundo estático em câmeras de vigilância:

$$g(x, y) = f_1(x, y) - f_2(x, y) \quad (3.2)$$

A **adição saturada** limita o resultado ao intervalo $[0, 255]$: valores acima de 255 são fixados em 255, evitando o *overflow* silencioso do tipo `uint8` (ex.: $200 + 100 = 44$ em vez de 300). A **subtração saturada** aplica o mesmo princípio pelo lado inferior: valores negativos são fixados em 0.

⚠ Saturação e *overflow*

Operações aritméticas em `uint8` sofrem *overflow* silencioso: $200 + 100 = 44$ (não 300). As funções `mm.addm` e `mm.subm` usam `cv2.add` e `cv2.subtract`, que realizam saturação automática. Para o *blending*, converta para `float32` antes de operar e aplique `np.clip(..., 0, 255).astype(np.uint8)` ao final, pois a operação envolve pesos fracionários.

A Figure 3.2 demonstra adição de uma constante (clareamento) e subtração de uma constante (escurecimento com saturação em 0).

```
fundo = 60

img_add = mm.addm(img_gray, fundo)
img_sub = mm.subm(img_gray, fundo)

mm.show(
    [img_gray, img_add, img_sub],
    titles=["Original", "addm (+60)", "subm (-60)"],
    cols=3
)
```



Figura 3.2: Operações aritméticas saturadas: adição de constante (clareamento) e subtração de constante (escurecimento com saturação em 0).

3.2.3 Mistura Ponderada (*Alpha Blending*)

A **mistura ponderada** (*alpha blending*) combina duas imagens utilizando pesos complementares α e $(1 - \alpha)$:

$$g(x, y) = \alpha f_1(x, y) + (1 - \alpha) f_2(x, y), \quad \alpha \in [0, 1] \quad (3.3)$$

Quando $\alpha = 1$, obtém-se apenas a imagem f_1 ; quando $\alpha = 0$, apenas f_2 . Valores intermediários produzem uma transição suave entre ambas, sendo amplamente utilizados em composição de imagens, sobreposição de camadas, marcas d'água e efeitos de fusão visual.

Para que a combinação produza um resultado coerente, é necessário alinhar previamente as regiões de interesse das imagens. Na Figure 3.4, utiliza-se um recorte do rosto do leopardo:

```
leo = img_gray_leo[250:-300, 100:-200]
```

e um recorte central da região facial do mandril:

```
mandrill = img_gray[100:400, 380:530]
```

As duas regiões são escolhidas de forma que olhos e estrutura facial permaneçam aproximadamente alinhados. Em seguida, o recorte do leopardo é redimensionado para coincidir com as dimensões do mandril antes da aplicação da mistura ponderada.

A operação é realizada em `float32` para evitar problemas de *overflow* durante as operações aritméticas, seguida de *clipping* e conversão final para `uint8`.

```
import os
from PIL.ExifTags import TAGS

base = "https://upload.wikimedia.org/wikipedia/commons"
arquivo = "Leopard_%28Panthera_pardus%29_portrait.jpg"
url = f"{base}/9/92/{arquivo}"
caminho = "imagens/leopardo.jpg"

# 1. Leitura com cache local
if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_numpy = np.array(img_obj)
img_gray_leo = mm.gray(img_numpy)

# 2. Extração e conversão de GPS (Tag 34853)
```

```

exif = img_obj._getexif()
if exif and (gps := exif.get(34853)):
    to_dec = lambda dms, ref: float(
        -(dms[0]+dms[1]/60+dms[2]/3600) if ref in 'SW'
        else (dms[0]+dms[1]/60+dms[2]/3600)
    )
    lat, lon = to_dec(gps[2], gps[1]), to_dec(gps[4], gps[3])
    print(f"GPS Decimal: {lat:.6f}, {lon:.6f}")
    print(f"Maps: https://www.google.com/maps/search/?api=1&query={lat},{lon}")

# 3. Diagnóstico de tipos, dimensões e acesso a pixels
print(f"\nTipo PIL : {type(img_obj)} | Dimensões (x,y): {img_obj.size}")
print(f"Pillow (0,0): {img_obj.getpixel((0, 0))}")
print(f"Tipo NumPy: {type(img_numpy)} | Dimensões [y,x,c]: {img_numpy.shape}")
print(f"NumPy [0,0]: {img_numpy[0, 0]}")

# 4. Exibição
mm.show(img_numpy)

```

```

GPS Decimal: -19.140200, 23.814872
Maps: https://www.google.com/maps/search/?api=1&query=-19.1402,23.814872222222224

Tipo PIL : <class 'PIL.JpegImagePlugin.JpegImageFile'> | Dimensões (x,y): (1242, 1324)
Pillow (0,0): (192, 134, 86)
Tipo NumPy: <class 'numpy.ndarray'> | Dimensões [y,x,c]: (1324, 1242, 3)
NumPy [0,0]: [192 134 86]

```



Figura 3.3: Retrato de um leopardo (*Panthera pardus*) em ambiente natural. Crédito: C. Brück (CC BY-SA 4.0).

```

leo = img_gray_leo[250:-300, 100:-200]
mandrill = img_gray[100:400, 380:530]
img_leopardo = mm.resize(leo, (mandrill.shape[1], mandrill.shape[0]), method='bilinear')

alphas = [1.0, 0.8, 0.6, 0.4, 0.2, 0.0]
imgs_blend = []

```

```
titles_blend = []

for a in alphas:
    imgs_blend.append(mm.blend(mandrill, img_leopardo, alpha=a))
    titles_blend.append(f"={a:.1f}")

mm.show(imgs_blend, titles=titles_blend, cols=6, figsize=(18, 16))
```

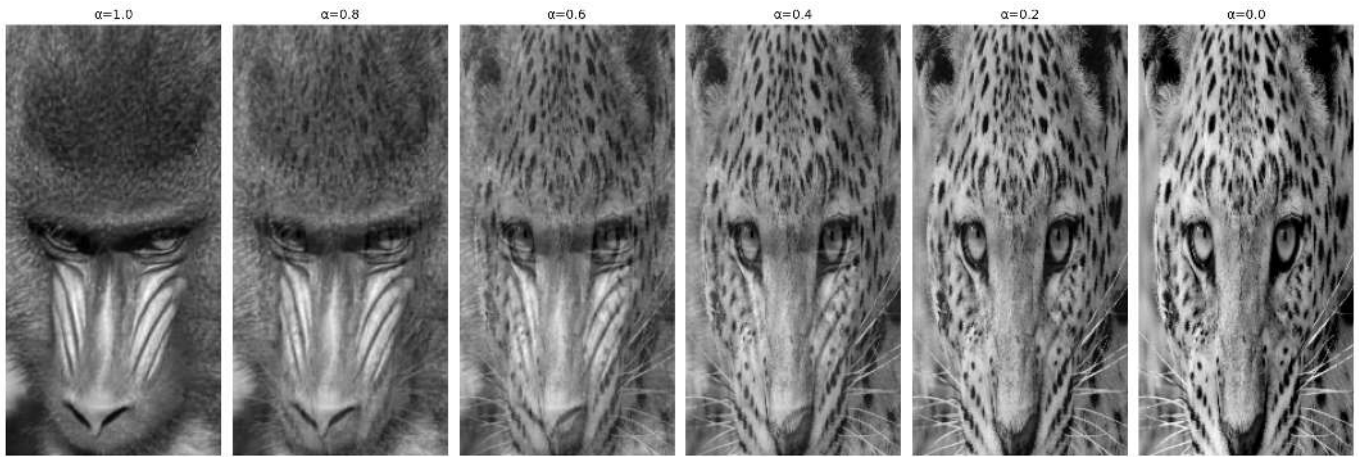


Figura 3.4: *Alpha blending* entre recortes alinhados de mandrill e do leopardo (Figure 3.3) para diferentes valores de α . Em $\alpha=1$ vê-se apenas mandrill; em $\alpha=0$, apenas o leopardo; valores intermediários fundem os olhares das duas imagens proporcionalmente.

3.2.4 Operações Lógicas e Máscaras Bit a Bit

As operações lógicas bit a bit (AND, OR e NOT) atuam diretamente sobre os bits de cada pixel e são a base para criação e aplicação de **máscaras** (*masks*) — imagens binárias com apenas 0 (preto) e 255 (branco) usadas para isolar **Regiões de Interesse (ROI)**.

O comportamento de cada operação decorre da representação binária do 255 (11111111) e do 0 (00000000):

- **AND** com a máscara: onde $m = 255$, os bits originais são preservados; onde $m = 0$, o pixel é zerado. Resultado: recorte da ROI.
- **OR** com a máscara: onde $m = 255$, o pixel é forçado a branco; onde $m = 0$, o valor original é mantido. Resultado: iluminação da ROI.
- **NOT** (sem máscara): inverte todos os bits ($g = 255 - f$), produzindo o negativo fotográfico da imagem.

A Figure 3.5 ilustra as três operações aplicadas à imagem do mandrill com uma máscara circular.

```
import cv2
h, w = img_gray.shape

# Máscara circular centrada na imagem
mask_circ = np.zeros((h, w), dtype=np.uint8)
cv2.circle(mask_circ, (w//2, h//2), min(h, w)//3 - 10, 255, -1)

# Operações via morph
img_not = mm.bnot(img_gray) # NOT: negativo fotográfico
img_and = mm.band(img_gray, mask_circ) # img_gray & mask_circ: preserva apenas a ROI circular
img_or = mm.bor(img_gray, mask_circ) # img_gray | mask_circ: ilumina a região da máscara

mm.show(
    [img_gray, img_and, img_or, img_not],
```



```
titles=["Original", "AND (ROI circular)", "OR (ilumina ROI)", "NOT (negativo)"],
cols=4
)
```

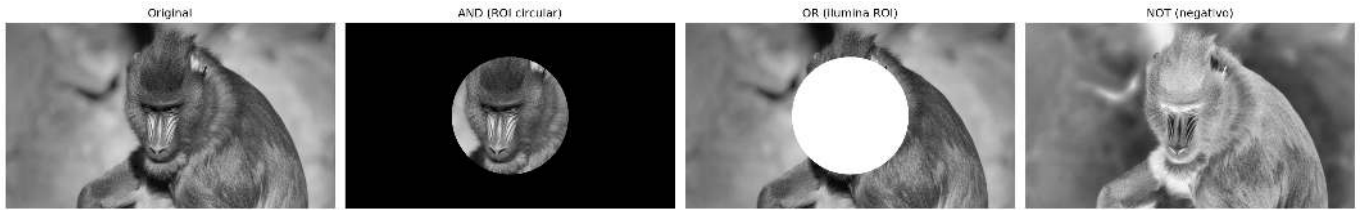


Figura 3.5: Operações lógicas bit a bit com máscara circular: AND (isolamento da ROI), OR (iluminação da ROI) e NOT (negativo).

3.3 Histograma de Imagens

O **histograma** de uma imagem em tons de cinza é uma função discreta que descreve a distribuição de frequências das intensidades:

$$h(r_k) = n_k, \quad k = 0, 1, \dots, L - 1 \quad (3.5)$$

onde r_k é o k -ésimo nível de intensidade, n_k é o número de pixels com essa intensidade e L é o total de níveis (tipicamente 256 para 8 bits). O histograma normalizado estima a probabilidade de cada nível:

$$p(r_k) = \frac{n_k}{MN} \quad (3.6)$$

onde MN é o total de pixels. Por ser uma **estatística global**, o histograma não carrega informação posicional, mas revela características essenciais como brilho médio, contraste e distribuição tonal. Na prática, `mm.hist(img)` retorna o vetor de contagens $h(r_k)$, que serve tanto para visualização (via `mm.histImg`) quanto para cálculos como **função de distribuição acumulada (CDF)** e equalização.

i Interpretação do Histograma

- **Estreito à esquerda:** imagem subexposta (escura).
- **Estreito à direita:** imagem superexposta (clara).
- **Concentrado no centro:** baixo contraste.
- **Distribuído por toda a faixa:** alto contraste, boa utilização dos tons disponíveis.

A Figure 3.6 apresenta o histograma da imagem do mandrill, bem como versões escurecida (`mm.subm`) e clareada (`mm.addm`). Observa-se o deslocamento da distribuição de intensidades para a esquerda e para a direita, respectivamente. Note que o intervalo representado no eixo x não corresponde necessariamente a toda a faixa de 0 a 255.

```
img_dark = mm.subm(img_gray, 80)
img_high = mm.addm(img_gray, 80)

images = [img_gray, img_dark, img_high]
titles = ["Original", "Escurecida (-80)", "Clareada (+80)"]

def hist_title(img, t):
    H = mm.hist(img)
    n0 = int(H[0])
    n255 = int(H[255]) if len(H) > 255 else 0
    return f"Histograma - {t}\n0:{n0:},px 255:{n255:},px"
```



```
mm.show(
    images + [mm.histImg(img) for img in images],
    titles=titles + [hist_title(img, t) for img, t in zip(images, titles)],
    rows=2, cols=3,
    figsize=(14, 8)
)
```

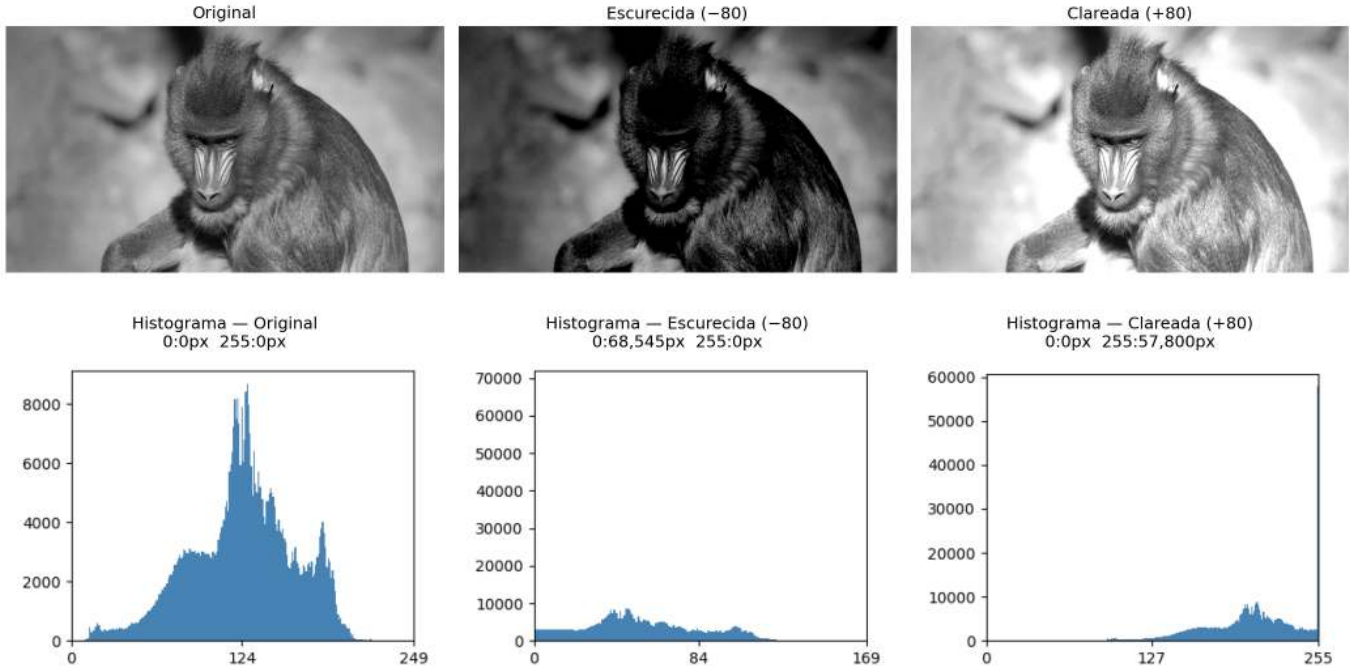


Figura 3.6: Histogramas da imagem original, de uma versão escurecida e de uma versão clareada. Os valores na segunda linha indicam a quantidade de pixels saturados em 0 (preto) e 255 (branco).

3.3.1 Equalização de Histograma

A **equalização de histograma** redistribui as intensidades para que o histograma resultante seja o mais uniforme possível. O mapeamento é dado pela **função de distribuição acumulada (CDF)**:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p(r_j) = \frac{L - 1}{MN} \sum_{j=0}^k n_j \quad (3.7)$$

A transformação é monotônica: níveis frequentes recebem intervalos maiores no domínio de saída (maior separação → mais contraste), enquanto níveis raros são comprimidos.

O algoritmo completo, em cinco etapas, é apresentado na Table 3.1.

Tabela 3.1: Algoritmo de equalização de histograma.

Etapa	Operação	Fórmula
1	Histograma	$h[k] \leftarrow$ número de pixels com intensidade k , $k = 0 \dots L - 1$
2	Probabilidade	$p[k] \leftarrow h[k]/MN$
3	CDF	$\text{cdf}[k] \leftarrow \sum_{j=0}^k p[j]$ (soma acumulada)
4	Look-Up Table (mapeamento)	$\text{lut}[k] \leftarrow \text{round}(\text{cdf}[k] \times (L - 1))$

Etapa	Operação	Fórmula
5	Aplicação	$g[i, j] \leftarrow \text{lut}[f[i, j]]$ (para todo pixel)

Note na Figure 3.7 que a equalização **redistribui** os tons existentes para posições mais espaçadas na faixa $[0, L - 1]$, mas não cria novos tons — a imagem equalizada continua com exatamente 3 tons distintos, agora em $\{1, 5, 7\}$ em vez de $\{2, 3, 4\}$.

```
img5 = np.array([[3, 4, 2, 3, 4],
                 [4, 3, 3, 4, 3],
                 [2, 3, 4, 3, 2],
                 [3, 4, 3, 2, 3],
                 [4, 3, 2, 3, 4]], dtype=np.uint8)
L = 8 # 3 bits: níveis 0..7

# 1. Histograma com tamanho garantido até L
h = mm.hist(img5, 3) # B=3 → 2³=8 níveis, tamanho garantido

p = h / h.sum() # 2. Probabilidade de cada nível p[k] = h[k] / MN

cdf = np.cumsum(p) # 3. CDF normalizada ou
# cdf = np.zeros(len(p))
# cdf[0] = p[0]
# for k in range(1, len(p)):
#     cdf[k] = cdf[k-1] + p[k] # cdf[k] = Σ p[j], j=0..k

lut = np.round(cdf * (L - 1)).astype(np.uint8) # 4. LUT: mapeamento para [0, L-1]

img5_eq = lut[img5] # 5. Aplica LUT pixel a pixel ou, equivalente a:
# l, c = img5.shape
# img5_eq = np.zeros((l, c), dtype=np.uint8)
# for i in range(l):
#     for j in range(c):
#         img5_eq[i, j] = lut[img5[i, j]] # g[i,j] = lut[f[i,j]]

print("Imagem original (5×5, 3 bits):")
print(img5)
print()
header = f"{'k':>3} {'h[k]':>6} {'p[k]':>7} {'CDF[k]':>8} {'lut[k]':>7}"
print(header)
print("-" * len(header))
for k in range(L):
    print(f"{k:>3} {h[k]:>6} {p[k]:>7.4f} {cdf[k]:>8.4f} {lut[k]:>7}")
print()
print("Imagem equalizada (5×5):")
print(img5_eq)

# Conta tons distintos
tons_orig = len(np.unique(img5))
tons_eq = len(np.unique(img5_eq))

mm.show(
    [img5, img5_eq, mm.histImg(img5, L-1), mm.histImg(img5_eq, L-1)],
    titles=[f"Original ({tons_orig} tons: {sorted(np.unique(img5).tolist())})",
           f"Equalizada ({tons_eq} tons: {sorted(np.unique(img5_eq).tolist())})",
           "Histograma - Original",
           "Histograma - Equalizada"],
    rows=2, cols=2, figsize=(5, 4), dpi=100
```

Imagem original (5×5, 3 bits):

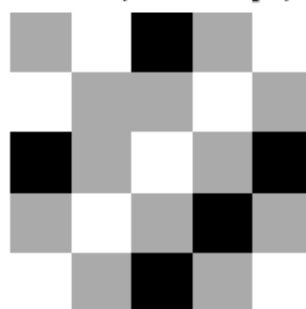
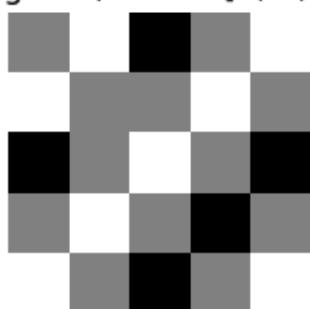
```
[[3 4 2 3 4]
 [4 3 3 4 3]
 [2 3 4 3 2]
 [3 4 3 2 3]
 [4 3 2 3 4]]
```

k	h[k]	p[k]	CDF[k]	lut[k]
0	0	0.0000	0.0000	0
1	0	0.0000	0.0000	0
2	5	0.2000	0.2000	1
3	12	0.4800	0.6800	5
4	8	0.3200	1.0000	7
5	0	0.0000	1.0000	7
6	0	0.0000	1.0000	7
7	0	0.0000	1.0000	7

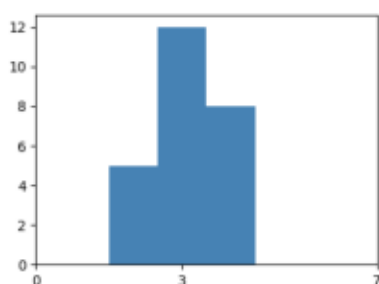
Imagem equalizada (5×5):

```
[[5 7 1 5 7]
 [7 5 5 7 5]
 [1 5 7 5 1]
 [5 7 5 1 5]
 [7 5 1 5 7]]
```

Original (3 tons: [2, 3, 4]) Equalizada (3 tons: [1, 5, 7])



Histograma — Original



Histograma — Equalizada

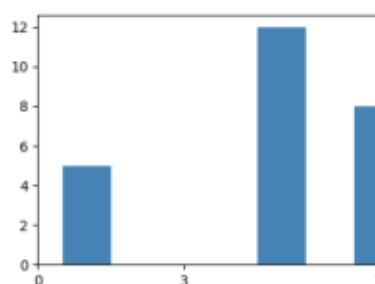


Figura 3.7: Equalização de histograma em imagem 5×5 com 3 bits ($L=8$): imagem original com tons concentrados em $\{2,3,4\}$, imagem equalizada com tons redistribuídos para $\{1,5,7\}$, e os respectivos histogramas evidenciando o espalhamento das frequências.

Limitação: a equalização global pode super-realçar ruídos e produzir contraste excessivo em regiões homogêneas. O **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*) reduz esse problema ao aplicar a equalização em blocos locais (*tiles*) e limitar a altura dos picos do histograma antes da equalização.

A Figure 3.8 compara a imagem original, a equalização global via `mm.equalize` e o CLAHE do OpenCV, exibindo também os histogramas resultantes. Diferentemente da equalização global, que utiliza uma única transformação baseada na CDF de toda a imagem, o CLAHE adapta o contraste a cada região, sendo particularmente útil em imagens com iluminação não uniforme.

No exemplo, foi utilizado `clipLimit=2.0` e `tileGridSize=(32,32)`. O parâmetro `clipLimit` define o quanto os picos do histograma local podem crescer antes de serem cortados (*clipped*). No OpenCV, esse valor é um fator relativo: o limite real é aproximadamente calculado como `clipLimit × (número de pixels do bloco / número de níveis de cinza)`. Por exemplo, em um bloco com 4096 pixels e uma imagem de 8 bits (256 níveis de cinza), a frequência média por nível é $4096/256 = 16$. Assim, `clipLimit=2.0` permite picos de aproximadamente $2 \times 16 = 32$ ocorrências antes do corte. As ocorrências excedentes não são descartadas: elas são redistribuídas entre os demais níveis de cinza do histograma, reduzindo a concentração excessiva em poucos níveis e evitando uma amplificação exagerada do contraste local. Valores menores limitam mais o contraste e reduzem a amplificação de ruído, enquanto valores maiores permitem um realce mais intenso, mas podem introduzir artefatos.

- `clipLimit=1.0`: realce suave e conservador;
- `clipLimit=2.0`: bom equilíbrio entre contraste e naturalidade;
- `clipLimit=4.0`: maior destaque para detalhes locais;
- `clipLimit=8.0`: contraste agressivo, com possível amplificação de ruído.

Assim, o CLAHE costuma produzir resultados mais naturais do que a equalização global, especialmente em imagens com sombras, reflexos ou iluminação desigual.

```
img_eq    = mm.equalize(img_gray)
clahe     = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(32, 32))
img_clahe = clahe.apply(img_gray)

images = [img_gray, img_eq, img_clahe]
titles = ["Original", "mm.equalize (CDF)", "CLAHE"]

mm.show(
    images + [mm.histImg(img) for img in images],
    titles=titles + [f"Histograma - {t}" for t in titles],
    rows=2, cols=3,
    figsize=(14, 8)
)
```

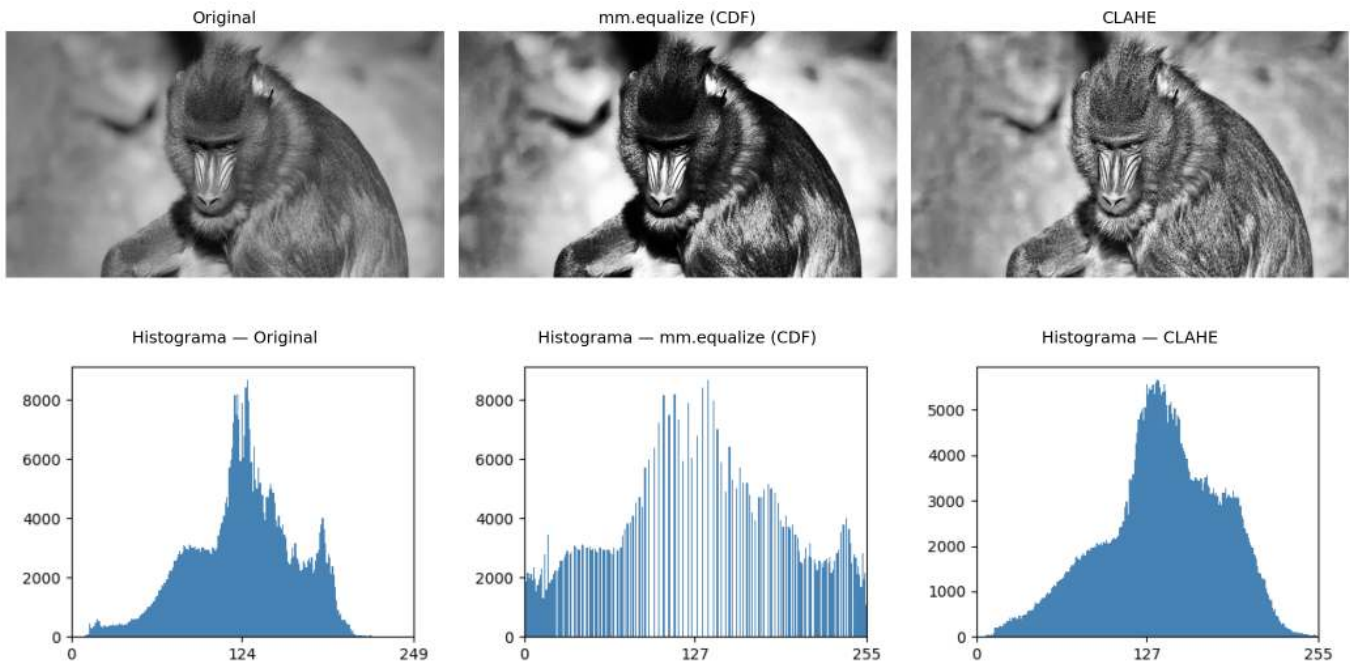


Figura 3.8: Equalização de histograma: mm.equalize (global via CDF) vs. CLAHE (adaptativa com limitação de contraste). Os histogramas revelam o espalhamento progressivo das intensidades.

3.3.2 Especificação de Histograma

Enquanto a equalização impõe uma distribuição uniforme, a **especificação de histograma** (*histogram matching*) permite que o histograma da imagem de saída siga uma distribuição **arbitrária** — por exemplo, o histograma de outra imagem de referência.

O procedimento envolve três etapas:

1. Calcular a CDF da imagem de entrada: $P_r(r_k)$.
2. Calcular a CDF da imagem de referência: $P_z(z_k)$.
3. Para cada nível r_k , encontrar o nível z que minimiza $|P_z(z) - P_r(r_k)|$.

$$T(r_k) = \arg \min_z |P_z(z) - P_r(r_k)| \quad (3.8)$$

Na Figure 3.9, transferimos o perfil tonal do leopardo (Figure 3.3) para a imagem do mandrill — uma aplicação direta do conceito visto no *blending*: em vez de fundir pixels, aqui fundimos distribuições tonais.

```
img_leop_gray = mm.gray(img_numpy)

def hist_specify(src, ref):
    """Mapeia src para o perfil tonal de ref via CDF."""
    cdf_src = np.cumsum(mm.hist(src) / src.size)
    cdf_ref = np.cumsum(mm.hist(ref) / ref.size)
    lut = np.array([np.argmin(np.abs(cdf_ref - v)) for v in cdf_src], dtype=np.uint8)
    return lut[src]

img_spec = hist_specify(img_gray, img_leop_gray)

images = [img_gray, img_leop_gray, img_spec]
titles = ["Mandrill (original)", "Leopardo (referência)", "Mandrill → perfil do leopardo"]

mm.show(
    images + [mm.histImg(img) for img in images],
```

```

titles=titles + [f"Histograma - {t}" for t in titles],
rows=2, cols=3,
figsize=(12, 9)
)

```

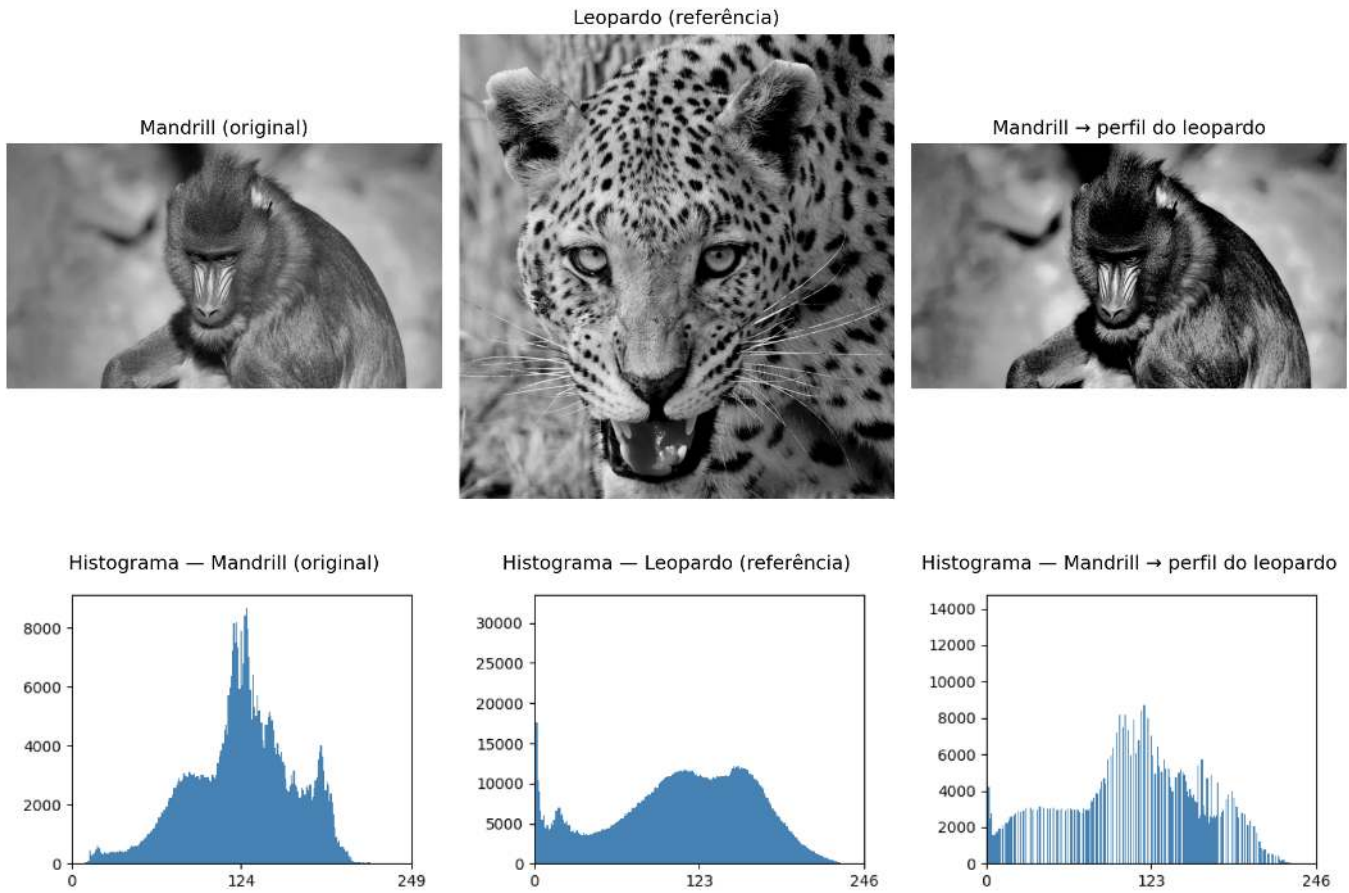


Figura 3.9: Especificação de histograma: mandril mapeada para o perfil tonal do leopardo. A CDF da saída aproxima a CDF de referência.

3.4 Fundamentos Espaciais: Vizinhança, Convolução e *Kernels*

As operações de filtragem espacial não atuam em um único pixel isolado, mas em uma **vizinhança** ao seu redor. Para isso, utiliza-se uma pequena matriz de coeficientes denominada **kernel** (ou máscara), que percorre toda a imagem por meio de uma **janela deslizante** (*sliding window*).

As janelas mais comuns são 3×3 , 5×5 e 7×7 . Em uma janela 3×3 , por exemplo, o pixel central é processado juntamente com seus oito vizinhos imediatos. Em cada posição da janela, os valores dos pixels são combinados com os coeficientes do *kernel*, produzindo um novo valor para o pixel central.

3.4.1 Vizinhança

Considere uma janela 3×3 centrada no pixel (x, y) :

$$\begin{bmatrix} (x-1, y-1) & (x, y-1) & (x+1, y-1) \\ (x-1, y) & (x, y) & (x+1, y) \\ (x-1, y+1) & (x, y+1) & (x+1, y+1) \end{bmatrix} \quad (3.9)$$

De forma geral, uma janela de tamanho $(2a+1) \times (2b+1)$ abrange todos os pixels situados até a posições na horizontal e até b posições na vertical em relação ao pixel central. Assim, uma janela 3×3 corresponde a

$a = b = 1$, uma janela 5×5 a $a = b = 2$, e assim por diante.

Matematicamente, a vizinhança é definida por

$$\mathcal{V}(x, y) = \{(x + s, y + t) : -a \leq s \leq a, -b \leq t \leq b\} \quad (3.10)$$

3.4.2 Tratamento de Bordas

Pixels próximos às bordas possuem parte de sua vizinhança fora da imagem. Para aplicar filtros nessas regiões, é necessário definir como os valores externos serão obtidos. As estratégias mais comuns são:

- **Zero-padding** (BORDER_CONSTANT): completa a região externa com zeros.
- **Replicação** (BORDER_REPLICATE): repete os valores da borda.
- **Reflexão** (BORDER_REFLECT_101): espelha os pixels vizinhos à borda.

O OpenCV utiliza, por padrão, BORDER_REFLECT_101, pois essa estratégia preserva melhor a continuidade dos níveis de cinza e reduz artefatos introduzidos pelo tratamento das bordas.

O exemplo a seguir compara os três métodos usando uma matriz 3×3 . Observe como cada estratégia preenche os pixels externos necessários para que filtros 3×3 possam ser aplicados também nos cantos da imagem.

```
import cv2
import numpy as np

img = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
], dtype=np.uint8)

bordas = {
    "CONSTANT (zero-padding)": cv2.BORDER_CONSTANT,
    "REPLICATE": cv2.BORDER_REPLICATE,
    "REFLECT_101": cv2.BORDER_REFLECT_101
}

print("Imagem original:")
print(mm.drawImg(img))

for k in [3, 5]:
    b = k // 2

    print(f"=== Kernel {k}x{k} ===")

    for nome, tipo in bordas.items():
        pad = cv2.copyMakeBorder(
            img, b, b, b, b,
            tipo,
            value=0
        )

        print(f"{nome}")
        print(mm.drawImg(pad))
```

```
Imagem original:
1 2 3
4 5 6
7 8 9

=== Kernel 3x3 ===
```

CONSTANT (zero-padding)

```
0 0 0 0 0
0 1 2 3 0
0 4 5 6 0
0 7 8 9 0
0 0 0 0 0
```

REPLICATE

```
1 1 2 3 3
1 1 2 3 3
4 4 5 6 6
7 7 8 9 9
7 7 8 9 9
```

REFLECT_101

```
5 4 5 6 5
2 1 2 3 2
5 4 5 6 5
8 7 8 9 8
5 4 5 6 5
```

=== Kernel 5x5 ===

CONSTANT (zero-padding)

```
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 1 2 3 0 0
0 0 4 5 6 0 0
0 0 7 8 9 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
```

REPLICATE

```
1 1 1 2 3 3 3
1 1 1 2 3 3 3
1 1 1 2 3 3 3
4 4 4 5 6 6 6
7 7 7 8 9 9 9
7 7 7 8 9 9 9
7 7 7 8 9 9 9
```

REFLECT_101

```
9 8 7 8 9 8 7
6 5 4 5 6 5 4
3 2 1 2 3 2 1
6 5 4 5 6 5 4
9 8 7 8 9 8 7
6 5 4 5 6 5 4
3 2 1 2 3 2 1
```

Observe que o resultado de um filtro pode variar significativamente conforme o tratamento adotado para as bordas da imagem.

Na biblioteca didática `morph.py`, os algoritmos implementados diretamente em Python (sem utilizar OpenCV ou *scikit-image*) não utilizam *padding*. Para cada pixel, em geral, os elementos da vizinhança são examinados individualmente, e apenas os vizinhos cujas coordenadas pertencem ao domínio da imagem participam do cálculo. Assim, nas regiões próximas às bordas, a vizinhança efetiva pode conter menos pixels do que a janela especificada.

3.4.3 Correlação e Convolução

Existem dois mecanismos matematicamente relacionados:

Correlação cruzada (*cross-correlation*) — o *kernel* é aplicado diretamente:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (3.11)$$

Convolução bidimensional — o *kernel* é rotacionado 180° antes da aplicação:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (3.12)$$

Para *kernels* simétricos (Gaussiano, Laplaciano, média) as duas operações produzem resultados idênticos. Para *kernels* assimétricos (Sobel, Prewitt) a diferença é significativa.

3.4.4 Correlação vs. Convolução no OpenCV

`cv2.filter2D` implementa **correlação**. Para convolução verdadeira com *kernel* assimétrico, rotacione o *kernel* 180° (`np.rot90(w, 2)`) antes de passar para `filter2D`.

3.4.4.1 Correlação (*OpenCV*: `cv2.filter2D`)

```
import cv2
import numpy as np

# imagem 4x4
img = np.array([
    [ 1,  2,  3,  4],
    [ 5,  6,  7,  8],
    [ 9, 10, 11, 12],
    [13, 14, 15, 16]
], dtype=np.float32)

# kernel assimétrico
w = np.array([
    [0, 1, 2],
    [0, 0, 0],
    [0, 0, 0]
], dtype=np.float32)

corr = cv2.filter2D(
    img, -1, w, # -1 indica que o tipo da saída é o mesmo da entrada
    borderType=cv2.BORDER_CONSTANT
)

print("Imagem original:")
print(mm.drawImg(img))

print("Kernel:")
print(mm.drawImg(w))

print("Resultado da correlação:")
print(mm.drawImg(corr))
```

Imagem original:

1	2	3	4
---	---	---	---

```

5   6   7   8
9  10  11  12
13 14  15  16

```

Kernel:

```

0  1  2
0  0  0
0  0  0

```

Resultado da correlação:

```

0   0   0   0
5   8  11   4
17  20  23   8
29  32  35  12

```

O método `cv2.filter2D` realiza correlação, isto é, aplica o *kernel* exatamente na orientação fornecida.

3.4.4.2 Convolução

```

import cv2
import numpy as np

# mesmo kernel
w_conv = np.rot90(w, 2)

conv = cv2.filter2D(
    img, -1, w_conv,
    borderType=cv2.BORDER_CONSTANT
)

print("Imagem original:")
print(mm.drawImg(img))

print("Kernel original:")
print(mm.drawImg(w))

print("Kernel rotacionado 180°:")
print(mm.drawImg(w_conv))

print("Resultado da convolução:")
print(mm.drawImg(conv))

```

```

Imagem original:
1   2   3   4
5   6   7   8
9  10  11  12
13 14  15  16

```

Kernel original:

```

0  1  2
0  0  0
0  0  0

```

Kernel rotacionado 180°:

```

0  0  0
0  0  0
2  1  0

```

Resultado da convolução:

```

5  16  19  22

```

9	28	31	34
13	40	43	46
0	0	0	0

A convolução utiliza o *kernel* rotacionado em 180°. Por isso, para reproduzir a definição matemática de convolução usando `cv2.filter2D`, é necessário aplicar previamente `np.rot90(w, 2)`.

Quando o *kernel* é simétrico (por exemplo, filtros de média ou Gaussianos), correlação e convolução produzem o mesmo resultado. A diferença aparece apenas para *kernels* assimétricos, como o exemplo apresentado.

3.4.5 O Papel do *Kernel*

Os coeficientes do *kernel* determinam completamente o efeito produzido pelo filtro, conforme resumido na Table 3.2.

Tabela 3.2: Interpretação típica dos coeficientes do *kernel*.

Característica	Efeito típico
Coeficientes positivos com soma 1	Suavização (<i>passa-baixa</i>)
Soma igual a 0, com valores positivos e negativos	Deteção de bordas (<i>passa-alta</i>)
Coeficiente central positivo dominante e vizinhos negativos	Realce de nitidez
Coeficientes assimétricos	Gradiente direcional

Exemplos:

Suavização:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Deteção de bordas:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Realce de nitidez:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Gradiente direcional (Sobel):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

A Figure 3.10 demonstra o mecanismo passo a passo: para cada posição da janela, multiplica-se cada coeficiente do *kernel* pelo pixel correspondente da vizinhança e somam-se os produtos obtidos. O resultado é exatamente o valor definido pela Equation 3.11 para aquela posição da imagem. Embora as imagens produzidas por `mm.conv0` e `cv2.filter2D` (ou `mm.conv`) sejam visualmente muito semelhantes, a implementação baseada em OpenCV é milhares de vezes mais rápida, como mostrado a seguir.

⚠ Desempenho: laços Python vs. operações vetorizadas

A função `mm.conv0` implementa a correlação diretamente em Python por meio de laços aninhados. Embora essa abordagem seja adequada para fins didáticos, ela executa um grande número de operações e torna-se lenta para imagens maiores.

Já `mm.conv` utiliza `cv2.filter2D`, implementado em C++ e otimizado para operações matriciais. No exemplo apresentado, a versão vetorizada foi mais de **3000 vezes mais rápida** que a implementação didática, produzindo um resultado visualmente equivalente.

As diferenças numéricas observadas concentram-se principalmente nas bordas da imagem. Em `mm.conv0`, os pixels da borda permanecem inalterados, enquanto `mm.conv` utiliza uma estratégia de reflexão das bordas (`cv2.BORDER_REFLECT_101`, padrão do `cv2.filter2D`).

Por isso, `mm.conv0` deve ser utilizado para compreender o algoritmo, enquanto `mm.conv` é a opção recomendada para aplicações práticas.

```
import time

w_mean = np.ones((3,3), dtype=np.float32) / 9.0
img_gray = img_leop_gray
# Demonstração numérica em patch 5x5
patch = img_gray[250:255, 250:255].astype(np.float32)
roi = patch[0:3, 0:3]
print("Patch 5x5 (intensidades):")
print(patch.astype(np.int32))
print(f"\nKernel 3x3 de média:\n{w_mean}")
print(f"\nCorrelação no pixel central [1,1]: \
      {(w_mean * roi).sum():.1f} (original: {patch[1,1]:.0f})")

# Comparação de tempo na imagem completa
t0 = time.perf_counter()
img_conv0 = mm.conv0(img_gray, w_mean)
t_conv0 = time.perf_counter() - t0

t0 = time.perf_counter()
img_conv = mm.conv(img_gray, w_mean)
t_conv = time.perf_counter() - t0

diff = np.abs(img_conv0.astype(int) - img_conv.astype(int)).max()
print(f"\nTempos na imagem {img_gray.shape[0]}x{img_gray.shape[1]}:")
print(f"  mm.conv0 (laços Python): {t_conv0:.3f} s")
print(f"  mm.conv (cv2.filter2D): {t_conv:.5f} s")
print(f"  Aceleração: {t_conv0/t_conv:.0f}x mais rápido")
print(f"  Diferença máxima: {diff} (bordas com padding diferente)")

mm.show(
    [img_gray, img_conv0, img_conv],
    titles=["Original", f"conv0 ({t_conv0:.2f}s)", f"conv ({t_conv:.4f}s)"],
    cols=3
)
```

Patch 5x5 (intensidades):

```
[[ 91  95 108 116 114]
 [106 107 108 111 114]
 [102 103 107 112 116]
 [ 83  90 111 125 123]
 [ 79  88 110 126 124]]
```

Kernel 3x3 de média:

```
[[0.11111111 0.11111111 0.11111111]]
```



```
[0.11111111 0.11111111 0.11111111]
[0.11111111 0.11111111 0.11111111]]
```

Correlação no pixel central [1,1]: 103.0 (original: 107)

Tempos na imagem 1324x1242:

mm.conv0 (laços Python): 6.950 s

mm.conv (cv2.filter2D): 0.00114 s

Aceleração: 6085× mais rápido

Diferença máxima: 39 (bordas com padding diferente)



Figura 3.10: Correlação com *kernel* de média 3×3: versão didática (mm.conv0) vs. vetorizada (mm.conv via cv2.filter2D). A diferença máxima de 39 ocorre nas bordas.

3.4.6 Exemplo Numérico: Correlação Passo a Passo

Para tornar concreto o mecanismo da Equation 3.11, considere o *kernel* de média 3×3 ($a = b = 1$, todos os coeficientes = $1/9 \approx 0,111$) aplicado ao *patch* 5×5 extraído da imagem do leopardo. A Figure 3.11 exhibe o *patch* com grade e destaca em amarelo a janela 3×3 centrada no pixel [1,1]:

```
patch = img_gray[250:255, 250:255]
B      = np.ones((3,3), dtype=np.uint8)

print("Patch 5x5 (intensidades):")
print(mm.drawImg(patch))

mm.drawImgKernel(patch, B, x=1, y=1, scale=40.0)
```

```
Patch 5x5 (intensidades):
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

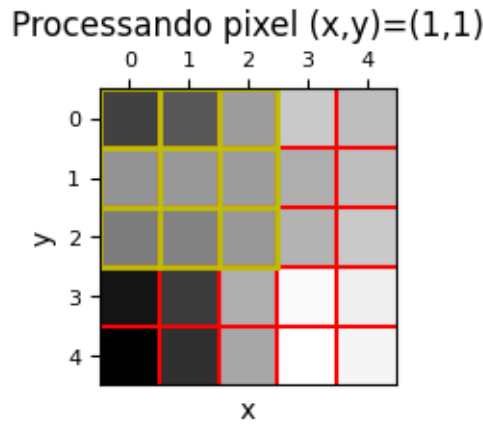


Figura 3.11: *Patch* 5×5 extraído da imagem do leopardo (posição [250:255, 250:255]). A janela amarela destaca a vizinhança 3×3 centrada no pixel [1,1] onde a correlação será calculada.

Para ilustrar o cálculo da correlação, considere o pixel na posição [1, 1] do *patch* 5×5 mostrado na Figure 3.11. Essa posição foi escolhida apenas por conveniência didática, pois possui uma vizinhança 3×3 completa ao seu redor.

Os valores dessa vizinhança correspondem à submatriz superior esquerda do *patch*:

$$\text{vizinhança} = \begin{bmatrix} 91 & 95 & 108 \\ 106 & 107 & 108 \\ 102 & 103 & 107 \end{bmatrix}$$

Aplicando a Equation 3.11 com o kernel de média:

$$g[1, 1] = \frac{91 + 95 + 108 + 106 + 107 + 108 + 102 + 103 + 107}{9} = \frac{927}{9} = 103$$

O resultado (103) é ligeiramente menor que o valor original do pixel central (107), pois a média incorpora vizinhos de menor intensidade, produzindo o efeito de suavização. Na prática, o algoritmo inicia o processamento em [0, 0] e repete esse mesmo cálculo para cada posição da imagem, deslocando a janela até cobrir todo o domínio.

3.5 Filtragem Espacial de Suavização

Os filtros de suavização (*smoothing filters*) atenuam variações bruscas de intensidade, reduzindo ruído e detalhes de alta frequência. São filtros **passa-baixa** — preservam as componentes de baixa frequência (estruturas grandes) e atenuam as de alta frequência (ruído, bordas).

3.5.1 Filtro de Média (*Box Filter*)

O filtro de média utiliza um *kernel* uniforme de tamanho $n \times n$, onde todos os coeficientes valem $1/n^2$:

$$w_{\text{média}} = \frac{1}{n^2} \begin{bmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{bmatrix}_{n \times n} \quad (3.13)$$

Cada pixel de saída é a média aritmética dos n^2 pixels de sua vizinhança. Note que a soma dos coeficientes é sempre 1 — o brilho médio da imagem é preservado. *Kernels* maiores produzem suavização mais agressiva, mas borram progressivamente as bordas.

A Figure 3.12 mostra o efeito do filtro de média com *kernels* 3×3 , 7×7 e 15×15 sobre um detalhe da imagem do leopardo. Os resultados foram obtidos com `mm.blur`, que implementa o filtro de média por meio da função `cv2.blur`, equivalente à convolução da imagem com um *kernel* uniforme cujos coeficientes são $h(x, y) = 1/N^2$; de forma equivalente, o mesmo resultado pode ser obtido com `mm.conv`, calculando ($g = f * h$). À medida que o *kernel* aumenta, mais pixels contribuem para cada valor de saída, intensificando a suavização, reduzindo o ruído e tornando detalhes finos e bordas progressivamente mais borrados.

```
# Detalhe da região do olho
y0, y1, x0, x1 = 580, 740, 680, 900
crop = lambda img: img[y0:y1, x0:x1]

img_gray_crop = crop(img_gray)

sizes = [3, 7, 15]
imgs = [img_gray_crop] + \
    [mm.blur(img_gray_crop, k) for k in sizes] # ou
    #[mm.conv(img_gray_crop, np.ones((k,k), dtype=np.float32)/(k*k)) for k in sizes]
titles = ["Original"] + [f"Média {k}x{k}" for k in sizes]

mm.show(imgs, titles=titles, cols=4)
```

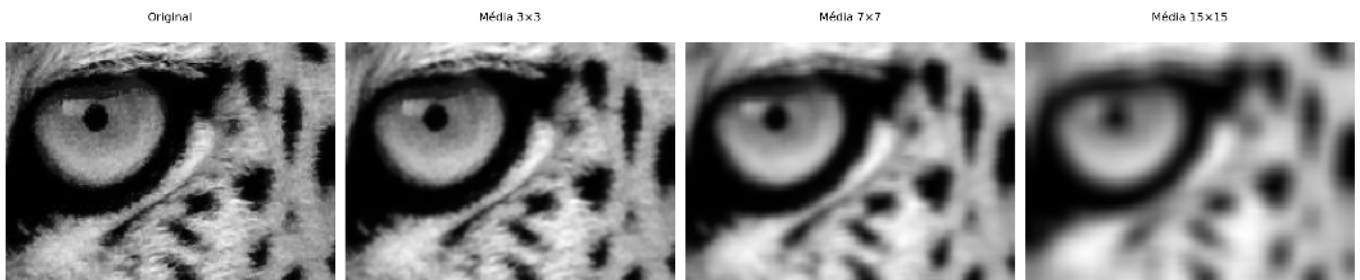


Figura 3.12: Filtro de média com *kernels* de tamanho crescente (3×3 , 7×7 , 15×15). O borramento das bordas aumenta com o tamanho do *kernel*.

3.5.2 Filtro Gaussiano

O filtro Gaussiano pesa os pixels da vizinhança de acordo com uma função Gaussiana bidimensional:

$$G(s, t) = \frac{1}{2\pi\sigma^2} e^{-\frac{s^2+t^2}{2\sigma^2}} \quad (3.14)$$

onde σ é o desvio padrão e controla o raio de influência. Pixels mais próximos do centro têm peso maior; pixels distantes são progressivamente ignorados.

A Figure 3.13 apresenta o *kernel* Gaussiano 5×5 gerado para $\sigma = 1$. O *kernel* foi construído a partir do produto externo de dois vetores Gaussianos unidimensionais e posteriormente normalizado para que a soma de seus coeficientes seja igual a 1. Observa-se que os maiores pesos concentram-se no centro da matriz, decrescendo radialmente em direção às bordas. Essa distribuição faz com que os pixels centrais tenham maior influência no resultado da filtragem, contribuindo para uma suavização mais natural e com melhor preservação de bordas do que o filtro de média.

```
# Gera kernel Gaussiano 5x5 via cv2
k_gauss = cv2.getGaussianKernel(5, 1)
w_gauss = k_gauss @ k_gauss.T          # @ => multiplicação matricial: G(s,t) = G(s)·G(t)
w_gauss = w_gauss / w_gauss.sum()      # garante soma = 1

print("Kernel Gaussiano 5x5 (=1), normalizado:")
for row in w_gauss:
```

```

print(" " + " ".join(f"{v:.4f}" for v in row))
print(f"\nSoma dos coeficientes: {w_gauss.sum():.6f}")
print(f"Peso central vs. canto: {w_gauss[2,2]:.4f} vs. {w_gauss[0,0]:.4f} "
      f"({w_gauss[2,2]/w_gauss[0,0]:.1f}× maior)")

mm.drawImgPlt((w_gauss * 1000).astype(np.uint8), scale=40)

```

Kernel Gaussiano 5×5 (=1), normalizado:

```

0.0030  0.0133  0.0219  0.0133  0.0030
0.0133  0.0596  0.0983  0.0596  0.0133
0.0219  0.0983  0.1621  0.0983  0.0219
0.0133  0.0596  0.0983  0.0596  0.0133
0.0030  0.0133  0.0219  0.0133  0.0030

```

Soma dos coeficientes: 1.000000

Peso central vs. canto: 0.1621 vs. 0.0030 (54.6× maior)

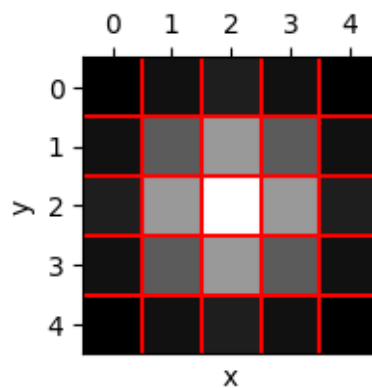


Figura 3.13: *Kernel* Gaussiano 5×5 (=1): pesos normalizados, maiores no centro e decrescentes radialmente. A grade facilita a leitura de cada coeficiente.

i Vantagem computacional da separabilidade

Considere um *kernel* quadrado de tamanho $n \times n$. Se esse filtro for separável (como o Gaussiano), a convolução 2D pode ser decomposta em duas convoluções 1D: uma horizontal e outra vertical. Nesse caso, o custo por pixel passa de aproximadamente $O(n^2)$ operações (convolução 2D direta) para $O(2n)$ operações (duas convoluções 1D). Assim, a complexidade é reduzida de forma significativa, tornando o processamento mais eficiente

Comparado ao filtro de média, o Gaussiano:

- **Preserva melhor as bordas** — a ponderação radial suaviza sem criar transições abruptas;
- **Não introduz anéis** (*ringing*) no domínio da frequência, pois a Gaussiana é sua própria transformada de Fourier (Capítulo 5);
- **É controlado por σ** — aumentar σ equivale a aumentar o raio de suavização de forma contínua e previsível.

A Figure 3.14 compara os filtros de média e Gaussiano aplicados à imagem do leopardo usando uma janela 9×9 . O filtro de média foi implementado por convolução com um *kernel* uniforme, em que todos os 81 pixels da vizinhança possuem o mesmo peso ($1/81$), enquanto o filtro Gaussiano foi obtido com `cv2.GaussianBlur`, utilizando pesos definidos por uma distribuição Gaussiana. Ambos reduzem ruído e suavizam a imagem, porém o filtro Gaussiano preserva melhor as bordas e os detalhes locais, como pode ser observado na região ampliada do olho.

A Figure 3.14 compara os filtros de média e Gaussiano aplicados a um detalhe da imagem do leopardo

com *kernels* 9×9 . O filtro de média foi obtido com `mm.blur`, equivalente à convolução com um *kernel* uniforme cujos coeficientes valem $1/81$, enquanto o filtro Gaussiano foi obtido com `mm.gaussian`, equivalente à convolução com um *kernel* gerado a partir de uma distribuição Gaussiana. Ambos promovem suavização e redução de ruído, porém o filtro Gaussiano atribui maior peso aos pixels centrais da vizinhança, preservando melhor as bordas e os detalhes locais, como pode ser observado na região ampliada do olho.

```
# w_media9 = np.ones((9,9), dtype=np.float32) / 81.0
# img_media9 = mm.conv(img_gray, w_media9) # ou
img_media9 = mm.blur(img_gray, 9)
# img_gauss9 = cv2.GaussianBlur(img_gray, (9,9), 0) # ou
img_gauss9 = mm.gaussian(img_gray, 9, 0)

# Detalhe da região do olho
y0, y1, x0, x1 = 580, 740, 680, 900
crop = lambda img: img[y0:y1, x0:x1]

img_gray_crop = crop(img_gray)

mm.show(
    [crop(img_gray), crop(img_media9), crop(img_gauss9)],
    titles=["Detalhe: Original", "Média 9x9", "Gaussiano 9x9"],
    cols=3, figsize=(12, 8)
)
```



Figura 3.14: Comparação entre filtro de média e Gaussiano (*kernel* 9×9 , $\sigma=0$). O Gaussiano preserva melhor as bordas, visível no detalhe do rosto.

3.6 Filtragem Espacial de Realce

Os filtros de realce (*sharpening filters*) enfatizam transições abruptas de intensidade, aumentando a nitidez e a visibilidade de bordas. São filtros **passa-alta** — amplificam as componentes de alta frequência (bordas, textura) e suprimem as de baixa frequência (regiões uniformes).

A intuição é simples: se subtrairmos de uma imagem sua versão suavizada (que contém apenas as baixas frequências), o que resta são as altas frequências — bordas e detalhes. Somando esse resíduo de volta à imagem original, o contraste local aumenta:

$$g = f + k(f - f_{\text{suave}}), \quad k > 0 \quad (3.15)$$

O Figure 3.15 ilustra esse processo em um sinal 1D sintético com três estruturas distintas: um degrau largo, um pico fino e uma rampa suave. No painel , o sinal original $f(x)$; no , a versão suavizada $f_{\text{suave}}(x)$ obtida por média móvel — note como o pico fino é atenuado. O painel exibe o resíduo $f - f_{\text{suave}}$, que retém apenas as transições abruptas. Por fim, o painel mostra $g(x)$: o pico, antes atenuado, é restaurado e amplificado

em relação ao original. Ajuste k e o tamanho da janela para observar o *trade-off* entre nitidez e amplificação de ruído.

Os filtros de realce formalizam essa ideia diretamente no *kernel*, sem precisar de duas etapas separadas.



Figura 3.15: Simulador: Filtragem Espacial de Realce 1D.

3.6.1 Laplaciano

O Laplaciano é um operador de **segunda derivada** isotrópico, ou seja, responde igualmente a variações em todas as direções, ao contrário de operadores de primeira derivada, como Sobel e Prewitt, que são direcionais:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad (3.16)$$

Uma propriedade importante da segunda derivada é que seu valor é **próximo de zero em regiões uniformes** e elevado nas transições de intensidade. Assim, ao subtrair o Laplaciano da imagem original, reforçam-se bordas e detalhes, aumentando o contraste local:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y) \quad (3.17)$$

Na forma discreta, a segunda derivada em x é aproximada por $f(x+1, y) - 2f(x, y) + f(x-1, y)$, e analogamente em y . Somando as duas direções, obtém-se o *kernel* w_4 (4-vizinhos) ou w_8 (8-vizinhos, incluindo diagonais):

$$w_4 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad w_8 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (3.18)$$

i Soma zero e centro negativo

Ambos os *kernels* têm **soma dos coeficientes igual a zero**: em regiões uniformes, a saída é 0 — o Laplaciano não altera o brilho médio, apenas detecta variações. O **centro negativo** indica que o pixel é comparado com seus vizinhos: quanto mais ele se destacar (para cima ou para baixo), maior o valor absoluto do Laplaciano naquele ponto.

No exemplo a seguir, o pixel central $[1, 1] = 107$ possui vizinhos $\{95, 106, 108, 103\}$. Como esses valores são próximos entre si, a região é quase uniforme e o Laplaciano retorna um valor baixo, produzindo pouco realce. Em regiões de borda, onde há diferenças maiores entre o pixel central e seus vizinhos, o Laplaciano assume valores mais elevados (positivos ou negativos), e a operação de Equation 3.17 intensifica essas transições.

A Figure 3.16 ilustra o cálculo do Laplaciano com o *kernel* w_4 em uma vizinhança 3×3 destacada dentro de um *patch* 5×5 . O exemplo mostra o valor obtido pelo operador e o correspondente pixel realçado na imagem de saída, que passa de 107 para 123.

```
patch = img_gray[250:255, 250:255]
w4 = np.array([[0, 1, 0],
               [1,-4, 1],
               [0, 1, 0]], dtype=np.float32)
B = np.ones((3,3), dtype=np.uint8)

p = patch.astype(np.float32)
lap_11 = int(p[0,1] + p[1,0] - 4*p[1,1] + p[1,2] + p[2,1])
real_11 = int(np.clip(p[1,1] - lap_11, 0, 255))

print("Patch original:")
print(mm.drawImg(patch))
print(f"Laplac. w4 em [1,1]: ", end="")
print(f"{p[0,1]:.0f} + {p[1,0]:.0f} - 4*{p[1,1]:.0f} + {p[1,2]:.0f} + {p[2,1]:.0f} = {lap_11}")
print(f"Pixel realçado [1,1]: {p[1,1]:.0f} - ({lap_11}) = {real_11}")

mm.drawImgKernel(patch, B, x=1, y=1, scale=40.0)
```

```
Patch original:
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124

Laplac. w4 em [1,1]: 95 + 106 - 4*107 + 108 + 103 = -16
Pixel realçado [1,1]: 107 - (-16) = 123
```

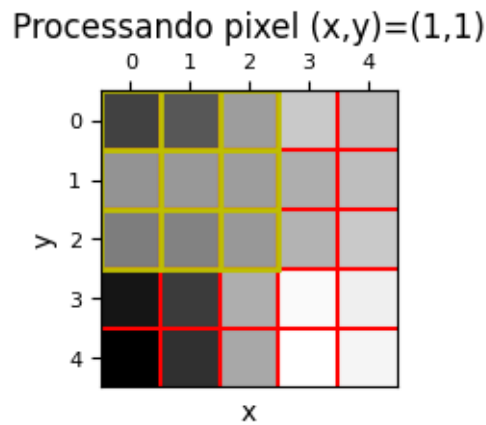


Figura 3.16: *Kernel* Laplaciano w_4 aplicado ao *patch* 5×5 : a janela amarela destaca a vizinhança 3×3 onde o operador de segunda derivada é calculado.

A Figure 3.17 compara a aplicação dos *kernels* Laplacianos w_4 e w_8 em um recorte maior da imagem do leopardo. Para cada caso, são mostradas a resposta bruta do operador, que evidencia as bordas e transições de intensidade, e a imagem obtida após o realce por subtração do Laplaciano. Observa-se que o *kernel* w_8 , por considerar também os vizinhos diagonais, produz uma resposta mais intensa e detecta variações em mais direções, resultando em um realce ligeiramente mais acentuado.

```
w4 = np.array([[0, 1, 0],
               [1,-4, 1],
               [0, 1, 0]], dtype=np.float32)
w8 = np.array([[1, 1, 1],
               [1,-8, 1],
               [1, 1, 1]], dtype=np.float32)

mm.show(
    [img_gray_crop, mm.laplacian_viz(img_gray_crop, w4), mm.laplacian(img_gray_crop, w4),
      img_gray_crop, mm.laplacian_viz(img_gray_crop, w8), mm.laplacian(img_gray_crop, w8)],
    titles=["Original", "Laplaciano w4", "Realce w4",
            "Original", "Laplaciano w8", "Realce w8"],
    rows=2, cols=3, figsize=(14, 8)
)
```



Figura 3.17: Laplaciano aplicado à imagem do leopardo: resposta bruta (bordas detectadas) com w4 e w8, e imagens realçadas pela subtração do Laplaciano. w8 é mais sensível às diagonais.

3.6.2 Operador de Sobel

O operador de Sobel estima as **derivadas parciais de primeira ordem** nas direções horizontal e vertical. Diferente do Laplaciano (segunda derivada), o Sobel é direcional e mais robusto ao ruído, pois cada *kernel* combina uma derivada com uma suavização Gaussiana perpendicular:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * f, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * f \quad (3.19)$$

G_x detecta bordas **verticais** (variação na direção x); G_y detecta bordas **horizontais** (variação na direção y). Os pesos $\{1, 2, 1\}$ na direção perpendicular correspondem à suavização Gaussiana 1D, que reduz a sensibilidade ao ruído.

i Sobel é correlação, não convolução

Os *kernels* de Sobel são **assimétricos** — a rotação de 180° altera o resultado. `cv2.Sobel` implementa correlação cruzada (como `cv2.filter2D`). Para obter a derivada direcional correta, os sinais já estão definidos para correlação: G_x retorna valores positivos onde a intensidade cresce da esquerda para a direita.

A magnitude do **gradiente** combina os dois componentes, representando a força da borda independente de direção:

$$|\nabla f| = \sqrt{G_x^2 + G_y^2} \quad (3.20)$$

E a **direção** do gradiente (perpendicular à borda) é:

$$\theta = \arctan \left(\frac{G_y}{G_x} \right) \quad (3.21)$$

Para ilustrar numericamente, calcula-se G_x e G_y manualmente no pixel central $[1, 1]$ do *patch* 5×5:

```
patch = img_gray[250:255, 250:255]
p     = patch.astype(np.float32)

# Gx no pixel [1,1]: coluna direita - coluna esquerda (ponderada)
gx = (p[0,2] + 2*p[1,2] + p[2,2]) - (p[0,0] + 2*p[1,0] + p[2,0])

# Gy no pixel [1,1]: linha inferior - linha superior (ponderada)
gy = (p[2,0] + 2*p[2,1] + p[2,2]) - (p[0,0] + 2*p[0,1] + p[0,2])

mag  = np.sqrt(gx**2 + gy**2)
theta = np.degrees(np.arctan2(gy, gx))

print("Patch 5x5:")
print(mm.drawImg(patch))
print(f"Gx em [1,1]: ({p[0,2]:.0f} + 2*{p[1,2]:.0f} + {p[2,2]:.0f}) \
      - ({p[0,0]:.0f} + 2*{p[1,0]:.0f} + {p[2,0]:.0f}) = {gx:.1f}")
print(f"Gy em [1,1]: ({p[2,0]:.0f} + 2*{p[2,1]:.0f} + {p[2,2]:.0f}) \
      - ({p[0,0]:.0f} + 2*{p[0,1]:.0f} + {p[0,2]:.0f}) = {gy:.1f}")

# Substituído o | pelo prefixo mag para não quebrar o interpretador de tabelas do Quarto
print(f"mag(Grad f) = sqrt({gx:.1f}^2 + {gy:.1f}^2) = {mag:.1f}")
print(f"Theta      = arctan({gy:.1f}/{gx:.1f}) = {theta:.1f} graus")
```

Patch 5x5:

```
91 95 108 116 114
106 107 108 111 114
102 103 107 112 116
83 90 111 125 123
79 88 110 126 124
```

```
Gx em [1,1]: (108 + 2*108 + 107)      - (91 + 2*106 + 102) = 26.0
Gy em [1,1]: (102 + 2*103 + 107)      - (91 + 2*95 + 108) = 26.0
mag(Grad f) = sqrt(26.0^2 + 26.0^2) = 36.8
Theta      = arctan(26.0/26.0) = 45.0 graus
```

O valor reduzido de $|\nabla f|$ nesse *patch* confirma que a região é quase uniforme, pois o gradiente assume valores elevados apenas onde há mudanças significativas de intensidade. A Figure 3.18 aplica o operador de Sobel a um recorte maior da imagem do leopardo. São apresentadas as respostas horizontal (G_x) e vertical (G_y), obtidas por convolução com os respectivos *kernels* de Sobel, além da magnitude $|\nabla f|$, calculada a partir da combinação de ambas. Enquanto G_x destaca bordas verticais e G_y bordas horizontais, a magnitude evidencia bordas em qualquer direção.

```
def norm255(img):
    mn, mx = img.min(), img.max() #+1e-9 para evitar divisão por zero
    return ((img - mn) / (mx - mn + 1e-9) * 255).astype(np.uint8)

sobelx = cv2.Sobel(img_gray_crop, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img_gray_crop, cv2.CV_64F, 0, 1, ksize=3)
magnitudo = np.sqrt(sobelx**2 + sobely**2)
direcao = np.degrees(np.arctan2(sobely, sobelx))

# magnitudo = mm.sobel(img_gray_crop) # ou, mas com clip em vez de norm255

mm.show()
```

```
[img_gray_crop, norm255(np.abs(sobelx)), norm255(np.abs(sobely)),
 norm255(magnitude), norm255(direcao % 180)],
titles=["Original", "Gx (bordas vert.)", "Gy (bordas horiz.)",
       "Magnitude |f|", "Direção (mod 180°)"],
cols=5
)
```



Figura 3.18: Operador de Sobel na imagem do leopardo: Gx detecta bordas verticais, Gy detecta bordas horizontais, e a magnitude $|f|$ combina ambos, revelando todas as bordas independente de direção.

3.6.3 Operador de Prewitt

O operador de Prewitt é estruturalmente idêntico ao Sobel, mas substitui a ponderação Gaussiana $\{1, 2, 1\}$ por pesos uniformes $\{1, 1, 1\}$:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} * f, \quad G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} * f \quad (3.22)$$

A magnitude e a direção do gradiente seguem as mesmas equações do Sobel (Equation 3.20 e Equation 3.21). A diferença prática é que o Prewitt é ligeiramente mais sensível ao ruído — a suavização perpendicular uniforme pondera menos o pixel central da linha — mas computacionalmente mais simples. Em imagens com baixo ruído os resultados são equivalentes.

```
Kx = np.array([[-1,0,1],[-1,0,1],[-1,0,1]], dtype=np.float32)
Ky = np.array([[-1,-1,-1],[0,0,0],[1,1,1]], dtype=np.float32)
f = img_gray_crop.astype(np.float32)

mm.show(
    [img_gray_crop,
     norm255(np.abs(cv2.filter2D(f, -1, Kx))),
     norm255(np.abs(cv2.filter2D(f, -1, Ky))),
     mm.prewitt(img_gray_crop)],
    titles=["Original", "Gx", "Gy", "Magnitude |f|"],
    cols=4
)
```

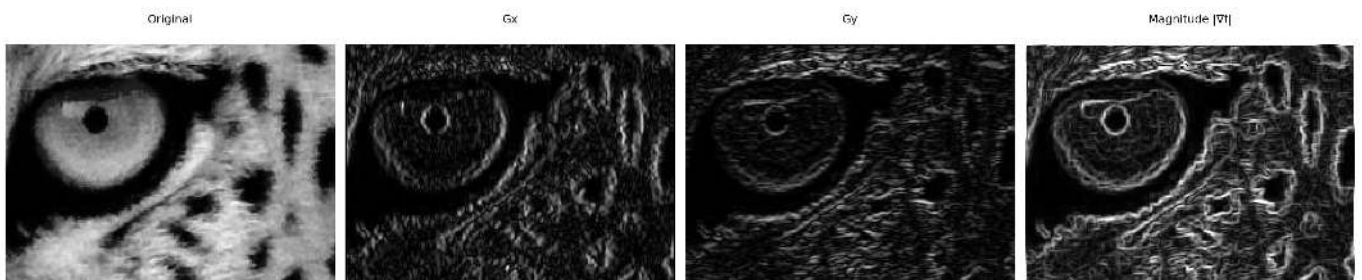


Figura 3.19: Operador de Prewitt: Gx, Gy e magnitude — comparável ao Sobel, mas sem ponderação Gaussiana perpendicular.

3.6.4 Unsharp Masking (USM)

O *Unsharp Masking* é uma técnica clássica de realce de nitidez originária da fotografia analógica, hoje amplamente usada em software de edição de imagens. A ideia central é extrair as **componentes de alta frequência** da imagem (bordas e detalhes) e somá-las de volta à original com um peso k :

Tabela 3.3: Etapas do *Unsharp Masking*.

Etapa	Operação	Descrição
1	$\bar{f} = f * G_{\sigma}$	Suaviza com Gaussiana — retém baixas frequências
2	$m = f - \bar{f}$	Máscara: diferença = altas frequências (bordas)
3	$g = f + k \cdot m$	Soma ponderada da máscara à original

Substituindo a etapa 2 na etapa 3, obtém-se a expressão compacta:

$$g = f + k(f - f * G_{\sigma}) = (1 + k)f - k(f * G_{\sigma}) \quad (3.23)$$

O parâmetro k controla a intensidade do realce:

- $k = 0$: sem realce ($g = f$);
- $k = 1$: USM clássico — duplica a contribuição das altas frequências;
- $k > 1$: *High Boost Filtering* — amplificação além do dobro, útil para imagens muito borradas.

⚠ Amplificação de ruído

O USM não distingue bordas de ruído — ambos são componentes de alta frequência. Para k elevado, o ruído presente na imagem é amplificado junto com as bordas. Por isso, é recomendável aplicar uma leve suavização antes do USM em imagens ruidosas, ou usar σ pequeno na Gaussiana.

Para ilustrar as etapas do USM, a Figure 3.20 aplica o método a um *patch* 30×30 da imagem do leopardo, utilizando $\sigma = 1$ e $k = 1$. Inicialmente, a imagem é suavizada por um filtro Gaussiano. Em seguida, a máscara de alta frequência é obtida pela diferença entre a imagem original e a suavizada. Por fim, essa máscara é somada à imagem original, reforçando bordas e detalhes. A figura apresenta as três etapas do processo e o resultado final do realce.

```
patch = img_gray_crop[35:65, 45:75]
k = 1.0

p = patch.astype(np.float32)
sigma = 1.0

# Etapa 1: suavização Gaussiana
ksize = int(6 * sigma + 1) | 1 # operador binário garante tamanho ímpar
p_suave = cv2.GaussianBlur(p, (ksize, ksize), sigma)

# Etapa 2: máscara de alta frequência
mascara = p - p_suave

# Etapa 3: realce
p_usm = np.clip(p + k * mascara, 0, 255).astype(np.uint8)

# p_usb = mm.usm(p, k) # ou

print(f"Pixel central [1,1]: original={int(p[1,1])} suave={p_suave[1,1]:.1f}")
```



```
f" mascara={mascara[1,1]:.1f} realcado={p_usm[1,1]}")

mm.show(
    [patch,
     p_suave.astype(np.uint8),
     np.clip(mascara + 128, 0, 255).astype(np.uint8), # tornar os valores negativos visíveis na figura
     p_usm],
    titles=["Patch original",
            f"Suavizado (={sigma})",
            "Máscara (alta freq., +128)",
            f"Realçado USM (k={k})"],
    cols=4, figsize=(12, 4)
)
```

Pixel central [1,1]: original=16 suave=20.2 mascara=-4.2 realcado=11

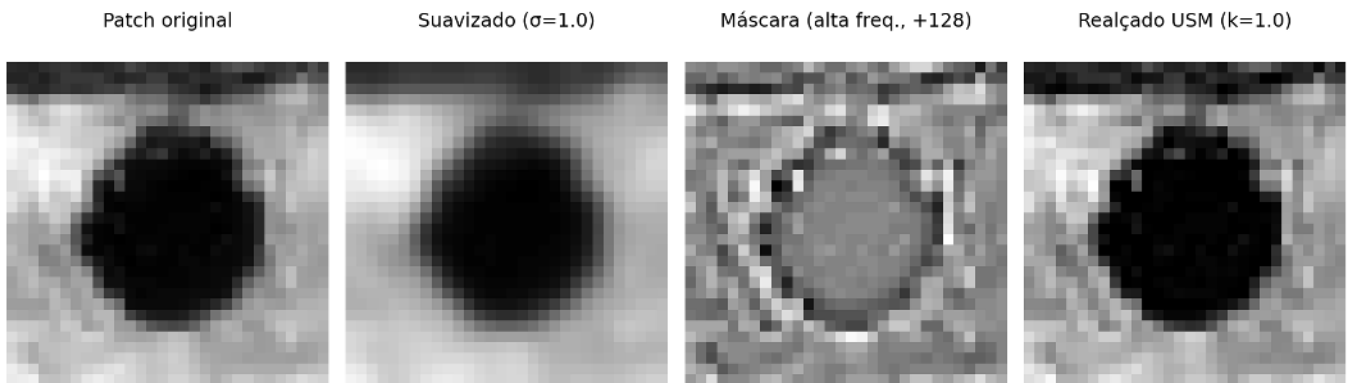


Figura 3.20: Realce de nitidez por *Unsharp Masking* na imagem do leopardo: a imagem suavizada é subtraída da original para gerar a máscara de alta frequência, que é então reintroduzida para realçar bordas e detalhes.

A Figure 3.21 aplica o método USM a um recorte maior da imagem do leopardo utilizando $\sigma = 1$ e diferentes valores do fator de ganho k . Em todos os casos, a máscara de alta frequência é obtida pela diferença entre a imagem original e sua versão suavizada por filtro Gaussiano. O parâmetro k controla a intensidade do realce: valores menores produzem um aumento sutil de nitidez, enquanto valores maiores reforçam progressivamente bordas e detalhes. Observa-se que, para valores elevados de k , surgem halos ao redor das bordas e o ruído presente na imagem passa a ser amplificado.

```
ks = [0.5, 1.0, 3.0, 5.0, 8.0]

imgs = [img_gray_crop] + [mm.usm(img_gray_crop, k) for k in ks]
titles = ["Original"] + [f"USM k={k}" for k in ks]

mm.show(imgs, titles=titles, cols=3, rows=2, figsize=(14, 8))
```

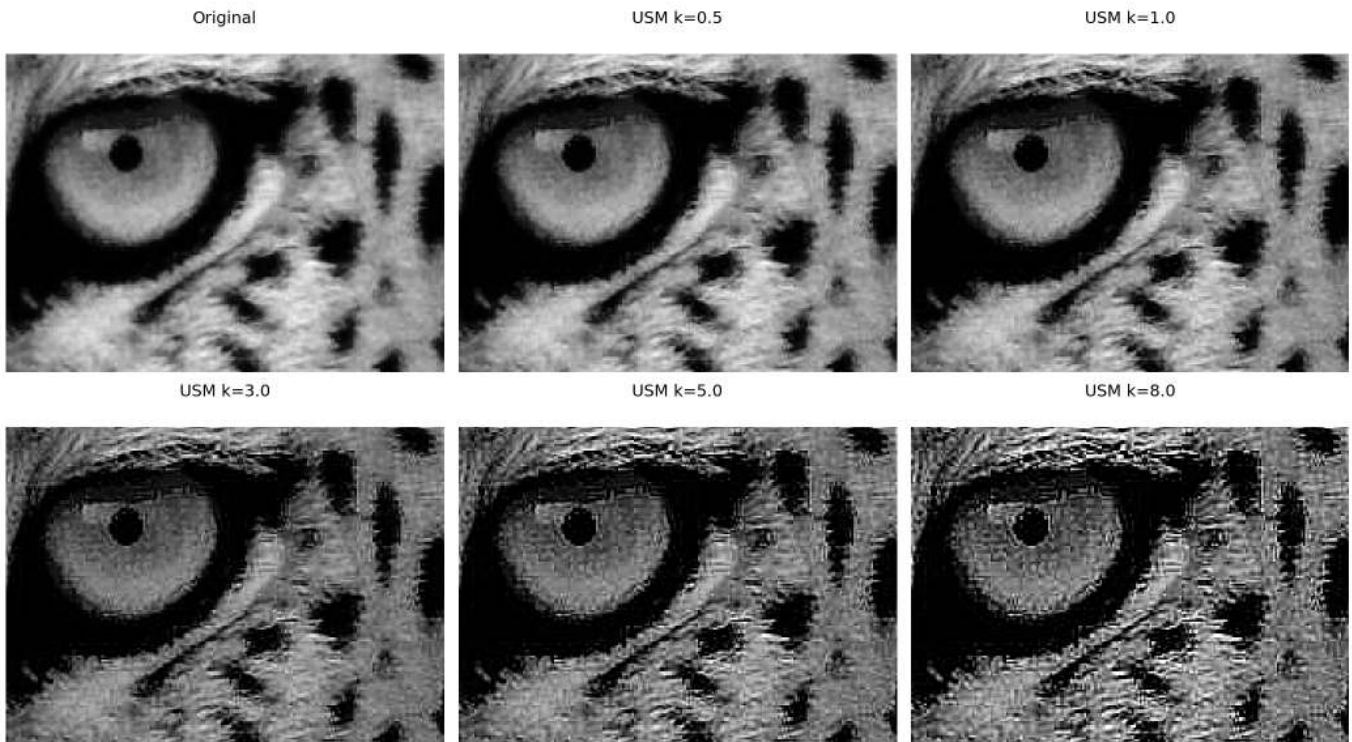


Figura 3.21: *Unsharp Masking* na imagem do leopardo com $\alpha=1$ e k variando de 0.5 a 8.0. Para $k>2$ surgem artefatos nas bordas (halos) e o ruído de fundo começa a ser visível.

3.6.5 Detector de Canny

O Canny combina quatro etapas em sequência — suavização Gaussiana, gradiente de Sobel, supressão de não-máximos e histerese por duplo limiar — para produzir bordas **finas, binárias e conectadas**. Diferente de Sobel e Prewitt, o resultado não é um mapa de gradiente contínuo, mas uma máscara onde cada pixel é borda ou não.

O parâmetro central é o par de limiares (T_{low}, T_{high}). Pixels com gradiente acima de T_{high} são bordas certas; abaixo de T_{low} , descartados. Os pixels ambíguos — entre os dois limiares — são decididos por **histerese**: tornam-se borda se estiverem conectados a uma borda certa, e descartados caso contrário. Isso evita tanto a perda de trechos fracos de bordas reais quanto a inclusão de ruído isolado. Uma heurística comum é $T_{high} = 3 \times T_{low}$.

```
mm.show(
    [img_gray_crop,
     mm.canny(img_gray_crop, 30, 90),
     mm.canny(img_gray_crop, 60, 180),
     mm.canny(img_gray_crop, 120, 240)],
    titles=["Original", "Canny (30/90)", "Canny (60/180)", "Canny (120/240)"],
    cols=4
)
```

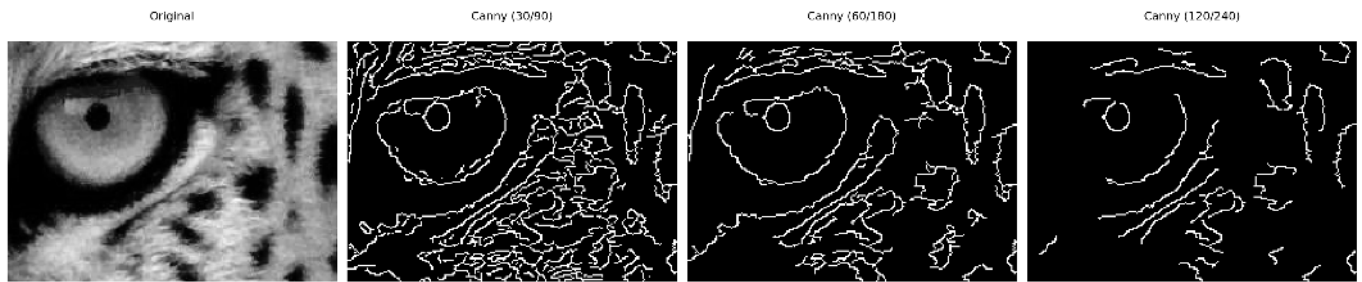


Figura 3.22: Detector de Canny com diferentes pares de limiar: limiares baixos capturam mais bordas (inclusive ruído); limiares altos retêm apenas as bordas mais fortes.

i Escolha dos limiares

Uma heurística comum é $T_{high} = 3 \times T_{low}$. Valores típicos dependem do intervalo de gradiente da imagem — `cv2.Canny` aceita valores absolutos em $[0, 255]$. Para imagens com contraste variável, calcular os limiares a partir de percentis da magnitude de Sobel é mais robusto do que valores fixos.

3.7 Filtros de Ordem: Filtro da Mediana

Os filtros de ordem (*order-statistic filters*) substituem o pixel central pelo valor de um **percentil** da distribuição de intensidades da vizinhança — ao contrário dos filtros lineares, que calculam combinações ponderadas. O mais importante é o **filtro da mediana**.

3.7.1 Ruído Impulsivo: Sal e Pimenta

O ruído **sal e pimenta** (*salt-and-pepper noise*) substitui pixels aleatórios por valores extremos: 0 (pimenta, preto) ou 255 (sal, branco). É comum em transmissão de imagens com erros de bit e em câmeras com sensores defeituosos.

Para entender por que os filtros lineares falham, considere uma vizinhança 3×3 onde um único pixel foi corrompido para 255:

$$\text{vizinhança} = \begin{bmatrix} 102 & 98 & 105 \\ 100 & \mathbf{255} & 97 \\ 103 & 99 & 101 \end{bmatrix}$$

Tabela 3.4: Média vs. mediana com um pixel corrompido. A mediana ignora o outlier; a média é deslocada ~40 níveis.

Método	Cálculo	Resultado
Média	$(102+98+\dots+255+\dots+101)/9$	140
Mediana	$\{97, 98, 99, 100, \mathbf{101}, 102, 103, 105, 255\}$	101

⚠ Por que filtros de média falham com ruído impulsivo?

A média é sensível a **outliers** — um único pixel com valor 255 em uma vizinhança de valor 100 eleva a saída para 140, espalhando o ruído pela imagem. A mediana, por ser um **estimador robusto**, seleciona o valor central da distribuição ordenada, descartando naturalmente os extremos sem nenhum ajuste especial.

O exemplo a seguir ilustra o comportamento da média e da mediana na presença de um pixel corrompido por ruído impulsivo. Observa-se que a média é fortemente influenciada pelo valor extremo (255), produzindo uma estimativa distante dos valores predominantes da vizinhança. Já a mediana permanece próxima do valor original da região, evidenciando sua maior robustez a *outliers* e justificando seu uso na remoção de ruído sal e pimenta.

```
# Exemplo numérico: média vs. mediana com pixel corrompido
vizinhanca = np.array([102, 98, 105, 100, 255, 97, 103, 99, 101])
print(f"Vizinhança: {sorted([int(x) for x in vizinhanca])}")
print(f"Média:      {vizinhanca.mean():.1f}  (deslocada pelo outlier 255)")
print(f"Mediana:    {int(np.median(vizinhanca))}  (ignora o outlier)")
print(f"Valor original: ~100")
```

```
Vizinhança: [97, 98, 99, 100, 101, 102, 103, 105, 255]
Média:      117.8  (deslocada pelo outlier 255)
Mediana:    101   (ignora o outlier)
Valor original: ~100
```

A Figure 3.23 apresenta o efeito do ruído sal e pimenta em diferentes densidades. O ruído foi gerado substituindo aleatoriamente uma fração dos pixels por valores mínimos (0, pimenta) e máximos (255, sal). À medida que a densidade aumenta de 2% para 10%, cresce a quantidade de pixels corrompidos, tornando a degradação visual mais evidente e dificultando a percepção de detalhes da imagem.

```
def add_salt_pepper(img, prob=0.05, seed=42):
    """Adiciona ruído sal e pimenta com probabilidade total prob."""
    noisy = img.copy()
    rnd = np.random.default_rng(seed).random(img.shape)
    noisy[rnd < prob / 2] = 0 # pimenta
    noisy[rnd > 1 - prob / 2] = 255 # sal
    n_corrompidos = int((rnd < prob/2).sum() + (rnd > 1-prob/2).sum())
    return noisy, n_corrompidos

probs = [0.02, 0.05, 0.10]
imgs_noise, titles_noise = [img_gray_crop], ["Original"]

for p in probs:
    noisy, n = add_salt_pepper(img_gray_crop, p)
    imgs_noise.append(noisy)
    titles_noise.append(f"Ruído {int(p*100)}%\n({n:,} pixels)")

mm.show(imgs_noise, titles=titles_noise, cols=4)
```

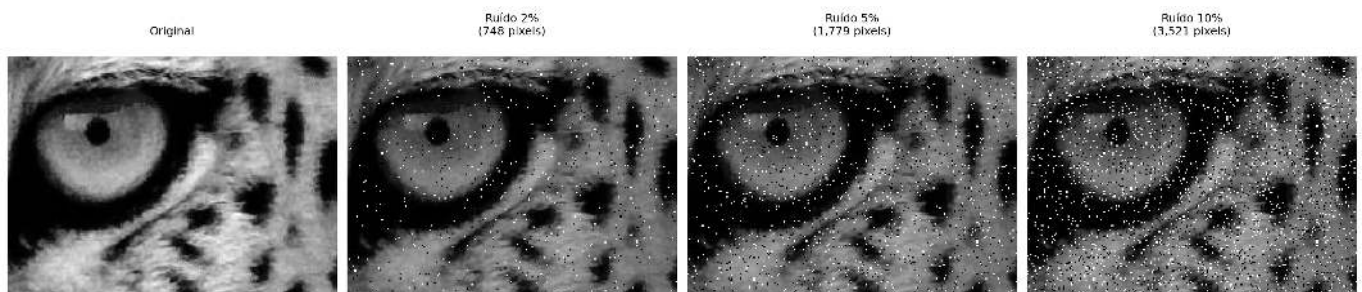


Figura 3.23: Ruído sal e pimenta com densidades crescentes (2%, 5%, 10%). O parâmetro *prob* indica a fração total de pixels corrompidos, metade sal (255) e metade pimenta (0).

3.7.2 Filtro da Mediana

O filtro da mediana substitui cada pixel pelo **valor mediano** dos pixels da sua vizinhança $n \times n$:

$$g(x, y) = \text{med}_{(s,t) \in \mathcal{V}_n} \{f(x + s, y + t)\} \quad (3.24)$$

O valor mediano é aquele que ocupa a posição central quando os n^2 valores da vizinhança são ordenados. Para uma janela 3×3 ($n^2 = 9$ pixels), a mediana é o 5º valor da sequência ordenada.

Para ilustrar, considere o mesmo *patch* 5×5 com um pixel corrompido artificialmente em $[1, 1]$:

```
patch = img_gray[250:255, 250:255].copy()
corrompido = patch.copy()
corrompido[1, 1] = 255 # injeta pixel sal no centro

# Vizinhança 3x3 centrada em [1,1]
viz_orig = sorted(patch[0:3, 0:3].ravel().tolist())
viz_corr = sorted(corrompido[0:3, 0:3].ravel().tolist())

print("Patch original:")
print(mm.drawImg(patch))
print("\nPatch com pixel corrompido [1,1]=255:")
print(mm.drawImg(corrompido))
print(f"\nVizinhança 3x3 original (ordenada): {viz_orig}")
print(f"Mediana original: {viz_orig[4]}")
print(f"\nVizinhança 3x3 corrompida (ordenada): {viz_corr}")
print(f"Mediana corrompida: {viz_corr[4]} ← ignora o 255")
print(f"Média corrompida: {np.mean(viz_corr):.1f} ← distorcida pelo 255")
```

```
Patch original:
 91  95 108 116 114
106 107 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Patch com pixel corrompido [1,1]=255:
```

```
 91  95 108 116 114
106 255 108 111 114
102 103 107 112 116
 83  90 111 125 123
 79  88 110 126 124
```

```
Vizinhança 3x3 original (ordenada): [91, 95, 102, 103, 106, 107, 107, 108, 108]
Mediana original: 106
```

```
Vizinhança 3x3 corrompida (ordenada): [91, 95, 102, 103, 106, 107, 108, 108, 255]
Mediana corrompida: 106 ← ignora o 255
Média corrompida: 119.4 ← distorcida pelo 255
```

O exemplo confirma: mesmo com o pixel corrompido a 255, a mediana retorna o valor central correto — o *outlier* ocupa a última posição na ordenação e é descartado naturalmente.

Por ser baseada em ordenação e não em soma, a mediana possui três propriedades fundamentais que a diferenciam dos filtros lineares:

- **Robusta** ao ruído impulsivo — outliers vão para as extremidades da sequência ordenada e não afetam o valor central;
- **Preservadora de bordas** — transições abruptas de intensidade são mantidas, pois a mediana seleciona um valor que já existe na vizinhança, sem criar novos níveis intermediários;

- **Não linear** — não pode ser expressa como convolução, portanto `mm.conv` não se aplica; usa-se `cv2.medianBlur`.

A Figure 3.24 compara diferentes técnicas de remoção de ruído sal e pimenta aplicadas a uma imagem com 10% de pixels corrompidos. Foram avaliados os filtros Gaussiano, Média, Mediana, Bilateral e Morfológico (próximo capítulo), permitindo observar o compromisso entre remoção de ruído e preservação de detalhes. Em geral, os filtros de média e Gaussiano reduzem o ruído, mas tendem a borrar as bordas, enquanto a mediana apresenta melhor desempenho para ruído impulsivo. O filtro bilateral preserva melhor as bordas, e o filtro morfológico remove boa parte dos pixels corrompidos sem degradar excessivamente a estrutura da imagem.

```
# 10% de ruído para evidenciar diferenças entre filtros
noisy_5, _ = add_salt_pepper(img_gray_crop, prob=0.1)

# Filtros
f_gauss = cv2.GaussianBlur(noisy_5, (5, 5), 1.0)
f_media = mm.conv(noisy_5, np.ones((5, 5), dtype=np.float32) / 20.0)
f_median3 = cv2.medianBlur(noisy_5, 3)
f_median5 = cv2.medianBlur(noisy_5, 5)
f_bilat = cv2.bilateralFilter(noisy_5, d=9, sigmaColor=75, sigmaSpace=75)

# filtros morfológicos no próximo capítulo, mas já adiantados aqui para comparação
# B = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
# f_morf = cv2.morphologyEx( # apresentado no próximo capítulo
#     cv2.morphologyEx(noisy_5, cv2.MORPH_OPEN, B),
#     cv2.MORPH_CLOSE, B)
B = mm.seccross() # elemento estruturante de caixa 3x3
f_morf = mm.asf(noisy_5, 'OC', B) # equivalente via morph

# MSE
def mse(a, b):
    return float(np.mean((a.astype(np.float32) - b.astype(np.float32))**2))

print(f"{'Filtro':<22} {'MSE':>8}")
print("-" * 32)
pares = [("Gaussiano 5x5", f_gauss),
         ("Média 5x5", f_media),
         ("Mediana 3x3", f_median3),
         ("Mediana 5x5", f_median5),
         ("Bilateral", f_bilat),
         ("Morf. open+close", f_morf)]
for nome, img in pares:
    print(f"{nome:<22} {mse(img_gray_crop, img):>8.2f}")

imgs = [img_gray_crop, noisy_5, f_gauss, f_media,
        f_median3, f_median5, f_bilat, f_morf]
titles = ["Original", "Ruído 10%", "Gaussiano 5x5", "Média 5x5",
         "Mediana 3x3", "Mediana 5x5", "Bilateral", "Morf. open+close"]

mm.show(
    imgs,
    titles=[f"Crop: {t}" for t in titles],
    cols=4, rows=2, figsize=(14, 8)
)
```

Filtro	MSE
Gaussiano 5x5	260.84
Média 5x5	872.57
Mediana 3x3	31.26

Mediana 5×5	65.86
Bilateral	1001.22
Morf. open+close	65.16

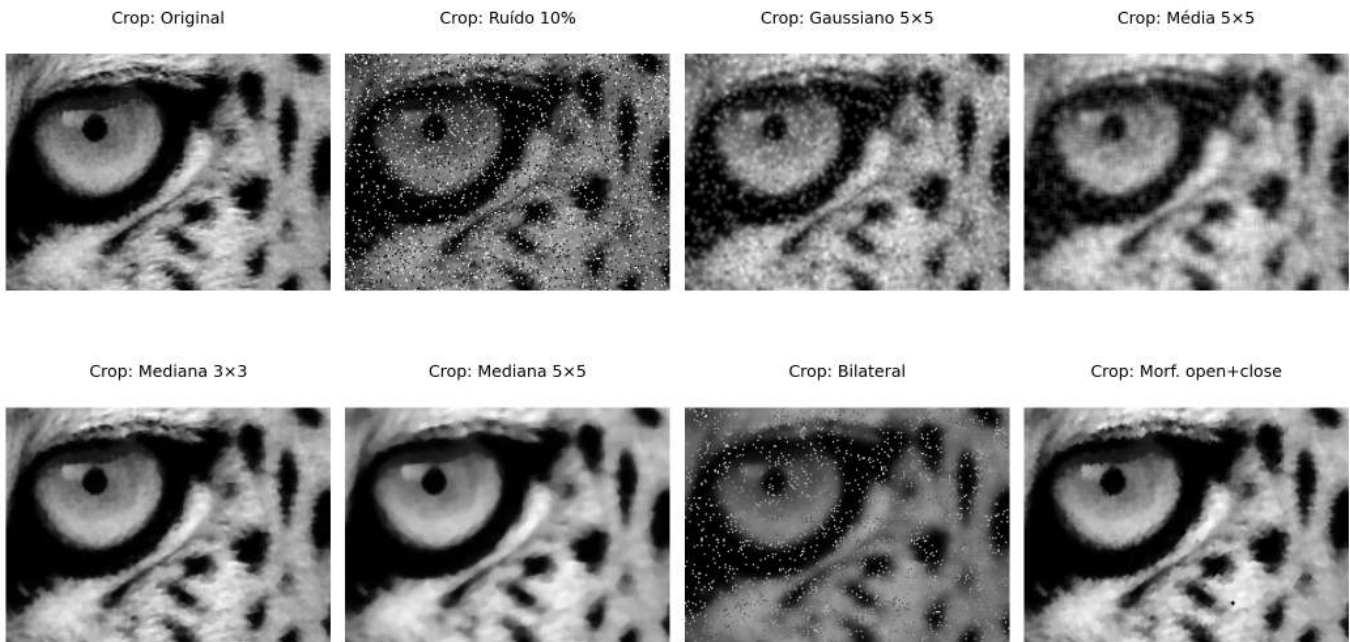


Figura 3.24: Comparação de filtros para remoção de ruído sal e pimenta (10%): Gaussiano, Média, Mediana, Bilateral e Morfológico. Linha superior: imagens completas; linha inferior: crop da região de interesse.

3.8 Aplicação Prática: Pré-processamento para Segmentação

Na prática, as técnicas deste capítulo raramente são usadas isoladamente. Um *pipeline* de **pré-processamento** típico combina várias etapas em sequência, adaptando-se ao tipo de imagem e à aplicação. A Figure 3.25 ilustra um *pipeline* completo:

1. **Equalização de histograma (CLAHE)**: normaliza o contraste independente das condições de iluminação;
2. **Filtro Gaussiano**: suaviza ruído de aquisição sem destruir bordas;
3. **Deteção de bordas (Sobel/Canny)**: extrai estruturas relevantes para segmentação.

i Ordem importa

A ordem das operações afeta o resultado final. Em geral: **(1) normalização de intensidade → (2) redução de ruído → (3) realce/segmentação**. Inverter a ordem pode amplificar ruído ou perder bordas antes de detectá-las.

```
# Etapa 1: CLAHE
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img_gray_crop)

# Etapa 2: Gaussiano
img_gauss = cv2.GaussianBlur(img_clahe, (5, 5), 0)

# Etapa 3: Canny
edges = cv2.Canny(img_gauss, 50, 150)

# Comparação: pipeline vs. Canny direto
edges_direct = cv2.Canny(img_gray_crop, 50, 150)
```

```
mm.show(
    [img_gray_crop, img_clahe, img_gauss, edges, edges_direct],
    titles=["Original", "1. CLAHE", "2. Gaussiano", "3. Canny (pipeline)", "Canny (direto)"],
    cols=5
)
```



Figura 3.25: *Pipeline* de pré-processamento: CLAHE → Gaussiano → Canny. Cada etapa prepara a imagem para a seguinte, resultando em bordas limpas e bem definidas.

3.9 Resumo

Neste capítulo foram apresentadas as principais técnicas de processamento no domínio espacial, da manipulação direta de pixels até a filtragem por vizinhança:

- **Operações de ponto:** aritméticas saturadas (`mm.addm`, `mm.subm`) e lógicas bit a bit (`mm.band`, `mm.bor`, `mm.bnot`) para recorte de ROI e combinação de imagens; *alpha blending* (`mm.blend`) para fusão ponderada com peso $\alpha \in [0, 1]$.
- **Histograma:** função discreta de distribuição de intensidades; visualizado com `mm.histImg` e calculado com `mm.hist`; base para diagnóstico tonal e para as técnicas de equalização e especificação.
- **Equalização:** redistribuição automática das intensidades pela CDF (`mm.equalize`), com variante adaptativa CLAHE para controle local do contraste.
- **Especificação de histograma:** transferência do perfil tonal de uma imagem de referência via mapeamento inverso da CDF — generalização da equalização para distribuições arbitrárias.
- **Correlação e convolução:** mecanismo de janela deslizante implementado em `mm.conv` (`cv2.filter2D`); diferenciados pela rotação de 180° do *kernel* — relevante apenas para kernels assimétricos.
- **Filtros de suavização:** média (*kernel* uniforme, borra bordas proporcionalmente ao tamanho) e Gaussiano (ponderação radial, separável, sem *ringing*, preserva melhor as bordas).
- **Filtros de realce:** Laplaciano (w_4/w_8 , segunda derivada isotrópica), Sobel (gradiente direcional de primeira ordem, com magnitude $|\nabla f|$ e direção θ) e *Unsharp Masking* (amplificação das altas frequências com parâmetro k).
- **Filtro da mediana:** não linear, robusto a outliers, preserva bordas — superior aos filtros lineares para ruído sal e pimenta.
- **Pipeline prático:** encadeamento CLAHE → Gaussiano → Canny como estratégia de pré-processamento; `mm.drawImgKernel` para visualização didática da janela deslizante.

O Capítulo 4 abordará a **morfologia matemática** (erosão, dilatação, abertura e fechamento), explorando em profundidade as funções `mm.ero` e `mm.dil` da biblioteca `morph.py`. Em seguida, o Capítulo 5 apresentará o **processamento no domínio da frequência**, com foco na Transformada de Fourier e em técnicas de filtragem espectral.

3.10 Uso do Gemini Notebook como Tutor Complementar

Nesta edição, incentivamos o uso do **Gemini Notebook** como ferramenta complementar de aprendizagem. Essa ferramenta de IA utiliza exclusivamente os documentos fornecidos pelo autor como base de conhecimento, garantindo respostas coerentes com o conteúdo do livro — incluindo as funções da biblioteca `morph.py` e os experimentos realizados neste capítulo.

Para cada capítulo, preparamos um projeto específico na plataforma com o PDF do capítulo, os *notebooks* e materiais auxiliares. Sugerimos explorar especialmente:

- **Guia de Estudo:** resumo estruturado dos conceitos, ideal para revisão antes de provas;
- **Conversa:** tire dúvidas sobre equalização, convolução, filtros e pipelines diretamente com o tutor;
- **Perguntas frequentes:** questões típicas sobre a diferença entre média e mediana, USM, Laplaciano vs. Sobel.

Estude com o Tutor Inteligente

Para interagir com o conteúdo deste capítulo, acesse o *link* a seguir. O ambiente contém materiais didáticos em diferentes formatos, gerados a partir do **PDF** do capítulo. Na plataforma, explore especialmente as opções **Guia de Estudo** e **Conversa** para aprofundar sua compreensão.

 **ACESSAR Gemini Notebook: CAPÍTULO 03**

Aviso sobre Conteúdo Gerado por IA

A IA é uma poderosa aliada nos estudos, mas o conteúdo gerado pode conter **erros ou imprecisões**. Consulte sempre **livros, artigos científicos e outras fontes acadêmicas confiáveis** para validar as informações. Sempre que possível, execute os exemplos práticos fornecidos neste capítulo para verificar os resultados.

3.11 Lista de Exercícios

1. (10%) Explique a diferença entre **convolução** e **correlação cruzada**. Para quais tipos de *kernel* os resultados são idênticos? Dê um exemplo de *kernel* assimétrico (como Sobel G_x) e mostre numericamente que os resultados diferem aplicando-o ao patch 5×5 do capítulo das duas formas.
2. (15%) Considere uma imagem 5×5 com intensidades concentradas entre os níveis 3 e 5 (baixo contraste, 3 bits). Aplique manualmente o algoritmo de equalização da Table 3.1, preenchendo todas as colunas da tabela (k , $h[k]$, $p[k]$, $\text{cdf}[k]$, $\text{lut}[k]$). Verifique o resultado com `mm.equalize`.
3. (15%) Usando `mm.conv`, aplique o filtro de média com kernels de tamanho 3×3 , 9×9 e 21×21 à imagem do mandrill. Para cada versão, calcule o **PSNR** (*Peak Signal-to-Noise Ratio*) em relação à original:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right), \quad \text{MSE} = \frac{1}{MN} \sum_{i,j} (f - g)^2$$

Plote o PSNR em função do tamanho do kernel e explique o que a queda progressiva indica sobre a relação entre suavização e perda de informação.

4. (15%) Usando `add_salt_pepper` com densidade de 5%, aplique e compare: (a) `mm.conv` com média 3×3 , (b) `cv2.GaussianBlur` com $\sigma = 1$, (c) `cv2.medianBlur` com janela 3×3 e (d) `cv2.medianBlur` com janela 5×5 . Exiba as imagens com `mm.show` em grade 2×4 (linha 1: imagens, linha 2: histogramas via `mm.histImg`). Explique por que a mediana supera os filtros lineares usando o argumento da Table 3.4.
5. (15%) Implemente `mm.conv0` usando apenas operações NumPy vetorizadas — sem laços Python e sem `cv2.filter2D` — com o operador de *stride tricks* (`np.lib.stride_tricks.sliding_window_view`). Compare o resultado e o tempo de execução com `mm.conv0` (laços) e `mm.conv` (cv2) para kernels 3×3 e 15×15 na imagem do mandrill.
6. (15%) Aplique o *Unsharp Masking* com $\sigma = 1$ e $k \in \{0.5, 1.0, 2.0, 4.0\}$ usando a função `usm` do capítulo. Para cada valor de k : (a) calcule a diferença absoluta $|g - f|$, (b) exiba as imagens e as diferenças com `mm.show`, e (c) plote o histograma das diferenças com `mm.histImg`. Identifique a partir de qual k os artefatos (halos e amplificação de ruído) tornam-se visualmente inaceitáveis.

7. (15%) Escolha uma imagem de raio-X ou tomografia disponível publicamente (ex.: via `mm.read` de URL) e projete um pipeline de pré-processamento com pelo menos 4 etapas sequenciais, justificando cada escolha com base nos conceitos do capítulo. Exiba com `mm.show` em grade: imagem original, cada etapa intermediária e o resultado final com seus histogramas (`mm.histImg`).

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de operações de intensidade, histograma, convolução e filtragem espacial.
- Szeliski (2022) para a visão computacional e aplicações práticas de filtragem.
- Bradski; Kaehler (2008) para a implementação prática com OpenCV e `morph.py`.



3.12 Parte Prática com Exercícios de Programação

Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f"\u2705 Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Para avaliar os testes, execute `TestSuite("EP03_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```

codigo = """
from morph import mm
# ... seu código aqui ...
"""
TestSuite("EP03_01").run_code(codigo)

```

3.12.1 EP03_01 + Adição Saturada de Constante

Em sistemas de vigilância por vídeo, câmeras em ambientes com iluminação variável produzem imagens subexpostas. O ajuste de brilho por **adição saturada de uma constante** é a operação mais simples para correção imediata, sendo aplicada em tempo real nos *chips* de câmeras embarcadas e em *pipelines* de pré-processamento de robôs móveis.

Ver na Figure 3.26 uma simulação deste EP.

3.12.1.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Constante:** Ler o inteiro k (valor a ser somado).
3. **Dados:** Ler os valores inteiros da matriz original linha a linha.
4. **Mapeamento:** Para cada pixel p , calcular o novo valor pela equação:

$$p' = \text{clip}(p + k)$$

5. **Saída:** Exibir a matriz resultante com dimensões $L \times C$.

3.12.1.2 📌 Restrições Computacionais

- **Saturação (*Clipping*):** Os valores devem ser confinados ao intervalo $[0, 255]$:

$$\text{clip}(x) = \max(0, \min(255, x))$$

- **Tipo:** O resultado final deve ser inteiro (sem casas decimais).
- **k pode ser negativo:** valores negativos escurecem a imagem; positivos clareiam.

3.12.1.3 🧠 Fundamentação Teórica

Parâmetro	Tipo	Impacto Visual
$k > 0$	Inteiro	Clareia a imagem; pixels próximos de 255 saturam em branco
$k < 0$	Inteiro	Escurece a imagem; pixels próximos de 0 saturam em preto
$k = 0$	Inteiro	Imagem inalterada

3.12.1.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .

- Linha 2: Inteiro C .
- Linha 3: Inteiro k .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz transformada em L linhas e C colunas, valores inteiros separados por espaço.

3.12.1.5 📌 Exemplos

Entrada	Saída	Observação
2	50 150 250	Saturação em 255 nos pixels altos
3	255 255 255	
50		
0 100 200		
210 240 255		
1	0 0 170 225	Saturação em 0 nos pixels baixos
4		
-30		
0 20 200 255		

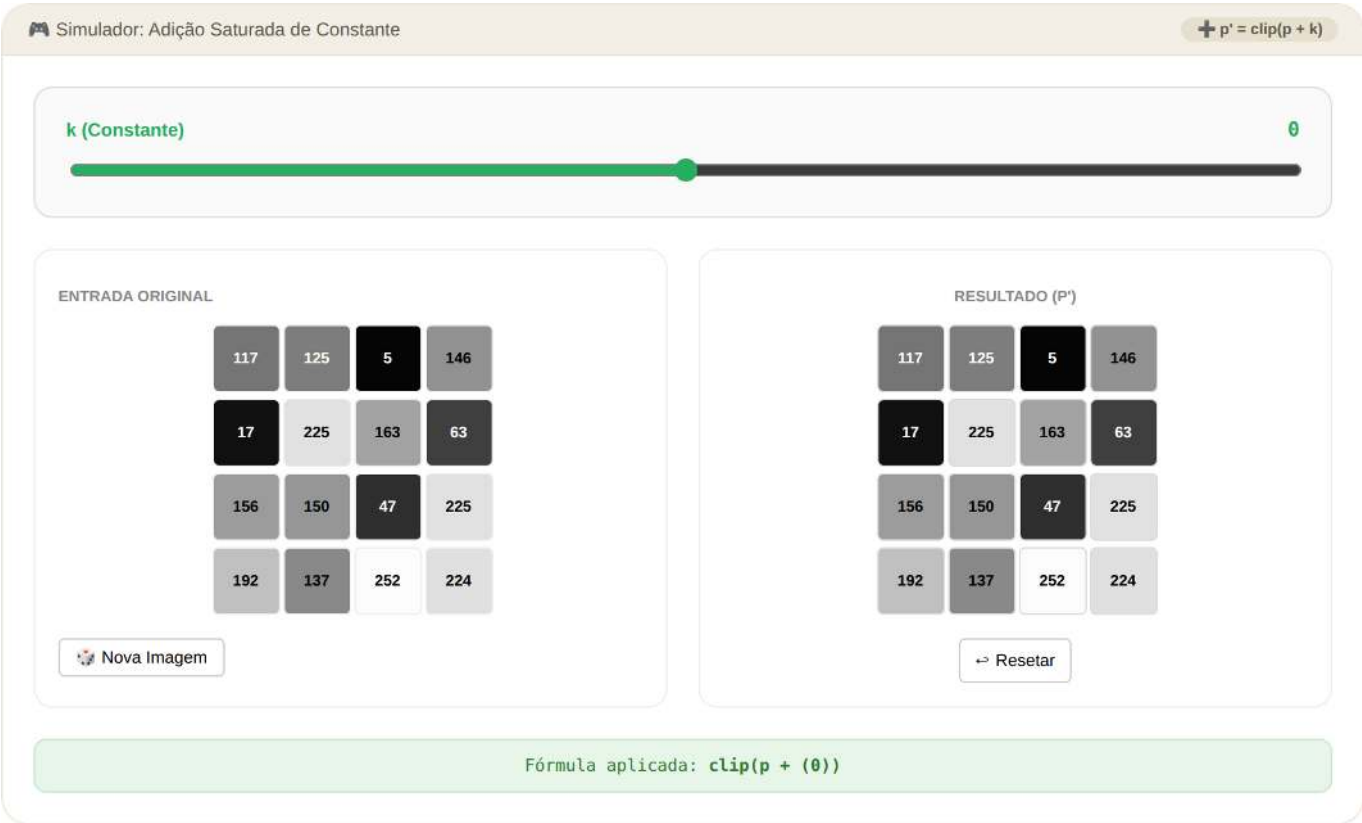


Figura 3.26: Simulador: Adição Saturada de Constante

```
%%writefile EP03_01.py
# Código Python

Writing EP03_01.py

TestSuite("EP03_01.py").run()
```


3.12.2 EP03_02 Alpha *Blending* de Duas Imagens

Em medicina nuclear, imagens de diferentes modalidades (tomografia computadorizada e ressonância magnética) são fundidas para auxiliar no diagnóstico. A **mistura ponderada** (*alpha blending*) é a operação fundamental desse processo, permitindo ao radiologista controlar interativamente o peso de cada modalidade na imagem exibida.

Ver na Figure 3.27 uma simulação deste EP.

3.12.2.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Parâmetro:** Ler o valor real $\alpha \in [0, 1]$.
3. **Dados:** Ler os valores inteiros da matriz f_1 (imagem 1) e em seguida da matriz f_2 (imagem 2).
4. **Mapeamento:** Para cada posição (i, j) , calcular:

$$g(i, j) = \text{clip}(\text{round}(\alpha \cdot f_1(i, j) + (1 - \alpha) \cdot f_2(i, j)))$$

5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.2.2 Restrições Computacionais

- **Arredondamento:** Aplicar **round** antes da conversão para inteiro.
- **Saturação:** Confinar ao intervalo $[0, 255]$ com $\text{clip}(x) = \max(0, \min(255, x))$.
- **Operação em float:** Realize a operação em ponto flutuante antes de arredondar.

3.12.2.3 Fundamentação Teórica

Valor de α	Resultado
$\alpha = 1.0$	Apenas f_1
$\alpha = 0.5$	Média aritmética de f_1 e f_2
$\alpha = 0.0$	Apenas f_2

3.12.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Real α .
- Linhas seguintes: Elementos de f_1 (L linhas com C valores cada).
- Linhas seguintes: Elementos de f_2 (L linhas com C valores cada).

Saída:

- Matriz resultante $L \times C$.

3.12.2.5 📌 Exemplos

Entrada	Saída	Observação
1 3 0.5 0 100 200 100 200 50	50 150 125	Média entre as duas imagens
1 3 1.0 10 20 30 90 80 70	10 20 30	Apenas f_1 (alpha=1)

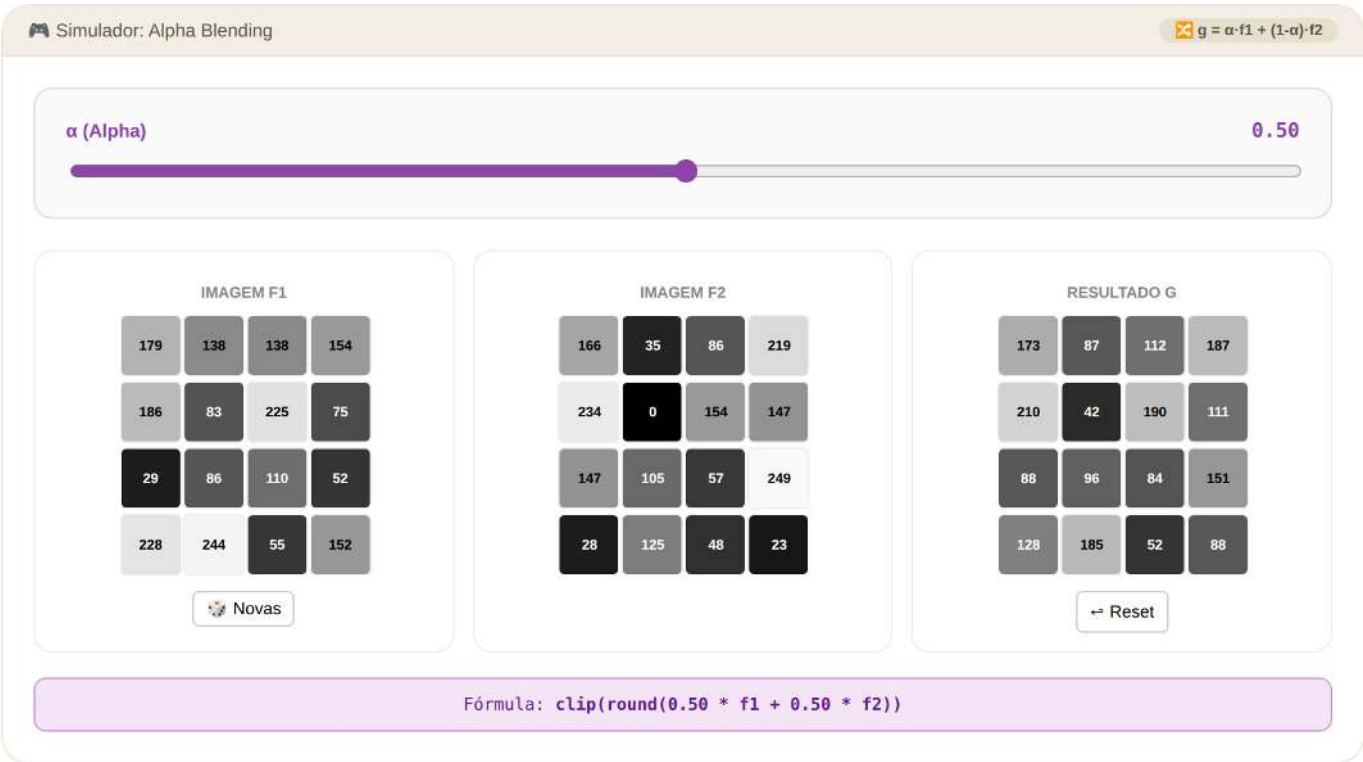


Figura 3.27: Simulador: Alpha *Blending* de Duas Imagens

```
%%writefile EP03_02.py
# Código Python

Writing EP03_02.py

TestSuite("EP03_02.py").run()
```

3.12.3 EP03_03 🧠 Inversão de Imagem (Negativo Fotográfico)

Em radiologia, as imagens de raio-X são tradicionalmente visualizadas em negativo: ossos aparecem em preto sobre fundo branco. A operação de **negativo fotográfico** é aplicada rotineiramente em PACS (*Picture Archiving and Communication Systems*) para facilitar a detecção de fraturas e densidades ósseas.

Ver na Figure 3.28 uma simulação deste EP.

3.12.3.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler os valores inteiros da matriz original.
3. **Mapeamento:** Para cada pixel p , calcular o negativo:

$$p' = 255 - p$$

4. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.3.2 Restrições Computacionais

- **Sem *clipping* necessário:** O resultado de $255 - p$ com $p \in [0, 255]$ é sempre $\in [0, 255]$.
- **Tipo inteiro:** Saída deve ser valores inteiros.
- **Equivalência lógica:** A operação é idêntica ao NOT bit a bit (`mm.bnot`) em imagens de 8 bits.

3.12.3.3 Fundamentação Teórica

Pixel Original p	Pixel Negativo p'	Observação
0 (preto)	255 (branco)	Inversão total
128 (cinza médio)	127 (cinza médio)	Valor central
255 (branco)	0 (preto)	Inversão total

3.12.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz negativa em L linhas e C colunas.

3.12.3.5 Exemplos

Entrada	Saída	Observação
140 128 200 255	255 127 55 0	Inversão de cada pixel
2 2 10 20 30 40	245 235 225 215	Matriz 2x2 invertida

```
%%writefile EP03_03.py
# Código Python
```

```
Writing EP03_03.py
```

```
TestSuite("EP03_03.py").run()
```



Figura 3.28: Simulador: Inversão de Imagem (Negativo Fotográfico)

3.12.4 EP03_04 Equalização de Histograma (L bits)

Em imagens de satélite de sensoriamento remoto, a variação de iluminação ao longo do dia produz imagens de baixo contraste. A **equalização de histograma** é aplicada automaticamente em satélites como o Landsat para redistribuir os tons, revelando detalhes de vegetação, relevo e zonas urbanas invisíveis na imagem original.

Ver na Figure 3.29 uma simulação deste EP.

3.12.4.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas), C (colunas) e B (número de bits, com $L_{\max} = 2^B$).
2. **Dados:** Ler a matriz de pixels f com valores em $[0, 2^B - 1]$.
3. **Histograma:** Calcular $h[k] = \text{número de pixels com intensidade } k$, para $k = 0 \dots 2^B - 1$.
4. **Probabilidade:** $p[k] = h[k] / (L \cdot C)$.
5. **CDF:** $\text{cdf}[k] = \sum_{j=0}^k p[j]$; função de distribuição acumulada.
6. **LUT:** $\text{lut}[k] = \text{round}(\text{cdf}[k] \cdot (2^B - 1))$; *Look-Up Table* (tabela de consulta).
7. **Aplicação:** $g[i, j] = \text{lut}[f[i, j]]$.
8. **Saída:** Exibir a matriz equalizada $L \times C$.

3.12.4.2 Restrições Computacionais

- **Arredondamento:** Usar arredondamento matemático (**round**) na LUT.
- **Bits:** O número de níveis é 2^B (ex.: $B = 3 \Rightarrow 8$ níveis, $B = 8 \Rightarrow 256$ níveis).
- **CDF acumulada:** $\text{cdf}[k] = \sum_{j=0}^k p[j]$, com $\text{cdf}[2^B - 1] = 1.0$.

3.12.4.3 Fundamentação Teórica

Etapa	Operação	Fórmula
1	Histograma	$h[k] \leftarrow \text{n}^{\circ} \text{ pixels com intensidade } k$
2	Probabilidade	$p[k] = h[k]/(L \cdot C)$
3	CDF	$\text{cdf}[k] = \sum_{j=0}^k p[j]$
4	LUT	$\text{lut}[\text{Look} - \text{UpTable}(\text{tabeladeconsulta})k] = \text{round}(\text{cdf}[k] \cdot (2^B - 1))$
5	Aplicação	$g[i, j] = \text{lut}[f[i, j]]$

3.12.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro B (número de bits).
- Linhas seguintes: Elementos inteiros da matriz.

Saída:

- Matriz equalizada em L linhas e C colunas.

3.12.4.5 Exemplos

Entrada	Saída	Observação
5	5 7 1 5 7	Exemplo 3 bits do capítulo
5	7 5 5 7 5	
3	1 5 7 5 1	
3 4 2 3 4	5 7 5 1 5	
4 3 3 4 3	7 5 1 5 7	
2 3 4 3 2		
3 4 3 2 3		
4 3 2 3 4		Histograma bimodal extremo
1	0 0 7 7	
4		
3		
0 0 7 7		

```
%%writefile EP03_04.py
# Código Python
```

```
Writing EP03_04.py
```

```
TestSuite("EP03_04.py").run()
```

3.12.5 EP03_05 Aplicação de Máscara AND Binária

Em sistemas de inspeção industrial por visão computacional, é necessário isolar regiões de interesse (ROI) em imagens de peças para verificar defeitos de fabricação. A operação **AND bit a bit com uma máscara**

Simulador: Equalização de Histograma

Escolha o número de bits e gere uma imagem para ver a equalização em tempo real.

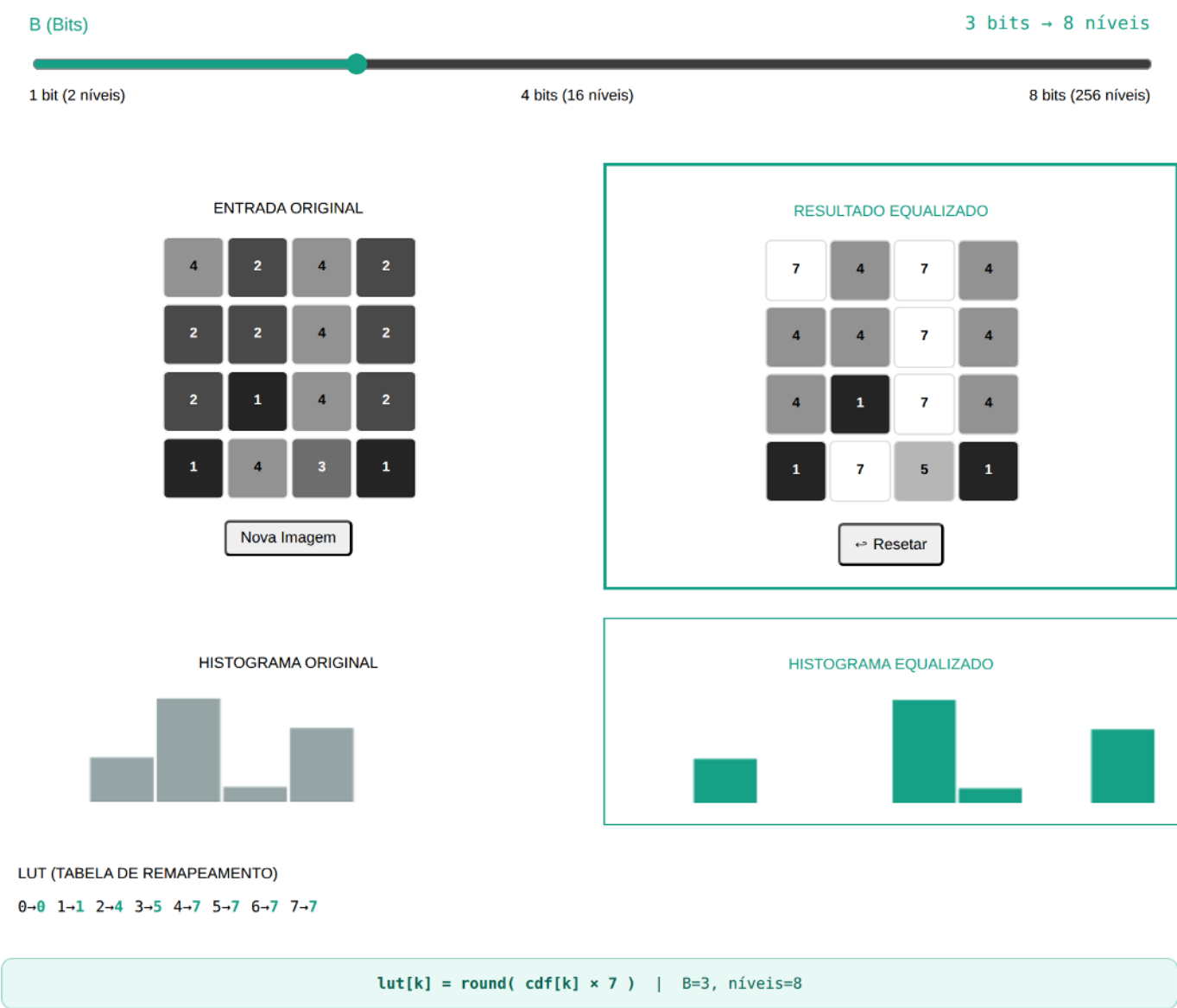


Figura 3.29: Simulador: Equalização de Histograma (L bits)

binária é o mecanismo fundamental para recortar exatamente a área de inspeção, zerando todos os pixels fora dela.

Ver na Figure 3.30 uma simulação deste EP.

3.12.5.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f (valores $\in [0, 255]$).
3. **Máscara:** Ler a matriz binária m (valores: apenas 0 ou 255).
4. **Mapeamento:** Para cada pixel (i, j) , aplicar o AND bit a bit:

$$g(i, j) = f(i, j) \text{ AND } m(i, j)$$

onde $255 = 11111111$ e $0 = 00000000$ em binário.

5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.5.2 Restrições Computacionais

- **AND com 255:** $p \text{ AND } 255 = p$ (todos os bits preservados).
- **AND com 0:** $p \text{ AND } 0 = 0$ (todos os bits zerados).
- **Máscara:** Os únicos valores possíveis na máscara são 0 e 255.
- **Implementação:** Em Python, o AND bit a bit entre inteiros usa o operador `&`.

3.12.5.3 Fundamentação Teórica

Pixel f	Máscara m	Resultado $f \text{ AND } m$
qualquer v	255 (11111111)	v (preservado)
qualquer v	0 (00000000)	0 (zerado)

3.12.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos de f (L linhas).
- Linhas seguintes: Elementos de m (L linhas com valores 0 ou 255).

Saída:

- Matriz resultante $L \times C$.

3.12.5.5 Exemplos

Entrada	Saída	Observação
2	100 150 0	Máscara seleciona região
3	0 80 120	
100 150 200		
50 80 120		
255 255 0		
0 255 255		Alternado preservado/zerado
1	10 0 30 0	
4		
10 20 30 40		
255 0 255 0		

Simulador de máscara AND binária: aplica operação AND pixel a pixel entre imagem e máscara interativa.

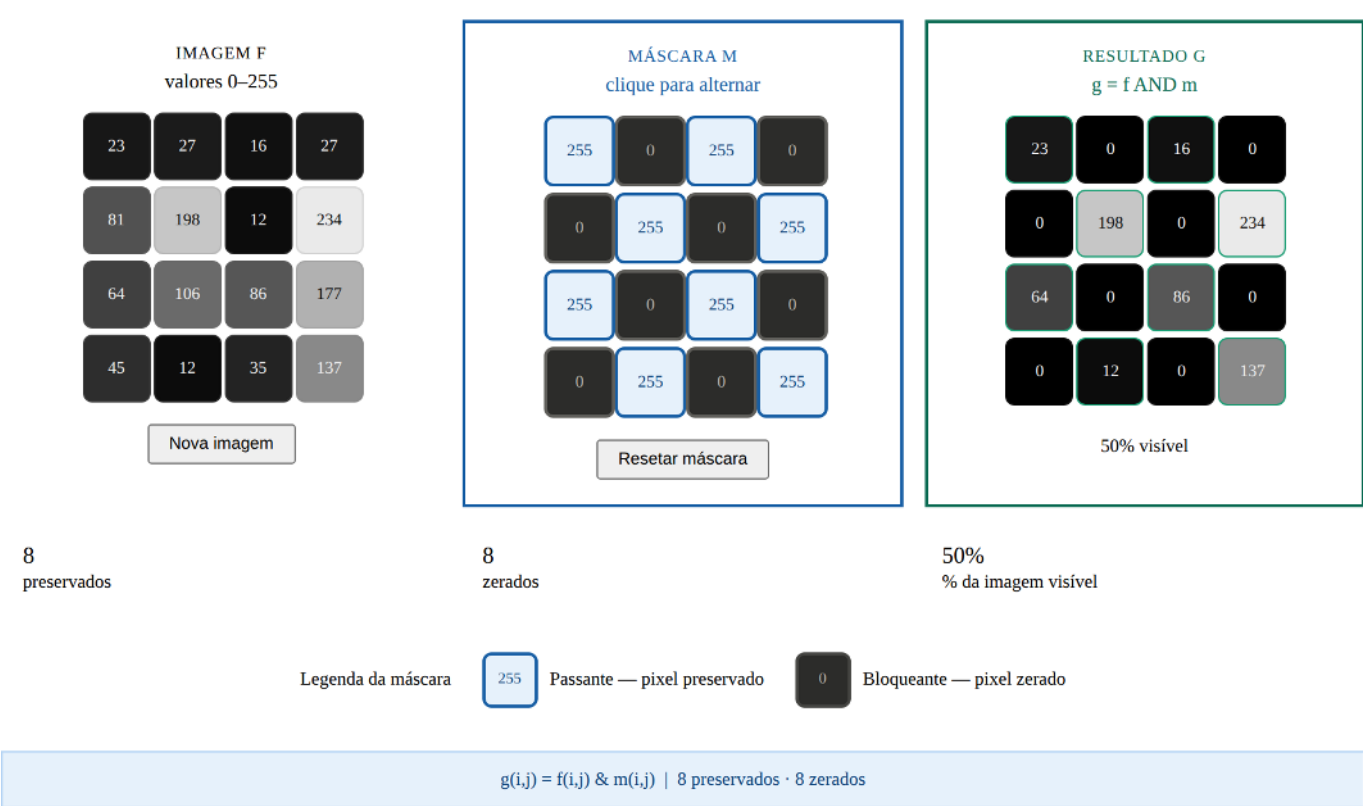


Figura 3.30: Simulador: Aplicação de Máscara AND Binária

```
%%writefile EP03_05.py
# Código Python

Writing EP03_05.py

TestSuite("EP03_05.py").run()
```

3.12.6 EP03_06 Filtro de Média com *Kernel* N×N

Em câmeras de veículos autônomos, imagens capturadas sob chuva ou névoa apresentam ruído gaussiano. O **filtro de média** é amplamente utilizado para sua redução em tempo real, sendo implementado diretamente

no **ISP** (*Image Signal Processor*) de sensores **CMOS** (*Complementary Metal-Oxide-Semiconductor*).

Os sensores CMOS são os sensores de imagem usados na maioria das câmeras modernas (*smartphones*, *webcams*, câmeras automotivas etc.). Eles convertem a luz em sinais elétricos, e o ISP processa esses sinais em tempo real — aplicando operações como redução de ruído, balanço de branco e outros ajustes de imagem.

Ver na Figure 3.31 uma simulação deste EP.

3.12.6.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas), C (colunas) e N (tamanho do *kernel*, sempre ímpar).
2. **Dados:** Ler a matriz de pixels f .
3. **Filtro de Média:** Para cada pixel (i, j) **interno** (sem bordas), calcular:

$$g(i, j) = \text{round} \left(\frac{1}{N^2} \sum_{s=-(r)}^r \sum_{t=-(r)}^r f(i+s, j+t) \right), \quad r = \lfloor N/2 \rfloor$$

4. **Tratamento de Borda:** Pixels na borda (onde a janela $N \times N$ ultrapassa os limites) devem ser **copiados diretamente** do original sem modificação.
5. **Saída:** Exibir a matriz resultante $L \times C$.

3.12.6.2 Restrições Computacionais

- **Raio:** $r = \lfloor N/2 \rfloor$ (metade do *kernel*, inteiro).
- **Pixels internos:** (i, j) com $r \leq i < L - r$ e $r \leq j < C - r$.
- **Arredondamento:** Usar arredondamento matemático antes de converter para inteiro.
- **Sem *clipping*:** A média de valores $\in [0, 255]$ permanece em $[0, 255]$.

3.12.6.3 Fundamentação Teórica

Tamanho N	Coefficiente	Pixels na janela	Efeito
3	$1/9 \approx 0.111$	9	Suave
5	$1/25 = 0.04$	25	Médio
7	$1/49 \approx 0.020$	49	Forte

3.12.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro N (ímpar, $N \geq 3$).
- Linhas seguintes: Elementos da matriz original.

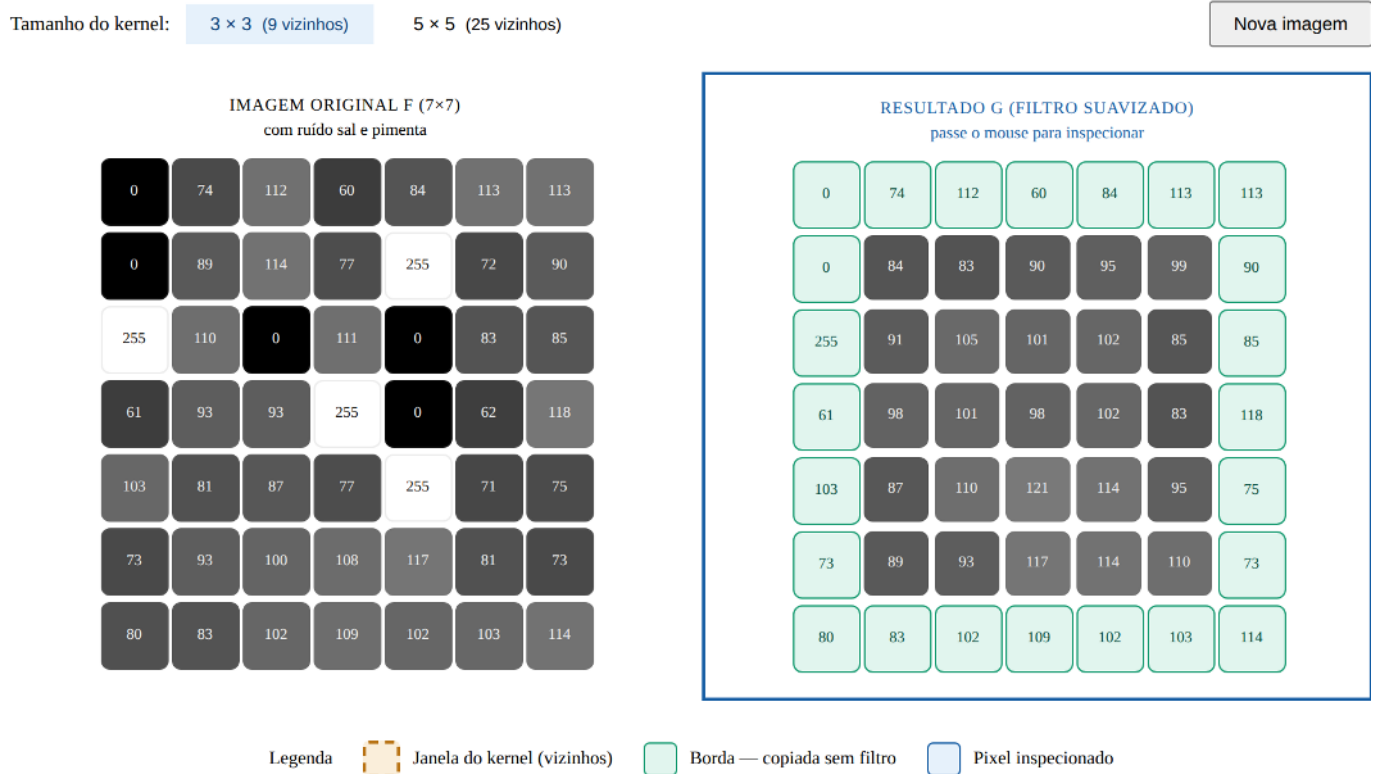
Saída:

- Matriz filtrada $L \times C$.

3.12.6.5 Exemplos

Entrada	Saída	Observação
3	10 20 30	Apenas borda ($3 \times 3 =$ borda total)
3	40 50 60	
3	70 80 90	
10 20 30		
40 50 60		
70 80 90		
5	0 0 0 0 0	Pixel isolado: todos os 9 pixels internos cuja janela 3×3 inclui o valor 100 recebem $\text{round}(100/9)=11$
5	0 11 11 11 0	
3	0 11 11 11 0	
0 0 0 0 0	0 11 11 11 0	
0 0 0 0 0	0 0 0 0 0	
0 0 100 0 0		
0 0 0 0 0		
0 0 0 0 0		

Simulador de filtro de média com kernel 3×3 ou 5×5 : passe o mouse nas células do resultado para visualizar a janela de vizinhança.



Passe o mouse sobre um pixel interno do resultado para ver o cálculo.

Figura 3.31: Simulador: Filtro de Média com *Kernel* $N \times N$

```
%%writefile EP03_06.py
# Código Python
```

Writing EP03_06.py

```
TestSuite("EP03_06.py").run()
```

3.12.7 EP03_07 🔍 Operador Laplaciano (w4) para Realce de Bordas

Em tomografias de alta resolução, a nitidez das bordas entre tecidos é crítica para diagnóstico. O **operador Laplaciano** é amplamente utilizado em *pipelines* de pré-processamento de imagens médicas para realçar automaticamente os contornos anatômicos antes da segmentação, evitando intervenção manual do radiologista.

Ver na Figure 3.32 uma simulação deste EP.

3.12.7.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f .
3. **Laplaciano (w4):** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$, calcular:

$$\nabla^2 f(i, j) = f(i - 1, j) + f(i + 1, j) + f(i, j - 1) + f(i, j + 1) - 4 \cdot f(i, j)$$

4. **Realce:** Calcular a imagem realçada:

$$g(i, j) = \text{clip}(f(i, j) - \nabla^2 f(i, j))$$

5. **Borda:** Pixels na borda são copiados diretamente: $g(i, j) = f(i, j)$.
6. **Saída:** Exibir a matriz realçada $L \times C$.

3.12.7.2 📌 Restrições Computacionais

- **Kernel w4:** $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ — apenas 4-vizinhos.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$ aplicado ao resultado do realce.
- **Sem arredondamento:** O Laplaciano usa apenas somas/subtrações de inteiros.

3.12.7.3 🧠 Fundamentação Teórica

Região	$\nabla^2 f$	Efeito do Realce
Uniforme	≈ 0	Sem alteração
Borda crescente	< 0	Pixel clareado
Borda decrescente	> 0	Pixel escurecido

3.12.7.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz realçada $L \times C$.

3.12.7.5 Exemplos

Entrada	Saída	Observação
3 3 0 0 0 0 100 0 0 0 0	0 0 0 0 255 0 0 0 0	Pico isolado: $\text{lap}=-400$, $g=100-(-400)=500 \rightarrow \text{clip}=255$
3 3 50 50 50 50 50 50 50 50 50	50 50 50 50 50 50 50 50 50	Região uniforme: Laplaciano=0, sem alteração

```
%%writefile EP03_07.py
# Código Python
```

Writing EP03_07.py

```
TestSuite("EP03_07.py").run()
```

3.12.8 EP03_08 Gradiente de Sobel: Gx e Gy

Em robôs exploradores de Marte (como o Perseverance), a detecção de obstáculos é realizada em tempo real por câmeras estereoscópicas. O **operador de Sobel** calcula o gradiente direcional da cena e é utilizado no algoritmo de detecção de bordas para identificar rochas, fissuras e desníveis do terreno que possam comprometer a navegação.

Ver na Figure 3.33 uma simulação deste EP.

3.12.8.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz f .
3. **Gx e Gy:** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$:

$$G_x(i, j) = [f(i - 1, j + 1) + 2f(i, j + 1) + f(i + 1, j + 1)] - [f(i - 1, j - 1) + 2f(i, j - 1) + f(i + 1, j - 1)]$$

$$G_y(i, j) = [f(i + 1, j - 1) + 2f(i + 1, j) + f(i + 1, j + 1)] - [f(i - 1, j - 1) + 2f(i - 1, j) + f(i - 1, j + 1)]$$

4. **Magnitude:** $|\nabla f(i, j)| = \text{clip}(\text{round}(\sqrt{G_x^2 + G_y^2}))$.
5. **Borda:** Pixels de borda recebem magnitude 0.
6. **Saída:** Exibir a magnitude $L \times C$.

Simulador do operador Laplaciano para realce de bordas: imagem original, laplaciano e resultado.

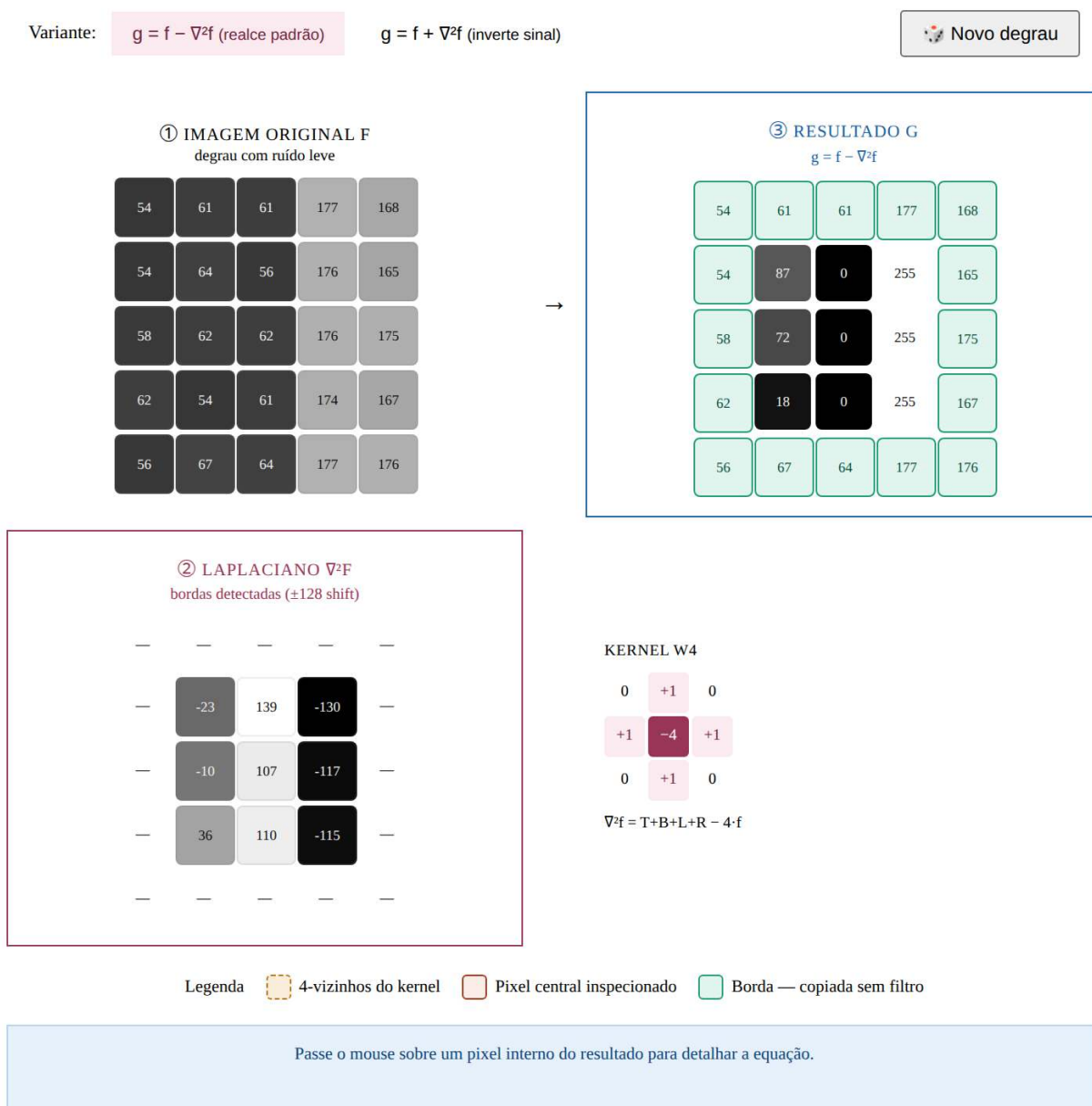


Figura 3.32: Simulador: Operador Laplaciano (w4) para Realce de Bordas

3.12.8.2 📌 Restrições Computacionais

- **Arredondamento:** Aplicar round antes de converter para inteiro.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$.
- **Raiz quadrada:** Usar $\sqrt{G_x^2 + G_y^2}$ (não a aproximação $|G_x| + |G_y|$).

3.12.8.3 🧠 Fundamentação Teórica

Operador	Detecta	Coeficientes diagonais
G_x	Bordas verticais	± 1
G_y	Bordas horizontais	± 1
$ \nabla f $	Todas as bordas	Combinado

3.12.8.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz.

Saída:

- Magnitude do gradiente, matriz $L \times C$.

3.12.8.5 📌 Exemplos

Entrada	Saída	Observação
3	0 0 0	Imagem nula: gradiente zero
3	0 0 0	
0 0 0	0 0 0	
0 0 0	0 0 0	
3	0 0 0	Borda vertical central: Gx alto
3	0 255 0	
0 0 255	0 0 0	
0 0 255		
0 0 255		

```
%%writefile EP03_08.py
# Código Python
```

```
Writing EP03_08.py
```

```
TestSuite("EP03_08.py").run()
```

Simulador do gradiente de Sobel: imagem original, Gx, Gy e magnitude do gradiente.

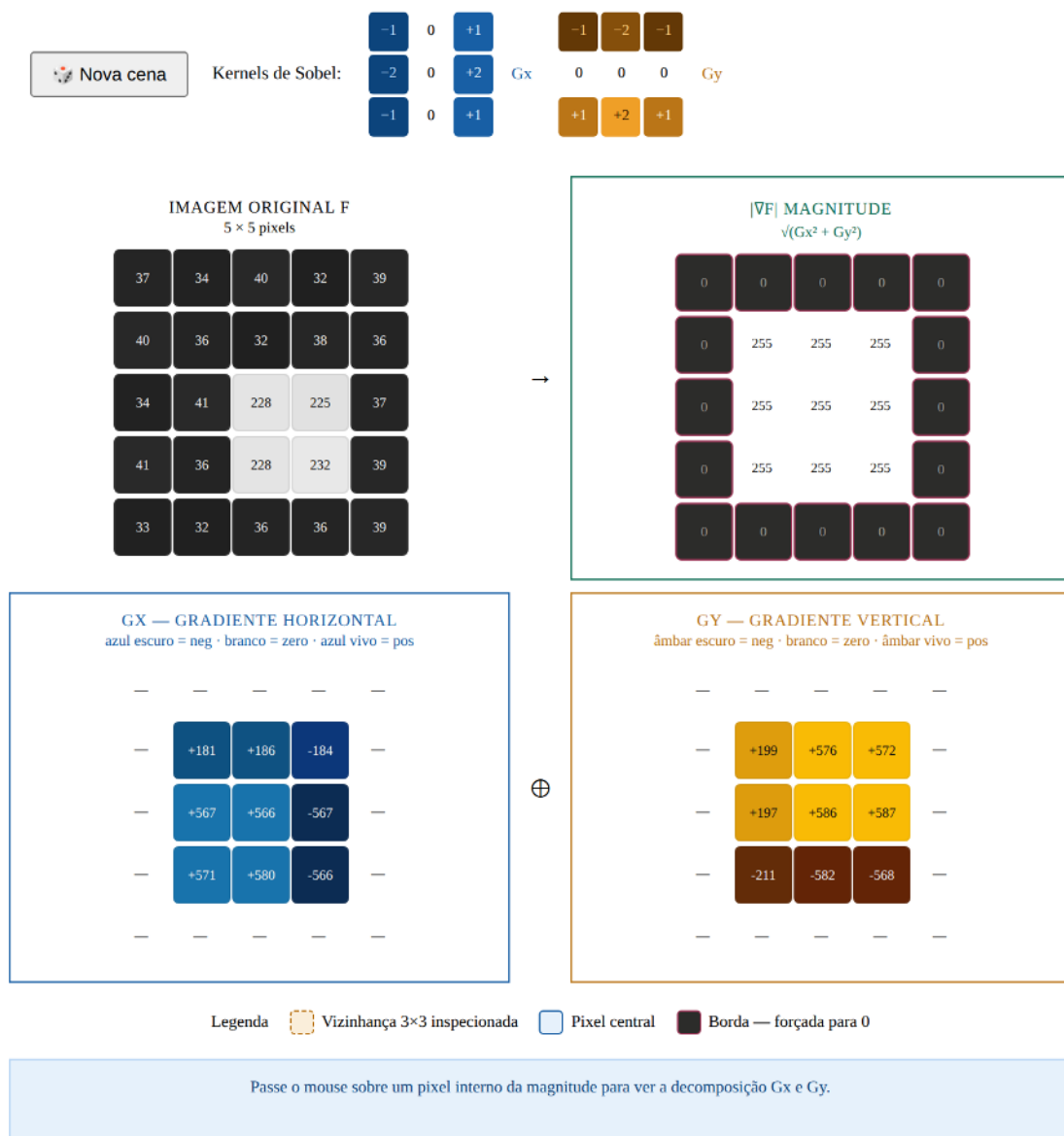


Figura 3.33: Simulador: Gradiente de Sobel: Gx e Gy

3.12.9 EP03_09 Filtro da Mediana 3×3

Imagens de radar de abertura sintética (SAR) usadas em monitoramento ambiental e militar sofrem de um tipo específico de ruído chamado *speckle*, que possui características similares ao ruído sal e pimenta. O **filtro da mediana** é o método padrão de remoção desse ruído porque preserva as bordas das estruturas enquanto elimina os pontos espúrios.

Ver na Figure 3.34 uma simulação deste EP.

3.12.9.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler a matriz de pixels f .
3. **Filtro da Mediana 3×3:** Para cada pixel **interno** (i, j) com $1 \leq i < L - 1$, $1 \leq j < C - 1$:
 - Coletar os 9 pixels da vizinhança 3×3 : $\{f(i + s, j + t) : s, t \in \{-1, 0, 1\}\}$.
 - Ordenar os 9 valores em ordem crescente.
 - Atribuir $g(i, j)$ ao valor central (posição índice 4, considerando índice 0).

$$g(i, j) = \text{mediana}\{f(i + s, j + t) : s, t \in \{-1, 0, 1\}\}$$

4. **Borda:** Copiar diretamente: $g(i, j) = f(i, j)$.
5. **Saída:** Exibir a matriz filtrada $L \times C$.

3.12.9.2 Restrições Computacionais

- **Janela:** Sempre $3 \times 3 = 9$ elementos.
- **Mediana:** O elemento central da sequência ordenada (índice 4 de 0 a 8).
- **Sem clipping:** A mediana de valores em $[0, 255]$ permanece em $[0, 255]$.
- **Não linear:** O filtro mediana não pode ser expresso como convolução linear.

3.12.9.3 Fundamentação Teórica

Ruído	Filtro de Média	Filtro de Mediana
Sal e pimenta (0 ou 255)	Espalha o ruído	Remove sem distorcer bordas
Gaussiano	Reduz eficazmente	Reduz parcialmente

3.12.9.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos da matriz.

Saída:

- Matriz filtrada $L \times C$.

3.12.9.5 Exemplos

Entrada	Saída	Observação
3	100 100 100	Ponto preto eliminado: mediana de $8 \times 100 + 1 \times 0 = 100$
3	100 100 100	
100 100 100	100 100 100	
100 0 100		
100 100 100		
3	50 50 50	Ponto branco (sal) eliminado
3	50 50 50	
50 50 50	50 50 50	
50 255 50		
50 50 50		

Injetar ruído impulsivo

Ruído sal (255) e pimenta (0) — ~30% dos pixels internos afetados

IMAGEM F — COM RUÍDO
sal (255) e pimenta (0) visíveis

RESULTADO G — SEM RUÍDO
passe o mouse para inspecionar

VETOR DE VIZINHANÇA 3x3 — ORDENADO
passe o mouse num pixel interno do resultado para ver

Legenda

Janela 3x3 capturada

Pixel central inspecionado

Borda — copiada sem filtro

Mediana (posição central)

Ruído (0 ou 255) — eliminado

Passe o mouse sobre um pixel interno do resultado para ver o processo de ordenação.

Figura 3.34: Simulador: Filtro da Mediana 3x3

```
%%writefile EP03_09.py
# Código Python

Writing EP03_09.py

TestSuite("EP03_09.py").run()
```

3.12.10 EP03_10 ✨ *Unsharp Masking* (USM)

Em sistemas de digitalização de documentos históricos e obras de arte, a nitidez das imagens é fundamental para leitura de textos manuscritos e detalhes ornamentais. O *Unsharp Masking* (USM) é o algoritmo de

realce de nitidez padrão utilizado em *scanners* profissionais e softwares como Adobe Photoshop, controlado pelo parâmetro k que determina a intensidade do realce.

Ver na Figure 3.35 uma simulação deste EP.

3.12.10.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Parâmetro:** Ler o valor real k (intensidade do realce, $k \geq 0$).
3. **Dados:** Ler a matriz de pixels f .
4. **Suavização:** Calcular $\bar{f}$ com filtro de média 3×3 (apenas pixels internos; bordas mantidas):

$$\bar{f}(i, j) = \frac{1}{9} \sum_{s=-1}^1 \sum_{t=-1}^1 f(i+s, j+t)$$

5. **Máscara de alta frequência:** $m(i, j) = f(i, j) - \bar{f}(i, j)$.
6. **Realce USM:** Para cada pixel interno:

$$g(i, j) = \text{clip}(\text{round}(f(i, j) + k \cdot m(i, j)))$$

7. **Borda:** $g(i, j) = f(i, j)$ (cópia direta).
8. **Saída:** Exibir a matriz realçada $L \times C$.

3.12.10.2 Restrições Computacionais

- **Arredondamento:** Aplicar `round` antes do clipping.
- **Saturação:** $\text{clip}(x) = \max(0, \min(255, x))$.
- **Operações em float:** Calcular $\bar{f}$ e m em ponto flutuante antes de arredondar o resultado final.
- $k = 0$: Sem realce — a saída é idêntica à entrada (exceto pelas bordas).

3.12.10.3 Fundamentação Teórica

Etapas	Operação	Descrição
1	$\bar{f} = f * \frac{1}{9} \mathbf{1}_{3 \times 3}$	Suavização (baixas frequências)
2	$m = f - \bar{f}$	Máscara (altas frequências)
3	$g = \text{clip}(\text{round}(f + k \cdot m))$	Realce ponderado

3.12.10.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Real k .
- Linhas seguintes: Elementos da matriz original.

Saída:

- Matriz realçada $L \times C$.

3.12.10.5 📌 Exemplos

Entrada	Saída	Observação
3	100 100 100	k=0: sem realce
3	100 100 100	
0.0	100 100 100	
100 100 100		
100 100 100		
100 100 100		
3	50 50 50	k=1: pixel central realçado e saturado
3	50 255 50	
1.0	50 50 50	
50 50 50		
50 200 50		
50 50 50		

```
%%writefile EP03_10.py
# Código Python

Writing EP03_10.py

TestSuite("EP03_10.py").run()
```

Simulador de Unsharp Masking (USM): visualização do pipeline com imagem original, versão desfocada, máscara de alta frequência e resultado realçado.

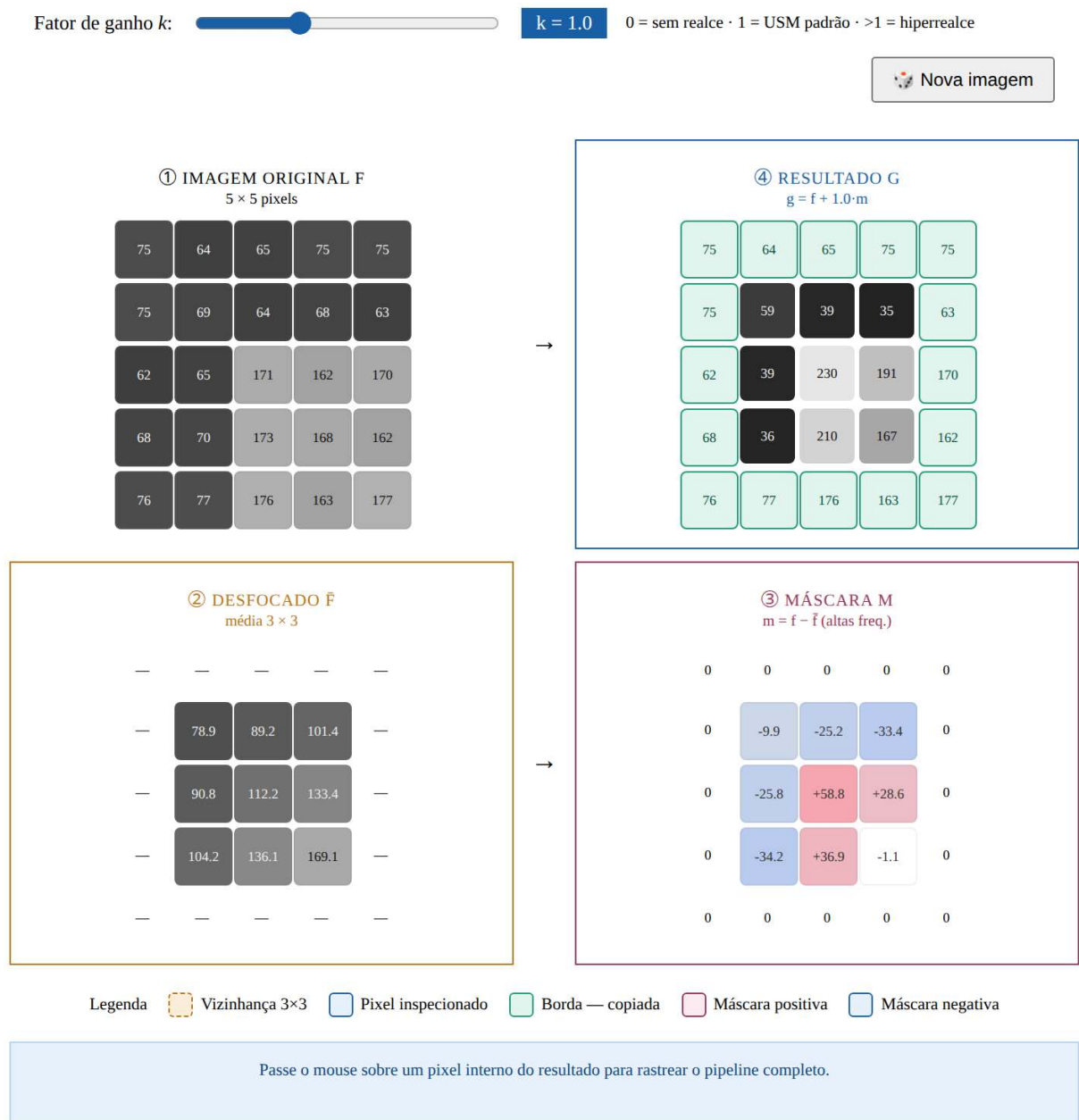


Figura 3.35: Simulador: *Unsharp Masking* (USM)

Capítulo 4

Morfologia Matemática e Segmentação de Imagens



Este capítulo apresenta dois temas fundamentais do Processamento Digital de Imagens (PDI): a **morfologia matemática** e a **segmentação de imagens**. A morfologia matemática fornece um arcabouço teórico baseado na teoria dos conjuntos para analisar, refinar e quantificar a forma de objetos em imagens binárias e em tons de cinza, por meio de operadores fundamentais como **erosão** e **dilatação**. A segmentação, por sua vez, tem como objetivo particionar a imagem em regiões de interesse, separando objetos do fundo e produzindo representações adequadas para análise e interpretação.

O capítulo inicia com a **limiarização**, uma das técnicas mais importantes de segmentação, introduzindo o método automático de **Otsu** e revisitando a análise de histogramas por meio da variância interclasses, apresentada no Capítulo 1. Em seguida, são estudados os principais operadores da morfologia matemática, incluindo erosão, dilatação, abertura, fechamento e reconstrução morfológica, que permitem refinar máscaras binárias e preservar estruturas relevantes dos objetos. Por fim, são apresentadas técnicas de segmentação baseada em regiões, como a **rotulação de componentes conexos**, a **transformada de distância** e o algoritmo **watershed** baseado em marcadores, culminando na extração de descritores geométricos e na geração de *bounding boxes* compatíveis com sistemas modernos de detecção de objetos.

4.1 Objetivos

Ao final deste capítulo, você será capaz de:

- **Aplicar limiarização:** Compreender o critério automático de Otsu por maximização da variância interclasses (σ_B^2) e selecionar estratégias adequadas de pré-processamento para facilitar a segmentação;
- **Dominar morfologia binária:** Compreender e aplicar erosão ($A \ominus B$) e dilatação ($A \oplus B$) como operadores fundamentais, derivando abertura ($A \circ B$), fechamento ($A \bullet B$) e operações baseadas em reconstrução morfológica, como `mm.clohole` e `mm.edgeoff`;
- **Aplicar morfologia em tons de cinza:** Utilizar gradiente morfológico e filtros *top-hat* para realce e análise de estruturas locais;
- **Rotular componentes conexos:** Identificar e separar regiões conectadas em imagens binárias por meio de algoritmos de rotulação;
- **Aplicar transformada de distância:** Interpretar e calcular distâncias ao fundo utilizando abordagens morfológicas e métricas geométricas;
- **Segmentar por regiões:** Construir *pipelines* de segmentação baseados em marcadores utilizando Transformada de Distância e o algoritmo **watershed**;
- **Extraír descritores geométricos:** Calcular propriedades como área, perímetro, centróide, circularidade e *bounding boxes* por meio de `cv2.connectedComponentsWithStats` e `cv2.findContours`;
- **Relacionar PDI e visão computacional:** Compreender como descritores extraídos por segmentação podem ser convertidos para formatos utilizados por detectores modernos, como YOLO.

```
import os, importlib, urllib.request
import numpy as np
import matplotlib.pyplot as plt

BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph
importlib.reload(morph)
from morph import mm

version = getattr(morph, "__version__", "local_file")
print(f" Ambiente pronto. Módulo 'morph' carregado (versão: {version}).")
```

Ambiente pronto. Módulo 'morph' carregado (versão: 1.1.5).

4.2 Limiarização

A **limiarização** (*thresholding*) é uma das formas mais simples e eficientes de segmentação de imagens. Seu objetivo é classificar cada pixel em duas classes de intensidade, normalmente associadas a *objeto* e *fundo*:

$$g(x, y) = \begin{cases} 255, & \text{se } f(x, y) > T \\ 0, & \text{caso contrário} \end{cases} \quad (4.1)$$

em que $f(x, y)$ representa a intensidade do pixel na imagem original e $g(x, y)$ a imagem binária resultante.

A escolha do limiar T é importante para a qualidade da segmentação. O método de **Otsu** determina automaticamente o limiar ótimo ao maximizar a **variância interclasses** σ_B^2 e definida por:

$$\sigma_B^2(T) = w_0(T) w_1(T) [\mu_0(T) - \mu_1(T)]^2 \quad (4.2)$$

em que:

- $w_0(T)$ e $w_1(T)$ são as probabilidades acumuladas das classes fundo e objeto;
- $\mu_0(T)$ e $\mu_1(T)$ são as médias de intensidade dessas classes;
- $\sigma_B^2(T)$ representa a variância interclasses para um dado limiar T .

O método funciona melhor quando o histograma apresenta dois grupos de intensidades relativamente separados. Para isso, o algoritmo avalia todos os limiares possíveis da imagem — tipicamente no intervalo $[0, 255]$ para imagens de 8 bits — e seleciona o valor que maximiza a variância entre classes, denotada por σ_B^2 :

$$T^* = \arg \max_{T \in [0, 255]} \sigma_B^2(T)$$

i Otsu assume histogramas bimodais

O método de Otsu produz melhores resultados quando o histograma apresenta dois picos bem definidos (*bimodalidade*), correspondentes ao fundo e ao objeto. Quanto maior a separação entre esses picos e mais pronunciado o máximo de σ_B^2 , mais confiável tende a ser o limiar obtido.

Em imagens com iluminação não uniforme ou múltiplas regiões de intensidade, técnicas de **limiarização adaptativa** — nas quais o limiar é calculado localmente — costumam produzir segmentações mais robustas.

O índice B em σ_B^2 significa *between classes* (*entre classes*). Assim, σ_B^2 representa a **variância entre as classes** (*between-class variance*).

4.2.1 Imagem de Moedas

A imagem utilizada para praticar a segmentação é uma fotografia de coleção de moedas de diferentes países e épocas (Figure 4.1). Crédito: GAZI.MD.AHAD (CC BY-SA 4.0). Ela apresenta objetos circulares com bordas bem definidas, sendo ideal para demonstrar limiarização, operadores morfológicos, transformada de distância, *watershed* e descritores de forma.

```
import os

url      = "https://upload.wikimedia.org/wikipedia/commons/2/25/GAZI.MD.AHAD_11.jpg"
caminho = "imagens/coins.jpg"

if not os.path.exists(caminho):
    os.makedirs("imagens", exist_ok=True)
    img_obj = mm.read(url, pil=True)
    mm.write(img_obj, caminho)
else:
    img_obj = mm.read(caminho, pil=True)

img_coins_color = np.array(img_obj)
img_coins_gray  = mm.gray(img_coins_color)

print(f"Dimensões [y,x,c]: {img_coins_color.shape}")
mm.show(img_coins_color, scale=30)
```

Dimensões [y,x,c]: (2560, 1920, 3)



Figura 4.1: Imagem com moedas de vários tipos. Crédito: GAZI.MD.AHAD (CC BY-SA 4.0).

4.2.2 Escolha do Pré-processamento para o Otsu

A qualidade do método de Otsu depende diretamente de quão **bimodal** é o histograma da imagem de entrada. A imagem original das moedas apresenta iluminação não uniforme e moedas escuras (cobre oxidado) com intensidades próximas às do fundo escuro, tornando o histograma pouco bimodal.

Para minimizar essas limitações, serão avaliadas duas técnicas clássicas de pré-processamento, apresentadas no capítulo anterior, aplicadas antes da etapa de limiarização. A Table 4.1 resume as características de cada abordagem. Essas técnicas buscam aumentar a separação entre objeto e fundo, tornando o histograma mais próximo de uma distribuição bimodal.

Tabela 4.1: Técnicas de pré-processamento avaliadas para melhorar a separação entre moedas e fundo antes da aplicação do método de Otsu.

Técnica	O que faz	Quando usar
CLAHE	Equalização de histograma adaptativa local	Baixo contraste global ou regional
Gaussiano	Suavização por convolução com gaussiana	Ruído de alta frequência (textura do fundo)

A função `cv2.createCLAHE(clipLimit, tileGridSize)` divide a imagem em blocos e aplica equalização de histograma em cada um deles, limitando a amplificação de ruído por meio do parâmetro `clipLimit`. Já o filtro Gaussiano (`cv2.GaussianBlur`) suaviza texturas finas que poderiam criar falsos picos no histograma.

Para comparar objetivamente qual pré-processamento produz a melhor entrada para o método de Otsu, serão apresentados, para cada versão da imagem, a própria imagem, o histograma com o limiar ótimo T^* destacado, a curva $\sigma_B^2(T)$ e o resultado da binarização.

i Critério de comparação

A versão que apresentar o maior valor de σ_B^2 em seu pico fornece a melhor separação entre as classes de fundo e objeto e, conseqüentemente, a melhor entrada para o método de Otsu (Figure 4.2).

```
import cv2, io

def otsu_criterio(img):
    h=mm.hist(img); p=h/h.sum(); # histograma e probabilidades
    sigma2=np.zeros(len(p))
    for T in range(1,len(p)): # percorre limiares
        w0,w1=p[:T].sum(),p[T:].sum() # probabilidades das classes
        if w0*w1==0: continue # evita divisão por zero
        mu0=(np.arange(T)*p[:T]).sum()/w0 # média fundo
        mu1=(np.arange(T,len(p))*p[T:]).sum()/w1 # média objeto
        sigma2[T]=w0*w1*(mu0-mu1)**2 #  $\sigma_B^2(T)$ 
    return sigma2,np.argmax(sigma2) # curva e T ótimo

def fig2img(fig):
    b=io.BytesIO(); fig.savefig(b,format='png',dpi=100) # figura → buffer
    plt.close(fig); b.seek(0)
    return np.array(plt.imread(b)) # buffer → array

def plot_curve(y,T,title,ylabel,color):
    fig,ax=plt.subplots(figsize=(4,3))
    ax.plot(y,color=color) if ylabel==" $\sigma_B^2$ " else ax.bar(range(len(y)),y,color=color,width=1)
    ax.axvline(T,color='red',lw=2,label=f"T*={T}") # limiar ótimo
    ax.set(xlabel="T" if ylabel==" $\sigma_B^2$ " else "Intensidade",ylabel=ylabel)
    ax.legend(fontsize=8); plt.tight_layout()
```



```

return fig2img(fig)

# Pré-processamentos
clahe=cv2.createCLAHE(clipLimit=2.0,tileGridSize=(8,8))
img_clahe = clahe.apply(img_coins_gray)
img_gauss = cv2.GaussianBlur(img_clahe,(5,5),0)
imgs0=[("Original",img_coins_gray),
        ("CLAHE",img_clahe),
        ("CLAHE+Gauss",img_gauss)]

# Tabela comparativa
print(f"{'Versão':<18}{'T*':>6}{'2B pico':>14}")
print("-"*40)

imgs,titles=[],[]
for nome,img in imgs0:
    sigma2,T=otsu_criterio(img) # calcula 2B(T)
    print(f"{'nome':<18}{'T':>6}{'sigma2[T]:>14.4e}")
    imgs += [
        img, # imagem
        plot_curve(mm.hist(img),T,f"Hist T*={T}", "Freq.", "steelblue"),
        plot_curve(sigma2,T,"2B(T)", "2B", "darkorange"),
        mm.threshold(img) # Otsu final
    ]
    titles += [nome,f"Hist T*={T}", "2B(T)",f"Otsu T*={T}"]

# Exibição final
mm.show(imgs,titles=titles,cols=4,figsize=(12,12),dpi=200)

```

Versão	T*	² B pico
Original	105	1.9437e+03
CLAHE	122	2.4695e+03
CLAHE+Gauss	123	2.4283e+03

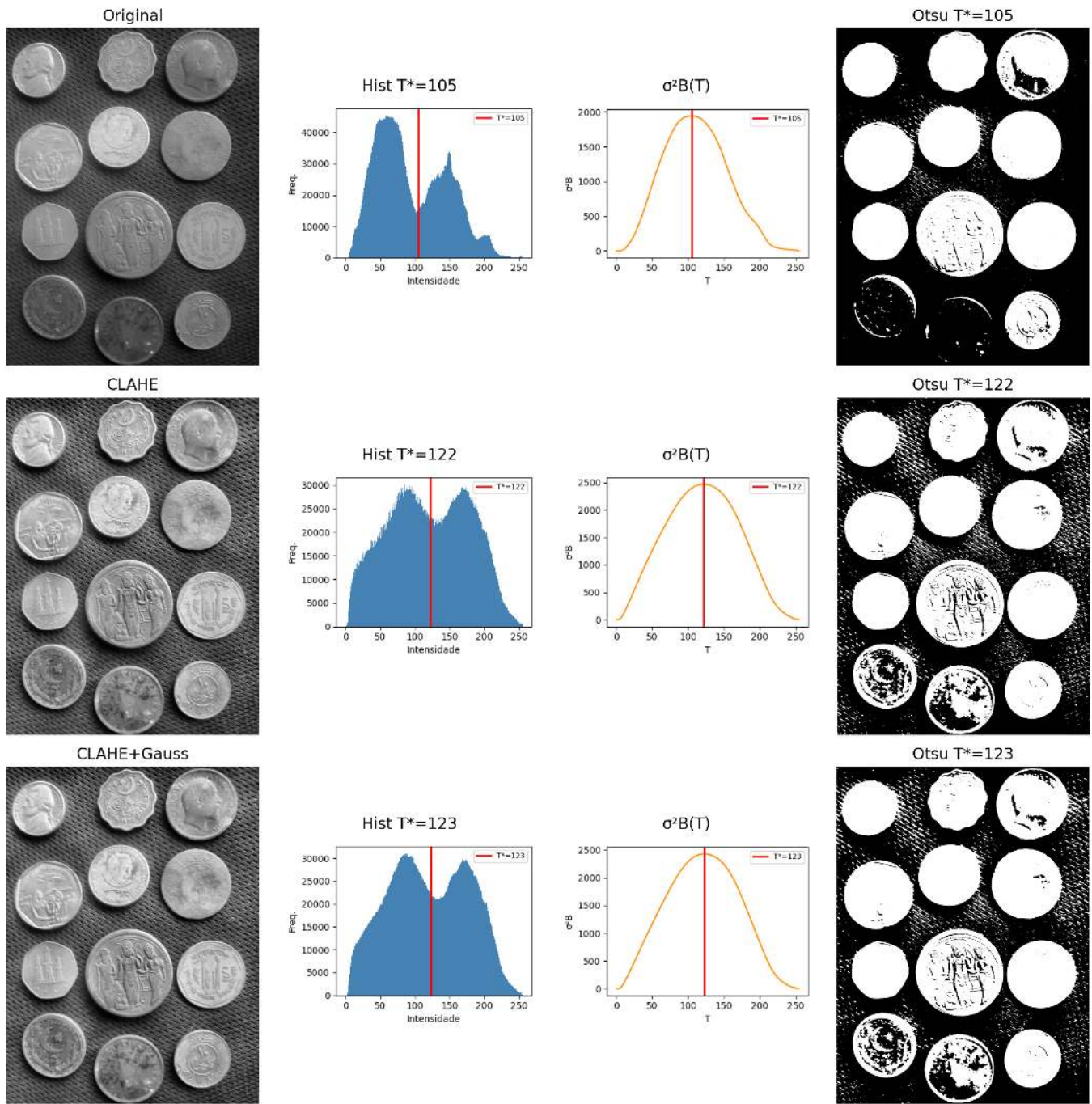


Figura 4.2: Comparação dos pré-processamentos: imagem | histograma+ T^* | $\sigma^2_B(T)$ | Otsu. A melhor separação bimodal indica o limiar mais confiável.

4.2.3 Resultado: CLAHE como Melhor Pré-processamento

A análise da Figure 4.2 indica que o **CLAHE** obteve o maior valor da variância inter-classes ($\sigma_B^2 \approx 2,47 \times 10^3$), com limiar ótimo $T^* = 122$. Embora a combinação **CLAHE+Gaussiano** tenha produzido resultado muito semelhante ($\sigma_B^2 \approx 2,43 \times 10^3$, $T^* = 123$), o critério quantitativo do método de Otsu favorece ligeiramente o uso do CLAHE isoladamente.

Em termos visuais, as imagens binarizadas obtidas com CLAHE e CLAHE+Gaussiano são praticamente equivalentes. A diferença entre as duas abordagens torna-se mais evidente na análise dos histogramas e dos valores de $\sigma_B^2(T)$ do que na inspeção direta das segmentações resultantes. Assim, a escolha do CLAHE baseia-se principalmente na maximização da separação estatística entre as classes de fundo e objeto.

💡 Interpretação dos resultados

Observe que os pré-processamentos com CLAHE e CLAHE+Gaussiano produzem histogramas e limiares ótimos muito próximos ($T^* = 122$ e $T^* = 123$). Consequentemente, as imagens binarizadas resultantes também são bastante semelhantes. Nesse caso, a decisão não é baseada em diferenças visuais marcantes, mas no critério objetivo do método de Otsu: o maior valor de σ_B^2 indica a melhor separação entre as classes.

4.3 Morfologia Matemática

A **morfologia matemática** é uma teoria baseada em conjuntos utilizada para analisar a forma e a estrutura de objetos em imagens. Diferentemente dos filtros lineares apresentados no Capítulo 3, os operadores morfológicos são **não lineares**, pois se baseiam em operações de mínimo, máximo e inclusão espacial, em vez de combinações lineares de intensidades. Esses operadores atuam sobre a vizinhança de cada pixel por meio de um **elemento estruturante** $\mathbb{B}$, responsável por definir a forma e o tamanho da região analisada.

Em imagens binárias e em tons de cinza com elementos planos, o elemento estruturante transladado para a posição x é definido espacialmente como:

$$\mathbb{B}_x = \{x + b \mid b \in \mathbb{B}\}$$

Nas regiões de borda da imagem, parte do conjunto $\mathbb{B}_x$ pode extrapolar o domínio físico da cena ($\mathbb{E}$). Para garantir a consistência matemática dos operadores primitivos nessas fronteiras, assume-se teoricamente que o espaço exterior ao domínio da imagem é preenchido com o **elemento neutro** da operação correspondente (infinito positivo para a erosão e infinito negativo para a dilatação), impedindo que o ambiente externo corrompa as estruturas internas do objeto.

Quando o elemento estruturante associa pesos a seus elementos — isto é, $b : \mathbb{B} \rightarrow \mathbb{Z}$ — ele é denominado **função estruturante** ou **elemento estruturante não plano**.

Desenvolvida por Matheron e Serra na década de 1960 para imagens binárias e posteriormente estendida a tons de cinza, a morfologia matemática fundamenta operadores como gradiente morfológico, *top-hat*, *watershed* e transformada de distância, todos derivados de dois primitivos: **erosão** e **dilatação** (Matheron, 1975; Serra, 1982).

4.3.1 Erosão e Dilatação

Os dois operadores primitivos são definidos de forma unificada para imagens em tons de cinza ($f : \mathbb{E} \rightarrow \mathbb{Z}$) e, por restrição ao domínio $\{0, 1\}$, também para imagens binárias.

4.3.1.1 Erosão

A **Erosão** de uma imagem f por uma função estruturante $b : \mathbb{B} \rightarrow \mathbb{Z}$ é definida formalmente por:

$$\varepsilon_b(f)(x) = (f \ominus b)(x) = \min_{z \in \mathbb{B}} \{ f(x + z) - b(z) \}, \quad \forall x \in \mathbb{E} \quad (4.3)$$

Na prática, a erosão substitui a intensidade do pixel x pelo mínimo valor resultante da diferença entre a imagem e o elemento estruturante na vizinhança definida pelo domínio $\mathbb{B}$. Valores positivos nos pesos de $b(z)$ forçam o resultado local para baixo, “cavando” o relevo da imagem mais profundamente e intensificando a erosão.

No caso **plano** (onde os pesos são nulos dentro do domínio, ou seja, $b \equiv 0$), a expressão simplifica-se para o mínimo local puro:

$$\varepsilon_B(f)(x) = \min\{f(y) : y \in \mathbb{B}_x\}$$

Em imagens binárias, essa operação equivale a exigir que o conjunto $\mathbb{B}$, transladado para a coordenada x , esteja **completamente contido** no objeto A :

$$A \ominus \mathbb{B} = \{z \in \mathbb{E} \mid \mathbb{B}_z \subseteq A\}$$

Efeito Visual: *Encolhe* objetos e estruturas claras, eliminando protuberâncias, picos brilhantes ou ruídos que sejam geometricamente menores que o domínio $\mathbb{B}$.

4.3.1.2 Implementação da erosão

A versão didática `mm.ero0` implementa o caso particular de erosão com **elemento estruturante plano**. Para cada pixel (y, x) , a função percorre os vizinhos espaciais permitidos por B e armazena o menor valor encontrado na imagem de entrada f , reproduzindo diretamente a operação de mínimo local descrita na Equation 4.3 para $b \equiv 0$.

Observe que os valores dos vizinhos são sempre lidos de forma estática da imagem original f ; a matriz de saída g é utilizada exclusivamente para registrar o mínimo acumulado da vizinhança corrente. Dessa forma, o resultado final é invariante em relação à ordem de varredura dos pixels (seja por linhas ou colunas).

A função auxiliar `_viz` calcula as coordenadas dos vizinhos válidos dentro dos limites físicos da imagem. Nas bordas, a inicialização do acumulador em 255 emula com exatidão o preenchimento por elemento neutro exigido pela teoria. Já a função de interface `mm.ero` recorre à implementação nativa e otimizada do OpenCV (`cv2.erode`) quando o elemento estruturante é plano, chaveando para a rotina geral `mm.ero1` caso o elemento possua pesos topográficos.

O exemplo computacional a seguir ilustra a aplicação de um elemento estruturante em cruz (`mm.secross()`) em destaque na Figure 4.3, comparando a execução da variante didática em laço (`mm.ero0`) com o motor computacional do OpenCV (`mm.ero`).

```
B_cruz = mm.secross()
mm.drawImgPlt(B_cruz, scale=20)
```

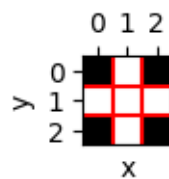


Figura 4.3: Elemento estruturante em formato de cruz (B_{cruz}) utilizado para conectividade-4.

```
def _viz(f, B, y, x):
    """Gera (vy, vx, b_val) para cada vizinho válido de (y,x)."""
    H, W = f.shape
    Bh, Bw = B.shape
    oh, ow = -Bh/2 + 0.5, -Bw/2 + 0.5 # offsets fixos
    for by, bx in np.ndindex(Bh, Bw):
        vy, vx = int(y + by + oh), int(x + bx + ow)
        if 0 <= vy < H and 0 <= vx < W:
            yield vy, vx, B[by, bx]

def ero(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Erosão (OpenCV ou com pesos)."""
    try:
        return cv2.erode(f, Bc)
```

```

except: return mm.ero1(f, Bc)

def ero0(f, Bc=np.ones((3,3),dtype='uint8')):
    """Erosão clássica sem pesos."""
    g = np.empty_like(f)
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            g[y,x] = 255
            for vy,vx,bv in mm._viz(f,Bc,y,x):
                if bv and g[y,x] > f[vy,vx]: g[y,x] = f[vy,vx]
    return g

# Script de Teste e Validação
B = mm.seccross()
print("Elemento estruturante B:")
print(mm.drawImage(B))

f = mm.randomImage(5,5)
print("Imagem original f:")
print(mm.drawImage(f))

print("Erosão Plana Didática (ero0):")
print(mm.drawImage(ero0(f, B)))

print("Erosão Otimizada OpenCV (ero):")
print(mm.drawImage(ero(f, B)))

```

```

Elemento estruturante B:
0 1 0
1 1 1
0 1 0

Imagem original f:
8 0 3 5 3
1 9 9 8 1
8 2 2 5 4
9 9 7 2 9
6 9 7 1 9

Erosão Plana Didática (ero0):
0 0 0 3 1
1 0 2 1 1
1 2 2 2 1
6 2 2 1 2
6 6 1 1 1

Erosão Otimizada OpenCV (ero):
0 0 0 3 1
1 0 2 1 1
1 2 2 2 1
6 2 2 1 2
6 6 1 1 1

```

A função `_viz` utiliza `yield` para gerar cada vizinho sob demanda, sem armazenar todos os resultados em memória em uma lista. No experimento abaixo, uma janela de 3000×3000 produz 9 milhões de vizinhos. A implementação baseada em lista consumiu mais de 1 GB de RAM e levou aproximadamente 46 s, enquanto a versão com `yield` consumiu memória desprezível e executou em 38 s. Em processamento de imagens, geradores são especialmente úteis para percorrer grandes vizinhanças de forma eficiente.

```
import tracemalloc

def lista(n): return [(i, i) for i in range(n)]
def gera(n):
    for i in range(n): yield i, i

for nome, f in [("LISTA", lista), ("YIELD", gera)]:
    tracemalloc.start()
    sum(x+y for x,y in f(9_000_000))
    _, pico = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print(f"{nome}: {pico/1024/1024:.1f} MB")
```

```
LISTA: 830.8 MB
YIELD: 0.0 MB
```

4.3.1.3 Dilatação

A **Dilatação** de uma imagem f por uma função estruturante $b : \mathbb{B} \rightarrow \mathbb{Z}$ é definida formalmente por:

$$\delta_b(f)(x) = (f \oplus b)(x) = \max_{z \in \mathbb{B}} \{ f(x - z) + b(z) \}, \quad \forall x \in \mathbb{E} \quad (4.4)$$

Na prática, a dilatação substitui a intensidade do pixel x pelo maior valor resultante da soma entre a imagem e o elemento estruturante na vizinhança definida. O argumento de inversão espacial ($x - z$) indica que a dilatação avalia implicitamente o elemento transposto (refletido) $\hat{b}$, propriedade fundamental para assegurar a dualidade matemática em relação à erosão.

No caso **plano** (onde os pesos são nulos dentro do domínio, ou seja, $b \equiv 0$), a expressão reduz-se ao máximo local puro:

$$\delta_B(f)(x) = \max \{ f(y) : y \in \mathbb{B}_x \}$$

Em imagens binárias, essa operação equivale a exigir que o conjunto refletido $\hat{\mathbb{B}}$, transladado para a coordenada x , possua **interseção não vazia** com o objeto A :

$$A \oplus \mathbb{B} = \{ z \in \mathbb{E} \mid \hat{\mathbb{B}}_z \cap A \neq \emptyset \}$$

Efeito Visual: *Expand*e as estruturas claras da imagem, aumentando o preenchimento de objetos, conectando componentes próximos e eliminando canais, fossas escuras ou vales que sejam geometricamente menores que o domínio $\mathbb{B}$.

4.3.1.4 Implementação da dilatação

A versão didática `mm.dil0` implementa o caso particular de dilatação com **elemento estruturante plano**. Para cada pixel (y, x) , a função percorre os vizinhos espaciais permitidos por B e armazena o maior valor encontrado na imagem de entrada f , reproduzindo diretamente a operação de máximo local para $b \equiv 0$.

Antes de iniciar a varredura espacial, o elemento estruturante sofre uma reflexão geométrica por meio da instrução `np.flip(Bc)` para construir explicitamente a matriz transposta $\hat{B}$ exigida pela teoria. Em máscaras perfeitamente simétricas (como cruzeiros, quadrados e discos centrados na origem), essa reflexão não altera o arranjo dos pixels; contudo, para elementos assimétricos, tal etapa é estritamente necessária para garantir a equivalência com as definições formais e salvaguardar as leis de dualidade.

Assim como verificado no operador de erosão, os valores dos vizinhos são sempre lidos de forma estática a partir da matriz original f , enquanto a matriz de saída g atua puramente como o registrador do máximo acumulado da vizinhança. Nas fronteiras da imagem, a inicialização do acumulador em 0 emula com exatidão

o preenchimento externo por elemento neutro ($-\infty$, ou zero em representações de 8 bits), garantindo que as bordas físicas da cena sejam dilatadas em perfeita conformidade com o padrão adotado pelo OpenCV.

O exemplo computacional abaixo ilustra a aplicação prática de um elemento em cruz (`mm.secross()`), validando a consistência entre a lógica em laços (`mm.dil0`) e o método nativo industrial (`mm.dil`).

```
def dil(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Dilatação (OpenCV ou com pesos)."""
    try:    return cv2.dilate(f, Bc)
    except: return mm.dil1(f, Bc)

def dil0(f, Bc=np.zeros((3,3),dtype='uint8')):
    """Dilatação plana seguindo rigorosamente a teoria."""
    g = np.empty_like(f)
    Bc = np.flip(Bc)      # reflexão explícita: B
    for y in range(f.shape[0]):
        for x in range(f.shape[1]):
            g[y,x] = 0 # Inicializa com o valor mínimo para buscar o máximo
            for vy,vx,bv in mm._viz(f,Bc,y,x):
                if bv and g[y,x] < f[vy,vx]:
                    g[y,x] = f[vy,vx]
    return g

# Script de Teste e Validação
B = mm.secross()
print("Elemento estruturante B:")
print(mm.drawImage(B))

f = mm.randomImage(5,5)
print("Imagem original f:")
print(mm.drawImage(f))

print("Dilatação Plana Didática (dil0):")
print(mm.drawImage(dil0(f, B)))

print("Dilatação Otimizada OpenCV (dil):")
print(mm.drawImage(dil(f, B)))
```

```
Elemento estruturante B:
0 1 0
1 1 1
0 1 0

Imagem original f:
9 2 0 8 4
9 3 8 6 1
1 4 1 1 3
2 9 4 9 6
6 8 6 8 7

Dilatação Plana Didática (dil0):
9 9 8 8 8
9 9 8 8 6
9 9 8 9 6
9 9 9 9 9
8 9 8 9 8

Dilatação Otimizada OpenCV (dil):
9 9 8 8 8
9 9 8 8 6
```

```

9 9 8 9 6
9 9 9 9 9
8 9 8 9 8

```

i Nota: O Confronto dos Sinais ($f(x+z)$ vs $f(x-z)$)

Compare as definições formais da erosão (Equation 4.3) e da dilatação (Equation 4.4). Considere um elemento estruturante assimétrico à direita $\mathbb{B} = \{0, 1\}$ (origem e um pixel à direita) aplicado na posição $x = 10$.

1. Na Erosão (Equation 4.3):

$$\min\{f(x+z) - b(z)\}$$

- $z = 0 \Rightarrow f(10+0) = \mathbf{f(10)}$
- $z = 1 \Rightarrow f(10+1) = \mathbf{f(11)}$

O operador consulta o pixel atual (10) e o pixel à direita (11), preservando a orientação original de $\mathbb{B}$.

2. Na Dilatação (Equation 4.4):

$$\max\{f(x-z) + b(z)\}$$

- $z = 0 \Rightarrow f(10-0) = \mathbf{f(10)}$
- $z = 1 \Rightarrow f(10-1) = \mathbf{f(9)}$

Devido ao sinal negativo ($-z$), avançar no elemento estruturante corresponde a recuar na imagem, fazendo com que a dilatação consulte o pixel à esquerda (9).

A função `_viz`, utilizada em `morph.py`, gera vizinhos por deslocamentos aditivos da forma $x+z$. Por esse motivo, a implementação de `mm.dil0` reflete previamente o elemento estruturante por meio de `np.flip(B)`. Após a reflexão, a varredura baseada em $x+z$ passa a acessar exatamente os mesmos pontos definidos pela expressão teórica $f(x-z)$ da dilatação em Equation 4.4.

Para elementos estruturantes simétricos (como discos, quadrados e cruzes centradas), a reflexão não altera a máscara. Já para elementos assimétricos, essa etapa é indispensável para que a implementação reproduza corretamente a definição matemática da dilatação e preserve a dualidade erosão–dilatação.

i Dualidade erosão–dilatação

Erosão e dilatação são **duais pelo complemento**. Isso significa que um operador pode ser completamente obtido a partir do outro, desde que se atue sobre o complemento da imagem utilizando o elemento estruturante refletido $\hat{B}$:

$$(A \ominus B)^c = A^c \oplus \hat{B} \quad \Longleftrightarrow \quad A \ominus B = (A^c \oplus \hat{B})^c$$

De forma análoga, a **dilatação também pode ser obtida a partir da erosão**:

$$(A \oplus B)^c = A^c \ominus \hat{B} \quad \Longleftrightarrow \quad A \oplus B = (A^c \ominus \hat{B})^c$$

Em termos práticos, a erosão de um objeto pode ser obtida pela dilatação de seu complemento, seguida da complementação do resultado (e vice-versa). Na implementação do pacote `morph.py`, as versões didáticas `mm.ero0` e `mm.dil0` tornam explícita essa estrutura por meio de laços (*loops*), enquanto `mm.ero` e `mm.dil` delegam as operações ao OpenCV visando maior eficiência computacional.

i Condições de contorno e imagens finitas

Na morfologia matemática clássica, definida sobre um domínio infinito (tipicamente $\mathbb{Z}^2$), essa dualidade é exata. Em imagens digitais, entretanto, trabalha-se com matrizes finitas, e o resultado passa a depender da forma como são tratados os pixels localizados fora da imagem.

Para que as identidades de dualidade permaneçam válidas, o complemento deve ser definido em relação ao mesmo universo e as condições de contorno adotadas para a erosão e para a dilatação devem ser complementares entre si. Por exemplo, se a erosão assume que os pixels externos pertencem ao objeto (255), então a dilatação aplicada ao complemento deve assumir que esses mesmos pixels externos pertencem ao fundo (0).

Quando diferentes estratégias de preenchimento são utilizadas (replicação, reflexão, valor constante etc.), a dualidade teórica pode deixar de ser satisfeita exatamente nas regiões próximas às bordas da imagem.

Para ilustrar numericamente os operadores morfológicos e a dualidade erosão–dilatação, a Figure 4.4 apresenta uma imagem binária 10×10 processada com um elemento estruturante em formato de “L”. Na implementação de `morph.py`, a origem de B é fixada no centro geométrico da máscara — posição (1,1) para um *kernel* 3×3 — e deve corresponder a um elemento ativo para que a erosão se comporte corretamente (conforme discutido anteriormente). O elemento estruturante B_L definido a seguir satisfaz essa condição. A Figure 4.5 complementa a análise com um simulador interativo da erosão, permitindo visualizar o deslocamento do elemento estruturante sobre a imagem e identificar as posições em que ele permanece completamente contido no objeto.

```
A = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,0,0,1,1,1,0,0,0,0],
    [0,0,1,1,1,1,1,0,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,1,0,0],
    [0,0,1,1,1,1,1,1,0,0],
    [0,0,0,1,1,1,1,1,0,0],
    [0,0,0,0,1,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

B_L = np.array([[1,0,0],
                [1,1,0],
                [1,1,0]], dtype=np.uint8) # origem no centro (1,1), elemento ativo

# Elemento estruturante refletido B
B_hat = np.flip(B_L)
print("Elemento estruturante B_L:")
print(mm.drawImg(B_L))
print("Elemento estruturante refletido B:")
print(mm.drawImg(B_hat))

# Erosão de A por B_L
img_ero0 = mm.ero0(A, B_L)

# Validação da dualidade: (A B)^c = A^c B
A_c = 255 - A # complemento de A
ero_c = 255 - img_ero0 # complemento da erosão - lado esquerdo
dil_Ac = mm.dil0(A_c, B_hat) # lado direito

dualidade_ok = np.array_equal(ero_c, dil_Ac)
print(f"Dualidade (A B)^c == A^c B : {dualidade_ok}")
# Transposto (reflexão em ambos os eixos): B usado na dilatação da dualidade
B_hat = np.flip(B_L)
mm.show(
    [A, A_c, img_ero0, ero_c, dil_Ac],
    titles=["A", "A ", "A B", "(A B)", "A B"],
    cols=5,
```

```
figsize=(15, 3)
)
```

Elemento estruturante B_L:

```
1 0 0
1 1 0
1 1 0
```

Elemento estruturante refletido B:

```
0 1 1
0 1 1
0 0 1
```

Dualidade $(A \ominus B)^c == A^c \oplus B$: True

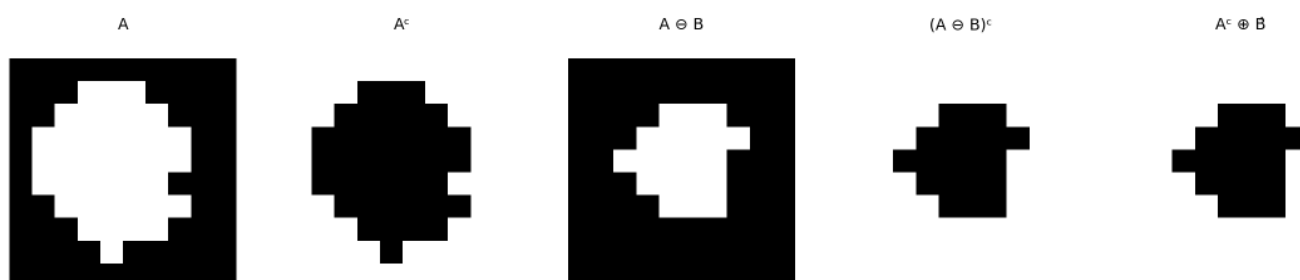


Figura 4.4: Erosão e dilatação em imagem binária 10×10 com elemento estruturante ‘L’ assimétrico 3×3. Validação da dualidade erosão–dilatação.

4.3.2 Abertura e Fechamento

Combinando erosão e dilatação obtêm-se dois operadores de grande utilidade prática: a **abertura** e o **fechamento**, definidos pelas Equações Equation 4.5 e Equation 4.6. Seus principais efeitos são resumidos na Table 4.2.

Abertura (*opening*) — erosão seguida de dilatação pelo mesmo B :

$$A \circ B = (A \ominus B) \oplus B \quad (4.5)$$

Fechamento (*closing*) — dilatação seguida de erosão pelo mesmo B :

$$A \bullet B = (A \oplus B) \ominus B \quad (4.6)$$

Na prática, o OpenCV aplica o mesmo elemento estruturante nas duas etapas. Para elementos estruturantes simétricos (os mais comuns), essa implementação coincide com a definição matemática apresentada acima.

Tabela 4.2: Propriedades de abertura e fechamento.

Operador	Sequência	Efeito principal
Abertura $A \circ B$	erosão $\rightarrow$ dilatação	Remove estruturas incapazes de conter o elemento estruturante; suaviza contornos externos
Fechamento $A \bullet B$	dilatação $\rightarrow$ erosão	Preenche buracos menores que B ; suaviza contornos internos

Propriedade importante: ambos são **idempotentes**. Por exemplo,

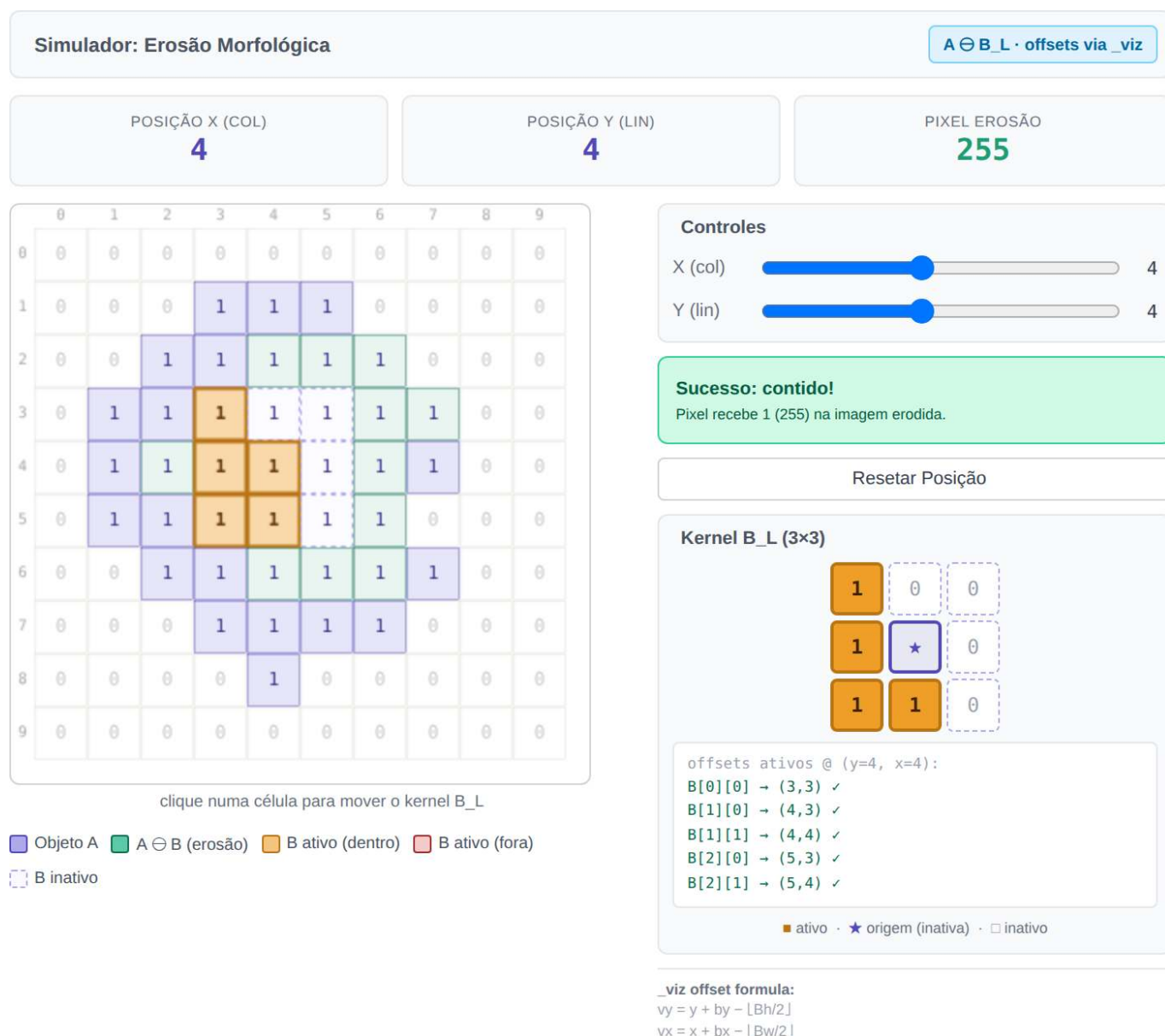


Figura 4.5: Simulador interativo de erosão morfológica: visualização do critério de inclusão do elemento estruturante assimétrico B_L sobre a imagem binária A .

$$(A \circ B) \circ B = A \circ B,$$

ou seja, após a primeira aplicação, novas aplicações do mesmo operador não alteram mais o resultado.

4.3.2.1 Implementação da abertura e do fechamento

Diferentemente da erosão e da dilatação, abertura e fechamento não introduzem novos mecanismos computacionais. Ambos são obtidos pela composição sequencial dos operadores primitivos já apresentados:

```
def open0(f, B):
    return mm.dil0(mm.ero0(f, B), B)

def close0(f, B):
    return mm.ero0(mm.dil0(f, B), B)
```

A função `mm.open` delega a operação para `cv2.morphologyEx(..., MORPH_OPEN, B)`, enquanto `mm.close` utiliza `cv2.morphologyEx(..., MORPH_CLOSE, B)`, produzindo o mesmo resultado de forma mais eficiente.

A abertura herda da erosão a capacidade de remover estruturas menores que o elemento estruturante e da dilatação a restauração parcial das regiões preservadas. O fechamento realiza o processo inverso: primeiro expande os objetos e depois restaura suas dimensões originais, preenchendo lacunas e buracos menores que o elemento estruturante.

4.3.2.2 Filtro Sequencial Alternado

Na prática, abertura e fechamento são frequentemente aplicados em sequência para remover simultaneamente ruído externo e preencher buracos internos. A função `mm.asf` (*Alternating Sequential Filter*) generaliza essa estratégia ao aplicar aberturas e fechamentos alternadamente com elementos estruturantes progressivamente maiores. As sequências disponíveis são apresentadas na Table 4.3.

Tabela 4.3: Sequências do filtro sequencial alternado `mm.asf`.

Sequência	Ordem	Uso típico
'OC'	abertura → fechamento	remove ruído externo antes de preencher pequenos buracos
'CO'	fechamento → abertura	preenche pequenos buracos antes de remover ruído externo
'OCO'	abertura → fechamento → abertura	ênfatisa a remoção de ruído externo
'COC'	fechamento → abertura → fechamento	ênfatisa o preenchimento de buracos e lacunas

O parâmetro `n` controla o número de escalas utilizadas pelo filtro. Em cada iteração i , o elemento estruturante é ampliado por soma de Minkowski (`mm.sesum(b, i)`), produzindo uma sequência de filtros morfológicos cada vez mais abrangentes. Diferentemente de uma única abertura ou fechamento com um elemento estruturante grande, o ASF realiza uma suavização progressiva em múltiplas escalas, preservando melhor a geometria dos objetos relevantes enquanto elimina estruturas menores. A Figure 4.6 apresenta um exemplo de aplicação da abertura, do fechamento e do ASF na imagem das moedas.

```
img_bin    = mm.threshold(img_clahe)
B_disk     = mm.sedisk(19)

img_open   = mm.open(img_bin, B_disk)           # erosão → dilatação
img_close  = mm.close(img_bin, B_disk)          # dilatação → erosão
img_oc     = mm.close(img_open, B_disk)         # abertura seguida de fechamento
img_asf    = mm.asf(img_bin, 'OC', mm.sedisk(3), n=7)
# ASF: disco base 3×3, cresce a cada iteração
```



```
mm.show(
    [img_bin, img_open, img_close, img_oc, img_asf],
    titles=["Binarização Otsu (CLAHE)", "Abertura (A∘B)", "Fechamento (A⋄B)",
           "Abertura→Fechamento", "ASF-OC (n=7)"],
    cols=5, figsize=(15, 12)
)
```

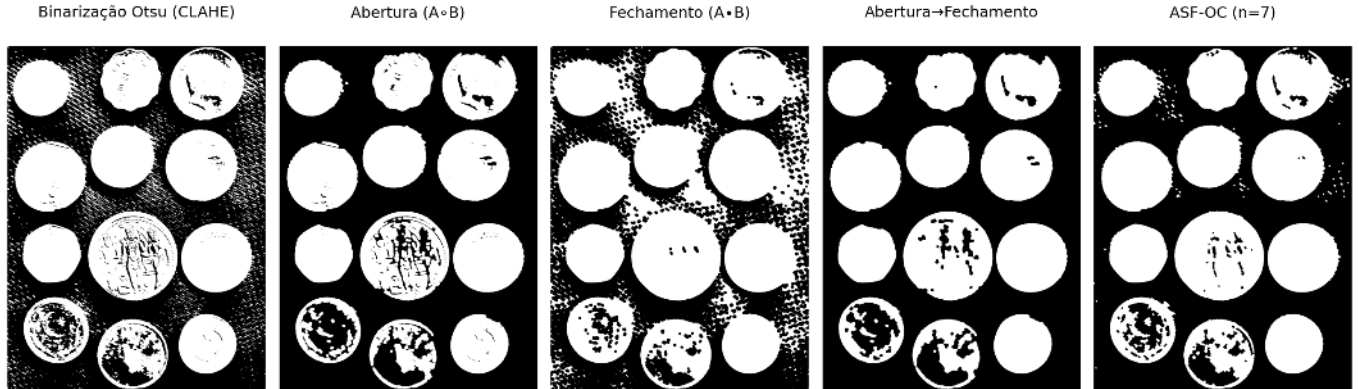


Figura 4.6: Abertura, fechamento, composição e filtro sequencial alternado aplicados à binarização Otsu das moedas com CLAHE. Elemento estruturante: disco 13×13 .

4.3.3 Operadores Geodésicos

Os operadores geodésicos introduzem uma restrição adicional aos operadores morfológicos clássicos por meio de uma imagem de controle denominada **máscara** g . Em vez de permitir que a erosão ou a dilatação se propaguem livremente pela imagem, o resultado de cada iteração é limitado ponto a ponto pelos valores da máscara, restringindo a evolução da operação às regiões permitidas.

4.3.3.1 Dilatação Geodésica

A **dilatação geodésica** de uma imagem marcador f sob uma imagem máscara g , utilizando um elemento estruturante plano b , é definida por:

$$f \oplus_g b = (f \oplus b) \wedge g, \quad (4.7)$$

onde $\wedge$ representa o mínimo ponto a ponto.

Em outras palavras, realiza-se inicialmente uma dilatação convencional sobre o marcador e, em seguida, o resultado é restringido pela máscara g . Dessa forma, a propagação nunca pode ultrapassar as regiões permitidas pela máscara.

A formulação clássica da dilatação geodésica pressupõe que o marcador esteja contido na máscara, isto é, $f \leq g$, garantindo que a evolução da operação permaneça sempre limitada pela máscara.

4.3.3.2 Implementação da dilatação geodésica

A função `mm.cdil` implementa diretamente esse operador e permite executar múltiplas iterações consecutivas:

```
def cdil(f, g, b=np.zeros((3,3),dtype='uint8'), n=1):
    """Dilatação geodésica do marcador f sob a máscara g."""
    y = f.copy()
    for _ in range(n):
        y = np.minimum(cv2.dilate(y, b), g)
    return y
```

A instrução `np.minimum(cv2.dilate(y, b), g)` implementa exatamente a definição matemática da dilatação geodésica, isto é, $(y \oplus b) \wedge g$.

Quando $n = 1$, a função executa uma única dilatação geodésica. Para $n > 1$, o resultado de cada etapa torna-se o marcador da etapa seguinte, produzindo uma propagação progressiva controlada pela máscara.

O simulador interativo Figure 4.7 permite acompanhar, passo a passo, a propagação do marcador f ao longo dos corredores do labirinto. A cada iteração de `mm.cdil`, a frente de dilatação avança para as células vizinhas livres — aquelas em que $g = 1$ —, enquanto as paredes ($g = 0$) permanecem intransponíveis. O número de passos necessários para que o marcador alcance a saída corresponde exatamente ao comprimento geodésico do caminho mais curto dentro da máscara, evidenciando a conexão direta entre a dilatação geodésica iterada e a noção de distância em grafos.



Figura 4.7: Simulador interativo de dilatação geodésica: o marcador f (verde) propaga-se passo a passo pelos caminhos livres da máscara g , sem atravessar paredes — ilustrando como $\delta_g^{(n)}(f)$ encontra a saída do labirinto.

4.3.3.3 Erosão Geodésica

De forma dual, a **erosão geodésica** de uma imagem marcador f sob uma imagem máscara g é definida por:

$$f \ominus_g b = (f \ominus b) \vee g, \quad (4.8)$$

onde $\vee$ representa o operador de máximo ponto a ponto.

Nesse caso, a erosão convencional do marcador é seguida por uma restrição inferior imposta pela máscara. Assim, nenhum pixel do resultado pode assumir valor inferior ao correspondente pixel da máscara.

A formulação clássica da erosão geodésica pressupõe a condição dual

$$f \geq g,$$

de modo que a máscara atue como limite inferior durante todo o processo.

4.3.3.4 Implementação da erosão geodésica

A função estática `mm.cero` materializa esse operador:

```
@staticmethod
def cero(f, g, b=np.zeros((3,3),dtype='uint8'), n=1):
    """Erosão geodésica do marcador f sob a máscara g."""
    y = f.copy()
    for _ in range(n):
        y = np.maximum(cv2.erode(y, b), g)
    return y
```

A instrução `np.maximum(cv2.erode(y, b), g)` implementa diretamente a expressão $(y \ominus b) \vee g$.

Assim como na dilatação geodésica, o parâmetro n define quantas erosões geodésicas sucessivas serão computadas antes de retornar a imagem final.

i Relação com a reconstrução morfológica

A reconstrução morfológica apresentada na próxima seção é obtida pela aplicação iterativa da dilatação geodésica (`mm.cdil`) até que um ponto fixo seja alcançado, isto é, até que nenhuma célula mude de valor entre duas iterações consecutivas. Em outras palavras, a reconstrução consiste em uma sequência de dilatações geodésicas sucessivas que se propagam dentro da máscara até não haver mais alterações. De forma dual, também é possível definir reconstruções baseadas em erosão geodésica por meio de aplicações sucessivas de `mm.cero`.

4.3.3.5 Exemplo: propagação em um labirinto via dualidade

A Figure 4.8 ilustra a resolução do problema de conectividade de um labirinto utilizando a dualidade morfológica por meio das funções `mm.cero` e `mm.suprec`.

Em vez de propagar um marcador pelos corredores livres através de dilatações geodésicas, o problema é formulado no domínio complementar. Inicialmente, a máscara original é invertida,

$$g = 1 - g_{orig},$$

fazendo com que as paredes passem a assumir valor 1 e os corredores valor 0. De forma análoga, o marcador é construído nesse mesmo domínio complementar, contendo um único valor 0 na posição de entrada do labirinto e valor 1 nos demais pixels.

Utilizando o elemento estruturante em cruz (`mm.secross()`), a erosão geodésica atua sobre o marcador complementado. A cada iteração de `mm.cero`, a região conectada ao marcador inicial sofre erosões sucessivas, enquanto a máscara impõe um limite inferior que impede a propagação através das paredes do labirinto.

As imagens intermediárias mostram estados da evolução após diferentes números de iterações ($n=5$, $n=12$ e $n=22$). O resultado final é obtido pela reconstrução geodésica por erosão (`mm.suprec`), que aplica erosões geodésicas sucessivas até atingir um ponto fixo, isto é, uma situação em que nenhuma alteração adicional ocorre entre duas iterações consecutivas.

No domínio complementar, a região reconstruída corresponde exatamente ao conjunto de corredores conectados à entrada do labirinto. Assim, a conectividade entre entrada e saída pode ser determinada diretamente a partir da imagem reconstruída.

```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors

# 1. Máscara original g (0 = Parede, 1 = Corredor)
g_orig = np.array([
    [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 1, 0, 1, 0, 1],
    [0, 1, 1, 1, 0, 1, 1, 1, 0, 1],
    [0, 1, 0, 1, 0, 0, 0, 0, 0, 1],
    [0, 1, 0, 1, 1, 1, 1, 1, 1, 1],
    [0, 1, 0, 0, 0, 0, 0, 0, 1, 0],
    [0, 1, 1, 1, 1, 1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0, 0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]], dtype=np.uint8)

# Inversão global no início (Dualidade Morfológica)
g = 1 - g_orig # Agora: 1 = Parede, 0 = Corredor
f = np.ones((10, 10), dtype=np.uint8)
f[0, 1] = 0 # Semente injetada como 0 no corredor

# Elemento estruturante em cruz
B_cruz = mm.secross()

# Processamento direto usando mm.cero e mm.suprec no domínio invertido
passo_5 = mm.cero(f, g, B_cruz, n=5)
passo_12 = mm.cero(f, g, B_cruz, n=12)
passo_22 = mm.cero(f, g, B_cruz, n=22)
ponto_fixo = mm.suprec(f, g, B_cruz)

# --- CONFIGURAÇÃO DA EXIBIÇÃO GRÁFICA ---

titles = [
    "Máscara (~g)", "Marcador (~f)",
    "Avanço (n=5)", "Avanço (n=12)",
    "Avanço (n=22)", "~mm.suprec"
]

images = [g, f, passo_5, passo_12, passo_22, ponto_fixo]

fig, axes = plt.subplots(1, 6, figsize=(16, 4), facecolor='#fcfcfc')

# Mapa de cores adaptado para o domínio complementar:
# No domínio invertido: 1 = Parede (Azul Escuro)
# Onde g == 0 e imagem == 1 = Corredor Livre (Cinza Claro)
# Onde g == 0 e imagem == 0 = Onda Geodésica Ativa (Ouro)
cmap_pipeline = mcolors.ListedColormap(['#ffc13b', '#e0e0e0', '#1e3d59'])

for i, ax in enumerate(axes):
    if i == 0 or i == 1:
        # Para as condições iniciais (~g e ~f)
        cmap_init = mcolors.ListedColormap(['#e0e0e0', '#1e3d59'])
        ax.imshow(images[i], cmap=cmap_init, vmin=0, vmax=1)
    else:
        # Renderização baseada nos estados complementares
        render_step = np.zeros_like(g, dtype=np.uint8)
        render_step[g == 1] = 2 # Parede (1 original do mapa de cores)
        render_step[g == 0] = 1 # Corredor padrão
        render_step[images[i] == 0] = 0 # Onda ativa (0 original do mapa de cores)
        ax.imshow(render_step, cmap=cmap_pipeline, vmin=0, vmax=2)

```

```

ax.set_title(titles[i], fontsize=10, fontweight='bold', color='#1e3d59', pad=12)
ax.set_xticks(np.arange(-0.5, 10, 1), minor=True)
ax.set_yticks(np.arange(-0.5, 10, 1), minor=True)
ax.grid(which='minor', color='ffffff', linestyle='-', linewidth=1)
ax.tick_params(which='both', bottom=False, left=False, labelbottom=False, labelleft=False)

# Indicadores gráficos de Entrada e Saída
ax.plot(1, 0, marker='v', color='#2ecc71', markersize=7, markeredgewidth=1.5)
ax.plot(8, 9, marker='o', color='#e74c3c', markersize=6, fillstyle='none', markeredgewidth=2)

plt.tight_layout()
plt.show()

```

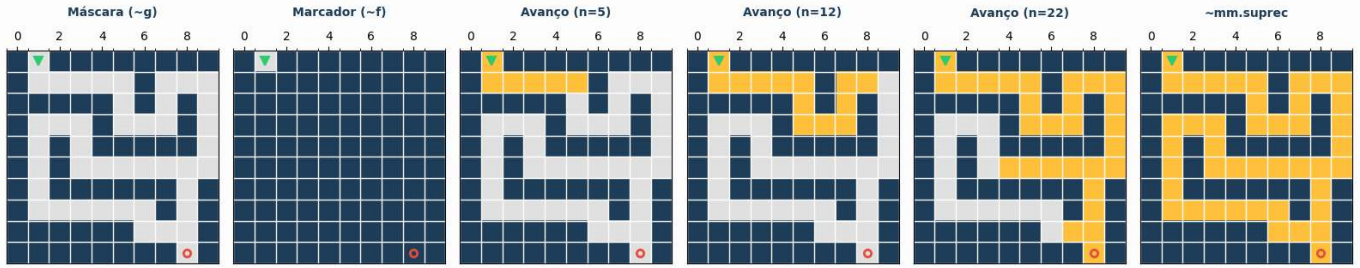


Figura 4.8: Resolução de labirinto via Operações Complementares: Invertendo a máscara e o marcador no início do *pipeline*, o fluxo é resolvido diretamente no domínio complementar através de `mm.cero` e `mm.suprec`, eliminando re-inversões redundantes.

4.3.4 Reconstrução Morfológica

A **reconstrução morfológica** propaga uma imagem **marcador** f dentro de uma imagem **máscara** g , garantindo que o resultado nunca ultrapasse os valores de intensidade impostos pela máscara. O operador fundamental que viabiliza essa propagação contida é a **dilatação geodésica**, definida pela Equation 4.7.

A reconstrução é obtida pela aplicação iterativa dessa dilatação condicionada. Inicialmente, o marcador é limitado pela máscara para estabelecer o estado inicial:

$$X^{(0)} = f \wedge g,$$

e as iterações subsequentes são definidas de forma recursiva por:

$$X^{(k)} = (X^{(k-1)} \oplus b) \wedge g.$$

A sequência cresce monotonicamente até atingir um ponto fixo, produzindo a **reconstrução morfológica por dilatação** (também conhecida na literatura como *inf-reconstrução*):

$$R_g^\delta(f) = \lim_{k \rightarrow \infty} X^{(k)} = X^{(k)} \quad \text{quando} \quad X^{(k)} = X^{(k-1)}. \quad (4.9)$$

A subida iterativa é interrompida assim que a estabilidade é alcançada, isto é, quando duas iterações consecutivas produzem matrizes com valores absolutamente idênticos.

4.3.4.1 Implementação da reconstrução morfológica

A rotina didática `mm.infrec` implementa diretamente o algoritmo iterativo de ponto fixo. Inicialmente, o marcador efetivo inicial $X^{(0)}$ é determinado pela operação `np.minimum(f, g)`. Para garantir que o laço de verificação execute a primeira passada sem disparar falsas convergências prematuras, a variável de controle

da iteração anterior (y_1) é inicializada preenchida com um valor sentinela fora do domínio de dados (ou simplesmente com uma matriz que force a primeira execução).

```
def infrec(f, g, b=np.zeros((3,3), dtype='uint8')):
    """Inf-reconstrução: dilata o marcador (f) até convergir sob a máscara g."""
    y = np.minimum(f, g)
    # Inicializa y1 com valores impossíveis para forçar a entrada no laço
    y1 = np.full_like(f, 256, dtype=np.int16)
    while not np.array_equal(y, y1):
        y1 = y.copy()
        # Aplica a dilatação geodésica: (y) sob (g)
        y = np.minimum(cv2.dilate(y, b), g)
    return y.astype('uint8')
```

No interior do laço `while`, a variável `y` armazena a estimativa corrente da reconstrução $X^{(k)}$, enquanto `y1` preserva a imagem do estágio imediatamente anterior $X^{(k-1)}$. A instrução de controle condicional `np.minimum(cv2.dilate(y, b), g)` traduz fielmente a dilatação geodésica teórica, onde a expansão morfológica convencional comandada pelo OpenCV é imediatamente “podada” e limitada pelas barreiras de intensidade da máscara `g`. O laço cessa quando nenhuma modificação de pixel é registrada entre os passos.

4.3.4.2 Vantagens da reconstrução morfológica

A reconstrução morfológica é significativamente mais robusta que a abertura convencional porque é capaz de remover estruturas indesejadas sem distorcer ou alterar a morfologia dos objetos que devem ser preservados.

Enquanto a abertura clássica suaviza cantos, elimina pontas e deforma contornos devido à imposição geométrica rígida do elemento estruturante, a reconstrução geodésica utiliza a máscara para recuperar com exatidão os limites e formatos originais dos objetos que possuem conectividade com o marcador original.

Em termos intuitivos, o marcador atua como uma semente de contágio que se expande progressivamente, mas apenas trafegando pelas regiões permitidas pela máscara. Componentes que não possuem nenhuma intersecção com o marcador jamais serão reconstruídas (sendo eliminadas), enquanto as componentes tocadas pela semente expandem-se até restaurar integralmente a sua geometria original.

Esse comportamento discriminatório e conservativo é ilustrado na Figure 4.9, utilizando o elemento estruturante em cruz apresentado na Figure 4.3.

```
# 1. Definição da Máscara (g): Imagem 10x10 com dois objetos isolados
g = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,1,1,0,0,0,1,1,0],
    [0,1,1,1,1,0,0,1,1,0],
    [0,1,1,1,1,0,0,0,0,0],
    [0,1,1,1,1,0,0,0,0,0],
    [0,1,1,1,0,0,0,0,0,0],
    [0,0,1,1,1,0,0,1,0,0],
    [0,0,1,1,1,0,0,1,1,0],
    [0,0,0,1,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

B_cruz = mm.secross()

# 2. Geração do Marcador (f)
f = mm.ero(g, mm.sebox())

# 3. Iterações da Dilatação Condicionada automatizadas em laço
iteracoes = []
titulos_iteracoes = []
img_atual = f.copy()
```



```

for i in range(1, 6):
    img_add = img_atual*80
    img_atual = mm.cdil(img_atual, g, B_cruz)
    iteracoes.append(img_add+img_atual)
    titulos_iteracoes.append(f"Dilatação Cond. (n={i})")

# Reconstrução Geodésica Final via biblioteca (ponto de controle)
img_reconstruida = mm.infrec(f, g, B_cruz)

# Verificação de convergência (o passo 5 deve ser idêntico à reconstrução final)
convergencia_ok = np.array_equal(img_reconstruida, iteracoes[-1])
print(f" Reconstrução alcançou estabilidade na iteração 5: {convergencia_ok}")

# 4. Exibição do pipeline evolutivo (Montagem dinâmica das listas)
mm.show(
    [g, f] + iteracoes + [img_reconstruida],
    titles=["Máscara (g)", "Marcador (f)"] + titulos_iteracoes + ["Reconstrução Final R_g(f)"],
    cols=8,
    figsize=(18, 3)
)

```

Reconstrução alcançou estabilidade na iteração 5: False

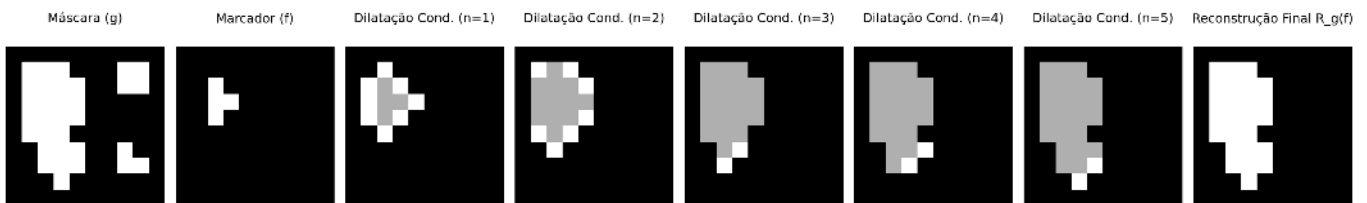


Figura 4.9: *Pipeline* de Reconstrução Morfológica por Dilatação Condicionada: a máscara contém dois objetos, o marcador isola apenas o núcleo do objeto principal, e as iterações subsequentes em laço reconstróem sua forma exata até a convergência.

4.3.5 Preenchimento de Buracos e Remoção de Bordas

Dois operadores baseados em reconstrução morfológica completam o *pipeline* de limpeza binária. Suas características principais são resumidas na Table 4.4.

Preenchimento de buracos (`mm.clohole`) remove cavidades completamente cercadas pelo objeto, independentemente do tamanho, sem alterar os contornos externos. O procedimento atua no complemento da imagem, utilizando como marcador uma restrição da moldura (*frame*) ao fundo:

$$\text{clohole}(f) = (R_{f^c}^\delta(\text{frame}(f) \wedge f^c))^c \quad (4.10)$$

Em termos operacionais, primeiro reconstrói-se o fundo externo e, em seguida, aplica-se a complementação para recuperar os objetos com buracos preenchidos.

Remoção de objetos de borda (`mm.edgeoff`) elimina todos os objetos que tocam a borda da imagem, preservando apenas os componentes totalmente internos. O marcador é obtido pela interseção entre a moldura (*frame*) e os objetos da imagem:

$$\text{edgeoff}(f) = f \setminus R_f^\delta(\text{frame}(f) \wedge f) \quad (4.11)$$

Tabela 4.4: Comparação entre os operadores *clohole* e *edgeoff*.

Operador	Marcador	Máscara	Efeito
<code>mm.clohole</code>	<i>frame</i> restrito ao fundo (f^c)	f^c	Preenche buracos internos
<code>mm.edgeoff</code>	<i>frame</i> restrito ao objeto (f)	f	Remove objetos conectados à borda

A evolução passo a passo dessas transformações geodésicas pode ser acompanhada nas figuras a seguir. A Figure 4.10 ilustra o mecanismo de inundação controlada do operador `mm.clohole`, no qual a reconstrução ocorre a partir do fundo externo e impede a propagação para regiões internas não conectadas ao exterior, resultando no preenchimento consistente das cavidades internas. Em contrapartida, a Figure 4.11 detalha a dinâmica do operador `mm.edgeoff`, em que apenas os componentes conectados à borda são reconstruídos e posteriormente removidos, preservando exclusivamente os objetos totalmente contidos no interior da imagem.

A conectividade da propagação geodésica é controlada pelo elemento estruturante: `mm.sebox()` (vizinhança de 8) inclui conexões diagonais, enquanto `mm.secross()` (vizinhança de 4) as exclui. Consequentemente, a escolha do elemento estruturante afeta quais componentes são alcançados pela reconstrução e, portanto, quais serão preservados ou removidos.

4.3.5.1 Conformidade com a implementação

As definições acima estão diretamente alinhadas com a implementação em `morph.py`, reproduzida a seguir:

```
@staticmethod
def clohole(f, b=np.ones((3,3),dtype='uint8')):
    # marcador restrito ao fundo da imagem
    marcador = mm.frame(f, border=1) & mm.neg(f)
    return mm.neg(mm.infrec(marcador, mm.neg(f), b))

@staticmethod
def edgeoff(f, b=np.ones((3,3),dtype='uint8')):
    # marcador restrito aos objetos da imagem
    marcador = mm.frame(f, border=1) & f
    return mm.subm(f, mm.infrec(marcador, f, b))
```

Essas implementações deixam explícito que ambos os operadores são instâncias diretas de reconstrução morfológica por dilatação geodésica com `mm.infrec`, diferindo apenas na escolha do marcador e da máscara: `clohole` atua no complemento da imagem, enquanto `edgeoff` atua diretamente no domínio dos objetos.

```
# Função auxiliar unificada para gerar o pipeline iterativo com realce visual
def gerar_pipeline_reconstrucao(marcador, mascara, kernel, passos=4, fator=80):
    iteracoes, titulos = [], []
    img_atual = marcador.copy()
    for i in range(1, passos + 1):
        img_visual = img_atual * fator
        img_atual = mm.cdil(img_atual, mascara, kernel)
        iteracoes.append(img_visual + img_atual)
        titulos.append(f"Iter. (n={i})")
    return iteracoes, titulos

# Imagem binária 10x10 com 3 cenários: objeto com buraco, objeto isolado e objeto na borda
f = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,1,1,1,0,0,0,0,1],
    [0,1,0,0,1,0,0,1,0,1],
    [0,1,0,0,1,0,0,1,0,1],
    [0,1,0,0,1,0,0,1,0,1],
```

```

[0,1,1,1,1,0,0,1,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,1,1,1,0,1,1,1,0],
[0,0,1,1,1,0,1,0,1,0],
[0,0,1,1,1,0,1,1,1,0],
[0,0,0,0,0,0,0,0,0,1]], dtype=np.uint8) * 255

B_cruz = mm.secross()
f_c = mm.neg(f)          # Utiliza o operador de negação nativo da mm

# PIPELINE CLOHOLE
# 0 marcador do clohole teórico é a borda externa
marcador_ch = mm.frame(f, border=1)
iters_ch, tits_ch = gerar_pipeline_reconstrucao(marcador_ch, f_c, B_cruz, passos=5)

# Resultado final calculado por extenso e validado com a função nativa mm.clohole
img_clohole = mm.neg(mm.infrec(marcador_ch, f_c, B_cruz))
print(f" Validação clohole: {np.array_equal(img_clohole, mm.clohole(f))}")

mm.show(
    [f, marcador_ch] + iters_ch + [img_clohole],
    titles=["f original", "Marcador (Borda)"] + tits_ch + ["clohole(f)"],
    cols=8, figsize=(18, 3), axis=True
)

```

Validação clohole: True

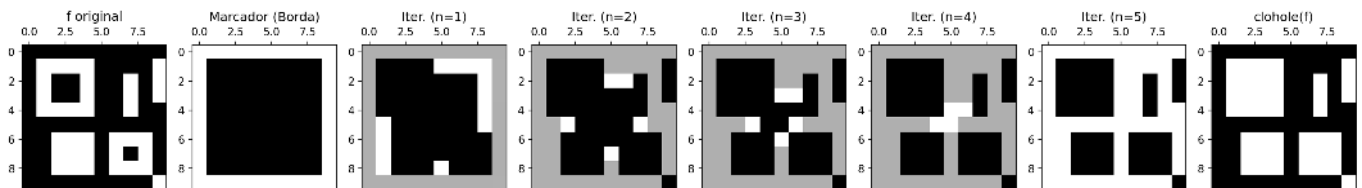


Figura 4.10: *Pipeline* de preenchimento de buracos (*clohole*): o marcador é obtido a partir da borda da imagem e restringido ao complemento f^c . As iterações de dilatação geodésica reconstruem o fundo externo; após a complementação final, os buracos internos ficam preenchidos.

```

# PIPELINE EDGEOFF
B_box = mm.sebox()
marcador_eo = marcador_ch & f
iters_eo, tits_eo = gerar_pipeline_reconstrucao(marcador_eo, f, B_box, passos=5)

# Resultado final por definição e validação nativa
img_edgoff = mm.edgoff(f, B_box)
print(f" Validação edgoff: {np.array_equal(img_edgoff, mm.edgoff(f))}")

mm.show(
    [f, marcador_eo] + iters_eo + [img_edgoff],
    titles=["f original", "Marcador (Borda)"] + tits_eo + ["edgoff(f)"],
    cols=8, figsize=(18, 3)
)

```

Validação edgoff: True

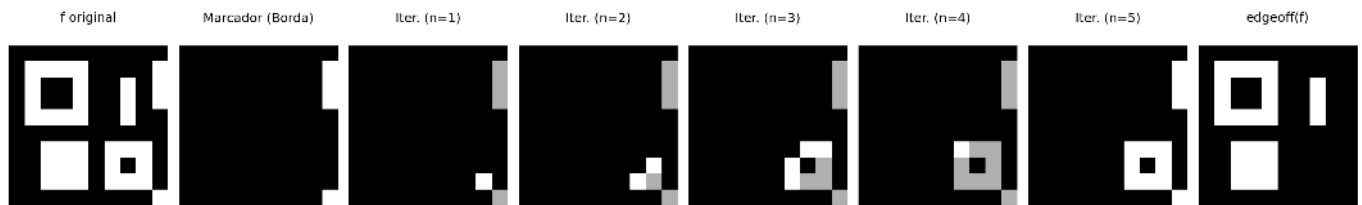
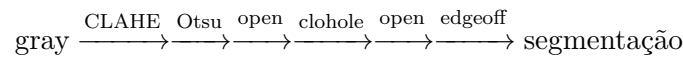


Figura 4.11: *Pipeline* de eliminação de estruturas de borda (*edgeoff*): o marcador captura as raízes conectadas às extremidades da matriz, a reconstrução delimita a extensão desses elementos e a subtração booleana preserva unicamente os objetos totalmente internos.

4.3.6 Pipeline de Limpeza Binária com CLAHE

Com base na análise anterior — na qual o CLAHE produziu o maior valor da variância interclasses ($\sigma_B^2 \approx 2,47 \times 10^3$), ver Figure 4.2, e o operador `mm.clohole` mostrou-se eficaz no preenchimento das cavidades internas —, o *pipeline* final de segmentação, ilustrado na Figure 4.12, é estruturado pelo seguinte fluxo computacional:



Após a etapa de `mm.clohole`, aplica-se uma segunda abertura morfológica com um elemento estruturante maior (`mm.sedisk(33)`, disco de diâmetro 33). Essa operação remove pequenas regiões residuais e artefatos que possam ter permanecido após a segmentação. Em particular, o preenchimento geodésico pode transformar pequenas cavidades isoladas em componentes conectados ao objeto, tornando conveniente uma etapa adicional de filtragem baseada em tamanho. O diâmetro foi escolhido de modo que as moedas permaneçam capazes de conter o elemento estruturante, enquanto componentes significativamente menores sejam eliminados.

Nesta imagem, nenhuma moeda está conectada à borda da matriz. Consequentemente, a aplicação de `mm.edgeoff` não altera o resultado obtido após a segunda abertura. Ainda assim, essa etapa é mantida no *pipeline* por robustez, pois em outras imagens podem existir objetos parcialmente visíveis ou conectados às bordas, que devem ser removidos antes da etapa de análise.

💡 Por que a abertura após o clohole?

O operador `mm.clohole` preenche todas as cavidades fechadas presentes nos objetos segmentados. Em algumas situações, pequenas regiões indesejadas podem permanecer após essa etapa ou tornar-se conectadas aos objetos principais. A abertura morfológica subsequente remove componentes menores que o elemento estruturante, preservando as moedas devido ao seu tamanho significativamente maior.

```
# Pré-processamento: apenas CLAHE
img_clahe0 = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(img_coins_gray)

# Etapa 1: Binarização Otsu
img_bin = mm.threshold(img_clahe0)

# Etapa 2: Abertura - remove ruídos brancos de fundo
img_open = mm.open(img_bin, mm.sedisk(9))

# Etapa 3: clohole - fecha todos os buracos internos
img_hole = mm.clohole(img_open)

# Etapa 4: Abertura (kernel grande) - remove artefatos do clohole
img_limpo = mm.open(img_hole, mm.sedisk(33))

# Etapa 5: edgeoff - remove objetos que tocam a borda
img_final = mm.edgeoff(img_limpo, border=1)
```

```

mm.show(
    [img_clahe0, img_bin,          img_open,
     img_hole,  img_limpo,        img_final],
    titles=["CLAHE",              "Otsu",          "Abertura (r=9)",
           "clohole",            "Abertura (r=33)", "Final (edgeoff)",
    cols=6, figsize=(18, 6)
)

```

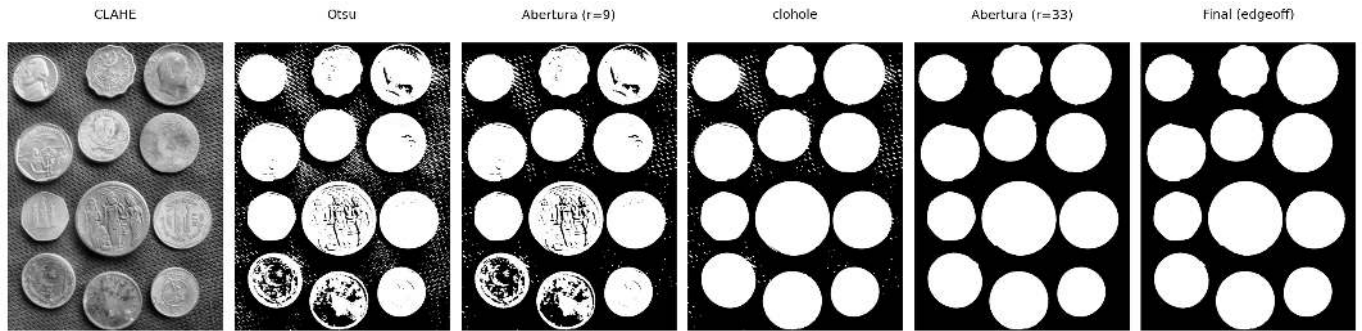


Figura 4.12: *Pipeline* completo de segmentação com CLAHE: Otsu $\rightarrow$ abertura ($r=9$) $\rightarrow$ clohole $\rightarrow$ abertura ($r=33$) $\rightarrow$ edgeoff.

4.3.7 Morfologia em Tons de Cinza

Os operadores morfológicos estendem-se naturalmente para imagens em tons de cinza. Nessa formulação, a erosão e a dilatação passam a atuar diretamente sobre os níveis de intensidade da imagem. Para elementos estruturantes planos ($b \equiv 0$), a erosão corresponde ao **mínimo local** e a dilatação ao **máximo local** dentro da vizinhança definida pelo elemento estruturante.

A interpretação intuitiva é simples: a erosão **escorece** regiões ao substituir cada pixel pelo menor valor presente em sua vizinhança, enquanto a dilatação **clareia** regiões ao utilizar o maior valor disponível. A combinação desses operadores permite construir transformações capazes de realçar bordas, remover tendências de iluminação e destacar estruturas locais.

Três operadores derivados são especialmente úteis:

Gradiente morfológico — destaca bordas como a diferença entre dilatação e erosão:

$$\text{grad}_B(f) = (f \oplus B) - (f \ominus B) \quad (4.12)$$

Top-hat — destaca estruturas brilhantes menores que o elemento estruturante (diferença entre a imagem original e sua abertura):

$$\text{top-hat}_B(f) = f - (f \circ B) \quad (4.13)$$

Black-hat — destaca estruturas escuras menores que o elemento estruturante (diferença entre o fechamento e a imagem original):

$$\text{black-hat}_B(f) = (f \bullet B) - f \quad (4.14)$$

O *Top-hat* extrai detalhes brilhantes que não sobrevivem à abertura, enquanto o *Black-hat* evidencia detalhes escuros removidos pelo fechamento. Já o gradiente morfológico realça transições abruptas de intensidade, produzindo uma representação semelhante à de um detector de bordas.

Para compreender o mecanismo desses operadores em nível local, a Figure 4.13 apresenta um simulador interativo de morfologia em tons de cinza. O simulador permite editar livremente o elemento estruturante,

visualizar seu deslocamento sobre a imagem e acompanhar simultaneamente o perfil unidimensional das intensidades. Dessa forma, torna-se possível observar diretamente como a erosão seleciona mínimos locais, como a dilatação seleciona máximos locais e como o gradiente morfológico emerge da diferença entre esses dois operadores.

O botão localizado no canto superior direito permite alternar entre a visualização original em tons de cinza e uma representação pseudocolorida (*colormap*) apenas nos três tipos gradientes. A versão colorida facilita a percepção visual das variações de intensidade, tornando mais evidente a ação dos operadores morfológicos sobre máximos, mínimos e transições locais da imagem.

Os operadores apresentados estão disponíveis em `morph.py` por meio das funções `mm.gradm`, `mm.tophat` e `mm.blackhat`.

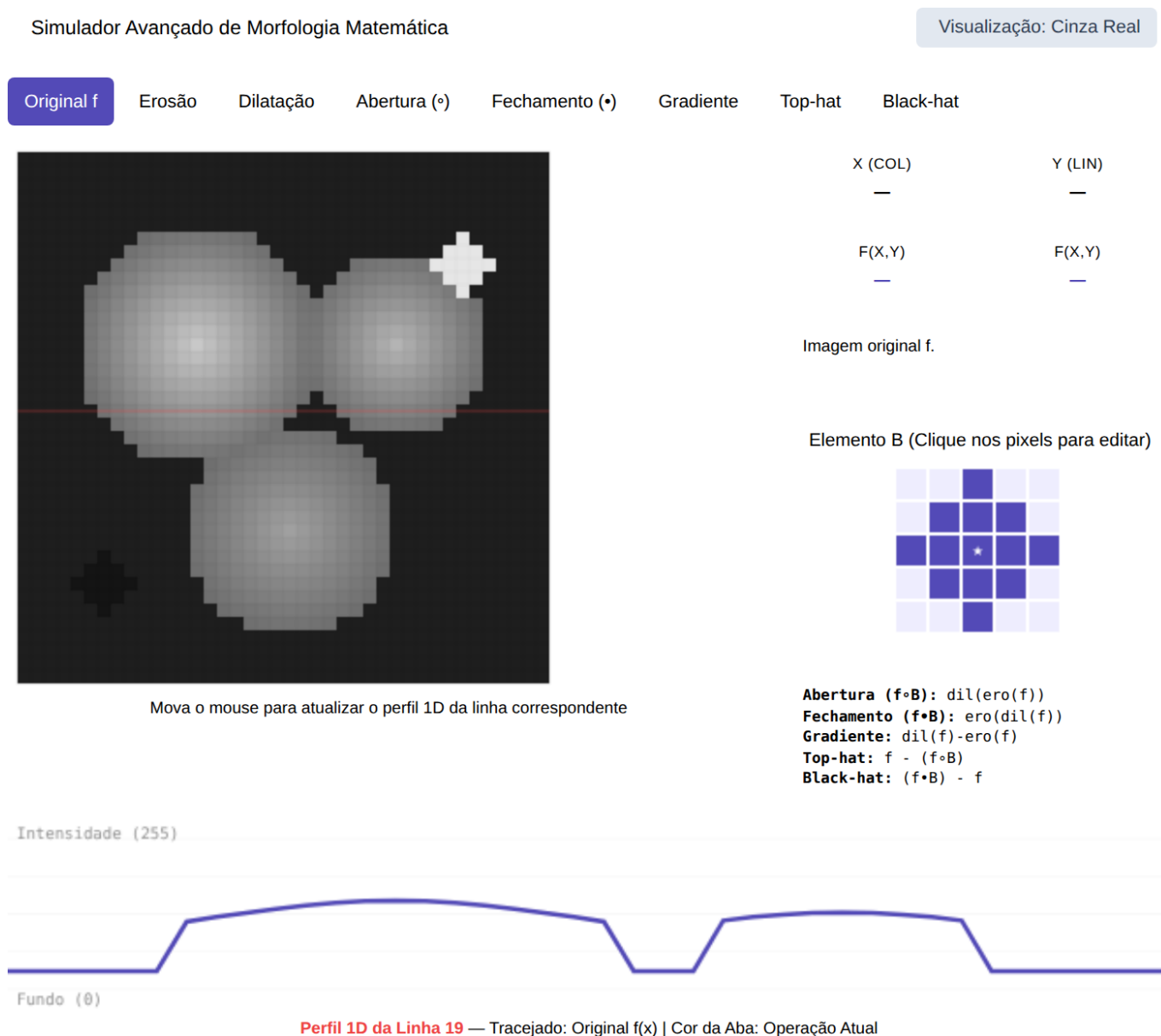


Figura 4.13: Simulador interativo avançado de morfologia com elemento estruturante editável e perfil 1D.

A Figure 4.14 ilustra os efeitos desses operadores sobre a imagem das moedas e seus respectivos histogramas. Observe que a erosão desloca a distribuição para intensidades mais baixas, enquanto a dilatação a desloca para intensidades mais altas. O gradiente concentra valores nas regiões de contorno, e os operadores *Top-hat* e *Black-hat* produzem histogramas fortemente concentrados em baixos níveis de intensidade, pois apenas pequenas estruturas locais são realçadas.


```

import io
import matplotlib.pyplot as plt
import numpy as np

def fig2img(fig):
    b = io.BytesIO(); fig.savefig(b, format='png', dpi=100); plt.close(fig); b.seek(0)
    return (plt.imread(b)[: , : , :3] * 255).astype(np.uint8)

def plot_hist(img, title):
    fig, ax = plt.subplots(figsize=(4, 3))
    h = mm.hist(img)
    # CORREÇÃO: range usa len(h) dinamicamente para casar com o retorno da biblioteca
    ax.bar(range(len(h)), h, color='steelblue', width=1, edgecolor='steelblue')
    ax.set(xlim=(0, 255)); plt.tight_layout()
    return fig2img(fig)

# 1. Processamento morfológico base
B = mm.sedisk(19)
operadores = [
    ("Original", img_coins_gray),
    ("Erosão", mm.ero(img_coins_gray, B)),
    ("Dilatação", mm.dil(img_coins_gray, B)),
    ("Gradiente Morf.", mm.gradm(img_coins_gray, B)),
    ("Top-hat", mm.tophat(img_coins_gray, B)),
    ("Black-hat", mm.blackhat(img_coins_gray, B))
]

# 2. Montagem dinâmica do par: [Imagem, Histograma]
imgs, titles = [], []
for nome, img in operadores:
    imgs += [img, plot_hist(img, f"Hist. {nome}")]
    titles += [nome, f"Hist. {nome}"]

# 3. Exibição final em grade de duas colunas (Imagem | Histograma)
mm.show(imgs, titles=titles, cols=4, figsize=(10, 8))

```

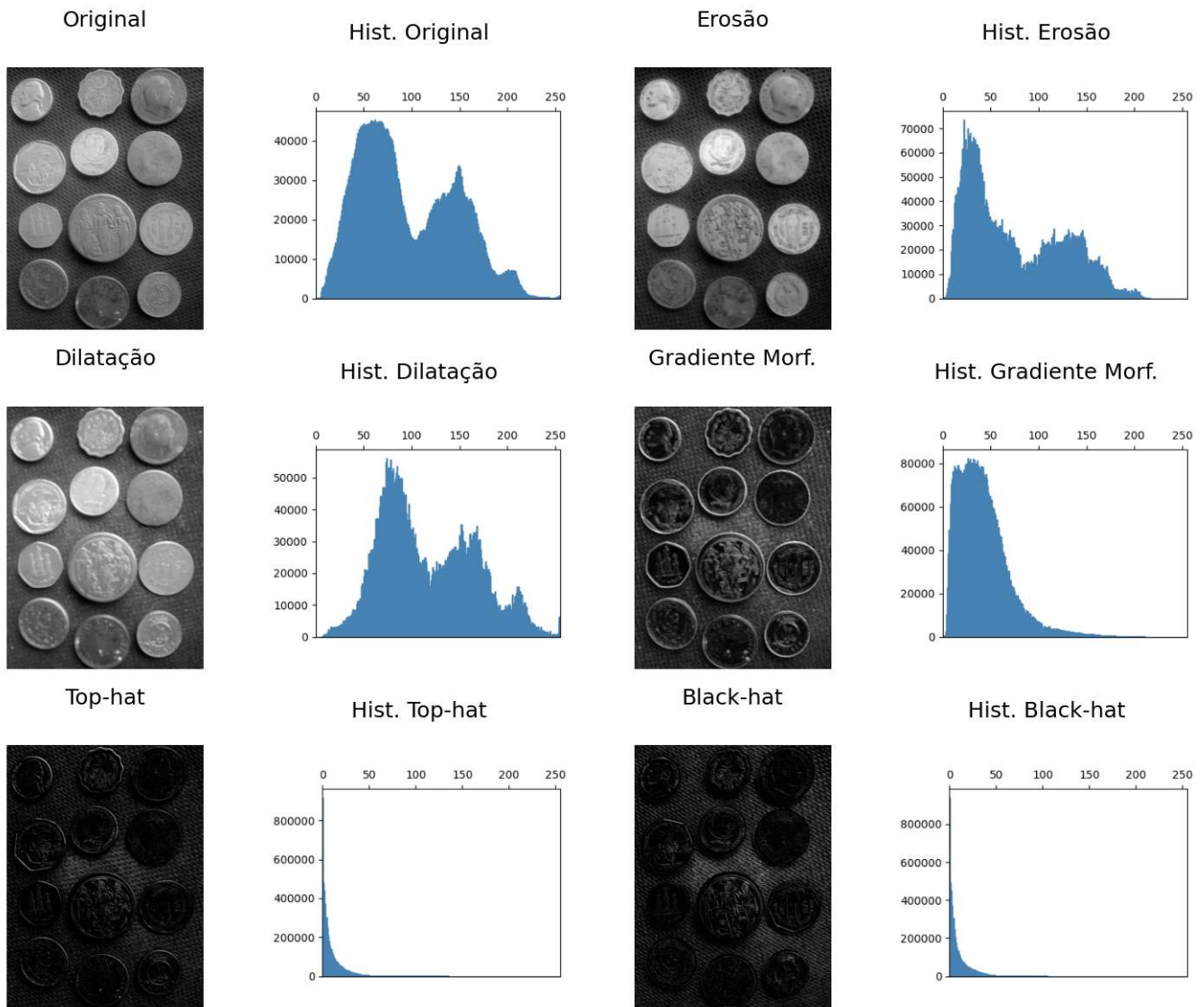


Figura 4.14: Morfologia em tons de cinza e seus respectivos histogramas. A erosão reduz intensidades locais, a dilatação as amplia, o gradiente destaca bordas, e os operadores Top-hat e Black-hat evidenciam detalhes locais brilhantes e escuros. Elemento estruturante: disco de diâmetro 19.

Os operadores morfológicos apresentados anteriormente serão agora utilizados como ferramentas de refinamento e geração de marcadores para métodos de segmentação mais avançados, apresentados a seguir.

4.4 Segmentação de Imagens: Fundamentação e Taxonomia

A **segmentação de imagens** consiste em dividir a imagem em regiões associadas a objetos ou estruturas de interesse. Em PDI, ela representa a transição entre o processamento de baixo nível — como filtragem e realce — e etapas de análise mais avançadas, como extração de características, reconhecimento e interpretação da cena.

Formalmente, o objetivo da segmentação consiste em decompor o domínio espacial completo de uma imagem, denotado por Ω , em uma partição de subconjuntos $\{R_1, R_2, \dots, R_n\}$ que satisfaça simultaneamente os critérios de **completeza** e **disjunção**:

$$\bigcup_{i=1}^n R_i = \Omega, \quad R_i \cap R_j = \emptyset \quad \forall i \neq j \quad (4.15)$$

Além das propriedades de completeza e disjunção expressas na Equation 4.15, cada sub-região R_i deve constituir um domínio **homogêneo** segundo um predicado de similaridade definido sobre propriedades locais — intensidade, cor ou textura — e, simultaneamente, ser **distinta** das regiões adjacentes.

As técnicas de segmentação podem ser organizadas em diferentes famílias. Neste capítulo serão enfatizadas as abordagens resumidas na Table 4.5, fundamentadas principalmente em critérios de intensidade, conectividade e proximidade espacial.

Tabela 4.5: Taxonomia simplificada das principais abordagens de segmentação e refinamento estudadas neste capítulo.

Abordagem	Critério de Segmentação	Operadores de Referência
Limiarização	Particionamento do espaço de intensidades	Critério de Otsu, limiarização global e local
Morfologia Matemática	Relações espaciais definidas por elementos/funções estruturantes	Erosão, dilatação, abertura, fechamento e reconstrução
Baseada em Regiões	Homogeneidade local e conectividade espacial	Rotulação de componentes conexos, Transformada de Distância e <i>Watershed</i>

Até este ponto, o desenvolvimento prático concentrou-se na **limiarização**, por meio da combinação entre equalização adaptativa CLAHE e o método global de Otsu. Essa etapa foi complementada por operadores de **reconstrução morfológica** baseados em dilatações geodésicas, implementados pelas funções `mm.infrec`, `mm.clohole` e `mm.edgeoff`, produzindo uma máscara binária limpa e adequada para análise.

Contudo, em cenários onde objetos distintos aparecem conectados na máscara binária — seja por contato físico, sobreposição parcial ou por pontes estreitas de pixels produzidas pela segmentação —, a limiarização deixa de ser suficiente para individualizar cada objeto. Nesses casos, múltiplos objetos passam a compor um único componente conexo, dificultando etapas posteriores de medição e interpretação.

Para superar essa limitação, as próximas seções introduzem três ferramentas complementares: a **Rotulação de Componentes Conexos**, a **Transformada de Distância** e o algoritmo de segmentação por **Watershed** baseado em marcadores. Em conjunto, essas técnicas permitem separar objetos adjacentes, identificar regiões individualmente e extrair descritores geométricos consistentes para análise quantitativa.

4.4.1 Rotulação

A **rotulação de componentes conexas** (*connected component labeling*) é o operador que atribui um identificador inteiro único a cada conjunto de pixels pertencentes à mesma componente conexa em uma imagem binária.

i Definição formal

Dada uma imagem binária f e uma relação de conectividade definida por um elemento estruturante B (tipicamente conectividade-4 ou conectividade-8), o algoritmo de rotulação da Figure 4.15 produz uma imagem g na qual todos os pixels pertencentes à mesma componente conexa recebem o mesmo rótulo inteiro positivo, enquanto pixels pertencentes a componentes distintas recebem rótulos diferentes.

A conectividade define quais pixels são considerados vizinhos diretos de um pixel (x, y) . As definições mais utilizadas são:

- **Conectividade-4:** considera apenas os quatro vizinhos ortogonais (norte, sul, leste e oeste).
- **Conectividade-8:** considera os quatro vizinhos ortogonais e os quatro diagonais, totalizando oito vizinhos.

A escolha da conectividade influencia diretamente a formação das componentes conexas e, consequentemente, o resultado da rotulação, como ilustrado na Figure 4.17. Um exemplo adicional pode ser explorado interativamente no simulador apresentado na Figure 4.16.

A implementação em `morph.py` disponibiliza duas versões desse operador. A função `mm.label0` reproduz explicitamente o algoritmo de *flood-fill* utilizando uma pilha e permite controlar a conectividade por meio do elemento estruturante adotado. Já `mm.label` delega a operação à implementação otimizada do OpenCV (`cv2.connectedComponents`). Em ambos os casos, o resultado é uma imagem rotulada na qual cada componente conexa recebe um identificador inteiro distinto.

O auxiliar `_viz` itera sobre a janela estruturante `b` centrada em (i, j) , gerando somente os vizinhos válidos dentro dos limites da imagem — a conectividade desejada é inteiramente determinada pela forma de `b` passada ao algoritmo.

Exemplo didático — efeito da conectividade:

```
# Imagem binária 10×10 com componentes diagonalmente adjacentes
f = np.array([
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,0,0,0,0,0,0,0,0],
    [0,0,1,0,0,0,0,0,0,0], # diagonal com linha anterior
    [0,0,0,1,0,0,1,1,0,0],
    [0,0,0,0,0,0,1,1,0,0],
    [0,0,0,0,0,0,0,0,0,0],
    [0,1,1,1,0,0,0,0,0,0],
    [0,1,0,1,0,0,0,1,0,0],
    [0,1,1,1,0,0,0,0,1,0], # diagonal com linha anterior
    [0,0,0,0,0,0,0,0,0,0]], dtype=np.uint8) * 255

# Elemento estruturante cruz (conectividade-4) e quadrado (conectividade-8)
B4 = mm.secross() # conectividade-4
B8 = mm.sebox()  # conectividade-8

# Rotulação com label0 (didático) e label (cv2)
lbl4_didatico = mm.label0(f, B4)
lbl8_didatico = mm.label0(f, B8)

_, lbl4_cv2 = cv2.connectedComponents(f, connectivity=4)
_, lbl8_cv2 = cv2.connectedComponents(f, connectivity=8)

# Validação cruzada
print(f" label0 (C4) == cv2 (C4): {np.array_equal(lbl4_didatico, lbl4_cv2)}")
print(f" label0 (C8) == cv2 (C8): {np.array_equal(lbl8_didatico, lbl8_cv2)}")
print(f" Componentes C4: {lbl4_cv2.max()} | Componentes C8: {lbl8_cv2.max()}")

# Normalização para visualização
def norm_label(lbl):
    out = np.zeros_like(lbl, dtype=np.uint8)
    for i, v in enumerate(np.unique(lbl)[1:], 1):
        out[lbl == v] = int(i * 255 / lbl.max())
    return out

mm.show(
    [f, norm_label(lbl4_cv2), norm_label(lbl8_cv2)],
    titles=[
        "f original",
        f"Rotulação C4\n({lbl4_cv2.max()} componentes)",
        f"Rotulação C8\n({lbl8_cv2.max()} componentes)"
    ],
    cols=3, figsize=(12, 4), axis=True
```

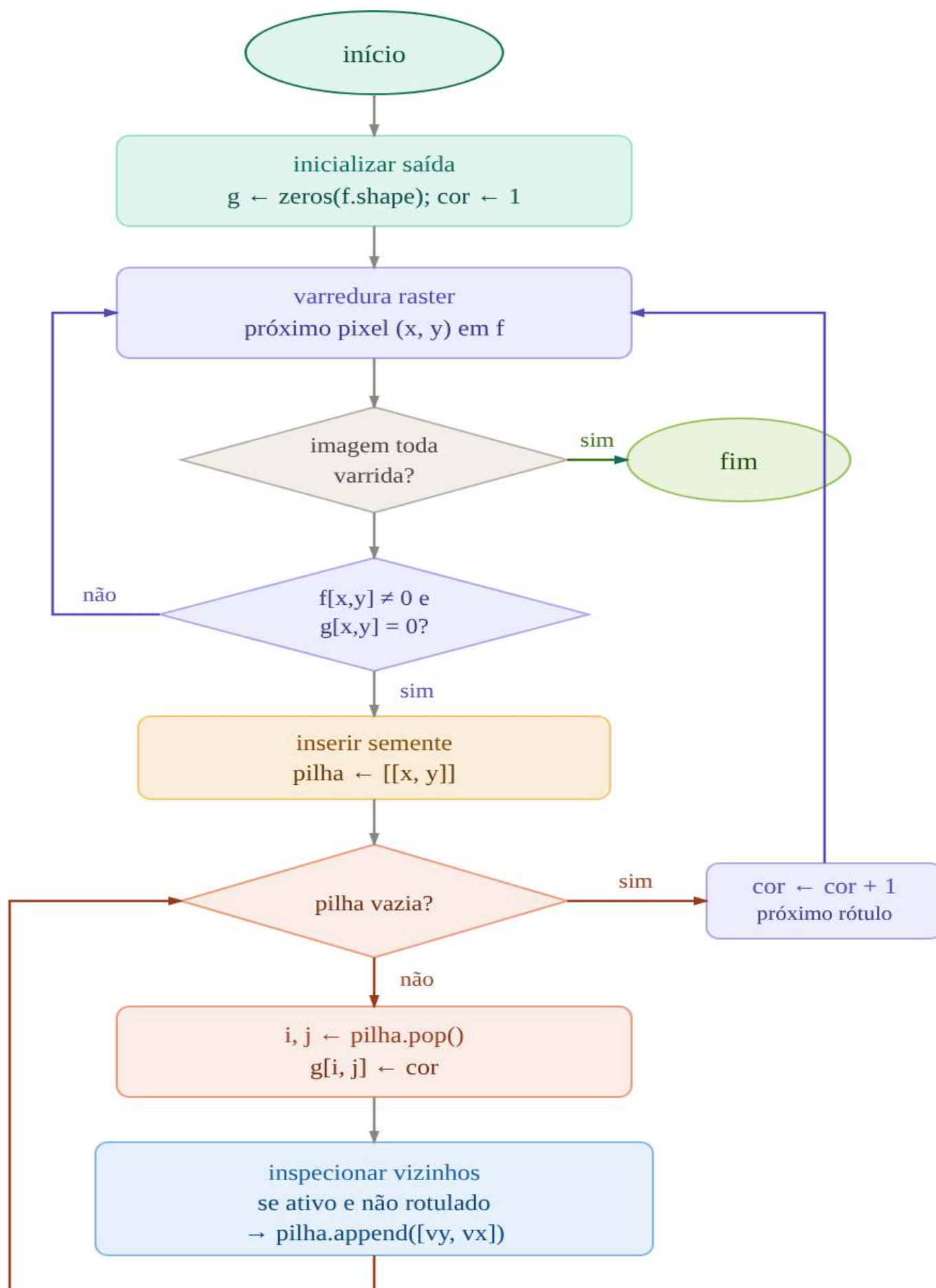


Figura 4.15: Algoritmo de rotulação por *flood-fill* com pilha: passo a passo, cards, linha do tempo, código Python e fluxograma.



Figura 4.16: Simulador interativo de rotulação de componentes conexas (*connected component labeling*): visualização da expansão flood-fill, conectividade-4 e conectividade-8.

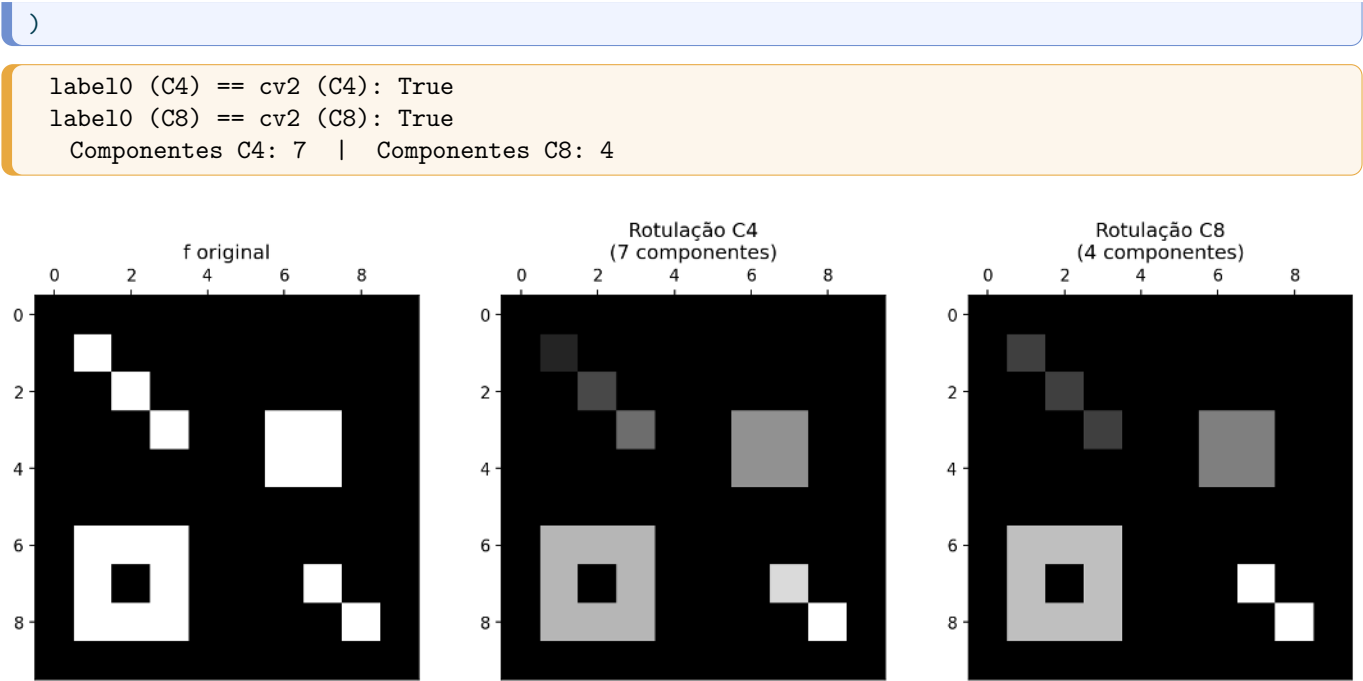


Figura 4.17: Efeito da conectividade na rotulação: a imagem binária 10×10 contém pixels diagonalmente adjacentes. Com conectividade-4, esses pixels formam componentes distintas; com conectividade-8, fundem-se em uma única componente.

4.4.2 Transformada de Distância

A **Transformada de Distância** (TD) é um operador que, aplicado a uma imagem binária f , produz uma imagem em níveis de cinza D na qual cada pixel pertencente ao objeto ($f(x, y) \neq 0$) recebe como valor a distância geométrica até o pixel de fundo ($f(x', y') = 0$) mais próximo:

$$D(x, y) = \min_{(x', y') : f(x', y') = 0} d((x, y), (x', y'))$$

onde $d(\cdot, \cdot)$ é uma métrica de distância — tipicamente a distância Euclidiana (L_2). O resultado é uma representação topográfica dos objetos: pixels localizados no interior assumem valores elevados, enquanto pixels próximos às bordas apresentam baixos valores de distância. Os **máximos locais** de D correspondem aos pontos mais afastados da borda do objeto, frequentemente próximos aos seus centros geométricos ou centros de máxima inscrição — propriedade particularmente útil para a geração automática de marcadores no algoritmo *watershed*.

i Definição formal usando erosões

A TD admite ainda uma interpretação morfológica iterativa, conforme algoritmo da Figure 4.18. Considere uma função estruturante b cujo valor central é nulo e cujos vizinhos possuem custos negativos associados ao deslocamento. Ao aplicar erosões sucessivas com essa função estruturante particular, os valores dos pixels dos objetos (que devem assumir a distância máxima possível da imagem) são progressivamente reduzidos segundo os custos definidos por b . O valor acumulado dessa propagação passa então a representar a distância ao fundo segundo a métrica induzida pela função estruturante.

Essa interpretação é implementada em `mm.dist1()`, que acumula erosões sucessivas utilizando a operação `mm.ero1()`. Já `mm.dist()` delega o cálculo da distância Euclidiana ao operador otimizado do OpenCV `cv2.distanceTransform(f, cv2.DIST_L2, 5)`, onde f é a imagem binária de entrada, `cv2.DIST_L2` especifica a métrica Euclidiana (L_2) e 5 indica o uso de uma máscara 5×5 para aproximar a distância com elevada precisão.

A função `dist1` produz uma transformada de distância discreta cuja métrica é determinada pela geometria e pelos pesos da função estruturante utilizada. Por exemplo, utilizando uma função estruturante em cruz com custo unitário para os quatro vizinhos ortogonais, obtém-se a distância de **Manhattan** (L_1). Outras escolhas de vizinhança e pesos induzem métricas diferentes. Já `mm.dist()` calcula uma aproximação eficiente da distância Euclidiana (L_2).

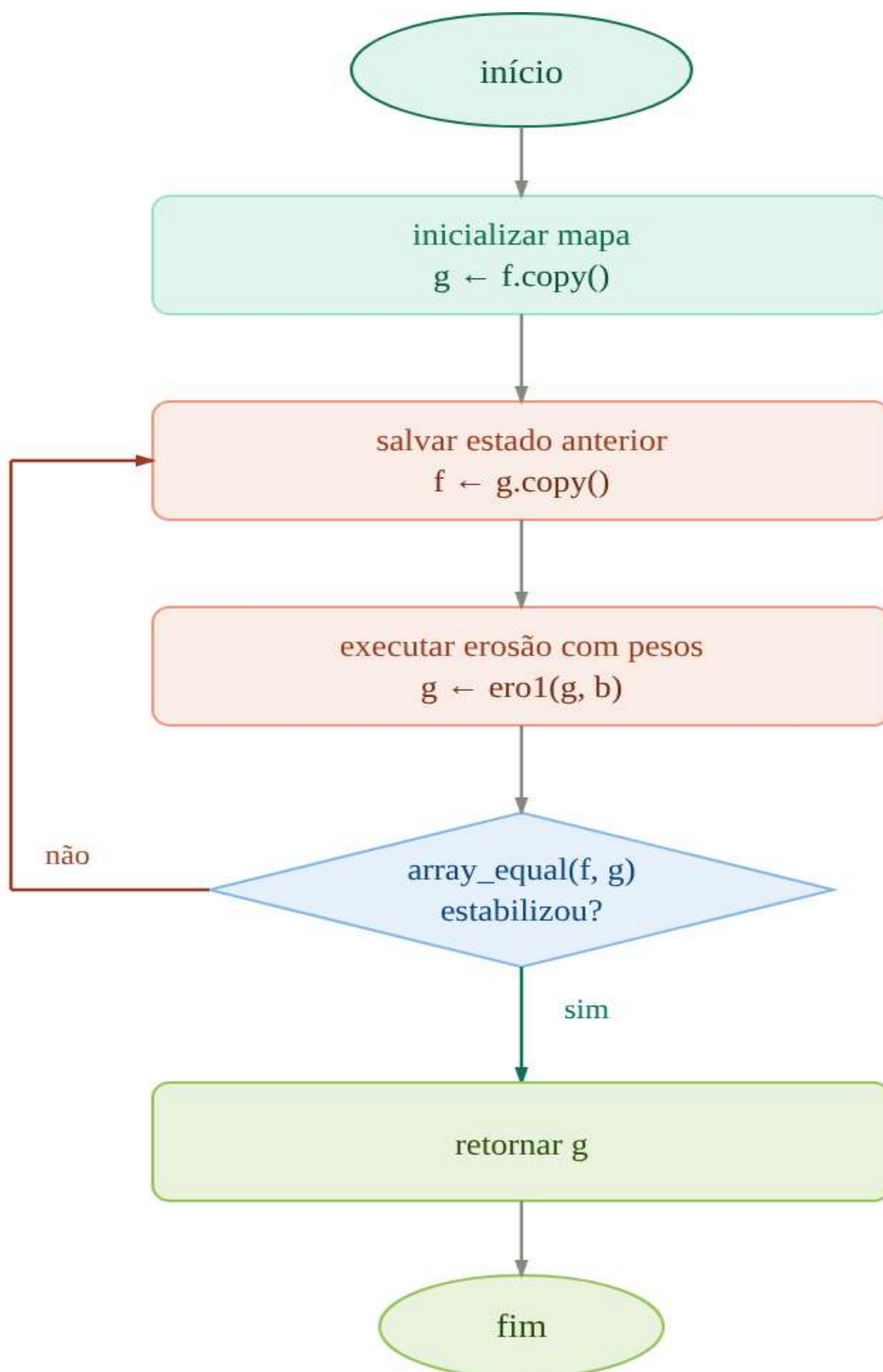
Por exigir sucessivas erosões sobre toda a imagem, a abordagem `dist1` possui custo computacional significativamente maior que `mm.dist()`, sendo empregada neste livro principalmente para fins didáticos e para evidenciar a relação entre morfologia matemática e transformadas de distância.

A Figure 4.19 apresenta um simulador interativo da TD: é possível posicionar o cursor sobre diferentes pixels do objeto e observar, em tempo real, o valor da distância associado àquela posição, isto é, a distância até o pixel de fundo mais próximo. A Figure 4.20 apresenta um exemplo prático dessa execução em ambiente Python.

```
import numpy as np

# Imagem binária 10×10 com um único pixel de fundo (0,0) e o resto como objeto (255)
f = np.ones((10,10), dtype=np.uint8) * 255
f[0,0] = 0

B_cruz = np.array([
    [-np.inf, -1, -np.inf],
    [-1,      0, -1],
    [-np.inf, -1, -np.inf]
], dtype=float)
```



Ponto Fixo Numérico: A distância emerge da propagação matemática do valor zero.

Figura 4.18: Algoritmo da Transformada de Distância: passo a passo, cards, linha do tempo, código Python e fluxograma.

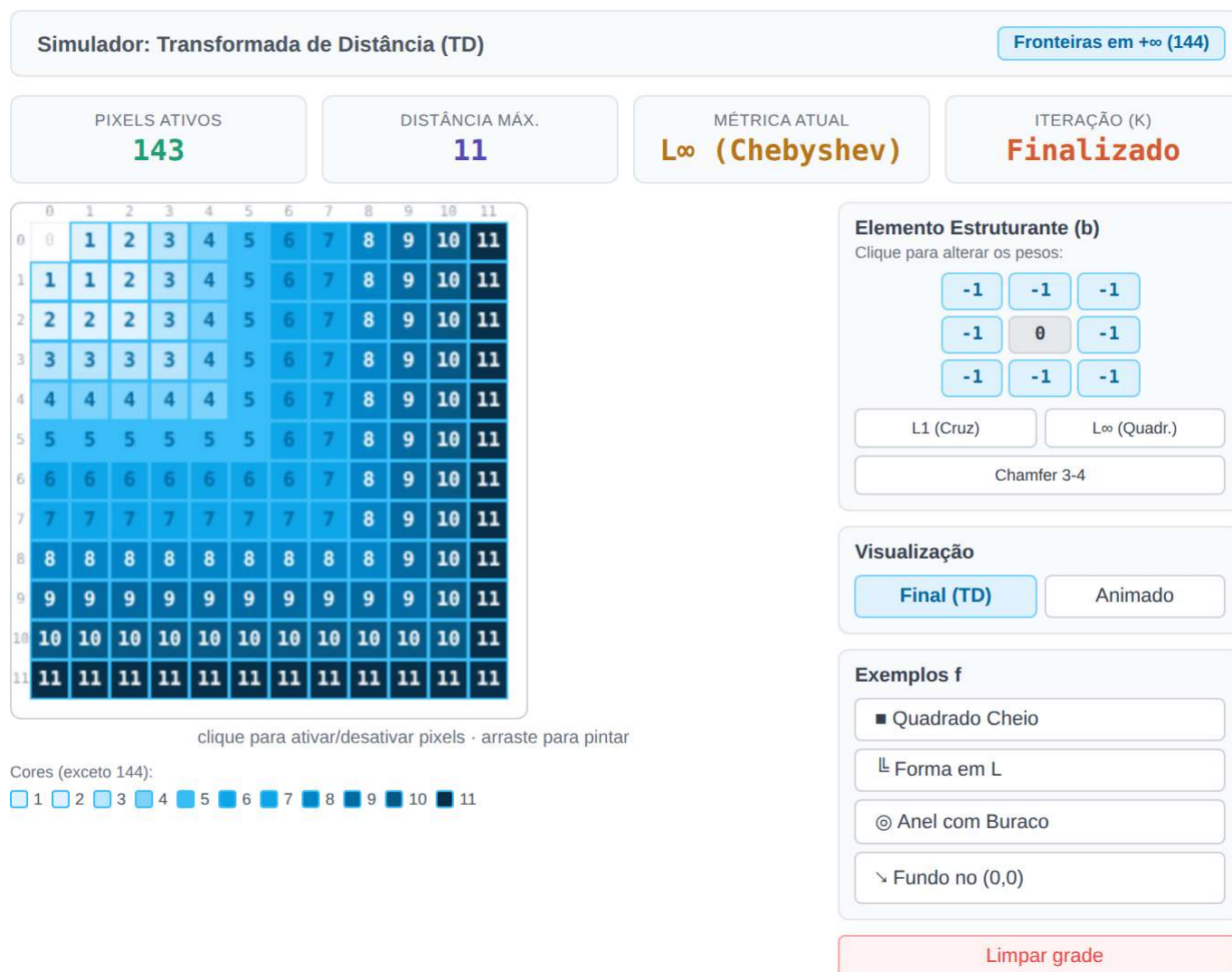


Figura 4.19: Simulador interativo da Transformada de Distância (TD) iterativa via erosão em tons de cinza. Pixels fora da imagem agora assumem o valor máximo (144), propagando os custos apenas a partir do fundo interno.

```

d_iter = mm.dist1(f, B_cruz)
d_l2 = mm.dist(f)

print(f"Máx. dist1 (erosões) : {d_iter.max()} px")
print(f"Máx. dist (L2) : {d_l2.max():.2f} px")
print(f"Posição do máximo dist1 : {np.where(d_iter == d_iter.max())}")
print(f"Posição do máximo dist : {np.where(d_l2 == d_l2.max())}")

mm.show(
    [f, d_iter, d_l2],
    titles=["f original", "mm.dist1\n(erosões com cruz)", "mm.dist\n(L2 Euclidiana)"],
    cols=3, figsize=(12, 4), axis=True
)

```

```

Máx. dist1 (erosões) : 18 px
Máx. dist (L2) : 12.00 px
Posição do máximo dist1 : (array([9]), array([9]))
Posição do máximo dist : (array([9]), array([9]))

```

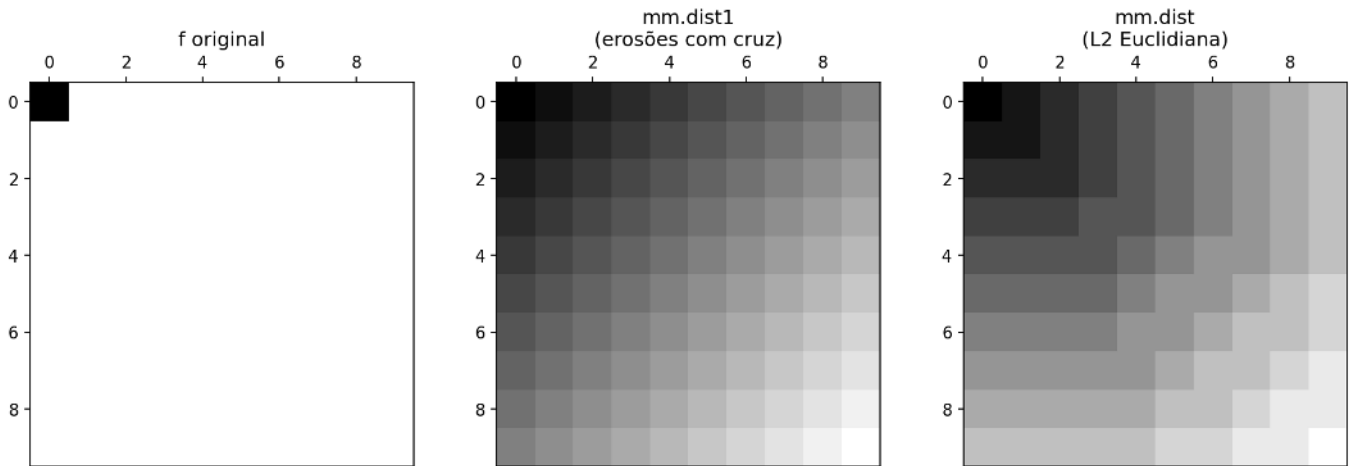


Figura 4.20: Transformada de Distância em imagem binária 10×10. Esquerda: imagem original (*foreground* = 255). Centro: `mm.dist1` iterativa (erosões com elemento cruz). Direita: `mm.dist` (L2 Euclidiana).

A anotação dos valores numéricos diretamente sobre os pixels permite verificar como `dist1` propaga as distâncias segundo a métrica induzida pela função estruturante utilizada. No caso do elemento cruz com custo unitário, os valores obtidos correspondem à distância de *Manhattan* (L_1). Embora `dist1` e `mm.dist` produzam valores numéricos distintos por adotarem métricas diferentes, ambas as transformadas preservam a estrutura topográfica dos objetos, fazendo com que seus máximos ocorram em regiões centrais semelhantes. Essa propriedade justifica o uso de `mm.dist` em aplicações práticas, devido à sua elevada eficiência computacional.

4.4.3 Transformada de Distância Euclidiana em quatro passos

O simulador da Figure 4.21 implementa o algoritmo da TDE de Lotufo; Zampirolli (2001) em duas etapas. Na primeira etapa, a função `edt1` realiza uma transformação unidimensional vertical de forma sequencial (*in-place*), percorrendo cada coluna em *raster* ($\downarrow$) e *anti-raster* ($\uparrow$) para calcular as distâncias na direção vertical (dois passos: Sul e Norte). Na segunda etapa, a função `edt2` utiliza esse resultado como entrada e realiza uma propagação horizontal por filas: para cada linha da matriz, duas filas de prioridade, E_q e W_q , são inicializadas percorrendo os índices de coluna em sentidos opostos (E_q de $W-1$ até 1, W_q de 2 até W), de modo que cada pixel atualizado enfileira imediatamente seus vizinhos para reproprocessamento dentro da mesma rodada (mais dois passos: Leste e Oeste). Esse mecanismo de fila permite aplicar erosões sucessivas com pesos ímpares crescentes ($b = 1, 3, 5, \dots$, incrementado a cada iteração do laço externo) sem que a propagação fique presa em valores desatualizados, já que cada rodada resolve a cadeia de dependências

horizontal por completo antes do próximo incremento de **b**. A convergência dessa propagação produz a Transformada de Distância Euclidiana em toda a matriz, combinando a informação vertical obtida em **edt1** com a propagação horizontal em fila realizada em **edt2**.

EDT² 2D Corrigido · Matriz 4x4

Passo 1: edt1 Vertical In-place · Passo 2: edt2 Horizontal In-place (mesma estrutura raster/anti-raster do edt1)



Figura 4.21: Simulador interativo 2D (4x4) com sincronização estrita de filas de propagação horizontal (b) para obter a convergência exata descrita no artigo.

4.4.3.1 Transformada de Distância Geodésica

A transformada de distância geodésica associa a cada pixel a menor distância até um marcador, sob a restrição imposta por uma máscara. Dessa forma, a propagação ocorre exclusivamente pelos pixels permitidos, preservando a conectividade do domínio.

A Figure 4.22 ilustra esse processo em um labirinto: (a) a máscara **g**; (b) a distância geodésica **D1** calculada a partir da entrada; (c) a distância **D2** calculada a partir da saída; e (d) o caminho mínimo obtido a partir dessas duas transformadas.

O caminho ótimo é determinado pela soma das distâncias ($D1 + D2$). Os pixels pertencentes à trajetória mínima são aqueles para os quais essa soma assume seu menor valor, definindo uma conexão entre entrada e saída com comprimento geodésico mínimo.

Esse princípio permite resolver labirintos sem a necessidade de explorar explicitamente todas as possibilidades de percurso. A solução emerge diretamente da propagação de distâncias em um domínio restrito. Essa abordagem é particularmente relevante em labirintos de elevada complexidade, como os construídos a partir de estruturas quasicristalinas e ciclos hamiltonianos descritos por Singh; Lloyd; Flicker (2024). Em Zampirolli et al. (2025), esse mesmo formalismo é empregado para resolver um labirinto complexo; a seguir, o método é ilustrado em uma versão simplificada do problema.

```
import numpy as np

# 1 = corredor, 0 = parede
g = np.array([
    [0,1,0,0,0,0,0,0,0],
    [0,1,1,1,1,1,0,1,1],
    [0,0,0,0,0,1,0,1,0],
])
```

```

[0,1,1,1,0,1,1,1,0,1],
[0,1,0,1,0,0,0,0,0,1],
[0,1,0,1,1,1,1,1,1,1],
[0,1,0,0,0,0,0,0,1,0],
[0,1,1,1,1,1,1,0,1,0],
[0,0,0,0,0,0,1,1,1,0],
[0,0,0,0,0,0,0,0,1,0]
], dtype=np.uint8)

# marcador da entrada
entrada = np.zeros_like(g, dtype=np.uint8)
entrada[0,1] = 1

# marcador da saída
saida = np.zeros_like(g, dtype=np.uint8)
saida[9,8] = 1

# distâncias geodésicas
D1 = mm.gdist(g, entrada)
D2 = mm.gdist(g, saida)

# soma das distâncias
S = D1 + D2

# menor valor válido da soma
dmin = np.min(S[S > 0])

# pixels pertencentes a um caminho ótimo
caminho = (S == dmin)

print("Distância geodésica mínima:", dmin)

mm.show(
    [g, D1, D2, caminho],
    titles=[
        "Labirinto",
        "Distância da Entrada",
        "Distância da Saída",
        f"Menor Caminho\n(d={dmin})"
    ],
    cols=4,
    figsize=(14,4),
    axis=True
)

```

Distância geodésica mínima: 15

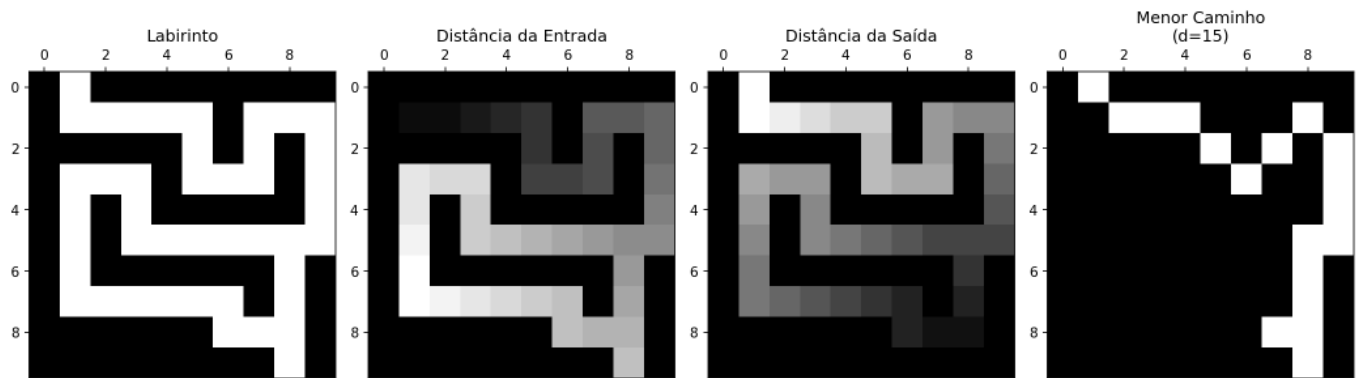


Figura 4.22: Menor caminho geodésico em um labirinto. As distâncias geodésicas são calculadas a partir da entrada e da saída usando `mm.gdist`. Os pixels cujo somatório das duas distâncias é igual à distância mínima entre os marcadores pertencem a um caminho ótimo.

4.4.4 Segmentação por *Watershed*

O algoritmo ***Watershed*** interpreta uma imagem em níveis de cinza como uma superfície topográfica, na qual valores elevados correspondem a montanhas e valores baixos correspondem a vales ou bacias de drenagem (*catchment basins*). No contexto da segmentação baseada em marcadores, os máximos da Transformada de Distância são frequentemente utilizados para identificar regiões internas aos objetos, fornecendo sementes confiáveis para o processo de inundação.

A segmentação é então realizada por uma simulação conceitual de inundação progressiva a partir desses marcadores. À medida que as bacias associadas a diferentes sementes se expandem, regiões vizinhas eventualmente entram em contato. Nesse instante são construídas barreiras virtuais, denominadas *watershed lines*, que passam a delimitar os objetos da cena. Esse mecanismo permite separar objetos adjacentes ou parcialmente sobrepostos, mesmo quando eles formam uma única componente conexa após a limiarização.

A implementação didática apresentada neste capítulo explora inicialmente o conceito de crescimento de regiões (*region growing*) confinado por uma máscara binária, conforme detalhado no algoritmo iterativo da Figure 4.23.

i Versão didática versus implementação clássica

A função `mm.watershed0` não implementa o algoritmo *watershed* clássico. Seu objetivo é ilustrar, de forma simplificada, a propagação de marcadores por crescimento de regiões (*region growing*), permitindo visualizar como diferentes sementes competem pela ocupação do espaço disponível. O crescimento é delimitado por uma máscara binária de suporte e monitorado por um controle de estagnação, gerando um resultado semelhante a uma partição de Voronoi restrita à geometria dos objetos de entrada. Já a função `mm.watershed` utiliza a implementação otimizada do OpenCV (`cv2.watershed`), que realiza a inundação sobre uma superfície topográfica definida pela imagem de entrada. Nesse caso, a propagação dos marcadores é influenciada pelos valores dos pixels, fazendo com que as linhas de separação se formem naturalmente sobre as cristas do relevo.

A Figure 4.24 apresenta um simulador iterativo que ilustra a propagação dos marcadores pela região de interesse. Cada marcador atua como uma fonte de inundação que expande sua área de influência até encontrar regiões provenientes de outras sementes. No algoritmo do OpenCV, os pixels pertencentes às linhas divisoras são identificados pelo valor `-1`, representando as fronteiras entre bacias adjacentes.

Pipeline Morfológico do *Watershed*

O *watershed* baseado em marcadores normalmente integra um fluxo mais amplo de segmentação. Em imagens reais, etapas de pré-processamento são frequentemente necessárias para melhorar o contraste, reduzir ruídos e gerar marcadores confiáveis. Esse fluxo completo está resumido na Table 4.6.

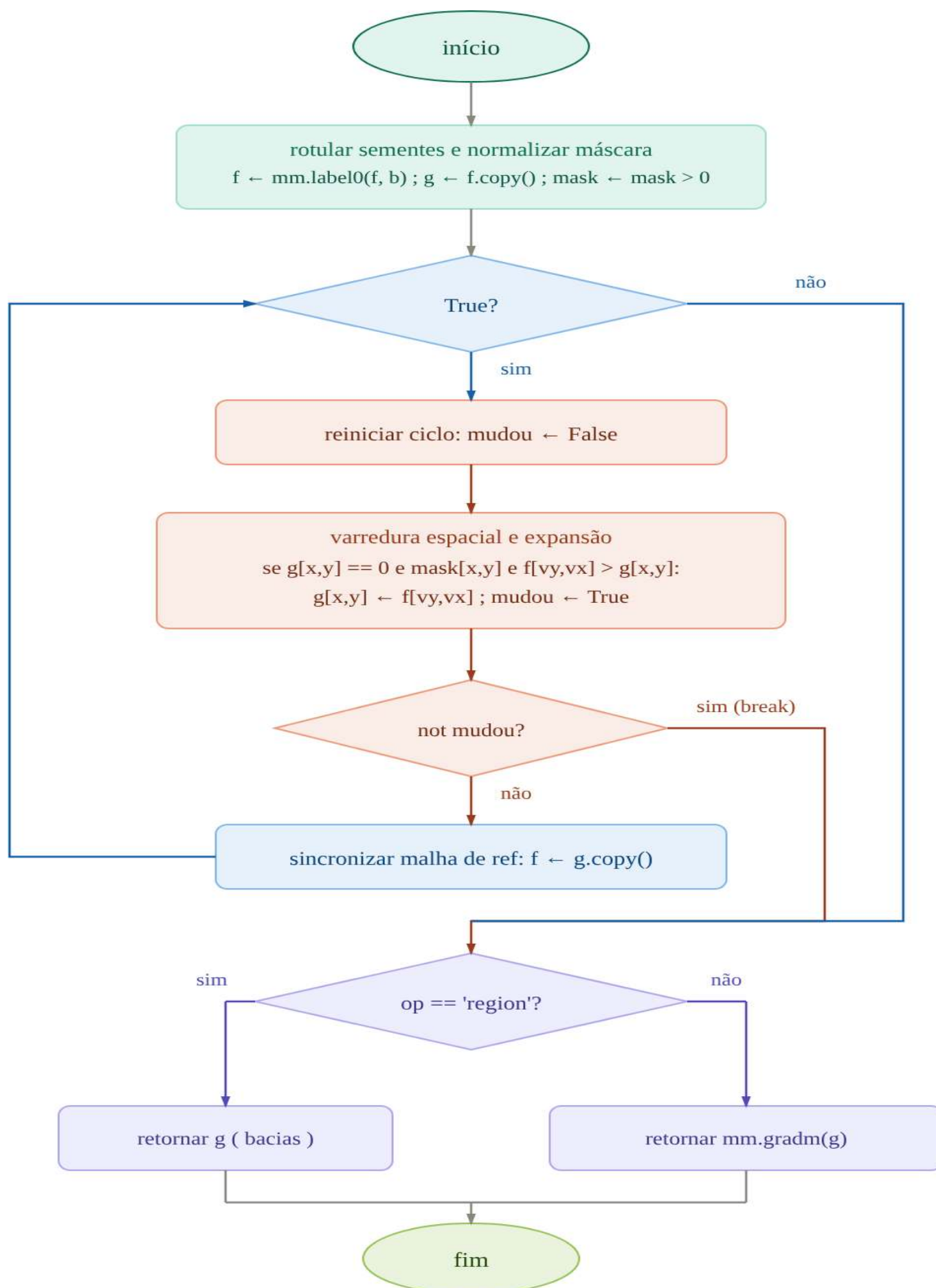


Figura 4.23: Algoritmo Didático de *Watershed* por Crescimento de Regiões Limitado por Máscara: passo a passo, cards, linha do tempo, código Python e fluxograma.

Simulador: Segmentação por Watershed

Propagação com Elemento Estruturante (b)

ÁREA (MÁSCARA)
83

MARCADORES
2

PREENCHIMENTO
100%

ITERAÇÃO (K)
Finalizado

arraste para desenhar/apagar a máscara ou as sementes

Ferramentas

☐ Máscara
☐ Apagar

☒ Sem. 1
☒ Sem. 2

☐ Sem. 3

Elem. Estruturante (b)

0	1	0
1	1	1
0	1	0

Cruz (C-4)
Quadr. (C-8)

Visualização

Final
Animado

Exemplos Iniciais

Limpar Tudo

Figura 4.24: Simulador interativo do Algoritmo *Watershed* por propagação morfológica. Desenhe a máscara, posicione os marcadores e ajuste o Elemento Estruturante para observar a inundação. Quando as bacias se encontram simultaneamente, o empate é resolvido assumindo uma das regiões de forma aleatória.

Tabela 4.6: *Pipeline* completo do *watershed* baseado em marcadores para imagens reais.

Etapa	Operação	Finalidade
1	CLAHE + Suavização	Realce de contraste e redução de ruído
2	Limiarização	Separação inicial entre objeto e fundo
3	Abertura/Fechamento	Remoção de ruídos e pequenas imperfeições
4	Dilatação da Máscara	Identificação do Fundo Certo (<i>Sure Background</i>)
5	Transformada de Distância + Limiar	Identificação do Objeto Certo (<i>Sure Foreground</i>)
6	Região Incerta	Diferença entre Fundo Certo e Objeto Certo
7	<code>cv2.watershed</code>	Propagação dos marcadores pela região incerta

Para enfatizar exclusivamente os conceitos de Transformada de Distância, marcadores e inundação topográfica, o exemplo da Figure 4.25 utiliza uma imagem binária sintética e adota um fluxo simplificado, resumido na Table 4.7.

Tabela 4.7: *Pipeline* simplificado utilizado no exemplo didático da Figure 4.25.

Etapa	Operação	Finalidade
1	Transformada de Distância	Construção da superfície topográfica
2	Limiar da TD	Extração dos marcadores (<i>Sure Foreground</i>)
3	Dilatação	Determinação do Fundo Certo (<i>Sure Background</i>)
4	Região Incerta	Diferença entre fundo e marcadores
5	<code>cv2.watershed</code>	Propagação dos marcadores e geração das fronteiras

```
import cv2

# 1. Imagem sintética e Transformada de Distância
f_sint = np.zeros((20, 20), dtype=np.uint8)
cv2.circle(f_sint, (6, 10), 5, 255, -1)
cv2.circle(f_sint, (14, 10), 5, 255, -1)
dist = mm.dist(f_sint)

# 2. Marcadores (picos da distância)
m = (dist > 0.8 * dist.max()).astype(np.uint8) * 255

# 3. Execução do Watershed0 condicional (m=marcadores primeiro, mask=f_sint)
w_reg = mm.watershed0(m, mask=f_sint, op='region')
w_line = mm.watershed0(m, mask=f_sint, op='line')

#w_reg = mm.watershedB(m, mask=f_sint, op='region')
#w_line = mm.watershedB(m, mask=f_sint, op='line')
```

```
w_reg = mm.watershed(m, mask=f_sint, op='region')
w_line = mm.watershed(m, mask=f_sint, op='line')

# 4. Exibição dos resultados
mm.show([f_sint, dist, m, w_reg, w_line], cols=5, figsize=(16, 4),
        titles=["Original", "Distância", "Marcadores", "Regiões", "Linhas"])
```

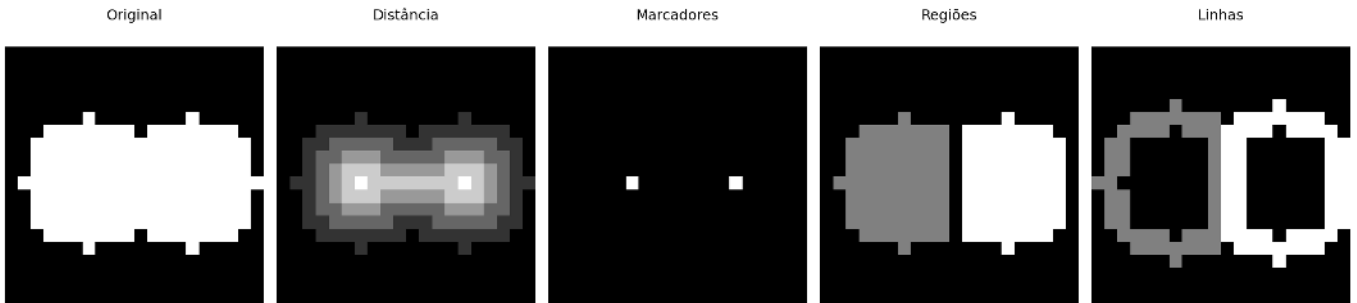


Figura 4.25: *Pipeline watershed* delimitado por máscara em imagem binária 20×20.

Aplicação em moedas sobrepostas:

```
img_base = img_coins_gray

# 1. Simular moedas sobrepostas/conectadas (usando a máscara binária base)
img_sobrepostas = mm.dil(img_final, mm.sebox(40))

# 2. Abertura morfológica para limpar ruídos
opening = mm.open(img_sobrepostas, mm.sebox(2))

# 3. Transformada de Distância
dist = mm.dist(opening)
dist_vis=(255*(dist/dist.max())).astype(np.uint8) if dist.max() > 0 else dist.astype(np.uint8)

# 4. Picos seguros (Marcadores das moedas)
picos = (dist > 0.5 * dist.max()).astype(np.uint8) * 255

# 5. Execução do Watershed (Ajustado para usar a nova assinatura)
# Passamos 'opening' direto em mask, pois ela delimita o escopo de expansão das moedas
ws_region = mm.watershed(picos, mask=opening, op='region')
ws_line = mm.watershed(picos, mask=opening, op='line')

# 6. Contagem de objetos (Ignora fundo 0)
labels = np.unique(ws_region)
labels = labels[labels > 0]
print(f"Objetos detectados: {len(labels)}")

# 7. Anotação Final
img_annotated = cv2.cvtColor(img_base, cv2.COLOR_GRAY2BGR)

for idx, label_id in enumerate(labels):
    mask_reg = (ws_region == label_id).astype(np.uint8)
    if mask_reg.sum() < 500: continue

    cy, cx = np.mean(np.where(mask_reg), axis=1).astype(int)
    cv2.putText(img_annotated, str(idx + 1), (cx - 25, cy + 20),
                cv2.FONT_HERSHEY_SIMPLEX, 3.2, (0, 255, 0), 8, cv2.LINE_AA)

# Desenha as linhas de separação em vermelho
```

```
mask_ann = mm.dil(ws_line, np.ones((11, 11), np.uint8))
img_annotated[mask_ann > 0] = [255, 0, 0]

# Exibição
mm.show(
    [img_base, img_sobrepostas, dist_vis, picos, ws_region, img_annotated],
    titles=["Original", "Sobrepostas", "Distância", "Marcadores", "Watershed", "Anotado"],
    cols=3, rows=2, figsize=(12, 8)
)
```

Objetos detectados: 12

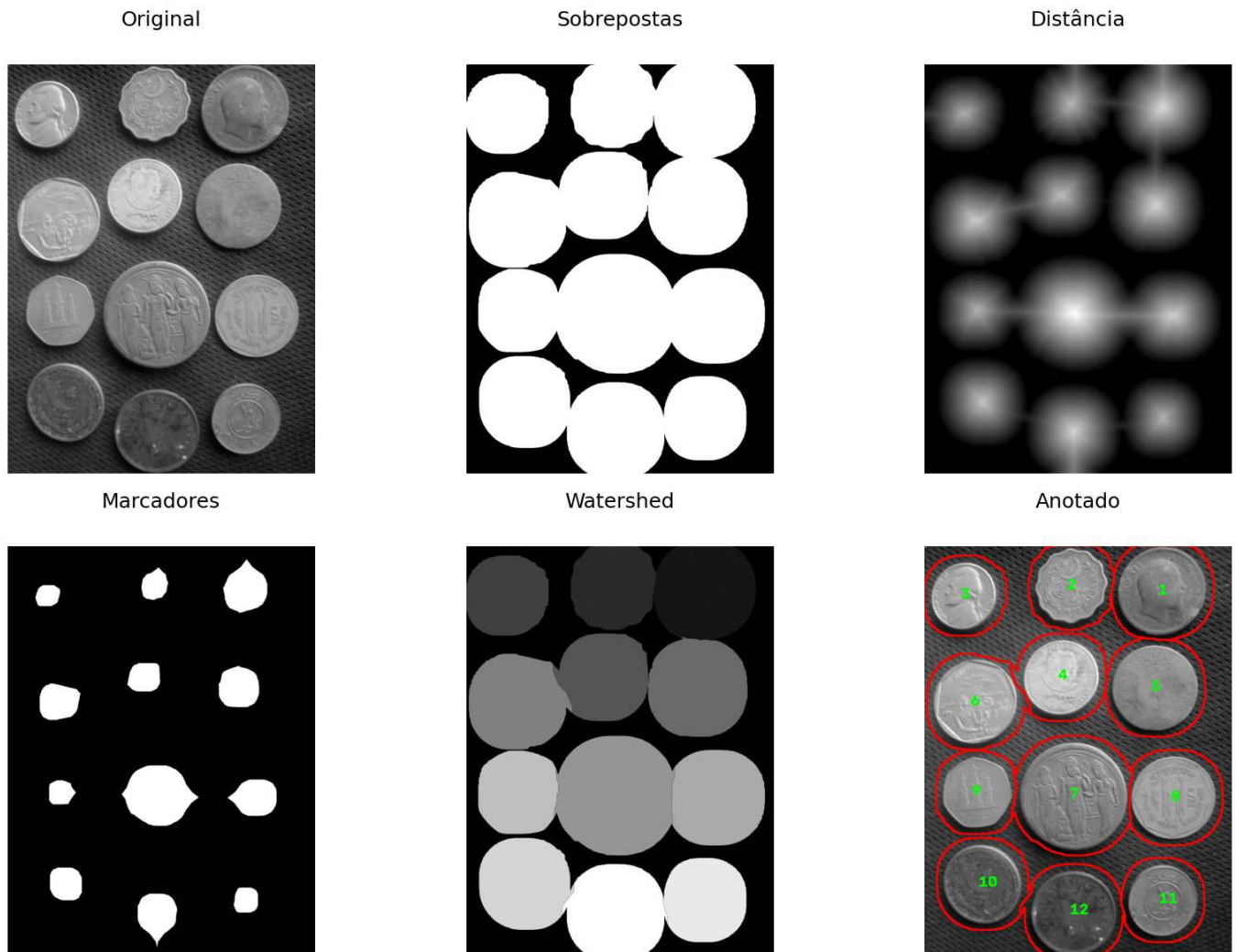


Figura 4.26: *Pipeline Watershed* para separação de moedas sobrepostas: da máscara binária dilatada até os contornos finais anotados sobre a imagem original.

4.5 Extração de Componentes e Descritores de Forma

Após a segmentação e o refinamento morfológico, o próximo passo consiste em identificar individualmente cada objeto presente na imagem e extrair suas propriedades geométricas. Essa etapa é fundamental para tarefas de medição, classificação e reconhecimento de padrões.

Um **componente conexo** é um conjunto maximal de pixels pertencentes ao objeto que permanecem mutuamente conectados segundo uma relação de conectividade previamente definida (4- ou 8-conectividade). Após a rotulação, cada componente recebe um identificador único, permitindo que suas características sejam analisadas individualmente.

O OpenCV oferece duas abordagens complementares para essa análise, resumidas na Table 4.8.

Tabela 4.8: Comparação entre as abordagens baseadas em componentes conexos e em contornos.

	<code>connectedComponentsWithStats</code>	<code>findContours</code>
Retorna	rótulo por pixel e estatísticas por componente	sequência de pontos que descreve a borda
Descritores diretos	área, <i>bounding box</i> e centróide	perímetro, forma e hierarquia
Objetos em contato	tende a fundir regiões conectadas	tende a produzir um único contorno externo
Uso típico	contagem, filtragem e rotulação	análise geométrica e descritores de forma

4.5.1 Rotulação e Estatísticas de Componentes

A função `cv2.connectedComponentsWithStats` realiza simultaneamente a rotulação dos componentes conexos e a extração de descritores básicos de cada região. O operador retorna:

- uma imagem de rótulos (`labels`);
- estatísticas geométricas (`stats`);
- coordenadas dos centróides (`centroids`).

As estatísticas incluem área, largura, altura e posição da *bounding box* mínima alinhada aos eixos da imagem.

A Figure 4.27 ilustra a aplicação desse operador após o *pipeline* de segmentação das moedas. Cada componente conexo recebe uma cor distinta e sua área é anotada diretamente sobre a imagem.

```
# 1. Rotulagem e estatísticas
n, labels, stats, centroids = cv2.connectedComponentsWithStats(
    img_final, connectivity=8
)

# 2. Coloração dos componentes
np.random.seed(4)
colors = np.random.randint(50, 255, (n, 3), dtype=np.uint8)
colors[0] = [0, 0, 0] # fundo preto
img_colored = colors[labels]
img_annotated = img_colored.copy()

# 3. Tabela de descritores
print(f"Componentes detectados (excluindo fundo): {n - 1}")
print(f"\n{'ID':>4} {'Área':>8} {'cx':>6} {'cy':>6} {'w':>6} {'h':>6}")
print("-" * 42)
for i in range(1, n):
    area = stats[i, cv2.CC_STAT_AREA]
    cx, cy = int(centroids[i, 0]), int(centroids[i, 1])
    w, h = stats[i, cv2.CC_STAT_WIDTH], stats[i, cv2.CC_STAT_HEIGHT]
    print(f"{i:>4} {area:>8} {cx:>6} {cy:>6} {w:>6} {h:>6}")
    for cor, esp in [(0,0,0), 10), ((255,0,0), 5)]:
        cv2.putText(img_annotated, f"{i}: {area}",
                    (cx - 150, cy + 18),
                    cv2.FONT_HERSHEY_SIMPLEX, 2.0, cor, esp, cv2.LINE_AA)

mm.show(
    [img_coins_gray, img_final, img_colored, img_annotated],
    titles=["Original", "Segmentação Final", "Componentes Conexos", "Áreas Anotadas"],
    cols=4, figsize=(18, 6)
)
```

Componentes detectados (excluindo fundo): 12

ID	Área	cx	cy	w	h
1	231969	1484	280	553	542
2	158967	917	250	453	468
3	139229	250	312	433	419
4	175550	857	821	474	465
5	222882	1442	889	539	531
6	210043	316	977	527	516
7	343376	934	1559	667	665
8	213641	1555	1574	530	515
9	147880	328	1543	432	438
10	187280	363	2111	487	492
11	150110	1487	2215	433	444
12	215387	932	2292	528	522

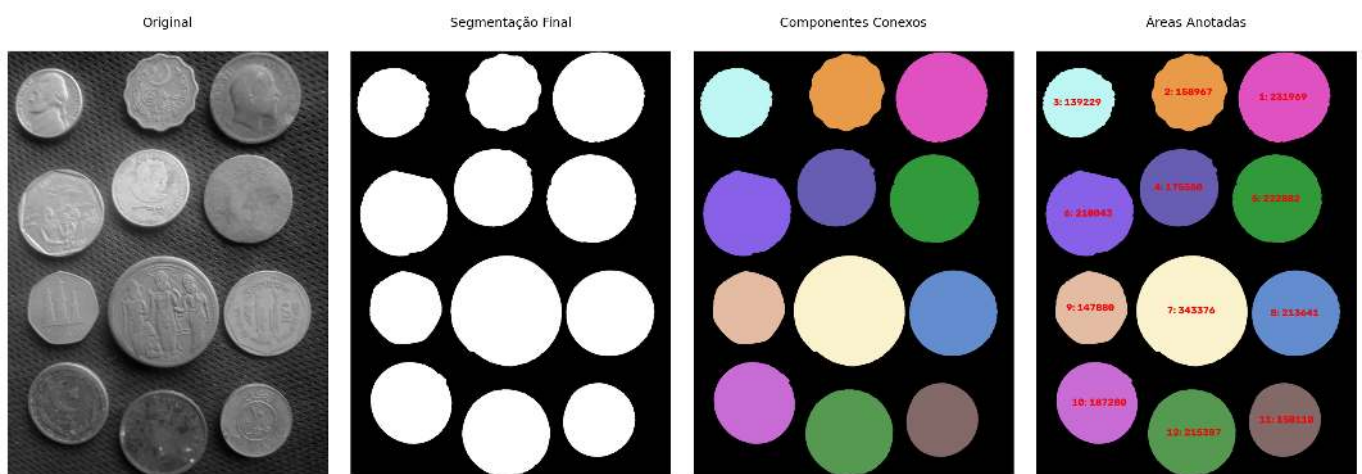


Figura 4.27: Componentes conexos extraídos após o *pipeline* CLAHE → Otsu → limpeza morfológica. Cada objeto é colorido com cor distinta e anotado com sua área em pixels.

4.5.2 Descritores de Forma

Enquanto `connectedComponentsWithStats` opera sobre regiões, `cv2.findContours` atua diretamente sobre suas fronteiras. O operador retorna, para cada objeto, uma sequência ordenada de pontos que descreve seu contorno.

A partir dessa representação é possível calcular descritores geométricos que não são fornecidos diretamente pela rotulação de componentes:

- **Área** (`cv2.contourArea`)
- **Perímetro** (`cv2.arcLength`);
- **Circularidade** ($C = \frac{4\pi A}{P^2}$, onde A é a área e P o perímetro);
- **Aproximação poligonal** (`cv2.approxPolyDP`);
- **Fecho convexo** (*convex hull*);
- **Hierarquia de contornos**, permitindo representar relações pai-filho entre regiões e seus buracos internos.

A circularidade assume valor máximo igual a 1 para um círculo perfeito e diminui à medida que o objeto se torna mais alongado ou apresenta irregularidades em sua borda.

A Figure 4.28 apresenta os contornos extraídos das moedas segmentadas, juntamente com os valores de circularidade calculados para cada objeto.

```
contornos, hierarquia = cv2.findContours(
    img_final, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

img_contornos = cv2.cvtColor(img_coins_gray, cv2.COLOR_GRAY2BGR)
img_circulares = img_contornos.copy()

print(f"Contornos detectados: {len(contornos)}")
print(f"\n{'ID':>4} {'Área':>8} {'Perímetro':>10} {'Circularidade':>14}")
print("-" * 42)

for i, cnt in enumerate(contornos, start=1):
    area = cv2.contourArea(cnt)
    perim = cv2.arcLength(cnt, closed=True)
    circ = (4 * np.pi * area / perim**2) if perim > 0 else 0
    M = cv2.moments(cnt)
    cx = int(M["m10"] / M["m00"]) if M["m00"] > 0 else 0
    cy = int(M["m01"] / M["m00"]) if M["m00"] > 0 else 0

    print(f"{i:>4} {area:>8.0f} {perim:>10.1f} {circ:>14.3f}")

    cv2.drawContours(img_contornos, [cnt], -1, (0, 255, 0), 3)
    cv2.drawContours(img_circulares, [cnt], -1, (0, 255, 0), 3)
    for cor, esp in [(0,0,0), 8), ((255,0,0), 3)]:
        cv2.putText(img_circulares, f"{circ:.2f}",
                    (cx - 80, cy + 15),
                    cv2.FONT_HERSHEY_SIMPLEX, 3.6, cor, esp, cv2.LINE_AA)

mm.show(
    [img_final, img_contornos, img_circulares],
    titles=["Segmentação Final", "Contornos", "Circularidade"],
    cols=3, figsize=(18, 6)
)
```

Contornos detectados: 12

ID	Área	Perímetro	Circularidade

1	214626	1769.6	0.861
2	149480	1465.1	0.875
3	186570	1654.9	0.856
4	147252	1458.6	0.870
5	212886	1756.3	0.867
6	342424	2232.5	0.863
7	209282	1767.7	0.842
8	222108	1809.7	0.852
9	174854	1616.4	0.841
10	138602	1471.5	0.804
11	158306	1538.7	0.840
12	231181	1847.4	0.851



Figura 4.28: Contornos extraídos com *findContours*. Cada moeda é anotada com sua circularidade — valores próximos de 1 confirmam forma circular.

4.5.3 Conexão com Detecção de Objetos Moderna

Os descritores extraídos nas seções anteriores — especialmente *bounding boxes*, centróides, áreas e medidas de forma — estabelecem uma ponte natural entre a segmentação morfológica clássica e os sistemas modernos de detecção de objetos. Embora as técnicas estudadas neste capítulo utilizem operações sobre pixels e regiões segmentadas, muitas das representações produzidas são diretamente compatíveis com os formatos empregados em modelos contemporâneos de visão computacional.

Detectores baseados em aprendizado profundo, como a família YOLO (*You Only Look Once*) (Redmon et al., 2016), operam diretamente sobre imagens coloridas e produzem, para cada objeto detectado, uma *bounding box* descrita pelo centro (cx, cy) e pelas dimensões (w, h), além de uma classe e uma pontuação de confiança. Essa representação compartilha a mesma estrutura geométrica básica obtida por `connectedComponentsWithStats`, embora seja produzida por um modelo aprendido e não por segmentação explícita.

A Figure 4.29 ilustra como as *bounding boxes* obtidas por morfologia podem ser exportadas no formato YOLO para compor conjuntos de dados utilizados no treinamento ou na avaliação de detectores.

```
H_img, W_img = img_coins_gray.shape
img_bbox = cv2.cvtColor(img_coins_gray, cv2.COLOR_GRAY2BGR)

CLASSE = 0 # 0 = moeda (única categoria neste exemplo)

print(f"{'cls':>4} {'cx_n':>8} {'cy_n':>8} {'w_n':>8} {'h_n':>8} ← formato YOLO")
print("-" * 54)

yolo_linhas = []
for i in range(1, n):
    x0 = stats[i, cv2.CC_STAT_LEFT]
    y0 = stats[i, cv2.CC_STAT_TOP]
    w = stats[i, cv2.CC_STAT_WIDTH]
    h = stats[i, cv2.CC_STAT_HEIGHT]

    cx_n = (x0 + w / 2) / W_img
    cy_n = (y0 + h / 2) / H_img
    w_n = w / W_img
    h_n = h / H_img

    yolo_linhas.append(f"{CLASSE} {cx_n:.4f} {cy_n:.4f} {w_n:.4f} {h_n:.4f}")
print(f"{CLASSE:>4} {cx_n:>8.4f} {cy_n:>8.4f} {w_n:>8.4f} {h_n:>8.4f}")
```

```

cv2.rectangle(img_bbox, (x0, y0), (x0 + w, y0 + h), (0, 255, 0), 4)
cv2.putText(img_bbox, f"moeda", (x0 + 8, y0 + 60),
            cv2.FONT_HERSHEY_SIMPLEX, 3.8, (255, 0, 0), 5, cv2.LINE_AA)

# Exportar arquivo de anotação no formato YOLO
with open("moedas.txt", "w") as f:
    f.write("\n".join(yolo_linhas))
print("\nAnotação salva em moedas.txt")

mm.show(
    [img_coins_gray, img_final, img_bbox],
    titles=["Original", "Segmentação Final", "Bounding Boxes (formato YOLO)"],
    cols=3, figsize=(18, 6)
)

```

cls	cx_n	cy_n	w_n	h_n	← formato YOLO
0	0.7753	0.1102	0.2880	0.2117	
0	0.4784	0.0984	0.2359	0.1828	
0	0.1331	0.1225	0.2255	0.1637	
0	0.4464	0.3217	0.2469	0.1816	
0	0.7523	0.3471	0.2807	0.2074	
0	0.1664	0.3812	0.2745	0.2016	
0	0.4862	0.6104	0.3474	0.2598	
0	0.8115	0.6154	0.2760	0.2012	
0	0.1724	0.6023	0.2250	0.1711	
0	0.1893	0.8258	0.2536	0.1922	
0	0.7763	0.8652	0.2255	0.1734	
0	0.4854	0.8953	0.2750	0.2039	

Anotação salva em moedas.txt

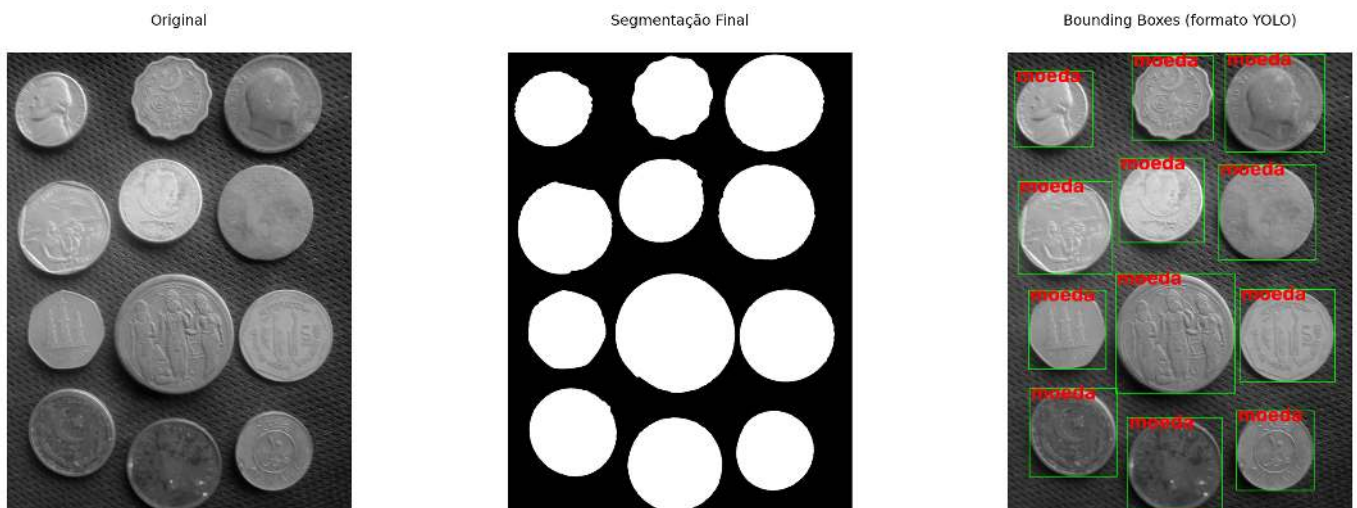


Figura 4.29: *Bounding boxes* derivadas dos componentes conexos sobrepostas à imagem original. As anotações são exportadas no formato YOLO (*classe cx cy w h*), com coordenadas normalizadas para o intervalo $[0,1]$.

O formato YOLO armazena cada objeto em uma linha contendo cinco campos:

classe cx cy w h

onde (cx, cy) representa o centro da *bounding box* e (w, h) suas dimensões. Todos os valores geométricos são normalizados para o intervalo $[0,1]$ em relação à largura e à altura da imagem. A **classe** é um identificador

inteiro associado a uma categoria definida pelo conjunto de dados (por exemplo, $0 \rightarrow \text{moeda}$). Quando há múltiplas categorias — como moeda de ouro (0), moeda de prata (1) e disco plástico (2) — basta atribuir o identificador correspondente a cada objeto antes da exportação, mantendo exatamente o mesmo formato de anotação. No exemplo anterior, essas informações foram armazenadas no arquivo `moedas.txt`.

O fluxo apresentado neste capítulo — segmentação $\rightarrow$ rotulação $\rightarrow$ extração de *bounding boxes* — corresponde conceitualmente à etapa de **anotação** (*labeling*) empregada na construção de conjuntos de treinamento para detectores modernos. Ferramentas especializadas, como Label Studio e Roboflow, automatizam esse processo em imagens complexas, mas a lógica fundamental permanece a mesma: associar a cada objeto uma região de interesse e uma classe. Em cenários controlados, com fundo uniforme e objetos bem separados, técnicas morfológicas podem inclusive gerar anotações automaticamente ou servir como ponto de partida para a rotulação manual, reduzindo significativamente o esforço de construção do conjunto de dados. Em aplicações reais mais complexas, entretanto, a validação humana continua sendo necessária para garantir a qualidade das anotações.

Avaliação: IoU (*Intersection over Union*)

Uma forma simples de avaliar a qualidade de uma segmentação consiste em compará-la com uma máscara de referência (*ground truth*). A métrica mais utilizada para essa finalidade é a **IoU** (*Intersection over Union*):

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \quad (4.16)$$

onde A representa a segmentação produzida pelo algoritmo e B a segmentação de referência.

O valor da IoU varia entre 0 e 1. Quanto maior o valor, maior a sobreposição entre as máscaras. Uma IoU igual a 1 indica correspondência perfeita entre a segmentação obtida e a referência.

A mesma métrica também é amplamente utilizada em detecção de objetos, sendo aplicada às *bounding boxes* previstas e anotadas. Nessa área, valores de IoU superiores a 0,5 são frequentemente adotados como critério mínimo para considerar uma detecção correta.

Além da Morfologia

As técnicas estudadas neste capítulo segmentam objetos explorando conectividade espacial, operações morfológicas e relevo topográfico. Existem, entretanto, abordagens alternativas baseadas em agrupamento de características, como o algoritmo *k-means*, modelos de mistura Gaussiana (GMM) e métodos mais recentes baseados em aprendizado profundo. Essas técnicas serão retomadas na Parte II do livro, dedicada à Visão Computacional.

4.6 Resumo

Neste capítulo foram apresentadas as principais técnicas de segmentação e morfologia matemática, concluindo o estudo do PDI no domínio espacial.

- **Pré-processamento e limiarização:** A combinação entre equalização adaptativa CLAHE e o método de Otsu mostrou-se eficaz para induzir separação bimodal no histograma e simplificar a binarização de imagens com iluminação não uniforme.
- **Erosão e dilatação:** Operadores morfológicos fundamentais baseados na busca por mínimos e máximos locais em uma vizinhança definida pelo elemento estruturante B . São operadores duais pelo complemento e foram implementados por meio das funções `mm.ero` e `mm.dil`.
- **Abertura e fechamento:** Composições de erosão e dilatação que permitem remover ruídos, suavizar contornos e preencher pequenas lacunas, preservando a estrutura global dos objetos.
- **Reconstrução morfológica:** Processo geodésico iterativo que propaga um marcador dentro dos limites impostos por uma máscara, constituindo a base de operadores como `mm.clohole` e `mm.edgeoff`.

- **Pipeline de limpeza binária:** Fluxo consolidado composto por CLAHE $\rightarrow$ Otsu $\rightarrow$ abertura $\rightarrow$ `mm.clohole` $\rightarrow$ abertura restrita $\rightarrow$ `mm.edgeoff`, produzindo máscaras adequadas para análise quantitativa.
- **Morfologia em tons de cinza:** Extensão algébrica baseada em mínimos e máximos ponderados, viabilizando operadores como gradiente morfológico e filtros *top-hat* para realce de estruturas locais.
- **Transformada de Distância e Watershed:** A Transformada de Distância permitiu gerar marcadores automáticos para o algoritmo *watershed*, possibilitando a separação de objetos adjacentes ou parcialmente sobrepostos.
- **Componentes conexos e descritores:** A rotulação de regiões (`cv2.connectedComponentsWithStats`) e a extração de contornos (`cv2.findContours`) permitiram calcular descritores geométricos como área, centróide, perímetro, circularidade e *bounding boxes*.
- **Conexão com Visão Computacional Moderna:** As *bounding boxes* extraídas por morfologia foram exportadas no formato YOLO, evidenciando a ligação entre técnicas clássicas de segmentação e sistemas modernos de detecção de objetos.

O Capítulo 5 introduzirá técnicas de processamento no **domínio da frequência**, abordando a **Transformada de Fourier**, filtragem espectral e os fundamentos da **compressão de imagens**, incluindo DCT, JPEG e *wavelets*.

4.7 Uso do Gemini Notebook como Tutor Complementar

Nesta edição, o uso do **Gemini Notebook** é incentivado como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo deste capítulo.

 Estude com o Tutor Inteligente

 [ACESSAR Gemini Notebook: CAPÍTULO 04](#)

Aviso sobre Conteúdo Gerado por IA

Embora seja uma ferramenta valiosa de apoio aos estudos, o Gemini Notebook pode eventualmente produzir respostas incompletas, imprecisas ou incorretas. Recomenda-se validar as informações consultando o material do capítulo, livros, artigos científicos e outras fontes acadêmicas confiáveis. Sempre que possível, execute e experimente os exemplos práticos apresentados ao longo do texto para consolidar a compreensão dos conceitos.

4.8 Lista de Exercícios

1. (10%) Implemente manualmente o critério de Otsu sem utilizar `cv2.threshold`. Calcule a variância interclasses $\sigma_B^2(T)$ para todos os limiares $T \in [0, 255]$ usando `mm.hist`, identifique o limiar ótimo T^* e compare o resultado com o valor obtido pelo OpenCV. Plote σ_B^2 em função de T e destaque o ponto de máximo.
2. (15%) Aplique limiarização adaptativa com blocos de tamanho 11, 31 e 51 a uma imagem contendo iluminação não uniforme. Compare os resultados com a limiarização global de Otsu e discuta as vantagens e limitações de cada abordagem.
3. (15%) Execute o *pipeline watershed* da imagem de moedas variando o limiar aplicado à Transformada de Distância (0.3, 0.5 e 0.7 vezes o valor máximo). Explique como esse parâmetro influencia a geração dos marcadores, a separação de objetos adjacentes e a ocorrência de sobre-segmentação.
4. (15%) Utilizando `mm.drawImg`, construa uma demonstração visual passo a passo da erosão de uma imagem binária 7×7 com elemento estruturante quadrado 3×3 . Para cada posição analisada, indique

se o elemento estruturante está completamente contido no objeto e justifique o valor atribuído ao pixel de saída.

5. (15%) Demonstre experimentalmente a dualidade entre erosão e dilatação verificando a identidade $(A \ominus B)^c = A^c \oplus \hat{B}$ utilizando `mm.ero`, `mm.dil` e `mm.bnot`. Calcule a diferença pixel a pixel entre os dois lados da equação e apresente o resultado utilizando `mm.histImg` ou visualização equivalente.
6. (15%) Implemente manualmente o gradiente morfológico utilizando apenas `mm.ero` e `mm.dil`, comparando o resultado com `cv2.morphologyEx(img, cv2.MORPH_GRADIENT, B)`. Avalie o efeito de diferentes elementos estruturantes (quadrado 3×3 , disco 5×5 e linha 1×9) sobre a detecção de bordas.
7. (15%) Construa um *pipeline* completo para contagem e classificação de moedas por tamanho (pequena, média e grande) utilizando área e circularidade como descritores. Gere uma máscara de referência (*ground truth*) manualmente e calcule a métrica IoU (*Intersection over Union*) para avaliar a qualidade da segmentação. Apresente os resultados em tabela e por meio de visualizações produzidas com `mm.show`.

Referências do Capítulo

A fundamentação teórica deste capítulo baseia-se nas seguintes obras:

- Gonzalez; Woods (2018) para os conceitos de segmentação, limiarização de Otsu, Transformada de Distância, *watershed*, morfologia matemática e descritores de forma.
- Matheron (1975) e Serra (1982) para a fundamentação teórica original, formulação algébrica e desenvolvimento da Morfologia Matemática.
- Szeliski (2022) para segmentação baseada em regiões, rotulação de componentes conexos, *watershed* baseado em marcadores e avaliação de segmentação por meio da métrica IoU.
- Bradski; Kaehler (2008) para a utilização prática da biblioteca OpenCV, incluindo funções como `cv2.distanceTransform`, `cv2.watershed`, `cv2.connectedComponentsWithStats` e `cv2.findContours`.
- Redmon et al. (2016) para a introdução aos detectores modernos da família YOLO e sua relação com descritores geométricos como *bounding boxes* extraídas por segmentação.
- Singh; Lloyd; Flicker (2024) para a construção de labirintos complexos baseados em ciclos hamiltonianos sobre mosaicos quasicristalinos, utilizados como exemplo de aplicação da Transformada de Distância Geodésica e de algoritmos de busca de caminhos.
- Zampirolli et al. (2025) para a implementação dos operadores morfológicos, transformadas geodésicas e resolução de labirintos por propagação de distâncias em domínios restritos.



4.9 Parte Prática com Exercícios de Programação

Esta lista transforma os conceitos do Capítulo 4 em uma trilha prática de segmentação e morfologia matemática. Os EPs começam com limiarização e avançam até rotulação e descritores de componentes, sempre com matrizes pequenas para que cada pixel possa ser conferido à mão.

! Regra comum dos EPs morfológicos

Nas operações com vizinhança, **não faça padding**. Para cada pixel, avalie apenas as posições do elemento estruturante que caem dentro do domínio da imagem. Essa é a mesma ideia das implementações didáticas em `morph.py`, como `mm.dil0`, `mm.ero0`, `mm.dil1` e `mm.label0`: a vizinhança é recortada pelo domínio válido da imagem.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Para avaliar os testes, execute `TestSuite("EP04_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```
codigo = """
from morph import mm
# ... seu código aqui ...
"""

TestSuite("EP04_01").run_code(codigo)
```

4.9.1 EP04_01 Limiarização Global por Limiar Fixo

Em **scanners de documentos** e em **sistemas de leitura de código de barras**, a primeira etapa do processamento é sempre separar o que é “objeto” (tinta, texto, barras) do que é “fundo” (papel, embalagem). A **limiarização global** faz exatamente isso: compara cada pixel a um único limiar T e decide, em tempo real, se ele pertence à classe clara ou à classe escura. É o operador de segmentação mais simples — e mesmo assim, está por trás de boa parte dos *pipelines* industriais de inspeção visual. Ver na Figure 4.30 uma simulação deste EP.

4.9.1.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Limiar:** Ler o inteiro T (limiar de decisão).
3. **Dados:** Ler os valores inteiros da matriz original linha a linha.
4. **Mapeamento:** Para cada pixel p , calcular o novo valor pela equação:

$$p' = \begin{cases} 255, & \text{se } p > T \\ 0, & \text{se } p \leq T \end{cases}$$

5. **Saída:** Exibir a matriz binarizada com dimensões $L \times C$.

4.9.1.2 Restrições Computacionais

- **Binarização:** A saída contém **apenas** os valores 0 ou 255.
- **Comparação estrita:** O critério usa $> T$ (pixels iguais a T tornam-se fundo).
- **Tipo:** O resultado final deve ser inteiro.
- **Observação:** Este EP segue a convenção da OpenCV (`cv2.THRESH_BINARY`): apenas pixels com valor **maior que** T tornam-se brancos (255); pixels com valor **igual a** T permanecem pretos (0).

4.9.1.3 Fundamentação Teórica

Parâmetro	Tipo	Impacto Visual
T pequeno	Inteiro	A maioria dos pixels torna-se branca
T grande	Inteiro	A maioria dos pixels torna-se preta
T bem escolhido	Inteiro	Separa nitidamente objeto e fundo

4.9.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro T .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz binarizada em L linhas e C colunas, valores 0 ou 255 separados por espaço.

4.9.1.5 Exemplos

Entrada	Saída	Observação
2 4 100 0 99 100 180 255 30 120 80	0 0 0 255 255 0 255 0	$T = 100$: apenas pixels com valor maior que 100 tornam-se brancos; por isso, 99 e 100 tornam-se pretos.
1 3 0 0 50 255	0 255 255	
		$T = 0$: apenas pixels com valor estritamente maior que 0 tornam-se brancos.

```
%%writefile EP04_01.py
# Código Python
```

```
Writing EP04_01.py
```

```
TestSuite("EP04_01.py").run()
```

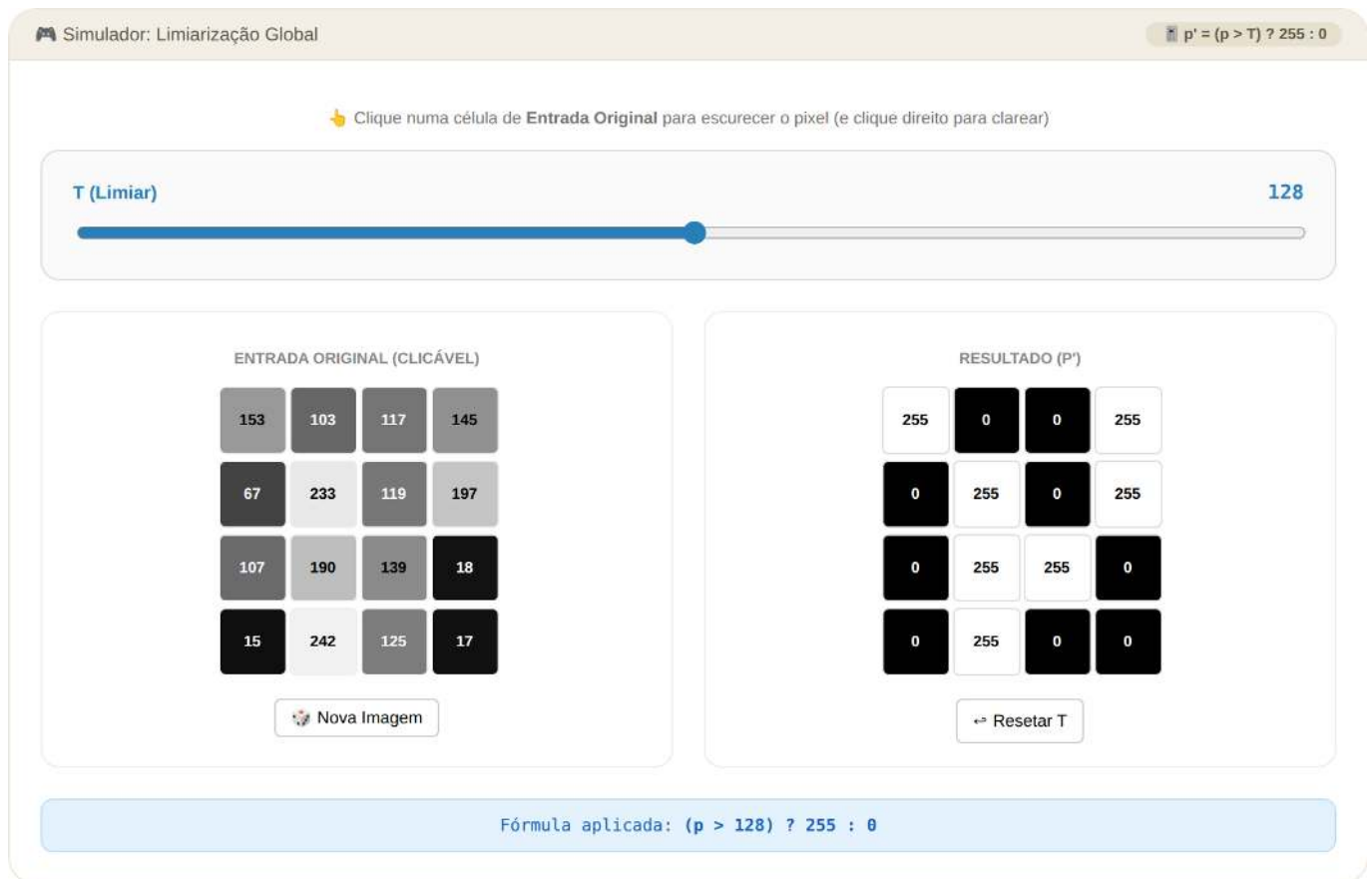


Figura 4.30: Simulador: Limiarização Global por Limiar Fixo

4.9.2 EP04_02 📊 Limiarização Automática de Otsu

Escolher manualmente o limiar T funciona quando a iluminação é estável, mas em **microscopia digital** e em **inspeção de lâminas de sangue**, cada amostra tem um contraste diferente — um limiar fixo falharia de imagem para imagem. O **método de Otsu** resolve isso encontrando, sozinho, o limiar que **maximiza a separação estatística** entre as duas classes de pixels, tornando a segmentação automática e adaptativa. Ver na Figure 4.31 uma simulação deste EP.

4.9.2.1 📋 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Dados:** Ler os valores inteiros da matriz original linha a linha.
3. **Histograma:** Construir o histograma $h[i]$, $i = 0, \dots, 255$, contando quantos pixels têm valor i .
4. **Busca do limiar:** Para cada candidato T de 1 a 255, calcular a **variância entre classes**:

$$\sigma_B^2(T) = \frac{n_0 \cdot n_1}{N^2} (m_0 - m_1)^2$$

onde n_0, n_1 são as quantidades de pixels com valor $< T$ e $\geq T$, m_0, m_1 são suas médias, e $N = L \times C$.

5. **Escolha:** O limiar ótimo T^* é o que maximiza $\sigma_B^2(T)$ (em caso de empate, manter o **primeiro** encontrado).
6. **Aplicação:** Binarizar a imagem usando T^* , aplicando:

$$p' = \begin{cases} 255, & \text{se } p > T^* \\ 0, & \text{se } p \leq T^* \end{cases}$$

4.9.2.2 📌 Restrições Computacionais

- **Candidatos válidos:** Ignorar T que deixe $n_0 = 0$ ou $n_1 = 0$ (classe vazia).
- **Empate:** Sempre manter o **primeiro** T que atingiu o valor máximo de σ_B^2 .
- **Tipo:** T^* e a matriz de saída devem ser inteiros.
- **Convenção OpenCV:** A binarização segue `cv2.THRESH_BINARY`; pixels com valor exatamente igual a T^* tornam-se pretos.

4.9.2.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto
$\sigma_B^2(T)$ alta	Classes bem separadas em T	T é um bom candidato a limiar
Histograma bimodal	Dois “morros” distintos	Otsu encontra o vale entre eles
Histograma unimodal	Um único “morro”	Otsu ainda escolhe <i>algum</i> T , mas a segmentação é pouco confiável

4.9.2.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linhas seguintes: Elementos inteiros da matriz original.

Saída:

- Matriz binarizada em L linhas e C colunas, valores 0 ou 255.

4.9.2.5 📌 Exemplos

Entrada	Saída	Observação
4	0 0 0 255	Histograma bimodal claro: 12 e 200
4	0 0 255 255	
12 12 12 200	0 255 255 255	
12 12 200 200	255 255 255 255	
12 200 200 200		Apenas dois valores: T^* fica no maior
200 200 200 200		
1	0 250	
2		
10 250		

```
%%writefile EP04_02.py
# Código Python
```

```
Writing EP04_02.py
```

```
TestSuite("EP04_02.py").run()
```

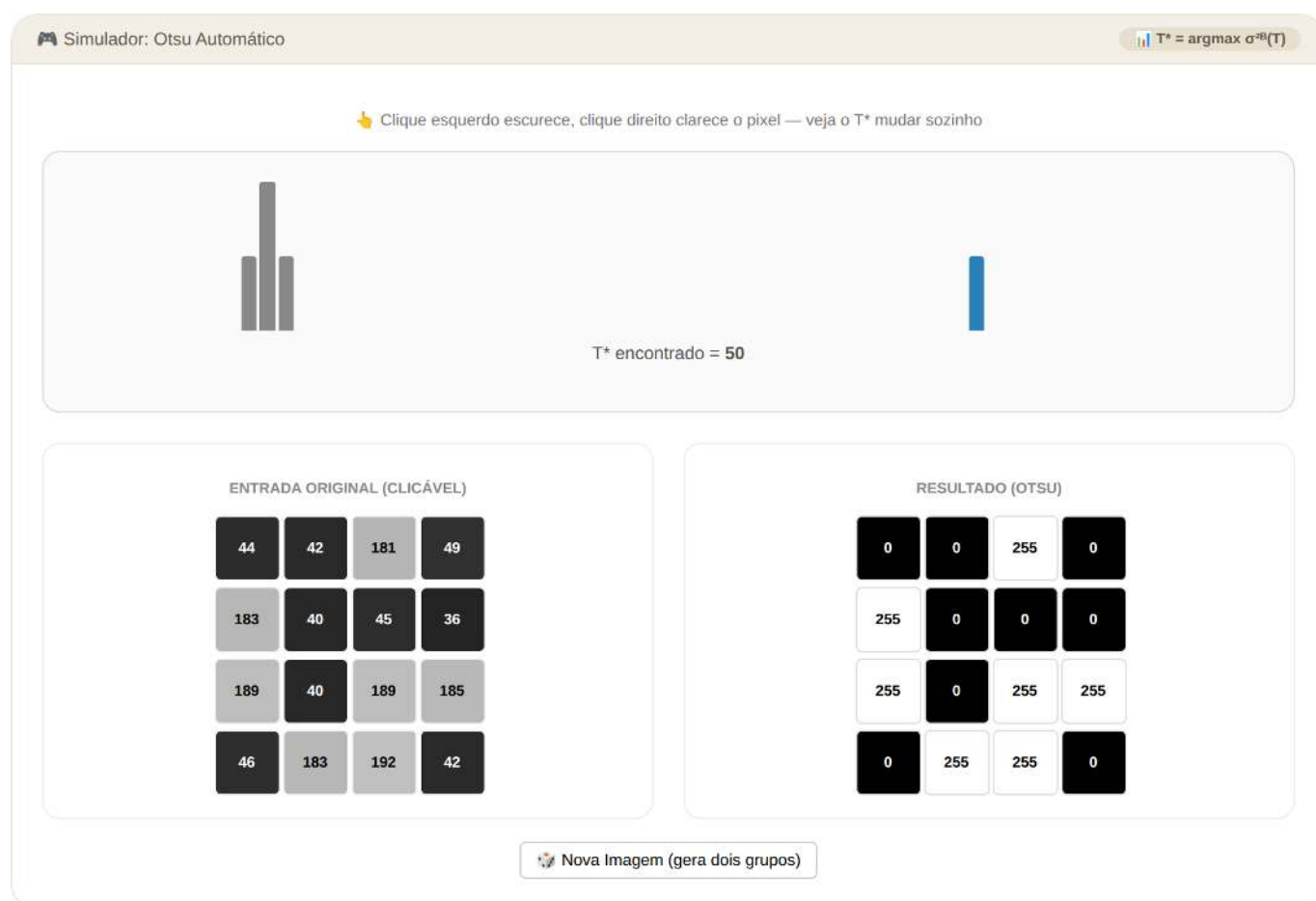



Figura 4.31: Simulador: Limiarização Automática de Otsu

4.9.3 EP04_03 🌱 Dilatação Binária Plana (mm.dil0)

Em **microscopia de partículas** e em **OCR de placas de carro desgastadas**, traços finos ou descontínuos precisam ser “engrossados” para que o reconhecimento funcione. A **dilatação morfológica** faz exatamente isso: expande regiões claras usando um elemento estruturante B — a mesma operação implementada em `morph.py` como `mm.dil0(f, B)`, usada quando B é **plano** (sem pesos, só 0/1). Ver na Figure 4.32 uma simulação deste EP.

4.9.3.1 📋 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Reflexão:** Construir B_{ref} , a versão de B refletida em 180° (linhas e colunas invertidas) — exatamente como faz `mm.dil0` internamente.
6. **Vizinhança sem padding:** Para cada pixel (y, x) , percorrer as posições (by, bx) de B_{ref} centradas em (y, x) , usando o deslocamento

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$ — **não preencher com zeros**.

7. **Mapeamento:** Calcular cada pixel de saída como o **máximo** entre $f(y, x)$ e todos os $f(v_y, v_x)$ válidos cuja posição correspondente em B_{ref} vale 1:

$$g(y, x) = \max \left(f(y, x), \max_{\substack{(v_y, v_x) \text{ válido} \\ B_{ref}(by, bx)=1}} f(v_y, v_x) \right)$$

8. **Saída:** Exibir a matriz g com dimensões $L \times C$.

4.9.3.2 🚫 Restrições Computacionais

- **Sem padding:** Jamais inventar vizinhos fora da imagem; usar apenas os que existem de fato.
- **Reflexão obrigatória:** B deve ser refletido antes de aplicado (é o que diferencia `mm.dil0` de uma simples busca por máximo).
- **Robustez de borda:** Se nenhuma posição válida de $B_{ref} = 1$ cair dentro do domínio para um dado pixel, ele **mantém seu valor original**.

4.9.3.3 🧠 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Dilatação	$g \geq f$ sempre (extensiva)	Regiões claras crescem, buracos escuros encolhem
B maior	Vizinhança mais ampla	Crescimento mais agressivo
Reflexão de B	$B_{ref}(y, x) = B(-y, -x)$	Garante a definição formal de Minkowski da dilatação

4.9.3.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .

- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz g em L linhas e C colunas, valores inteiros separados por espaço.

4.9.3.5 Exemplos

Entrada	Saída	Observação
3	0 9 0	B em cruz simétrico: ponto isolado se expande em cruz
3	9 9 9	
3	0 9 0	
3		
0 1 0		B horizontal: cada pixel “puxa” o máximo dos vizinhos da linha
1 1 1		
0 1 0		
0 0 0		
0 9 0		
0 0 0		
1	200 200 200 80	
4		
1		
3		
1 1 1		
10 200 5 80		

```
%%writefile EP04_03.py
# Código Python
```

```
Writing EP04_03.py
```

```
TestSuite("EP04_03.py").run()
```

4.9.4 EP04_04 Erosão Binária Plana (mm.ero0)

Se a dilatação engrossa, a **erosão** afina. Em **sistemas de contagem de células**, ela é usada para **separar células que se tocam**: ao “comer” as bordas de cada região, conexões finas entre objetos desaparecem antes mesmo de qualquer contagem ser feita. Em **morph.py**, essa é a operação **mm.ero0(f, B)** — a **dual** exata da dilatação, e a única das duas que **não** reflete o elemento estruturante. Ver na Figure 4.33 uma simulação deste EP.

4.9.4.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Vizinhança sem padding (sem reflexão!):** Para cada pixel (y, x) , percorrer as posições (by, bx) de B na ordem original (sem refletir), usando o mesmo deslocamento do EP04_03:

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

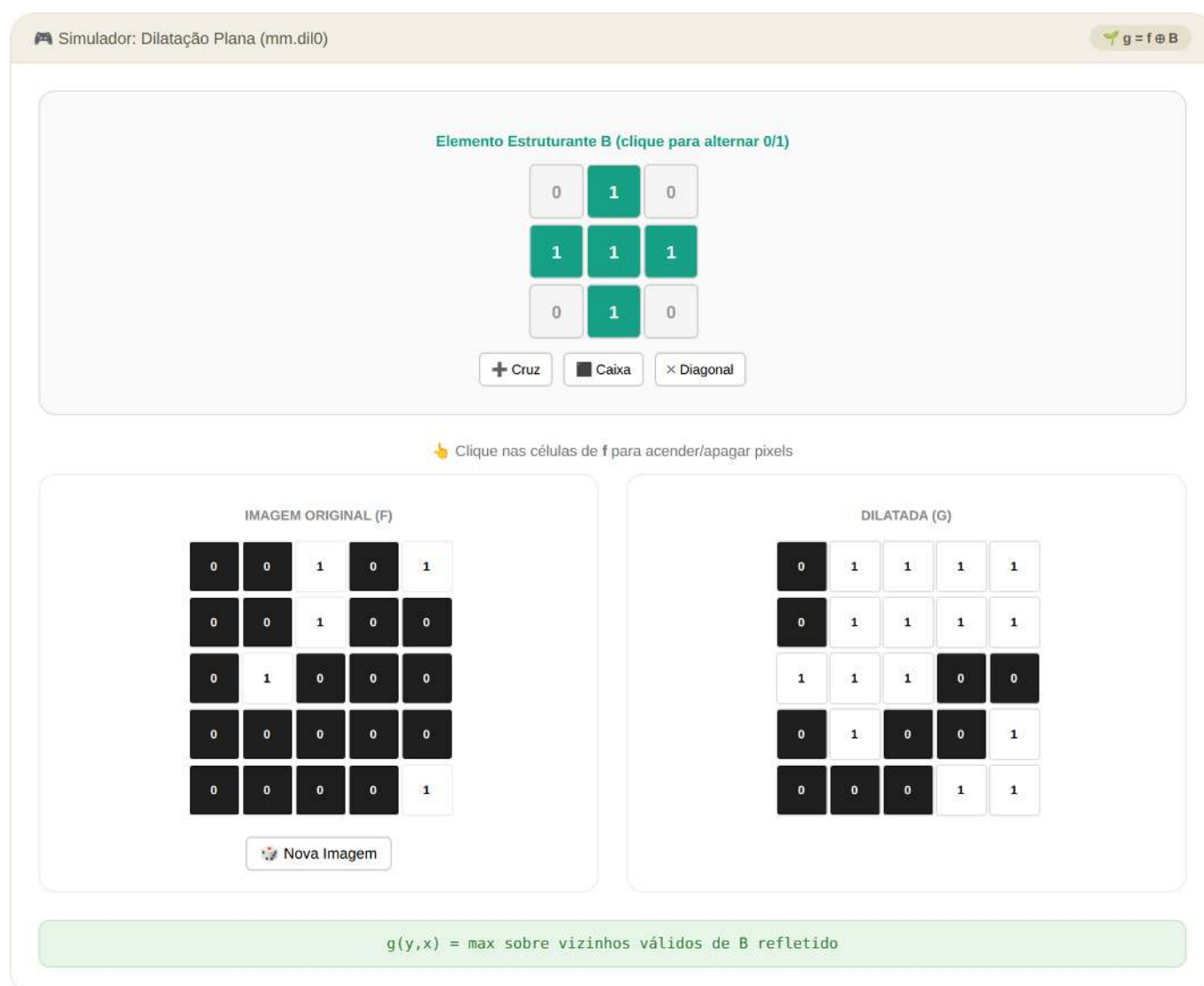


Figura 4.32: Simulador: Dilatação Binária Plana (mm.dil0)

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$. 6. **Mapeamento:** Calcular cada pixel de saída como o **mínimo** entre $f(y, x)$ e todos os $f(v_y, v_x)$ válidos cuja posição correspondente em B vale 1:

$$g(y, x) = \min \left(f(y, x), \min_{\substack{(v_y, v_x) \text{ válido} \\ B(by, bx)=1}} f(v_y, v_x) \right)$$

7. **Saída:** Exibir a matriz g com dimensões $L \times C$.

4.9.4.2 Restrições Computacionais

- **Sem reflexão:** Diferente da dilatação, B é usado **exatamente como lido** — refletir aqui seria um erro conceitual grave.
- **Sem padding:** Vizinhos fora da imagem são simplesmente ignorados, nunca tratados como 0.
- **Robustez de borda:** Se nenhuma posição válida de $B = 1$ cair dentro do domínio, o pixel mantém seu valor original.

4.9.4.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Erosão	$g \leq f$ sempre (anti-extensiva)	Regiões claras encolhem, ruído pontual desaparece
Dualidade	$\text{ero}(f, B) = -\text{dil}(-f, B_{ref})$	Erosão e dilatação são “espelhos” matemáticos
B maior	Erosão mais agressiva	Objetos finos somem completamente

4.9.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz g em L linhas e C colunas, valores inteiros separados por espaço.

4.9.4.5 Exemplos

Entrada	Saída	Observação
3	9 0 9	O “buraco” central (0) se propaga em cruz
3	0 0 0	
3	9 0 9	
3		
0 1 0		
1 1 1		
0 1 0		
9 9 9		
9 0 9		
9 9 9		

Entrada	Saída	Observação
1	10 5 5 80	B horizontal: cada pixel “puxa” o mínimo dos vizinhos da linha
4		
1		
3		
1 1 1		
10 200 5 80		

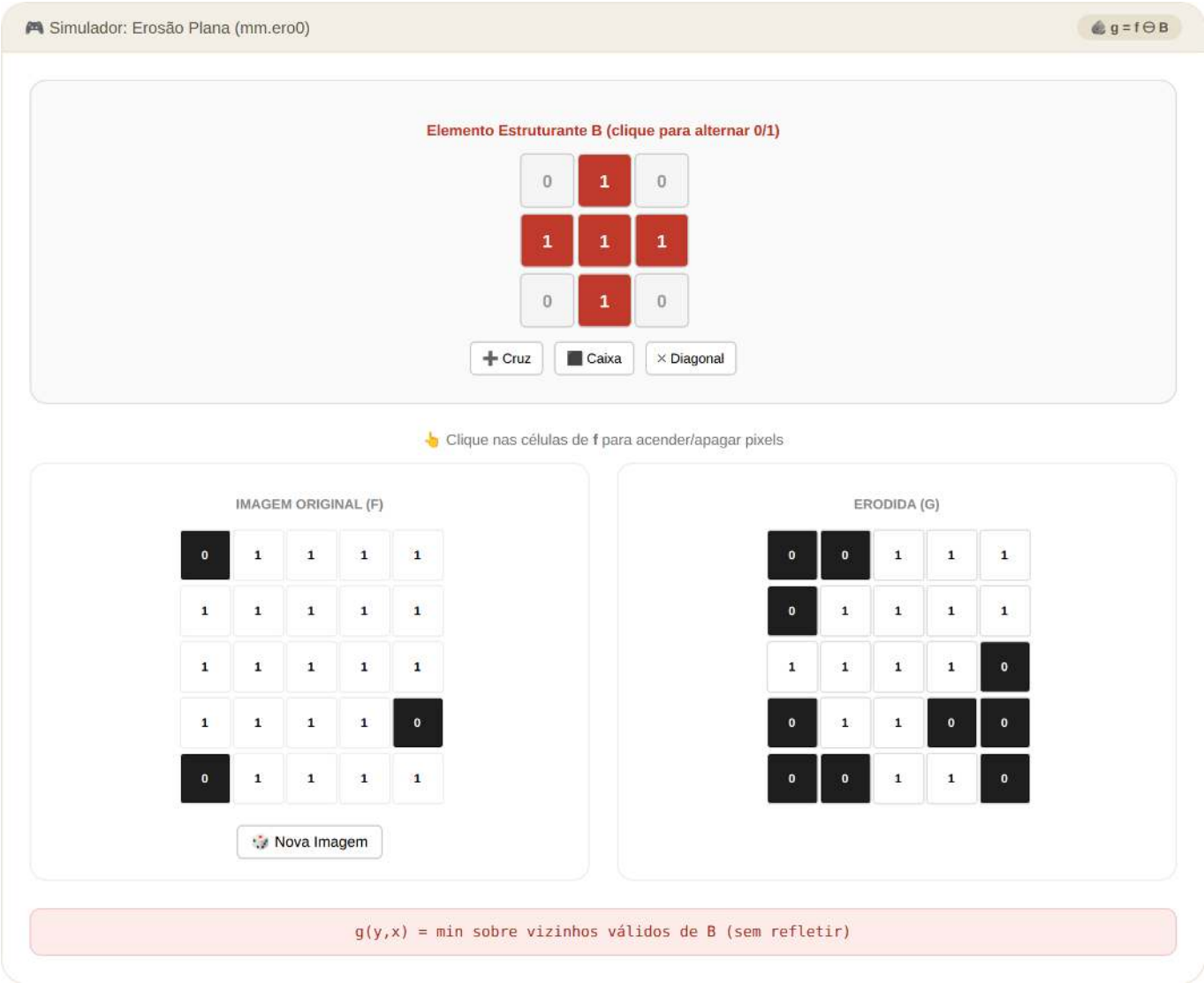


Figura 4.33: Simulador: Erosão Binária Plana (mm.ero0)

```
%%writefile EP04_04.py
# Código Python

Writing EP04_04.py

TestSuite("EP04_04.py").run()
```


4.9.5 EP04_05 Abertura Morfológica (Remoção de Ruído)

Imagens capturadas por **sensores de baixo custo**, como os de drones agrícolas, costumam vir salpicadas de pequenos pontos de ruído — pixels isolados que não representam nada real. Aplicar erosão seguida de dilatação com o **mesmo** elemento estruturante produz a **abertura**: ela “limpa” pontos e protuberâncias finas, mas devolve ao objeto principal praticamente seu tamanho original. É a combinação clássica usada em **pré-processamento de imagens de satélite** antes de qualquer contagem de área plantada. Ver na Figure 4.34 uma simulação deste EP.

4.9.5.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Erosão:** Calcular $e = f \ominus B$, usando exatamente o algoritmo do EP04_04 (sem refletir B , sem padding).
6. **Dilatação:** Calcular $g = e \oplus B$, usando exatamente o algoritmo do EP04_03 (refletindo B , sem padding) — mas agora aplicado sobre e , não sobre f .
7. **Saída:** Exibir a matriz resultante g (a **abertura** de f por B) com dimensões $L \times C$.

4.9.5.2 Restrições Computacionais

- **Ordem fixa:** É **sempre** erosão primeiro, depois dilatação — a ordem inversa define outro operador (fechamento, do próximo EP).
- **Mesmo B :** O elemento estruturante usado na erosão e na dilatação deve ser idêntico.
- **Sem padding em nenhuma das duas etapas.**

4.9.5.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Anti-extensividade	$g \subseteq f$ sempre	A abertura nunca cria pixel novo, só remove
Idempotência	$\text{abertura}(\text{abertura}(f)) = \text{abertura}(f)$	Aplicar de novo não muda mais nada
Pontos isolados	Menores que B	São completamente eliminados
Núcleo do objeto	Maior que B	É recuperado quase intacto pela dilatação final

4.9.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros (0 ou 1) da matriz f .

Saída:

- Matriz resultante em L linhas e C colunas, valores 0 ou 1.

4.9.5.5 Exemplos

Entrada	Saída	Observação
7	0 0 0 0 0 0	Pontos isolados e a protuberância fina desaparecem; o quadrado central sobrevive
7	0 0 0 0 0 0	
3	0 0 1 1 1 0 0	
3	0 0 1 1 1 0 0	
1 1 1	0 0 1 1 1 0 0	
1 1 1	0 0 0 0 0 0 0	
1 1 1	0 0 0 0 0 0 0	
0 0 0 0 0 0 0		
0 1 0 0 0 1 0		
0 0 1 1 1 0 0		
0 0 1 1 1 0 0		
0 0 1 1 1 1 0		
0 0 0 0 0 0 0		
0 1 0 0 0 0 1		

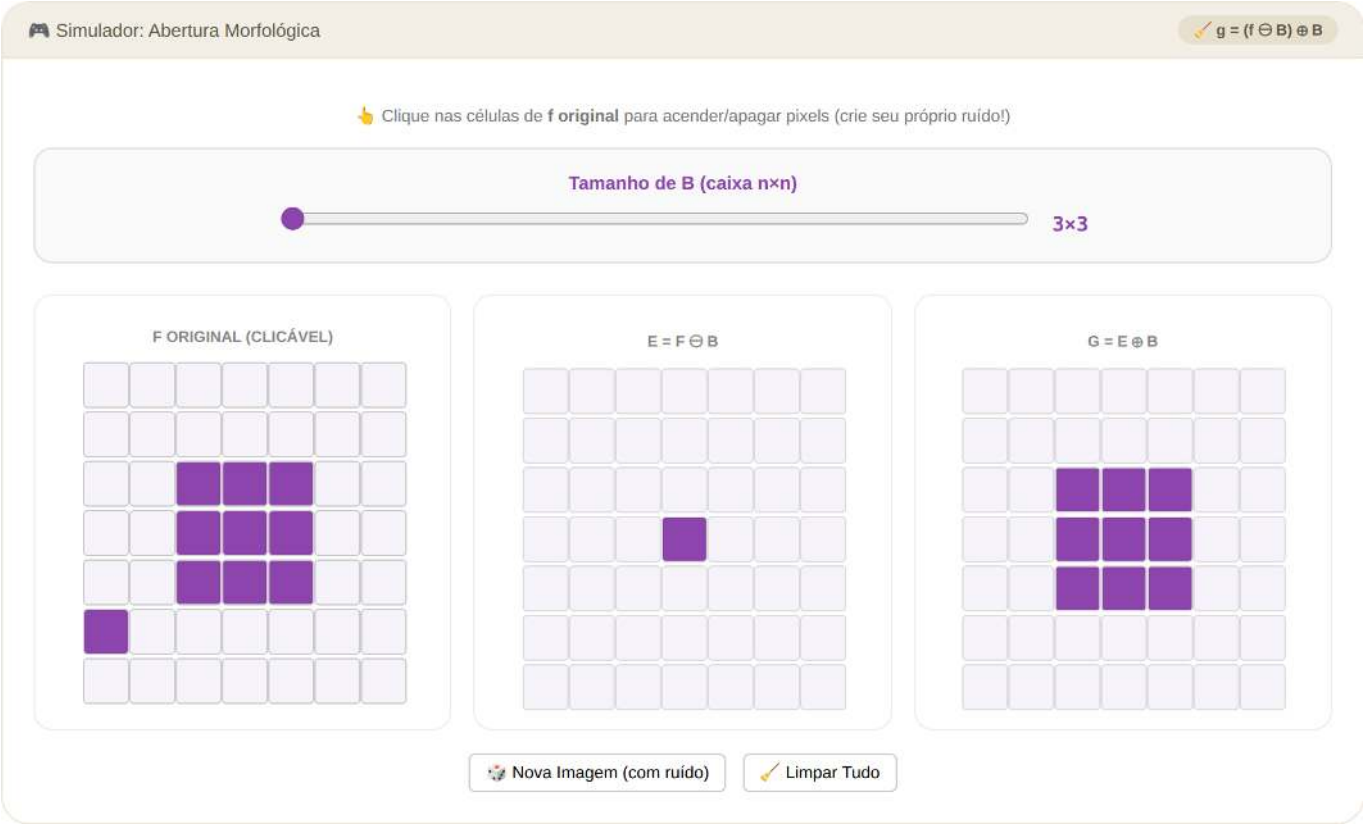


Figura 4.34: Simulador: Abertura Morfológica (erosão + dilatação)

```
%%writefile EP04_05.py
# Código Python

Writing EP04_05.py

TestSuite("EP04_05.py").run()
```

4.9.6 EP04_06 Fechamento Morfológico (Preenchimento de Falhas)

Em **digitalização de impressões digitais**, sulcos da pele às vezes ficam interrompidos por sujeira ou ressecamento, criando pequenas falhas na curva contínua que deveria existir. O **fechamento** — dilatação seguida de erosão com o mesmo elemento estruturante — é o operador dual da abertura: ele **preenche buracos pequenos e reentrâncias estreitas**, sem alterar significativamente o contorno externo do objeto. É a etapa padrão antes de extrair o esqueleto de uma impressão digital. Ver na Figure 4.35 uma simulação deste EP.

4.9.6.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Dilatação:** Calcular $d = f \oplus B$, usando exatamente o algoritmo do EP04_03 (refletindo B , sem padding).
6. **Erosão:** Calcular $g = d \ominus B$, usando exatamente o algoritmo do EP04_04 (sem refletir B , sem padding) — agora aplicado sobre d , não sobre f .
7. **Saída:** Exibir a matriz resultante g (o **fechamento** de f por B) com dimensões $L \times C$.

4.9.6.2 Restrições Computacionais

- **Ordem fixa:** É sempre dilatação primeiro, depois erosão — a ordem inversa é a abertura do EP04_05.
- **Mesmo B :** O elemento estruturante usado na dilatação e na erosão deve ser idêntico.
- **Sem padding em nenhuma das duas etapas.**

4.9.6.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Extensividade	$g \supseteq f$ sempre	O fechamento nunca remove pixel, só adiciona
Idempotência	$\text{fecha}(\text{fecha}(f)) = \text{fecha}(f)$	Aplicar de novo não muda mais nada
Buracos pequenos	Menores que $\overline{B}$	São completamente preenchidos
Dualidade	$\text{fecha}(f) = \overline{\text{abre}(\overline{f})}$	É a abertura aplicada ao “negativo” da imagem

4.9.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros (0 ou 1) da matriz f .

Saída:

- Matriz resultante em L linhas e C colunas, valores 0 ou 1.

4.9.6.5 Exemplos

Entrada	Saída	Observação
8	0 0 1 1 1 1 0 0	Os dois buracos internos não-adjacentes são totalmente preenchidos
8	0 0 1 1 1 1 0 0	
3	0 0 1 1 1 1 0 0	
3	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
1 1 1	0 0 1 1 1 1 0 0	
0 0 0 0 0 0 0 0	0 0 1 1 1 1 0 0	
0 0 1 1 1 1 0 0		
0 0 1 1 1 1 0 0		
0 0 1 0 1 1 0 0		
0 0 1 1 0 1 0 0		
0 0 1 1 1 1 0 0		
0 0 1 1 1 1 0 0		
0 0 0 0 0 0 0 0		

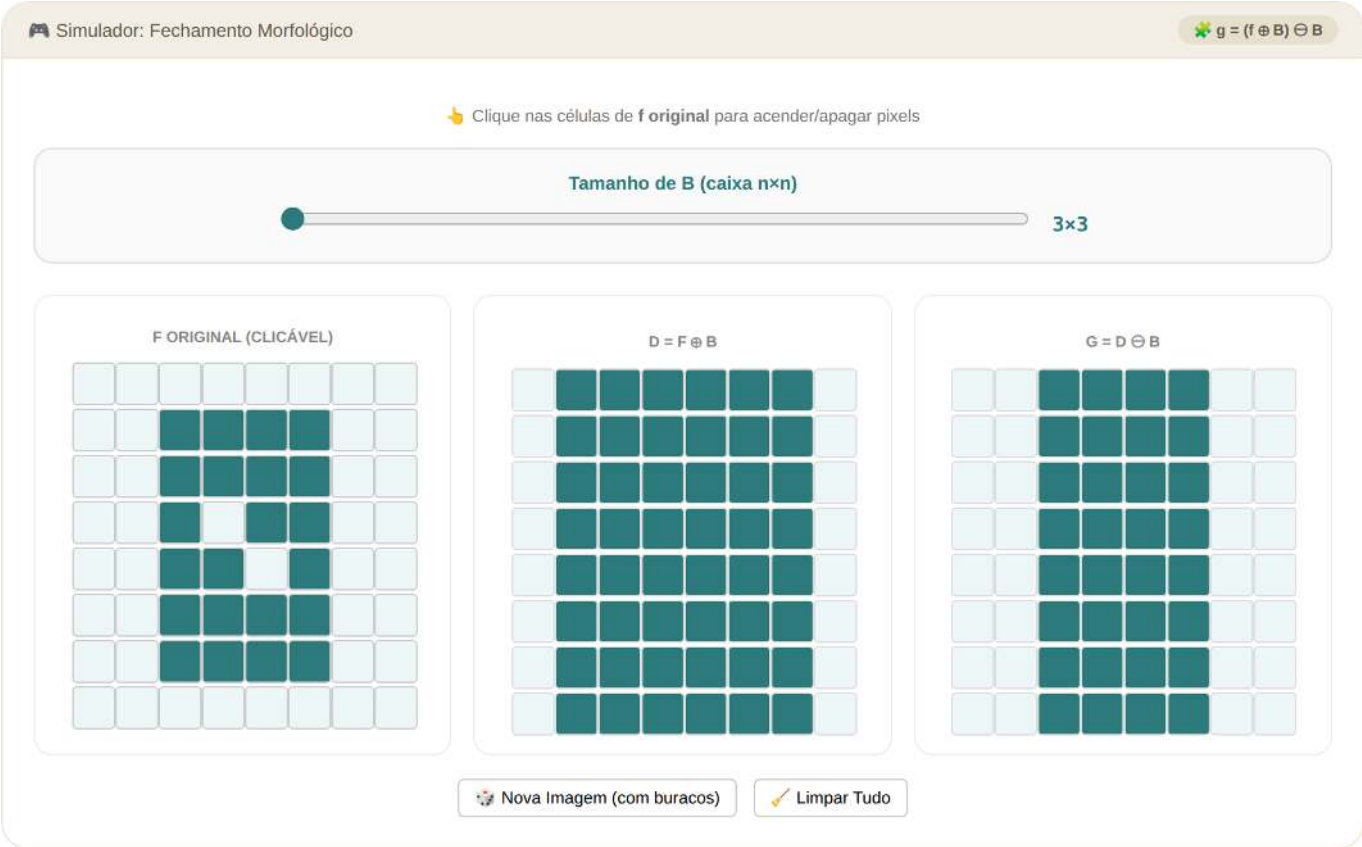


Figura 4.35: Simulador: Fechamento Morfológico (dilatação + erosão)

```
%%writefile EP04_06.py
# Código Python

Writing EP04_06.py

TestSuite("EP04_06.py").run()
```

4.9.7 EP04_07 Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)

Até agora, o elemento estruturante só dizia “este vizinho conta” ou “não conta” — mas em **modelos digitais de elevação** (usados em SIG e em planejamento de drenagem urbana), cada vizinho deveria ter um **peso diferente** dependendo da distância ou da direção do relevo. As versões **ponderadas** da dilatação e da erosão, implementadas em `morph.py` como `mm.dil1(f, b)` e `mm.ero1(f, b)`, somam (ou subtraem) o peso de cada vizinho antes de tomar o máximo (ou mínimo) — generalizando tudo o que foi feito nos EPs anteriores. Ver na Figure 4.36 uma simulação deste EP.

4.9.7.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de b :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante ponderado.
3. **Pesos:** Ler a matriz b de pesos **inteiros** (podem ser negativos, zero ou positivos), linha a linha.
4. **Dados:** Ler a matriz f (a imagem original), linha a linha.
5. **Vizinhança sem padding:** Para cada pixel (y, x) , percorrer **todas** as posições (by, bx) de b (não apenas onde valeria 1 — aqui **todo** peso participa), usando o mesmo deslocamento dos EPs anteriores:

$$v_y = y + by + o_y, \quad v_x = x + bx + o_x, \quad o_y = -\frac{L_B}{2} + 0,5, \quad o_x = -\frac{C_B}{2} + 0,5$$

Descartar todo (v_y, v_x) fora de $[0, L) \times [0, C)$.

6. **Dilatação ponderada:** Calcular

$$g_{dil}(y, x) = \max \left(f(y, x), \max_{(v_y, v_x) \text{ válido}} (f(v_y, v_x) + b(by, bx)) \right)$$

7. **Erosão ponderada:** Calcular, **usando o mesmo b e sem refletir:**

$$g_{ero}(y, x) = \min \left(f(y, x), \min_{(v_y, v_x) \text{ válido}} (f(v_y, v_x) - b(by, bx)) \right)$$

8. **Saída:** Exibir **primeiro** a matriz g_{dil} completa, e **depois** a matriz g_{ero} completa.

4.9.7.2 Restrições Computacionais

- **Nenhuma das duas reflete b** — a versão ponderada não usa reflexão, mesmo na dilatação (diferente de `mm.dil0`).
- **Todos os pesos participam:** Não existe aqui o filtro “ $B = 1$ ”; mesmo peso 0 entra na conta.
- **Sem padding:** vizinhos fora da imagem são ignorados, nunca virtualmente preenchidos.
- **Tipo:** A saída pode conter valores negativos ou maiores que 255 — **não** há *clipping* neste EP.
- **Dica:** Para remover mensagens de overflow ao ultrapassar limites do tipo `uint8`, incluir no início do código:

```
import warnings
warnings.filterwarnings("ignore")
```

4.9.7.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
Peso positivo	“Puxa” o valor do vizinho para cima na dilatação	Simula relevo que sobe naquela direção
Peso negativo	Reduz a contribuição do vizinho	Simula distância ou atenuação direcional
Dualidade ponderada	$ero1(f, b) = -dil1(-f, b)$	A simetria entre as duas operações se mantém mesmo com pesos

4.9.7.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (podem ser negativos) da matriz b .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Primeiro a matriz g_{dil} em L linhas e C colunas.
- Em seguida a matriz g_{ero} em L linhas e C colunas.

4.9.7.5 Exemplos

Entrada	Saída	Observação
3	50 60 61	Peso central 2 acelera o crescimento na dilatação e o encolhimento na erosão
3	80 90 91	
3	81 91 92	
3	8 9 19	
0 1 0	9 10 20	
1 2 1	39 40 50	
0 1 0		
10 20 30		
40 50 60		
70 80 90		

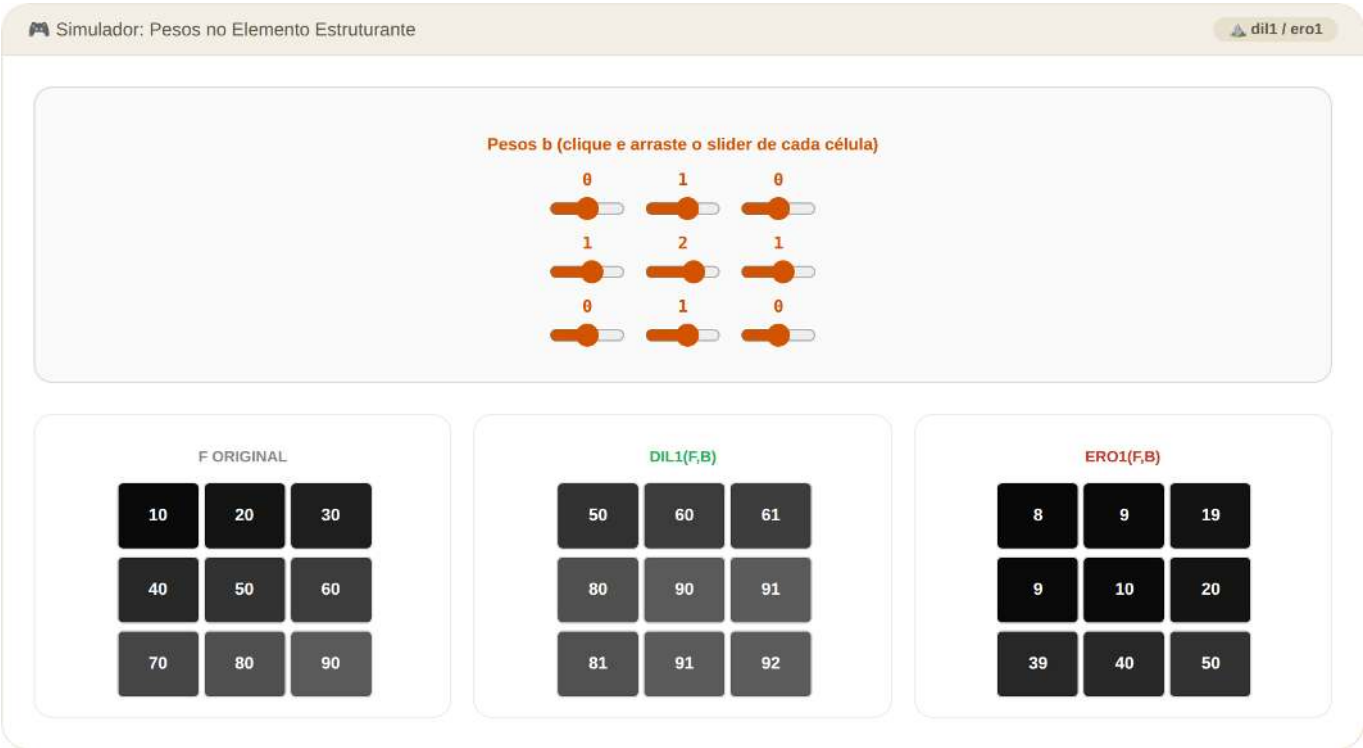


Figura 4.36: Simulador: Dilatação e Erosão com Pesos (mm.dil1 / mm.ero1)


```
import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite
```

```
%%writefile EP04_07.py
# Código Python
```

```
Writing EP04_07.py
```

```
TestSuite("EP04_07.py").run()
```

4.9.8 EP04_08 Gradiente Morfológico, Top-hat e Black-hat

Em **inspeção automática de placas de circuito**, três perguntas aparecem o tempo todo: onde estão as **bordas** dos componentes? Quais **detalhes claros e pequenos** (como pontos de solda) se destacam do fundo? Quais **reentrâncias escuras** (como fissuras) o fundo esconde? Um único par erosão/dilatação responde às três: o **gradiente morfológico** evidencia contornos, o **top-hat** revela picos estreitos, e o **black-hat** revela vales estreitos — três ferramentas, uma só vizinhança. Ver na Figure 4.37 uma simulação deste EP.

4.9.8.1 Diretrizes de Implementação

1. **Dimensões da imagem:** Ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** Ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** Ler a matriz B com valores 0 ou 1, linha a linha.
4. **Dados:** Ler a matriz f (a imagem original, em tons de cinza), linha a linha.
5. **Operadores de base:** Calcular, exatamente como nos EPs 04_03 a 04_06:
 - $d = f \oplus B$ (dilatação),
 - $e = f \ominus B$ (erosão),
 - abertura = $e \oplus B$,
 - fechamento = $d \ominus B$.
6. **Gradiente morfológico:** $\text{grad}(y, x) = d(y, x) - e(y, x)$.
7. **Top-hat:** $\text{tophat}(y, x) = f(y, x) - \text{abertura}(y, x)$.
8. **Black-hat:** $\text{blackhat}(y, x) = \text{fechamento}(y, x) - f(y, x)$.
9. **Saída:** Exibir, **nesta ordem**, as três matrizes completas: gradiente, top-hat, black-hat.

4.9.8.2 Restrições Computacionais

- **Sem padding em nenhuma etapa intermediária** — dilatação, erosão, abertura e fechamento seguem as mesmas regras de vizinhança dos EPs anteriores.
- **Não há clipping:** as três saídas podem conter qualquer valor inteiro (o gradiente é sempre ≥ 0 , mas top-hat e black-hat também).
- **Reaproveitamento:** d e e devem ser calculados **uma única vez** e reaproveitados para montar abertura, fechamento e gradiente.

4.9.8.3 Fundamentação Teórica

Operador	Fórmula	O que revela
Gradiente	$d - e$	Bordas: zero em regiões planas, alto nas transições
Top-hat	$f - \text{abertura}(f)$	Elementos claros e finos , menores que B
Black-hat	$\text{fechamento}(f) - f$	Elementos escuros e finos , menores que B

4.9.8.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro L_B .
- Linha 4: Inteiro C_B .
- Próximas L_B linhas: elementos inteiros (0 ou 1) da matriz B .
- Próximas L linhas: elementos inteiros da matriz f .

Saída:

- Matriz gradiente em L linhas e C colunas.
- Matriz top-hat em L linhas e C colunas.
- Matriz black-hat em L linhas e C colunas.

4.9.8.5 Exemplos

Entrada	Saída	Observação
9	(gradiente: halo	Pico isolado vira top-hat; vale isolado vira black-hat; ambos aparecem no gradiente
9	$3 \times 3 = 70$ em torno de	
3	(2, 2) e halo $3 \times 3 = 8$ em	
3	torno de (6, 6), resto 0)	
1 1 1	(top-hat: único 70 em	
1 1 1	(2, 2), resto 0)	
1 1 1	(black-hat: único 8 em	
10 10 10 10 10 10 10 10 10	(6, 6), resto 0)	
10 10 10 10 10 10 10 10 10		
10 10 80 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 2 10 10		
10 10 10 10 10 10 10 10 10		
10 10 10 10 10 10 10 10 10		

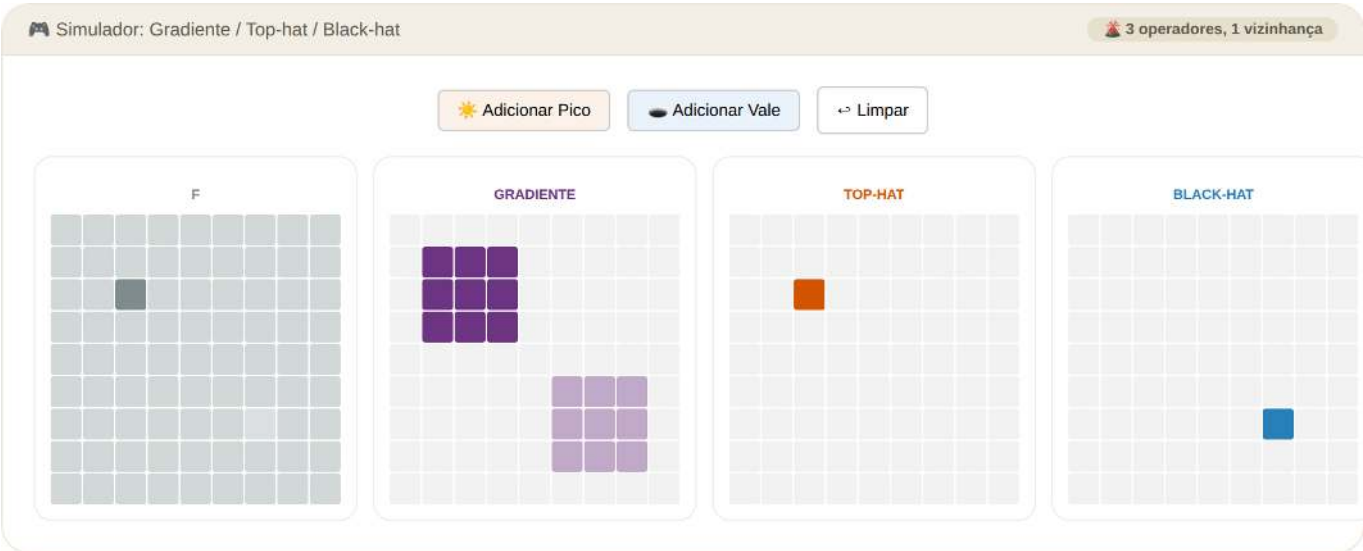


Figura 4.37: Simulador: Gradiente Morfológico, Top-hat e Black-hat

```
%%writefile EP04_08.py
# Código Python
```

```
Writing EP04_08.py
```

```
TestSuite("EP04_08.py").run()
```

4.9.9 EP04_09 Transformada de Distância e o “Miolo” do Objeto

Em **robótica móvel**, ao planejar uma rota dentro de um corredor, o robô quer saber não apenas *onde* existe espaço livre, mas também **quão longe** cada ponto livre está da parede mais próxima. Os caminhos mais seguros tendem a passar pelo “miolo” do corredor, longe dos obstáculos.

A **transformada de distância morfológica** atribui a cada pixel um valor que representa sua distância até a borda mais próxima, segundo a métrica definida pelo elemento estruturante. Pixels próximos à borda recebem valores baixos, enquanto pixels mais internos recebem valores maiores. O pixel de valor máximo corresponde à região mais protegida do objeto, frequentemente associada ao seu centro morfológico.

Ver na Figure 4.38 uma simulação deste EP.

4.9.9.1 Diretrizes de Implementação

1. **Dimensões da imagem:** ler os inteiros L (linhas) e C (colunas) da imagem f .
2. **Dimensões de B :** ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** ler a matriz b , contendo valor 0 no centro e valores negativos nas demais posições.
4. **Imagem:** ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Preparação:** multiplicar a imagem por $L \times C$, garantindo que os pixels internos tenham valor inicial suficientemente alto para a propagação das distâncias.
6. **Transformada de distância:** calcular a matriz de distâncias utilizando o método `mm.dist1(f,b)`.
7. **Saída:** exibir a matriz resultante da transformada de distância.

4.9.9.2 Restrições Computacionais

- Utilizar a implementação de erosão ponderada fornecida pela biblioteca.
- O elemento estruturante pode conter valores negativos arbitrários.
- A transformada deve ser obtida pela aplicação iterativa de erosões ponderadas até atingir um ponto fixo.

 **Nota Crucial sobre Leitura de Matrizes:** Como o elemento estruturante pode conter valores inteiros negativos (por exemplo, -1 e -99), **não utilize a função `mm.readImg` para ler a matriz b** . Essa função converte os dados para o tipo `uint8`, provocando *underflow* e corrompendo os valores negativos. Leia as L_B linhas de b manualmente utilizando o tipo padrão `int`. A imagem f pode continuar sendo lida normalmente por `mm.readImg`.

4.9.9.3 Fundamentação Teórica

Conceito	Significado	Impacto Visual
$\text{dist}(y, x)$	Distância morfológica até a borda mais próxima segundo a métrica definida por b	Pixels mais internos recebem valores maiores
Valor máximo	Pixel mais distante da borda	Aproxima o centro morfológico do objeto
Elemento estruturante ponderado	Define os custos de deslocamento entre pixels vizinhos	Determina a métrica de distância utilizada

Conceito	Significado	Impacto Visual
Objetos finos	Regiões estreitas do objeto	Produzem valores baixos de distância

4.9.9.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L .
- Linha 2: inteiro C .
- Linha 3: inteiro L_B .
- Linha 4: inteiro C_B .
- Próximas L_B linhas: elementos inteiros da matriz b .
- Próximas L linhas: elementos binários (0 ou 1) da matriz f .

 **Nota de implementação:** Os elementos da matriz f (0 ou 1) devem ser multiplicados por **255** para gerar uma imagem binária adequada (0 e 255) antes de aplicar a Transformada de Distância (TD).

Saída:

- Matriz da transformada de distância em L linhas e C colunas.

4.9.9.5 Exemplo

Entrada	Saída	Observação
5	0 0 0 0 0 0 0 0	Resultado da transformada de distância.
9	0 1 1 1 1 1 1 0	
3	0 1 2 2 2 2 2 0	
3	0 1 1 1 1 1 1 0	
-99 -1 -99	0 0 0 0 0 0 0 0	
-1 0 -1		
-99 -1 -99		
0 0 0 0 0 0 0 0		
0 1 1 1 1 1 1 0		
0 1 1 1 1 1 1 0		
0 1 1 1 1 1 1 0		
0 0 0 0 0 0 0 0		

Nota: o valor -99 atua como uma aproximação prática de $-\infty$, impedindo a propagação pelas diagonais. Dessa forma, apenas os vizinhos horizontal e vertical contribuem para a distância, produzindo a distância de Manhattan.

```
%%writefile EP04_09.py
# Código Python
```

```
Writing EP04_09.py
```

```
TestSuite("EP04_09.py").run()
```

4.9.10 EP04_10 Separação de *Blobs*, Rotulação e Descritores

Em uma **linha de produção de moedas**, é comum que peças encostem umas nas outras na esteira, formando uma única mancha conectada na imagem — uma contagem ingênua erraria o total. A solução

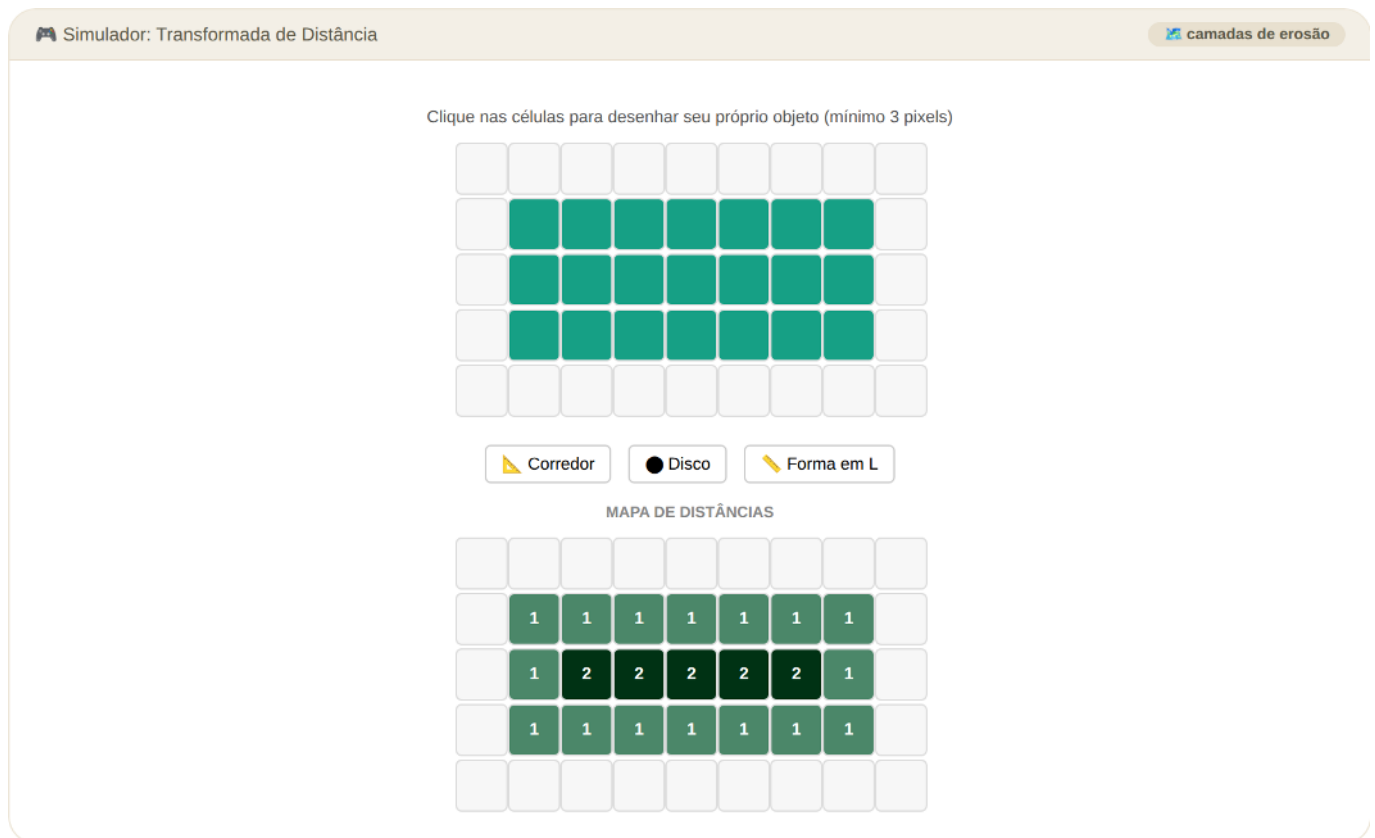


Figura 4.38: Simulador: Transformada de Distância

clássica combina operações morfológicas e análise de conectividade: primeiro uma **erosão** reduz ou rompe conexões frágeis entre objetos, e depois a **rotulação de componentes conectados** separa cada objeto em uma região distinta. Por fim, **descritores geométricos** (área e caixa delimitadora) resumem cada componente encontrado.

Ver na Figure 4.39 uma simulação deste EP.

4.9.10.1 Diretrizes de Implementação

1. **Dimensões da imagem:** ler os inteiros L (linhas) e C (colunas) de f .
2. **Dimensões de B :** ler os inteiros L_B (linhas) e C_B (colunas) do elemento estruturante.
3. **Elemento estruturante:** ler a matriz B , contendo valores 0 ou 1, linha a linha.
4. **Dados:** ler a matriz binária f (valores 0 ou 1), linha a linha.
5. **Separação:** calcular

$$f_{ero} = f \ominus B$$

usando erosão binária plana (como no EP04_04), eliminando conexões frágeis entre objetos.

6. **Rotulação:** sobre f_{ero} , identificar componentes conectados usando conectividade definida pela vizinhança B . A rotulação deve seguir varredura *raster*: ao encontrar um pixel 1 ainda não rotulado, atribuir um novo rótulo inteiro crescente a partir de 1 e propagar esse rótulo a toda a região conectada.
7. **Descritores:** para cada rótulo k , calcular:
 - **Área:** número de pixels pertencentes ao rótulo;
 - **Caixa delimitadora:**

$$(y_{min}, x_{min}, y_{max}, x_{max})$$

8. **Saída:** exibir o número total de rótulos e, em seguida, uma linha por rótulo no formato:

$$k, \text{ área}, y_{\min}, x_{\min}, y_{\max}, x_{\max}$$

4.9.10.2 Restrições Computacionais

- A erosão deve ser aplicada antes da rotulação.
- A conectividade é fixa e definida pela vizinhança acima.
- O elemento estruturante B não interfere na conectividade da rotulação.
- Sem padding em qualquer etapa.
- A ordem dos rótulos segue a primeira descoberta em varredura *raster*.

4.9.10.3 Fundamentação Teórica

Conceito	Significado	Impacto
Ponte fina	Conexão estreita entre objetos	Pode ser removida pela erosão morfológica
Conectividade	Definida pelo conjunto $\mathcal{N}(y, x)$	Determina quais pixels pertencem ao mesmo componente
Área	Número de pixels por componente	Estimativa direta do tamanho do objeto
Caixa delimitadora	Extensão espacial do rótulo	Resumo geométrico do componente

4.9.10.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L
- Linha 2: inteiro C
- Linha 3: inteiro L_B
- Linha 4: inteiro C_B
- Próximas L_B linhas: matriz B
- Próximas L linhas: matriz f

Saída:

- Linha 1: número total de rótulos encontrados
- Linhas seguintes:

$$k, \text{ área}, y_{\min}, x_{\min}, y_{\max}, x_{\max}$$

```
%%writefile EP04_10.py
# Código Python
```

```
Writing EP04_10.py
```

```
TestSuite("EP04_10.py").run()
```



Figura 4.39: Simulador: Separação de Blobs, Rotulação e Descritores

Parte II

Parte II — Visão Computacional

Capítulo 5

Análise de Documentos e Inspeção Industrial



Na **Parte I — Processamento Digital de Imagens (PDI)**, foram estudadas técnicas para transformação e aprimoramento de imagens, como operações morfológicas, filtragem espacial, convoluções, limiarização, segmentação e processamento no domínio da frequência.

A **Parte II — Visão Computacional (VC)** amplia esse escopo ao tratar da interpretação automática do conteúdo visual, envolvendo a extração de informações, o reconhecimento de padrões e a tomada de decisões a partir de imagens.

Este capítulo apresenta essa transição por meio de duas aplicações representativas:

1. **Análise Automatizada de Documentos**, aplicada ao processamento de formulários, avaliações e outros documentos estruturados por meio de sistemas de reconhecimento óptico de marcas (*Optical Mark Recognition* – OMR);
2. **Inspeção Industrial Automatizada**, voltada ao controle de qualidade e à detecção de defeitos em linhas de produção.

Essas aplicações integram técnicas de detecção de estruturas geométricas, extração de descritores invariantes, reconhecimento de padrões e classificação de objetos, constituindo a base de diversos sistemas modernos de inspeção visual e automação.

5.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- **Avaliar a influência do pré-processamento** na qualidade do reconhecimento automático de informações em documentos;
- **Realizar reconhecimento óptico de caracteres (OCR)** para converter documentos digitalizados em texto codificado;
- **Aplicar técnicas de processamento de linguagem natural**, incluindo tradução automática, ao texto obtido por OCR;
- Aplicar técnicas de **alinhamento e retificação geométrica de documentos** utilizando a Transformada de Hough e transformações projetivas;
- **Implementar sistemas de reconhecimento óptico de marcas (OMR)** para leitura automatizada de avaliações e formulários;
- **Detectar e segmentar regiões de interesse** com base em operações morfológicas, contornos e propriedades geométricas;
- **Decodificar marcadores bidimensionais e códigos de barras**, integrando bibliotecas de Visão Computacional a *pipelines* de processamento documental;
- **Desenvolver *pipelines* de Visão Computacional** para análise automatizada de documentos.

Este capítulo marca a transição do **Processamento Digital de Imagens**, voltado à transformação de imagens, para a **Visão Computacional**, cujo objetivo é interpretar o conteúdo visual, extrair informações e apoiar processos automatizados de análise e tomada de decisão.

5.2 Configuração do Ambiente

Os exemplos deste capítulo utilizam bibliotecas amplamente empregadas em PDI-VC. O bloco abaixo instala os pacotes necessários; em ambientes que já os possuam, a execução pode ser ignorada.

```
import sys, subprocess, importlib, shutil

# 1. Dependências de Sistema
if 'google.colab' in sys.modules:
    print("[AMBIENTE] Google Colab. Configurando dependências do sistema...")
    subprocess.run(
        "apt-get update && apt-get install -y poppler-utils "
        "libzbar0 tesseract-ocr tesseract-ocr-por",
        shell=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
    )
elif shutil.which("tesseract") is None:
    # Ambiente local sem tesseract: tenta instalar via apt-get (requer sudo/root)
    if shutil.which("apt-get"):
        print("[AMBIENTE] Local. Instalando tesseract-ocr via apt-get (pode pedir senha)...")
        resultado = subprocess.run(
            "sudo apt-get update && sudo apt-get install -y tesseract-ocr tesseract-ocr-por",
            shell=True
        )
        if resultado.returncode != 0 or shutil.which("tesseract") is None:
            print(
                "[AVISO] Não foi possível instalar automaticamente. "
                "Instale manualmente: sudo apt install tesseract-ocr tesseract-ocr-por"
            )
    else:
        print(
            "[AVISO] tesseract não encontrado e apt-get indisponível. "
            "Instale manualmente antes de executar as células de OCR."
        )

# 2. Dependências do Python (Instala apenas as ausentes)
pkgs = {
    "cv2": "opencv-python", "skimage": "scikit-image", "numpy": "numpy",
    "pdf2image": "pdf2image", "pandas": "pandas", "tabulate": "tabulate",
    "PyPDF2": "PyPDF2", "bcrypt": "bcrypt", "pyarrow": "pyarrow",
    "pyzbar": "pyzbar", "pytesseract": "pytesseract", "deep_translator": "deep-translator"
}
for mod, pkg in pkgs.items():
    if importlib.util.find_spec(mod) is None:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg])

# 3. Imports Globais do Pipeline
import cv2, numpy as np, matplotlib.pyplot as plt
from skimage import io, data, color
```

Além dessas bibliotecas, será utilizado o módulo didático `morph.py`, desenvolvido para simplificar operações de leitura, visualização e processamento de imagens ao longo deste livro. O código a seguir verifica sua disponibilidade, realiza o *download* quando necessário e confirma a versão carregada.

```
import os
import urllib.request

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

print(f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')}")
```

Ambiente pronto. morph 1.1.5

5.3 Bases de Imagens para Experimentação

Os exemplos apresentados nesta parte do livro utilizam, sempre que possível, documentos digitalizados, folhas de respostas, códigos de barras, *QR Codes* e outras imagens provenientes de aplicações reais. Para tornar os experimentos integralmente reproduzíveis — inclusive em ambientes sem acesso à internet ou aos arquivos originais do MCTest —, também são empregadas imagens públicas amplamente utilizadas no ensino e na pesquisa em Processamento Digital de Imagens e Visão Computacional (PDI-VC).

💡 Por que utilizar um banco de imagens de *benchmark*?

Imagens como `camera()` e `coins()` são empregadas há décadas em livros-texto, artigos científicos e materiais didáticos de PDI-VC. Sua utilização oferece importantes vantagens:

- **Reprodutibilidade:** qualquer leitor obtém exatamente as mesmas imagens, independentemente do computador ou sistema operacional utilizado, sem necessidade de *downloads* externos ou de arquivos específicos deste livro;
- **Comparabilidade:** os resultados produzidos podem ser comparados diretamente com aqueles reportados na literatura, uma vez que os mesmos conjuntos de imagens são amplamente adotados como referência;
- **Foco nos algoritmos:** por serem imagens compactas, bem documentadas e distribuídas sem restrições de uso para fins educacionais e científicos, permitem concentrar a atenção nas técnicas de processamento, reduzindo interferências relacionadas à aquisição ou ao gerenciamento dos dados.

5.3.1 Imagens Públicas com `skimage.data`

O módulo `skimage.data` disponibiliza uma coleção de imagens de referência amplamente utilizada em atividades de ensino, pesquisa e validação de algoritmos em PDI-VC. A inspeção do atributo `data.__all__` mostra que a versão atual reúne **42 itens**, incluindo fotografias naturais, documentos digitalizados, imagens médicas, microscopia, texturas, padrões sintéticos, modelos tridimensionais e sequências temporais. Convém observar que alguns desses itens correspondem a funções utilitárias, como `data_dir()` e `download_all()`, e não a imagens propriamente ditas.

Como o objetivo deste capítulo é apresentar aplicações de análise documental e inspeção visual, a Table 5.2 reúne uma **seleção representativa** das imagens mais relevantes, organizada de acordo com suas principais aplicações em Visão Computacional. A Figure 5.1 apresenta uma amostra dessas imagens, agrupadas conforme a mesma classificação adotada na tabela.

Imagens utilizadas neste capítulo

Embora o `skimage.data` disponibilize dezenas de imagens de referência, apenas quatro são empregadas diretamente nos experimentos deste capítulo. Elas foram selecionadas por reproduzirem, de forma controlada, características frequentemente encontradas em documentos digitalizados e em sistemas de inspeção visual industrial. A Tabela 5.1 resume o papel de cada uma delas ao longo deste capítulo.

Tabela 5.1: Imagens do `skimage.data` utilizadas nos experimentos deste capítulo.

Imagem	Aplicação no capítulo
<code>data.page()</code>	Página digitalizada utilizada nos experimentos de correção de iluminação, realce local (CLAHE) e limiarização automática por Otsu.
<code>data.text()</code>	Documento contendo texto impresso, empregado para ilustrar segmentação, extração de contornos e etapas típicas de OCR e OMR.
<code>data.coffee()</code>	Fotografia colorida com variações naturais de iluminação, utilizada para exemplificar técnicas aplicáveis a cenas reais não documentais.
<code>data.brick()</code>	Textura de referência empregada em exemplos de inspeção superficial e detecção de defeitos por análise de variância local.

Tabela 5.2: Seleção de imagens públicas representativas disponíveis no módulo `skimage.data`, organizadas por área de aplicação.

Categoria	Função	Descrição
 Documentos & OCR/OMR	<code>data.page()</code>	Página digitalizada de documento — normalização de fundo, CLAHE e Otsu.
 Documentos & OCR/OMR	<code>data.text()</code>	Texto impresso — segmentação e extração de contornos em cenários de OCR/OMR.
 Segmentação, Morfologia & Contornos	<code>data.coins()</code>	Conjunto de moedas — referência clássica para segmentação e <i>watershed</i> .
 Segmentação, Morfologia & Contornos	<code>data.clock()</code>	Relógio analógico — detecção de formas e contornos.
 Segmentação, Morfologia & Contornos	<code>data.binary_blobs()</code>	Blobs binários sintéticos — conectividade e morfologia matemática.
 Segmentação, Morfologia & Contornos	<code>data.moon()</code>	Superfície lunar — segmentação de crateras por relevo de intensidade.
Textura & Inspeção Industrial	<code>data.brick()</code>	Textura uniforme de tijolos — detecção de defeitos por variância local.
Textura & Inspeção Industrial	<code>data.checkerboard()</code>	Padrão xadrez — calibração de câmera e transformações geométricas.
 Fotografias Clássicas de PDI/VC	<code>data.camera()</code>	Fotógrafo com tripé — imagem de referência mais citada na literatura de PDI.
 Fotografias Clássicas de PDI/VC	<code>data.astronaut()</code>	Retrato colorido de astronauta — filtragem e realce em cor.
 Fotografias Clássicas de PDI/VC	<code>data.coffee()</code>	Xícara de café — cena real com variação de iluminação e cor.
 Fotografias Clássicas de PDI/VC	<code>data.cat()</code> / <code>data.chelsea()</code>	Fotografias coloridas de gatos — detecção de bordas e realce.

Categoria	Função	Descrição
 Fotografias Clássicas de PDI/VC	<code>data.horse()</code>	Silhueta binária de cavalo — descritores de forma e contorno.

```
import matplotlib.pyplot as plt
from skimage import data

# Imagens ordenadas por categoria; a cor do título reproduz a cor da categoria na tab. anterior
imgs = [
    ("page",      data.page(),      "#2563eb"),    # Documentos & OCR/OMR
    ("text",      data.text(),      "#2563eb"),
    ("coins",     data.coins(),     "#16a34a"),    # Segmentação & morfologia
    ("binary_blobs", data.binary_blobs(), "#16a34a"),
    ("brick",     data.brick(),     "#ea580c"),    # Textura & inspeção industrial
    ("checkerboard", data.checkerboard(), "#ea580c"),
    ("camera",    data.camera(),    "#7c3aed"),    # Fotografias clássicas de PDI/VC
    ("coffee",    data.coffee(),    "#7c3aed"),
]

fig, ax = plt.subplots(2, 4, figsize=(11, 5.5))
for a, (nome, img, cor) in zip(ax.ravel(), imgs):
    a.imshow(img, cmap="gray")
    a.set_title(nome, color=cor, fontweight="bold")
    a.axis("off")
plt.tight_layout()
```

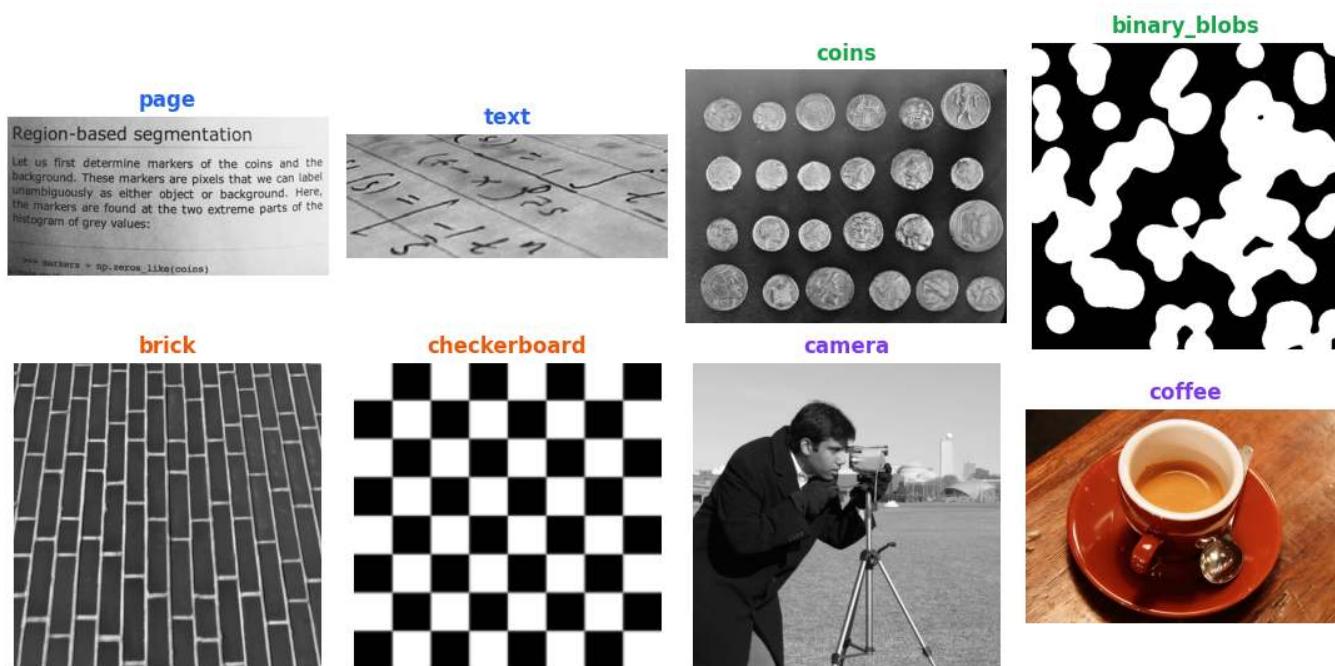


Figura 5.1: Amostra de imagens públicas do *skimage.data*, agrupadas por área de aplicação.

5.4 Normalização de Fundo e Equalização Local de Contraste

A qualidade da segmentação depende diretamente das características da imagem de entrada. Em documentos digitalizados, variações de iluminação, sombras, regiões superexpostas e diferenças de tonalidade do papel reduzem o contraste entre o primeiro plano e o fundo, dificultando a aplicação de métodos de limiarização global, como o algoritmo de Otsu.

Para minimizar esses efeitos, são empregadas duas técnicas complementares de pré-processamento:

- **Normalização de fundo:** estima a componente de baixa frequência da imagem por meio de uma forte suavização e, em seguida, normaliza a imagem original em relação a esse fundo estimado. Esse procedimento reduz gradientes de iluminação e compensa variações lentas de intensidade, preservando as estruturas de interesse.
- **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*), apresentado no **Capítulo 4**: divide a imagem em pequenas regiões (*tiles*) e realiza a equalização do histograma de cada região de forma independente. O contraste é limitado para evitar a amplificação excessiva do ruído, tornando a técnica especialmente adequada para imagens com variações locais de iluminação.

A [Figure 5.2](#) compara essas estratégias utilizando a imagem `page()` da biblioteca `skimage.data`. São apresentados seis resultados: (a) a imagem original; (b) a binarização direta pelo método de Otsu, utilizada como referência; (c) o fundo estimado por filtragem Gaussiana; (d) a imagem após a normalização de fundo; (e) a binarização obtida após a aplicação do CLAHE seguida do método de Otsu; e (f) a binarização obtida após a normalização de fundo seguida da aplicação do método de Otsu.

A comparação permite observar o efeito produzido por cada etapa do pré-processamento e sua influência na qualidade da segmentação. Em particular, a normalização de fundo reduz as variações globais de iluminação, enquanto o CLAHE aumenta o contraste local entre caracteres e fundo. Dependendo das características da imagem, uma ou outra estratégia pode produzir resultados superiores, não existindo uma técnica universalmente mais adequada.

```
import cv2
from skimage import data
from morph import mm

img = data.page() # ou: img = mm.gray(img_final) - com imagem da folha de prova

# Método 1: Normalização de fundo + Otsu
# Estima o fundo com um filtro Gaussiano de sigma grande (variações lentas de luz)
# e divide pixel a pixel para cancelar o gradiente de iluminação
bg = cv2.GaussianBlur(img, (0, 0), sigmaX=25)
img_norm = cv2.divide(img, bg, scale=255)
img_norm_otsu = mm.threshold(img_norm)

# Método 2: CLAHE + Otsu
# tileGridSize define o tamanho de cada região local (tile)
# clipLimit controla o teto de amplificação - valores altos aumentam contraste
# mas também amplificam ruído
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img)
img_clahe_otsu = mm.threshold(img_clahe)

# Método 0: Otsu direto (sem pré-processamento) - referência
img_otsu = mm.threshold(img)

mm.show(
    [img, img_otsu, bg, img_norm, img_clahe_otsu, img_norm_otsu],
    titles=["(a) Original", "(b) Otsu direto", "(c) Gaussiano", \
           "(d) Fundo normalizado (img/bg)", "(e) CLAHE + Otsu", \
           "(f) Normaliz. fundo + Otsu"],
    cols=3,
    figsize=(14, 8)
)
```

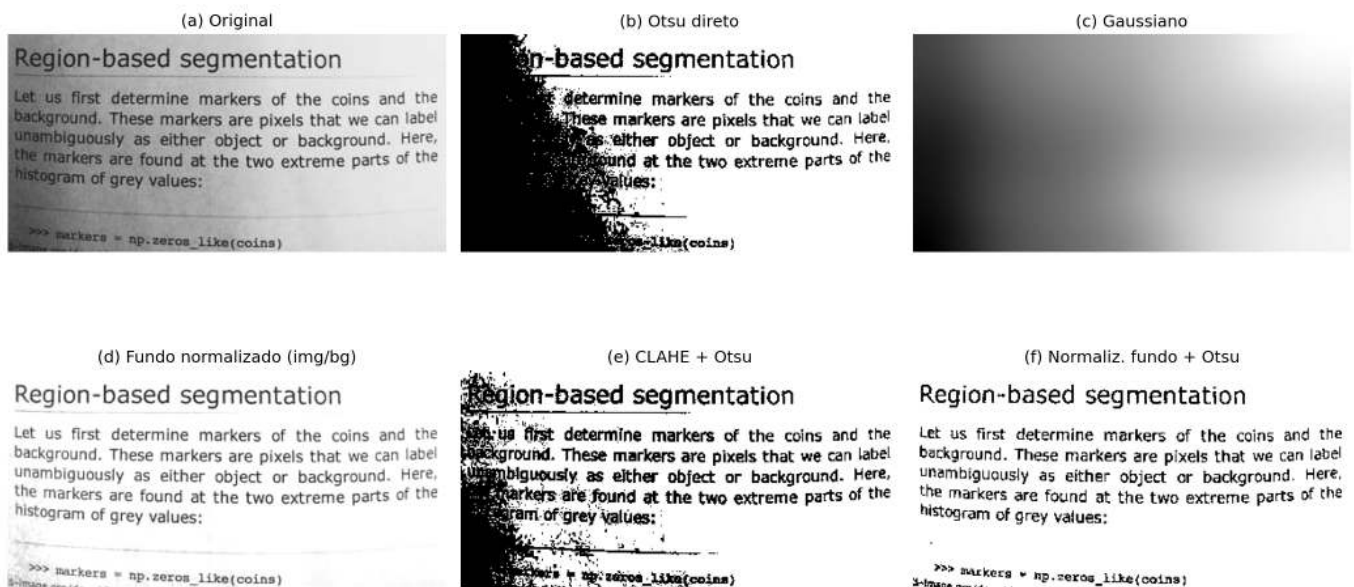



Figura 5.2: Comparação entre estratégias de pré-processamento para binarização da imagem de texto: (a) imagem original; (b) limiarização direta pelo método de Otsu; (c) fundo estimado por filtragem Gaussiana; (d) imagem após normalização de fundo (divisão pela imagem suavizada); (e) CLAHE seguido de limiarização por Otsu; e (f) normalização de fundo seguida de limiarização por Otsu. As imagens ilustram o efeito de cada técnica na compensação de variações de iluminação e na qualidade da segmentação.

Por que funciona? — Normalização de fundo vs. CLAHE

Normalização de fundo: ao dividir a imagem por uma versão fortemente suavizada de si mesma, eliminam-se as variações lentas de luminosidade (gradiente de luz, sombra de borda) sem afetar os detalhes finos — texto, linhas, bolhas. O resultado é uma imagem com iluminação aproximadamente uniforme, onde o limiar global de Otsu passa a funcionar bem em toda a página.

CLAHE: um histograma global equalizado “estica” os tons de toda a imagem de uma vez — útil quando a iluminação é uniforme, mas problemático quando não é. O CLAHE divide a imagem em pequenos blocos (*tiles*) e equaliza cada um separadamente, com um limite máximo de amplificação (`clipLimit`) para não explodir o ruído. É especialmente eficaz para realçar regiões subexpostas localmente, mas não elimina gradientes globais — por isso, aplicá-lo após a normalização de fundo tende a produzir resultados mais consistentes.

5.5 Reconhecimento Óptico de Caracteres (OCR)

Após a binarização, a etapa seguinte do processamento documental consiste na conversão da representação visual dos caracteres em texto codificado digitalmente, processo denominado **Reconhecimento Óptico de Caracteres** (*Optical Character Recognition* — **OCR**).

De forma geral, um sistema de OCR compreende três etapas:

1. **Segmentação:** identifica linhas, palavras e caracteres na imagem, utilizando projeções horizontais e verticais ou detecção de componentes conexos.
2. **Extração de características:** representa cada caractere por atributos visuais, como bordas, curvaturas e padrões de traço.
3. **Reconhecimento:** associa os atributos extraídos ao caractere mais provável. Sistemas atuais utilizam predominantemente redes neurais recorrentes (LSTM) ou arquiteturas baseadas em *transformers*.

Neste livro, utiliza-se o **Tesseract OCR**, acessado por meio da biblioteca `pytesseract` (instalação: `pip install pytesseract`). Desenvolvido originalmente pela Hewlett-Packard entre 1985 e 1995 e atualmente

mantido pelo Google, o Tesseract é descrito em Smith (2007) e Smith (2013). Nas versões recentes, o reconhecimento textual é realizado por redes neurais LSTM.

O desempenho do OCR depende da qualidade da imagem de entrada. Ruído, baixo contraste, distorções geométricas e iluminação irregular reduzem a taxa de reconhecimento. Por esse motivo, etapas como retificação, normalização de fundo e equalização adaptativa (CLAHE) integram o pré-processamento da imagem.

O Tesseract precisa de uma imagem binarizada?

O Tesseract incorpora internamente uma etapa de binarização adaptativa antes do reconhecimento dos caracteres. Por esse motivo, fornecer ao OCR uma imagem previamente binarizada nem sempre produz os melhores resultados.

Como a limiarização é uma operação irreversível, ela pode eliminar variações sutis de intensidade nas bordas dos caracteres, como o *antialiasing*, que podem auxiliar o mecanismo de reconhecimento. Em muitos casos, uma imagem em tons de cinza, com boa iluminação e contraste, produz uma transcrição mais fiel do que sua versão binarizada.

Para investigar esse efeito, compara-se o texto extraído pelo Tesseract a partir de quatro versões da mesma imagem, apresentadas na Figure 5.3: (a) imagem original; (b) imagem submetida à equalização adaptativa (CLAHE) seguida da limiarização pelo método de Otsu; (c) imagem submetida à normalização de fundo seguida da limiarização pelo método de Otsu; e (d) imagem submetida apenas à normalização de fundo, preservando os tons de cinza.

A comparação entre as versões (c) e (d) mostra que, em trechos contendo caracteres visualmente semelhantes, a versão em tons de cinza (d) produziu uma transcrição mais fiel ao texto original do que a versão binarizada (c). Esse resultado indica que a limiarização aplicada no pré-processamento pode eliminar informações úteis ao reconhecimento. Assim, embora a binarização seja essencial para diversas operações de processamento de imagens, ela não constitui necessariamente a melhor entrada para o OCR. A escolha da técnica de pré-processamento deve considerar a etapa subsequente do *pipeline* documental.

```
import pytesseract
from skimage import data
import cv2
from morph import mm

img = data.page()

# Reaproveitando os resultados da seção anterior
img_otsu = mm.threshold(img)

bg = cv2.GaussianBlur(img, (0, 0), sigmaX=25)
img_norm = cv2.divide(img, bg, scale=255)      # tons de cinza, sem Otsu
img_norm_otsu = mm.threshold(img_norm)        # binarizada

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
img_clahe = clahe.apply(img)
img_clahe_otsu = mm.threshold(img_clahe)

# Configuração do Tesseract
# --psm 6: assume um único bloco uniforme de texto (adequado à imagem `page`)
config = "--psm 6"

texto_original = pytesseract.image_to_string(img, config=config)
texto_clahe_otsu = pytesseract.image_to_string(img_clahe_otsu, config=config)
texto_norm_otsu = pytesseract.image_to_string(img_norm_otsu, config=config)
texto_norm_gray = pytesseract.image_to_string(img_norm, config=config)
```

```

for nome, texto in zip(
    ["(a) Original", "(b) CLAHE + Otsu", "(c) Normaliz. fundo + Otsu",
     "(d) Normaliz. fundo (tons de cinza)"],
    [texto_original, texto_clahe_otsu, texto_norm_otsu, texto_norm_gray]
):
    print(f"--- {nome} ---")
    print(texto.strip(), "\n")

mm.show(
    [img, img_clahe_otsu, img_norm_otsu, img_norm],
    titles=["(a) Original", "(b) CLAHE + Otsu", "(c) Normaliz. fundo + Otsu",
           "(d) Normaliz. fundo (cinza)"],
    cols=4,
    figsize=(16, 4)
)

```

```

--- (a) Original ---
ion-based segmentation
Fst determine markers of the coins and the
These markers are pixels that we can label
ly a5 either object or background. Here,
are found at the two extreme parts of the
eS
Be iatetcoins)

--- (b) CLAHE + Otsu ---
Rédion-based segmentation
Menus fist determine markers of the coins and the
yee ground. These markers are pixels that we can label
Sere Epbloueusly. aS either object or background. Here,
Reverkers are fourid at the two extreme parts of the
Bsesern.of Grey Values:
eessere i neo zdtoa lixstcoins)

--- (c) Normaliz. fundo + Otsu ---
Region-based segmentation

Let us first determine markers of the coins and the
background. These markers are pixels that we can label
unambiguously as either object or background. Here,
the markers are found at the two extreme parts of the
histogram of grey values:

sa" Mathers © np.ceros_Like(coins)

--- (d) Normaliz. fundo (tons de cinza) ---
Region-based segmentation

Let us first determine markers of the coins and the
background. These markers are pixels that we can label
unambiguously as either object or background. Here,
the markers are found at the two extreme parts of the
histogram of grey values:

4 2? Barkers = np. zeros_like(coins)

```

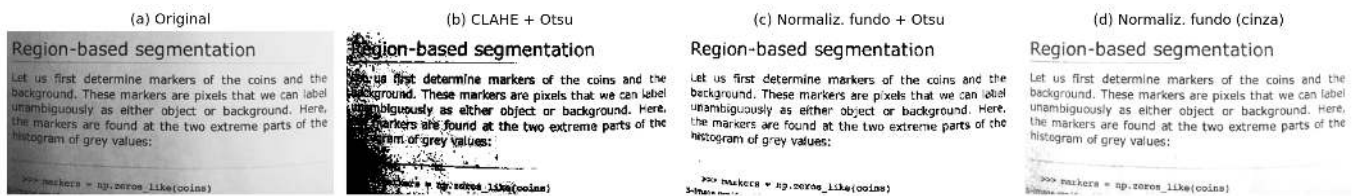


Figura 5.3: Comparação do texto extraído pelo Tesseract a partir de quatro versões da mesma imagem: (a) imagem original; (b) CLAHE seguido de limiarização por Otsu; (c) normalização de fundo seguida de limiarização por Otsu; e (d) normalização de fundo em tons de cinza, sem limiarização. A comparação evidencia que a binarização externa nem sempre favorece o reconhecimento, uma vez que o Tesseract já realiza sua própria binarização adaptativa internamente.

Por que o pré-processamento melhora o OCR?

O desempenho do OCR depende diretamente da qualidade da imagem de entrada. Baixo contraste, iluminação não uniforme, ruído e distorções geométricas dificultam a separação entre texto e fundo e aumentam a probabilidade de erros de reconhecimento.

Técnicas como a normalização de fundo e a equalização adaptativa (CLAHE) corrigem gradientes de iluminação e realçam o contraste local entre os caracteres e o fundo, produzindo imagens mais adequadas ao reconhecimento automático. A limiarização, por sua vez, deve ser aplicada com cautela: por tratar-se de uma operação irreversível, pode eliminar variações sutis de intensidade nas bordas dos caracteres — como o *antialiasing* — que o próprio motor de OCR utiliza internamente para resolver ambiguidades entre símbolos visualmente semelhantes. Por esse motivo, imagens em tons de cinza, corrigidas apenas quanto à iluminação, frequentemente produzem transcrições mais fiéis do que suas versões binarizadas. Em documentos capturados por câmeras de dispositivos móveis, o pré-processamento tende a proporcionar maior ganho de desempenho do que em documentos digitalizados por *scanner*, nos quais a iluminação costuma ser mais uniforme.

5.6 Tradução Automática do Texto Reconhecido

Após o reconhecimento óptico de caracteres, o texto obtido pode ser submetido a técnicas de processamento de linguagem natural, como correção ortográfica, indexação, sumarização e tradução automática.

A tradução automática constitui uma etapa independente do OCR. Enquanto o OCR converte os caracteres presentes na imagem em texto codificado, a tradução opera sobre esse texto no idioma original do documento. Dessa forma, erros de reconhecimento podem ser propagados para a tradução, comprometendo a qualidade do resultado. Sistemas atuais de tradução automática utilizam predominantemente arquiteturas neurais baseadas em mecanismos de atenção e *transformers* (Bahdanau; Cho; Bengio, 2015; Vaswani et al., 2017).

Neste exemplo, utiliza-se o texto obtido a partir da imagem submetida apenas à normalização de fundo, sem limiarização (item **d** da Figure 5.3), por apresentar a transcrição mais fiel entre as estratégias avaliadas na seção anterior.

A tradução é realizada por meio da biblioteca `deep-translator` (instalação: `pip install deep-translator`), que fornece uma interface para diferentes serviços de tradução automática, incluindo o Google Translate.

```
from deep_translator import GoogleTranslator

# Texto obtido pelo OCR a partir da imagem com normalização de fundo (tons de cinza)
texto_en = texto_norm_gray

texto_pt = GoogleTranslator(source="en", target="pt").translate(texto_en)
```

```
print("--- Texto original gerado pela imagem normalizada em tons de cinza (OCR, EN) ---")
print(texto_en)

print("\n--- Texto traduzido (PT-BR) ---")
print(texto_pt)

mm.show(
    [img_norm],
    titles=["Imagem com normalização de fundo (tons de cinza)"],
    cols=1,
    figsize=(6, 4)
)
```

```
--- Texto original gerado pela imagem normalizada em tons de cinza (OCR, EN) ---
Region-based segmentation
```

Let us first determine markers of the coins and the background. These markers are pixels that we can label unambiguously as either object or background. Here, the markers are found at the two extreme parts of the histogram of grey values:

```
4 2? Barkers = np. zeros_like(coins)
```

```
--- Texto traduzido (PT-BR) ---
Segmentação baseada em região
```

Vamos primeiro determinar os marcadores das moedas e o fundo. Esses marcadores são pixels que podemos rotular inequivocamente como objeto ou plano de fundo. Aqui, os marcadores são encontrados nas duas partes extremas do histograma de valores de cinza:

```
4 2? Ladradores = np. zeros_like(moedas)
```

Imagem com normalização de fundo (tons de cinza)

Region-based segmentation

Let us first determine markers of the coins and the background. These markers are pixels that we can label unambiguously as either object or background. Here, the markers are found at the two extreme parts of the histogram of grey values:

```
>>> markers = np.zeros_like(coins)
```

Figura 5.4: Fluxo de reconhecimento e tradução automática. A imagem pré-processada por normalização de fundo, sem limiarização, é utilizada como entrada para o Tesseract OCR, e o texto reconhecido é traduzido do inglês para o português por meio da biblioteca *deep-translator*.

Por que a qualidade do OCR influencia a tradução?

A tradução automática utiliza como entrada o texto produzido pelo OCR. Erros de reconhecimento, como caracteres incorretos, palavras incompletas ou fragmentadas, são propagados para a etapa de tradução e podem alterar o significado do texto.

Consequentemente, a qualidade da tradução depende diretamente da fidelidade da transcrição obtida pelo OCR. Como discutido anteriormente, essa fidelidade nem sempre é maximizada por uma binarização externa: imagens em tons de cinza, corrigidas apenas quanto à iluminação, podem preservar informações relevantes para a distinção entre caracteres visualmente semelhantes. Assim, o pré-processamento da imagem — e a escolha adequada de suas etapas em função da tarefa subsequente — contribui para melhorar não apenas o reconhecimento dos caracteres, mas também o desempenho de etapas posteriores de processamento de linguagem natural, como tradução, indexação e sumarização.

5.7 Fundamentos de OMR e Inspeção Industrial

O **Reconhecimento Óptico de Marcas** (*Optical Mark Recognition* — OMR) é uma técnica de Visão Computacional destinada à identificação automática de marcações em posições previamente definidas de um formulário. Suas aplicações incluem folhas de respostas, questionários, formulários administrativos e outros documentos estruturados.

Diferentemente do OCR (*Optical Character Recognition*), que reconhece caracteres e palavras, o OMR determina a presença, a ausência ou a intensidade de marcas em regiões previamente conhecidas. Em vez de interpretar texto, explora propriedades geométricas e estatísticas associadas ao preenchimento dessas regiões.

Os sistemas modernos de OMR processam imagens obtidas por *scanners*, câmeras ou dispositivos móveis, automatizando tarefas que anteriormente dependiam de equipamentos especializados.

De forma geral, um sistema de OMR compreende as seguintes etapas:

1. **Aquisição:** conversão do documento físico em formato digital;
2. **Pré-processamento:** correção geométrica, redução de ruídos e binarização;
3. **Localização das regiões de interesse:** identificação das áreas destinadas às marcações;
4. **Análise das marcações:** avaliação do preenchimento das regiões candidatas;
5. **Interpretação:** conversão das marcações em respostas ou dados estruturados.

Esses princípios se estendem naturalmente à **Inspeção Industrial Automatizada**. Em linhas de produção, o mesmo encadeamento — aquisição, pré-processamento, segmentação, extração de características e decisão — é empregado para detectar defeitos superficiais, verificar a integridade de componentes e medir dimensões com precisão subpixel. A diferença reside no domínio de aplicação: enquanto o OMR opera sobre documentos com estrutura predefinida, a inspeção industrial lida com objetos cujas variações geométricas e radiométricas devem ser modeladas de forma mais flexível.

Nas seções seguintes, ambas as aplicações são desenvolvidas por meio de projetos práticos que reproduzem etapas típicas de sistemas reais.

5.8 Projetos Práticos: Construção de um *Pipeline* de Análise Documental

Os conceitos deste capítulo serão desenvolvidos por meio de projetos que reproduzem etapas típicas de sistemas reais de análise documental, introduzindo técnicas reutilizáveis em aplicações de OCR, OMR, inspeção visual e processamento de formulários.

5.8.1 Alinhamento Automático de Documentos (*OCR/OMR Pre-processing*)

A correção de inclinação (*deskew*) é uma etapa fundamental no processamento de documentos. Rotações introduzidas durante a digitalização ou captura comprometem a localização de regiões de interesse e reduzem a precisão das etapas subsequentes.

Neste projeto será desenvolvido um sistema para estimar automaticamente a orientação predominante do documento e corrigir sua inclinação. Para isso, serão empregadas técnicas clássicas de detecção de bordas com o operador de Canny e detecção de retas pela Transformada de Hough. A partir das linhas identificadas, será estimado o ângulo de rotação e aplicada uma transformação afim para produzir uma versão alinhada do documento.

Como formulários e folhas de resposta são frequentemente distribuídos em formato PDF, o *pipeline* inicia-se com a rasterização de cada página, convertendo-a em uma imagem matricial. Neste capítulo, essa etapa será realizada com a biblioteca `pdf2image`, gerando imagens PNG com resolução de 300 DPI (*dots per inch*). A partir delas, poderão ser aplicadas as técnicas de detecção de bordas, Transformada de Hough, segmentação, extração de contornos e reconhecimento automático de padrões estudadas ao longo do capítulo.

```
import os
import urllib.request
from pdf2image import convert_from_path
from skimage import data
import cv2

# Diretório dos microdados e folhas de respostas do exame institucional
file_path = "dados/provas_qrcode_EP.pdf"
url_github = (
    "https://raw.githubusercontent.com/fzampirolli/"
    "pdi-vc/master/all/cap06/dados/provas_qrcode_EP.pdf"
)

# Se o arquivo não existir localmente, baixa automaticamente do GitHub
if not os.path.exists(file_path):
    print(f"[DOWNLOAD] Baixando PDF do GitHub: {url_github}")
    try:
        # Garante que a pasta 'dados' existe antes de salvar
        os.makedirs(os.path.dirname(file_path), exist_ok=True)
        urllib.request.urlretrieve(url_github, file_path)
        print("[DOWNLOAD] PDF baixado com sucesso!")
    except Exception as e:
        print(f"[DOWNLOAD] Falha ao baixar o arquivo: {e}")

print(f"PDF de folhas de prova digitalizadas: {file_path}")

if os.path.exists(file_path):
    # Rasterização das páginas com resolução otimizada de 300 DPI
    pages = convert_from_path(file_path, dpi=300)
    for i, page in enumerate(pages):
        saida = f"test{i+1:02d}.png"
        page.save(saida)
        print(f"[INGESTÃO] Página PDF convertida com sucesso: {saida}")
else:
    print("[AVISO] Arquivo PDF não localizado no path. Ativando fallback via skimage.data.")
    # Injeta matriz de texto pública para garantir a execução contínua do pipeline
    img_fallback = data.text()
    cv2.imwrite("test02.png", img_fallback)
    print("[INGESTÃO] Imagem de fallback estruturada: test02.png")

# Carrega e exibe a imagem rasterizada inicial utilizando o ecossistema morph
if os.path.exists('test02.png'):
```



```
img_original = mm.read('test02.png')
else:
    # Fallback definitivo caso até o skimage falhe
    img_original = np.ones((400, 400), dtype=np.uint8) * 255
mm.show(img_original, figsize=(4, 3))
```

PDF de folhas de prova digitalizadas: dados/provas_qrcode_EP.pdf
 [INGESTÃO] Página PDF convertida com sucesso: test01.png
 [INGESTÃO] Página PDF convertida com sucesso: test02.png
 [INGESTÃO] Página PDF convertida com sucesso: test03.png



Figura 5.5: *Pipeline* de ingestão de documentos: rasterização adaptativa de páginas PDF para matrizes discretas em formato PNG, exibindo a página dois.

5.8.2 Algoritmo de Retificação de Inclinação (*Deskew*)

A etapa de *deskew* tem como objetivo estimar e corrigir a inclinação global de um documento digitalizado, alinhando seu conteúdo aos eixos da imagem. A Figure 5.7 apresenta o fluxo completo de processamento, desde a imagem original até o resultado após a correção geométrica. Complementarmente, o simulador da Figure 5.6 permite visualizar o funcionamento da Transformada de Hough e compreender como a orientação predominante é estimada.

O procedimento é composto por três etapas principais:

1. detecção de bordas pelo operador de Canny;
2. estimação da orientação predominante por meio da Transformada de Hough Linear;
3. correção da inclinação utilizando uma transformação afim de rotação.

Após a retificação, o documento passa a apresentar orientação aproximadamente horizontal, favorecendo as etapas subsequentes de segmentação, rotulagem de componentes conexos e reconhecimento de caracteres e marcas.

5.8.3 Modelagem Matemática

As subseções seguintes formalizam, em termos matemáticos, as etapas descritas anteriormente, relacionando o gradiente da imagem, a parametrização de retas no espaço de Hough e a matriz de rotação empregada na correção geométrica.

5.8.3.1 Detecção de Bordas

Inicialmente, a imagem é suavizada por um filtro Gaussiano, reduzindo o efeito de ruídos de alta frequência que podem gerar bordas espúrias. Os conceitos de filtragem espacial e convolução foram apresentados no

Capítulo 3.

Em seguida, o operador de Canny estima o gradiente da imagem. Seja $f(x, y)$ a intensidade da imagem e α o ângulo de rotação.

A magnitude do gradiente é dada por

$$|\nabla f(x, y)| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}.$$

em que:

- $f(x, y)$ representa a intensidade da imagem na posição (x, y) ;
- $\frac{\partial f}{\partial x}$ e $\frac{\partial f}{\partial y}$ são as derivadas parciais nas direções horizontal e vertical;
- $|\nabla f(x, y)|$ é a magnitude do gradiente.

Após o cálculo do gradiente, o algoritmo aplica a supressão de não máximos (*non-maximum suppression*) e a limiarização por histerese, produzindo uma imagem binária contendo as principais bordas do documento.

5.8.3.2 Transformada de Hough

A imagem binária de bordas é processada pela Transformada de Hough Linear, cujo objetivo é detectar estruturas aproximadamente retilíneas. Em vez da representação cartesiana da reta, $y = ax + b$, utiliza-se a representação na forma normal,

$$\rho = x \cos \theta + y \sin \theta,$$

em que:

- x e y são as coordenadas de um ponto pertencente à reta;
- ρ é a distância perpendicular entre a reta e a origem do sistema de coordenadas da imagem;
- θ é o ângulo formado entre a normal à reta e o eixo horizontal da imagem.

Nessa representação, cada ponto de borda (x, y) gera uma curva no espaço de parâmetros (ρ, θ) . A interseção das curvas produzidas por pontos pertencentes à mesma reta origina máximos em uma matriz bidimensional denominada **acumulador**. Assim, os picos do acumulador correspondem às retas predominantes da imagem, como bordas do documento, linhas de formulários ou linhas de texto.

Para estimar a inclinação global do documento, consideram-se apenas as retas cujos ângulos satisfazem

$$-45^\circ \leq \theta \leq 45^\circ.$$

Essa restrição elimina orientações incompatíveis com a disposição esperada do documento e reduz a influência de retas verticais ou de estruturas irrelevantes. Seja $\theta_1, \theta_2, \dots, \theta_n$ o conjunto dos ângulos das retas selecionadas. A estimativa da inclinação global é obtida pela mediana,

$$\hat{\theta} = \text{med}(\theta_1, \theta_2, \dots, \theta_n),$$

em que:

- θ_i é o ângulo da i -ésima reta detectada pela Transformada de Hough;
- n é o número de retas consideradas após a filtragem angular;
- $\hat{\theta}$ é a estimativa da inclinação global do documento.

A mediana é adotada por ser menos sensível à presença de detecções isoladas (*outliers*) do que a média aritmética, produzindo uma estimativa mais estável da orientação predominante.

5.8.3.3 Rotação Afim

Seja $\hat{\theta}$ a inclinação estimada na etapa anterior. A correção geométrica consiste em aplicar uma transformação afim de rotação em torno do centro da imagem, de modo que a orientação predominante passe a coincidir com o eixo horizontal. As transformações afins foram estudadas no **Capítulo 2**, juntamente com as operações de translação, escala, cisalhamento e rotação.

Seja (x, y) a posição de um pixel em relação ao centro da imagem e (x', y') sua posição após a rotação. A transformação é descrita por

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = R(\alpha) \begin{bmatrix} x \\ y \end{bmatrix},$$

em que

$$R(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix},$$

sendo:

- (x, y) as coordenadas originais do pixel em relação ao centro da imagem;
- (x', y') as coordenadas do pixel após a rotação;
- α o ângulo de rotação aplicado para compensar a inclinação estimada do documento;
- $R(\alpha)$ a matriz de rotação.

Na prática, o ângulo aplicado corresponde ao oposto da inclinação estimada,

$$\alpha = -\hat{\theta},$$

em que $\hat{\theta}$ representa a orientação predominante obtida pela Transformada de Hough.

Como as coordenadas transformadas nem sempre coincidem com posições inteiras da malha de pixels, é necessário reamostrar a imagem para determinar os novos valores de intensidade. Na implementação apresentada neste capítulo, a função `mm.rotate` realiza essa operação utilizando interpolação bicúbica, reduzindo os artefatos de reamostramento e preservando a continuidade visual de bordas e caracteres.

```
def retificar_inclinacao_documento(img):

    gray = mm.gray(img) if img.ndim == 3 else img
    edges = cv2.Canny(cv2.GaussianBlur(gray, (5,5), 0), 50, 150)
    lines = cv2.HoughLines(edges, 1, np.pi/180, 200)

    if lines is None:
        return edges, img

    angulos = []
    for line in lines:
        angulo = np.rad2deg(line[0][1]) - 90
        if -45 < angulo < 45:
            angulos.append(angulo)

    if not angulos:
        return edges, img

    return edges, mm.rotate(img, np.median(angulos), interp="bicubic")

# Execução do pipeline de deskew
img_edges, img_final = retificar_inclinacao_documento(img_original)

# Exibição múltipla padronizada com o formato nativo do livro
```

 **Simulador: Correção de Inclinação (Deskew)**

Canny → Hough → Rotação

ÂNGULO REAL
+0.0°

ESTIMADO (HOUGH)
+0.0°

ERRO
0.0°

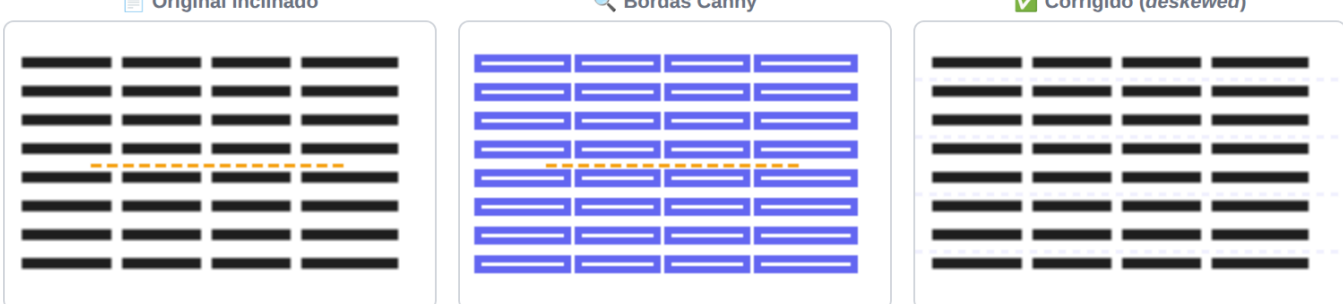
↺ Resetar

Inclinação aplicada: +0.0°

Original inclinado

Bordas Canny

✓ Corrigido (deskewed)



 **Pipeline de processamento**

- 1 Canny:** detecta bordas dos segmentos de texto — pixels de alto gradiente que formam os contornos das linhas.
- 2 Hough:** cada pixel de borda vota em todas as retas que passam por ele. A *mediana* dos ângulos das retas com mais votos estima a inclinação global.
- 3 Rotação inversa:** aplica `cv2.warpAffine` com ângulo negativo, alinhando o documento ao eixo horizontal.

✓ Documento alinhado — nenhuma correção necessária.

Figura 5.6: Simulador interativo de correção de inclinação (*deskew*): mova o *slider* para inclinar o documento e observe as três etapas do *pipeline* — imagem inclinada, bordas Canny e resultado corrigido.

```
mm.show(
    [img_original, img_edges, img_final],
    titles=["Imagem Original", "Bordas de Canny", "Documento Retificado"],
    cols=3,
    figsize=(12, 4)
)
```

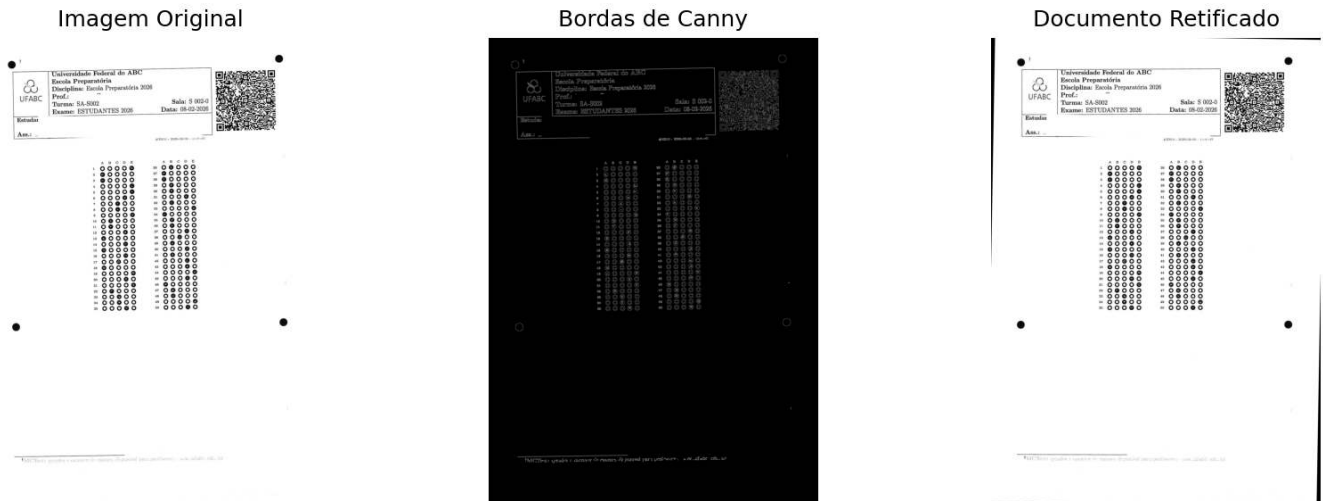


Figura 5.7: *Pipeline* de retificação axial: exibição comparativa entre a entrada rotacionada original, o mapa de gradientes estruturais de Canny e o resultado final alinhado com fundo normalizado em branco.

Por que funciona? — Transformada de Hough

Na Transformada de Hough, cada pixel de borda contribui com votos para todas as retas que podem passar por sua posição. Em vez de selecionar apenas a reta com maior número de votos, o algoritmo considera todas as retas cuja quantidade de votos excede um limiar mínimo e calcula seus respectivos ângulos. A inclinação global do documento é então estimada pela mediana desses ângulos, uma medida robusta a valores discrepantes. Assim, retas espúrias produzidas por sombras, ruídos ou outros elementos da imagem exercem pouca influência sobre a estimativa final, desde que a maioria das retas detectadas corresponda às bordas do documento.

5.8.4 Limitações Práticas

Embora apresente bom desempenho em condições usuais de digitalização, o método depende da existência de estruturas lineares suficientemente definidas para serem detectadas pela Transformada de Hough, como bordas da página, linhas de formulários ou linhas de texto. Sua precisão pode ser reduzida em imagens com baixa resolução, ruído excessivo, sombras intensas ou grandes inclinações. Em geral, documentos digitalizados com resolução próxima de 300 DPI e iluminação homogênea fornecem resultados adequados para aplicações de OCR e OMR.

A implementação apresentada neste capítulo possui finalidade didática, ilustrando os princípios da correção automática de inclinação por meio da detecção de bordas, da Transformada de Hough e da rotação afim. Por utilizar apenas a orientação das estruturas lineares predominantes, o método pode ser aplicado a diferentes tipos de documentos, sem depender de marcadores específicos.

Em sistemas reais de análise documental, entretanto, o alinhamento normalmente utiliza marcadores geométricos previamente conhecidos. No modelo de folha de respostas empregado pelo ecossistema MCTest, por exemplo, são utilizados quatro discos pretos de referência, além das regiões correspondentes ao cabeçalho, ao *QRCode* e aos quadros de respostas. A localização desses elementos permite estimar simultaneamente a rotação, a escala e a translação da folha, tornando o registro menos sensível à quantidade de texto, à ausência

de linhas estruturais e às variações de impressão ou digitalização.

Por esse motivo, a abordagem baseada na Transformada de Hough é utilizada neste capítulo para introduzir os fundamentos do problema, enquanto as etapas posteriores adotam o alinhamento por marcadores geométricos, estratégia predominante em sistemas de OMR e análise documental.

5.8.5 Detecção de Bordas e Contornos

A localização precisa das regiões de interesse é uma etapa essencial em sistemas de OMR. No modelo de folha de respostas utilizado neste capítulo, o cabeçalho e o quadro de respostas estão contidos em um retângulo virtual delimitado por quatro discos pretos posicionados nos cantos. A identificação desses marcadores permite localizar a região de interesse e corrigir distorções geométricas introduzidas durante a aquisição da imagem.

O procedimento é composto por cinco etapas. Inicialmente, aplica-se um **fechamento morfológico** (dilatação seguida de erosão), operação estudada no **Capítulo 4**, utilizando um elemento estruturante em disco (`mm.sedisk(33)`). Essa operação reduz pequenas descontinuidades e preserva os discos de referência, tornando-os mais homogêneos. Em seguida, a imagem é invertida (`mm.neg`), de modo que os discos passem a constituir componentes claros sobre fundo escuro.

Na etapa seguinte, aplica-se a operação `mm.edgeoff` (**Capítulo 4**), que remove componentes conectados às bordas da imagem, eliminando artefatos como sombras de digitalização, marcas de corte e outros objetos espúrios nas margens. Os componentes remanescentes são então analisados a partir de seus contornos e filtrados por propriedades geométricas, como área e circularidade, para identificar os discos candidatos. A implementação do MCTest torna esse processo mais robusto ao selecionar, entre todos os candidatos, os quatro cujos centros formam um retângulo com largura compatível com a da imagem, reduzindo a ocorrência de falsos positivos.

Por fim, os centros dos quatro discos são ordenados espacialmente (superior esquerdo, superior direito, inferior esquerdo e inferior direito) e utilizados como pontos de controle em uma transformação de perspectiva (*perspective warp*). Essa transformação retifica a imagem, produzindo uma representação alinhada e com dimensões conhecidas, adequada às etapas subsequentes de segmentação e reconhecimento.

As principais etapas desse *pipeline*, desde o processamento morfológico até a imagem retificada, são ilustradas a seguir.

```
import cv2
import numpy as np
from morph import mm

# img: imagem em escala de cinza da folha de prova
if img_final.ndim == 2:
    img = img_final
else:
    img = mm.gray(img_final)

# 1. Fechamento morfológico: preserva os discos escuros, removendo tudo menor que o disco
img_close = mm.close(img, mm.sedisk(41))

# 2. Inversão: discos escuros tornam-se componentes claros sobre fundo escuro
img_neg = mm.neg(img_close)

# 3. Remove componentes conectados que tocam a borda da imagem
img_edgeoff = mm.edgeoff(img_neg)

# 4. Extração dos contornos externos
contornos, _ = cv2.findContours(img_edgeoff, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# 5. Filtragem por área e circularidade, mantendo apenas os 4 discos
```

```

centros = []
for c in contornos:
    area = cv2.contourArea(c)
    perimetro = cv2.arcLength(c, True)
    if area < 50 or perimetro == 0:
        continue
    circularidade = 4 * np.pi * area / (perimetro ** 2)
    if circularidade > 0.8:
        M = cv2.moments(c)
        cx, cy = M["m10"] / M["m00"], M["m01"] / M["m00"]
        centros.append((cx, cy))

# Verificação robusta: interrompe o pipeline com mensagem clara em vez de AssertionError
if len(centros) != 4:
    print(f"[AVISO] Esperado 4 discos marcadores, encontrado {len(centros)}.")
    print("  Verifique se a imagem é uma folha de respostas MCTest válida")
    print("  ou ajuste os parâmetros de circularidade e área mínima.")
    img_retificada = img # fallback: preserva a imagem sem retificação
else:
    # 6. Ordenação dos centros: superior-esquerdo, superior-direito,
    # inferior-esquerdo, inferior-direito
    pts = np.array(centros, dtype=np.float32)
    soma = pts.sum(axis=1)
    diff = pts[:, 0] - pts[:, 1]
    tl = pts[np.argmin(soma)]
    br = pts[np.argmax(soma)]
    tr = pts[np.argmax(diff)]
    bl = pts[np.argmin(diff)]
    pts_ordenados = np.array([tl, tr, bl, br], dtype=np.float32)

    # 7. Retificação por transformação de perspectiva (warp)
    largura, altura = 800, 800
    destino = np.array(
        [[0, 0], [largura, 0], [0, altura], [largura, altura]], dtype=np.float32
    )
    M_persp = cv2.getPerspectiveTransform(pts_ordenados, destino)
    img_retificada = cv2.warpPerspective(img, M_persp, (largura, altura))

    mm.show(
        [img_close, img_edgeoff, img_retificada],
        titles=["Fechamento (sedisk 41)", "edgeoff", "Retificada (warp)"],
        cols=3,
        figsize=(12, 4)
    )

mm.write(img_retificada, "img_beetween_disks.png")

```

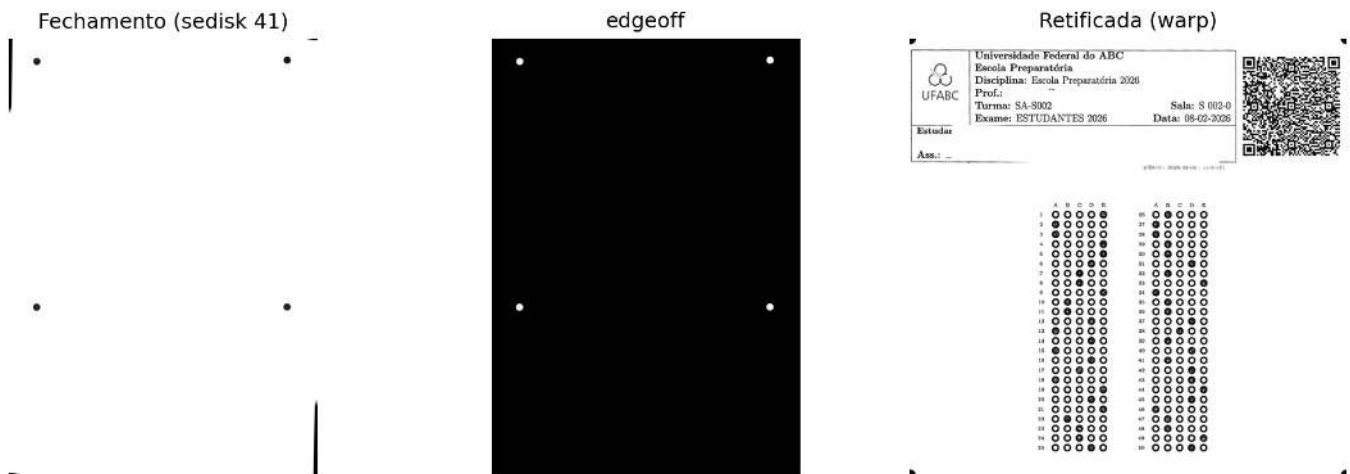



Figura 5.8: Detecção dos discos marcadores, extração de contornos e retificação por transformação de perspectiva.

Por que funciona? — Do fechamento morfológico à retificação

Fechamento morfológico: a dilatação seguida de erosão preenche pequenas descontinuidades e suaviza os contornos dos objetos sem alterar significativamente sua forma global. Ao utilizar um elemento estruturante grande (`sedisk(41)`), detalhes finos, como textos, linhas do formulário e pequenos ruídos, tendem a ser incorporados ao fundo durante o processamento, enquanto objetos de maior escala, como os discos de referência, permanecem preservados e tornam-se mais homogêneos.

Circularidade: após o isolamento dos componentes candidatos, a métrica $C = \frac{4\pi A}{P^2}$ quantifica o quanto sua forma se aproxima de um círculo. Seu valor é igual a 1 para um círculo perfeito e diminui à medida que o contorno se torna mais irregular. Assim, um limiar como $C > 0,8$ permite descartar a maior parte dos falsos positivos sem recorrer a modelos de aprendizado. Um simulador dessa métrica é apresentado na Figure 5.9.

Transformação de perspectiva (*perspective warp*): uma vez identificados os quatro discos de referência, seus centros são utilizados como pontos de controle para estimar a transformação projetiva que mapeia a imagem capturada para o plano do documento. Essa transformação corrige as distorções introduzidas pela perspectiva durante a aquisição da imagem, produzindo uma representação frontal com dimensões conhecidas e adequada às etapas subsequentes de segmentação e reconhecimento.

5.8.6 Isolamento, Segmentação e Decodificação do *QRCode*

Após a retificação geométrica da folha de respostas, realiza-se a detecção e a decodificação do *QRCode* presente no formulário. Esse marcador armazena informações utilizadas pelo sistema de OMR (*Optical Mark Recognition*), como a identificação do estudante, o código da prova e sua variação, possibilitando a recuperação do gabarito correspondente no banco de dados. Por segurança, essas informações são criptografadas antes da geração do *QRCode*. Assim, a sequência decodificada corresponde a uma *string* hexadecimal, cuja interpretação é realizada exclusivamente pelo sistema MCTest. O procedimento é composto por três etapas: pré-processamento morfológico, isolamento da região do *QRCode* e decodificação de seu conteúdo.

Inicialmente, a imagem retificada em escala de cinza é binarizada por meio da operação `mm.threshold`. Em seguida, aplica-se uma **abertura morfológica** (erosão seguida de dilatação), estudada no **Capítulo 4**, utilizando um elemento estruturante quadrado (`mm.sebox(2)`). Essa operação remove pequenos ruídos e suaviza imperfeições sem comprometer a estrutura do marcador. Por fim, a imagem é invertida (`mm.neg`), de modo que o *QRCode* passe a constituir um componente claro sobre fundo escuro, facilitando a extração de seus contornos.

A localização do *QRCode* é realizada pela análise dos contornos externos da imagem binarizada. Entre os componentes detectados, seleciona-se aquele com maior área e geometria aproximadamente quadrada,

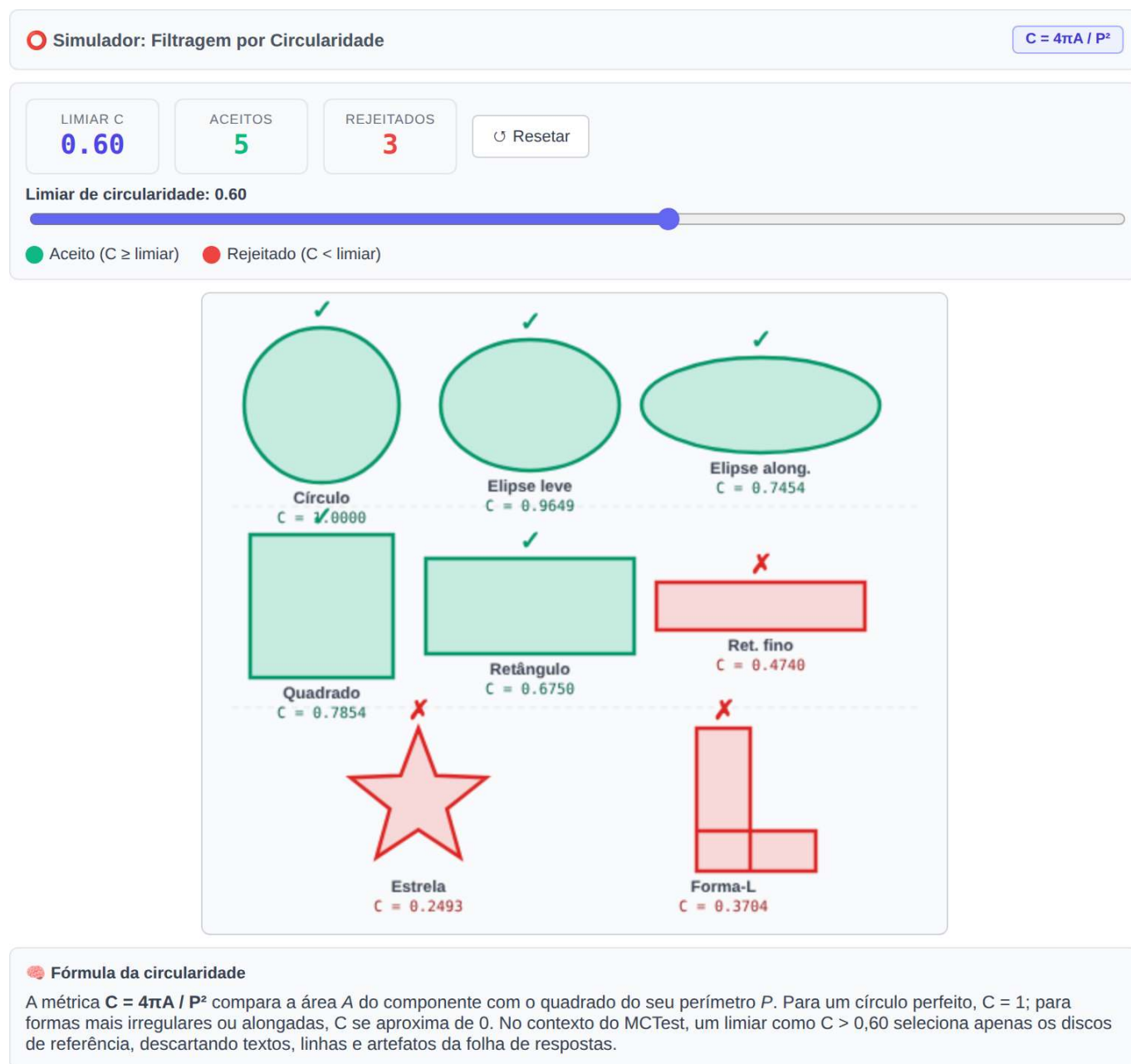


Figura 5.9: Simulador interativo de filtragem por circularidade: mova o *slider* para ajustar o limiar C e observe quais componentes são aceitos (verde) ou rejeitados (vermelho).

descartando os demais elementos impressos da folha. Em seguida, a região correspondente é expandida por uma pequena margem de segurança, garantindo a preservação integral do marcador.

O *QRCode* é então extraído diretamente da imagem retificada em escala de cinza, preservando sua qualidade radiométrica. Como essa região geralmente apresenta dimensões reduzidas, aplica-se um redimensionamento com interpolação cúbica, aumentando a resolução espacial e favorecendo a identificação de seus módulos. A leitura é realizada pelo detector de *QRCode* do OpenCV (`cv2.QRCodeDetector`), que recupera a sequência de caracteres originalmente codificada.

O fluxo completo desse processamento, desde o pré-processamento morfológico até a decodificação do *QRCode*, é ilustrado na Figure 5.10. O trecho exibido na saída corresponde apenas ao início da *string* hexadecimal criptografada; sua interpretação completa é realizada internamente pelo MCTest após a decodificação.

```
import cv2
import numpy as np
from morph import mm

# f: imagem retificada convertida para o tipo correto de 8 bits (0-255)
f = img_retificada.astype('uint8')

# 1. Limiarização: conversão da imagem em tons de cinza para binária
f_thresh = mm.threshold(f)

# 2. Abertura morfológica: elimina pequenos ruídos e suaviza o contorno dos blocos
f_open = mm.open(f_thresh, mm.sebox(2))

# 3. Inversão morfológica: módulos escuros tornam-se componentes claros sobre fundo escuro
f_inv = mm.neg(f_open)

# 4. Conversão segura para uint8 com escala 0-255
img_uint8 = (f_inv.astype(np.uint8) * 255) if f_inv.max() == 1 else f_inv.astype(np.uint8)

# 5. Detecção de contornos externos
contornos, _ = cv2.findContours(img_uint8, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

if not contornos:
    raise ValueError("Nenhum contorno encontrado. Verifique o limiar ou a imagem de entrada.")

# 6. Filtragem pelo maior contorno com proporção aproximadamente quadrada
# (aspect ratio entre 0.7 e 1.3 descarta retângulos alongados da folha)
def is_square_like(contorno, tol=0.3):
    x, y, w, h = cv2.boundingRect(contorno)
    ratio = w / h if h > 0 else 0
    return (1 - tol) <= ratio <= (1 + tol)

candidatos = [c for c in contornos if is_square_like(c)]

if not candidatos:
    raise ValueError(
        "Nenhum contorno quadrado encontrado. "
        "Verifique se o QRCode está presente na imagem ou ajuste a tolerância."
    )

# Seleciona o maior candidato quadrado por área de bounding box
maior_contorno = max(candidatos, key=lambda c: cv2.boundingRect(c)[2] * cv2.boundingRect(c)[3])
x, y, w, h = cv2.boundingRect(maior_contorno)

# 7. Expansão da bounding box com margem de segurança (evita truncamento do QRCode)
margem = 5
h_img, w_img = img_uint8.shape[:2]
```

```

x1 = max(x - margem, 0)
y1 = max(y - margem, 0)
x2 = min(x + w + margem, w_img)
y2 = min(y + h + margem, h_img)

# 8. Recorte da região de interesse a partir da imagem original (nítida, em cinza)
img_qrcode_final = img_retificada[y1:y2, x1:x2]

# 9. Ampliação para resolução mínima de decodificação (400 px no lado maior)
# cv2.QRCodeDetector requer módulos com ao menos 3-4 px de largura para decodificar
# com segurança; imagens menores que ~400 px tendem a falhar.
lado = max(img_qrcode_final.shape[:2])
escala = max(400 / lado, 1.0)
img_para_leitura = cv2.resize(
    img_qrcode_final, None,
    fx=escala, fy=escala,
    interpolation=cv2.INTER_CUBIC
)

# 10. Inicialização do detector nativo de QRCode do OpenCV
detector = cv2.QRCodeDetector()

# 11. Detecção geométrica e decodificação dos dados textuais
dados, pontos, qrcode_reto = detector.detectAndDecode(img_para_leitura)

# Visualização intermediária: progressão da binarização ao isolamento do QRCode
mm.show(
    [f_thresh, f_open, f_inv, img_qrcode_final],
    titles=["1. Limiarização", "2. Abertura morfológica", "3. Inversão", "Imagem final"],
    cols=3, figsize=(12, 4)
)

# Validação e saída dos metadados extraídos
if dados:
    print(f"QRCode decodificado com sucesso: \n{dados[:50]}...")
else:
    raise ValueError(
        "Falha na decodificação do QRCode. "
        "Verifique o limiar, as margens da região ou a qualidade da imagem."
    )

```

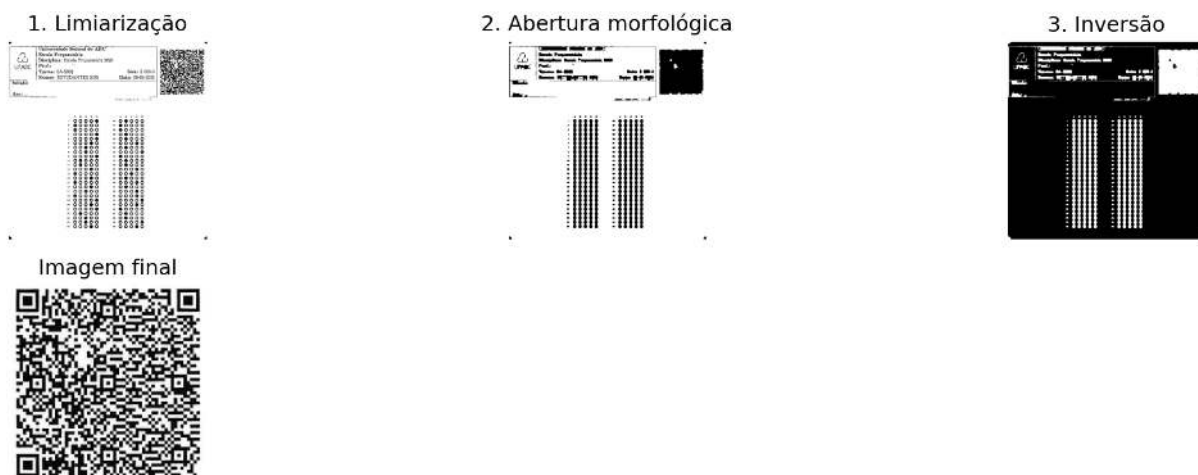


Figura 5.10: *Pipeline* de processamento do *QRCode*: limiarização, abertura morfológica, inversão e recorte final para decodificação.

QRCode decodificado com sucesso:

325a356b71367266556955646b7233454149624a694f417730...

Simulador: Pipeline de Isolamento do QRCode

Threshold → Fechamento → Contorno Quadrado → Recorte

Selecione a etapa:

0 · Original

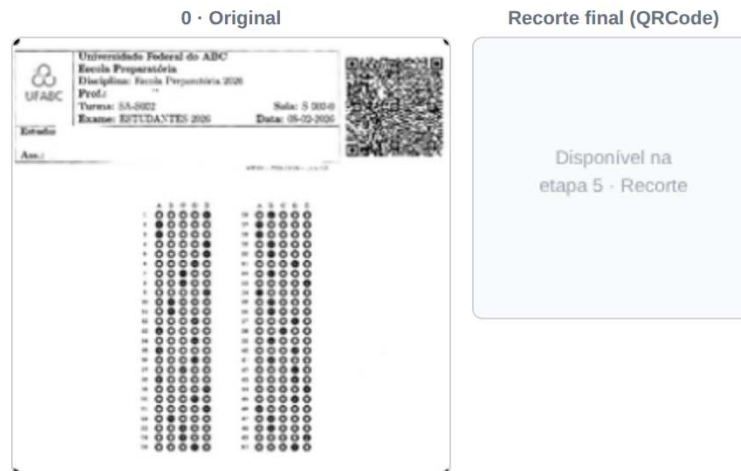
1 · Limiarização

2 · Fechamento

3 · Contorno

4 · Recorte

Imagem original em escala de cinza: documento com QRCode no canto superior direito e bolhas de respostas.



Etapas do pipeline (replica fiel do algoritmo OpenCV de referência)

- 1 **Limiarização** (`mm.threshold`): Segmenta os módulos escuros do QRCode isolando-os do fundo claro.
- 2 **Fechamento morfológica** (`mm.close + mm.sebox(k)`): dilatação seguida de erosão remove ruídos e preenche discontinuidades. `sebox(0)` = kernel 3×3, `sebox(1)` = 5×5, e assim por diante. Note que aqui se usa *fechamento* (≠ abertura usada na célula de código acima), o que estimula a comparação entre os dois operadores.
- 4 **Deteção de contornos** (`cv2.findContours, RETR_EXTERNAL`): Busca o maior contorno externo com proporção aproximadamente quadrada (razão largura/altura entre 0.7 e 1.3), descartando retângulos alongados da folha.
- 5 **Recorte + margem de segurança**: Extrai a *bounding box* do contorno selecionado, com margem de 5px, a partir da imagem original em escala de cinza.

Figura 5.11: Simulador interativo do *pipeline* de isolamento do QRCode: navegue pelas etapas de filtragem, ajuste o elemento estruturante do fechamento morfológica e veja a detecção do contorno quadrado sobre a imagem original do gabarito.

Por que funciona? — Isolamento e decodificação do QRCode

Abertura morfológica: diferentemente do fechamento, a abertura (erosão seguida de dilatação) remove pequenos ruídos e protuberâncias sem alterar significativamente a geometria dos objetos maiores. Assim, preserva a estrutura do QRCode enquanto elimina componentes espúrios que poderiam dificultar sua localização.

Seleção por geometria: o QRCode possui formato aproximadamente quadrado ($w/h \approx 1$). A combinação desse critério com a seleção do componente de maior área descarta linhas do formulário, textos e outros elementos impressos, permitindo isolar o marcador sem o uso de modelos de aprendizado.

Redimensionamento antes da decodificação: quando o QRCode ocupa poucos pixels na imagem, seus módulos tornam-se difíceis de distinguir. O redimensionamento com interpolação cúbica aumenta a resolução espacial da região de interesse, facilitando a identificação dos padrões do código pelo `cv2.QRCodeDetector` e tornando a decodificação mais robusta.

5.8.7 Decodificação de Códigos de Barras

Na seção anterior, a decodificação de *QR Codes* foi realizada utilizando o detector nativo do OpenCV (`cv2.QRCodeDetector`), que integra em uma única interface a detecção geométrica do símbolo, sua retificação e a extração da informação codificada.

Para códigos de barras lineares (1D), como EAN-13, Code 39 e Code 128, uma alternativa amplamente utilizada é a biblioteca `pyzbar`. Diferentemente do `QRCodeDetector`, ela suporta diversas simbologias de códigos de barras e também pode ser empregada na leitura de *QR Codes*.

O procedimento consiste em localizar automaticamente cada símbolo presente na imagem e interpretar a sequência de barras e espaços correspondente, produzindo a cadeia de caracteres codificada. Além dos dados decodificados, a biblioteca fornece informações como a simbologia identificada e a posição do código na imagem, permitindo sua posterior validação ou processamento.

A Figure 5.12 apresenta um exemplo de código de barras e o resultado de sua decodificação utilizando a biblioteca `pyzbar`.

```
import os
import urllib.request
import numpy as np
from pyzbar.pyzbar import decode
from morph import mm

barcode_path = 'dados/barcode.png'
url_github = (
    "https://raw.githubusercontent.com/fzampirolli/"
    "pdi-vc/master/all/cap06/dados/barcode.png"
)

# Se o arquivo não existir localmente, baixa automaticamente do GitHub
if not os.path.exists(barcode_path):
    print(f"[DOWNLOAD] Baixando código de barras do GitHub: {url_github}")
    try:
        os.makedirs(os.path.dirname(barcode_path), exist_ok=True)
        urllib.request.urlretrieve(url_github, barcode_path)
        print("[DOWNLOAD] Imagem baixada com sucesso!")
    except Exception as e:
        print(f"[DOWNLOAD] Falha ao baixar o arquivo: {e}")

if os.path.exists(barcode_path):
    image = mm.read(barcode_path)
else:
    # Fallback sintético: gera um padrão de barras verticais que simula um Code-128
    print("[AVISO] Arquivo 'dados/barcode.png' não encontrado.")
    print("Usando imagem sintética para demonstração do pipeline.")
    h, w = 100, 400
    img_synth = np.ones((h, w), dtype=np.uint8) * 255
    # Barras escuras em posições regulares (padrão simplificado)
    for x in range(20, w - 20, 8):
        if (x // 8) % 3 != 0:
            img_synth[:, x:x+4] = 0
    image = img_synth

mm.show(image)

# Executa o decode do pyzbar
try:
    barcodes = decode(image)
except ImportError:
```



```
print("[ERRO] zbar do sistema não encontrada. Rode: !apt-get install -y libzbar0")
barcodes = []

if barcodes:
    dados_bc = barcodes[0].data.decode("utf-8")
    tipo = barcodes[0].type
    print(f"Código de barras decodificado com sucesso [{tipo}]:\n{dados_bc}")
else:
    print("[INFO] Nenhum código de barras detectado na imagem.")
    print(
        "Em imagem sintética isso é esperado - "
        "substitua pelo arquivo real para decodificar."
    )
```



Figura 5.12: Decodificação de código de barras linear com *pyzbar*: imagem de entrada e dados extraídos.

```
Código de barras decodificado com sucesso [EAN13]:
00000000000055
```

5.9 O MCTest como Estudo de Caso: do Protótipo ao Sistema em Produção

Até este ponto, as principais etapas do *pipeline* de processamento foram apresentadas e analisadas individualmente, incluindo a correção da inclinação (*deskew*), a detecção de marcadores, a retificação por perspectiva e a leitura de *QR Codes*. Embora essa abordagem facilite a compreensão de cada técnica, aplicações reais exigem a integração dessas etapas em um fluxo de processamento único e consistente.

O **MCTest** constitui um exemplo dessa integração. Desenvolvido na UFABC e disponibilizado como software de código aberto, o sistema é utilizado desde 2012 na correção automatizada de avaliações, oferecendo suporte a diferentes modelos de folhas de resposta, gabaritos individualizados e geração automática de relatórios de desempenho (Zampirolli, 2023).

A partir deste ponto, o foco deixa de ser a implementação isolada de algoritmos e passa a ser a organização desses algoritmos em uma aplicação completa. Além da qualidade dos métodos de processamento de imagens, um sistema dessa natureza deve atender a requisitos como robustez diante de diferentes condições de aquisição, facilidade de manutenção e capacidade de evolução para novas funcionalidades.

Nas próximas seções, será analisado o módulo de Visão Computacional do **MCTest**, implementado no arquivo `CVMCTest.py`. O objetivo é mostrar como os conceitos apresentados ao longo deste capítulo são combinados em um *pipeline* de processamento empregado em uma aplicação real.

5.9.1 Obtenção e Preparação do Módulo

O arquivo `CVMCTest.py` integra o sistema **MCTest** e, em sua versão original, depende de modelos, configurações e outros componentes do *framework* Django. Como essas dependências não estão disponíveis no ambiente utilizado neste capítulo, o módulo precisa ser adaptado para execução de forma independente.

Para isso, o arquivo é obtido diretamente do repositório do projeto por meio da biblioteca **requests**. Em seguida, comandos `sed` são empregados para remover as importações e dependências específicas do ambiente Web, produzindo uma versão autocontida do módulo. Essa adaptação preserva a implementação dos algoritmos de Visão Computacional, permitindo sua execução e análise sem a necessidade de instalar ou configurar toda a infraestrutura do sistema **MCTest**.


```
import requests
CVMCTest = requests.get(
    "https://raw.githubusercontent.com/fzampirolli/mctest/master/exam/CVMCTest.py"
)
with open('CVMCTest.py', 'w') as writefile:
    writefile.write(CVMCTest.text)
```

Cada comando `sed` remove importações específicas do ambiente Django, tornando o arquivo `CVMCTest.py` utilizável de forma independente neste capítulo.

```
# remove linhas with "from django.", ...
!sed --in-place '/from django./d' CVMCTest.py
!sed --in-place '/from exam./d' CVMCTest.py
!sed --in-place '/from mctest./d' CVMCTest.py
!sed --in-place '/from student./d' CVMCTest.py
!sed --in-place '/from topic./d' CVMCTest.py
!sed --in-place '/from .models import VariationExam/d' CVMCTest.py
```

i 🧠 Por que remover as dependências do Django?

Na implementação original, o arquivo `CVMCTest.py` faz parte de uma aplicação desenvolvida com o *framework* Django e, por isso, importa modelos, configurações e outros componentes específicos desse ambiente. Como esses elementos não estão disponíveis neste capítulo, o módulo não pode ser importado diretamente.

Os comandos `sed` removem apenas essas dependências, sem alterar as rotinas de Visão Computacional implementadas no arquivo. Dessa forma, o módulo pode ser executado de maneira independente, preservando o comportamento dos algoritmos apresentados.

Esse procedimento ilustra um princípio importante de engenharia de software: separar a lógica da aplicação da infraestrutura em que ela está inserida, facilitando a reutilização, os testes e o estudo de componentes específicos.

5.9.2 Extração da Área de Respostas

Após a leitura da folha, a função `getAnswerArea` executa automaticamente as etapas de detecção dos marcadores de referência e retificação por perspectiva apresentadas nas seções anteriores. Como resultado, é obtida uma imagem contendo apenas a região destinada às respostas, alinhada e com dimensões padronizadas.

Essa padronização simplifica as etapas subsequentes de processamento, pois a localização dos campos de marcação passa a ser conhecida e independente da posição original da folha durante a digitalização.

A Figure 5.13 apresenta a folha de respostas original em escala de cinza, enquanto a Figure 5.14 mostra a região de respostas obtida após a aplicação da função `getAnswerArea`.

```
import os
from pdf2image import convert_from_path
from skimage import data as skdata
import cv2
from morph import mm

file = "dados/provas_qrcode_EP.pdf"
MYFILES = 'extra02.qrcode'

if os.path.exists(file):
    pages = convert_from_path(file, 200) # dpi 100=min 500=max
    numPAGES = 0
    for page in pages:
        myfile0 = MYFILES + '_p' + str(numPAGES) + '.png'
        page.save(myfile0)
```

```

numPAGES += 1
print(f"[INGESTÃO] Página convertida: {myfile0}")
pages.clear()
img_color = mm.read(myfile0)
img_inicial = mm.gray(img_color)
else:
    print("[AVISO] Arquivo 'dados/provas_qrcode.pdf' não encontrado.")
    print("        Usando imagem pública skimage.data.page() como substituto.")
    img_inicial = skdata.page()

mm.show(img_inicial)

```

```

[INGESTÃO] Página convertida: extra02.qrcode_p0.png
[INGESTÃO] Página convertida: extra02.qrcode_p1.png
[INGESTÃO] Página convertida: extra02.qrcode_p2.png

```

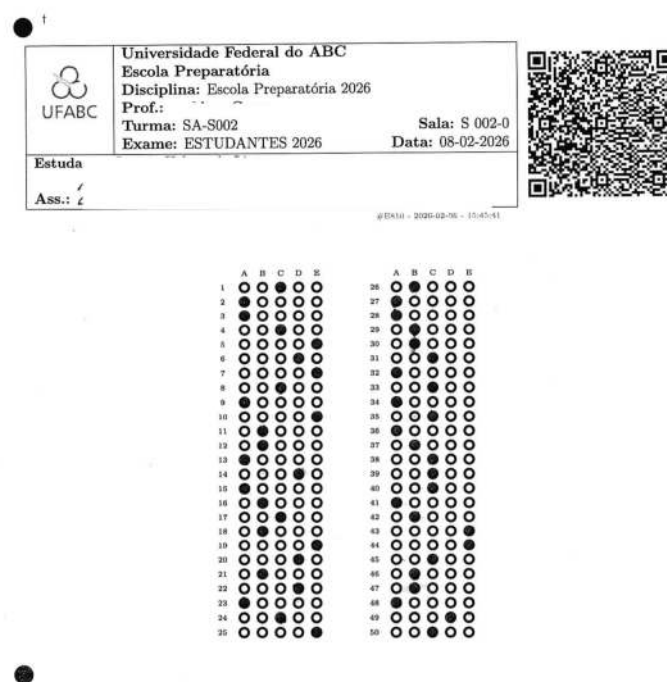


Figura 5.13: Imagem da folha de respostas em escala de cinza carregada a partir do PDF rasterizado.

```

import CVMCTest
countPage = 0
img_getAnswerArea = CVMCTest.cvMCTest.getAnswerArea(img_inicial, countPage)

```

```
mm.show(img_getAnswerArea)
```

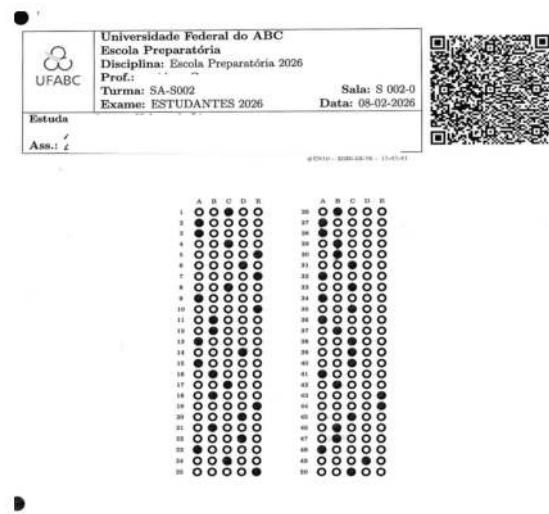


Figura 5.14: Área de respostas extraída por *getAnswerArea*: região retificada contendo os quadros de marcação.

Nota de compatibilidade: versões recentes do NumPy (2.0) removeram o alias `np.int0`. Caso `CVMCTest.py` utilize esse tipo, o comando abaixo aplica a correção diretamente no arquivo antes de recarregá-lo:

```
!sed -i 's/box = np.int0(cv2.boxPoints(rect))/box = cv2.boxPoints(rect).astype(np.intp)/' \
./CVMCTest.py
```

```
import importlib
import CVMCTest

importlib.reload(CVMCTest)
```

```
<module 'CVMCTest' from '/home/fz/fz/VSCoDe/pdi-vc/all/cap06/CVMCTest.py'>
```

5.9.3 Segmentação do *QRCode*

Após a extração da área de respostas, o `MCTest` realiza duas etapas preparatórias para a leitura das marcações: a segmentação do *QRCode* e a localização dos quadros que contêm as questões.

A função `segmentQRcode` isola a região da imagem correspondente ao *QRCode*, apresentada na Figure 5.15. Em seguida, essa região é processada pela função `getQRCode`, responsável por sua decodificação e pela extração dos metadados da prova.

Caso o *QRCode* não possa ser decodificado, o processamento da folha continua normalmente. As respostas do estudante ainda são lidas e registradas no arquivo CSV de saída; apenas as informações obtidas a partir do *QRCode*, como a identificação da prova ou do estudante, permanecem indisponíveis.

```
import CVMCTest
imgQRcode = CVMCTest.cvMCTest.segmentQRcode(img_getAnswerArea, countPage)
mm.show(imgQRcode)
```



Figura 5.15: Região do *QRCode* isolada por *segmentQRcode* dentro da área de respostas retificada.

A função `CVMCTest.cvMCTest.getQRcode(img, countPage)` integra as etapas de segmentação e decodificação do *QRCode*. Internamente, ela utiliza `CVMCTest.cvMCTest.decodeQRcode(imgQRcode)` para interpretar a *string* hexadecimal codificada no símbolo e construir o dicionário `qr`, além de retornar o indicador lógico `myFlagArea`, que informa se a leitura foi realizada com sucesso.

O dicionário `qr` reúne os metadados da prova utilizados nas etapas subsequentes de processamento. Seus principais campos são:

- **date:** identificador temporal da prova, composto pela data de geração e por um *timestamp* interno do MCTest.
- **idClassroom, idExam e idStudent:** identificadores da turma, da prova e do estudante.
- **term:** período letivo.
- **stylesheet:** folha de estilo utilizada na geração do formulário.
- **var1 a var5:** quantidade de questões em cada nível de dificuldade.
- **text:** número de questões dissertativas.
- **answer:** número de alternativas por questão.
- **numquest:** número total de questões.
- **correct e dbtext:** campos preenchidos durante a correção, contendo o gabarito e informações adicionais.
- **variations e variant:** informações sobre as versões da prova.

Esses metadados identificam a prova e o estudante, permitindo selecionar o gabarito correspondente e parametrizar as etapas seguintes de leitura e correção das respostas.

```
myFlagArea, qr = CVMCTest.cvMCTest.getQRcode(img_inicial, countPage)
myFlagArea, qr
```

```
(True,
{'date': '260208-1770403541363',
 'idClassroom': '955',
 'idExam': '810',
 'idStudent': '448898',
 'term': '0',
 'stylesheet': '1',
 'var1': '50',
 'var2': '0',
 'var3': '0',
 'var4': '0',
 'var5': '0',
 'text': '0',
 'answer': '5',
```

```
'numquest': 50,
'correct': '',
'dbtext': '',
'variations': '0',
'variant': '0'})
```

Esses metadados permitem identificar a prova, recuperar o gabarito correspondente e parametrizar as etapas subsequentes de processamento.

Após a decodificação do *QRCode*, o processamento retorna à área de respostas para localizar os quadros que contêm as marcações do estudante.

5.9.4 Localização dos Quadros de Respostas

A imagem produzida por `getAnswerArea` contém toda a região útil da folha, incluindo o cabeçalho, onde se encontra o *QRCode*, e os quadros destinados às respostas. Como a leitura das marcações utiliza apenas esses quadros, a região correspondente ao cabeçalho é descartada por meio do recorte `img_getAnswerArea[300:, :]`.

A Figura 5.16 apresenta essa região de interesse. Embora esse recorte seja posteriormente utilizado pela função `findSquares` para localizar os quadros de respostas, o `MCTest` realiza inicialmente a leitura do *QRCode*, pois ele contém os metadados necessários para identificar a prova e configurar as etapas subsequentes do processamento.

```
img_getAnswerArea_aux = img_getAnswerArea[300:, :]
mm.show(img_getAnswerArea_aux)
```

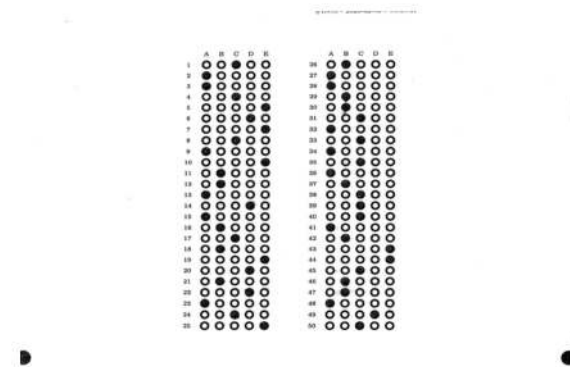


Figura 5.16: Recorte inferior da área de respostas, concentrando os quadros de bolhas a serem segmentados.

A função `findSquares` recebe a imagem da área de respostas e os metadados armazenados em `qr`, retornando as coordenadas dos quadros que delimitam os grupos de questões.

Cada elemento de `rectSquares` contém as coordenadas dos vértices superior esquerdo e inferior direito de um quadro de respostas, que serão utilizadas na etapa de segmentação das bolhas.

```
rectSquares = CVMCTest.cvMCTest.findSquares(qr, img_getAnswerArea, countPage)
rectSquares
```

```
[[[np.int64(395), np.int64(350)], [np.int64(950), np.int64(516)]],
 [[np.int64(394), np.int64(602)], [np.int64(951), np.int64(767)]]]
```

5.9.5 Leitura Automática das Respostas

Conhecidos os metadados da prova (`qr`) e as coordenadas dos quadros de respostas (`rectSquares`), o `MCTest` identifica automaticamente as alternativas marcadas pelo estudante.

O código a seguir integra as etapas apresentadas anteriormente. Para cada quadro delimitado em `rectSquares`, as funções `setColumns` e `setLines` estimam, respectivamente, o número de alternativas por questão e o número de questões a partir da distribuição espacial das bolhas. Em seguida, `segmentAnswers` determina a alternativa assinalada em cada questão e `setAnswersOneLine` reúne os resultados de todos os quadros no campo `qr['answers']`.

No modo de operação adotado neste capítulo, em que o `MCTest` é executado de forma independente de seu banco de dados, o conteúdo de `qr['answers']` é comparado ao gabarito armazenado na primeira página do arquivo PDF, correspondente ao modelo de prova sem enunciados utilizado nos exemplos.

A Figure 5.17 apresenta os quadros de respostas processados pelo algoritmo, enquanto a saída do programa exibe o conteúdo final de `qr['answers']`.

```
testAnswers = []
if myFlagArea:

    imgQ_all = []

    for countSquare in range(len(rectSquares)):
        p1, p2 = rectSquares[countSquare]

        if True:
            imgQi = CVMCTest.cvMCTest.imgAnswers[p1[0]:p2[0], p1[1]:p2[1]]
            [NUM_COLUMNS, img] = CVMCTest.cvMCTest.setColumns(imgQi, countPage, countSquare)
            [NUM_LINES, img] = CVMCTest.cvMCTest.setLines(imgQi, countPage, countSquare)
            NUM_RESPOSTAS = NUM_COLUMNS
            NUM_QUESTOES = NUM_LINES

            imgQiNC = CVMCTest.cvMCTest.imgAnswers[p1[0]:p2[0], p1[1]:p2[1]]
            testAnswers.append(CVMCTest.cvMCTest.segmentAnswers(
                [imgQi, imgQiNC], countPage, countSquare, NUM_QUESTOES, qr

            ))

            imgQ_all.append(imgQiNC)

    qr = CVMCTest.cvMCTest.setAnswersOneLine(testAnswers, qr)
    # deixa as respostas de cada quadro em uma linha

mm.show(imgQ_all)
print(f"Respostas lidas das {len(qr['answers'].split(","))} questões: \
      \n{qr['answers'][:-19]}...")
```

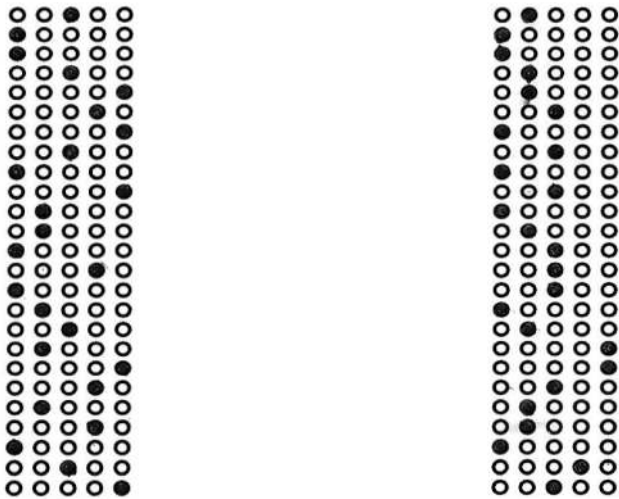


Figura 5.17: Respostas lidas automaticamente pelo MCTest após segmentação e classificação de todas as bolhas.

Respostas lidas das 50 questões:

C,A,A,C,E,D,E,C,A,E,B,B,A,D,A,B,C,B,E,D,B,D,A,C,E,B,A,A,B,B,C,A,C,A,C,A,B,C,C,C,...

💡 Conectando os Pontos

O campo `qr['answers']` representa o resultado final do *pipeline* de Visão Computacional apresentado neste capítulo. Sua obtenção integra todas as etapas estudadas, desde a rasterização do documento e a retificação geométrica até a extração da área de respostas, a decodificação do *QRCode*, a localização dos quadros e a identificação das alternativas marcadas.

Esse *pipeline* ilustra a transição de um protótipo para um sistema em produção. No MCTest, os algoritmos de Visão Computacional permanecem essencialmente os mesmos; as principais diferenças concentram-se em aspectos de engenharia de software, como tratamento de exceções, suporte a diferentes modelos de formulários, integração com banco de dados, interface Web e mecanismos de auditoria e manutenção.

Nos experimentos deste capítulo, a primeira página do arquivo PDF contém o gabarito da prova, enquanto as páginas subsequentes correspondem às folhas de respostas dos estudantes. Após a obtenção de `qr['answers']`, o MCTest compara automaticamente as respostas lidas com o gabarito para calcular a pontuação de cada estudante.

Na utilização completa do sistema, os resultados da correção são consolidados em um arquivo CSV e enviados ao professor juntamente com um arquivo compactado contendo informações auxiliares para auditoria. Entre esses arquivos estão os recortes das questões em que foram detectadas múltiplas marcações ou outras situações que exigem revisão manual. O professor pode, então, inspecionar essas imagens, decidir a interpretação mais adequada e, se necessário, atualizar o arquivo CSV antes da importação definitiva das notas.

5.10 Inspeção Industrial Automatizada

A inspeção visual automatizada é uma aplicação da Visão Computacional em que imagens de peças ou produtos são analisadas para verificar o atendimento a critérios de qualidade previamente definidos. Em uma linha de produção, as imagens podem ser obtidas por câmeras ou outros dispositivos de aquisição e processadas automaticamente para identificar defeitos, medir dimensões ou verificar a presença de componentes.

A estratégia de inspeção depende das características do produto, do tipo de defeito de interesse e da disponibilidade de uma imagem de referência. Neste capítulo são apresentadas duas abordagens clássicas:

- **Subtração de imagens:** compara a imagem da peça inspecionada com uma imagem de referência

considerada livre de defeitos. Regiões em que a diferença de intensidade excede um limiar são classificadas como possíveis defeitos. Essa abordagem pressupõe que as imagens estejam geometricamente alinhadas e tenham sido adquiridas sob condições semelhantes de iluminação.

- **Análise de textura:** utiliza características da textura da superfície para identificar regiões cuja aparência difere do padrão esperado, sem a necessidade de uma imagem de referência. Essa abordagem é adequada para materiais que apresentam textura aproximadamente homogênea, como tecidos, papéis e superfícies metálicas.

Nas próximas seções, essas duas estratégias são ilustradas por meio de exemplos construídos a partir de imagens da biblioteca `skimage.data`. O objetivo é apresentar os princípios de funcionamento de cada abordagem em experimentos que possam ser reproduzidos integralmente pelo leitor.

5.10.1 Referências e *Datasets* Públicos

Em aplicações de inspeção industrial, o desempenho de algoritmos de detecção de defeitos é frequentemente avaliado em *datasets* públicos, que fornecem imagens representativas e, em muitos casos, anotações de referência (*ground truth*). Neste capítulo, entretanto, os exemplos utilizam imagens sintéticas derivadas de `skimage.data` (ver Figure 5.18), permitindo reproduzir todos os experimentos sem dependência de bases de dados externas.

Para estudos mais abrangentes e comparação entre algoritmos, destacam-se os seguintes *datasets* públicos:

- **MVTec Anomaly Detection Dataset (MVTec AD):** conjunto de imagens de objetos e texturas, contendo amostras sem defeitos e com defeitos, acompanhadas de máscaras de segmentação em nível de pixel para as imagens anômalas (Bergmann et al., 2019). Disponível em: <https://www.mvtec.com/company/research/datasets/mvtec-ad>.
- **Kolektor Surface-Defect Dataset (KolektorSDD):** conjunto de imagens de componentes industriais com defeitos superficiais anotados, utilizado em estudos de detecção e segmentação de defeitos (Tabernik et al., 2020). Disponível em: <https://www.vicos.si/resources/kolektorsdd/>.
- **NEU Surface Defect Database:** conjunto de imagens de superfícies de aço laminado, organizado em seis categorias de defeitos superficiais, frequentemente empregado na avaliação de métodos de classificação e detecção (Song; Yan, 2013). Disponível em: http://faculty.neu.edu.cn/songkechen/zh_CN/zdylm/263270/list/index.htm.

```
import numpy as np
from skimage import data, color
from morph import mm

# Imagem de referência (produto sem defeito)
product_color = data.coffee()
product_gray = color.rgb2gray(product_color)

# Inserção de defeito simulado: arranhão escuro de 10×100 px
defect_image = np.copy(product_gray)
defect_image[100:110, 200:300] = 0.1

# Detecção por subtração e limiarização
difference = np.abs(product_gray - defect_image)
defect_threshold = 0.15 # ajustável conforme a aplicação
detected_defect = (difference > defect_threshold).astype(np.uint8) * 255

mm.show(
    [product_gray, defect_image, detected_defect],
    titles=["Referência", "Com defeito", "Defeito detectado"],
    cols=3,
    figsize=(12, 4)
```

)

```
status = "Defeito detectado." if detected_defect.any() else "Produto conforme."
print(status)
```



Figura 5.18: Detecção de defeito por subtração de imagem: produto de referência, imagem com defeito simulado e máscara de anomalia detectada.

Defeito detectado.

i Subtração de Imagens: Registro Geométrico e Princípio de Funcionamento

A subtração de imagens pressupõe que a imagem de inspeção esteja geometricamente alinhada à imagem de referência. Diferenças de posicionamento, rotação, escala ou perspectiva produzem regiões de diferença que podem ser confundidas com defeitos.

Em aplicações com aquisição controlada, esse alinhamento é obtido durante a captura por meio de gabaritos mecânicos, esteiras e câmeras fixas, permitindo comparar diretamente imagens sucessivas. Um exemplo é a inspeção de uma caixa de ferramentas sempre posicionada na mesma orientação para verificar a ausência de algum item.

Quando esse controle não é possível, emprega-se o **registro de imagens**, que estima uma transformação geométrica para compensar diferenças de translação, rotação, escala e, quando necessário, perspectiva. Após o registro, realiza-se a comparação pixel a pixel entre as duas imagens. Em regiões sem alterações, as diferenças de intensidade tendem a ser próximas de zero; onde existe um defeito, surgem diferenças locais que podem ser destacadas por limiarização. A utilização da diferença absoluta permite detectar tanto defeitos mais claros quanto mais escuros que a referência.

O desempenho do método depende principalmente da qualidade do alinhamento geométrico e da escolha do limiar utilizado para separar pequenas variações de aquisição das diferenças associadas aos defeitos. A Figure 5.19 ilustra o efeito do desalinhamento entre as imagens e a importância do registro geométrico antes da aplicação da subtração.

5.10.2 Detecção de Defeitos por Análise de Textura

Em aplicações nas quais não existe uma imagem de referência, a detecção de defeitos pode ser baseada nas características da textura da superfície. Nesse caso, busca-se identificar regiões cuja aparência difere do padrão predominante do material.

Neste exemplo, utiliza-se a **variância local** como medida de heterogeneidade. Para cada posição da imagem, calcula-se a variância dos níveis de intensidade em uma vizinhança de dimensões fixas. Regiões com baixa variância tendem a apresentar textura mais uniforme, enquanto alterações locais, como riscos, manchas ou imperfeições, podem produzir valores mais elevados dessa medida.

A Figure 5.20 ilustra esse procedimento utilizando a imagem `skimage.data.brick()`. Inicialmente, calcula-se o mapa de variância local por meio de uma janela deslizante. Em seguida, aplica-se uma limiarização para destacar as regiões cuja variância excede o valor especificado, identificando possíveis áreas de interesse para inspeção.

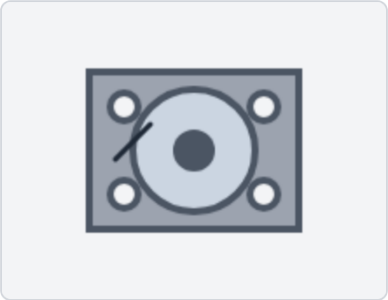
 **Simulador: Subtração de Imagens e Registro Geométrico** |Referência – Inspeção| > Limiar

 **Desalinhamento da Captura (Esteira)**
Translação horizontal (Δx): 0 px
Translação vertical (Δy): 0 px
Rotação (θ): 0.0°

 **Parametrização do Inspetor**
Limiar de tolerância (T): 35

 **Anomalia localizada! Defeito superficial detectado na estrutura interna (103 px afetados).**

1. Referência estável


2. Inspeção (captura real)


3. Máscara de anomalia (subtração direta)


 **Desafio Pedagógico**
Com o modo **Subtração direta** ativo, utilize os controles de translação e rotação para simular pequenos desalinhamentos entre a imagem de referência e a imagem de inspeção. Observe que diferenças de poucos pixels ou de alguns graus já podem produzir regiões de diferença ao longo das bordas da peça, mesmo na ausência de novos defeitos. Para reduzir essas respostas indesejadas, seria necessário aumentar o **limiar de tolerância**, o que também pode diminuir a sensibilidade à detecção de defeitos de baixo contraste, como o arranhão escuro presente no lado esquerdo da peça. Em seguida, alterne para **Subtração com registro**. Após o alinhamento geométrico das imagens, as diferenças decorrentes do posicionamento são reduzidas, permitindo que a subtração evidencie predominantemente o defeito introduzido.

Figura 5.19: Simulador interativo de inspeção industrial por subtração de imagens: controle as distorções geométricas de desalinhamento (registro) e o limiar de detecção para observar o impacto nos falsos positivos.

Modelagem Matemática

Considere uma imagem em níveis de cinza representada por

$$f : \Omega \subset \mathbb{Z}^2 \rightarrow \mathbb{R},$$

em que Ω é o domínio da imagem e $f(x, y)$ representa a intensidade do pixel nas coordenadas (x, y) . Em imagens de 8 bits, essas intensidades pertencem ao intervalo $[0, 255]$. Neste exemplo, entretanto, elas foram normalizadas para o intervalo $[0, 1]$, sem alterar o funcionamento do algoritmo.

Para cada posição da imagem, considera-se uma vizinhança quadrada $W_{x,y}$ de dimensão 15×15 pixels.

A média local é dada por

$$\mu(x, y) = \frac{1}{|W_{x,y}|} \sum_{(u,v) \in W_{x,y}} f(u, v),$$

e a variância local é calculada por

$$\sigma^2(x, y) = \frac{1}{|W_{x,y}|} \sum_{(u,v) \in W_{x,y}} f(u, v)^2 - \mu(x, y)^2.$$

Na implementação a seguir, essas duas médias são obtidas pela função `cv2.blur`,

```
mean = cv2.blur(img, (15,15))
mean2 = cv2.blur(img**2, (15,15))
var = mean2 - mean**2
```

Em seguida, calcula-se a diferença entre os mapas de variância da imagem de referência e da imagem inspecionada,

$$D(x, y) = |\sigma_d^2(x, y) - \sigma_r^2(x, y)|,$$

em que $\sigma_r^2(x, y)$ e $\sigma_d^2(x, y)$ são, respectivamente, as variâncias locais da imagem de referência e da imagem contendo o defeito. Após a normalização do mapa $D(x, y)$, aplica-se uma limiarização para obter a máscara das possíveis anomalias.

```
import numpy as np
import cv2
from skimage import data as skdata
from morph import mm

# Imagem de textura uniforme (tijolo)
texture = skdata.brick().astype(np.float32) / 255.0

# Inserção de defeito sintético: mancha clara 20x80 px
texture_defect = np.copy(texture)
texture_defect[60:80, 80:160] = 0.95

# Mapa de variância local (janela 15x15)
def variancia_local(img, ksize=15):
    img_f = img.astype(np.float32)
    mean = cv2.blur(img_f, (ksize, ksize))
    mean2 = cv2.blur(img_f ** 2, (ksize, ksize))
    return np.clip(mean2 - mean ** 2, 0, None)

var_ref = variancia_local(texture)
```

```

var_defect = variancia_local(texture_defect)
diff_var   = np.abs(var_defect - var_ref)

# Normaliza e limiariza
diff_norm = (diff_var / diff_var.max() * 255).astype(np.uint8)
_, mask    = cv2.threshold(diff_norm, 30, 255, cv2.THRESH_BINARY)

mm.show(
    [texture, texture_defect, diff_norm, mask],
    titles=["Textura original", "Com defeito", " $\Delta$  variância local", "Anomalia detectada"],
    cols=4,
    figsize=(16, 4)
)

status = "Defeito de textura detectado." if mask.any() else "Superfície conforme."
print(status)

```

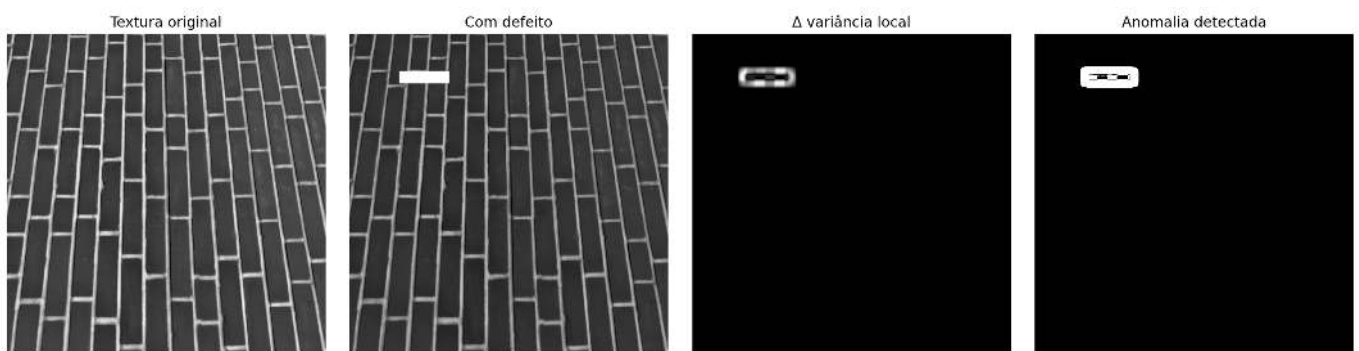


Figura 5.20: Detecção de heterogeneidade de textura: mapa de variância local e máscara de anomalia.

Defeito de textura detectado.

Por que funciona? — Análise de Textura

A variância local mede a dispersão das intensidades em uma vizinhança da imagem. Em regiões cuja textura permanece uniforme, essa medida tende a variar pouco. Quando um defeito modifica o padrão da superfície, a distribuição das intensidades também se altera, produzindo diferenças na variância local.

Neste capítulo, a detecção é realizada comparando os mapas de variância da imagem de referência e da imagem com defeito. Após a normalização, aplica-se uma limiarização para destacar as regiões em que essa diferença excede um valor especificado.

Os principais parâmetros do método são o tamanho da janela utilizada no cálculo da variância e o limiar empregado na segmentação. Janelas menores são mais sensíveis a detalhes finos, enquanto janelas maiores produzem mapas mais suaves e podem reduzir a resposta a defeitos de pequenas dimensões.

5.11 Resumo

Neste capítulo foram apresentados métodos de Visão Computacional aplicados à análise de documentos e à inspeção visual automatizada. As principais técnicas estudadas foram:

- **Pré-processamento de documentos:** aplicação de normalização de fundo, equalização adaptativa (CLAHE) e limiarização por Otsu para reduzir os efeitos de iluminação não uniforme e melhorar a segmentação do texto.
- **Reconhecimento óptico de caracteres (OCR):** conversão de imagens de documentos em texto codificado por meio do Tesseract OCR, evidenciando a influência do pré-processamento na qualidade

do reconhecimento.

- **Tradução automática:** aplicação de técnicas de processamento de linguagem natural para traduzir o texto obtido pelo OCR.
- **Retificação geométrica de documentos:** utilização do detector de bordas de Canny, da Transformada de Hough e de transformações projetivas para corrigir a perspectiva de documentos digitalizados.
- **Localização e retificação de formulários:** emprego de operações morfológicas, análise de contornos e transformação de perspectiva para identificar marcadores de referência e extrair automaticamente regiões de interesse.
- **Leitura de códigos bidimensionais e unidimensionais:** detecção e decodificação de *QR Codes* e códigos de barras para identificação automática de documentos e metadados.
- **Reconhecimento óptico de marcas (OMR):** leitura automatizada de formulários e folhas de respostas, ilustrada por um estudo de caso do sistema MCTest.
- **Inspeção industrial:** detecção de defeitos por comparação com uma imagem de referência e por análise da variância local.

Ao longo do capítulo, os algoritmos foram implementados e avaliados com imagens da biblioteca `skimage.data` e com documentos reais, permitindo reproduzir os experimentos apresentados.

Próximos Passos

Os métodos apresentados neste capítulo mostram como técnicas de Processamento Digital de Imagens, Visão Computacional e Processamento de Linguagem Natural podem ser integradas em *pipelines* para análise automatizada de documentos.

No próximo capítulo serão estudadas técnicas de **extração de características** e **reconhecimento de padrões**, com ênfase em descritores capazes de representar imagens por atributos numéricos para comparação, classificação e reconhecimento automático. Esses conceitos constituem a base para os capítulos dedicados ao aprendizado de máquina e ao aprendizado profundo aplicados à Visão Computacional.

5.12 Uso do Gemini Notebook como Tutor Complementar

Como apoio ao estudo deste capítulo, recomenda-se a utilização do **Gemini Notebook** como tutor complementar. A ferramenta emprega modelos de inteligência artificial para responder perguntas, elaborar resumos e explicar conceitos com base nos documentos fornecidos como fonte de consulta, permitindo ao estudante revisar o conteúdo de forma interativa.

 Estude com o Tutor Inteligente

 [ACESSAR Gemini Notebook: CAPÍTULO 06](#)

Uso Crítico das Respostas

As respostas geradas pelo Gemini Notebook são produzidas automaticamente por um modelo de inteligência artificial e podem conter omissões ou imprecisões. Por esse motivo, elas devem ser utilizadas como material de apoio, e não como substitutas do estudo do capítulo.

Sempre que houver dúvidas, consulte o texto deste livro, execute os exemplos apresentados e, quando necessário, complemente a consulta com livros, artigos científicos e outras fontes acadêmicas confiáveis.

5.13 Lista de Exercícios

Os exercícios a seguir consolidam os conceitos apresentados neste capítulo por meio de adaptações, experimentos e extensões dos algoritmos desenvolvidos ao longo do texto.

1. (10%) Investigue a influência do ângulo de inclinação na etapa de *deskew*. Gere versões rotacionadas da imagem `skimage.data.page()` para ângulos entre -10° e 10° , aplique o algoritmo apresentado no capítulo e compare o ângulo estimado com o ângulo utilizado na rotação. Apresente os resultados em uma tabela e discuta a precisão do método.
2. (15%) Aplique normalização de fundo e CLAHE (utilizando pelo menos três combinações de `clipLimit` e `tileGridSize`) à imagem `skimage.data.page()` degradada artificialmente com gradiente de iluminação e sombra lateral. Segmente cada versão por meio do método de Otsu e compare os resultados utilizando o número de componentes conexos espúrios e a métrica IoU em relação a uma máscara de referência construída manualmente.
3. (15%) Investigue a sensibilidade da filtragem por circularidade, $C = \frac{4\pi A}{P^2}$, na detecção dos marcadores circulares. Utilizando o simulador da Figure 5.9, gere discos sintéticos com ruído geométrico crescente e avalie os limiares $C \in \{0,5, 0,6, 0,7, 0,8\}$. Apresente uma tabela relacionando o limiar ao número de falsos positivos e falsos negativos e discuta o compromisso entre sensibilidade e especificidade.
4. (15%) A partir dos quatro marcadores detectados, implemente a retificação por perspectiva utilizando `cv2.getPerspectiveTransform` e `cv2.warpPerspective`. Em seguida, perturbe artificialmente as coordenadas dos pontos de controle com ruído gaussiano de desvio-padrão $\sigma \in \{1, 3, 5\}$ pixels e avalie o erro de reprojeção obtido após a homografia inversa.
5. (15%) Adapte o *pipeline* de aquisição e retificação desenvolvido neste capítulo para processar documentos contendo códigos de barras lineares em substituição aos *QR Codes*. Rasterize o PDF com `pdf2image` a 300 DPI, aplique o *deskew*, decodifique o símbolo com `pyzbar` e apresente a imagem retificada juntamente com a sequência de caracteres obtida.
6. (15%) Estenda a leitura de bolhas do MCTest para identificar três situações: **OK**, **BRANCO** (nenhuma alternativa marcada) e **DUPLA MARCAÇÃO** (duas ou mais alternativas acima de um limiar de preenchimento). Avalie pelo menos três valores desse limiar, apresente os resultados em um `pandas.DataFrame` e discuta sua influência na classificação das respostas.
7. (15%) Construa um *pipeline* de inspeção industrial combinando subtração de imagens e análise de variância local da textura sobre um conjunto de imagens sintéticas contendo defeitos simulados. Para cada imagem, gere uma máscara de referência (*ground truth*), calcule a métrica IoU (*Intersection over Union*) das duas abordagens para diferentes limiares de decisão e apresente os resultados em tabelas e visualizações produzidas com `mm.show`.
8. (Bônus – 10%) Implemente manualmente a estimação do ângulo de inclinação sem utilizar `cv2.HoughLines` ou `cv2.HoughLinesP`. A partir do mapa de bordas obtido pelo detector de Canny, construa o acumulador da Transformada de Hough, $\rho = x \cos \theta + y \sin \theta$, para $\theta \in [-45^\circ, 45^\circ]$, identifique os máximos do acumulador e estime a inclinação pela mediana das retas detectadas. Compare os resultados com a implementação do OpenCV e discuta a influência de retas espúrias na estimativa final.

Referências do Capítulo

A fundamentação teórica e os estudos de caso apresentados neste capítulo apoiam-se nas seguintes referências:

- Gonzalez; Woods (2018), para os fundamentos de detecção de bordas, limiarização, segmentação, operações morfológicas, reconhecimento óptico de caracteres e transformações geométricas aplicadas à análise de documentos.
- Szeliski (2022), para a Transformada de Hough, o registro e alinhamento de imagens, as transformações projetivas (homografias) e os fundamentos da inspeção visual automatizada.

- Bradski; Kaehler (2008), para a utilização da biblioteca OpenCV nas etapas de detecção de bordas, Transformada de Hough, transformações geométricas, análise de contornos e decodificação de *QR Codes*.
- Smith (2007) e Smith (2013), para a arquitetura, o funcionamento e a evolução do mecanismo de reconhecimento óptico de caracteres **Tesseract OCR**, empregado nos exemplos de OCR apresentados neste capítulo.
- Bahdanau; Cho; Bengio (2015) e Vaswani et al. (2017), para os fundamentos da tradução automática baseada em redes neurais, incluindo mecanismos de atenção e arquiteturas *transformer*.
- Zampiroli (2023), para a descrição do sistema MCTest, utilizado como estudo de caso de um *pipeline* completo para leitura e correção automatizada de folhas de respostas.
- Bergmann et al. (2019), Tabernik et al. (2020) e Song; Yan (2013), para os *datasets* públicos de inspeção industrial **MVTec AD**, **KolektorSDD** e **NEU Surface Defect Database**, utilizados como referência para avaliação e comparação de algoritmos de detecção de defeitos.



5.14 Parte Prática com Exercícios de Programação

Os exercícios de programação (EP) desta seção complementam os conceitos apresentados ao longo do Capítulo 6 por meio da implementação de algoritmos relacionados à inspeção industrial e à análise de documentos. O objetivo é consolidar os fundamentos estudados, reproduzindo, em escala reduzida, etapas de um *pipeline* típico de Visão Computacional.

Diferentemente dos capítulos anteriores, cujos exercícios enfatizavam operações mais diretamente relacionadas aos dados de imagem, os EPs deste capítulo concentram-se nas **grandezas intermediárias** produzidas durante o processamento, como áreas, perímetros, circularidade, ângulos de retas, graus de preenchimento de bolhas, mapas de variância e mapas de diferença. Essa abordagem permite compreender e validar cada etapa do *pipeline* de forma independente, sem depender de bibliotecas especializadas para aquisição de imagens, detecção de marcadores ou decodificação de códigos — com exceção do exercício de encerramento do capítulo (EP06_08), que propositalmente introduz o uso do OpenCV para a segmentação e a decodificação real de um *QR Code*, fechando o ciclo entre os conceitos teóricos e as ferramentas empregadas na prática.

Os exercícios seguem a mesma sequência conceitual do capítulo, em ordem crescente de complexidade. Inicialmente, são abordadas métricas de avaliação de segmentação, utilizadas para quantificar a qualidade de máscaras binárias. Em seguida, estudam-se critérios geométricos para seleção de marcadores, classificação de marcações em formulários e estimação da inclinação de documentos por meio da Transformada de Hough. Na parte final, os exercícios exploram a normalização de iluminação, a detecção de defeitos por análise de textura e a integração entre registro geométrico e subtração de imagens em um *pipeline* simplificado de inspeção industrial.

Cada exercício representa uma etapa isolada de um sistema real de Visão Computacional, permitindo validar individualmente conceitos que, em aplicações industriais, são combinados em um único *pipeline* de inspeção.

Legenda de Dificuldade

Nível	Significado	EPs
	Muito fácil / fácil — implementação de um único conceito ou algoritmo simples	EP06_01, EP06_02

Nível	Significado	EPs
	Fácil–médio — tratamento de múltiplos casos ou utilização de critérios estatísticos simples	EP06_03, EP06_04
	Médio — processamento matricial ponto a ponto	EP06_05
	Difícil — processamento matricial com operações em vizinhança (janela deslizante)	EP06_06
	Muito difícil — integração de múltiplas etapas de um <i>pipeline</i> de Visão Computacional	EP06_07
	Especial — uso de biblioteca especializada (cv2) para segmentação geométrica e decodificação real de código de barras/QRCode	EP06_08

! Diretrizes para a Resolução dos Exercícios de Programação

Salvo indicação em contrário, todos os exercícios utilizam a convenção de coordenadas matriciais [linha] [coluna], com origem em (0, 0) no canto superior esquerdo da imagem. Quando houver necessidade de arredondamento numérico, deve-se utilizar o arredondamento padrão para o inteiro mais próximo (*round half away from zero*, com `np.floor(img + 0.5)`). Comparações com limiares (por exemplo, circularidade, variância, diferença de intensidade ou grau de preenchimento) devem ser consideradas **estritas** ($>$), exceto quando o enunciado especificar explicitamente outro critério. Cada exercício foi elaborado para enfatizar um conceito específico apresentado no capítulo. Recomenda-se implementar inicialmente a solução de forma direta e, somente após sua validação, buscar alternativas mais eficientes ou mais gerais.

🎯 Objetivo deste Caderno

Este caderno foi elaborado para apoiar o desenvolvimento, a validação e os testes das soluções dos **Exercícios de Programação (EPs)** em um ambiente interativo, como o Google Colab ou o Jupyter Notebook. Após verificar o funcionamento da implementação com os casos de teste apresentados, o código pode ser submetido ao Moodle para a avaliação oficial.

Download

Execute a célula a seguir para obter os arquivos `morph.py` e `testsuite.py`, utilizados pelos exercícios deste capítulo.

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)
```

```
import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.2 | TestSuite: 1.1.2

Executando os Testes

Após implementar a solução, execute `TestSuite("EP06_01.extensão").run()` em uma nova célula, substituindo `extensão` pela linguagem utilizada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema obtém automaticamente os casos de teste do repositório do curso, executa o programa e apresenta o resultado da avaliação.

Em Python, também é possível testar a solução diretamente a partir de uma *string*, sem a necessidade de salvar o código em um arquivo. Para isso, armazene o programa em uma variável e utilize o método `run_code`:

```
codigo = """
# ... seu código aqui ...
"""

TestSuite("EP06_01").run_code(codigo)
```

5.14.1 EP06_01 Avaliação de Segmentação por IoU (*Intersection over Union*)

Ao longo deste capítulo, diversas etapas do *pipeline* produzem **máscaras binárias**, como na segmentação de documentos, na localização de *QR Codes* e na detecção de defeitos. Para avaliar objetivamente a qualidade dessas segmentações, é necessário compará-las com uma máscara de referência (*ground truth*).

Uma das métricas mais utilizadas para esse fim é a **IoU** (*Intersection over Union*, ou Interseção sobre União), definida como a razão entre a área de interseção e a área de união de duas máscaras binárias. Quanto maior o valor da IoU, maior a concordância entre a segmentação produzida pelo algoritmo e a referência.

5.14.1.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (número de linhas) e C (número de colunas).
2. **Máscara de referência:** Ler os $L \times C$ elementos binários (0 ou 1) da matriz `ref`.
3. **Máscara predita:** Ler os $L \times C$ elementos binários (0 ou 1) da matriz `pred`.
4. **Interseção:** Contar o número de posições (i, j) para as quais `ref[i][j] = 1` e `pred[i][j] = 1`.
5. **União:** Contar o número de posições (i, j) para as quais `ref[i][j] = 1` ou `pred[i][j] = 1`.
6. **Caso degenerado:** Se a união for igual a 0, definir $\text{IoU} = 1,0$, pois ambas as máscaras são vazias.
7. **Cálculo:** Caso a união seja maior que zero, calcular

$$\text{IoU} = \frac{|\text{Intersecao}|}{|\text{Uniao}|}.$$

8. **Classificação:** Determinar a classificação qualitativa utilizando o valor de IoU **antes** do arredondamento.
9. **Arredondamento:** Exibir a IoU com quatro casas decimais.
10. **Saída:** Imprimir, nessa ordem, a interseção, a união, a IoU e a classificação.

5.14.1.2 Restrições Computacionais

- Se a união for igual a 0, não deve ser realizada a divisão; a IoU deve ser definida como 1,0.

- As faixas de classificação utilizam comparações não estritas ($\geq$).
- A classificação deve ser realizada utilizando o valor da IoU em precisão completa, antes do arredondamento para exibição.

5.14.1.3 Fundamentação Teórica

A IoU é definida por

$$\text{IoU} = \frac{|R \cap P|}{|R \cup P|},$$

em que:

- R representa o conjunto de pixels pertencentes à máscara de referência;
- P representa o conjunto de pixels pertencentes à máscara predita;
- $|R \cap P|$ corresponde ao número de pixels pertencentes simultaneamente às duas máscaras;
- $|R \cup P|$ corresponde ao número de pixels pertencentes a pelo menos uma das máscaras.

Faixa de IoU	Classificação	Interpretação
$\text{IoU} \geq 0,90$	EXCELENTE	Concordância muito elevada entre as máscaras.
$0,70 \leq \text{IoU} < 0,90$	BOM	Pequenas diferenças entre as máscaras.
$0,50 \leq \text{IoU} < 0,70$	ACEITAVEL	Concordância parcial entre as máscaras.
$\text{IoU} < 0,50$	RUIM	Baixa concordância entre as máscaras.

A IoU depende apenas da sobreposição entre as máscaras e, portanto, é independente do tamanho da imagem.

5.14.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro L .
- Linha 2: inteiro C .
- Próximas L linhas: elementos binários (0 ou 1) da matriz **ref**.
- Próximas L linhas: elementos binários (0 ou 1) da matriz **pred**.

Saída:

- Linha 1: **Intersecao:** X
- Linha 2: **Uniao:** Y
- Linha 3: **IoU:** Z
- Linha 4: **Classificacao:** NOME

O valor de IoU deve ser impresso com quatro casas decimais.

5.14.1.5 Exemplos

Entrada	Saída	Observação
2	Intersecao: 1	A metade da região de referência foi corretamente segmentada.
2	Uniao: 2	
1 1	IoU: 0.5000	
0 0	Classificacao: ACEITAVEL	
1 0		
0 0		
2	Intersecao: 0	Ambas as máscaras são vazias; por convenção, IoU = 1,0.
2	Uniao: 0	
0 0	IoU: 1.0000	
0 0	Classificacao: EXCELENTE	
0 0		
0 0		

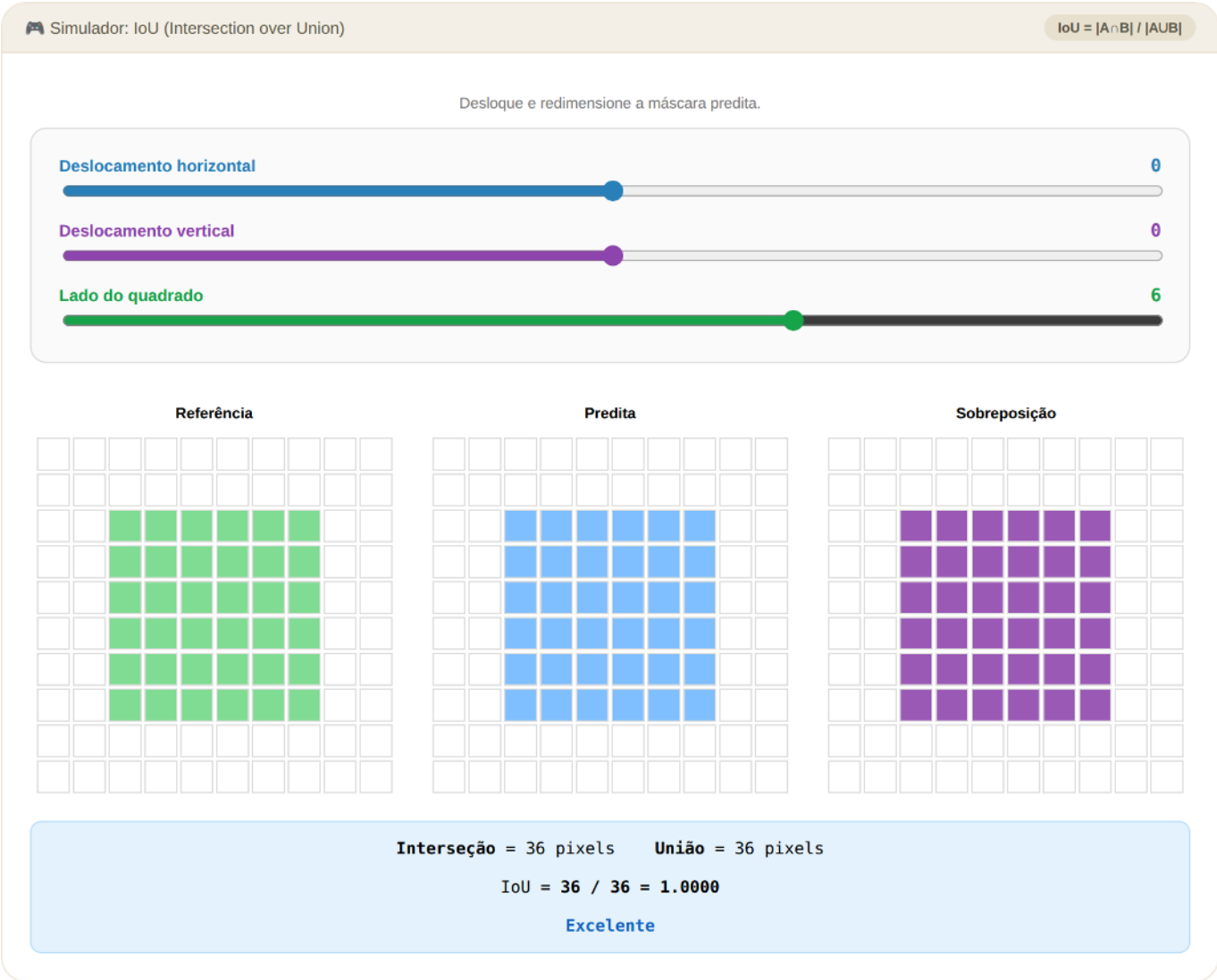


Figura 5.21: Simulador: IoU entre máscara de referência e máscara predita

```
%%writefile EP06_01.py
# Código Python
```

Overwriting EP06_01.py

```
TestSuite("EP06_01.py").run()
```

5.14.2 EP06_02 ● Filtro de Marcadores por Circularidade

Após a segmentação de uma imagem, é comum que diversos componentes conexos sejam identificados. Em aplicações como a retificação de documentos, apenas alguns desses componentes correspondem aos marcadores de referência utilizados para o alinhamento da imagem. Um critério frequentemente empregado para selecionar esses marcadores é a **circularidade**, que mede o quão próxima a forma de um componente está de um círculo.

Neste exercício, cada componente é descrito por sua área A e seu perímetro P . O objetivo é calcular sua circularidade e decidir, a partir de um limiar fornecido, se o componente deve ser aceito ou rejeitado como candidato a marcador.

5.14.2.1 📋 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro N (número de candidatos) e o limiar de circularidade C_{limiar} (número real).
2. **Dados dos candidatos:** Para cada um dos N candidatos, ler a área A (inteiro) e o perímetro P (número real).
3. **Circularidade:** Calcular $C = \frac{4\pi A}{P^2}$, em que:
 - A é a área do componente;
 - P é o perímetro do componente;
 - C é a circularidade.
4. **Caso degenerado:** Se $P = 0$, considerar $C = 0$ e classificar diretamente o candidato como REJEITADO.
5. **Classificação:** Se $C > C_{\text{limiar}}$, classificar o candidato como ACEITO; caso contrário, classificá-lo como REJEITADO.
6. **Arredondamento:** Exibir o valor de C com quatro casas decimais.
7. **Saída:** Para cada candidato, imprimir o valor de C seguido da classificação. Ao final, imprimir o número total de candidatos aceitos.

5.14.2.2 🚫 Restrições Computacionais

- Utilizar a constante π da biblioteca padrão da linguagem (por exemplo, `math.pi`), sem aproximações.
- A comparação deve ser realizada com o valor de C em precisão completa, antes do arredondamento para exibição.
- O critério de aceitação é estrito ($C > C_{\text{limiar}}$).
- Se $P = 0$, a divisão não deve ser realizada.

5.14.2.3 🧠 Fundamentação Teórica

A circularidade é um descritor geométrico definido por $C = \frac{4\pi A}{P^2}$, em que:

- A é a área do componente;
- P é o perímetro do componente;
- C é a circularidade.

Para um círculo perfeito, $C = 1$. À medida que a forma se torna mais alongada ou irregular, o perímetro cresce mais rapidamente que a área, reduzindo o valor de C .

Forma	Circularidade aproximada	Interpretação
Círculo	1,0000	Forma circular.
Quadrado	0,7854	Forma aproximadamente compacta.
Forma alongada ou irregular	$C \ll 1$	Baixa circularidade.

Forma	Circularidade aproximada	Interpretação
$P = 0$	0 (convenção adotada)	Contorno degenerado.

A circularidade é invariante à translação, à rotação e à escala, sendo amplamente utilizada para distinguir componentes aproximadamente circulares de outros formatos.

5.14.2.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro N .
- Linha 2: número real C_{limiar} .
- Próximas N linhas: área A (inteiro) e perímetro P (real), separados por espaço.

Saída:

- Uma linha para cada candidato, no formato **C ACEITO** ou **C REJEITADO**, com C apresentado com quatro casas decimais.
- Última linha: **Total aceitos: X**.

5.14.2.5 📌 Exemplos

Entrada	Saída	Observação
3	0.9941 ACEITO	Candidato aproximadamente circular, forma compacta e forma alongada.
0.6	0.7854 ACEITO	
78 31.4	0.1745 REJEITADO	
100 40	Total aceitos: 2	
50 60		
1	0.0000 REJEITADO	Perímetro nulo: contorno degenerado.
0.9	Total aceitos: 0	
10 0		



Figura 5.22: Simulador: Filtro de Marcadores por Circularidade

```
%%writefile EP06_02.py
# Código Python
```



```
Overwriting EP06_02.py
```

```
TestSuite("EP06_02.py").run()
```

5.14.3 EP06_03 ● Classificação de Marcações em Folhas de Resposta (OMR)

Após a retificação da folha e a segmentação dos quadros de respostas, o MCTest estima, para cada bolha, um **grau de preenchimento**, representado por um valor entre 0 e 100. A partir desses valores, o sistema deve determinar automaticamente a alternativa marcada, identificando também questões em branco e casos de múltiplas marcações.

Neste exercício, você implementará essa etapa de decisão do *pipeline* de OMR. A classificação depende de um limiar de preenchimento: pequenas variações nesse valor podem alterar o resultado da leitura automática.

5.14.3.1 📋 Diretrizes de Implementação

1. **Parâmetros:** Ler os inteiros Q (número de questões) e K (número de alternativas por questão, com $2 \leq K \leq 26$) e o limiar de preenchimento Th (número real entre 0 e 100).
2. **Graus de preenchimento:** Para cada uma das Q questões, ler os K valores reais correspondentes às alternativas A, B, C, ..., na ordem de entrada.
3. **Contagem de marcações:** Para cada questão, contar quantas alternativas possuem grau de preenchimento **estritamente maior** que Th .
4. **Classificação:**
 - Se nenhuma alternativa exceder Th , classificar a questão como **BRANCO**.
 - Se exatamente uma alternativa exceder Th , imprimir a letra correspondente (A, B, C, ...).
 - Se duas ou mais alternativas excederem Th , classificar a questão como **DUPLA_MARCACAO**.
5. **Saída por questão:** Imprimir, na ordem de leitura, a classificação de cada questão.
6. **Totais:** Ao final, imprimir o número de questões OK (uma única marcação), **BRANCO** e **DUPLA_MARCACAO**.

5.14.3.2 📌 Restrições Computacionais

- **Comparação estrita:** apenas valores maiores que Th são considerados marcações válidas; valores exatamente iguais ao limiar não devem ser contabilizados.
- **Letras das alternativas:** o índice 0 corresponde à alternativa A, o índice 1 à alternativa B e assim sucessivamente.
- **Múltiplas marcações:** sempre que duas ou mais alternativas excederem o limiar, a classificação deve ser **DUPLA_MARCACAO**, independentemente dos respectivos graus de preenchimento.

5.14.3.3 🧠 Fundamentação Teórica

Situação	Classificação	Interpretação
Exatamente uma alternativa acima do limiar	Letra da alternativa	Resposta válida
Nenhuma alternativa acima do limiar	BRANCO	Questão não respondida
Duas ou mais alternativas acima do limiar	DUPLA_MARCACAO	Resposta ambígua

O limiar de preenchimento controla a sensibilidade do algoritmo. Valores muito baixos tendem a aumentar o número de **DUPLA_MARCACAO**, enquanto valores muito altos podem aumentar a quantidade de questões classificadas como **BRANCO**.

5.14.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro Q .
- Linha 2: Inteiro K .
- Linha 3: Número real Th .
- Próximas Q linhas: K números reais, correspondentes aos graus de preenchimento das alternativas.
- Linha 1: Inteiro Q e K .

Saída:

- Q linhas, cada uma contendo a classificação da respectiva questão.
- Linha final: OK: x BRANCO: y DUPLA_MARCACAO: z .

5.14.3.5 Exemplos

Entrada	Saída	Observação
3	B	Na primeira questão apenas B supera o limiar; na segunda nenhuma alternativa o supera; na terceira, A e B excedem o limiar.
4	BRANCO	
50	DUPLA_MARCACAO	
10 85 5 12	OK: 1 BRANCO: 1	
20 15 18 22	DUPLA_MARCACAO: 1	
90 88 10 5		
1	BRANCO	Valores iguais ao limiar não são considerados marcações válidas.
2	OK: 0 BRANCO: 1	
50.0	DUPLA_MARCACAO: 0	
50 50		

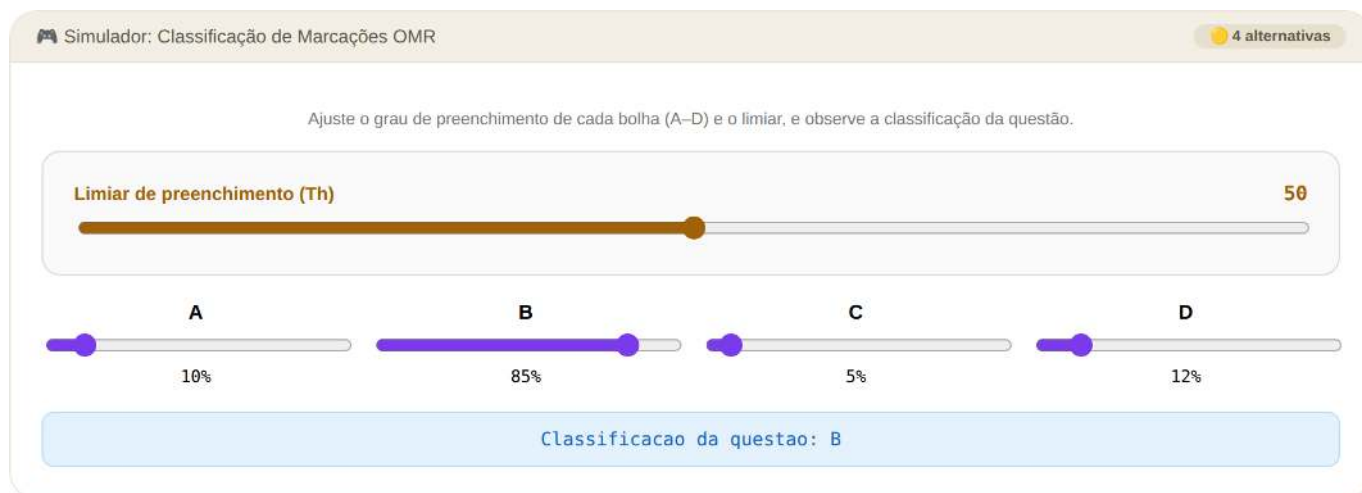


Figura 5.23: Simulador: Classificação de Marcação OMR

```
%%writefile EP06_03.py
# Código Python
```

Overwriting EP06_03.py

```
TestSuite("EP06_03.py").run()
```

5.14.4 EP06_04 Estimador de Inclinação por Mediana Angular (*Deskew*)

Após a detecção de bordas e a aplicação da Transformada de Hough, obtém-se um conjunto de retas candidatas à orientação predominante do documento. Cada reta fornece uma estimativa do ângulo de inclinação, calculada por

$$\text{ângulo} = \text{rad2deg}(\theta) - 90.$$

Entretanto, nem todas as retas correspondem às linhas do documento: algumas resultam de ruídos, sombras ou outros elementos da imagem. Neste exercício, você implementará a etapa de estimação robusta do ângulo de inclinação, filtrando os valores plausíveis e calculando sua mediana.

5.14.4.1 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro M , correspondente ao número de ângulos estimados.
2. **Ângulos:** Ler os M valores reais, em graus.
3. **Filtragem:** Manter apenas os ângulos que satisfaçam **estritamente** $-45 < \text{ângulo} < 45$.
4. **Ausência de candidatos:** Se nenhum ângulo permanecer após a filtragem, imprimir exatamente `SEM_CORRECAO`.
5. **Mediana:** Caso existam ângulos válidos:
 - se a quantidade for ímpar, a mediana é o elemento central da sequência ordenada;
 - se for par, a mediana é a média aritmética dos dois elementos centrais.
6. **Saída:** Imprimir a mediana arredondada para duas casas decimais (arredondamento padrão, *round half away from zero*, com `np.floor(img + 0.5)`).

5.14.4.2 Restrições Computacionais

- **Intervalo aberto:** ângulos iguais a -45 ou 45 não devem ser considerados.
- **Precisão:** calcular a mediana utilizando os valores originais; o arredondamento deve ser realizado apenas na saída.
- **Caso vazio:** se não houver ângulos válidos, nenhuma mediana deve ser calculada.

5.14.4.3 Fundamentação Teórica

Situação	Resultado
Maioria dos ângulos concentrada em torno da inclinação real	A mediana aproxima a orientação do documento.
Poucos ângulos discrepantes (<i>outliers</i>)	A mediana sofre pouca influência desses valores.
Ângulos fora do intervalo ($-45^\circ, 45^\circ$)	São descartados antes do cálculo.
Nenhum ângulo válido	Não é aplicada correção (<code>SEM_CORRECAO</code>).

A mediana é utilizada por ser mais robusta que a média na presença de poucos valores discrepantes, produzindo uma estimativa mais estável da inclinação predominante do documento.

5.14.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro M .
- Linha 2: M números reais, correspondentes aos ângulos em graus.

Saída:

- Uma única linha contendo o ângulo estimado, com duas casas decimais, ou a palavra `SEM_CORRECAO` caso nenhum ângulo seja válido.

5.14.4.5 📌 Exemplos

Entrada	Saída	Observação
5 -50 -10.5 2.3 2.3 47	2.30	Apenas os ângulos no intervalo $(-45, 45)$ são considerados; a mediana é 2,3.
4 -46 50 45 -45	SEM_CORRECAO	Nenhum ângulo pertence ao intervalo aberto $(-45, 45)$.

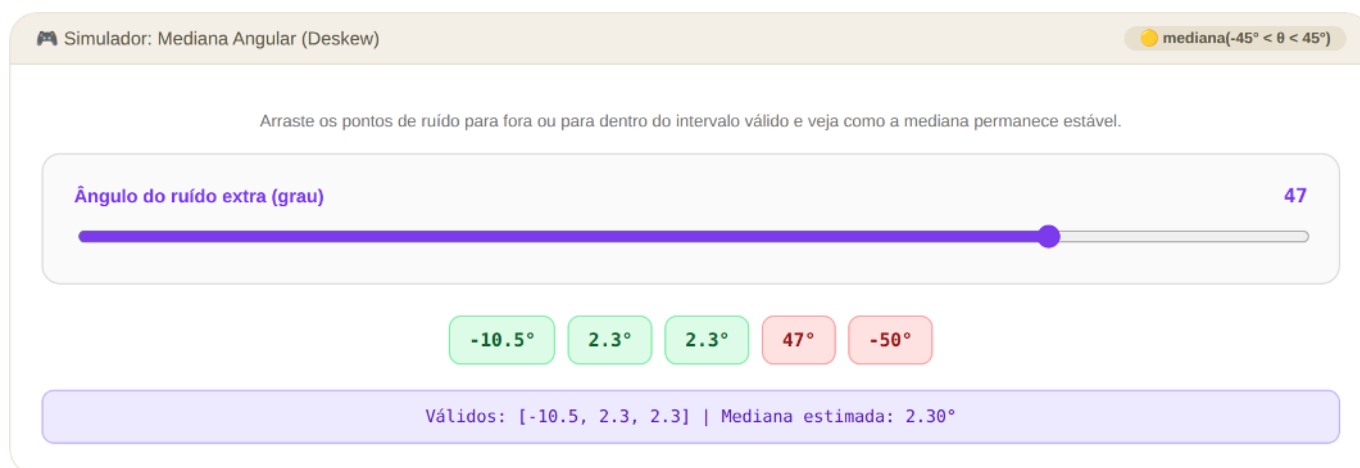


Figura 5.24: Simulador: Estimador de Inclinação por Mediana Angular

```
%%writefile EP06_04.py
# Código Python
```

Overwriting EP06_04.py

```
TestSuite("EP06_04.py").run()
```

5.14.5 EP06_05 ● Normalização de Fundo por Divisão (Correção de Iluminação)

Um formulário foi fotografado sob iluminação não uniforme, fazendo com que um lado da folha apareça mais claro que o outro. Nessas condições, a limiarização global por Otsu pode produzir resultados insatisfatórios, pois um único limiar não separa adequadamente texto e fundo em toda a imagem. A solução apresentada no capítulo consiste em **normalizar o fundo**, dividindo a imagem original por uma versão fortemente suavizada de si mesma, que representa a iluminação de baixa frequência.

Neste exercício, a imagem original e o fundo suavizado (equivalente ao resultado de um `cv2.GaussianBlur` com σ elevado) já são fornecidos. Sua tarefa é implementar a etapa de normalização que produz a imagem corrigida.

5.14.5.1 📌 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros L (linhas) e C (colunas).
2. **Imagem original:** Ler os $L \times C$ valores inteiros da matriz `img` (intensidades entre 0 e 255).
3. **Fundo estimado:** Ler os $L \times C$ valores inteiros da matriz `bg` (intensidades entre 0 e 255, sempre estritamente maiores que zero).
4. **Normalização:** Para cada posição (i, j) , calcular

$$\text{valor}(i, j) = \frac{\text{img}(i, j)}{\text{bg}(i, j)} \times 255.$$

5. **Arredondamento:** Arredondar o resultado para o inteiro mais próximo (*round half away from zero*, com `np.floor(img + 0.5)`).
6. **Saturação:** Limitar o valor obtido ao intervalo $[0, 255]$.
7. **Saída:** Imprimir a matriz `img_norm` resultante.

5.14.5.2 Restrições Computacionais

- **Divisão por zero:** a entrada garante $bg(i, j) > 0$ em todas as posições.
- **Ordem das operações:** primeiro arredondar, depois aplicar a saturação.
- **Processamento independente:** cada pixel deve ser normalizado individualmente, sem utilizar informações dos pixels vizinhos.

5.14.5.3 Fundamentação Teórica

Situação	Efeito da normalização
$img(i, j) = bg(i, j)$	Resultado igual a 255, correspondente ao fundo normalizado.
$img(i, j) < bg(i, j)$	Resultado menor que 255, preservando regiões mais escuras, como texto.
$img(i, j) > bg(i, j)$ Fundo com iluminação não uniforme	Resultado superior a 255, posteriormente saturado. A divisão reduz as variações lentas de iluminação, tornando a imagem mais homogênea.

A divisão pelo fundo estimado reduz os efeitos da iluminação não uniforme e preserva o contraste entre o primeiro plano e o fundo, facilitando as etapas posteriores de segmentação.

5.14.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Próximas L linhas: elementos da matriz `img`.
- Próximas L linhas: elementos da matriz `bg`.

Saída:

- Matriz `img_norm`, com L linhas e C colunas, contendo valores inteiros separados por espaço.

5.14.5.5 Exemplos

Entrada	Saída	Observação
2	153 255	Valores superiores a 255 devem ser saturados; $180/200 \times 255 = 229,5$ resulta em 230 após o arredondamento.
2	230 128	
60 120		
180 40		
100 100		
200 80		Apenas o primeiro valor permanece abaixo de 255 após a normalização.
1	128 255 255	
3		
30 60 90		
60 60 60		



Figura 5.25: Simulador: Normalização de Fundo por Divisão

```
%%writefile EP06_05.py
# Código Python
```

```
Overwriting EP06_05.py
```

```
TestSuite("EP06_05.py").run()
```

5.14.6 EP06_06 ● Mapa de Variância Local para Detecção de Textura

Uma fábrica de tecidos precisa inspecionar rolos de pano em tempo real, sem dispor de uma imagem de referência — cada rolo apresenta pequenas variações naturais. Nessa situação, a estratégia apresentada no capítulo consiste em analisar a **homogeneidade local da textura**: regiões uniformes apresentam baixa variância de intensidade em pequenas vizinhanças, enquanto riscos, manchas e falhas de fabricação produzem aumentos locais dessa variância.

Neste exercício, você implementará o núcleo desse método, calculando a variância local em uma janela deslizante e gerando uma máscara binária que identifica as regiões cuja variância excede um limiar.

5.14.6.1 📋 Diretrizes de Implementação

1. **Dimensões e parâmetros:** Ler os inteiros L , C , k (tamanho da janela, sempre ímpar) e T (limiar de variância).
2. **Imagem:** Ler os $L \times C$ valores inteiros da matriz de textura (intensidades entre 0 e 255).
3. **Tratamento das bordas:** Quando a janela ultrapassar os limites da imagem, utilizar **replicação de borda**, isto é, repetir o valor do pixel válido mais próximo.
4. **Média local:** Para cada posição (i, j) , calcular

$$\mu(i, j) = \frac{1}{k^2} \sum_{(p, q) \in \text{janela}} \text{textura}(p, q).$$

5. **Variância local:** Calcular a variância populacional da janela,

$$\sigma^2(i, j) = \frac{1}{k^2} \sum_{(p, q) \in \text{janela}} (\text{textura}(p, q) - \mu(i, j))^2,$$

ou, de forma equivalente,

$$\sigma^2(i, j) = \overline{x^2} - \mu(i, j)^2,$$

em que $\overline{x^2}$ representa a média dos quadrados das intensidades.

6. **Arredondamento:** Arredondar a variância para o inteiro mais próximo (*round half away from zero*, com `np.floor(res_norm + 0.5)`).
7. **Limiarização:** Definir $\text{máscara}(i, j) = 1$ se a variância arredondada for **estritamente maior** que T ; caso contrário, definir $\text{máscara}(i, j) = 0$.
8. **Saída:** Imprimir a máscara binária resultante.

5.14.6.2 Restrições Computacionais

- **Replicação de borda:** utilizar o valor do pixel válido mais próximo sempre que a janela ultrapassar os limites da imagem.
- **Variância populacional:** utilizar denominador k^2 , nunca $k^2 - 1$.
- **Comparação estrita:** a máscara deve ser calculada utilizando a condição $\sigma_{\text{arred}}^2 > T$.
- **Janela ímpar:** o valor de k é sempre ímpar, garantindo um pixel central.

5.14.6.3 Fundamentação Teórica

Situação	Variância local	Interpretação
Região uniforme	Baixa	Intensidades semelhantes na vizinhança.
Região contendo defeito	Alta	A presença de intensidades distintas aumenta a dispersão dos valores.
Janela pequena	Maior sensibilidade a detalhes e ruído	Detecta alterações localizadas.
Janela grande	Resposta mais suave	Evidencia defeitos maiores, porém reduz a precisão de sua localização.

A variância local mede a dispersão das intensidades em uma vizinhança. Regiões homogêneas apresentam baixa variância, enquanto alterações na textura aumentam essa medida, permitindo identificar possíveis defeitos por meio de uma simples limiarização.

5.14.6.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Inteiro k (ímpar).
- Linha 4: Inteiro T .
- Próximas L linhas: elementos inteiros da matriz de textura.

Saída:

- Máscara binária (valores 0 ou 1), com L linhas e C colunas.

5.14.6.5 Exemplos

Entrada	Saída	Observação
3	0 0 0	O defeito aumenta a variância em todas as janelas que o contêm.
3	1 1 1	
3	1 1 1	
50		
10 10 10		
10 10 10		A textura é uniforme; a variância é nula em toda a imagem.
10 90 10		
2	0 0	
2	0 0	
3		
5		
100 100		
100 100		

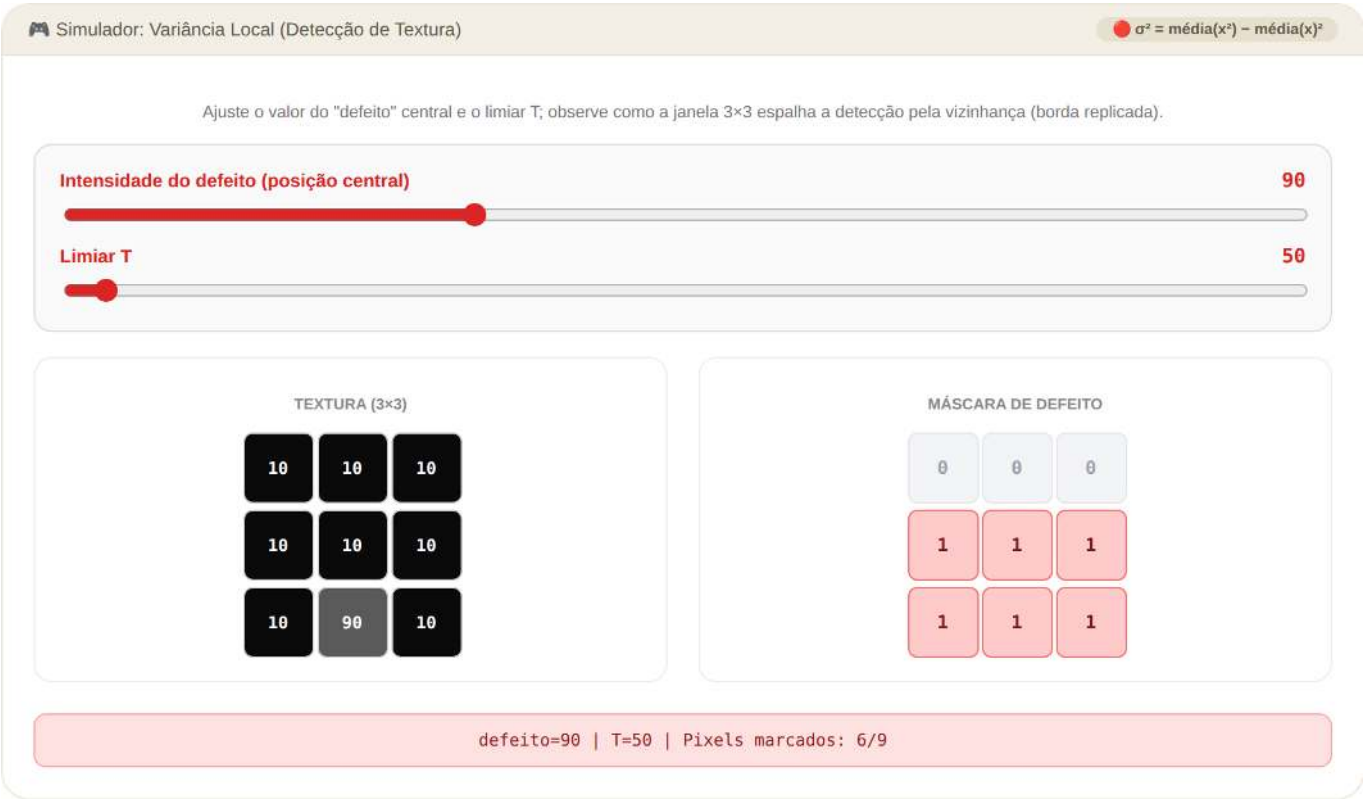


Figura 5.26: Simulador: Mapa de Variância Local para Detecção de Textura

```
%%writefile EP06_06.py
# Código Python

Overwriting EP06_06.py

TestSuite("EP06_06.py").run()
```

5.14.7 EP06_07 Pipeline de Inspeção Industrial: Registro por Translação e Subtração

Em uma linha de produção, uma câmera fixa fotografa cada peça que passa pela esteira, comparando-a a uma imagem de referência sem defeitos. O problema: pequenas vibrações da esteira deslocam a peça em relação à posição de referência a cada captura. Se a subtração de imagens for aplicada diretamente, sem

correção, o deslocamento por si só já gera diferenças enormes — **falsos positivos** que mascaram os defeitos reais.

Este é o exercício mais completo do capítulo: você deve **primeiro registrar** (alinhar geometricamente) a imagem capturada usando um deslocamento conhecido (dx, dy) , fornecido por um sensor de posição da esteira, e **só então aplicar a subtração** com limiarização, exatamente como descrito na seção de inspeção industrial.

5.14.7.1 Diretrizes de Implementação

1. **Dimensões e parâmetros:** Ler L, C (dimensões das imagens), o deslocamento inteiro conhecido dx, dy (podendo ser negativos) e o limiar de detecção T (inteiro).
2. **Imagens:** Ler a matriz de referência (**ref**, $L \times C$, sem defeitos) e a matriz capturada (**cap**, $L \times C$, possivelmente deslocada e com defeito).
3. **Registro por translação:** Construir a imagem alinhada **alin** aplicando o deslocamento (dx, dy) recebido:

$$\text{alin}(i, j) = \begin{cases} \text{cap}(i + dy, j + dx), & \text{se } (i + dy, j + dx) \in [0, L) \times [0, C) \\ 0, & \text{caso contrário} \end{cases}$$

4. **Preenchimento de borda:** As posições que “saem” da imagem capturada após o deslocamento recebem o valor **0** (*zero-padding* — fora do campo de visão da câmera; **note que este exercício usa zero, diferente da replicação de borda do EP06_06**).
5. **Diferença absoluta:** Calcular, pixel a pixel,

$$\text{diff}(i, j) = |\text{ref}(i, j) - \text{alin}(i, j)|$$

6. **Limiarização:** Definir $\text{mascara}(i, j) = 1$ se $\text{diff}(i, j) > T$; caso contrário, $\text{mascara}(i, j) = 0$.
7. **Saída:** Nesta ordem — (a) a matriz **alin** ($L \times C$); (b) a máscara de defeito ($L \times C$); (c) uma última linha com o total de pixels classificados como defeituosos.

5.14.7.2 Restrições Computacionais

- **Zero-padding, não replicação:** posições fora dos limites da imagem capturada, após o deslocamento, valem exatamente 0 — este é o ponto que mais diferencia este exercício do EP06_06.
- **Comparação estrita:** $\text{diff}(i, j) > T$.
- **Sinal de (dx, dy) :** o deslocamento pode ser positivo ou negativo; a fórmula do passo 3 deve ser aplicada literalmente, sem inverter os sinais.
- **Todos os valores são inteiros:** não há arredondamento nesta etapa.

5.14.7.3 Fundamentação Teórica

Etapa omitida	Consequência
Pular o registro geométrico	A borda inteira da imagem (introduzida pelo deslocamento) é marcada como “defeito” — falso positivo sistemático
Registro com (dx, dy) incorreto	Peça e referência ficam desalinhadas; a subtração detecta contornos deslocados, não defeitos reais
Limiar T muito baixo	Ruído de captura (variações de 1–2 níveis de cinza) é confundido com defeito
Limiar T muito alto	Defeitos sutis deixam de ser detectados

O registro geométrico e a subtração são etapas complementares: o primeiro garante que ambas as imagens representem exatamente a mesma cena no mesmo referencial espacial; o segundo isola o que realmente mudou entre elas — idealmente, apenas os defeitos.

5.14.7.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Linha 2: Inteiro C .
- Linha 3: Dois inteiros dx e dy , separados por espaço.
- Linha 4: Inteiro T .
- Próximas L linhas: elementos inteiros da matriz `ref`.
- Próximas L linhas: elementos inteiros da matriz `cap`.

Saída:

- L linhas com a matriz `alin`.
- L linhas com a máscara de defeito (0/1).
- Última linha: Total de pixels defeituosos: X .

5.14.7.5 Exemplos

Entrada	Saída	Observação
3	50 50 0	$dx = 1$ desloca a leitura uma coluna à direita; a última coluna de <code>alin</code> fica sem correspondência (vira 0) e é sistematicamente marcada; o defeito real (90) também é detectado.
3	50 90 0	
1 0	50 50 0	
30	0 0 1	
50 50 50	0 1 1	
50 50 50	0 0 1	
50 50 50	Total de pixels defeituosos: 4	
0 50 50		
0 50 90		
0 50 50		
2	10 10	Sem deslocamento ($dx = dy = 0$): <code>alin</code> é idêntica a <code>cap</code> ; apenas o defeito real (60) é detectado.
2	10 60	
0 0	0 0	
20	0 1	
10 10	Total de pixels defeituosos: 1	
10 10		
10 10		
10 60		

```
%%writefile EP06_07.py
# Código Python
```

```
Overwriting EP06_07.py
```

```
TestSuite("EP06_07.py").run()
```

5.14.8 EP06_08 Segmentação e Decodificação Real de *QRCode* com OpenCV

Nos exercícios anteriores, as grandezas intermediárias do *pipeline* de processamento de imagens — como áreas, perímetros, variâncias e deslocamentos — foram fornecidas diretamente ou calculadas a partir de matrizes numéricas, sem a necessidade de bibliotecas especializadas de Visão Computacional. Neste exercício de encerramento do capítulo, essa restrição é removida de forma intencional: será utilizada a biblioteca **OpenCV** (`cv2`) para localizar e decodificar um *QRCode* real presente em uma cena.

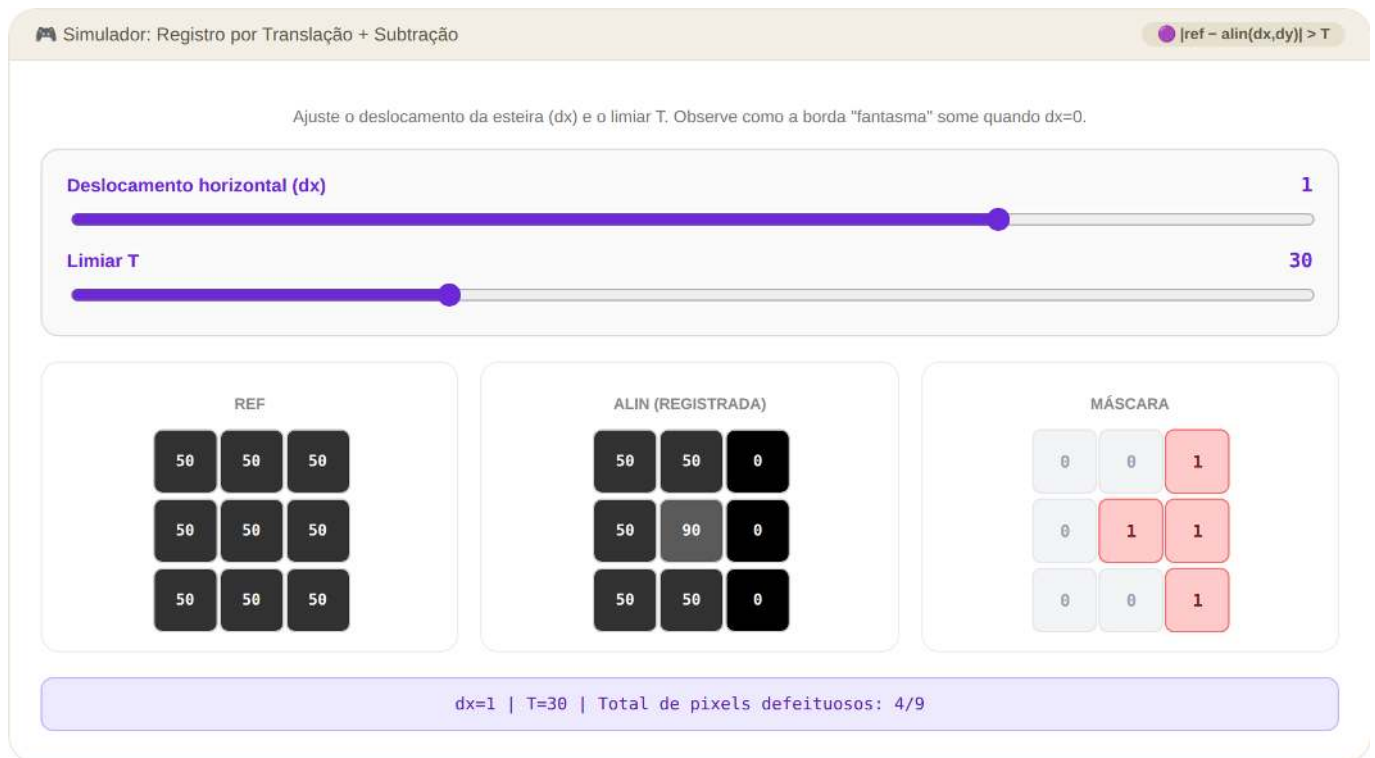


Figura 5.27: Simulador: *Pipeline* de Inspeção — Registro por Translação e Subtração

A proposta reproduz um fluxo simplificado de sistemas empregados em inspeção visual, automação industrial e leitura automática de documentos. Para manter a entrada de dados acessível ao contexto educacional, o carregamento da imagem será integrado à biblioteca didática `morph`, por meio da função `mm.readImg`.

A cena é fornecida no formato **PGM ASCII (P2)** e contém um único *QRCode* válido, além de diversos **objetos distratores**, como retângulos, regiões de ruído texturizado e blocos isolados. A segmentação baseada apenas em propriedades geométricas — como área e formato aproximadamente quadrado — é necessária para reduzir o espaço de busca, mas não é suficiente para identificar o código correto. A confirmação final será realizada exclusivamente pela tentativa de decodificação utilizando `cv2.QRCodeDetector`, procedimento compatível com aplicações reais de reconhecimento automático.

5.14.8.1 Diretrizes de Implementação

1. Leitura das dimensões e parâmetros

Ler, nesta ordem, por meio da entrada padrão:

- uma linha contendo o número de linhas L ;
- uma linha contendo o número de colunas C ;
- uma linha contendo os quatro parâmetros do algoritmo separados por espaço:
 - limiar de binarização T (inteiro);
 - área mínima $A_{\min}$ (inteiro);
 - tolerância de aspecto tol (real);
 - margem M (inteiro, em pixels).

2. Carregamento da imagem

Utilizar a função didática `f = mm.readImg(L, C)` para ler os $L \times C$ valores da imagem em tons de cinza, obtendo um *array* NumPy do tipo `uint8`.

3. Binarização

Aplicar limiarização binária invertida utilizando o limiar T . Todo pixel da imagem original com

intensidade estritamente maior que T deve ser convertido para 255, enquanto os demais devem assumir o valor 0.

4. Detecção de contornos

Extrair os componentes conectados externos utilizando `cv2.findContours(...)` com os parâmetros:

- `cv2.RETR_EXTERNAL`;
- `cv2.CHAIN_APPROX_SIMPLE`.

5. Filtragem geométrica

Para cada contorno encontrado:

- calcular o retângulo delimitador (x , y , w , h) por meio de `cv2.boundingRect`;
- manter apenas os candidatos que satisfaçam simultaneamente:

Área mínima

$$w \times h > A_{\min}$$

Razão de aspecto

$$\left| \frac{w}{h} - 1 \right| \leq \text{tol}$$

6. Ordenação dos candidatos

Ordenar os candidatos pela área do retângulo delimitador

$$w \times h$$

em ordem decrescente.

Em caso de empate, preservar a ordem originalmente retornada por `cv2.findContours`.

7. Verificação por decodificação

Para cada candidato, seguindo a ordem estabelecida:

- expandir o retângulo em M pixels nas quatro direções;
- limitar os índices para permanecerem dentro da imagem;
- extrair o recorte diretamente da imagem original f ;
- aplicar `cv2.QRCodeDetector().detectAndDecode(...)` sobre esse recorte.

8. Critério de parada

Interromper imediatamente o processamento quando o primeiro candidato produzir uma *string* decodificada não vazia.

9. Caso não encontrado

Se nenhum candidato for decodificado com sucesso, imprimir exatamente: `QRCODE_NAO_ENCONTRADO`

10. Saída (caso encontrado)

Imprimir duas linhas.

Primeira linha: `linha coluna altura largura` utilizando o retângulo delimitador **original**, antes da expansão pela margem M .

Segunda linha: `texto_decodificado`

5.14.8.2 Restrições Computacionais

- Utilizar funções do OpenCV para realizar a binarização, a detecção de contornos, o cálculo do retângulo delimitador e a decodificação do QRCode.
- A filtragem geométrica deve ocorrer obrigatoriamente antes da etapa de decodificação.
- Utilizar exclusivamente o limiar fixo T fornecido na entrada. Não é permitido utilizar métodos automáticos de limiarização, como Otsu ou limiarização adaptativa.
- Garantir que os recortes enviados ao decodificador permaneçam dentro dos limites da imagem.

5.14.8.3 Fundamentação Teórica

Etapa	Papel no pipeline	Consequência se omitida
Filtragem geométrica	Reduz o espaço de busca selecionando apenas regiões compatíveis com a geometria esperada de um QRCode.	O decodificador processaria todos os contornos, incluindo ruídos e objetos distratores.
Decodificação	Confirma semanticamente se o candidato contém um QRCode válido.	Objetos geometricamente semelhantes poderiam ser classificados incorretamente como QRCode.
Margem M	Preserva a <i>quiet zone</i> ao redor do código, facilitando sua detecção.	A ausência dessa margem pode impedir o alinhamento e a leitura correta do código.

Este exercício integra conceitos estudados ao longo do capítulo em um único *pipeline* de Visão Computacional. A segmentação reduz o conjunto de regiões candidatas por meio de características geométricas, enquanto a etapa de decodificação valida o conteúdo da região utilizando um algoritmo especializado de reconhecimento.

5.14.8.4 Especificação de Entrada e Saída (VPL)

Estrutura de Entrada

```
L
C
T A_min tol M
[matriz da imagem]
```

Estrutura de Saída (Sucesso)

```
linha coluna altura largura
texto_decodificado
```

Estrutura de Saída (Falha)

```
QRCODE_NAO_ENCONTRADO
```

5.14.8.5 Arquivos de Referência (.pgm)

Para fins de validação, depuração local e análise de matrizes reais de pixels, os arquivos de imagem gerados no padrão ASCII P2 encontram-se disponíveis no diretório do projeto. Você pode utilizá-los para testar em decodificados do seu celular a aderência do seu código (salvar *.pgm localmente para visualizar):

-  **Caso 1: Padrão Normal** – Contém um único código perfeitamente centralizado com distratores geométricos simples na periferia.
-  **Caso 2: Cenário Complexo** – Apresenta maior densidade de ruído texturizado e múltiplos distratores candidatos que testam os limites da filtragem por aspecto.

- 🏠 **Caso 3: Mensagem Expandida** – Contém um QRCode estruturado a partir de uma cadeia de caracteres de maior comprimento, gerando maior densidade de módulos internos.
- 🏠 **Caso 4: Geometria Compacta** – Avalia o comportamento do pipeline sob condições otimizadas de contraste e posicionamento limítrofe.
- 🏠 **Caso 5: Cenário de Exclusão** – Imagem composta puramente por elementos distratores de alta área, projetada para validar o comportamento de falha controlada do programa.

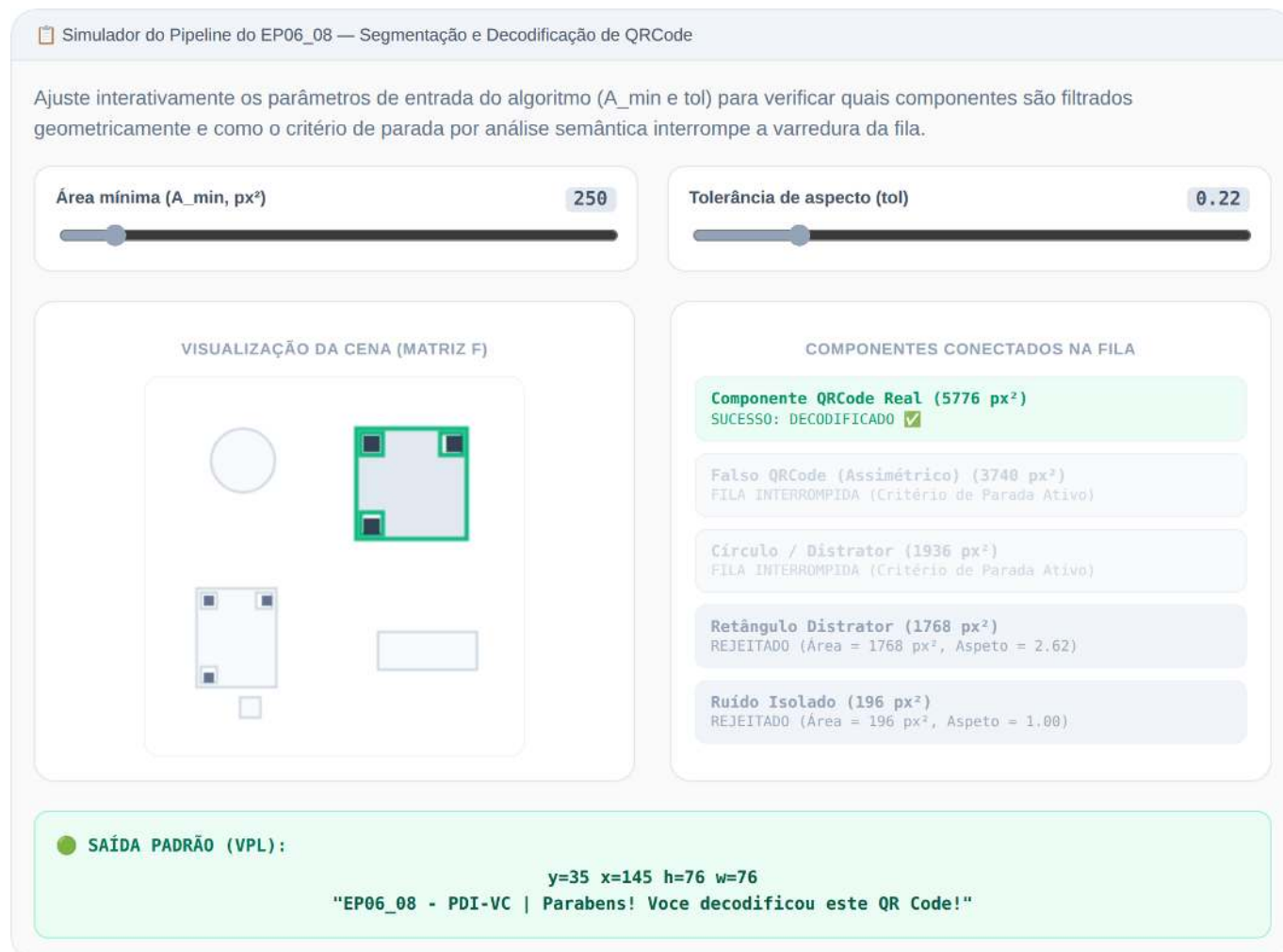


Figura 5.28: Simulador: Segmentação Geométrica + Verificação por Decodificação de QRCode

```
%%writefile EP06_08.py
# Código Python
```

```
Overwriting EP06_08.py
```

```
TestSuite("EP06_08.py").run()
```


Capítulo 6

Classificação de Imagens e Reconhecimento de Padrões



No **Capítulo 6**, a transição da Parte I para a Parte II foi apresentada por meio de duas aplicações que já exigiam decisões automatizadas: o reconhecimento de marcas em folhas de resposta (OMR) e a detecção de defeitos em inspeção industrial. Em ambos os casos, no entanto, as decisões dependiam de regras geométricas e de limiares definidos manualmente, como determinar se um disco era suficientemente circular ou se uma região era escura o bastante.

Este capítulo formaliza o problema mais geral subjacente a essas aplicações: dado um conjunto de exemplos rotulados, como treinar um sistema para **classificar automaticamente** novas imagens ou regiões de interesse? Essa questão está no núcleo do **Reconhecimento de Padrões**, disciplina que fundamenta grande parte das tarefas modernas de Visão Computacional, desde a classificação de imagens até a detecção de objetos e a segmentação semântica, exploradas nos próximos capítulos.

Serão estudados os principais **descritores clássicos de imagem** (cor, textura e forma/gradiente) e o classificador **k-Vizinhos mais Próximos** (*k-Nearest Neighbors* — k-NN), escolhido por sua simplicidade conceitual e por evidenciar, de forma direta, a relação entre o espaço de características, as métricas de distância e as fronteiras de decisão — conceitos que permanecem centrais mesmo em classificadores baseados em redes neurais profundas, estudados no capítulo final desta parte.

6.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- Compreender o **pipeline clássico de reconhecimento de padrões**: aquisição, pré-processamento, extração de descritores, classificação e avaliação;
- **Extraair e interpretar descritores clássicos** de cor, textura (*Local Binary Patterns* — LBP) e forma/gradiente (*Histogram of Oriented Gradients* — HOG);
- **Implementar e treinar um classificador k-NN** para tarefas de classificação de imagens;
- **Avaliar classificadores** por meio de métricas como acurácia, matriz de confusão, precisão e revocação;
- **Analisar o efeito do parâmetro k** e da dimensionalidade do espaço de características no desempenho do classificador;
- **Reconhecer as limitações dos descritores artesanais** (*hand-crafted features*) e compreender a motivação para a transição, nos próximos capítulos, para descritores aprendidos automaticamente.

6.2 Configuração do Ambiente

Os exemplos deste capítulo utilizam bibliotecas amplamente empregadas em Processamento Digital de Imagens, Visão Computacional e Aprendizado de Máquina. O bloco abaixo instala os pacotes necessários; em ambientes que já os possuam, a execução pode ser ignorada.

```

# Instala apenas bibliotecas ausentes
import importlib
import subprocess
import sys

PACOTES = {
    "cv2": "opencv-python",
    "skimage": "scikit-image",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "matplotlib": "matplotlib",
    "pandas": "pandas",
    "seaborn": "seaborn",
    "tabulate": "tabulate",
    "kaleido": "kaleido",
}

for modulo, pacote in PACOTES.items():
    try:
        importlib.import_module(modulo)
    except ImportError:
        subprocess.run(
            [sys.executable, "-m", "pip", "install", "-q", pacote],
            check=True
        )

# =====
# Bibliotecas
# =====

# Computação científica
import numpy as np
import pandas as pd

# Visualização
import matplotlib.cm as cm
import matplotlib.pyplot as plt
import seaborn as sns

# Visão computacional
import cv2
from skimage import data as skdata
from skimage.feature import hog, local_binary_pattern

# Aprendizado de máquina
from sklearn.datasets import load_digits, make_classification
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)

```

Assim como no capítulo anterior, será utilizado o módulo didático `morph.py`, responsável por padronizar a leitura, a exibição e o processamento de imagens ao longo do livro.

```
import os
import urllib.request

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

print(f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')}")
```

Ambiente pronto. morph 1.1.6

6.3 Um Problema Concreto: Classificação de Frutas

Antes de apresentar os fundamentos teóricos, considere o seguinte problema, que será utilizado como exemplo ao longo deste capítulo para ilustrar os principais conceitos de reconhecimento de padrões.

O cenário: Uma fazenda automatizada utiliza um sistema de Visão Computacional para separar maçãs, bananas e laranjas em linhas de embalagem.

O desafio: As frutas chegam à esteira em diferentes posições e orientações, sob condições de iluminação que podem variar. Além disso, folhas, sombras e pequenas oclusões podem dificultar sua identificação. Como desenvolver um sistema capaz de classificá-las corretamente?

Uma possível abordagem:

1. Extrair descritores que representem características relevantes das frutas:
 - **Cor:** distribuição predominante das cores;
 - **Textura:** diferenças na superfície da casca;
 - **Forma:** características geométricas do contorno.
2. Treinar um classificador utilizando exemplos previamente rotulados.
3. Utilizar o modelo treinado para classificar automaticamente novas frutas.

A Figure 6.1 ilustra, de forma conceitual, como diferentes frutas podem ser representadas em um espaço de características tridimensional.

💡 Reflita antes de continuar

Se cada fruta fosse representada apenas pelos valores de intensidade de seus pixels, seria possível distingui-las de forma confiável? Que tipos de informação poderiam ser extraídos da imagem para facilitar essa tarefa?

```
np.random.seed(42)

centros = {
    "Maçã": [0.8, 0.2, 0.9],
    "Banana": [0.3, 0.1, 0.2],
    "Laranja": [0.9, 0.8, 0.8],
}

fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(projection="3d")
```

```

for fruta, centro, cor, marcador in zip(
    centros,
    centros.values(),
    ["red", "gold", "orange"],
    ["o", "s", "^"],
):
    X = np.clip(np.random.normal(centro, 0.08, (70, 3)), 0, 1)
    ax.scatter(X[:, 0], X[:, 1], X[:, 2],
               c=cor, marker=marcador, s=35,
               alpha=0.7, label=fruta)

ax.set(
    xlim=(0,1), ylim=(0,1), zlim=(0,1),
    xlabel="Intensidade de cor",
    ylabel="Textura",
    zlabel="Forma",
    title="Espaço de Características"
)
ax.view_init(elev=25, azim=-60)
ax.legend(title="Frutas")
ax.zaxis.labelpad = 0.01

plt.tight_layout()
plt.show()

```

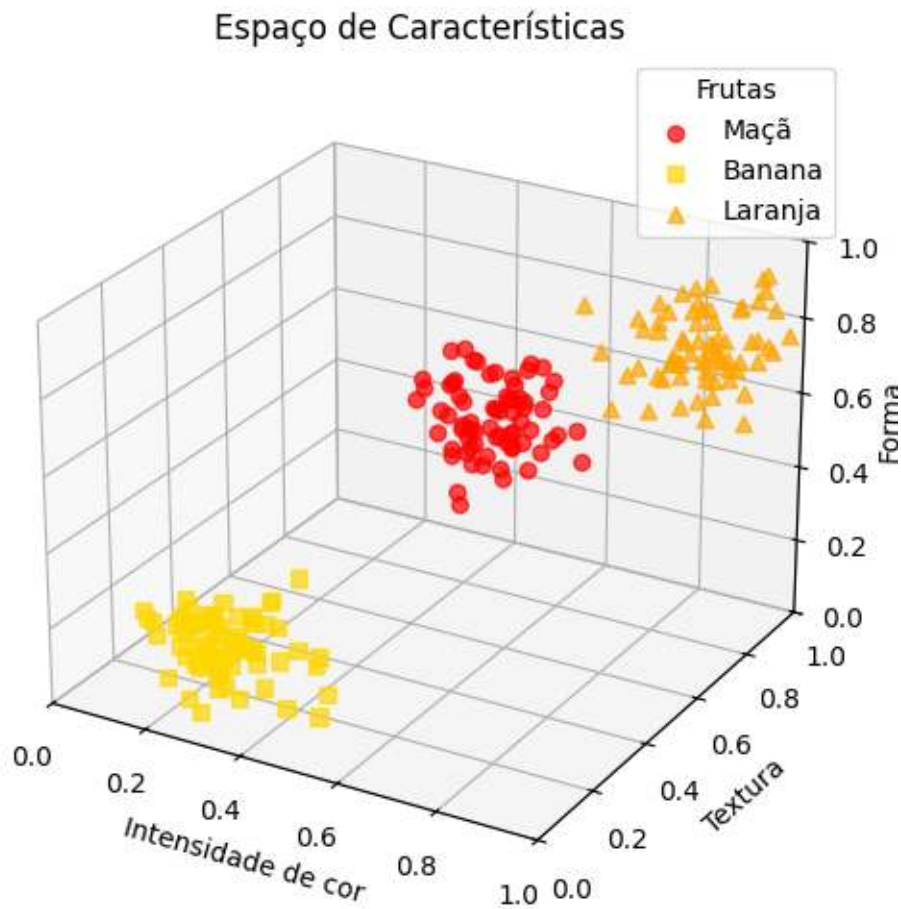
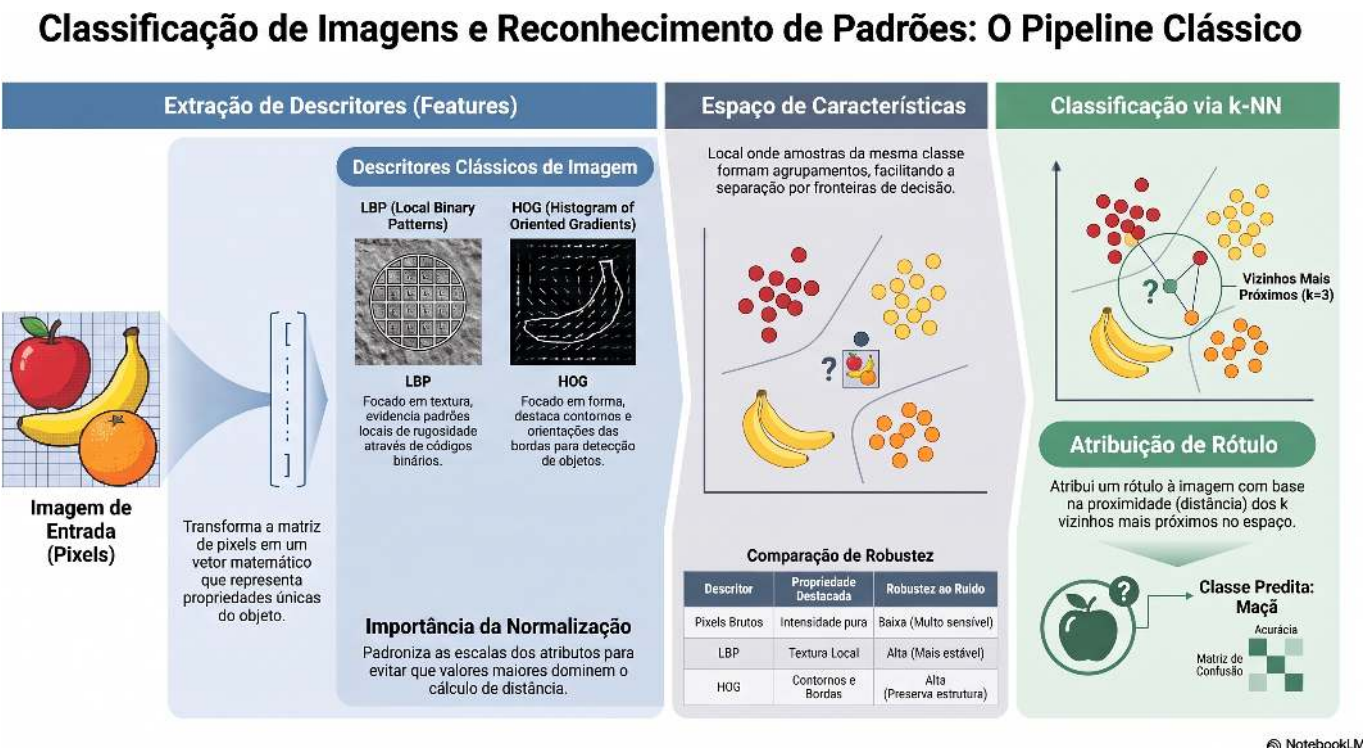


Figura 6.1: Exemplo motivacional: diferentes frutas formando agrupamentos distintos em um espaço de características.

6.4 Panorama do Capítulo: O *Pipeline* Clássico de Classificação de Imagens

Antes de prosseguir, é útil apresentar uma visão integrada do que será estudado. A Figure 7.1 mostra o fluxo geral de um sistema clássico de classificação de imagens, desde a imagem de entrada até a etapa de atribuição do rótulo final. Nas próximas seções, cada uma das etapas desse processo será estudada em detalhes.



4. **Classificação:** aplicação de um modelo que associa o vetor de características a uma classe;
5. **Avaliação:** análise do desempenho do modelo em um conjunto de dados independente daquele utilizado para o treinamento.

O conjunto de todos os vetores de características possíveis constitui o **espaço de características** (*feature space*). Um bom descritor produz representações que aproximam, nesse espaço, observações da mesma classe e afastam observações de classes distintas. Essa propriedade favorece métodos de classificação baseados em proximidade, como o **k-NN**, e também beneficia diversos outros classificadores.

A Figure 6.1 ilustra esse conceito de forma esquemática: cada fruta é representada por um ponto em um espaço de características de três dimensões (cor, textura e forma). Embora esse espaço seja apenas uma simplificação didática, ele mostra como amostras da mesma classe tendem a formar agrupamentos, enquanto classes diferentes ocupam regiões distintas, facilitando a tarefa de classificação.

6.6 Extração de Descritores Clássicos

Antes da popularização das redes neurais profundas, os descritores de imagem eram, em sua maioria, projetados manualmente por especialistas (*hand-crafted features*), com base em propriedades estatísticas ou geométricas conhecidas. Três famílias clássicas são particularmente relevantes:

- **Descritores de cor:** histogramas de intensidade ou de matiz, que capturam a distribuição dos valores de cor de uma região, já introduzidos no **Capítulo 3** por meio da função `mm.hist`;
- **Descritores de textura:** capturam padrões locais de repetição, rugosidade ou orientação, como o *Local Binary Patterns* (LBP), estudado a seguir;
- **Descritores de forma/gradiente:** descrevem a distribuição das bordas e das orientações do gradiente, como o *Histogram of Oriented Gradients* (HOG), amplamente empregado na detecção de pessoas e outros objetos.

Para comparar a informação capturada por cada abordagem, a Figure 6.3 mostra como diferentes técnicas “enxergam” a mesma imagem.

```
# Carregar imagem de exemplo
imagem = skdata.camera()

# Aplicar descritores
lbp_img = local_binary_pattern(
    imagem,
    P=8,
    R=1,
    method="uniform"
)

# Converter o LBP para RGB apenas para facilitar a visualização
lbp_norm = (lbp_img - lbp_img.min()) / (lbp_img.max() - lbp_img.min() + 1e-8)
lbp_rgb = (cm.nipy_spectral(lbp_norm)[..., :3] * 255).astype("uint8")

hog_features, hog_img = hog(
    imagem,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
)

# Exibição padronizada
mm.show(
    [imagem, lbp_rgb, hog_img],
    titles=[
```



```

    "Imagem Original\n(como o humano vê)",
    "LBP: Textura\n(cada cor = um código LBP)",
    "HOG: Gradientes e Contornos\n(regiões claras = maior intensidade)",
],
cols=3,
figsize=(12, 4),
)

print("Observe como cada descritor destaca propriedades diferentes:")
print("• LBP: evidencia padrões locais de textura.")
print("• HOG: evidencia contornos e orientações das bordas.")
print("• Imagem original: contém apenas os valores de intensidade.")

```

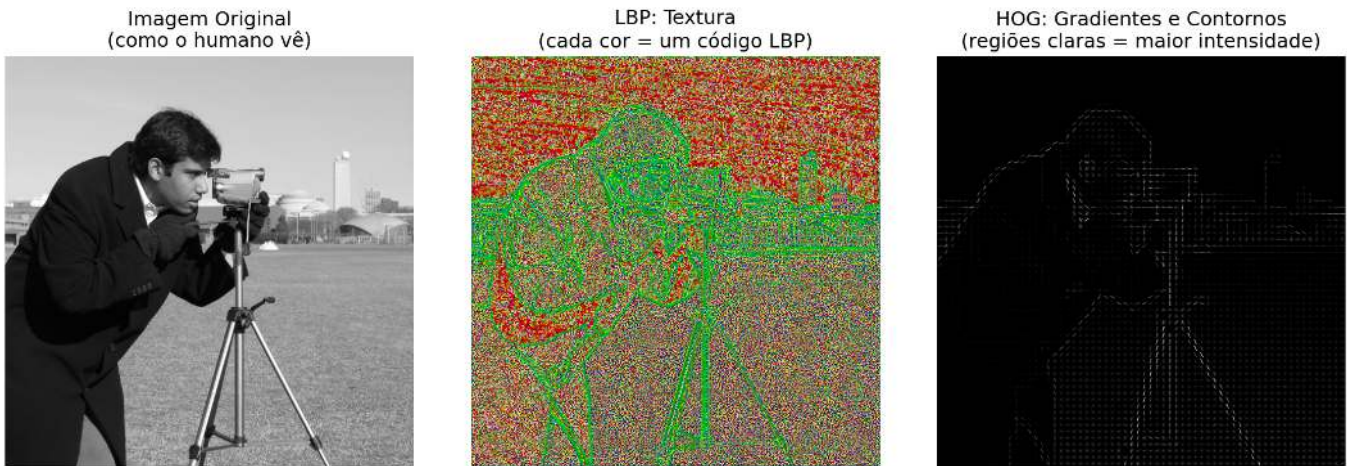


Figura 6.3: Comparação visual de diferentes descritores aplicados à mesma imagem. Cada descritor revela aspectos distintos da cena.

Observe como cada descritor destaca propriedades diferentes:

- LBP: evidencia padrões locais de textura.
- HOG: evidencia contornos e orientações das bordas.
- Imagem original: contém apenas os valores de intensidade.

6.6.1 Local Binary Patterns (LBP)

O LBP é um descritor de textura que codifica, para cada pixel central g_c , a relação entre sua intensidade e a dos P vizinhos dispostos em uma vizinhança circular de raio R :

$$\text{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p, \quad s(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

em que:

- (x_c, y_c) são as coordenadas do pixel central;
- g_c é a intensidade do pixel central;
- g_p é a intensidade do p -ésimo pixel vizinho;
- P é o número de vizinhos considerados;
- R é o raio da vizinhança circular;
- p é o índice do vizinho, com $p = 0, 1, \dots, P-1$;
- $s(z)$ é a função limiar definida na equação, em que $z = g_p - g_c$; ela assume valor 1 quando $z \geq 0$ e 0 quando $z < 0$;
- 2^p corresponde ao peso binário associado ao p -ésimo vizinho.

O código LBP obtido descreve o padrão local de contraste ao redor do pixel. O histograma desses códigos forma um vetor de características compacto para representar a textura da imagem (Figure 6.4). Neste capítulo utiliza-se a variante **uniforme**, que agrupa padrões não uniformes em uma única categoria, reduzindo a dimensionalidade e aumentando a robustez do descritor.

```
plt.figure(figsize=(6, 4))

plt.hist(
    lbp_img.ravel(),
    bins=np.arange(-0.5, lbp_img.max() + 1.5, 1),
    density=True,
    edgecolor="black",
)

plt.title("Histograma dos códigos LBP")
plt.xlabel("Código LBP")
plt.ylabel("Frequência relativa")
plt.xticks(range(int(lbp_img.max()) + 1))
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()
```

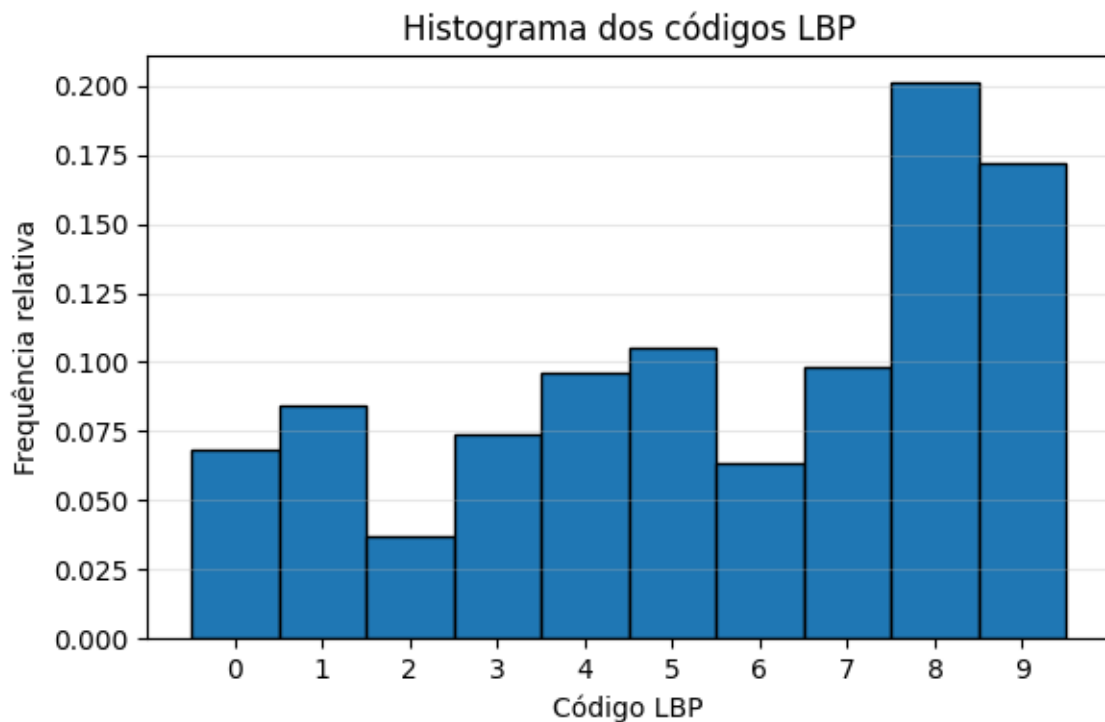


Figura 6.4: Histograma dos códigos LBP da imagem do cameramen. Cada barra representa a frequência relativa de um código LBP, formando o vetor de características utilizado para descrever a textura.

💡 Função `local_binary_pattern`

A implementação utilizada neste capítulo é fornecida pela biblioteca `scikit-image`:

```
local_binary_pattern(
    imagem,
    P=8,
    R=1,
    method="uniform"
)
```

onde:

- **image**: imagem em escala de cinza;
- **P**: número de vizinhos igualmente espaçados na vizinhança circular;
- **R**: raio da vizinhança, em pixels;
- **method**: estratégia de codificação. Neste capítulo utiliza-se o valor "uniform".

A equação apresentada anteriormente descreve o **LBP original**. Na implementação adotada neste capítulo, a opção **method="uniform"** calcula inicialmente esse código e, em seguida, remapeia os padrões não uniformes para uma única categoria, reduzindo a dimensionalidade do descritor e tornando-o mais robusto a pequenas variações locais.

A Figure 6.3 apresenta a representação visual do LBP, enquanto a Figure 6.4 mostra o histograma dos códigos LBP utilizado como vetor de características.

O **Projeto Prático 2** (seção **Comparação de Descritores para Classificação de Texturas**) emprega o LBP na classificação de diferentes tipos de textura sintética.

6.6.2 Histogram of Oriented Gradients (HOG)

O HOG é um descritor que representa a forma de um objeto por meio da distribuição das orientações do gradiente local. Assim como no operador de Canny (**Capítulo 6**), calcula-se inicialmente o gradiente:

$$|\nabla f(x, y)| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}, \quad \theta(x, y) = \text{atan2}\left(\frac{\partial f}{\partial y}, \frac{\partial f}{\partial x}\right).$$

em que:

- $f(x, y)$ é a intensidade da imagem no pixel (x, y) ;
- $\frac{\partial f}{\partial x}$ e $\frac{\partial f}{\partial y}$ são, respectivamente, as derivadas parciais da imagem nas direções horizontal e vertical;
- $|\nabla f(x, y)|$ é a magnitude do vetor gradiente no pixel (x, y) , indicando a intensidade da variação local da imagem;
- $\theta(x, y)$ é a orientação do vetor gradiente no pixel (x, y) , calculada pela função atan2 , cujo resultado pertence ao intervalo $(-\pi, \pi]$.

Embora $\theta(x, y)$, como calculado pela função atan2 , pertença ao intervalo $(-\pi, \pi]$, a implementação padrão do HOG utiliza o **gradiente não sinalizado** (*unsigned*): orientações opostas (por exemplo, 0 e π) são tratadas como equivalentes, e os ângulos são mapeados para o intervalo $[0, \pi)$ antes da construção do histograma. Essa escolha torna o descritor invariante à direção do contraste (por exemplo, uma borda clara-escura e uma borda escura-clara produzem a mesma orientação).

A imagem é então dividida em **células** (*cells*). Para cada célula, constrói-se um histograma das orientações do gradiente, ponderado pela magnitude correspondente. A concatenação dos histogramas de todas as células forma o vetor de características HOG, que representa a distribuição espacial das orientações do gradiente e captura informações sobre a forma e os contornos do objeto (Figure 6.5).

```
n = 100

plt.figure(figsize=(8, 3))
plt.bar(
    range(n),
```

```

hog_features[:n],
width=0.9
)

plt.title("Primeiros componentes do vetor HOG")
plt.xlabel(f"Índice do componente (0-{n-1}), de um total de {hog_features.shape[0]}")
plt.ylabel("Valor normalizado")
plt.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

```

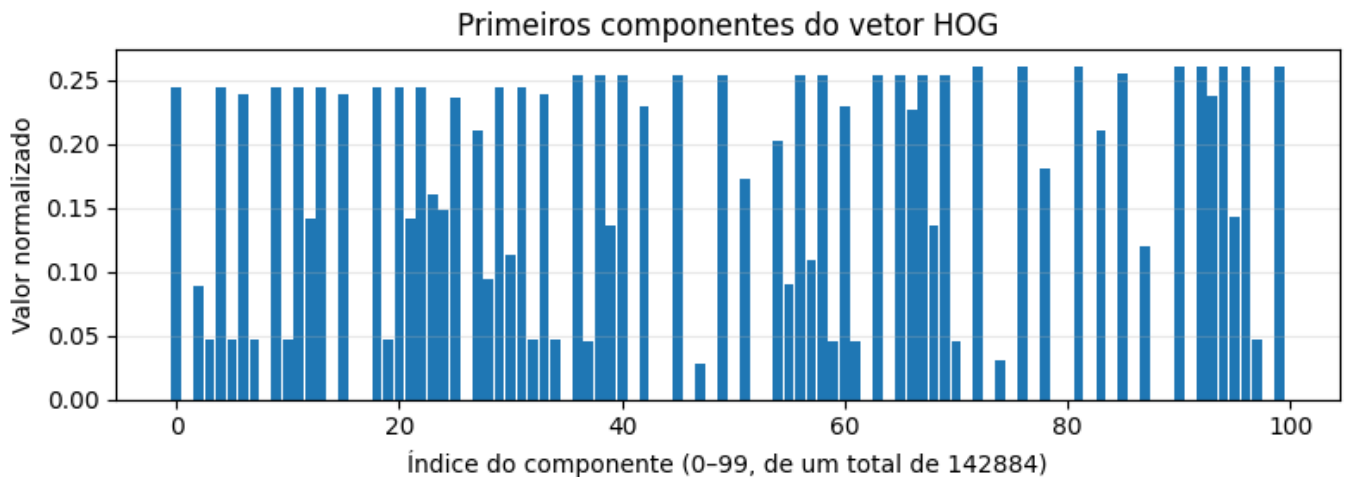


Figura 6.5: Primeiros 100 componentes do vetor de características HOG.

💡 Função hog

A extração do descritor HOG é realizada pela função:

```

hog(
    image,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
)

```

Os principais parâmetros são:

- **image**: imagem de entrada;
- **orientations**: número de divisões angulares do histograma de orientações em cada célula;
- **pixels_per_cell**: tamanho, em pixels, de cada célula onde o histograma é calculado;
- **cells_per_block**: número de células utilizadas na normalização do descritor;
- **visualize**: quando **True**, retorna também uma imagem ilustrando os gradientes utilizados pelo HOG.

A equação apresentada anteriormente descreve o cálculo da magnitude e da orientação do gradiente, que constituem a base do descritor HOG. Na implementação adotada neste capítulo, a função `hog()` utiliza essas informações para construir histogramas de orientações em cada célula da imagem e, em seguida, realiza a normalização em blocos (`cells_per_block`), reduzindo a sensibilidade do descritor a variações de iluminação e contraste.

A Figure 6.3 apresenta a representação visual do HOG, enquanto a Figure 6.5 ilustra os primeiros componentes do vetor de características extraído da imagem.

O **Projeto Prático 1** (seção **Classificação de Dígitos Manuscritos com k-NN**) compara o desempenho de descritores HOG com o uso direto das intensidades dos pixels como vetor de características.

6.6.3 O Impacto da Escala e a Normalização de Características

O classificador k -NN toma suas decisões com base na distância entre os vetores de características. Por isso, a escala de cada característica influencia diretamente o resultado da classificação. Se uma variável apresentar valores muito maiores do que as demais (por exemplo, uma intensidade de cor variando de 0 a 255, enquanto um índice de circularidade varia de 0 a 1), ela tende a dominar o cálculo da distância, reduzindo a influência dos outros descritores.

Para evitar esse problema, aplica-se uma etapa de **normalização das características**, geralmente por meio da padronização (*Z-score standardization*). Nesse procedimento, cada característica passa a ter média igual a zero e desvio-padrão igual a um, tornando comparáveis grandezas originalmente medidas em escalas diferentes.

A padronização é realizada pela transformação

$$z = \frac{x - \mu}{\sigma},$$

em que:

- x é o valor original da característica;
- μ é a média dessa característica calculada sobre o conjunto de treinamento;
- σ é o desvio-padrão da característica;
- z é o valor padronizado.

Após essa transformação, todas as características passam a possuir média igual a zero e desvio-padrão igual a um, permitindo que contribuam de forma equilibrada para o cálculo das distâncias.

Classe StandardScaler

A padronização utilizada neste capítulo é realizada pela classe `StandardScaler`, da biblioteca `scikit-learn`:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_norm = scaler.fit_transform(X)
```

em que:

- `StandardScaler()`: cria o objeto responsável pela padronização;
- `fit_transform(X)`: calcula a média e o desvio-padrão de cada característica do conjunto X e retorna a matriz padronizada.

Na prática, o método `fit_transform()` executa duas etapas: primeiro (`fit`), estima a média (μ) e o desvio-padrão (σ) de cada característica; em seguida (`transform`), aplica a transformação de padronização apresentada anteriormente a todos os valores da matriz de entrada.

A Figure 6.6 mostra o efeito da normalização. Visualmente, a distribuição dos pontos permanece a mesma; o que muda é a escala dos eixos. Sem a normalização, a característica de maior magnitude domina o cálculo das distâncias entre as amostras. Após a padronização, todas as características passam a contribuir de forma equilibrada para o cálculo das distâncias utilizadas pelo classificador k -NN.

```
# Dados sintéticos com escalas muito diferentes
np.random.seed(42)
X_demo = np.random.randn(20, 2) * [100, 1]
y_demo = np.array([0] * 10 + [1] * 10)
```

```

print("Efeito da normalização:")
print("  Característica 1: escala 100")
print("  Característica 2: escala 1")
print("\nSem normalização, a primeira característica domina o cálculo das distâncias.")
print("Com normalização, ambas contribuem de forma equilibrada.")
print("\nA normalização é essencial quando as características possuem escalas diferentes.")
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Sem normalização
axes[0].scatter(
    X_demo[y_demo == 0, 0], X_demo[y_demo == 0, 1],
    c="blue", label="Classe 0"
)
axes[0].scatter(
    X_demo[y_demo == 1, 0], X_demo[y_demo == 1, 1],
    c="red", label="Classe 1"
)
axes[0].set_title("Sem Normalização\n(escalas diferentes)")
axes[0].set_xlabel("Característica 1 (escala 100)")
axes[0].set_ylabel("Característica 2 (escala 1)")
axes[0].legend()

# Com normalização
scaler = StandardScaler()
X_norm = scaler.fit_transform(X_demo)

axes[1].scatter(
    X_norm[y_demo == 0, 0], X_norm[y_demo == 0, 1],
    c="blue", label="Classe 0"
)
axes[1].scatter(
    X_norm[y_demo == 1, 0], X_norm[y_demo == 1, 1],
    c="red", label="Classe 1"
)
axes[1].set_title("Com Normalização\n(características balanceadas)")
axes[1].set_xlabel("Característica 1")
axes[1].set_ylabel("Característica 2")
axes[1].legend()

plt.tight_layout()
plt.show()

```

Efeito da normalização:

Característica 1: escala 100
 Característica 2: escala 1

Sem normalização, a primeira característica domina o cálculo das distâncias.
 Com normalização, ambas contribuem de forma equilibrada.

A normalização é essencial quando as características possuem escalas diferentes.

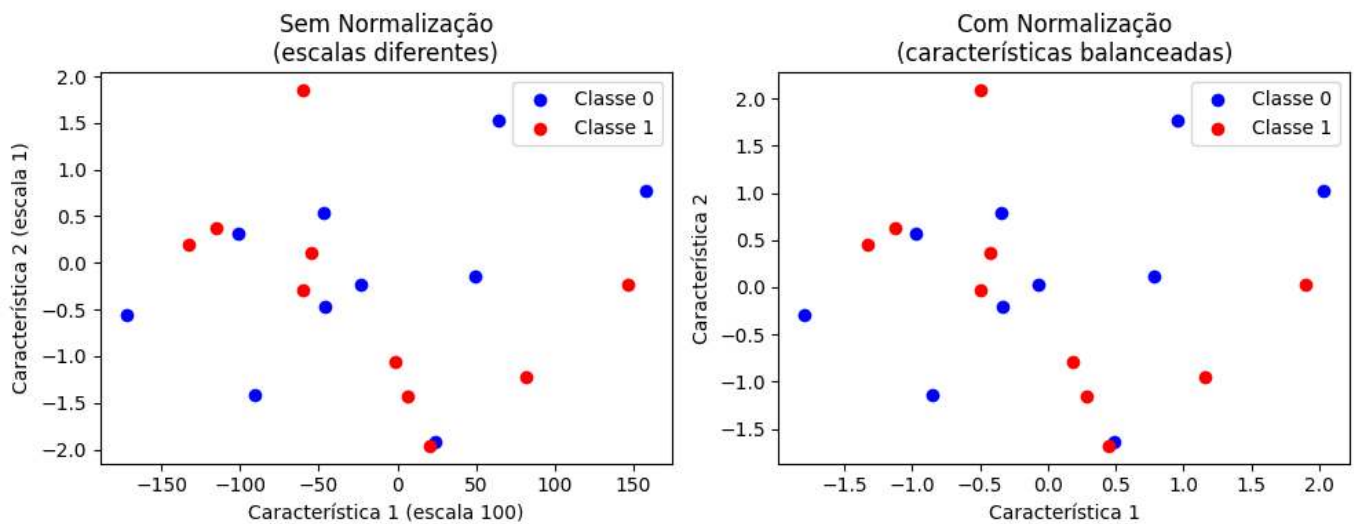


Figura 6.6: Importância da normalização das características para o classificador k-NN.

6.7 Mapa Conceitual

Até aqui, foram apresentados os descritores clássicos (LBP, HOG, pixels brutos) e a forma como eles organizam as amostras em um espaço de características. A Figure 6.7 sintetiza esse percurso e situa essas etapas dentro do fluxo geral de um sistema clássico de classificação de imagens, indicando também as etapas seguintes — classificação (k-NN) e avaliação dos resultados — que serão formalizadas nas próximas seções.

6.8 Descritores na Prática

Após conhecer os principais descritores clássicos, é natural perguntar como eles influenciam o desempenho de um classificador em situações próximas às encontradas na prática.

Nesta seção, compara-se o uso de três representações distintas das mesmas imagens: intensidades dos pixels, descritores LBP e descritores HOG. Para tornar o experimento mais realista, adiciona-se ruído sintético aos dados, em intensidades diferentes para cada descritor — uma forma simplificada de simular o fato de que, na prática, diferentes representações toleram de forma desigual as imperfeições da captura (ruído do sensor, pequenas variações de posição, etc.).

A Figure 6.8 apresenta as matrizes de confusão obtidas para cada descritor, permitindo identificar em quais classes ocorrem os principais erros de classificação. A interpretação dessas matrizes foi introduzida no **Capítulo 1**, quando foram apresentados os conceitos de **Verdadeiro Positivo (VP)**, **Falso Positivo (FP)**, **Verdadeiro Negativo (VN)** e **Falso Negativo (FN)**. Esses conceitos foram explorados nos **EPs 01_02** (métricas de classificação) e **01_03** (*mean Average Precision* – mAP), disponíveis em:

- https://fzampirolli.github.io/pdi-vc/eps/py.pt/EP01_02.html
- https://fzampirolli.github.io/pdi-vc/eps/py.pt/EP01_03.html

Neste capítulo, as matrizes de confusão são empregadas para analisar como diferentes descritores influenciam o desempenho do classificador.

Em seguida, a Figure 6.9 resume a acurácia global obtida por cada descritor.

Os resultados mostram que o desempenho do classificador depende diretamente da representação escolhida para descrever as imagens. Enquanto o uso direto das intensidades dos pixels é mais sensível às degradações introduzidas, os descritores LBP e HOG preservam melhor as informações relevantes para a classificação, resultando em maior desempenho nesse cenário. É importante ressaltar que os níveis de ruído aplicados a cada descritor foram escolhidos apenas para fins didáticos, de modo a ilustrar o princípio geral de que

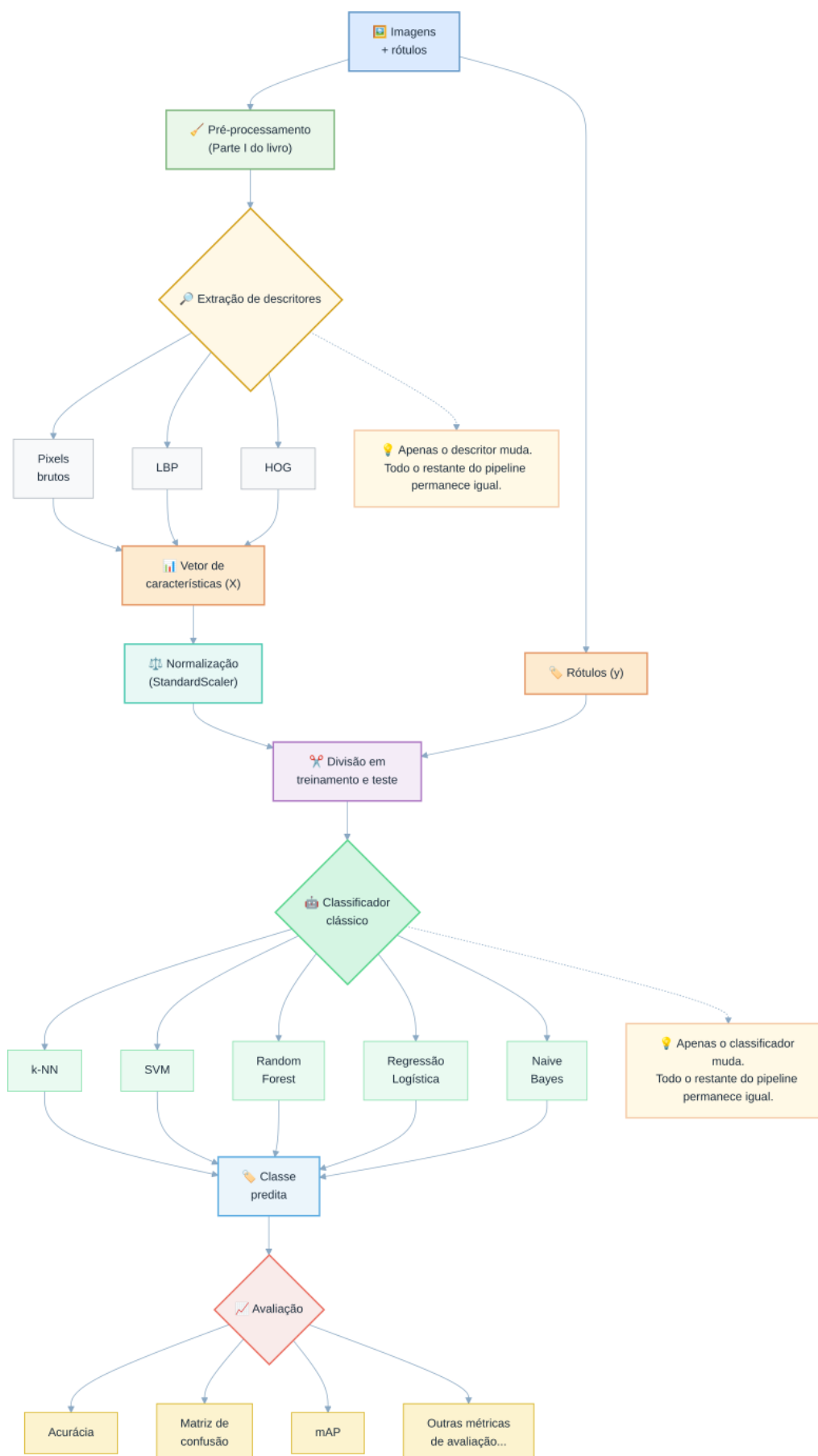


Figura 6.7: Mapa conceitual do processo de classificação de imagens utilizando descritores clássicos (LBP, HOG, pixels brutos) e classificador tradicional (k-NN). Não se aplica a modelos de *deep learning* (CNNs, YOLO), que aprendem *features end-to-end* diretamente dos pixels.

descritores mais elaborados *podem* ser mais robustos a degradações — o que não significa que essa relação se verifique sempre, como o estudo de caso da próxima seção demonstrará.

💡 Como o experimento é realizado

Como o objetivo desta seção é comparar apenas o efeito dos descritores, gera-se um conjunto de dados sintético simples: três nuvens de pontos gaussianas, centradas nos mesmos valores de “cor, textura e forma” já utilizados na Figure 6.1 — o mesmo padrão empregado desde o início do capítulo para representar as três classes de frutas.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
centros = {
    "Maçã": [0.8, 0.2, 0.9],
    "Banana": [0.3, 0.1, 0.2],
    "Laranja": [0.9, 0.8, 0.8],
}

n_por_classe = 100
X = np.vstack([
    np.random.normal(centro, 0.12, (n_por_classe, 3))
    for centro in centros.values()
])
y = np.repeat(list(centros.keys()), n_por_classe)
```

em que:

- `centros`: dicionário com o ponto médio de cada classe no espaço de características (cor, textura, forma);
- `n_por_classe`: número de amostras geradas por classe;
- `np.random.normal(centro, 0.12, (n_por_classe, 3))`: gera `n_por_classe` amostras ao redor de cada centro, com desvio-padrão 0,12 em cada dimensão;
- `np.repeat(list(centros.keys()), n_por_classe)`: gera o vetor de rótulos correspondente, na mesma ordem dos centros.

Em seguida, utiliza-se o fluxo de treinamento e avaliação:

- `train_test_split(X, y, test_size=0.3)`: divide os dados em treinamento (70%) e teste (30%);
- `KNeighborsClassifier(n_neighbors=5)`: cria um classificador k -NN com $k = 5$ vizinhos;
- `fit(X_train, y_train)`: ajusta o modelo aos dados de treinamento;
- `predict(X_test)`: classifica as amostras de teste;
- `accuracy_score(y_test, y_pred)`: calcula a acurácia;
- `confusion_matrix(y_test, y_pred)`: gera a matriz de confusão.

Neste experimento, o conjunto de treinamento, o classificador e o método de avaliação permanecem exatamente os mesmos. A única diferença entre os experimentos é a representação utilizada para cada imagem (pixels brutos, LBP ou HOG), permitindo avaliar exclusivamente a influência do descritor sobre o desempenho do classificador.

```
classes = ["Maçã", "Banana", "Laranja"]

np.random.seed(42)

# Mesmos centros de classe (cor, textura, forma) utilizados no
# exemplo anterior, agora reaproveitados para gerar os dados
# sintéticos de treino e teste deste experimento.
centros = {
```

```

    "Maçã": [0.8, 0.2, 0.9],
    "Banana": [0.3, 0.1, 0.2],
    "Laranja": [0.9, 0.8, 0.8],
}

n_por_classe = 100
X = np.vstack([
    np.random.normal(centro, 0.12, (n_por_classe, 3))
    for centro in centros.values()
])
y = np.repeat(list(centros.keys()), n_por_classe)

# Simular descritores com diferentes níveis de sensibilidade ao ruído.
# Quanto maior o ruído adicionado, pior tende a ser a representação.
descritores = {
    "Pixels Brutos": X + 0.5 * np.random.randn(*X.shape),
    "LBP": X + 0.3 * np.random.randn(*X.shape),
    "HOG": X + 0.2 * np.random.randn(*X.shape),
}

# Criar uma única figura com 3 subplots lado a lado para as matrizes
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
resultados = {}

for idx, (nome, Xd) in enumerate(descritores.items()):
    X_train, X_test, y_train, y_test = train_test_split(Xd, y, test_size=0.3, random_state=42)

    knn = KNeighborsClassifier(n_neighbors=5)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    resultados[nome] = acc

    # Matriz de confusão no subplot correspondente

    cm = confusion_matrix(y_test, y_pred, labels=classes)

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        cbar=False,
        xticklabels=classes,
        yticklabels=classes,
        ax=axes[idx],
    )

    axes[idx].set_title(
        f"{nome}\nAcurácia: {acc:.3f}",
        fontsize=11,
        fontweight="bold"
    )
    axes[idx].set_xlabel("Classe Predita")
    axes[idx].set_ylabel("Classe Real")

plt.tight_layout()
plt.show()

```

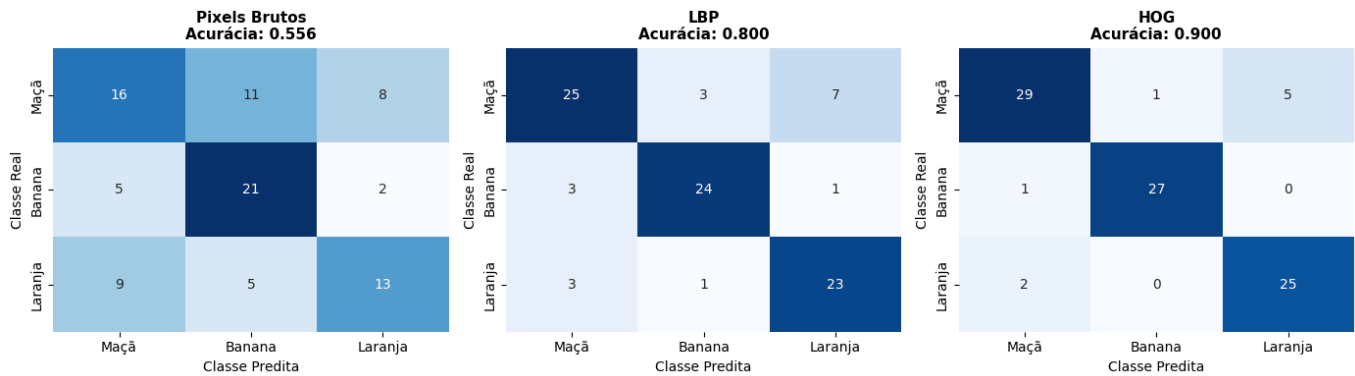


Figura 6.8: Matrizes de confusão obtidas pelo classificador k-NN utilizando três descritores diferentes. As linhas representam a classe real (Maçã, Banana e Laranja) e as colunas a classe predita. Quanto maior a concentração de valores na diagonal principal, melhor o desempenho do descritor.

```
# Comparação visual em uma figura isolada
plt.figure(figsize=(6, 3.5))
nomes = list(resultados.keys())
acuracia = list(resultados.values())
colors = ['#6366f1', '#f97316', '#22c55e']

bars = plt.bar(nomes, acuracia, color=colors, width=0.5)
plt.ylabel('Acurácia Global')
plt.title('Desempenho Geral dos Descritores sob Ruído Realista', fontsize=12, fontweight='bold')
plt.ylim(0.5, 1.0)
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Adicionar os valores acima das barras usando round padrão para exibição
for bar, val in zip(bars, acuracia):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{val:.3f}', ha='center', fontweight='bold', fontsize=10)

plt.tight_layout()
plt.show()

print("Análise dos resultados:")
print("- Pixels Brutos: Sensíveis a variações locais de iluminação e ruído.")
print("- LBP: Boa tolerância para variações monotônicas de iluminação global.")
print("- HOG: Excelente para contornos e formas estáveis sob pequenas flutuações geométricas.")
```

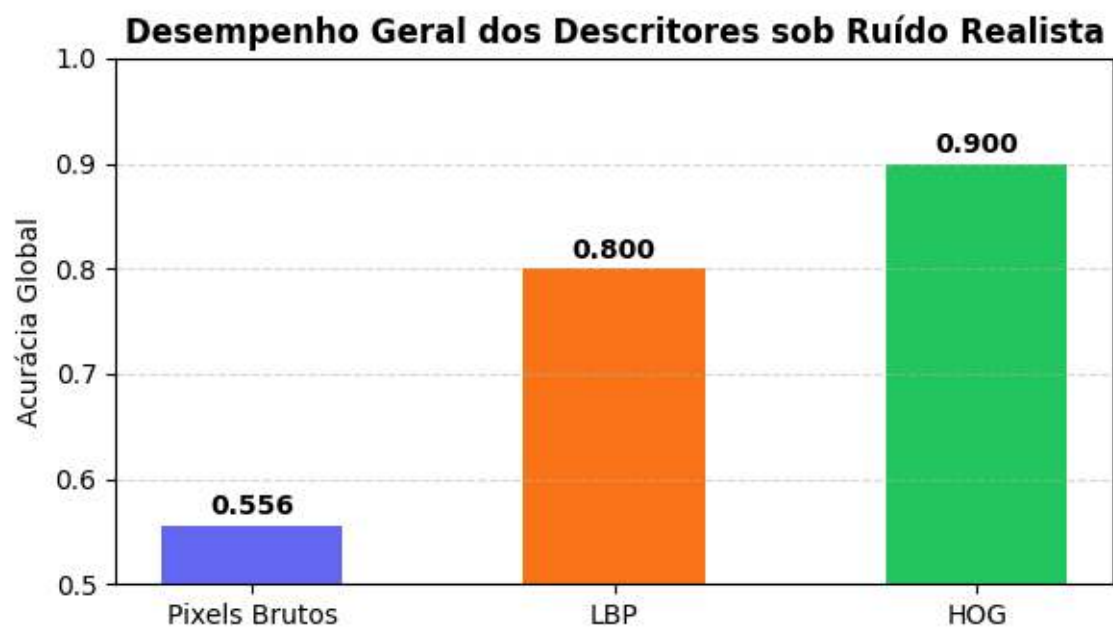


Figura 6.9: Comparação detalhada de acurácia global em cenário realista. Observe como os descritores extraídos estruturalmente (LBP e HOG) superam o uso de intensidades puras de pixels brutos.

Análise dos resultados:

- Pixels Brutos: Sensíveis a variações locais de iluminação e ruído.
- LBP: Boa tolerância para variações monotônicas de iluminação global.
- HOG: Excelente para contornos e formas estáveis sob pequenas flutuações geométricas.

6.9 Um Problema Concreto: Simulando Descritores de Frutas

Retomando o problema de classificação de frutas apresentado no início do capítulo, cada imagem pode ser representada por um vetor de características (*feature vector*) obtido a partir da extração de descritores de cor, textura e forma. A Table 6.1 apresenta alguns descritores frequentemente utilizados em aplicações de Visão Computacional, incluindo técnicas introduzidas no Capítulo 3 e neste capítulo.

Tabela 6.1: Conjunto de descritores cromáticos, texturais e geométricos utilizados para representar imagens de frutas.

Característica	Descrição
R, G, B	intensidade média dos canais vermelho, verde e azul
NC	intensidade média em níveis de cinza (<i>grayscale</i>)
LBP	descritor de textura (<i>Local Binary Pattern</i>)
HOG	descritor de forma (<i>Histogram of Oriented Gradients</i>)
Área	número de pixels do objeto
Perímetro	comprimento do contorno
Circularidade	medida de quão circular é o objeto
Razão largura/altura	proporção entre largura e altura da região

Nesse exemplo, cada imagem é representada pelo vetor

$$X = (R, G, B, NC, LBP, HOG, \text{Área}, \text{Perímetro}, \text{Circularidade}, \text{Razão}).$$

i Simplificação adotada nesta tabela

Na prática, LBP e HOG não são valores escalares, mas histogramas com dezenas ou centenas de componentes. Nesta seção, cada um deles é representado por um único valor apenas para simplificar a apresentação. Em aplicações reais, essas posições seriam substituídas pelas componentes completas dos respectivos histogramas.

Em aplicações reais, nem todos os descritores contribuem igualmente para distinguir as classes. Alguns fornecem informações mais relevantes, enquanto outros podem ser redundantes ou pouco discriminativos.

Para reproduzir esse cenário de forma controlada, será utilizado o `make_classification()`, da biblioteca `scikit-learn`. A função gera um conjunto de dados sintético cujas características podem ser interpretadas como descritores de imagens, permitindo definir quantas delas serão informativas para a classificação.

Neste exemplo, são geradas dez características sintéticas, das quais apenas sete (`n_informative=7`) participam da separação entre as três classes. As demais simulam atributos pouco informativos ou redundantes. A Figure 6.10 apresenta uma representação conceitual desse processo.

Antes de introduzir o algoritmo que será estudado em detalhe neste capítulo, cabe uma observação: para identificar, entre as dez características sintéticas, quais são mais discriminativas — e assim selecionar duas delas para a visualização em 2D —, utiliza-se um *Random Forest* apenas como ferramenta auxiliar de diagnóstico. O KNN, foco deste capítulo, é apresentado a seguir.

```
# Configuração para reprodução
np.random.seed(42)

# Geração de dados sintéticos com características controladas
X, y = make_classification(
    n_samples=300,
    n_features=10,
    n_informative=7,
    n_redundant=2,
    n_repeated=1,          # Uma característica é cópia de outra
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42,
)

# Criar figura com dois subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Subplot 1: Visualização das classes em 2D (usando duas características informativas)
# Identificar quais características são mais informativas
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)
importancias = rf.feature_importances_
caracteristicas_informativas = np.argsort(importancias)[-2:] # Duas mais importantes

cores = {0: "#e74c3c", 1: "#f1c40f", 2: "#e67e22"} # Maçã, Banana, Laranja
rotulos = {0: "Maçã", 1: "Banana", 2: "Laranja"}

for classe in range(3):
    idx = y == classe
    ax1.scatter(X[idx, caracteristicas_informativas[0]],
                X[idx, caracteristicas_informativas[1]],
                c=cores[classe], label=rotulos[classe],
                alpha=0.6, s=50, edgecolors="white", linewidth=0.5)

ax1.set_xlabel(f"Característica {caracteristicas_informativas[0]+1} (informativa)", fontsize=11)
```

```

ax1.set_ylabel(f"Característica {caracteristicas_informativas[1]+1} (informativa)", fontsize=11)
ax1.set_title("Classes no Espaço de Características\n(2 características informativas)",
              fontsize=12, fontweight="bold")
ax1.legend(loc="upper right")
ax1.grid(alpha=0.3)

# Subplot 2: Importância das características
bars = ax2.bar(range(1, 11), importancias, color="#4a90d9", alpha=0.7)
ax2.set_xlabel("Índice da Característica", fontsize=11)
ax2.set_ylabel("Importância", fontsize=11)
ax2.set_title("Importância de cada Característica\npara a Classificação",
              fontsize=12, fontweight="bold")
ax2.set_xticks(range(1, 11))
ax2.grid(axis="y", alpha=0.3)

# Colorir barras para destacar características
cores_barras = ["#e74c3c" if i < 7 else "#95a5a6" for i in range(10)]
for bar, cor in zip(bars, cores_barras):
    bar.set_color(cor)

# Adicionar legenda
from matplotlib.patches import Patch
legenda_elements = [
    Patch(facecolor="#e74c3c", label="Características Informativas (7)"),
    Patch(facecolor="#95a5a6", label="Características Redundantes (3)")
]
ax2.legend(handles=legenda_elements, loc="upper right")

# Anotar o número de características informativas
ax2.axhline(y=0.15, color="red", linestyle="--", alpha=0.3)
ax2.text(0.5, 0.17, "Limiar de importância", fontsize=9, color="red", alpha=0.7)

plt.tight_layout()
plt.show()

print("\n Análise dos dados gerados:")
print(f" • Total de amostras: {X.shape[0]}")
print(f" • Número de características: {X.shape[1]}")
print(f" • Características informativas: 7 (colunas 1 a 7 do gráfico)")
print(f" • Características redundantes: 2 (colunas 8 e 9)")
print(f" • Características repetidas: 1 (coluna 10)")
print(f" • Distribuição das classes: {np.bincount(y)}")

```

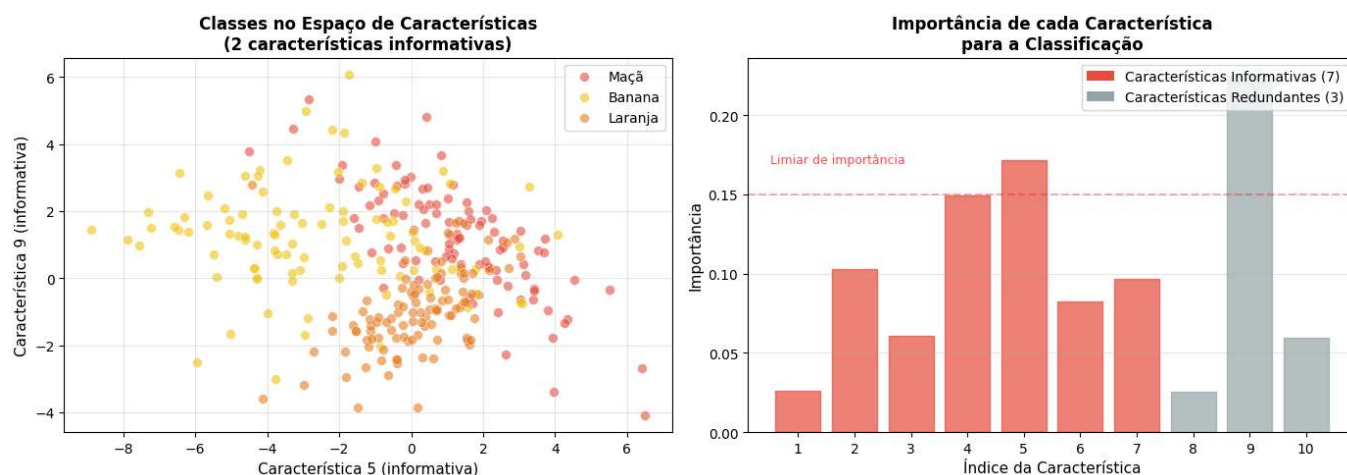


Figura 6.10: Ilustração do processo de geração de dados sintéticos com *make_classification*. À esquerda, visualização das três classes em um espaço bidimensional formado por duas características informativas. À direita, importância relativa de cada característica para a classificação, evidenciando que apenas 7 das 10 características são efetivamente discriminativas, enquanto as demais são redundantes (2) ou repetidas (1).

Análise dos dados gerados:

- Total de amostras: 300
- Número de características: 10
- Características informativas: 7 (colunas 1 a 7 do gráfico)
- Características redundantes: 2 (colunas 8 e 9)
- Características repetidas: 1 (coluna 10)
- Distribuição das classes: [101 98 101]

6.10 Classificador k-NN: Como Funciona por Dentro

O **k-Nearest Neighbors (k-NN)** é um dos algoritmos de classificação mais simples e intuitivos da Aprendizagem de Máquina. Diferentemente de muitos classificadores, ele não constrói explicitamente um modelo durante a etapa de treinamento. Em vez disso, armazena as amostras rotuladas e, quando uma nova amostra precisa ser classificada, procura aquelas que mais se assemelham a ela.

O princípio do algoritmo baseia-se na hipótese de que amostras com características semelhantes tendem a pertencer à mesma classe. Para quantificar essa proximidade, o k-NN utiliza uma medida de distância entre os vetores de características.

Como exemplo, considere a Table 6.2, que apresenta uma versão simplificada do problema de classificação de frutas utilizando apenas duas características: intensidade de cor e circularidade, ambas normalizadas no intervalo de 0 a 1.

Tabela 6.2: Exemplo simplificado de classificação de frutas utilizando duas características normalizadas.

Amostra	Cor	Circularidade	Classe
Fruta 1	0,82	0,88	Maçã
Fruta 2	0,30	0,20	Banana
Fruta 3	0,88	0,85	Maçã
Fruta ? (teste)	0,80	0,90	?

Observando apenas essas duas características, percebe-se que a fruta de teste está muito mais próxima das amostras rotuladas como **Maçã** do que da amostra rotulada como **Banana**. Na seção seguinte, essa noção intuitiva de proximidade será formalizada por meio de uma métrica de distância, utilizada pelo algoritmo para identificar os vizinhos mais próximos e decidir a classe da nova amostra.

6.10.1 Métrica de Distância

A proximidade entre duas amostras é normalmente quantificada pela **distância euclidiana**, definida por

$$d(x, x_i) = \|x - x_i\|_2 = \sqrt{\sum_{j=1}^n (x_j - x_{i,j})^2},$$

em que:

- x é a amostra de teste;
- x_i é uma amostra do conjunto de treinamento;
- n é o número de características;
- x_j e $x_{i,j}$ representam a j -ésima característica.

Na implementação deste capítulo, x corresponde a uma linha de `X_test` e x_i a uma linha de `X_train`. O método `predict()` calcula automaticamente a distância entre x e todas as amostras de treinamento.

No exemplo da Table 6.2:

$$d(\text{teste}, \text{Fruta 1}) \approx 0,028, \quad d(\text{teste}, \text{Fruta 2}) \approx 0,860, \quad d(\text{teste}, \text{Fruta 3}) \approx 0,094.$$

Como as menores distâncias correspondem às Frutas 1 e 3, essas amostras serão utilizadas na etapa de decisão.

6.10.2 Regra de Decisão

Após ordenar as distâncias, o algoritmo seleciona os k vizinhos mais próximos. Seja $N_k(x)$ esse conjunto. A classe predita é dada por

$$\hat{y} = \text{moda}\{y_i : x_i \in N_k(x)\},$$

em que y_i é o rótulo da amostra x_i e $\hat{y}$ é a classe atribuída à amostra de teste.

No exemplo, para $k = 3$, os vizinhos são Fruta 1 (Maçã), Fruta 3 (Maçã) e Fruta 2 (Banana). Como **Maçã** recebe dois votos, essa é a classe predita.

Classe KNeighborsClassifier

Neste capítulo, o algoritmo é implementado com a classe `KNeighborsClassifier`, da biblioteca `scikit-learn`:

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
```

em que:

- `KNeighborsClassifier(n_neighbors=3)`: define o valor de k ;
- `fit(X_train, y_train)`: armazena as amostras de treinamento (`X_train`) e seus rótulos (`y_train`);
- `predict(X_test)`: retorna as classes preditas para as amostras de `X_test`.

Internamente, `predict()` executa as etapas descritas anteriormente: calcula as distâncias, identifica os k vizinhos mais próximos e determina a classe por votação majoritária.

A Figure 6.11 ilustra esse procedimento em um conjunto bidimensional. A figura destaca os vizinhos utilizados na classificação, enquanto o console apresenta as etapas do algoritmo: cálculo das distâncias, ordenação, seleção dos vizinhos, votação e predição da classe.

```
def knn_passo_a_passo(X, y, x, k=3):
    """Executa as cinco etapas do algoritmo k-NN.
    Parâmetros: X (treino), y (rótulos), x (teste) e k (número de vizinhos).
    """
    # 1. Distâncias
    dist = [(np.linalg.norm(x-xi), yi, i) for i, (xi, yi) in enumerate(zip(X, y))]
    print(f"1. Distâncias calculadas: {len(dist)}")

    # 2. Ordenação
    dist.sort(key=lambda t: t[0])
    print("2. Distâncias ordenadas")

    # 3. Seleção
    vizinhos = dist[:k]
    print(f"3. {k} vizinhos mais próximos:")
    for d, c, _ in vizinhos:
        print(f"    {d:.4f} → {c}")

    # 4. Votação
    votos = {}
    for _, c, _ in vizinhos:
        votos[c] = votos.get(c, 0) + 1
    print("4. Votos:", votos)

    # 5. Decisão
    classe = max(votos, key=votos.get)
    print("5. Classe predita:", classe)

    return classe, vizinhos

# Dados de exemplo
np.random.seed(4)
X = np.r_[np.random.randn(15,2)+[2,2],
          np.random.randn(15,2)+[-2,-2]]
y = np.array(["Classe A"]*15 + ["Classe B"]*15)
x = np.array([0.5,0.5])

classe, vizinhos = knn_passo_a_passo(X, y, x)

# Visualização
plt.figure(figsize=(5,5))

for c, rotulo in [("Classe A","Classe A"), ("Classe B","Classe B")]:
    P = X[y==c]
    plt.scatter(P[:,0], P[:,1], s=80, label=rotulo)

plt.scatter(*x, marker="*", s=220, edgecolors="black", label="Teste")

for _, _, i in vizinhos:
    plt.scatter(*X[i], s=220, facecolors="none", edgecolors="black", linewidths=2)
    plt.plot([x[0], X[i,0]], [x[1], X[i,1]], "--", lw=1)

plt.xlabel("Característica 1")
plt.ylabel("Característica 2")
plt.title(f"k-NN ($k=3$): classe predita = {classe}")
```

```
plt.legend()
plt.grid(alpha=.3)
plt.axis("equal")
plt.tight_layout()
plt.show()
```

1. Distâncias calculadas: 30
2. Distâncias ordenadas
3. 3 vizinhos mais próximos:
 - 1.0850 → Classe A
 - 1.6379 → Classe B
 - 1.6382 → Classe A
4. Votos: {np.str_('Classe A'): 2, np.str_('Classe B'): 1}
5. Classe predita: Classe A

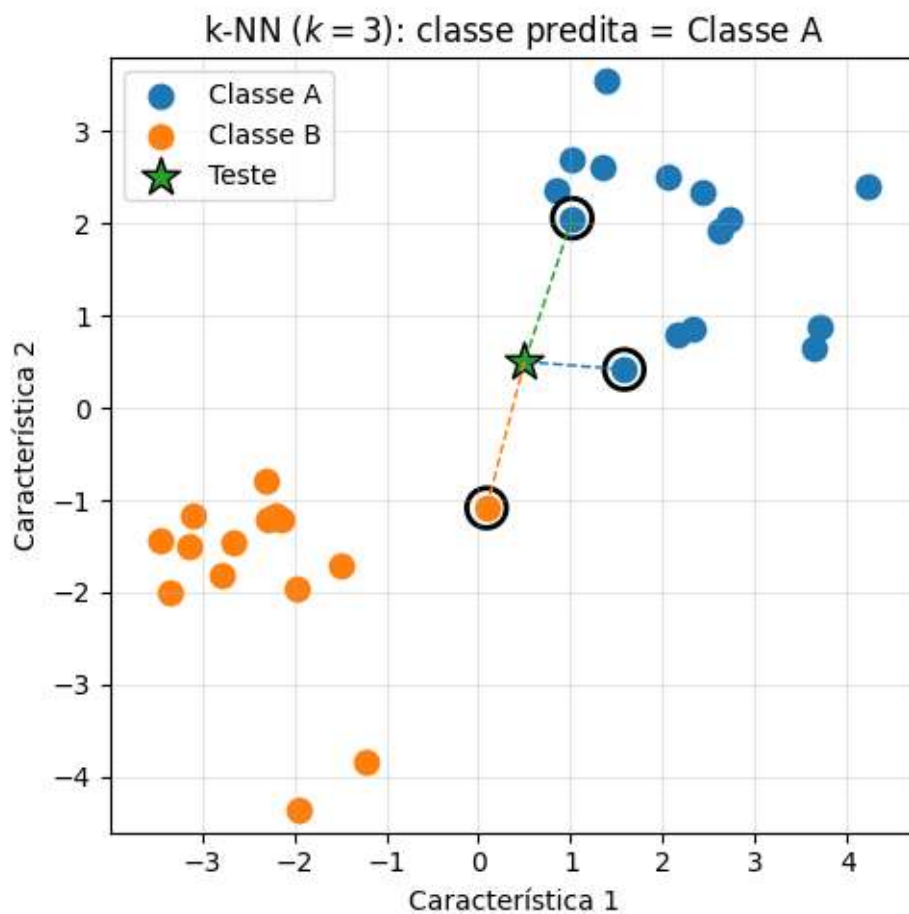


Figura 6.11: Classificação de uma nova amostra pelo algoritmo k-NN. O ponto de teste (estrela) é classificado a partir dos três vizinhos mais próximos, destacados por círculos.

6.10.3 O Papel do Parâmetro k

O parâmetro k determina quantos vizinhos participam da decisão de classificação.

- **Valores pequenos de k** (por exemplo, $k = 1$) tornam o classificador mais sensível a ruídos e variações locais, produzindo fronteiras de decisão mais irregulares e favorecendo o *overfitting*.
- **Valores maiores de k** produzem fronteiras de decisão mais suaves, porém podem reduzir a sensibilidade a estruturas locais, favorecendo o *underfitting*.

Em problemas com duas classes, é comum utilizar valores ímpares de k para reduzir a ocorrência de empates. Outro aspecto importante é a **maldição da dimensionalidade** (*curse of dimensionality*). À medida que o

número de características aumenta, as distâncias entre as amostras tendem a se tornar mais semelhantes, dificultando a identificação de vizinhos realmente representativos.

i Resumo

O algoritmo k-NN pode ser resumido em três etapas:

1. extrair o vetor de características da nova amostra;
2. identificar os k vizinhos mais próximos;
3. classificar a amostra pela classe mais frequente entre esses vizinhos.

O simulador da Figure 6.12 permite explorar visualmente o efeito do parâmetro k sobre a fronteira de decisão.



Figura 6.12: Simulador interativo da fronteira de decisão do k-NN: adicione pontos de treinamento e ajuste o valor de k para observar o efeito sobre a região de decisão.

6.11 Projeto Prático 1: Classificação de Dígitos Manuscritos com k-NN

As seções anteriores apresentaram o algoritmo k-NN por meio de um exemplo simplificado de classificação de frutas, utilizando apenas duas características. A seguir, o mesmo algoritmo é aplicado a um conjunto de dados de imagens, no qual cada amostra é representada por um vetor de maior dimensão.

Como estudo de caso, utiliza-se a base pública `load_digits`, disponibilizada pela biblioteca `scikit-learn`. Esse conjunto de dados contém 1797 imagens de dígitos manuscritos das classes de 0 a 9, cada uma com resolução de 8×8 pixels em níveis de cinza. Cada imagem é representada por um vetor com 64 características,

correspondentes às intensidades dos pixels, e cada vetor possui um rótulo indicando o dígito correspondente.

A base `load_digits` é disponibilizada pela biblioteca `scikit-learn` e é utilizada neste capítulo para ilustrar a aplicação do algoritmo k-NN. Além de estar disponível diretamente no `scikit-learn`, ela dispensa etapas adicionais de obtenção e preparação dos dados, permitindo concentrar a atenção na implementação e na avaliação do classificador.

A Figure 6.13 apresenta uma amostra das imagens da base de dados.

```
digits = load_digits()
print(f"Total de amostras: {digits.data.shape[0]}, dimensão do vetor: {digits.data.shape[1]}")
print(f"Classes: {[int(i) for i in sorted(set(digits.target))]}")

n_amostras = 16
imgs = list(digits.images[:n_amostras])
imgs_titles = [str(label) for label in digits.target[:n_amostras]]
mm.show(imgs, titles=imgs_titles, cols=8, figsize=(12, 4))
```

```
Total de amostras: 1797, dimensão do vetor: 64
Classes: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

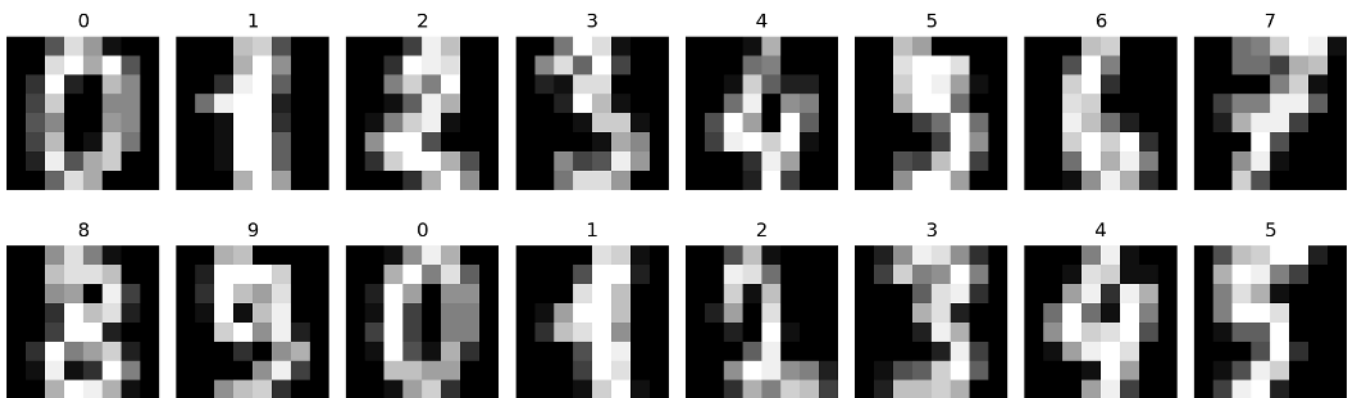


Figura 6.13: Amostra de dígitos manuscritos da base `load_digits`, utilizada como estudo de caso de classificação.

6.11.1 Classificação com Vetores de Intensidade

Neste primeiro experimento, cada imagem de dimensão 8×8 é representada diretamente pelas intensidades de seus 64 pixels, sem a extração de descritores adicionais. Assim, cada amostra corresponde a um vetor de 64 características, utilizado como entrada do classificador k-NN.

Em seguida, o conjunto de dados é dividido em subconjuntos de treinamento e teste, preservando a proporção das dez classes por meio do parâmetro `stratify=y`. O classificador é treinado com $k = 3$ e avaliado sobre o conjunto de teste utilizando a acurácia e a matriz de confusão apresentada na Figure 6.14.

```
X, y = digits.data, digits.target

X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

n_neighbors = 3
knn_pixels = KNeighborsClassifier(n_neighbors=n_neighbors)
knn_pixels.fit(X_treino, y_treino)
pred_pixels = knn_pixels.predict(X_teste)

acc_pixels = accuracy_score(y_teste, pred_pixels)
print(f"Acurácia (vetores de intensidade, k={n_neighbors}): {acc_pixels:.4f}")
```

```

cm = confusion_matrix(y_teste, pred_pixels)

plt.figure(figsize=(5,4))
plt.imshow(cm, cmap="Blues")

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(
            j, i, cm[i, j],
            ha="center", va="center",
            color="white" if cm[i, j] > cm.max()/2 else "black",
            fontsize=9
        )

plt.title("Matriz de Confusão - Pixels Brutos")
plt.xlabel("Classe Predita")
plt.ylabel("Classe Real")
plt.xticks(range(10))
plt.yticks(range(10))
plt.colorbar(fraction=0.046)
plt.tight_layout()
plt.show()

```

Acurácia (vetores de intensidade, k=3): 0.9870

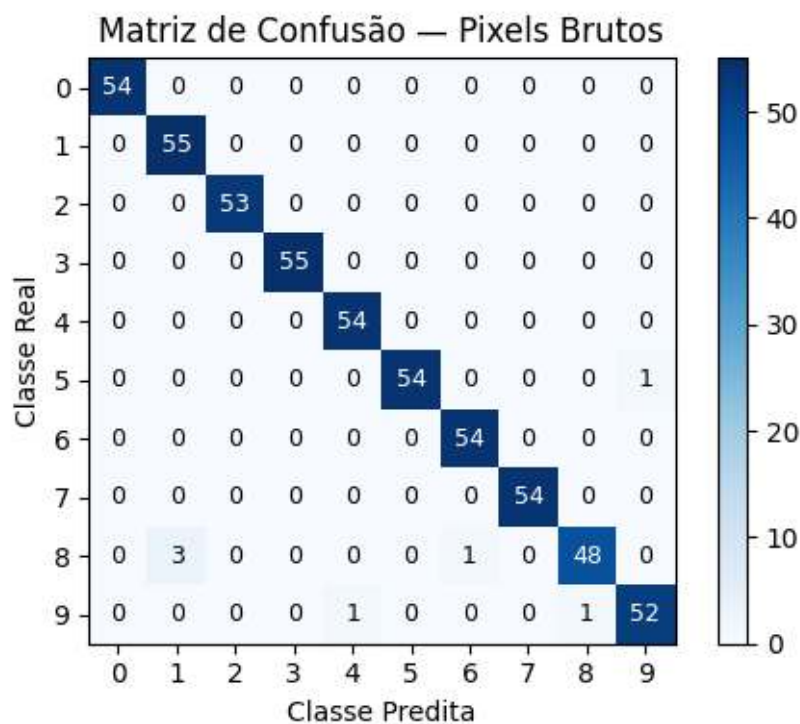


Figura 6.14: Matriz de confusão do classificador k-NN treinado com vetores de intensidade brutos (pixels).

6.11.2 Classificação com Descritores HOG

No experimento anterior, cada imagem foi representada diretamente pelas intensidades de seus pixels. Nesta seção, essa representação é substituída por descritores HOG (*Histogram of Oriented Gradients*), que codificam informações sobre a distribuição das orientações dos gradientes da imagem.

Mantêm-se o mesmo particionamento dos dados, o mesmo classificador k-NN e o mesmo protocolo de avaliação, alterando apenas a representação das imagens. A Figure 6.15 compara os resultados obtidos com

vetores de intensidade e com descritores HOG.

```
descritores_hog = np.array([
    hog(img, orientations=8, pixels_per_cell=(4, 4), cells_per_block=(1, 1))
    for img in digits.images
])
print(f"Dimensão do vetor HOG: {descritores_hog.shape[1]}")

Xh_treino, Xh_teste, yh_treino, yh_teste = train_test_split(
    descritores_hog, y, test_size=0.3, random_state=42, stratify=y
)

n_neighbors = 3
knn_hog = KNeighborsClassifier(n_neighbors=n_neighbors)
knn_hog.fit(Xh_treino, yh_treino)
pred_hog = knn_hog.predict(Xh_teste)
acc_hog = accuracy_score(yh_teste, pred_hog)
print(f"Acurácia (descriptor HOG, k={n_neighbors}): {acc_hog:.4f}")

plt.figure(figsize=(4, 3))
plt.bar(["Pixels brutos", "HOG"], [acc_pixels, acc_hog], color=["#6366f1", "#f97316"])
plt.ylim(0, 1.1) # Aumenta o limite superior para dar espaço
plt.ylabel("Acurácia")
plt.title("Comparação de Descritores")
for i, v in enumerate([acc_pixels, acc_hog]):
    plt.text(i, v + 0.02, f"{v:.3f}", ha="center") # Aumenta o deslocamento vertical
plt.tight_layout()
```

Dimensão do vetor HOG: 32
Acurácia (descriptor HOG, k=3): 0.7593

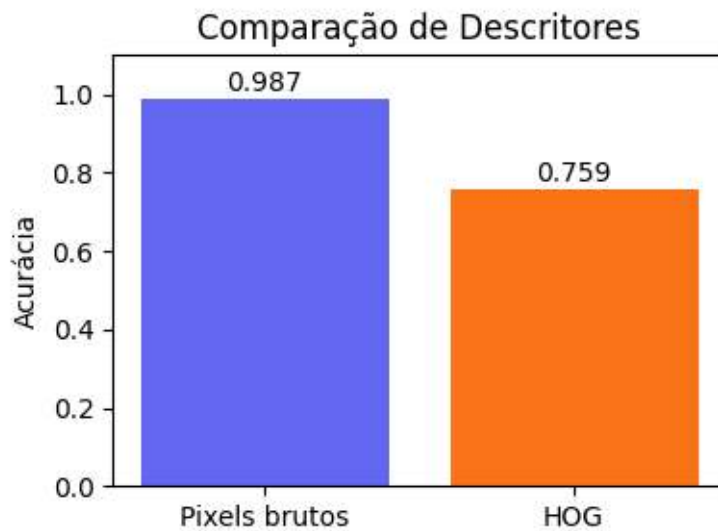


Figura 6.15: Comparação de acurácia entre descritores de pixels brutos e HOG para o classificador k-NN (k=3) na base de dígitos.

i Por que isso acontece?

Na Figure 6.15, o classificador treinado com vetores de **intensidade dos pixels** alcança maior acurácia (0.987) do que aquele baseado em descritores **HOG** (0.759). Esse resultado está relacionado às características da base `load_digits`.

As imagens possuem resolução de apenas 8×8 pixels, encontram-se aproximadamente centralizadas

e apresentam pouca variação de iluminação, escala e orientação. Nesse cenário, as intensidades dos pixels preservam praticamente toda a informação necessária para distinguir as classes. Em contraste, o HOG resume a imagem em histogramas de orientações dos gradientes, reduzindo parte do detalhamento espacial disponível nos pixels originais.

Essa redução de informação pode dificultar a separação de dígitos visualmente semelhantes, como 3 e 8 ou 4 e 9, especialmente quando a resolução da imagem é baixa.

Em problemas com imagens de maior resolução ou sujeitas a variações de iluminação, posição, escala ou pequenas deformações, descritores como o HOG tendem a representar melhor a estrutura local da imagem do que os valores individuais dos pixels. Assim, este experimento ilustra um princípio importante da Aprendizagem de Máquina: **a representação dos dados deve ser escolhida de acordo com as características do problema e não pela complexidade do descritor.**

6.12 Avaliação de Classificadores

Nas seções anteriores, a qualidade do classificador foi analisada por meio da acurácia e da matriz de confusão. Nesta seção, essas ferramentas são complementadas por métricas utilizadas na avaliação de modelos e por um procedimento para selecionar o valor do parâmetro k .

A acurácia corresponde à proporção de amostras classificadas corretamente. Embora seja uma medida simples e amplamente utilizada, ela pode ser insuficiente quando as classes apresentam distribuições muito desbalanceadas.

A partir da matriz de confusão — introduzida no **Capítulo 1** e utilizada ao longo deste capítulo — podem ser calculadas métricas por classe, como precisão e revocação:

$$\text{Precisão} = \frac{VP}{VP + FP}, \quad \text{Revocação} = \frac{VP}{VP + FN},$$

em que VP , FP e FN representam, respectivamente, o número de verdadeiros positivos, falsos positivos e falsos negativos da classe analisada. A precisão quantifica a proporção de predições positivas corretas, enquanto a revocação mede a capacidade do classificador de identificar os exemplos pertencentes à classe.

6.12.1 Escolha de k por Validação Cruzada

Nos experimentos anteriores, adotou-se $k = 3$ para ilustrar o funcionamento do algoritmo. Entretanto, esse parâmetro influencia diretamente o desempenho do classificador e, na prática, deve ser selecionado a partir dos dados.

Uma abordagem amplamente utilizada é a **validação cruzada** (*cross-validation*), na qual o conjunto de treinamento é dividido em partições sucessivas para estimar o desempenho do modelo em dados não utilizados durante o treinamento.

O código a seguir calcula a acurácia média obtida por validação cruzada de cinco partições (*5-fold cross-validation*) para diferentes valores de k . A Figure 6.16 apresenta os resultados, permitindo identificar a região em que o classificador atinge melhor desempenho.

```
valores_k = range(1, 16)
acuracias_medias = []

for k in valores_k:
    modelo = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(modelo, X, y, cv=5)
    acuracias_medias.append(scores.mean())

melhor_k = list(valores_k)[int(np.argmax(acuracias_medias))]
print(f"Melhor valor de k encontrado: {melhor_k} (acurácia média={max(acuracias_medias):.4f})")
```

```
plt.figure(figsize=(6, 4))
plt.plot(list(valores_k), acuracias_medias, marker="o", color="#4f46e5")
plt.axvline(melhor_k, color="#f97316", linestyle="--", label=f"melhor k = {melhor_k}")
plt.xlabel("k")
plt.ylabel("Acurácia média (validação cruzada)")
plt.title("Seleção de k por Validação Cruzada")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```

Melhor valor de k encontrado: 2 (acurácia média=0.9672)

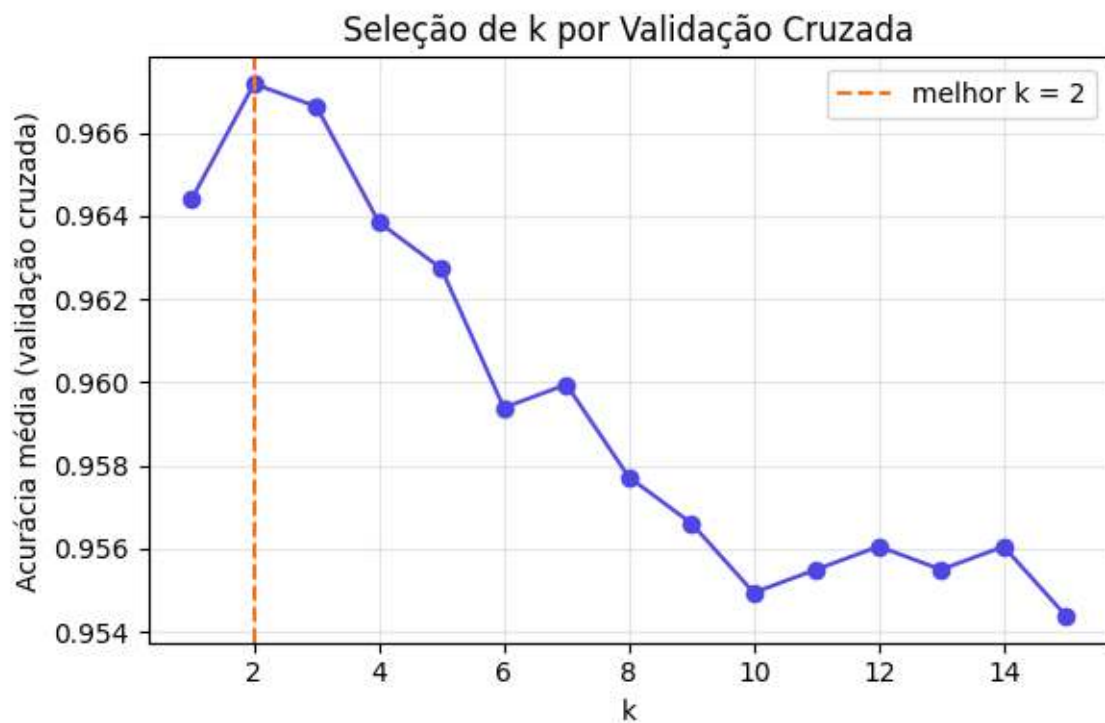


Figura 6.16: Acurácia média por validação cruzada (5 partições) em função do parâmetro k , para a base de dígitos com vetores de intensidade brutos.

6.12.2 O Compromisso entre Viés e Variância: Diagnóstico de *Overfitting* e *Underfitting*

O hiperparâmetro k influencia a complexidade da fronteira de decisão do classificador k -NN e, consequentemente, sua capacidade de generalização. Em termos gerais, valores pequenos de k tornam o modelo mais sensível às amostras de treinamento, enquanto valores maiores produzem fronteiras de decisão mais suaves.

Esses comportamentos estão associados ao compromisso entre **viés** (*bias*) e **variância** (*variance*). Valores muito pequenos de k tendem a aumentar o risco de **sobreajuste** (*overfitting*), especialmente em conjuntos de dados ruidosos, ao passo que valores muito grandes podem levar ao **subajuste** (*underfitting*), reduzindo a capacidade do modelo de capturar estruturas locais dos dados.

Enquanto a Figure 6.16 apresentou apenas a acurácia média obtida por validação cruzada, a Figure 6.17 compara as acurácias de treinamento e de teste para diferentes valores de k . As regiões destacadas no gráfico representam o comportamento esperado do algoritmo: maior risco de sobreajuste para valores pequenos de k , uma região intermediária que frequentemente produz bom equilíbrio entre viés e variância e maior risco de subajuste para valores elevados de k .

Entretanto, essas regiões devem ser interpretadas apenas como uma referência conceitual. O comportamento observado depende das características do conjunto de dados. Na base `load_digits`, por exemplo, as imagens

apresentam pouca variabilidade e boa separação entre as classes, de modo que valores pequenos de k podem apresentar desempenho semelhante — ou até superior — aos demais, sem evidenciar um sobreajuste significativo.

```
k_values = range(1, 16)
train_acc, test_acc = [], []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k).fit(X_treino, y_treino)
    train_acc.append(accuracy_score(y_treino, knn.predict(X_treino)))
    test_acc.append(accuracy_score(y_teste, knn.predict(X_teste)))

plt.figure(figsize=(9,5))
plt.plot(k_values, train_acc, "o-", lw=2, label="Treinamento")
plt.plot(k_values, test_acc, "s-", lw=2, label="Teste")

plt.axvspan(1, 3, color="#fca5a5", alpha=.25, label="Maior risco de overfitting")
plt.axvspan(3,11, color="#86efac", alpha=.25, label="Compromisso entre viés e variância")
plt.axvspan(11,15, color="#93c5fd", alpha=.25, label="Maior risco de underfitting")

plt.xlabel("Número de vizinhos ($k$)")
plt.ylabel("Acurácia")
plt.xticks(k_values)
plt.grid(alpha=.3)
plt.legend(loc="upper right")
plt.tight_layout()
plt.show()

maior_acc = max(test_acc)
melhores_k = [k for k, a in zip(k_values, test_acc) if np.isclose(a, maior_acc)]

print("Interpretação")
print("- Valores pequenos de k: maior risco de overfitting.")
print("- Valores intermediários: melhor compromisso entre viés e variância.")
print("- Valores grandes de k: maior risco de underfitting.")
print("\nAs regiões coloridas representam tendências gerais;")
print("o comportamento observado depende do conjunto de dados.")
print(f"\nMaior acurácia no teste: {maior_acc:.3f}")
print(f"Valores de k que atingiram essa acurácia: {melhores_k}")
```

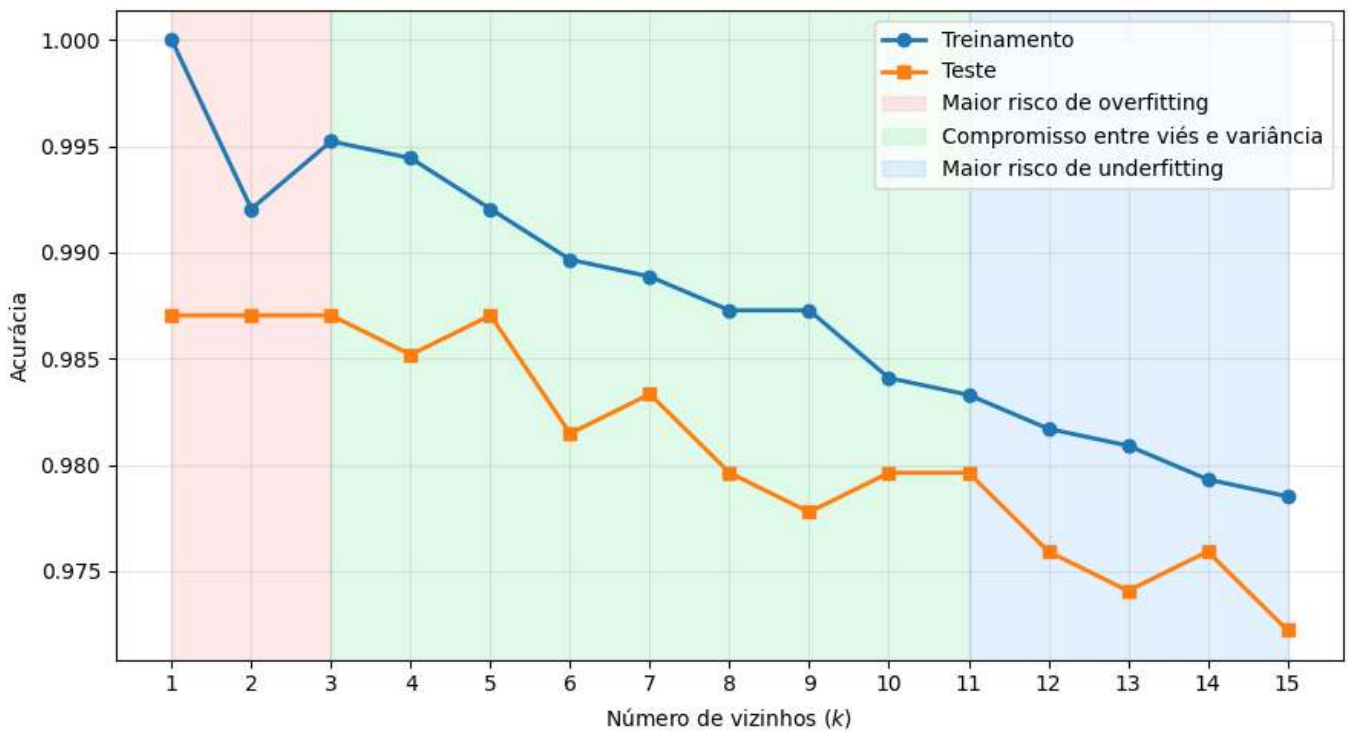


Figura 6.17: Acurácia nos conjuntos de treinamento e teste para diferentes valores de k . As regiões coloridas representam, de forma conceitual, tendências de comportamento do classificador: maior risco de sobreajuste (vermelho), compromisso entre viés e variância (verde) e maior risco de subajuste (azul).

Interpretação

- Valores pequenos de k : maior risco de overfitting.
- Valores intermediários: melhor compromisso entre viés e variância.
- Valores grandes de k : maior risco de underfitting.

As regiões coloridas representam tendências gerais;
o comportamento observado depende do conjunto de dados.

Maior acurácia no teste: 0.987

Valores de k que atingiram essa acurácia: [1, 2, 3, 5]

6.13 Projeto Prático 2: Comparação de Descritores para Classificação de Texturas

No **Capítulo 6**, a variância local foi utilizada como descritor de textura para **detectar** anomalias em superfícies industriais, distinguindo amostras **conformes** e **defeituosas**. Neste projeto, o problema é reformulado como uma tarefa de **classificação multiclasse**, na qual diferentes representações da imagem são utilizadas como entrada para um classificador.

Serão considerados três tipos de descritores: as **intensidades dos pixels**, o **Local Binary Patterns (LBP)** e o **Histogram of Oriented Gradients (HOG)**. Para cada representação, será extraído um vetor de características que servirá de entrada para o algoritmo dos k vizinhos mais próximos (k -NN). Ao final, serão comparadas as acurácias obtidas por cada descritor nas condições definidas para este experimento.

A avaliação será realizada por meio de **validação cruzada estratificada em cinco partições (5-fold stratified cross-validation)**. Nesse procedimento, o conjunto de dados é dividido em cinco subconjuntos preservando a proporção entre as classes. Em cada iteração, uma partição é utilizada para teste e as quatro restantes para treinamento, repetindo-se o processo até que todas as partições tenham sido utilizadas como conjunto de teste. Ao término das cinco execuções, são calculadas a acurácia média e o desvio-padrão para

cada descritor.

O conjunto de dados é composto por três classes de texturas sintéticas: **granular**, obtida a partir de ruído gaussiano suavizado; **listrada**, formada por padrões senoidais periódicos; e **manchada**, composta por regiões circulares sobrepostas. Para introduzir variabilidade entre as amostras, todas as imagens recebem uma perturbação por ruído gaussiano de baixa intensidade. A Figure 6.18 apresenta exemplos das três classes utilizadas no experimento.

```
rng = np.random.default_rng(42)

def gerar_textura(classe, tamanho=64, ruido=0.10):
    """Gera uma textura sintética 64x64 pertencente a uma das três classes."""
    if classe == "granular":
        escala = rng.uniform(0.14, 0.22)
        img = rng.normal(0.5, escala, (tamanho, tamanho))
        img = cv2.GaussianBlur(img.astype(np.float32), (3, 3), 0)

    elif classe == "listrada":
        n_periodos = rng.uniform(4, 8)
        amplitude = rng.uniform(0.22, 0.38)
        eixo_x = np.linspace(0, n_periodos * np.pi, tamanho)
        base = 0.5 + amplitude * np.sin(eixo_x)
        img = np.tile(base, (tamanho, 1)).astype(np.float32)
        img += rng.normal(0, 0.09, (tamanho, tamanho)).astype(np.float32)

    elif classe == "manchada":
        img = np.full((tamanho, tamanho), 0.5, dtype=np.float32)
        n_manchas = rng.integers(5, 11)
        for _ in range(n_manchas):
            cx, cy = rng.integers(0, tamanho, 2)
            raio = int(rng.integers(3, 11))
            intensidade = float(rng.uniform(0.15, 0.9))
            cv2.circle(img, (int(cx), int(cy)), raio, intensidade, -1)
        img = cv2.GaussianBlur(img, (5, 5), 0)

    else:
        raise ValueError(f"Classe desconhecida: {classe}")

    img = img + rng.normal(0, ruido, (tamanho, tamanho)).astype(np.float32)
    img = np.clip(img, 0, 1)
    return (img * 255).astype(np.uint8)

classes_textura = ["granular", "listrada", "manchada"]
amostras = [gerar_textura(c) for c in classes_textura]

mm.show(amostras, titles=classes_textura, cols=3, figsize=(9, 3))
```

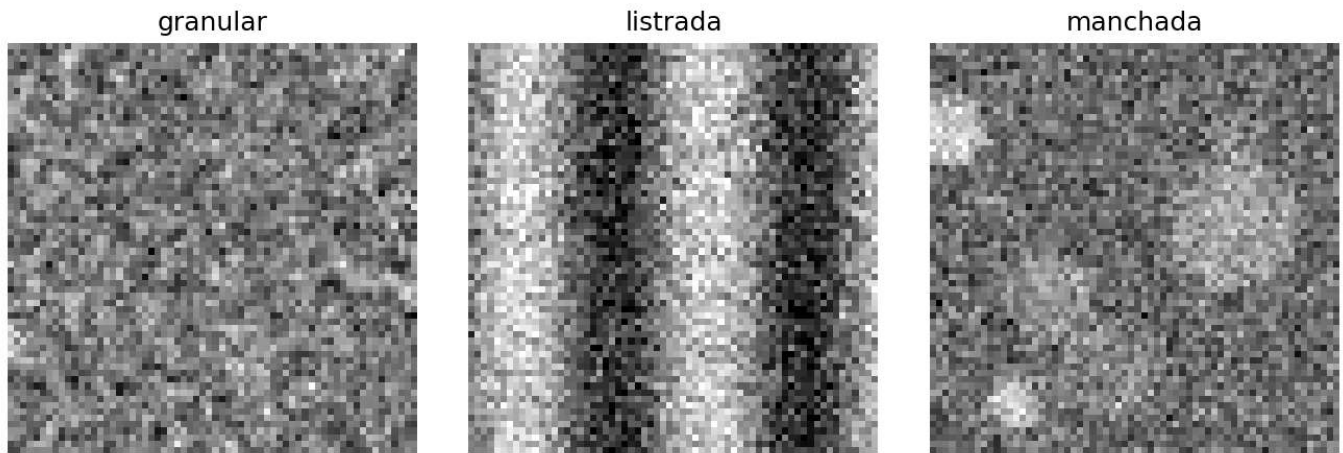


Figura 6.18: Amostras sintéticas das três classes de textura utilizadas no experimento de classificação, geradas com ruído gaussiano e variabilidade intra-classe.

6.13.1 Pipeline de Extração de Características e Avaliação Comparativa

Para comparar o desempenho de diferentes formas de representação das imagens, foi gerado um conjunto de dados balanceado contendo 60 amostras para cada classe. A partir desse conjunto, foram extraídos três tipos de vetores de características, cada um representando aspectos distintos da informação visual:

1. **Pixels brutos:** vetor obtido pelo achatamento (*flattening*) da matriz de intensidades da imagem, resultando em um vetor de $64 \times 64 = 4096$ atributos;
2. **Histograma LBP uniforme:** histograma normalizado das frequências dos padrões locais produzidos pelo operador LBP uniforme, composto por 10 atributos;
3. **Descritor HOG:** vetor formado por histogramas de gradientes orientados, que representam a distribuição espacial das orientações das bordas, totalizando 128 atributos.

Como esses descritores possuem escalas e dimensionalidades distintas, os vetores de características são padronizados utilizando o `StandardScaler`, de modo que cada atributo apresente média zero e desvio-padrão unitário. Essa etapa evita que atributos com maior amplitude influenciem desproporcionalmente o cálculo das distâncias euclidianas empregado pelo classificador.

A avaliação é realizada utilizando o algoritmo k -NN com $k = 5$, sob o mesmo protocolo de validação cruzada estratificada em cinco partições descrito na seção anterior. A acurácia média obtida ao longo das cinco execuções, ver Figure 6.19, fornece uma estimativa mais estável do desempenho do classificador, reduzindo a dependência de uma única divisão entre treinamento e teste.

```
rng = np.random.default_rng(42)

# 1. Geração da base de dados
X_bruto, X_lbp, X_hog, y_textura = [], [], [], []

for classe in classes_textura:
    for _ in range(60):
        img = gerar_textura(classe, ruído=0.10)

        # Extração 1: Pixels brutos
        X_bruto.append(img.ravel())

        # Extração 2: Histograma LBP uniforme
        lbp = local_binary_pattern(img, P=8, R=1, method="uniform")
        hist_lbp, _ = np.histogram(lbp, bins=10, range=(0, 10), density=True)
        X_lbp.append(hist_lbp)
```



```

# Extração 3: Descritor HOG
feat_hog = hog(img, orientations=8, pixels_per_cell=(16, 16), cells_per_block=(1, 1))
X_hog.append(feat_hog)

y_textura.append(classe)

y_textura = np.array(y_textura)
descritores = {
    "Pixels Brutos": np.array(X_bruto),
    "LBP (Textura)": np.array(X_lbp),
    "HOG (Forma)": np.array(X_hog)
}

# 2. Avaliação estatística via 5-fold cross-validation
resultados_media = {}
resultados_desvio = {}

knn = KNeighborsClassifier(n_neighbors=5)
scaler = StandardScaler()

for nome, X_dados in descritores.items():
    X_norm = scaler.fit_transform(X_dados)
    scores = cross_val_score(knn, X_norm, y_textura, cv=5, scoring="accuracy")
    resultados_media[nome] = scores.mean()
    resultados_desvio[nome] = scores.std()
    print(f"{nome:15s} -> Acurácia Média: {scores.mean():.4f} (± {scores.std():.4f})")

# 3. Plotagem do gráfico comparativo formal
plt.figure(figsize=(7, 4.5))
nomes_desc = list(resultados_media.keys())
medias = list(resultados_media.values())
desvios = list(resultados_desvio.values())

bars = plt.bar(nomes_desc, medias, yerr=desvios, capsize=6,
               color=["#6366f1", "#9333ea", "#f97316"],
               width=0.45, edgecolor="black", alpha=0.85)
plt.ylabel("Acurácia Média (5-Fold CV)", fontsize=11)
plt.title("Análise Comparativa de Descritores para Classificação de Texturas",
          fontsize=12, fontweight="bold")
plt.ylim(0.3, 1.1)
plt.grid(axis="y", linestyle="--", alpha=0.5)

for bar in bars:
    h = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2,
             h + 0.03, f"{h:.3f}", ha="center", fontweight="bold")

plt.tight_layout()
plt.show()

```

```

Pixels Brutos    -> Acurácia Média: 0.6889 (± 0.0478)
LBP (Textura)    -> Acurácia Média: 0.7611 (± 0.0648)
HOG (Forma)      -> Acurácia Média: 0.4889 (± 0.0624)

```

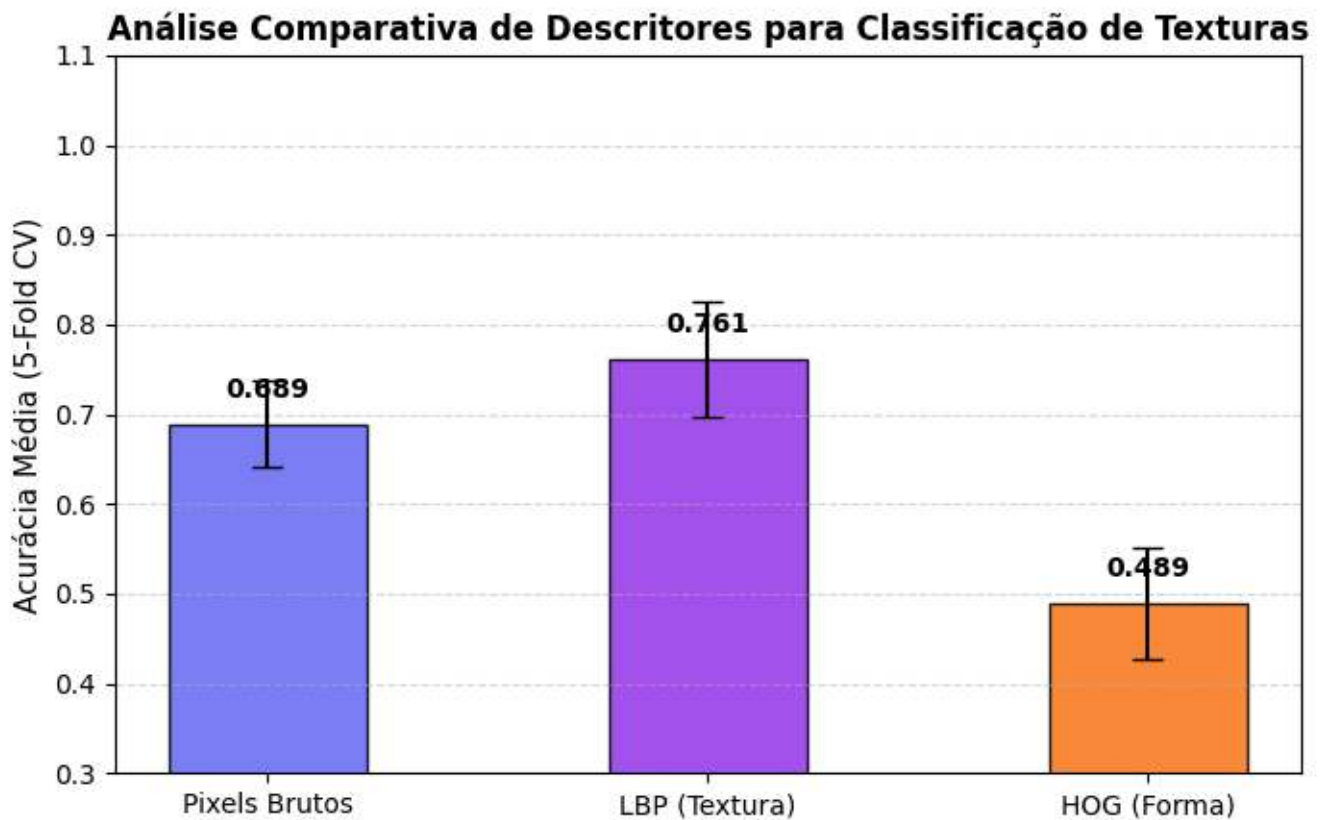



Figura 6.19: Acurácia média obtida via validação cruzada (5-fold) para os descritores de Pixels Brutos, LBP e HOG aplicados à base de texturas sintéticas.

i Por que funciona? — LBP como representação de texturas

O descritor LBP representa a textura de uma imagem por meio de um histograma normalizado que contabiliza a frequência dos padrões locais de intensidade. Em vez de armazenar diretamente os valores dos pixels ou suas posições, essa representação resume a distribuição das microestruturas presentes na imagem, produzindo um vetor de características compacto.

Neste experimento, foram comparados três tipos de descritores: pixels brutos, LBP e HOG. Os vetores formados pelos **pixels brutos** preservam todas as intensidades da imagem, mas também incorporam variações decorrentes de ruído e pequenos deslocamentos espaciais, o que pode dificultar a comparação entre amostras por meio da distância euclidiana.

O descritor **HOG** representa a distribuição das orientações dos gradientes, sendo adequado para descrever formas e contornos. Como as imagens utilizadas neste projeto diferem principalmente pelas propriedades de textura, e não pela presença de contornos bem definidos, essa representação tende a capturar menos informações discriminativas do que o LBP.

Já o **LBP** foi desenvolvido especificamente para caracterizar padrões locais de textura. Seu histograma descreve a frequência das microestruturas presentes na imagem, independentemente de sua posição exata, tornando a representação menos sensível a pequenas variações espaciais e a mudanças monotônicas de iluminação.

Embora os histogramas das diferentes classes apresentem distribuições distintas, o ruído gaussiano introduzido na geração das imagens aumenta a variabilidade entre amostras da mesma classe e pode produzir regiões de sobreposição no espaço de características. Como consequência, algumas texturas podem ser confundidas pelo classificador. Ainda assim, quando as características relevantes para distinguir as classes estão associadas aos padrões locais de textura, espera-se que descritores projetados para essa finalidade, como o LBP, produzam representações mais informativas do que aquelas baseadas apenas nas intensidades dos pixels ou nas orientações dos gradientes.

6.13.2 Diagnóstico Fino do Classificador: Precisão, Revocação e F1-Score

A acurácia resume o desempenho do classificador em um único valor, mas não indica como esse desempenho se distribui entre as diferentes classes. Para uma análise mais detalhada, utilizam-se métricas calculadas individualmente para cada classe.

A **precisão** (*precision*) mede a proporção de amostras classificadas como pertencentes a uma classe que realmente pertencem a ela. A **revocação** (*recall*) mede a proporção de amostras da classe que foram corretamente identificadas pelo classificador. O **F1-score** corresponde à média harmônica entre precisão e revocação, fornecendo um indicador que equilibra ambas as medidas.

O relatório também informa o **suporte** (*support*), isto é, o número de amostras de cada classe presentes no conjunto de teste. Essa informação é importante para contextualizar as métricas, pois resultados obtidos sobre poucas amostras tendem a apresentar maior variabilidade.

A Figure 6.20 apresenta essas métricas para as três classes de textura. Em conjunto, elas permitem identificar diferenças de desempenho que não são evidentes apenas pela acurácia. Por exemplo, uma classe pode apresentar alta precisão e menor revocação, indicando que o classificador comete poucos falsos positivos, mas deixa de identificar parte das amostras que realmente pertencem àquela classe. Esse tipo de análise auxilia na compreensão das limitações do modelo e na identificação de possíveis estratégias para seu aprimoramento.

```
X_lbp_data = np.array(X_lbp)
y_textura_data = np.array(y_textura)

# Realizar um split de treino/teste para o relatório detalhado
Xt_treino, Xt_teste, yt_treino, yt_teste = train_test_split(
    X_lbp_data, y_textura_data, test_size=0.3, random_state=42, stratify=y_textura_data
)

# Escalar os dados
scaler = StandardScaler()
Xt_treino_scaled = scaler.fit_transform(Xt_treino)
Xt_teste_scaled = scaler.transform(Xt_teste)

# Treinar o classificador k-NN
knn_textura = KNeighborsClassifier(n_neighbors=5)
knn_textura.fit(Xt_treino_scaled, yt_treino)

y_pred = knn_textura.predict(Xt_teste_scaled)

report = classification_report(
    yt_teste,
    y_pred,
    target_names=classes_textura,
    output_dict=True
)

print("=== RELATÓRIO DE CLASSIFICAÇÃO DETALHADO ===")
print(f"{'Classe':<12} {'Precisão':>10} {'Revocação':>12} {'F1-score':>10} {'Suporte':>10}")
for classe in classes_textura:
    r = report[classe]
    print(f"{'classe':<12} {r['precision']:>10.2f} {r['recall']:>12.2f} "
          f"{r['f1-score']:>10.2f} {r['support']:>10.0f}")

precision = precision_score(yt_teste, y_pred, average=None, labels=classes_textura)
recall = recall_score(yt_teste, y_pred, average=None, labels=classes_textura)
f1 = f1_score(yt_teste, y_pred, average=None, labels=classes_textura)

fig, ax = plt.subplots(figsize=(10, 5))
x = np.arange(len(classes_textura))
```

```

width = 0.25

bars1 = ax.bar(x - width, precision, width, label='Precisão', color='#6366f1', alpha=0.8)
bars2 = ax.bar(x, recall, width, label='Revocação', color='#f97316', alpha=0.8)
bars3 = ax.bar(x + width, f1, width, label='F1-Score', color='#22c55e', alpha=0.8)

ax.set_xlabel('Classe', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Métricas por Classe - Classificação de Texturas', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(classes_textura)
ax.legend(loc='upper right')
ax.set_ylim(0, 1.35)
ax.grid(axis='y', alpha=0.3)

for bars in [bars1, bars2, bars3]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.02,
                f'{height:.2f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

print("Interpretação das métricas:")
print("- Precisão: entre as amostras classific. como pertencentes à classe,",
      "quantas estavam corretas?")
print("- Revocação: entre as amostras que realmente pertencem à classe, quantas",
      "foram identificadas?")
print("- F1-Score: média harmônica entre precisão e revocação.")
print("- Suporte: número de amostras reais de cada classe presentes no conjunto de teste.")
print("\nO suporte não mede desempenho; ele apenas informa quantos exemplos de cada",
      "classe foram utilizados na avaliação.")

```

=== RELATÓRIO DE CLASSIFICAÇÃO DETALHADO ===

Classe	Precisão	Revocação	F1-score	Suporte
granular	0.67	0.78	0.72	18
lustrada	0.55	0.61	0.58	18
manchada	1.00	0.72	0.84	18

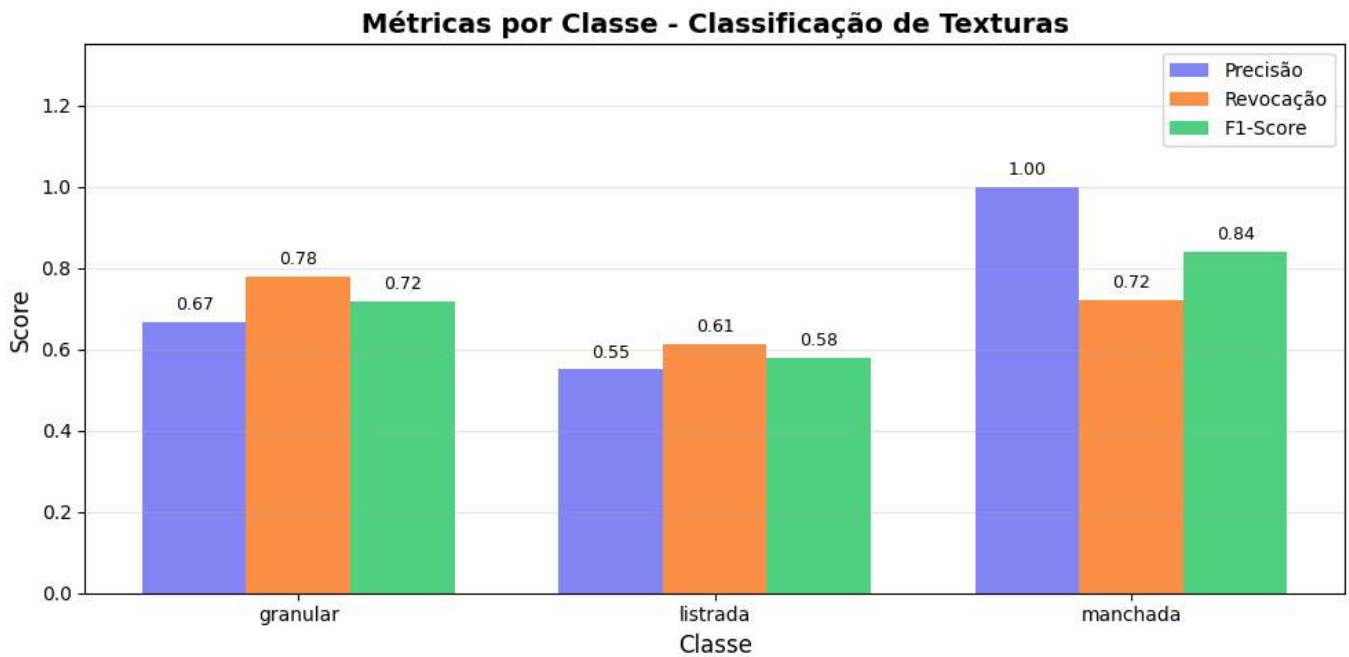


Figura 6.20: Métricas de avaliação detalhadas para o classificador k -NN com descritores LBP

Interpretação das métricas:

- **Precisão:** entre as amostras classificadas como pertencentes à classe, quantas estavam corretas?
- **Revocação:** entre as amostras que realmente pertencem à classe, quantas foram identificadas?
- **F1-Score:** média harmônica entre precisão e revocação.
- **Suporte:** número de amostras reais de cada classe presentes no conjunto de teste.

O suporte não mede desempenho; ele apenas informa quantos exemplos de cada classe foram utilizados na avaliação.

6.14 Limitações dos Descritores Artesanais

Os experimentos deste capítulo mostram que descritores clássicos podem ser bastante eficazes em tarefas de classificação, mas também apresentam limitações importantes:

- **Especificidade:** cada descritor foi desenvolvido para representar um determinado tipo de informação, como cor, textura ou forma. Assim, um descritor adequado para uma tarefa pode não ser o mais apropriado para outra.
- **Dependência de hiperparâmetros:** o desempenho de descritores como LBP e HOG depende da escolha de parâmetros, como raio de vizinhança, número de pontos amostrados, tamanho da célula e número de orientações, que precisam ser ajustados conforme a aplicação.
- **Representação limitada:** descritores de cor, textura e gradiente capturam propriedades de baixo nível da imagem, mas não representam diretamente conceitos semânticos mais complexos, como objetos ou cenas.
- **Maldição da dimensionalidade:** descritores muito extensos podem reduzir a eficácia de classificadores baseados em distância, como o k -NN.

Essas limitações motivam a evolução das técnicas estudadas nos próximos capítulos. O **Capítulo 8** apresenta métodos clássicos para detecção e correspondência de características em imagens, enquanto o **Capítulo 9** introduz as **Redes Neurais Convolucionais**, capazes de aprender automaticamente representações adequadas para cada tarefa a partir dos dados.

6.15 Resumo

Neste capítulo foram apresentados os fundamentos do reconhecimento de padrões aplicado a imagens. Os principais conceitos estudados foram:

- **Pipeline de reconhecimento de padrões:** aquisição, pré-processamento, extração de descritores, classificação e avaliação.
- **Descritores clássicos:** descritores de cor, LBP para textura e HOG para forma, utilizados para representar diferentes características das imagens.
- **Normalização de características:** padronização (*Z-score*) para evitar que atributos de maior magnitude dominem o cálculo das distâncias.
- **Classificador k -NN:** classificação baseada nos k vizinhos mais próximos no espaço de características.
- **Escolha do parâmetro k :** influência do valor de k sobre o desempenho do classificador e uso da validação cruzada para sua seleção.
- **Avaliação de classificadores:** acurácia, matriz de confusão, precisão, revocação e F1-score como métricas complementares de desempenho.
- **Limitações dos descritores artesanais:** especificidade, dependência de hiperparâmetros e dificuldade em representar informações de alto nível.

Os conceitos foram ilustrados por meio de experimentos com a base pública `load_digits`, texturas sintéticas geradas para fins didáticos e dados simulados de descritores de frutas.

6.16 Uso do Gemini Notebook como Tutor

Nesta edição, o **Gemini Notebook** é apresentado como uma ferramenta de apoio ao estudo. O sistema utiliza exclusivamente os documentos disponibilizados pelo autor como fonte de conhecimento, permitindo explorar os conceitos do capítulo por meio de perguntas, resumos e explicações relacionadas ao material estudado.

 Estude com o Tutor Inteligente

 [ACESSAR Gemini Notebook: CAPÍTULO 07](#)

Aviso sobre Conteúdo Gerado por IA

As respostas fornecidas pelo Gemini Notebook podem conter imprecisões ou omissões. Sempre que necessário, confirme as informações utilizando o material deste capítulo e outras fontes acadêmicas confiáveis. A execução dos exemplos práticos apresentados ao longo do texto continua sendo a melhor forma de consolidar os conceitos estudados.

6.17 Lista de Exercícios

Os exercícios a seguir exploram e estendem os conceitos apresentados neste capítulo por meio de adaptações dos algoritmos implementados, análises experimentais e comparações entre diferentes abordagens.

1. **(10%)** Implemente um descritor de cor (histograma RGB ou HSV, com pelo menos 16 *bins* por canal) para as três classes de frutas simuladas na Figure 6.1. Treine um classificador k -NN com esse descritor, compare sua acurácia com a obtida pelos descritores LBP e HOG (Figure 6.9) e discuta em quais situações a informação de cor é mais discriminativa.
2. **(15%)** Investigue o efeito da normalização de características (*Z-score*) sobre o desempenho do k -NN em um espaço de atributos heterogêneo, combinando descritores de cor, LBP e HOG em um único vetor. Compare os resultados obtidos com e sem normalização para pelo menos três valores de k .
3. **(15%)** Reproduza a análise de *overfitting* e *underfitting* da Figure 6.17 variando o tamanho do conjunto de treinamento (por exemplo, 20%, 50% e 80% da base `load_digits`). Discuta como a quantidade de

exemplos influencia a escolha do valor de k .

4. **(15%)** Estenda o Projeto Prático 2 adicionando uma quarta classe sintética de textura. Avalie precisão, revocação e F1-score para cada classe, seguindo o padrão da Figure 6.20, e analise o impacto da nova classe na matriz de confusão.
5. **(15%)** Implemente manualmente o classificador k -NN, sem utilizar `sklearn`:
`sklearn.neighbors.KNeighborsClassifier`,
completando a função `knn_passo_a_passo` apresentada no capítulo. Compare a acurácia e o tempo de execução da implementação manual com a implementação do `scikit-learn` em conjuntos de dados de tamanhos crescentes e relacione os resultados à maldição da dimensionalidade.
6. **(15%)** Investigue a influência dos parâmetros `orientations`, `pixels_per_cell` e `cells_per_block` do descritor HOG na base `load_digits`. Avalie pelo menos quatro combinações de parâmetros e discuta o compromisso entre dimensionalidade do descritor e desempenho do classificador.
7. **(15%)** Avalie a influência dos parâmetros P (número de vizinhos) e R (raio) do descritor LBP na classificação das texturas sintéticas do Projeto Prático 2, considerando $P \in \{4, 8, 16\}$ e $R \in \{1, 2, 3\}$. Analise como esses parâmetros afetam a capacidade discriminativa do descritor.
8. **(Bônus – 10%)** Implemente manualmente a validação cruzada k -fold para o classificador k -NN na base `load_digits`, sem utilizar `cross_val_score`, e compare os resultados com os obtidos pela implementação do `scikit-learn` apresentada na Figure 6.16.

Referências do Capítulo

A fundamentação teórica e os experimentos apresentados neste capítulo baseiam-se nas seguintes referências:

- Gonzalez; Woods (2018), pelos fundamentos de descritores estatísticos de textura e das operações de pré-processamento aplicadas à extração de características.
- Szeliski (2022), pela apresentação do pipeline clássico de reconhecimento de padrões, da extração de descritores e da avaliação de classificadores em Visão Computacional.
- Duda; Hart; Stork (2001), pelos fundamentos teóricos do reconhecimento de padrões, do classificador k -NN e da relação entre viés e variância.
- Cover; Hart (1967), pela formulação original do algoritmo dos k vizinhos mais próximos.
- Ojala; Pietikäinen; Mäenpää (2002), pela formulação do descritor *Local Binary Patterns* (LBP) e de sua variante uniforme, utilizada neste capítulo.
- Dalal; Triggs (2005), pela formulação do descritor *Histogram of Oriented Gradients* (HOG), empregado na representação de forma e contorno.
- Pedregosa et al. (2011), pela implementação do classificador k -NN, das métricas de avaliação e da validação cruzada na biblioteca `scikit-learn`.
- Quilici-Gonzalez; Zampirolli (2014), pela apresentação didática de classificadores tradicionais de Reconhecimento de Padrões, como Árvores de Decisão, Regras de Classificação e Máquinas de Vetores de Suporte (SVM), complementares ao classificador k -NN explorado neste capítulo.
- Quilici-Gonzalez; Zampirolli; Souza (2026), pela atualização e ampliação desses conteúdos em sua 2ª edição, atualmente em produção.

 Executar  Colab  Abrir pdi-vc  GitHub

6.18 Parte Prática com Exercícios de Programação

A presente lista de exercícios de programação (EP) consolida as formulações teóricas apresentadas ao longo do Capítulo 7 — Classificação de Imagens e Reconhecimento de Padrões — por meio de uma trilha prática aplicada. Diferentemente da manipulação direta de pixels dos capítulos anteriores, os EPs deste

capítulo trabalham com as **grandezas intermediárias** de um *pipeline* real de reconhecimento de padrões — vetores de características, distâncias, rótulos previstos e reais, códigos binários locais e histogramas de orientação — permitindo validar manualmente cada etapa do raciocínio sem depender de bibliotecas externas de aprendizado de máquina.

O encadeamento dos exercícios reproduz o fluxo conceitual do capítulo: inicia-se com a implementação manual da regra de decisão do classificador **k-NN** sobre um pequeno espaço de características; em seguida, revisita-se, sob a ótica da normalização de características, o classificador implementado no primeiro exercício da lista; avança-se para o cálculo das métricas de **avaliação** (matriz de confusão, precisão e revocação) a partir de rótulos previstos e reais; prossegue-se com a codificação manual do descritor de textura **LBP** a partir de uma vizinhança 3×3 ; aprofunda-se no cálculo do histograma de orientações do descritor **HOG** para uma única célula; avança-se, em seguida, para a integração de **extração de descritores**, **classificação k-NN** e **avaliação multi-classe** em um *pipeline* completo de reconhecimento de texturas; e conclui-se com a aplicação desse mesmo *pipeline* sobre uma **imagem real** (formato PGM), em que o descritor LBP é calculado diretamente sobre os pixels de um mosaico de texturas.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.7 | TestSuite: 1.1.2

Executando os Testes

Para avaliar os testes, execute `TestSuite("EP07_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```
codigo = """
# ... seu código aqui ...
"""

TestSuite("EP07_01").run_code(codigo)
```


Resumo dos Métodos do `morph.py` (Cap. 7)

A biblioteca `morph.py` disponibiliza duas versões para a maioria dos algoritmos: uma **didática** (métodos terminados em 0), implementada passo a passo em NumPy, e outra **clássica**, baseada nas bibliotecas `scikit-learn` e `scikit-image`. As implementações didáticas são utilizadas nos **Exercícios de Programação (EPs)**, pois não dependem de bibliotecas externas e executam dentro do limite de memória do ambiente **VPL** do Moodle. Já as versões clássicas são mais eficientes e indicadas para experimentos em ambientes como Colab e Jupyter Notebook, mas normalmente **não podem ser utilizadas nos EPs** do Moodle, pois a biblioteca `scikit-learn` excede a memória disponível no VPL.

1. **Leitura dos dados (`readClasses`, `readDataset`, `readTrain`, `readTest`)**
Padronizam a entrada dos conjuntos de treinamento e teste, retornando as matrizes de características (X) e os vetores de rótulos (y).
2. **Classificação (`knn0` / `knn`)**
Implementam o algoritmo dos **k-vizinhos mais próximos (k-NN)** para classificação binária e multiclasse, utilizando distância Euclidiana ou Manhattan.
3. **Normalização (`zscore0` / `zscore`)**
Aplicam a normalização *z-score* aos atributos, reduzindo diferenças de escala antes da classificação.
4. **Avaliação (`confusion0` / `confusion`)**
Calculam a matriz de confusão e métricas como acurácia, precisão e revocação, tanto para problemas binários quanto multiclasse.
5. **Descritor de textura (`lbp0` / `lbp`)**
Calculam o **Local Binary Pattern (LBP)**, permitindo obter o mapa LBP, o código de um pixel ou o histograma de uma região da imagem.
6. **Descritor de forma (`hog0` / `hog`)**
Calculam o **Histogram of Oriented Gradients (HOG)**, produzindo histogramas das orientações dos gradientes para representar informações de forma e contorno.

6.18.1 EP07_01 Classificador k-NN Passo a Passo

O `KNeighborsClassifier` do `scikit-learn`, usado ao longo do capítulo, esconde por trás de uma única chamada (`.fit` / `.predict`) uma regra de decisão bastante simples: para cada nova observação, calcular a distância a todos os exemplos de treinamento, selecionar os k mais próximos e votar pela classe majoritária entre eles.

Antes de confiar na biblioteca, você foi encarregado de implementar essa regra do zero, para um espaço de características bidimensional, exatamente como o simulador interativo de fronteira de decisão do capítulo faz internamente a cada clique do usuário.

6.18.1.1 Diretrizes de Implementação

1. **Quantidade e parâmetro:** Ler o inteiro N (número de exemplos de treinamento) e o inteiro ímpar k (número de vizinhos).
2. **Exemplos de treinamento:** Para cada um dos N exemplos, ler três valores: as coordenadas x e y (reais) e o rótulo r (inteiro, 0 ou 1).
3. **Consultas:** Ler o inteiro Q (número de pontos de consulta) e, em seguida, as coordenadas x_q , y_q (reais) de cada consulta.
4. **Distância:** Para cada consulta, calcular a distância euclidiana até **todos** os exemplos de treinamento:

$$d(x_q, x_i) = \sqrt{(x_q - x_i)^2 + (y_q - y_i)^2}.$$

5. **Seleção dos vizinhos:** Ordenar os exemplos por distância crescente e selecionar os k primeiros. Em caso de **empate de distância** na fronteira do k -ésimo vizinho, desempate pelo exemplo lido **primeiro** na entrada (ordem de leitura estável).

6. **Votação majoritária:** Contar os votos de cada classe entre os k vizinhos selecionados. Se houver **empate na votação** (apenas possível quando k é par, o que não deve ocorrer pela diretriz do item 1, mas trate defensivamente), atribua a classe do vizinho mais próximo entre as classes empatadas.
7. **Saída:** Para cada consulta, na ordem de entrada, imprimir a classe prevista. Ao final, imprimir o total de consultas classificadas como classe 1.

6.18.1.2 Restrições Computacionais

- **Métrica fixa:** utilize exclusivamente a distância euclidiana (não a *squared distance*) para a ordenação, embora o resultado da comparação seja o mesmo.
- **k sempre ímpar:** a entrada garante k ímpar e $k \leq N$; ainda assim, implemente o desempate do item 6 por robustez.
- **Estabilidade:** ao ordenar por distância, preserve a ordem relativa de exemplos com a mesma distância (ordenação estável).

6.18.1.3 Fundamentação Teórica

Elemento	Papel no k-NN
Espaço de características	Conjunto de todos os vetores (x, y) possíveis
Distância euclidiana	Medida de similaridade entre observações
k pequeno	Fronteira irregular, alta variância
k grande	Fronteira suave, alto viés
Votação majoritária	Regra de decisão $\hat{y} = \text{moda}\{y_i : x_i \in N_k(x)\}$

6.18.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros N e k , separados por espaço.
- Próximas N linhas: três valores por linha — x , y (reais) e r (inteiro $\in \{0, 1\}$), separados por espaço.
- Próxima linha: inteiro Q .
- Próximas Q linhas: dois valores por linha — x_q , y_q (reais), separados por espaço.

Saída:

- Q linhas, cada uma com a classe prevista (0 ou 1) para a respectiva consulta, na ordem de entrada.
- Última linha: **Total classe 1: X.**

6.18.1.5 Exemplos

Entrada	Saída	Observação
4 3	0	Consulta próxima do agrupamento de classe 0.
0 0 0	Total classe 1: 0	
1 0 0		
5 5 1		
6 5 1		
1		
1 1		

Entrada	Saída	Observação
4 1	0	Com $k = 1$, cada consulta herda a classe do vizinho mais próximo.
0 0 0	1	
1 0 0	Total classe 1: 1	
5 5 1		
6 5 1		
2		
0.9 0.1		
5.5 5.1		

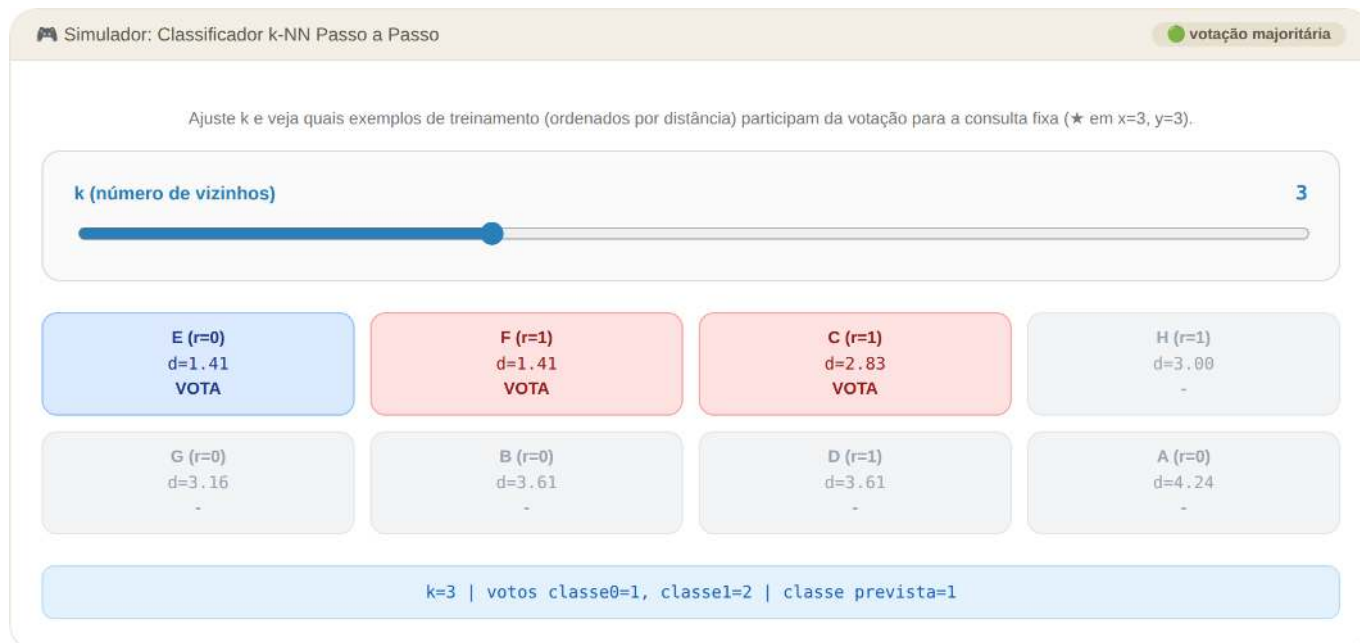


Figura 6.21: Simulador: Classificador k-NN Passo a Passo

```
%%writefile EP07_01.py
# Código Python
```

```
Writing EP07_01.py
```

```
TestSuite("EP07_01.py").run()
```

6.18.2 EP07_02 ● Normalização *Z-score* e Robustez do k-NN a Escalas Distintas

Este exercício revisita o classificador implementado no **EP07_01**, desta vez sob a ótica discutida na seção *O Impacto da Escala e a Normalização de Características* do capítulo: o k-NN decide com base na distância entre vetores, de modo que uma característica medida em uma escala muito maior do que as demais tende a **dominar** o cálculo da distância, mesmo quando ela não é a mais relevante para separar as classes.

Um sistema de inspeção registra, para cada peça, sua **área** (em pixels, podendo chegar a centenas ou milhares) e sua **circularidade** (sempre entre 0 e 1). Você foi encarregado de classificar novas peças por k-NN de duas formas — com e sem a padronização *Z-score* apresentada no capítulo — e de reportar em quais casos as duas abordagens **divergem**.

6.18.2.1 📄 Diretrizes de Implementação

1. **Quantidade e parâmetro:** Ler o inteiro N (número de exemplos de treinamento) e o inteiro ímpar k .

2. **Exemplos de treinamento:** Para cada um dos N exemplos, ler três valores: a área x_1 (real), a circularidade x_2 (real) e o rótulo r (inteiro, 0 ou 1).
3. **Consultas:** Ler o inteiro Q e, em seguida, as coordenadas x_1, x_2 de cada consulta.
4. **Classificação sem normalização:** Para cada consulta, classifique-a por k-NN diretamente sobre (x_1, x_2) , com distância euclidiana e as mesmas regras de desempate do EP07_01 (ordem de leitura para distâncias empatadas; vizinho mais próximo entre classes empatadas na votação).
5. **Parâmetros de normalização:** Calcule a média μ_j e o desvio-padrão **populacional** σ_j (divisão por N , não por $N - 1$ — a mesma convenção adotada pela classe `StandardScaler`) de cada característica $j \in \{1, 2\}$, **exclusivamente sobre o conjunto de treinamento**.
6. **Padronização:** Transforme cada característica de treinamento e de consulta por

$$z_j = \frac{x_j - \mu_j}{\sigma_j}.$$

Se $\sigma_j = 0$ (característica constante no treinamento), defina $z_j = 0$ para todas as amostras dessa característica, evitando a divisão por zero.

7. **Classificação com normalização:** Repita a classificação k-NN do item 4, agora sobre os vetores padronizados (z_1, z_2) , com as mesmas regras de desempate.
8. **Saída:** Para cada consulta, na ordem de entrada, imprimir as classes previstas. Ao final, imprimir o número de consultas em que as duas classificações **divergem**.

6.18.2.2 Restrições Computacionais

- **Ajuste apenas no treino:** μ_j e σ_j são calculados unicamente a partir do conjunto de treinamento e reaplicados às consultas — nunca recalculados a partir delas. Essa prática evita o **vazamento de dados** (*data leakage*), mencionado na seção de normalização do capítulo.
- **Desvio-padrão populacional:** utilize $\sigma_j = \sqrt{\frac{1}{N} \sum_i (x_{i,j} - \mu_j)^2}$, e não a versão amostral (divisão por $N - 1$).
- **Característica constante:** trate $\sigma_j = 0$ como caso especial (item 6); não deve ocorrer erro de divisão por zero.
- **Regras de desempate:** reutilize exatamente as convenções do EP07_01, tanto na seleção dos k vizinhos quanto na votação majoritária.

6.18.2.3 Fundamentação Teórica

Elemento	Papel
Padronização <i>Z-score</i>	Reescala cada característica para média 0 e desvio-padrão 1, tornando escalas heterogêneas comparáveis
Ajuste (<i>fit</i>) apenas no treino	Garante que a avaliação sobre as consultas reflita apenas o que o modelo aprendeu no treinamento
Distância euclidiana sem normalização	Dominada pela característica de maior amplitude — aqui, a área
Predição divergente	Evidencia que a escala das características, e não apenas o algoritmo ou os dados, pode determinar a fronteira de decisão do k-NN

Este exercício reforça, de forma controlada, a razão pela qual o `StandardScaler` é aplicado antes do k-NN ao longo do capítulo: sem essa etapa, características de circularidade — mesmo sendo altamente discriminativas — podem ser praticamente ignoradas pelo classificador diante de uma característica de área com amplitude centenas de vezes maior.

6.18.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros N e k , separados por espaço.
- Próximas N linhas: três valores por linha — x_1 , x_2 (reais) e r (inteiro $\in \{0, 1\}$), separados por espaço.
- Próxima linha: inteiro Q .
- Próximas Q linhas: dois valores por linha — x_1 , x_2 (reais) da consulta, separados por espaço.

Saída:

- Q linhas, no formato `SemNorm=<0|1> ComNorm=<0|1>`, na ordem de entrada das consultas.
- Última linha: `Divergiu: <int>`.

6.18.2.5 Exemplos

Entrada	Saída	Observação
4 3 10 0.9 0 12 0.85 0 900 0.2 1 950 0.25 1 1 500 0.88	SemNorm=1 ComNorm=0 Divergiu: 1	Sem normalização, a área (escala de centenas) domina a distância e a consulta é classificada como classe 1. Após a padronização, a circularidade — muito mais próxima das amostras de classe 0 — passa a pesar de forma comparável, e a predição muda para 0.
2 1 0 0.5 0 100 0.5 1 1 60 0.5	SemNorm=1 ComNorm=1 Divergiu: 0	A circularidade é constante no treinamento ($\sigma_2 = 0$); pela regra do item 6, $z_2 = 0$ para todas as amostras, e a classificação depende apenas da área em ambos os casos.

Simulador: Normalização Z-score e Distância k-NN

padronização de características

Cada exemplo tem duas características — área (pixels) e circularidade (0 a 1). Alterne entre distância bruta e padronizada (Z-score) para a mesma consulta (★) e observe se a classe prevista muda.

★ consulta: área=500, circularidade=0.88

Sem normalização

Com normalização (Z-score)

P3 (r=1)
 área=900, circ=0.2
 d=400.001
VOTA

P4 (r=1)
 área=950, circ=0.25
 d=450.000
VOTA

P2 (r=0)
 área=12, circ=0.85
 d=488.000
VOTA

P1 (r=0)
 área=10, circ=0.9
 d=490.000
VOTA

SEM normalização | k=3 | votos classe0=1, classe1=2 | classe prevista=1

Figura 6.22: Simulador: Efeito da Normalização Z-score na Distância k-NN

```
%%writefile EP07_02.py
# Código Python
```

```
Writing EP07_02.py
```

```
TestSuite("EP07_02.py").run()
```

6.18.3 EP07_03 ● Avaliação por Matriz de Confusão

Um classificador binário de qualidade de solda foi treinado e testado em uma linha de produção. Para cada peça inspecionada, o sistema registrou o rótulo **real** (obtido por um especialista) e o rótulo **previsto** pelo classificador, em que 1 representa “defeituosa” e 0 representa “conforme”.

A gerência de qualidade quer saber não apenas a acurácia do sistema, mas também sua **precisão** (quando o sistema aponta defeito, com que frequência ele está certo?) e sua **revocação** (de todas as peças realmente defeituosas, quantas o sistema conseguiu identificar?) — a distinção discutida na seção de avaliação de classificadores do capítulo.

6.18.3.1 📋 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro N (número de peças inspecionadas).
2. **Dados de cada peça:** Para cada uma das N peças, ler dois inteiros — o rótulo real y e o rótulo previsto $\hat{y}$ (ambos $\in \{0, 1\}$).
3. **Matriz de confusão:** Considerando a classe 1 (defeituosa) como **positiva**, contar:
 - VP (Verdadeiro Positivo): $y = 1$ e $\hat{y} = 1$;
 - FP (Falso Positivo): $y = 0$ e $\hat{y} = 1$;
 - FN (Falso Negativo): $y = 1$ e $\hat{y} = 0$;
 - VN (Verdadeiro Negativo): $y = 0$ e $\hat{y} = 0$.
4. **Métricas:** Calcular

$$\text{Acurácia} = \frac{VP + VN}{N}, \quad \text{Precisão} = \frac{VP}{VP + FP}, \quad \text{Revocação} = \frac{VP}{VP + FN}.$$

5. **Casos degenerados:** Se $VP + FP = 0$ (nenhuma predição positiva), imprima `Precisao: indefinida`. Se $VP + FN = 0$ (nenhum caso positivo real), imprima `Revocacao: indefinida`.
6. **Arredondamento:** Todas as métricas numéricas devem ser arredondadas para 4 casas decimais (*round half away from zero*) apenas na exibição.

6.18.3.2 🚫 Restrições Computacionais

- **Convenção de classe positiva fixa:** a classe 1 é sempre a classe positiva neste exercício, independentemente de sua frequência relativa.
- **Proteção de divisão por zero:** implemente os casos degenerados do item 5 antes de realizar a divisão.
- **Ordem de saída:** siga exatamente a ordem especificada na seção de saída, mesmo nos casos degenerados.

6.18.3.3 🧠 Fundamentação Teórica

Métrica	Pergunta que responde	Sensível a desbalanceamento?
Acurácia	Qual fração das peças foi classificada corretamente?	Sim — pode mascarar erros na classe minoritária
Precisão	Das peças apontadas como defeituosas, quantas realmente são?	Penaliza falsos positivos
Revocação	Das peças realmente defeituosas, quantas foram detectadas?	Penaliza falsos negativos

Em um contexto industrial, uma **revocação** baixa é frequentemente mais grave do que uma **precisão** baixa: deixar passar uma peça defeituosa (falso negativo) tende a ser mais custoso do que inspecionar manualmente uma peça boa apontada por engano (falso positivo).

6.18.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N .
- Próximas N linhas: dois inteiros por linha — y e $\hat{y}$, separados por espaço.

Saída (nesta ordem exata):

```
VP=<int> FP=<int> FN=<int> VN=<int>
Acuracia: <valor ou métrica indefinida>
Precisao: <valor ou indefinida>
Revocacao: <valor ou indefinida>
```

6.18.3.5 Exemplos

Entrada	Saída	Observação
4 1 1 0 1 1 0 0 0	VP=1 FP=1 FN=1 VN=1 Acuracia: 0.5000 Precisao: 0.5000 Revocacao: 0.5000	Um erro de cada tipo.
3 0 0 0 0 0 0	VP=0 FP=0 FN=0 VN=3 Acuracia: 1.0000 Precisao: indefinida Revocacao: indefinida	Nenhum caso positivo real nem previsto.

 Simulador: Precisão x Revocação
 linha de produção

Escolha um cenário de inspeção e observe como acurácia, precisão e revocação reagem de forma diferente.

Cenário A: erros equilibrados

Cenário B: muitos falsos negativos

Cenário C: nenhum defeito real

Cenário D: muitos falsos positivos

Peça 1
real=1, prev=1

Peça 2
real=0, prev=1

Peça 3
real=1, prev=0

Peça 4
real=0, prev=0

VP=1 FP=1 FN=1 VN=1 | Acuracia=0.5000 Precisao=0.5000 Revocacao=0.5000

Figura 6.23: Simulador: Precisão x Revocação

```
%%writefile EP07_03.py
# Código Python
```

```
Writing EP07_03.py
```

```
TestSuite("EP07_03.py").run()
```


6.18.4 EP07_04 ● Codificação Manual do Descritor LBP

A função `local_binary_pattern` do `scikit-image`, utilizada no projeto de classificação de texturas, calcula automaticamente o código LBP de cada pixel de uma imagem. Antes de utilizá-la como uma caixa-preta, você foi encarregado de implementar manualmente o cálculo do código LBP clássico ($P = 8$, $R = 1$) para o pixel central de uma vizinhança 3×3 , exatamente como definido na equação do capítulo.

Além do código, o sistema de inspeção de texturas também precisa saber se aquele padrão é **uniforme** — um padrão é uniforme quando o número de transições ($0 \rightarrow 1$ ou $1 \rightarrow 0$) ao percorrer os 8 bits **circularmente** (voltando do último bit ao primeiro) é **no máximo 2**, propriedade explorada pela variante *uniforme* do LBP mencionada no capítulo.

6.18.4.1 📋 Diretrizes de Implementação

1. **Quantidade:** Ler o inteiro T (número de vizinhanças a processar).
2. **Dados de cada vizinhança:** Para cada uma das T vizinhanças, ler uma matriz 3×3 de inteiros (intensidades), fornecida em 3 linhas de 3 valores cada. O pixel central é a posição `[1][1]`.
3. **Ordem dos vizinhos:** Percorra os 8 vizinhos em sentido **horário**, iniciando no canto superior esquerdo, na seguinte ordem de posições `[linha][coluna]`: `[0][0]`, `[0][1]`, `[0][2]`, `[1][2]`, `[2][2]`, `[2][1]`, `[2][0]`, `[1][0]`. Esse é o índice $p = 0, 1, \dots, 7$ da equação do LBP.
4. **Função limiar:** Para cada vizinho p com intensidade g_p e centro g_c , calcule $s(g_p - g_c)$, que vale 1 se $g_p \geq g_c$ e 0 caso contrário.
5. **Código LBP:** Calcule

$$\text{LBP} = \sum_{p=0}^7 s(g_p - g_c) 2^p.$$

6. **Transições:** Considerando a sequência circular de bits $s_0, s_1, \dots, s_7$ (na ordem do item 3), conte quantos pares consecutivos **adjacentes na sequência circular** (incluindo o par s_7, s_0) diferem entre si.
7. **Classificação:** Se o número de transições for ≤ 2 , classifique como **UNIFORME**; caso contrário, **NAO_UNIFORME**.
8. **Saída:** Para cada vizinhança, na ordem de entrada, imprimir o código LBP (inteiro decimal, 0–255), o número de transições e a classificação.

6.18.4.2 📌 Restrições Computacionais

- **Ordem fixa dos vizinhos:** a ordem do item 3 é obrigatória — invertê-la produz um código numericamente diferente, mesmo representando o mesmo padrão visual.
- **Comparação não estrita:** $s(z) = 1$ quando $z \geq 0$ (o próprio capítulo define a igualdade como incluída no caso 1).
- **Contagem circular:** não esqueça o par que fecha o ciclo (s_7 com s_0); ignorar esse par é um erro comum que classifica incorretamente padrões uniformes.

6.18.4.3 🧠 Fundamentação Teórica

Padrão (bits $s_0 \dots s_7$)	Transições	Interpretação
00000000 ou 11111111	0	Região homogênea (mancha clara ou escura)
00001111	2	Borda simples entre duas regiões
01010101	8	Textura de contraste alternado — não uniforme

Padrões uniformes concentram-se em regiões de textura suave ou bordas simples; padrões não uniformes tendem a corresponder a ruído de alta frequência. Por isso, o histograma LBP *uniforme*, usado no projeto de

classificação de texturas, agrupa todos os padrões não uniformes em um único compartimento, reduzindo a dimensionalidade do descritor.

6.18.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro T .
- Para cada vizinhança: 3 linhas com 3 inteiros cada (matriz 3×3).

Saída:

- T linhas, no formato `LBP=<int> transicoes=<int> <UNIFORME|NAO_UNIFORME>`.

6.18.4.5 Exemplos

Entrada	Saída	Observação
1 10 10 10 10 50 10 10 10 10	LBP=0 transicoes=0 UNIFORME	Centro é o mais claro; todos os vizinhos geram bit 0.
1 90 90 90 10 50 10 90 90 90	LBP=119 transicoes=4 NAO_UNIFORME	Vizinhos claros e escuros alternados na vizinhança.

 Simulador: Código LBP de uma Vizinhança 3x3

P=8, R=1

Clique em uma célula da vizinhança para alternar entre claro e escuro (o centro, em destaque, é fixo) e observe o código LBP resultante. O rótulo pequeno em cada célula é o índice p da equação.

Exemplo 1: mancha homogênea

Exemplo 2: padrão alternado

p0
90

p1
90

p2
90

p7
10

50

p3
10

p6
90

p5
90

p4
90

bits(p0..p7)=11101110 | LBP=119 | transições=4 | NAO_UNIFORME

Figura 6.24: Simulador: Código LBP

```
%%writefile EP07_04.py
# Código Python

Writing EP07_04.py

TestSuite("EP07_04.py").run()
```

Francisco de Assis Zampirolli

UFABC

349

6.18.5 EP07_05 Histograma de Orientações de uma Célula HOG

A função `hog` do `scikit-image`, empregada no projeto de classificação de dígitos, divide a imagem em pequenas **células** e, para cada uma, constrói um histograma das orientações do gradiente ponderado pela magnitude — exatamente a etapa central descrita na seção sobre o descritor HOG do capítulo.

Você foi encarregado de implementar esse cálculo para uma única célula, a partir dos valores de magnitude e orientação do gradiente **já calculados** para cada pixel da célula (dispensando o cálculo das derivadas parciais).

6.18.5.1 Diretrizes de Implementação

1. **Dimensões:** Ler os inteiros n (a célula tem $n \times n$ pixels) e B (número de compartimentos do histograma).
2. **Magnitudes:** Ler n linhas com n valores reais cada, representando $|\nabla f(x, y)|$ para cada pixel da célula.
3. **Orientações:** Ler mais n linhas com n valores reais cada, representando $\theta(x, y)$ em **graus**, já convertido para o intervalo **não sinalizado** $[0^\circ, 180^\circ)$, como convencionalmente utilizado pelo HOG.
4. **Compartimentos:** Os B compartimentos cobrem $[0^\circ, 180^\circ)$ em faixas iguais de largura $180/B$ graus. Um pixel com orientação θ pertence ao compartimento $\lfloor \theta / (180/B) \rfloor$; se esse índice for igual a B (possível apenas quando θ é exatamente 180° , o que não deve ocorrer pela diretriz do item 3), utilize o compartimento $B - 1$.
5. **Histograma bruto:** Para cada pixel, acumule sua **magnitude** (não sua contagem) no compartimento correspondente:

$$H[b] = \sum_{(x,y) : \text{bin}(\theta(x,y))=b} |\nabla f(x, y)|.$$

6. **Normalização L2:** Após construir H , normalize-o para obter $\hat{H}$:

$$\hat{H}[b] = \frac{H[b]}{\sqrt{\sum_{j=0}^{B-1} H[j]^2 + \epsilon}}, \quad \epsilon = 10^{-6}.$$

7. **Saída:** Imprimir o histograma bruto H (arredondado a 2 casas decimais) em uma linha, seguido do histograma normalizado $\hat{H}$ (arredondado a 4 casas decimais) em outra linha, ambos com os B valores separados por espaço, na ordem dos compartimentos.

6.18.5.2 Restrições Computacionais

- **Binning não sinalizado:** o intervalo de orientações é $[0, 180)$, não $[0, 360)$ — gradientes em direções opostas (diferença de 180°) contribuem para o **mesmo** compartimento, convenção padrão do HOG para detecção de objetos.
- **Acumulação por magnitude, não por contagem:** o histograma pondera cada pixel por sua magnitude de gradiente, não simplesmente conta quantos pixels caem em cada compartimento.
- **Constante de estabilização:** o $\epsilon = 10^{-6}$ no denominador da normalização evita divisão por zero quando a célula é completamente homogênea (todas as magnitudes nulas).

6.18.5.3 De onde vêm as matrizes de entrada

Antes deste EP, cada pixel (x, y) da imagem passa por:

$$G_x = f(x+1, y) - f(x-1, y), \quad G_y = f(x, y+1) - f(x, y-1)$$

$$|\nabla f| = \sqrt{G_x^2 + G_y^2}, \quad \theta_{\text{sinal}} = \text{atan2}(G_y, G_x)$$

Como o HOG ignora a polaridade do contraste, o ângulo é dobrado para o intervalo não sinalizado:

$$\theta = \theta_{\text{sinal}} \bmod 180^\circ$$

Repetindo isso para todos os pixels de uma célula $n \times n$, obtêm-se as duas matrizes de entrada deste exercício: **magnitudes** $|\nabla f|$ e **orientações** $\theta \in [0^\circ, 180^\circ)$.

6.18.5.4 Fundamentação Teórica

Etapas	Papel
Magnitude do gradiente	Pondera a contribuição de cada pixel — bordas fortes pesam mais que ruído fraco
Orientação não sinalizada	Torna o descritor invariante à polaridade do contraste (claro→escuro vs. escuro→claro)
Histograma por célula	Resume a distribuição local de bordas em um vetor compacto
Normalização L2	Reduz a sensibilidade do descritor a variações globais de iluminação e contraste

A concatenação dos histogramas normalizados de todas as células da imagem — não implementada neste exercício — forma o vetor de características HOG completo, utilizado como entrada do classificador k-NN no projeto do capítulo.

6.18.5.5 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros n e B .
- Próximas n linhas: n magnitudes reais cada.
- Próximas n linhas: n orientações reais (graus, $[0, 180)$) cada.

Saída:

- Linha 1: os B valores do histograma bruto, arredondados a 2 casas decimais.
- Linha 2: os B valores do histograma normalizado, arredondados a 4 casas decimais.

6.18.5.6 Exemplos

Entrada	Saída	Observação
2 2 1.0 2.0 3.0 4.0 10 100 170 20	5.00 5.00 0.7071 0.7071	Bin de largura 90° : $[0, 90)$ e $[90, 180)$; magnitudes 1 e 4 caem no bin 0, 2 e 3 no bin 1.
2 4 0.0 0.0 0.0 0.0 0 0 0 0	0.00 0.00 0.00 0.00 0.0000 0.0000 0.0000 0.0000	

```
%%writefile EP07_05.py
# Código Python
```

Writing EP07_05.py



Figura 6.25: Simulador Didático: Histograma HOG de uma Celula (mapeamento de ângulos para bins)

```
TestSuite("EP07_05.py").run()
```

6.18.6 EP07_06 Pipeline Completo: Descritores + k-NN + Avaliação Multi-Classe

Este exercício integra as três etapas centrais do capítulo em um único *pipeline*, reproduzindo em miniatura o **Projeto Prático 2** (classificação de texturas sintéticas por LBP): um conjunto de histogramas de descritores **já extraídos** (como se fossem histogramas LBP) é utilizado para treinar um classificador k-NN, que por sua vez é avaliado sobre um conjunto de teste independente por meio de uma matriz de confusão multi-classe.

Diferentemente do EP07_01, aqui o espaço de características tem dimensão arbitrária H (o tamanho do histograma), existem mais de duas classes, e a métrica de distância é um parâmetro de entrada — permitindo reproduzir o experimento de comparação de métricas discutido no capítulo.

6.18.6.1 Diretrizes de Implementação

1. **Classes:** Ler o inteiro C (número de classes) seguido de C nomes de classe (*strings* sem espaço), na ordem em que devem aparecer na matriz de confusão.
2. **Configuração:** Ler o inteiro H (dimensão dos histogramas), a *string* M (métrica: **euclidiana** ou **manhattan**) e o inteiro ímpar k .
3. **Treinamento:** Ler o inteiro N e, em seguida, N linhas, cada uma contendo o nome da classe seguido de H valores reais (o histograma de descritor).
4. **Teste:** Ler o inteiro Q e, em seguida, Q linhas, cada uma contendo o nome da classe **real** seguido de H valores reais (o histograma de descritor da amostra de teste).
5. **Distância:** Para cada amostra de teste, calcule a distância a cada exemplo de treinamento usando a métrica M :

$$d_{\text{euclidiana}}(u, v) = \sqrt{\sum_{j=1}^H (u_j - v_j)^2}, \quad d_{\text{manhattan}}(u, v) = \sum_{j=1}^H |u_j - v_j|.$$

6. **Classificação k-NN:** Selecione os k exemplos de treinamento mais próximos (desempate de distância pela ordem de leitura, como no EP07_01) e classifique pela classe majoritária entre eles. Em caso de **empate de votação** entre duas ou mais classes, escolha a que aparece **primeiro** na lista de classes do item 1.
7. **Matriz de confusão:** Construa uma matriz $C \times C$ em que a linha corresponde à classe real e a coluna à classe prevista, seguindo a ordem de classes do item 1.
8. **Acurácia:** Calcule a acurácia global como a razão entre acertos e Q .
9. **Saída:** Para cada amostra de teste, na ordem de entrada, imprimir a classe prevista. Em seguida, imprimir a matriz de confusão (uma linha por classe real, valores separados por espaço, na ordem das classes). Por fim, imprimir a acurácia arredondada a 4 casas decimais.

6.18.6.2 Restrições Computacionais

- **Métrica selecionável:** implemente ambas as distâncias; a métrica M define qual é utilizada em toda a execução (não é possível misturar métricas na mesma chamada).
- **Desempate de votação determinístico:** o critério do item 6 (ordem da lista de classes) deve ser seguido mesmo quando o empate envolve mais de duas classes.
- **Independência de treino e teste:** não há necessidade de validar que as amostras de teste não aparecem no treino — assuma que a entrada é válida.

6.18.6.3 Fundamentação Teórica

Etapas do exercício	Etapas correspondentes no capítulo
Histogramas de treino/teste já extraídos	<code>descriptor_lbp</code> aplicado às texturas sintéticas
Distância euclidiana ou Manhattan	Parâmetro <code>metric</code> do <code>KNeighborsClassifier</code>

Etapa do exercício	Etapa correspondente no capítulo
Votação majoritária com k vizinhos	<code>KNeighborsClassifier.predict</code>
Matriz de confusão $C \times C$	<code>confusion_matrix</code> do <code>scikit-learn</code>
Acurácia global	<code>accuracy_score</code> do <code>scikit-learn</code>

Este exercício evidencia, de forma controlada, um resultado discutido no capítulo: a **escolha da métrica de distância** e do **valor de k** pode alterar a classe prevista para uma mesma amostra, mesmo mantendo fixo o descritor utilizado — reforçando que, no reconhecimento de padrões clássico, o descritor, a métrica e o classificador formam um sistema interdependente, e não peças isoladas.

6.18.6.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: inteiro C seguido de C nomes de classe.
- Linha 2: inteiro H , *string* M e inteiro k .
- Linha 3: inteiro N .
- Próximas N linhas de treinamento: nome da classe seguido de H reais.
- Próxima linha: inteiro Q .
- Próximas Q linhas de teste: nome da classe real seguido de H reais.

Saída:

- Q linhas com a classe prevista de cada amostra de teste, na ordem de entrada.
- C linhas com a matriz de confusão (uma linha por classe real).
- Última linha: **Acuracia:** <valor>.

6.18.6.5 📌 Exemplos

Entrada (resumida)	Saída	Observação
2 granular listrada	granular	Com $k = 1$, cada teste é classificado pelo vizinho de treino mais próximo.
2 euclidiana 1	listrada	
4	1 0	
granular 0.9 0.1	0 1	
granular 0.8 0.2	Acuracia: 1.0000	
listrada 0.1 0.9		
listrada 0.2 0.8		
2		
granular 0.85 0.15		
listrada 0.15 0.85		

i Note

Este simulador usa um conjunto simplificado de **3 classes** (**granular**, **listrada**, **manchada**) sobre pontos 2D fictícios, apenas para ilustrar o *pipeline* de votação, desempate e matriz de confusão do k-NN. No **EP07_07**, você aplicará essa mesma lógica a um mosaico de imagem real, que introduz uma quarta classe (**xadrez**) e substitui os pontos 2D por histogramas LBP extraídos diretamente dos pixels da imagem.

```
%%writefile EP07_06.py
# Código Python
```

Overwriting EP07_06.py

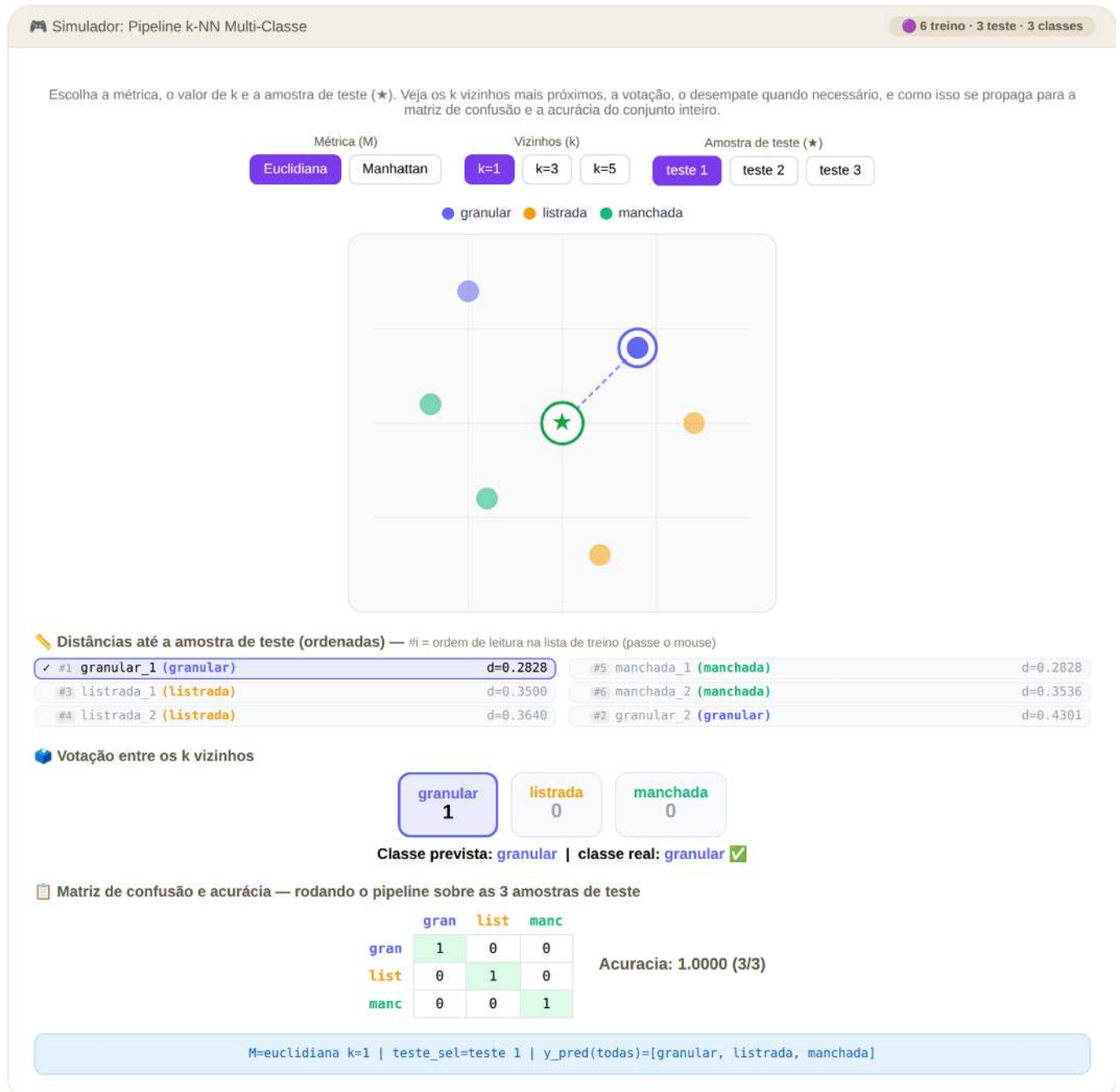


Figura 6.26: Simulador: Pipeline k-NN Multi-Classe (votacao, desempate e matriz de confusao)

```
TestSuite("EP07_06.py").run()
```

6.18.7 EP07_07 ● Classificação Real de um Mosaico de Texturas via LBP + k-NN

Nos exercícios anteriores, o descritor LBP (**EP07_04**) e o classificador k-NN multiclasse (**EP07_06**) foram estudados separadamente, sempre a partir de dados já fornecidos na entrada — vizinhanças 3×3 isoladas ou histogramas previamente extraídos. Neste exercício de encerramento do capítulo, o programa deverá **ler uma imagem real**, no formato **PGM ASCII (P2)**, calcular o descritor LBP diretamente a partir dos pixels e, em seguida, classificar cada região por meio do k-NN, reproduzindo, em escala reduzida, o fluxo completo de um sistema de reconhecimento de texturas. Essa abordagem também antecipa a ideia de **classificação por mosaico de regiões**, relacionada à segmentação semântica estudada em um capítulo posterior.

O simulador interativo do **EP07_06** utilizava apenas três classes (**granular**, **listrada** e **manchada**) representadas por pontos bidimensionais fictícios. Neste exercício, acrescenta-se uma quarta classe, **xadrez**, e os pontos são substituídos por histogramas LBP extraídos de uma imagem real.

A imagem de entrada é um **mosaico** formado por uma grade $G \times G$ de blocos quadrados de $S \times S$ pixels. Cada bloco contém uma amostra de uma das quatro classes de textura sintética do capítulo: **granular**, **listrada**, **manchada** ou **xadrez** (padrão de tabuleiro com intensidades alternadas). Como nos demais exercícios do livro, o carregamento da imagem é realizado pela função didática `mm.readImg`.

💡 Por que um mosaico único, e não várias imagens?

A entrada reúne as $G \times G$ amostras de textura em um único arquivo **PGM**, apenas para simplificar a leitura dos dados e evitar a abertura de vários arquivos. Para o algoritmo, isso não altera o processamento: cada bloco é tratado de forma independente, como se fosse uma imagem isolada. A única exceção é a **exclusão da borda** (item 4 a seguir).

6.18.7.1 📋 Diretrizes de Implementação

1. Leitura das dimensões da imagem

Ler, por meio da entrada padrão, duas linhas contendo, respectivamente, o número de linhas L e o número de colunas C do mosaico (ambos múltiplos do tamanho de bloco S , com $L = C$).

2. Carregamento da imagem

Utilizar a função didática

```
f = mm.readImg(L, C)
```

para ler os $L \times C$ valores de intensidade (tons de cinza, `uint8`) do mosaico.

3. Parâmetros da grade

Ler o inteiro G (número de blocos por lado) e o inteiro S (tamanho do lado de cada bloco, em pixels), satisfazendo $L = C = G \times S$.

4. Cálculo do código LBP por pixel

Para cada pixel **interior** da imagem (ou seja, que não esteja na borda global de `f` — linha ou coluna 0 ou $L - 1 / C - 1$), calcule o código LBP com $P = 8$ vizinhos e raio $R = 1$, percorrendo os vizinhos em sentido **horário** a partir do canto superior esquerdo, exatamente como no EP07_04: `[lin-1][col-1]`, `[lin-1][col]`, `[lin-1][col+1]`, `[lin][col+1]`, `[lin+1][col+1]`, `[lin+1][col]`, `[lin+1][col-1]`, `[lin][col-1]`.

Pixels na borda global da imagem **não** possuem vizinhança completa e devem ser **ignorados** (não contribuem para nenhum histograma). Isso inclui pixels de borda que caem no interior de um bloco (a exclusão é sempre em relação à borda da imagem inteira, não à borda de cada bloco).

5. Histograma LBP uniforme por bloco (10 compartimentos)

Para cada bloco (i, j) da grade $(i, j = 0, \dots, G - 1)$, acumule, entre seus pixels válidos (item 4), um histograma $H^{(i,j)}$ de 10 compartimentos:

- Considerando a sequência circular de bits $s_0, \dots, s_7$ do pixel (mesma regra de transições do EP07_04): se o número de transições for ≤ 2 (padrão **uniforme**), o pixel contribui para o compartimento $\text{popcount}(s_0, \dots, s_7) \in \{0, \dots, 8\}$ (número de bits iguais a 1);
- Caso contrário (padrão **não uniforme**), o pixel contribui para o compartimento 9.

Ao final, normalize o histograma de cada bloco dividindo pelo número de pixels válidos nele contidos, obtendo $\hat{H}^{(i,j)}$, com $\sum_{b=0}^9 \hat{H}^{(i,j)}[b] = 1$.

6. Protótipos de treinamento

Ler o inteiro Ncl (número de classes) seguido de Ncl nomes de classe (ordem que define a matriz de confusão e o desempate de votação, como no EP07_06); em seguida, ler a *string* M (métrica: **euclidiana** ou **manhattan**) e o inteiro ímpar k ; por fim, ler o inteiro N (número de protótipos) e, para cada um, o nome da classe seguido de 10 valores reais (histograma protótipo já normalizado).

7. Classificação k-NN de cada bloco

Para cada bloco, calcule a distância de $\hat{H}^{(i,j)}$ a cada um dos N protótipos, usando a métrica M (mesmas fórmulas do EP07_06). Selecione os k protótipos mais próximos (desempate de distância pela ordem de leitura dos protótipos) e classifique pela classe majoritária (desempate de votação pela ordem das classes do item 6).

8. Rótulos reais e avaliação

Ler, em uma única linha, os $G \times G$ nomes de classe **reais** de cada bloco, em ordem de leitura por linha da grade (bloco $(0, 0)$, $(0, 1)$, ..., $(0, G - 1)$, $(1, 0)$, ...). Construa a matriz de confusão $Ncl \times Ncl$ (linha = classe real, coluna = classe prevista) e a acurácia global.

9. Saída

Imprimir, para cada bloco (na mesma ordem de leitura dos rótulos reais do item 8), a classe prevista. Em seguida, imprimir a matriz de confusão (uma linha por classe real, na ordem do item 6). Por fim, imprimir a acurácia, arredondada a 4 casas decimais.

6.18.7.2 📌 Restrições Computacionais

- **Descritor fixo:** $P = 8$, $R = 1$ e 10 compartimentos (conforme item 5) são fixos neste exercício — não são lidos da entrada.
- **Exclusão de borda global, não de bloco:** um pixel no limite entre dois blocos, mas no interior da imagem, é válido e contribui normalmente para o histograma do bloco ao qual pertence.
- **Ordem de leitura como critério de desempate:** tanto o desempate de distância (item 7) quanto o de votação (item 7) seguem exatamente as mesmas convenções do EP07_01 e do EP07_06.
- **Protótipos como entrada, não aprendidos:** diferentemente do Projeto Prático 2, os histogramas de treinamento são fornecidos diretamente na entrada; o programa não deve gerar texturas sintéticas.

6.18.7.3 🧠 Fundamentação Teórica

Etapa do exercício	Etapa correspondente no capítulo
Leitura da imagem via <code>mm.imread</code>	Aquisição da imagem no <i>pipeline</i> de reconhecimento de padrões

Etapa do exercício	Etapa correspondente no capítulo
Código LBP por pixel (EP07_04)	<code>local_binary_pattern(imagem, P=8, R=1, method="uniform")</code>
Histograma de 10 compartimentos por bloco	Função <code>descritor_lbp</code> do Projeto Prático 2 (<code>bins=10, range=(0, P+2)</code>)
Classificação k-NN com métrica selecionável (EP07_06)	<code>KNeighborsClassifier</code> treinado sobre <code>X_textura</code>
Matriz de confusão $N_{cl} \times N_{cl}$ e acurácia	<code>confusion_matrix</code> e <code>accuracy_score</code> sobre <code>yt_teste</code>

Este exercício evidencia, com pixels reais em vez de valores sintéticos, uma limitação discutida na seção final do capítulo: classes de textura visualmente distintas para um observador humano — como **granular** e **manchada** — podem produzir histogramas LBP semelhantes quando a vizinhança considerada é pequena ($R = 1$), pois ambas apresentam alta frequência de padrões não uniformes na escala de um único pixel. Já a classe **xadrez**, por possuir bordas regulares e repetitivas, tende a ser separada com maior facilidade. Espera-se que a matriz de confusão produzida reflita exatamente esse padrão de confusão parcial.

6.18.7.4 Especificação de Entrada e Saída (VPL)

Entrada:

```
L
C
[matriz L x C da imagem]
G S
Ncl nome_classe_1 ... nome_classe_Ncl
M k
N
nome_classe h0 h1 ... h9      (repetida N vezes)
rotulo(0,0) rotulo(0,1) ... rotulo(G-1,G-1)
```

Saída:

- $G \times G$ linhas com a classe prevista de cada bloco, na ordem de leitura da grade.
- N_{cl} linhas com a matriz de confusão (uma linha por classe real, valores separados por espaço).
- Última linha: **Acuracia:** <valor>.

6.18.7.5 Exemplo (verificação manual)

Para conferir a implementação do descritor antes de testá-la sobre um mosaico completo, considere uma imagem 6×6 **homogênea**, com todos os pixels de intensidade 100, tratada como um único bloco ($G = 1$, $S = 6$). Como todo pixel interior tem os 8 vizinhos com intensidade igual à do centro ($g_p \geq g_c$ em todos os casos), todos os bits s_p valem 1, o número de transições é 0 (uniforme) e o compartimento é `popcount(11111111) = 8`. O histograma do único bloco é, portanto, 0 0 0 0 0 0 0 0 1 0.

Entrada (resumida)	Saída	Observação
6	uniforme	Distância do bloco ao protótipo
6	1 0	uniforme é exatamente 0; a classe
[36 valores iguais a 100]	0 0	outra não aparece no rótulo real,
1 6	Acuracia: 1.0000	por isso sua linha na matriz de
2 uniforme outra		confusão é nula.
euclidiana 1		
2		
uniforme 0 0 0 0 0 0 0 1 0		
outra 0.1 0.1 0.1 0.1 0.1 0.1 0.1 0.1		
0.1 0.1		
uniforme		

6.18.7.6 📁 Arquivos de Referência (.pgm)

Para depuração local, dois mosaicos de teste no padrão ASCII P2 são disponibilizados (anexados a esta entrega; ao integrá-los ao repositório do capítulo, salve-os em `all/cap07/dados/EP07/`):

- 📁 **Caso 1 — Mosaico simples (Caso1_Mosaico_Simples.pgm)**: grade 2×2 de blocos de 24×24 pixels, uma amostra de cada uma das quatro classes, com baixo ruído — útil para validar a leitura da imagem e a lógica de classificação em um cenário controlado.
- 📁 **Caso 2 — Mosaico misto (Caso2_Mosaico_Misto.pgm)**: grade 3×3 de blocos de 16×16 pixels, com classes repetidas e maior variabilidade — cenário em que a confusão entre **granular** e **manchada** discutida na Fundamentação Teórica tende a se manifestar.

A Figure 6.27 exhibe os dois mosaicos, para inspeção visual antes da implementação.

```
import os
import urllib.request
import numpy as np

def garantir_e_baixar_arquivo(nome_arquivo):
    diretorio_local = "dados/EP07"
    caminho_local = os.path.join(diretorio_local, nome_arquivo)

    # Criar o diretório local se ele não existir
    if not os.path.exists(diretorio_local):
        os.makedirs(diretorio_local)

    # Se o arquivo não existir localmente, baixa do repositório remoto
    if not os.path.exists(caminho_local):
        url_base = "https://raw.githubusercontent.com/fzampirolli/"
        url_base += "pdi-vc/master/all/cap07/dados/EP07"
        url_arquivo = f"{url_base}/{nome_arquivo}"
        print(f"Baixando {nome_arquivo} do GitHub...")
        try:
            urllib.request.urlretrieve(url_arquivo, caminho_local)
        except Exception as e:
            raise IOError(f"Erro ao baixar {nome_arquivo} do GitHub. ",
                          "Verifique a conexão ou a URL. Detalhes: {e}")

    return caminho_local

def ler_pgm_p2(caminho):
    with open(caminho) as f:
        linhas = [l for l in f.read().split() if l]
    assert linhas[0] == "P2"
```

```

C, L = int(linhas[1]), int(linhas[2])
maxv = int(linhas[3])
valores = list(map(int, linhas[4:4 + L * C]))
return np.array(valores, dtype=np.uint8).reshape(L, C)

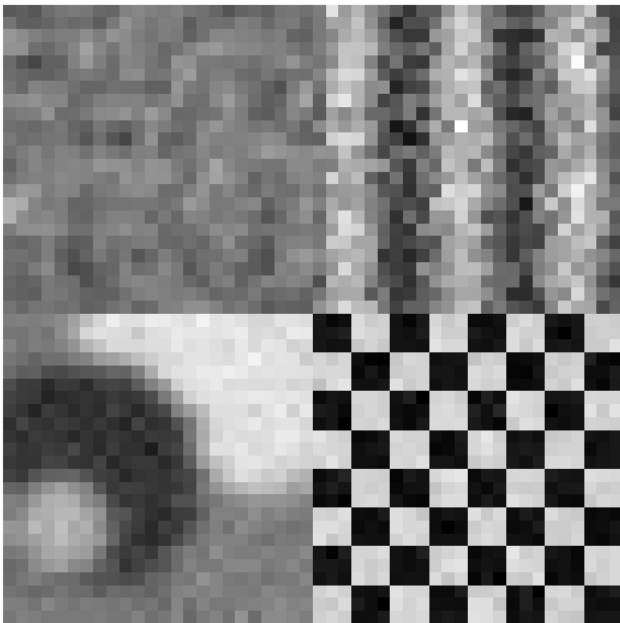
# Garante o download e obtém o caminho correto
arq_caso1 = garantir_e_baixar_arquivo("Caso1_Mosaico_Simples.pgm")
arq_caso2 = garantir_e_baixar_arquivo("Caso2_Mosaico_Misto.pgm")

# Lê as matrizes PGM
caso1 = ler_pgm_p2(arq_caso1)
caso2 = ler_pgm_p2(arq_caso2)

mm.show(
    [caso1, caso2],
    titles=[
        "Caso 1: Mosaico Simples\n(2x2 blocos, 1 amostra/classe)",
        "Caso 2: Mosaico Misto\n(3x3 blocos, classes repetidas)",
    ],
    cols=2,
    figsize=(8, 4),
)

```

Caso 1: Mosaico Simples
(2x2 blocos, 1 amostra/classe)



Caso 2: Mosaico Misto
(3x3 blocos, classes repetidas)

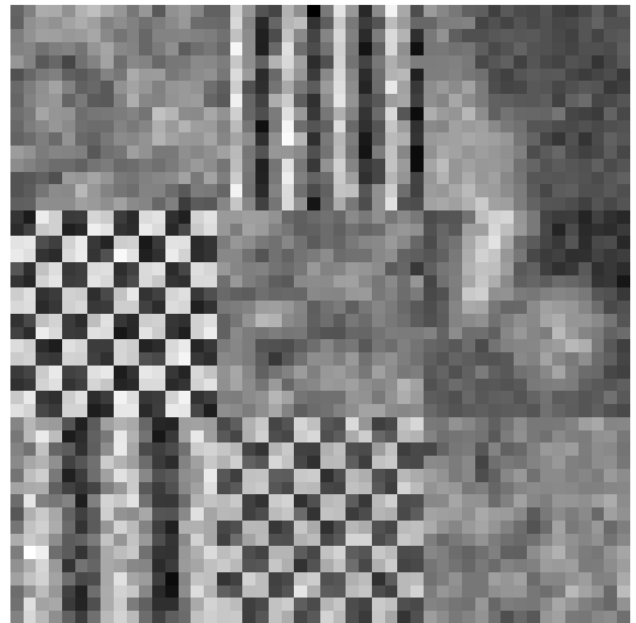


Figura 6.27: Mosaicos de referência (formato PGM ASCII) utilizados no EP07_07. Caso 1: grade 2x2 com uma amostra de cada classe. Caso 2: grade 3x3 com classes repetidas e maior variabilidade.

```

%%writefile EP07_07.py
# Código Python

```

Overwriting EP07_07.py

```

TestSuite("EP07_07.py").run()

```

Simulador: Classificação de Mosaico via LBP + k-NN ● pipeline completo

Mosaico 3x3 de blocos 12x12 (L=C=36). LBP (P=8,R=1) calculado pixel a pixel, com exclusão da borda global. Ajuste k e a métrica e observe a classificação de cada bloco frente a 8 protótipos (2 por classe).

⚠ Texturas sintéticas geradas por código, não os arquivos .pgm reais do EP07_07. Use este simulador para entender o fluxo do algoritmo, não como referência de dificuldade entre as classes.

k (número de vizinhos) 1 **Métrica** euclidiana ▼

	Real: granular Pred: granular ✓	Real: listrada Pred: listrada ✓	Real: manchada Pred: manchada ✓
	Real: xadrez Pred: granular ✗	Real: granular Pred: granular ✓	Real: manchada Pred: manchada ✓
	Real: listrada Pred: listrada ✓	Real: xadrez Pred: granular ✗	Real: granular Pred: granular ✓

Classes preditas (ordem de leitura da grade):
granular listrada manchada granular granular manchada listrada granular granular

Matriz de confusão (linhas=real, colunas=predita; ordem granular,listrada,manchada,xadrez):

```

3 0 0 0
0 2 0 0
0 0 2 0
2 0 0 0

```

Acuracia: 0.7778

Figura 6.28: Simulador: Classificação de um Mosaico de Texturas via LBP + k-NN

Capítulo 7

Compreendendo Cenas: Correspondência de Características, Detecção e Segmentação



No **Capítulo 7**, foi estudado como representar imagens por meio de descritores e utilizar essas representações para tarefas de classificação. Neste capítulo, o problema é ampliado: além de reconhecer padrões, torna-se necessário estabelecer correspondências entre diferentes imagens, localizar automaticamente objetos de interesse e interpretar a organização espacial de uma cena.

Esses problemas constituem algumas das principais tarefas da Visão Computacional e representam uma etapa natural após a classificação de imagens. Para resolvê-los, serão apresentados métodos clássicos de correspondência de características, detecção de objetos e segmentação de imagens, bem como uma visão geral das abordagens modernas baseadas em *Deep Learning*, preparando a transição para o **Capítulo 9**.

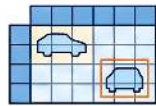
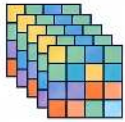
7.1 Objetivos do Capítulo

Ao concluir este capítulo, o estudante deverá ser capaz de:

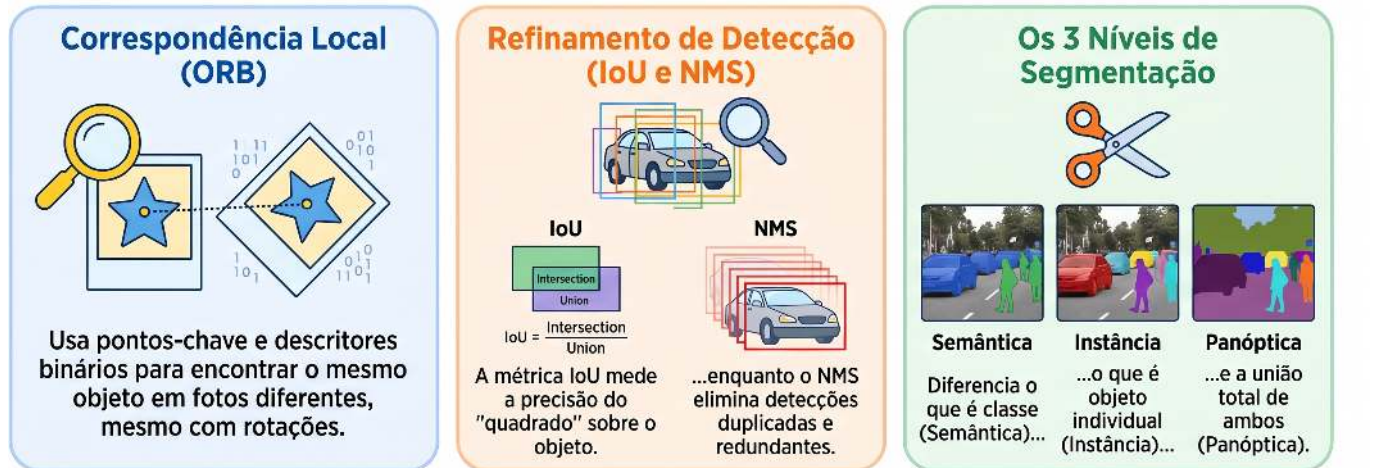
- **Estabelecer correspondências entre imagens** utilizando detectores e descritores locais, estimando transformações geométricas por meio de homografias para o registro e a correção de perspectiva;
- **Localizar objetos de interesse** em imagens utilizando métodos clássicos de detecção e avaliar os resultados por meio de métricas como a *Intersection over Union (IoU)* e da técnica de *Non-Maximum Suppression (NMS)*;
- **Diferenciar os paradigmas de segmentação** — semântica, de instâncias e panóptica — compreendendo que a segmentação panóptica unifica a segmentação semântica e a segmentação de instâncias, proporcionando uma descrição mais completa da cena, e aplicar métodos clássicos de segmentação;
- **Extraír descritores geométricos e topológicos** de objetos segmentados e exportá-los como anotações estruturadas (CSV ou formato YOLO), validando a qualidade das anotações por meio da métrica IoU;
- **Relacionar os métodos clássicos de correspondência, detecção e segmentação** com as abordagens modernas baseadas em *Deep Learning*, estudadas no **Capítulo 9**.

A *Figure 7.1* apresenta uma visão geral dos principais conceitos e das relações entre os tópicos abordados neste capítulo, servindo como um mapa conceitual para orientar a leitura.

Visão Computacional: Do Pixel à Compreensão da Cena



Extrair significado e contexto de imagens através de algoritmos clássicos de correspondência, detecção e segmentação.



© NotebookLM

Figura 7.1: Visão geral dos principais conceitos abordados neste capítulo, incluindo correspondência de características, detecção de objetos, segmentação de imagens e suas relações com abordagens modernas baseadas em *Deep Learning*. **Fonte:** elaborado com auxílio do Gemini Notebook (Google, 2025).

7.2 Configuração do Ambiente

```
import importlib
import subprocess
import sys

for mod, pkg in {
    "cv2": "opencv-python",
    "skimage": "scikit-image",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "matplotlib": "matplotlib",
}.items():
    try:
        importlib.import_module(mod)
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

import cv2
import numpy as np
import importlib
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from skimage import data as skdata
from skimage.filters import threshold_otsu
from skimage.measure import label
from skimage.morphology import remove_small_objects, opening, disk

import os
import urllib.request
```

```
if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
importlib.reload(morph);
from morph import mm

print(f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')}")
```

Ambiente pronto. morph 1.1.7

7.3 Detectores e Descritores Locais de Características

No capítulo anterior, cada imagem foi representada por um único vetor de características utilizado para classificação. Em diversas aplicações, entretanto, é necessário comparar apenas partes da imagem, estabelecendo correspondências entre regiões observadas em diferentes instantes, posições ou pontos de vista. Essa tarefa requer uma representação local da imagem, capaz de identificar estruturas suficientemente distintas para serem reencontradas em outras imagens.

Esse processo é realizado em duas etapas complementares. Inicialmente, um **detector** identifica pontos de interesse (*keypoints*), normalmente associados a cantos ou regiões com variações significativas de intensidade. Em seguida, um **descriptor** representa numericamente a vizinhança de cada ponto detectado, permitindo comparar regiões correspondentes entre diferentes imagens.

Uma vez obtidos os pares (*ponto*, *descriptor*), a correspondência (*matching*) consiste em encontrar, para cada descriptor de uma imagem, o descriptor mais semelhante na outra. Essas correspondências constituem a base de diversas aplicações, como registro de imagens, reconstrução tridimensional, navegação visual e realidade aumentada.

7.3.1 O Algoritmo ORB (*Oriented FAST and Rotated BRIEF*)

Neste capítulo será utilizado o **ORB**, um detector e descriptor local que combina eficiência computacional e robustez a rotações. O algoritmo reúne três componentes principais:

- **FAST** (*Features from Accelerated Segment Test*), responsável pela detecção dos pontos-chave;
- **BRIEF** (*Binary Robust Independent Elementary Features*), responsável pela construção do descriptor binário;
- um mecanismo de **estimativa da orientação da vizinhança**, que torna o descriptor aproximadamente invariante à rotação.

O detector **FAST** percorre todos os pixels da imagem. Para cada pixel candidato, analisa um círculo de **16 pixels** ao seu redor. Se um conjunto de pixels consecutivos apresentar intensidade significativamente maior ou menor que a intensidade do pixel central, esse pixel é considerado um **ponto-chave** (*keypoint*). Em seguida, candidatos muito próximos são filtrados, preservando apenas os mais representativos.

Após a detecção dos pontos-chave, o descriptor **BRIEF** é calculado em uma vizinhança ao redor de cada ponto-chave, e não apenas sobre os 16 pixels utilizados pelo FAST. Nessa região, um **padrão de amostragem**, formado por um conjunto fixo de pares de pontos (x, y) distribuídos em uma janela ao redor do ponto-chave, é utilizado para realizar comparações de intensidade segundo a Equation 7.1. No ORB, a **orientação predominante da vizinhança** é estimada a partir da distribuição das intensidades nessa região, e o padrão de amostragem é rotacionado de acordo com essa orientação. Além disso, o ORB utiliza uma versão otimizada do BRIEF, denominada **rBRIEF** (*Rotated BRIEF*), na qual os pares de pontos são selecionados para produzir descritores mais discriminativos e com baixa correlação entre seus bits.

$$\tau(p; x, y) = \begin{cases} 1, & \text{se } I(x) < I(y), \\ 0, & \text{caso contrário,} \end{cases} \quad (7.1)$$

em que:

- p é o ponto-chave detectado pelo FAST;
- x e y são dois pontos da vizinhança de p , escolhidos pelo padrão de amostragem do BRIEF;
- $I(\cdot)$ representa a intensidade de um pixel;
- $\tau(p; x, y)$ é o resultado da comparação binária entre os pontos x e y .

Cada comparação gera um bit do descritor. A concatenação de todas essas comparações forma o descritor binário associado ao ponto-chave.

Como o descritor é binário, a similaridade entre dois pontos é medida pela **distância de Hamming**, correspondente ao número de bits diferentes entre dois descritores. Essa métrica pode ser calculada de forma muito eficiente por operações lógicas sobre os bits, tornando o ORB adequado para aplicações em tempo real.

7.3.2 Explorando o Simulador do ORB

A Figure 7.3 ilustra, de forma interativa, como o descritor **ORB** é construído. Cada segmento representa um dos **32 pares de pontos** (x, y) utilizados na Equation 7.1. A cor do segmento indica o resultado da comparação de intensidades: **verde** quando $I(x) < I(y)$ (bit igual a 1) e **vermelho** caso contrário (bit igual a 0). A seta **amarela** representa a orientação predominante da vizinhança, estimada a partir do centróide de intensidade. O padrão de amostragem do BRIEF é rotacionado de acordo com essa orientação, tornando o descritor aproximadamente invariante à rotação.

Inicialmente, deixe o ruído em zero e compare a sequência de bits com o *slider* em 0° e depois em 15° (anote o valor do campo “Assinatura do Descritor Binário” em cada caso):

```
0° : 00000000000000100000011111100010
15° : 00000000000000100000011111100010
```

Nesse exemplo, ambos os descritores são idênticos, e o painel “Dist. Hamming” confirma distância zero — evidência de que a estimativa de orientação está compensando corretamente a rotação da imagem.

Agora clique em “Adicionar Ruído” e repita o experimento. Como o ruído é gerado aleatoriamente a cada execução, os valores abaixo são apenas um exemplo — os seus serão diferentes, mas devem apresentar uma distância de Hamming de magnitude semelhante (tipicamente entre 6 e 14 bits, de um total de 32):

```
0° : 01000000001000110110011111101010
15° : 1011000000101110010011111100010
```

O ruído altera parte das comparações de intensidade, modificando alguns bits do descritor. A diferença entre dois descritores é medida pela **distância de Hamming**, correspondente ao número de posições em que os bits diferem. Para os descritores acima, essa distância é igual a **10**.

Uma forma eficiente de calcular essa distância em Python consiste em aplicar a operação **XOR** ($\wedge$), que identifica os bits diferentes, seguida do método `bit_count()`, que contabiliza quantos bits iguais a 1 existem no resultado.

```
def hamming(a: int, b: int) -> int:
    return (a ^ b).bit_count()
a = 0b01000000001000110110011111101010
b = 0b1011000000101110010011111100010
hamming(a, b)
```

```
10
```

A Figure 7.2 compara essa implementação com uma versão baseada na comparação de caracteres e com a implementação otimizada do OpenCV (`cv2.NORM_HAMMING`).

```

import random, timeit, cv2, numpy as np, pandas as pd
import matplotlib.pyplot as plt

BITS, N = 256, 100_000
A = [''.join(random.choice('01') for _ in range(BITS)) for _ in range(N)]
B = [''.join(random.choice('01') for _ in range(BITS)) for _ in range(N)]
Ai, Bi = map(lambda L: [int(x,2) for x in L], (A,B))
Acv = np.array([[int(s[i:i+8],2) for i in range(0,BITS,8)] for s in A], np.uint8)
Bcv = np.array([[int(s[i:i+8],2) for i in range(0,BITS,8)] for s in B], np.uint8)

H = [
    ("Strings", lambda: sum(sum(x!=y for x,y in zip(a,b)) for a,b in zip(A,B))),
    ("XOR+bit_count()", lambda: sum((a^b).bit_count() for a,b in zip(Ai,Bi))),
    ("OpenCV", lambda: sum(cv2.norm(a,b,cv2.NORM_HAMMING) for a,b in zip(Acv,Bcv)))
]

df = pd.DataFrame(
    [(n, timeit.timeit(f, number=1)) for n,f in H],
    columns=["Método", "Tempo (s)"]
)
df["Speedup"] = (df.iloc[0,1]/df["Tempo (s)"]).round(1)

print(df)

plt.figure(figsize=(6,3))
plt.bar(df["Método"], df["Tempo (s)"])
plt.ylabel("Tempo (s)")
plt.show()

```

	Método	Tempo (s)	Speedup
0	Strings	1.353989	1.0
1	XOR+bit_count()	0.011943	113.4
2	OpenCV	0.184508	7.3

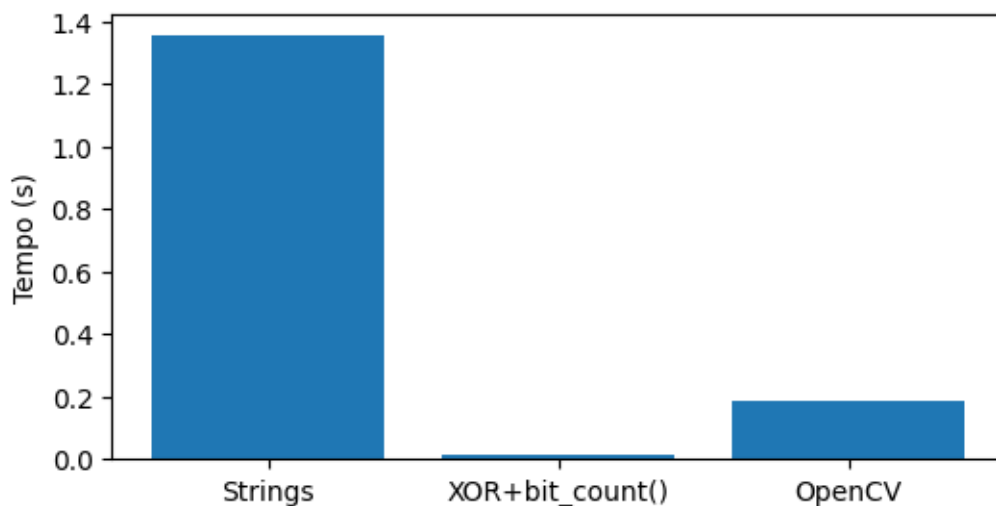


Figura 7.2: Comparação do desempenho de diferentes implementações da distância de Hamming.

7.4 Correspondência de Características e Homografia com ORB

Para ilustrar o processo de correspondência de características, utiliza-se uma cena sintética obtida pela rotação de 20° da imagem original. Essa transformação simula uma segunda captura da mesma cena sob

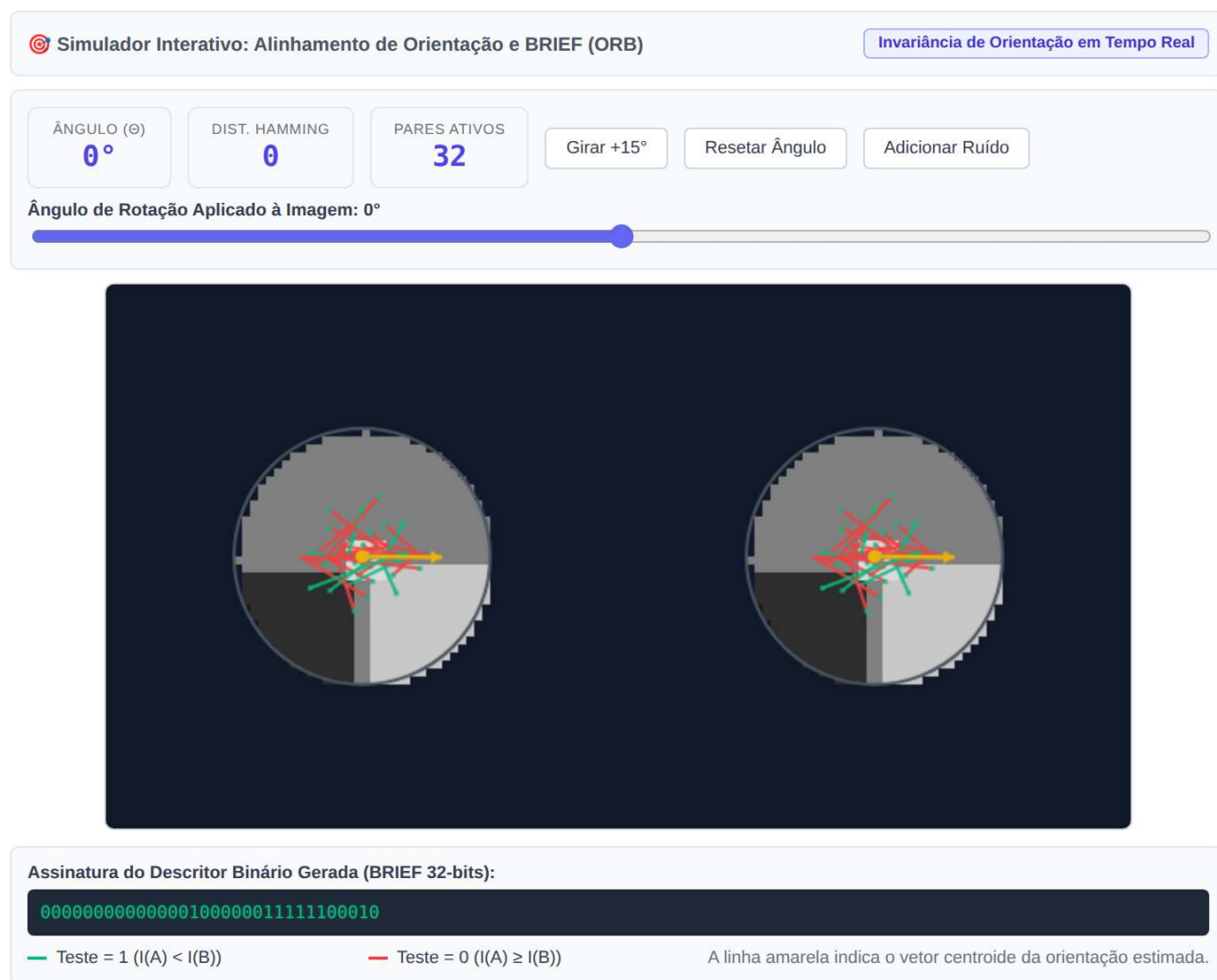


Figura 7.3: Simulador interativo do descritor ORB: explore a lógica de rotação e construção do descritor binário. Altere a rotação para observar como o padrão de amostragem de testes binários do BRIEF (linhas verdes e vermelhas) se orienta dinamicamente para garantir a invariância angular.

outro ponto de vista. A Figure 7.4 apresenta a imagem de referência e sua versão rotacionada, que serão utilizadas nas etapas seguintes.

```
caminho_local = "dados/fig-08-Gemini-faces3.png"
url_imagem = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/all/cap08/"
url_imagem += caminho_local

if not os.path.exists(caminho_local):
    mm.write(mm.read(url_imagem), caminho_local)

img_original = mm.read(caminho_local)
img_cena = mm.rotate(img_original, angle=20)

mm.show([img_original, img_cena],
        titles=["Original", "Rotação (20°)",],
        cols=2, figsize=(8, 4))
```



Figura 7.4: Imagem original gerada pelo Gemini e versão rotacionada (20°), simulando uma mudança de ponto de vista.

7.4.1 Detectando e Correspondendo Características com ORB

Com as duas imagens disponíveis, o **ORB** detecta os pontos-chave e calcula seus descritores binários. Em seguida, o **BFMatcher** da biblioteca **cv2** estabelece as correspondências entre os descritores utilizando a distância de Hamming e a verificação mútua (*cross-check*). Por fim, as correspondências são ordenadas da menor para a maior distância de Hamming, priorizando os pares potencialmente mais confiáveis. No código a seguir, que gera a Figure 7.5, destacam-se os seguintes passos:

1. `cv2.ORB_create(nfeatures=500)` instancia o detector ORB, limitando a busca aos **500 pontos-chave** mais representativos de cada imagem. Essa restrição reduz o custo computacional e evita a seleção de pontos pouco distintivos.
2. `orb.detectAndCompute(...)` executa, em uma única chamada, a detecção dos pontos-chave pelo **FAST** e o cálculo dos descritores pelo **BRIEF orientado**, retornando a lista de pontos-chave (**kp**) e seus descritores binários de **256 bits (des)**.
3. `cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)` cria um comparador por força bruta (*Brute-Force Matcher*), que utiliza a **distância de Hamming** — a mesma métrica explorada na Figure 7.3 — para comparar cada descritor da imagem original com todos os descritores da imagem rotacionada. O parâmetro `crossCheck=True` preserva apenas os pares em que a melhor correspondência é recíproca, isto é, quando o melhor correspondente de A é B e, simultaneamente, o melhor correspondente de B é A. Esse critério elimina grande parte das correspondências ambíguas.
4. `matches = sorted(...)` ordena as correspondências da menor para a maior distância de Hamming. Quanto menor essa distância, maior a similaridade entre os descritores e, consequentemente, maior a probabilidade de a correspondência estar correta.

A Figure 7.5 apresenta apenas as cinco correspondências com menor distância de Hamming. Embora esses pares sejam os mais promissores, ainda não existe qualquer restrição geométrica entre os pontos correspondentes. Como consequência, algumas ligações podem representar falsas correspondências (*false matches*), justificando o uso do **RANSAC** na etapa seguinte para identificar apenas as correspondências geometricamente consistentes.

```
orb = cv2.ORB_create(nfeatures=500)
kp1, des1 = orb.detectAndCompute(img_original, None)
kp2, des2 = orb.detectAndCompute(img_cena, None)
print(f"Pontos de interesse detectados: {len(kp1)} (original), {len(kp2)} (cena)")

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(
    bf.match(des1, des2),
    key=lambda m: m.distance
)
print(f"Correspondências encontradas: {len(matches)}")

def draw_matches_destacado(img1, kp1, img2, kp2, matches, espessura=2, raio_ponto=4,
                           seed=42, cor_fixa=None):
    """Desenha as imagens lado a lado com linhas conectando os pontos correspondentes.
    Se cor_fixa=None, cada correspondência recebe uma cor aleatória (facilita distinguir
    ligações individuais). Se cor_fixa for definida (ex: verde), todas as linhas usam a
    mesma cor - útil para destacar um subconjunto específico, como os inliers do RANSAC.
    """
    h1, w1 = img1.shape[:2]
    h2, w2 = img2.shape[:2]
    h = max(h1, h2)
    canvas = np.zeros((h, w1 + w2, 3), dtype=np.uint8)
    canvas[:h1, :w1] = cv2.cvtColor(img1, cv2.COLOR_GRAY2BGR) if img1.ndim == 2 else img1
    canvas[:h2, w1:w1+w2] = cv2.cvtColor(img2, cv2.COLOR_GRAY2BGR) if img2.ndim == 2 else img2

    rng = np.random.RandomState(seed) # seed fixa = cores reproduzíveis a cada execução

    for m in matches:
        pt1 = tuple(np.round(kp1[m.queryIdx].pt).astype(int))
        pt2 = tuple(np.round(kp2[m.trainIdx].pt).astype(int) + np.array([w1, 0]))

        cor = cor_fixa if cor_fixa is not None else \
            tuple(int(c) for c in rng.randint(60, 256, size=3))

        cv2.line(canvas, pt1, pt2, cor, espessura, lineType=cv2.LINE_AA)
        cv2.circle(canvas, pt1, raio_ponto, cor, -1, lineType=cv2.LINE_AA)
        cv2.circle(canvas, pt2, raio_ponto, cor, -1, lineType=cv2.LINE_AA)

    return canvas

# experimente as 5 piores: matches[-5:]
img_matches = draw_matches_destacado(
    img_original, kp1, img_cena, kp2, matches[:5],
    espessura=10, raio_ponto=15
)
mm.show([img_matches], titles=["Top 5 Correspondências ORB"], cols=1, figsize=(10, 5))
```

Pontos de interesse detectados: 500 (original), 500 (cena)
Correspondências encontradas: 300



Figura 7.5: As 5 melhores correspondências de características ORB entre a imagem original e a cena sintética, antes da filtragem por RANSAC.

A função `draw_matches_destacado()` possui apenas finalidade de visualização: ela posiciona as imagens lado a lado e desenha linhas entre os pares correspondentes, não interferindo na estimação das correspondências.

7.5 Modelagem Matemática: Homografia e RANSAC

7.5.1 Homografia

Nos Exercícios 10 e 11 do Capítulo 2, as funções `cv2.getPerspectiveTransform` e `cv2.warpPerspective` foram utilizadas para corrigir a perspectiva de imagens a partir de **quatro pares de pontos correspondentes** informados manualmente. Neste capítulo, essas correspondências passam a ser obtidas automaticamente pelo ORB, tornando possível estimar a transformação entre duas imagens sem intervenção do usuário.

Matematicamente, essa transformação é descrita por uma **homografia**, representada por uma matriz 3×3 que relaciona as coordenadas de um mesmo plano observado sob diferentes pontos de vista:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

Após a normalização das coordenadas homogêneas, obtém-se o ponto correspondente

$$\left(\frac{x'}{w'}, \frac{y'}{w'} \right).$$

Como a homografia é definida a menos de um fator de escala, ela possui **oito graus de liberdade**. Consequentemente, são necessários, no mínimo, **quatro pares de pontos correspondentes** para estimar seus parâmetros.

Na prática, porém, as correspondências produzidas automaticamente pelo ORB podem conter associações incorretas (*outliers*). Para estimar a homografia de forma confiável mesmo na presença desses erros, utiliza-se o algoritmo **RANSAC**, apresentado na próxima seção.

7.5.2 RANSAC

O **RANSAC** (*Random Sample Consensus*) é um algoritmo de estimação robusta capaz de ajustar um modelo geométrico mesmo na presença de observações incorretas (*outliers*). Neste capítulo, o modelo de interesse é uma **homografia**, estimada a partir das correspondências produzidas pelo ORB.

Em cada iteração, o algoritmo:

1. seleciona aleatoriamente um pequeno subconjunto de correspondências (quatro pares de pontos, no caso da homografia);
2. estima uma homografia candidata a partir desse subconjunto;
3. verifica quais correspondências são compatíveis com essa transformação, classificando-as como *inliers* ou *outliers*;
4. registra a homografia que produz o maior número de *inliers*;
5. reestima a homografia utilizando apenas os *inliers* encontrados.

Embora o exemplo deste capítulo utilize uma homografia, o RANSAC é um algoritmo de propósito geral e pode ser empregado para estimar diversos modelos geométricos, como retas, circunferências, planos e outras transformações. Em todos os casos, o princípio é o mesmo: gerar modelos candidatos a partir de pequenas amostras aleatórias e selecionar aquele que apresenta o maior consenso entre os dados.

Para compreender esse processo de forma gradual, são apresentados dois simuladores.

O primeiro, mostrado na Figure 7.6, utiliza o exemplo mais simples possível: o ajuste de uma **reta** a um conjunto de pontos contendo aproximadamente **25% de outliers**. O objetivo é compreender as etapas fundamentais do algoritmo — amostrar, estimar um modelo, identificar os *inliers* e repetir o processo — sem a complexidade do registro entre imagens.



Figura 7.6: Simulador interativo do algoritmo RANSAC: ajuste o limiar de distância e execute o algoritmo para observar a separação entre *inliers* e *outliers*.

O segundo simulado, apresentado na Figure 7.7, aproxima-se do problema estudado neste capítulo. Em vez de um único conjunto de pontos, são consideradas **duas imagens** contendo correspondências entre pontos-chave. Algumas correspondências estão corretas (*inliers*), enquanto outras são incorretas (*outliers*), resultantes de erros no processo de correspondência dos descritores. Nesse simulador, o modelo estimado é uma **transformação de similaridade** (rotação, escala e translação), mais simples que uma homografia

completa, mas suficiente para ilustrar o problema de registro entre imagens.

Em ambos os simuladores, o algoritmo executado segue exatamente o mesmo princípio utilizado posteriormente para estimar a homografia. A única diferença está no modelo geométrico ajustado.

Ajuste o limiar de distância e execute o algoritmo em cada simulador para observar como o RANSAC identifica os *inliers*, descarta os *outliers* e estima um modelo consistente utilizando apenas as correspondências válidas.

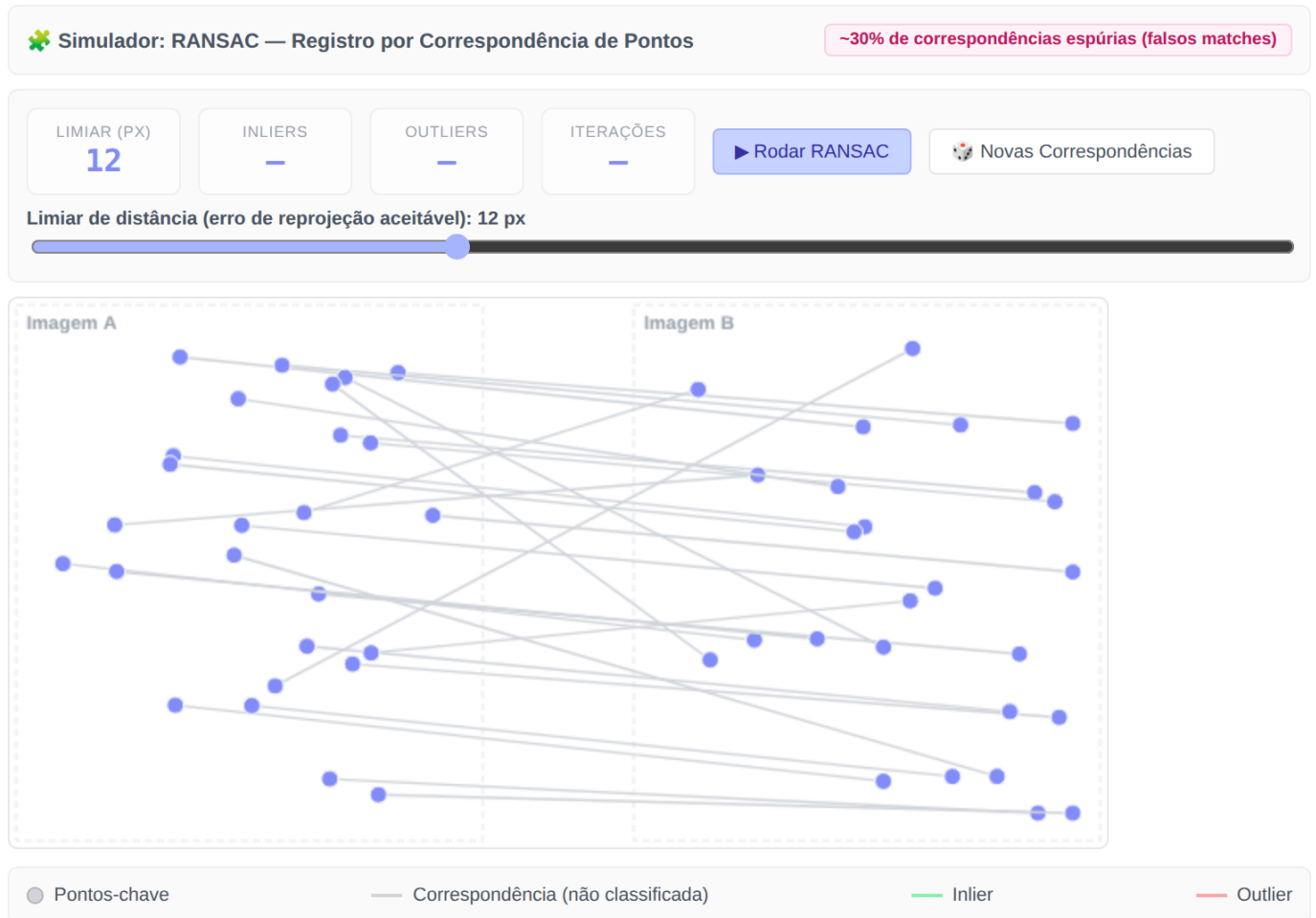


Figura 7.7: Simulador interativo do RANSAC aplicado ao registro de imagens: pontos-chave de duas imagens são casados por um descritor, algumas correspondências são espúrias (*outliers*), e o RANSAC estima a transformação de similaridade que alinha a maioria delas.

7.5.3 Estimando a Homografia com RANSAC

Após obter as correspondências entre os pontos-chave pelo ORB, o próximo passo consiste em estimar a **homografia** entre as duas imagens. Para isso, utiliza-se a função `cv2.findHomography()`, que emprega o algoritmo **RANSAC** para calcular essa transformação e retornar uma máscara indicando quais correspondências foram classificadas como *inliers*.

O código a seguir realiza quatro operações principais:

1. extrai as coordenadas dos pontos correspondentes em cada imagem;
2. estima a homografia por meio de `cv2.findHomography(..., cv2.RANSAC)`;
3. recebe a máscara produzida pelo RANSAC, na qual cada correspondência é classificada como *inlier* ou *outlier*;
4. utiliza essa máscara para selecionar apenas as correspondências classificadas como *inliers*.

A Figure 7.8 apresenta apenas as correspondências classificadas como *inliers*. Observa-se que esses pares

de pontos são compatíveis com uma mesma transformação geométrica, enquanto as correspondências inconsistentes (*outliers*) são descartadas. Como consequência, a homografia estimada representa de forma mais fiel a relação geométrica entre as duas imagens.

```
pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])

H, mascara_inliers = cv2.findHomography(pts1, pts2, cv2.RANSAC, ransacReprojThreshold=5.0)
n_inliers = int(mascara_inliers.sum())
print(f"Matriz de homografia estimada:\n{H}\n")
print(f"Inliers: {n_inliers} de {len(matches)} correspondências \
      ({100*n_inliers/len(matches):.1f}%)")

matches_inliers = [m for m, ok in zip(matches, mascara_inliers.ravel()) if ok]

img_inliers = draw_matches_destacado(
    img_original, kp1, img_cena, kp2, matches_inliers,
    espessura=2, raio_ponto=4, cor_fixa=(0, 200, 0) # verde (BGR)
)
mm.show([img_inliers], titles=[f"Correspondências Inliers (RANSAC) - \
                               {n_inliers}/{len(matches)}"], cols=1, figsize=(10, 5))
```

```
Matriz de homografia estimada:
[[ 9.39407240e-01  3.41706397e-01 -1.77390658e+02]
 [-3.41929137e-01  9.39717641e-01  5.27534018e+02]
 [-1.12175998e-07 -4.86089683e-08  1.00000000e+00]]

Inliers: 285 de 300 correspondências      (95.0%)
```



Figura 7.8: Correspondências classificadas como *inliers* (verde) pelo RANSAC ao estimar a homografia entre as duas imagens.

7.5.4 Registrando a Imagem

Após estimar a homografia, o passo seguinte consiste em utilizá-la para registrar a imagem da cena no sistema de coordenadas da imagem original. Esse processo permite alinhar as duas imagens, facilitando a comparação entre elas.

O código realiza três operações principais:

1. aplica a transformação projetiva por meio de `cv2.warpPerspective()`, utilizando a opção `cv2.WARP_INVERSE_MAP`, que aplica internamente a transformação inversa sem a necessidade de calcular explicitamente H^{-1} ;
2. calcula a diferença absoluta pixel a pixel entre a imagem registrada e a imagem original por meio de `cv2.absdiff()`;

3. exibe a imagem original, a cena rotacionada, a imagem registrada e um mapa das diferenças entre as duas imagens.

A Figure 7.9 apresenta o resultado do registro. Observa-se que a imagem registrada torna-se visualmente muito próxima da imagem original, indicando que a homografia estimada conseguiu alinhar corretamente as duas vistas da mesma cena. O mapa de diferenças evidencia apenas as regiões em que ainda existem pequenas discrepâncias decorrentes de erros de interpolação, quantização e da própria estimativa da homografia.

```
h, w = img_original.shape[:2]

# WARP_INVERSE_MAP: aplica H "de trás para frente", dispensando o cálculo manual de H^-1
img_registrada = cv2.warpPerspective(img_cena, H, (w, h), flags=cv2.WARP_INVERSE_MAP)

erro = cv2.absdiff(img_original, img_registrada)

mm.show( # mostra mm.gray(erro)>10 em níveis de cinza
    [img_original, img_cena, img_registrada, mm.gray(erro)>10],
    titles=["Original", "Cena (rotacionada)", "Registrada", "Diferença"],
    cols=2,
    figsize=(14, 8),
)
```



Figura 7.9: Registro da imagem da cena utilizando a homografia inversa estimada pelo RANSAC.

i 🧠 Por que funciona? — Robustez por consenso

Muitos métodos de estimação ajustam um modelo utilizando **todas** as observações disponíveis, procurando minimizar o erro total entre os dados observados e o modelo ajustado (abordagem conhecida como **mínimos quadrados**). Quando existem *outliers*, essas observações incorretas podem deslocar significativamente o resultado obtido.

O **RANSAC** segue uma estratégia diferente. Em vez de utilizar todos os dados simultaneamente, ele estima sucessivos modelos a partir de pequenas amostras aleatórias. Cada modelo é então avaliado pelo

número de correspondências compatíveis com ele. Ao final das iterações, é selecionado o modelo que apresenta o maior consenso entre os dados, isto é, o maior número de *inliers*.

Dois parâmetros exercem papel fundamental no algoritmo:

- o **limiar de reprojeção**, que define a distância máxima para que uma correspondência seja classificada como *inlier*;
- o **número de iterações**, que deve ser suficientemente grande para aumentar a probabilidade de selecionar pelo menos uma amostra livre de *outliers*.

Embora bastante robusto, o RANSAC pressupõe que exista um modelo geométrico predominante nos dados. Seu desempenho tende a diminuir quando a proporção de *inliers* é muito pequena ou quando diferentes estruturas geométricas coexistem na mesma cena, dificultando a identificação de um único modelo dominante.

7.6 Detecção de Objetos: *Haar Cascade* (Viola-Jones)

A correspondência de características responde à pergunta: “**onde está o mesmo objeto ou padrão observado anteriormente?**”. A **detecção de objetos** resolve um problema mais geral: localizar automaticamente instâncias de uma **categoria** (por exemplo, faces humanas), mesmo que os objetos específicos nunca tenham sido observados durante o treinamento. Enquanto o ORB necessita de duas imagens para estabelecer correspondências entre pontos, o **Haar Cascade** opera sobre uma única imagem, identificando diretamente as regiões candidatas a conter o objeto procurado.

O algoritmo **Haar Cascade**, proposto por Viola e Jones (Viola; Jones, 2001; Viola; Jones, 2004), combina características Haar, imagens integrais e uma cascata de classificadores para realizar a detecção de objetos de forma eficiente. Embora atualmente existam métodos mais recentes baseados em redes neurais convolucionais, o Haar Cascade permanece disponível na biblioteca OpenCV e constitui um exemplo clássico para o estudo de técnicas de detecção de objetos.

Seu funcionamento baseia-se em três componentes principais:

1. **Características Haar:** filtros retangulares simples que medem diferenças de intensidade entre regiões vizinhas da imagem, explorando **padrões de contraste característicos** do objeto, como a região dos olhos geralmente mais escura que a testa;
2. **Imagem integral:** estrutura de dados que permite calcular rapidamente a soma dos pixels de qualquer região retangular da imagem:

$$I_{\text{integral}}(x, y) = \sum_{x' \leq x, y' \leq y} I(x', y'),$$

reduzindo significativamente o custo computacional da avaliação das características Haar. Com essa estrutura, a soma dos pixels de qualquer retângulo pode ser obtida com apenas quatro acessos à imagem integral;

3. **Cascata de classificadores:** durante o treinamento, o algoritmo AdaBoost seleciona e combina classificadores simples em uma sequência de estágios. Na detecção, regiões que claramente **não correspondem ao objeto** são descartadas logo nos primeiros estágios, enquanto apenas as candidatas mais promissoras passam pelas etapas seguintes, mais precisas e computacionalmente mais caras. Essa estratégia permite realizar uma busca eficiente em diferentes posições e escalas da imagem.

O código a seguir, que gera a Figure 7.10, utiliza classificadores treinados previamente e disponibilizados pelo OpenCV para detectar **faces** e, em seguida, restringe a busca por **olhos** apenas ao interior de cada face detectada. Essa estratégia reduz falsos positivos e diminui o custo computacional, pois evita realizar a busca por olhos em toda a imagem.

Durante a detecção, uma janela percorre a imagem em diferentes posições e escalas. A função `detectMultiScale()` realiza essa busca automaticamente. O parâmetro `scaleFactor` controla o fator de redução entre escalas consecutivas da janela, enquanto `minNeighbors` define o número mínimo de detecções vizinhas necessárias para confirmar um objeto, reduzindo detecções espúrias. O parâmetro `minSize` estabelece o menor tamanho de objeto considerado durante a busca.

A função retorna uma lista de retângulos, cada um descrito pelas coordenadas do canto superior esquerdo e pelas dimensões (`x`, `y`, `largura`, `altura`). Esses retângulos delimitam as regiões classificadas como objetos pelo detector e são utilizados para desenhar as caixas mostradas na Figure 7.10.

```
def get_cascade(nome):
    """Baixa (se necessário) e carrega um classificador Haar Cascade do OpenCV."""
    caminho = f"haarcascades/{nome}"
    os.makedirs("haarcascades", exist_ok=True)
    if not os.path.exists(caminho):
        url = f"https://raw.githubusercontent.com/opencv/opencv/master/data/haarcascades/{nome}"
        urllib.request.urlretrieve(url, caminho)
    return cv2.CascadeClassifier(caminho)

# Mais utilizado
#face_cascade = get_cascade("haarcascade_frontalface_default.xml")

# Mais preciso, porém mais lento
face_cascade = get_cascade("haarcascade_frontalface_alt2.xml")

# Compromisso entre velocidade e precisão
#face_cascade = get_cascade("haarcascade_frontalface_alt.xml")

# Muito rápido, porém menos preciso
#face_cascade = get_cascade("haarcascade_frontalface_alt_tree.xml")

# para os olhos, mais utilizado
#eye_cascade = get_cascade("haarcascade_eye.xml")
eye_cascade = get_cascade("haarcascade_eye_tree_eyeglasses.xml")

img_original = mm.read(caminho_local)

# Escala de cinza + equalização de histograma (Capítulo 3), como o Haar Cascade espera
img_rgb = mm.rotate(img_original, angle=0)
img_gray = cv2.equalizeHist(cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY))

# Detecta faces e, dentro de cada uma, tenta detectar os olhos
faces = face_cascade.detectMultiScale(
    img_gray,
    scaleFactor=1.1,      # Passos maiores entre escalas: mais rápido, porém menos sensível
    minNeighbors=3,       # N° mínimo de detecções sobrepostas para confirmar uma face
    minSize=(90, 90)      # Ignora regiões candidatas menores que 30×30 px
)

img_annotada = img_rgb.copy()
for (x, y, w, h) in faces:
    cv2.rectangle(img_annotada, (x, y), (x + w, y + h), (0, 255, 0), 3)
    olhos = eye_cascade.detectMultiScale(
        img_gray[y:y+h, x:x+w],
        scaleFactor=1.02,
        minNeighbors=4,
        minSize=(15, 15)
    )
    for (ex, ey, ew, eh) in olhos:
        cv2.rectangle(img_annotada, (x+ex, y+ey), (x+ex+ew, y+ey+eh), (255, 0, 0), 4)

print(f"Regiões detectadas como face: {len(faces)}")
mm.show([img_annotada], titles=
    ["Detecção Haar Cascade: faces (verde) e olhos (vermelho)"],
    cols=1, figsize=(8, 8))
```

Regiões detectadas como face: 5

Detecção Haar Cascade: faces (verde) e olhos (vermelho)



Figura 7.10: Detecção de faces e olhos com *Haar Cascade*.

i 🧠 Por que funciona? — E por que também falha

Na imagem utilizada neste exemplo, o classificador detecta a face e os olhos, mas também pode marcar uma segunda região sobre parte do fundo da imagem como se fosse uma face. Esse é um exemplo de **falso positivo**: a distribuição local de intensidades nessa região é suficientemente semelhante aos padrões aprendidos durante o treinamento para que a cascata a classifique incorretamente como um rosto.

Esse comportamento evidencia uma das principais limitações do *Haar Cascade*. Como o método baseia sua decisão apenas em características Haar, isto é, diferenças de intensidade entre regiões retangulares, ele não representa explicitamente a forma ou o significado dos objetos presentes na imagem. Assim, texturas e padrões de contraste semelhantes aos encontrados em faces podem produzir detecções incorretas. Além disso, seu desempenho tende a diminuir diante de grandes variações de pose, oclusões, expressões faciais e condições de iluminação diferentes daquelas presentes nos dados utilizados para o treinamento.

Apesar dessas limitações, o *Haar Cascade* permanece útil em aplicações que priorizam baixo custo computacional. Em situações que exigem maior capacidade de generalização diante de variações na aparência dos objetos, métodos modernos baseados em redes neurais profundas costumam apresentar melhor desempenho.

7.7 Detecção de Objetos: Caixas Delimitadoras, IoU e NMS

Embora os métodos de detecção de objetos utilizem estratégias bastante diferentes — do *Haar Cascade* aos detectores modernos baseados em redes neurais profundas —, seus resultados são normalmente representados por **caixas delimitadoras** (*bounding boxes*), definidas pelas coordenadas $(x_{min}, y_{min}, x_{max}, y_{max})$.

7.7.1 Intersection over Union (IoU)

A métrica **Intersection over Union (IoU)** quantifica a sobreposição entre duas caixas delimitadoras, por exemplo, a detecção produzida por um algoritmo e a anotação de referência (*ground truth*):

$$\text{IoU}(A, B) = \frac{\text{área}(A \cap B)}{\text{área}(A \cup B)} \in [0, 1].$$

Quanto mais próximo de 1, maior a concordância entre as caixas; o valor 0 indica ausência de sobreposição. Em avaliações de detectores de objetos, é comum considerar uma detecção correta quando $\text{IoU} \geq 0,5$, embora aplicações específicas possam adotar limiares diferentes.

A métrica IoU não é utilizada apenas para avaliar detectores. Ela também constitui o critério empregado pelo algoritmo de **Supressão de Não-Máximos** para decidir quando duas caixas delimitadoras representam o mesmo objeto e, portanto, uma delas deve ser eliminada.

7.7.2 Supressão de Não-Máximos (NMS)

Durante a detecção, é comum que várias caixas delimitadoras sejam associadas ao mesmo objeto. A **Supressão de Não-Máximos** (*Non-Maximum Suppression*, NMS) elimina essa redundância em três etapas:

1. ordena as detecções pela pontuação de confiança, da maior para a menor;
2. mantém a caixa de maior confiança e elimina aquelas cujo IoU com ela excede um determinado limiar;
3. repete o processo com as caixas restantes até que não existam sobreposições relevantes.

O exemplo da Figure 7.11 ilustra esse procedimento com dois objetos, cada um inicialmente representado por várias caixas sobrepostas. Neste exemplo, cada caixa delimitadora recebe uma **pontuação de confiança**, que representa o grau de confiança do detector de que aquela região contém o objeto procurado.

Em vez de implementar o algoritmo manualmente, utiliza-se a função `cv2.dnn.NMSBoxes`, que implementa a Supressão de Não-Máximos empregada em diversos detectores modernos. A função `desenhar_caixas` exibe, lado a lado, as caixas antes e depois do NMS, evidenciando a redução de cinco detecções para apenas duas, preservando apenas a caixa de maior confiança para cada objeto.

```
def desenhar_caixas(caixas, pontuacoes, indices, cor, tamanho=(450, 450)):
    """Desenha as caixas (e suas pontuações) indicadas em `indices` sobre um fundo branco."""
    tela = np.full((tamanho[1], tamanho[0], 3), 255, dtype=np.uint8)
    for i in indices:
        x0, y0, x1, y1 = caixas[i]
        cv2.rectangle(tela, (x0, y0), (x1, y1), cor, 2)
        cv2.putText(tela, f"{pontuacoes[i]:.2f}", (x0, y0 - 8),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, cor, 1, cv2.LINE_AA)
    return tela

caixas = np.array([
    [50, 50, 150, 150], [60, 55, 155, 145], [58, 60, 160, 150],
    [300, 300, 400, 420], [310, 305, 395, 415],
])
pontuacoes = np.array([0.90, 0.75, 0.60, 0.95, 0.70])

# cv2.dnn.NMSBoxes espera caixas no formato (x, y, largura, altura)
caixas_xywh = np.column_stack([
    caixas[:, 0], caixas[:, 1],
    caixas[:, 2] - caixas[:, 0], caixas[:, 3] - caixas[:, 1]
])

mantidas = cv2.dnn.NMSBoxes(
    bboxes=caixas_xywh.tolist(),
```

```

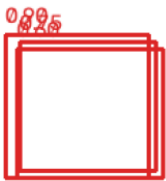
scores=pontuacoes.tolist(),
score_threshold=0.0, # nenhum corte por confiança mínima aqui
nms_threshold=0.5 # limiar de IoU para considerar duas caixas redundantes
).flatten()

img_antes = desenhar_caixas(caixas, pontuacoes, range(len(caixas)), cor=(220, 38, 38))
img_depois = desenhar_caixas(caixas, pontuacoes, mantidas, cor=(22, 163, 74))

mm.show(
    [img_antes, img_depois],
    titles=[f"Antes do NMS ({len(caixas)} caixas)", \
            f"Depois do NMS ({len(mantidas)} caixa(s))"],
    cols=2, figsize=(9, 4.5)
)

```

Antes do NMS (5 caixas)



Depois do NMS (2 caixa(s))



Figura 7.11: Efeito da Supressão de Não-Máximos (NMS): múltiplas detecções redundantes (esquerda) são reduzidas a uma detecção por objeto (direita).

i 🧠 Por que funciona? — Eliminação de Redundâncias

O **NMS** (*Non-Maximum Suppression*) não altera a qualidade das detecções produzidas pelo detector. Sua função é eliminar **caixas delimitadoras redundantes** que representam o mesmo objeto, preservando apenas aquela com maior pontuação de confiança.

Sua eficácia depende do limiar de IoU adotado. Um valor muito baixo pode eliminar caixas correspondentes a objetos diferentes cuja sobreposição seja elevada, enquanto um valor muito alto pode manter várias caixas sobrepostas para um mesmo objeto.

O NMS também **não elimina falsos positivos isolados**. Se uma região incorreta for detectada por apenas uma caixa, ela será preservada, pois não existe outra detecção redundante para compará-la. Por esse motivo, o NMS é aplicado como uma etapa de **pós-processamento**, utilizando apenas as caixas delimitadoras e suas pontuações de confiança produzidas pelo detector.

7.8 Segmentação Visual: Semântica, de Instâncias e Panóptica

Uma caixa delimitadora indica aproximadamente a posição de um objeto, mas não identifica quais pixels lhe pertencem. A **segmentação visual** resolve esse problema atribuindo um rótulo a cada pixel da imagem. Conforme a informação produzida, distinguem-se três paradigmas principais, resumidos na Table 7.1.

Tabela 7.1: Comparação entre os principais paradigmas de segmentação visual.

Paradigma	Pergunta que responde	Distingue objetos da mesma classe?
Semântica	“A que classe pertence cada pixel?”	Não. Todos os pixels de uma mesma classe recebem o mesmo rótulo.
De instâncias	“Quais pixels pertencem a cada objeto individual?”	Sim. Cada objeto recebe um identificador próprio.
Panóptica	Combina as duas anteriores.	Sim. Cada pixel recebe uma classe e, quando aplicável, um identificador de instância.

Na segmentação **semântica**, cada pixel recebe apenas o rótulo de sua classe. Na segmentação **de instâncias**, além da classe, objetos distintos pertencentes à mesma categoria são diferenciados entre si. Já a segmentação **panóptica** combina essas duas informações, atribuindo a cada pixel uma classe e, quando aplicável, um identificador de instância.

7.8.1 Segmentação Semântica por Limiarização

Antes da popularização dos métodos baseados em redes neurais profundas — tema do **Capítulo 9** —, muitas aplicações de segmentação eram resolvidas por técnicas clássicas de processamento de imagens. Um dos exemplos mais simples consiste em separar objeto e fundo por **limiarização**, produzindo uma máscara binária. Essa máscara caracteriza uma **segmentação semântica**, pois cada pixel passa a pertencer a uma das duas classes: **moeda** ou **fundo**.

A Figure 7.12 utiliza a imagem de moedas da biblioteca `scikit-image`. Esse exemplo retoma os conceitos de limiarização e morfologia matemática estudados no **Capítulo 4**, em substituição ao exemplo anterior, que utilizava outra imagem de moedas.

Inicialmente, aplica-se um **fechamento morfológico** para suavizar pequenas imperfeições na superfície das moedas. Em seguida, a **limiarização de Otsu** produz a máscara binária que separa moedas e fundo. Após a remoção dos objetos conectados à borda da imagem, o preenchimento de regiões internas e a eliminação de pequenos ruídos por operações morfológicas, obtém-se uma máscara semântica adequada para as etapas seguintes.

7.8.2 Segmentação de Instâncias por Rotulação

Uma máscara semântica informa apenas a classe de cada pixel, mas não distingue objetos diferentes pertencentes à mesma categoria. Para obter uma **segmentação de instâncias**, aplica-se a **rotulação de componentes conectados**, que atribui um identificador diferente a cada região conexa da máscara binária.

Neste exemplo, cada componente conectado corresponde a uma moeda individual. Assim, a rotulação produz uma segmentação de instâncias, permitindo identificar, contar e medir separadamente cada moeda presente na imagem.

```
img_moedas = skdata.coins()

# 1. Fechamento morfológico: suaviza bordas e preenche pequenas falhas na
# superfície das moedas antes da limiarização
```

```
fechamento = mm.close(img_moedas, mm.sedisk(4))

# 2. Limiarização de Otsu (Capítulo 4): separa moedas (claras) do fundo (escuro)
mascara_bruta = mm.threshold(fechamento)

# 3. Remove objetos conectados à borda da imagem - moedas cortadas nas
#     extremidades não formam instâncias completas e atrapalhariam a contagem
mascara_sem_borda = mm.edgeoff(mascara_bruta)

# 4. Fechamento de buracos: preenche eventuais regiões internas não detectadas
#     pelo threshold, garantindo que cada moeda seja um disco sólido
mascara_semantica = mm.clohole(mascara_sem_borda)

# 5. Abertura morfológica: remove ruído residual e desconecta moedas que
#     porventura se tocam, preparando a máscara para a Rotulação
mascara_limpa = mm.open(mascara_semantica, mm.sedisk(6))

# 6. Rotulação de componentes conectados: cada moeda isolada recebe um
#     identificador de instância distinto
rotulos_instancias = mm.label(mascara_limpa)

print(f"Instâncias (moedas individuais) identificadas: {np.max(rotulos_instancias)}")

mm.show(
    [img_moedas, mascara_bruta, mascara_sem_borda, mascara_semantica,
     mascara_limpa, rotulos_instancias],
    titles=[
        "Original",
        "1. Limiarização (Otsu)",
        "2. Remoção de bordas",
        "3. Preenchimento de buracos\n(máscara semântica: moeda x fundo)",
        "4. Abertura (limpeza)",
        "5. Rotulação\n(segmentação de instâncias)"
    ],
    cols=3, figsize=(10, 6)
)
```

Instâncias (moedas individuais) identificadas: 24

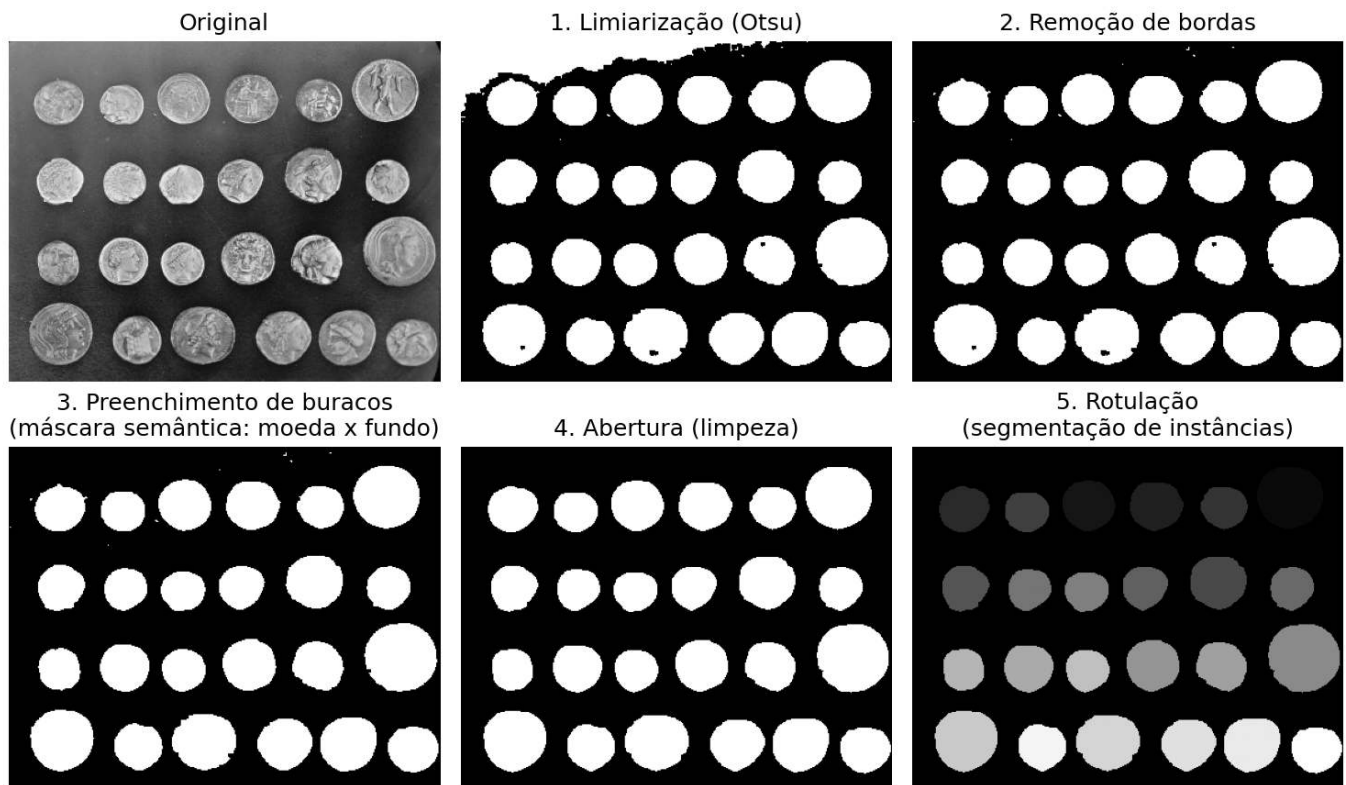


Figura 7.12: Segmentação clássica (não baseada em aprendizado profundo): máscara semântica (moeda x fundo) via limiarização de Otsu, e segmentação de instâncias via rotulação de componentes conectados.

i 🧠 Por que funciona? — E onde a abordagem clássica encontra limitações

Neste exemplo, a segmentação é facilitada pelo contraste entre as moedas e o fundo e pela relativa homogeneidade de intensidade no interior de cada moeda. A limiarização de Otsu separa eficientemente essas duas regiões, enquanto as operações morfológicas removem pequenas imperfeições da máscara binária. Por fim, a rotulação de componentes conectados atribui um identificador distinto a cada região conexa, produzindo uma segmentação de instâncias.

Essa estratégia, entretanto, depende diretamente da qualidade da máscara binária. Se dois objetos estiverem unidos, sobrepostos ou apresentarem contraste insuficiente em relação ao fundo, eles poderão ser representados por uma única região ou deixar de ser segmentados corretamente. Além disso, métodos baseados predominantemente na intensidade dos pixels possuem capacidade limitada para distinguir objetos de classes diferentes com aparência semelhante.

Métodos modernos de segmentação baseados em redes neurais profundas aprendem representações visuais diretamente a partir dos dados de treinamento, o que geralmente lhes confere maior capacidade de lidar com variações de iluminação, textura, forma e oclusão. A segmentação panóptica amplia essa abordagem ao combinar, em uma única representação, a classificação semântica de todos os pixels e a identificação individual dos objetos presentes na cena.

7.8.3 Alternativa: Transformada de Distância + *Watershed*

A abertura morfológica pode separar moedas que se tocam por meio da erosão da máscara binária. Entretanto, essa operação modifica o contorno de todos os objetos, inclusive daqueles que já estavam isolados. Uma alternativa, introduzida no **Capítulo 4**, consiste em combinar a **transformada de distância** com o algoritmo *watershed*, utilizando marcadores obtidos a partir da própria máscara binária.

O procedimento é realizado em três etapas:

1. calcula-se a **transformada de distância** da máscara binária, atribuindo a cada pixel do objeto sua

- distância até o fundo mais próximo;
2. aplica-se um limiar à transformada de distância para obter **marcadores** localizados nas regiões centrais das moedas;
3. utiliza-se o algoritmo *watershed* para expandir esses marcadores até as fronteiras entre os objetos, separando moedas que se encontram em contato.

A Figure 7.13 apresenta essas etapas. Os marcadores são obtidos nas regiões centrais da transformada de distância e utilizados como sementes para o *watershed*, que propaga cada rótulo até encontrar as fronteiras entre objetos vizinhos.

Comparando com a abertura morfológica, ambas as abordagens conseguem separar moedas em contato. Entretanto, como o *watershed* utiliza marcadores para dividir regiões conectadas, sem aplicar erosão diretamente à máscara, os contornos originais tendem a ser mais bem preservados, favorecendo a obtenção de medidas geométricas, como área, perímetro e circularidade.

```
# Reaproveita a máscara semântica ANTES da abertura agressiva (sem erosão do contorno)
mascara_base = mascara_semantica

# 1. Transformada de distância: cada pixel da máscara recebe a distância até o fundo próximo
distancia = mm.dist(mascara_base)

# Marcadores obtidos por limiarização da transformada de distância.
# As regiões centrais das moedas permanecem conectadas e são utilizadas
# como sementes para o algoritmo watershed.
marcadores = mm.label(
    np.uint8(distancia > 0.5 * distancia.max())
)

# 3. Watershed: propaga cada marcador dentro da máscara até os pontos de contato entre moedas
rotulos_watershed = mm.watershed(marcadores, mascara_base)

print(f"Abertura morfológica (mascara_limpa): {np.max(rotulos_instancias)} instâncias")
print(f"Distância + watershed: {np.max(rotulos_watershed)} instâncias")

mm.show(
    [mascara_base, distancia, mm.dil(marcadores, mm.sedisk(5)), rotulos_watershed],
    titles=[
        "Máscara semântica\n(sem abertura, sem erosão)",
        "Transformada de distância",
        "Marcadores\n(máximos regionais)",
        "Watershed\n(instâncias separadas)"
    ],
    cols=4, figsize=(11, 3.2)
)
```

```
Abertura morfológica (mascara_limpa): 24 instâncias
Distância + watershed: 24 instâncias
```

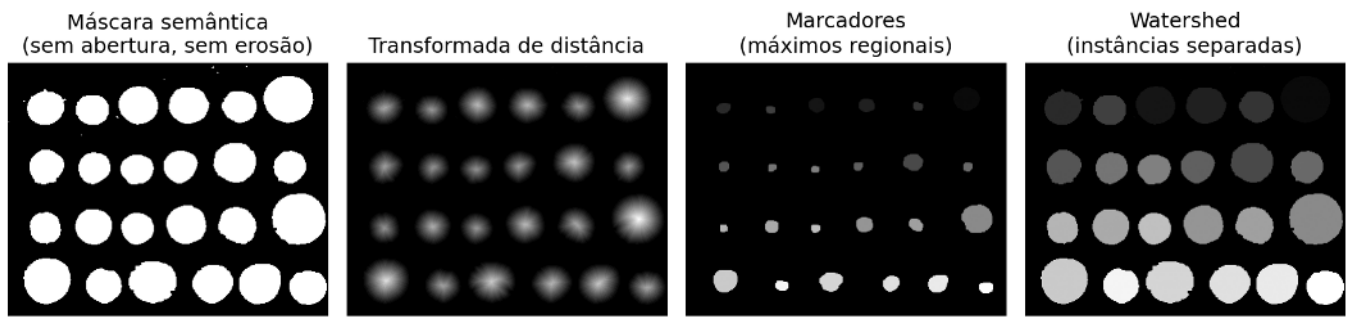


Figura 7.13: Separação de moedas que se tocam via transformada de distância + *watershed*, sem erosão do contorno das moedas.

i 🧠 Por que funciona? — Marcadores e *Watershed*

A transformada de distância atribui valores maiores aos pixels mais afastados do fundo, que normalmente se encontram nas regiões mais centrais dos objetos. Ao aplicar um limiar a essa transformada, obtêm-se marcadores localizados no interior de cada moeda, favorecendo a obtenção de um marcador para cada objeto.

O algoritmo *watershed* utiliza esses marcadores como sementes e propaga seus rótulos até que duas regiões em crescimento se encontrem. Os pontos de encontro entre essas regiões definem as fronteiras entre objetos adjacentes.

O desempenho desse método depende da qualidade dos marcadores. Objetos muito alongados, com formas irregulares ou contendo múltiplos máximos na transformada de distância podem gerar marcadores adicionais, resultando em uma segmentação excessiva (*oversegmentation*). Em aplicações mais complexas, métodos modernos de segmentação de instâncias baseados em redes neurais profundas aprendem diretamente, a partir dos dados de treinamento, representações adequadas para separar os objetos, dispensando a construção explícita de marcadores e outras heurísticas geométricas.

7.8.4 Medição e Extração de Atributos com `mm.measure`

Após a segmentação e a rotulação dos objetos, o próximo passo consiste em medir suas propriedades geométricas. Para isso, a biblioteca `morph` disponibiliza a função `mm.measure`, que recebe uma imagem **binária** e retorna, para cada objeto, um conjunto de descritores geométricos organizado em uma lista de dicionários.

Entre os descritores calculados destacam-se a **área**, o **perímetro**, o **centroide**, a **caixa delimitadora** (*bounding box*), a **circularidade**, a **solidez** e o número de **vértices** do contorno aproximado. Esses vértices são obtidos aproximando-se o contorno por um polígono simplificado, calculado pela função `approxPolyDP`, que preserva a forma geral do objeto utilizando um número reduzido de segmentos.

A circularidade é definida por

$$\text{Circularidade} = \frac{4\pi \cdot \text{Área}}{\text{Perímetro}^2},$$

assumindo valor igual a 1 para um círculo perfeito e valores menores para formas progressivamente menos circulares.

A solidez é dada por

$$\text{Solidez} = \frac{\text{Área}}{\text{Área do Fecho Convexo}},$$

indicando o quanto o objeto preenche seu fecho convexo. Valores próximos de 1 caracterizam objetos convexos, enquanto valores menores indicam a presença de concavidades.

O número de vértices fornece uma indicação da complexidade da forma do objeto. Por exemplo, um triângulo tende a produzir três vértices e um retângulo quatro, enquanto objetos de contorno curvo, como moedas, costumam resultar em polígonos com maior número de vértices, dependendo da precisão adotada na aproximação.

Como as moedas deste exemplo apresentam contornos aproximadamente circulares e convexos, espera-se que a circularidade assuma valores elevados (tipicamente entre 0,8 e 0,9) e que a solidez permaneça muito próxima de 1. O número de vértices depende do parâmetro utilizado na aproximação poligonal (**precision**), aumentando à medida que a aproximação preserva mais detalhes do contorno. Em conjunto, esses descritores podem ser empregados na classificação de objetos, na identificação de componentes espúrios remanescentes da segmentação e na realização de medições quantitativas sobre a imagem.

```
# mm.measure recebe uma imagem binária; cada componente conexo corresponde
# a uma moeda individual.
medidas = mm.measure(mascara_limpa, precision=0.01)

if medidas:
    print(f"Total de objetos catalogados: {len(medidas)}")
    print(f"{'ID':<4}{'Área':<10}{'Perímetro':<12}{'Circularidade':<14}",end="")
    print(f"{'Solidez':<10}{'Vértices':<8}")
    for m in medidas:
        print(f"{m['id']:<4}{m['area']:<10.1f}{m['perimeter']:<12.2f}"
              f"{m['circularity']:<15.3f}{m['solidity']:<10.3f}{m['vertices']:<8}")
else:
    print("Nenhuma medida retornada.")
```

Total de objetos catalogados: 24

ID	Área	Perímetro	Circularidade	Solidez	Vértices
1	1376.5	140.47	0.877	0.979	11
2	1299.5	136.47	0.877	0.978	10
3	1628.0	154.37	0.859	0.980	12
4	1866.0	162.71	0.886	0.985	11
5	2127.0	178.37	0.840	0.973	12
6	2328.0	181.68	0.886	0.983	10
7	1086.5	124.81	0.876	0.979	13
8	1053.5	121.50	0.897	0.983	9
9	1399.5	140.47	0.891	0.985	13
10	1449.0	147.05	0.842	0.972	13
11	1642.5	153.30	0.878	0.982	13
12	2987.5	207.58	0.871	0.981	13
13	1036.5	121.15	0.887	0.981	12
14	1050.5	123.15	0.870	0.979	11
15	1110.0	126.23	0.875	0.975	11
16	1135.0	127.05	0.884	0.986	11
17	1250.5	135.64	0.854	0.975	13
18	1802.0	160.71	0.877	0.982	10
19	1067.5	122.33	0.896	0.986	11
20	1154.5	129.64	0.863	0.977	10
21	1356.0	139.88	0.871	0.978	13
22	1549.5	149.30	0.874	0.982	12
23	1592.0	149.54	0.895	0.984	13
24	2509.5	187.92	0.893	0.987	14

Observa-se que foram identificadas **24 moedas**. As medidas de solidez permanecem próximas de 1, indicando contornos essencialmente convexos, enquanto a circularidade varia entre aproximadamente 0,84 e 0,90 devido às irregularidades do contorno discretizado. O número de vértices varia entre 9 e 14, refletindo a aproximação poligonal utilizada para representar cada moeda.

7.8.5 Exportação de Anotações: de `mm.measure` ao Formato YOLO

Além dos descritores geométricos, `mm.measure` calcula, para cada objeto, sua **caixa delimitadora** (*bounding box*). Essas informações podem ser exportadas automaticamente pela função `mm.saveMeasures`, permitindo gerar arquivos de anotação para diferentes aplicações sem a necessidade de rotulação manual.

A função suporta três formatos de saída:

- `fmt="csv"`: exporta todos os descritores geométricos, sendo útil para análise exploratória, medição e classificação baseada em atributos;
- `fmt="txt"`: grava os descritores em formato tabular simples;
- `fmt="yolo"`: gera anotações compatíveis com o formato utilizado pelos detectores da família **YOLO** (*You Only Look Once*).

No formato YOLO, cada objeto é representado por uma linha contendo

$$\text{classe } x_c \ y_c \ w_n \ h_n,$$

em que (x_c, y_c) representa o centro da caixa delimitadora e (w_n, h_n) suas dimensões, todos normalizados pelas dimensões da imagem:

$$x_c = \frac{x + w/2}{W}, \quad y_c = \frac{y + h/2}{H}, \quad w_n = \frac{w}{W}, \quad h_n = \frac{h}{H},$$

em que (x, y) correspondem às coordenadas do canto superior esquerdo da caixa delimitadora, (w, h) às suas dimensões em pixels e (W, H) à largura e à altura da imagem. A normalização torna as anotações independentes da resolução da imagem, permitindo utilizar o mesmo formato em imagens de diferentes tamanhos.

O código a seguir exporta as medidas extraídas das moedas para os formatos **CSV** e **YOLO**. A saída em CSV preserva todos os descritores geométricos calculados por `mm.measure`, enquanto o arquivo no formato YOLO contém apenas a classe e a caixa delimitadora normalizada de cada objeto, conforme exigido pelos detectores dessa família.

Essa representação será utilizada novamente no **Capítulo 9**, dedicado aos métodos de detecção de objetos baseados em *Deep Learning*.

```
# Exporta as medidas extraídas das moedas em dois formatos de anotação
altura_img, largura_img = img_moedas.shape[:2]

mm.saveMeasures("moedas.csv", medidas, fmt="csv")
mm.saveMeasures("moedas.txt", medidas, fmt="yolo",
                img_width=largura_img, img_height=altura_img, class_id=0)

print("--- Trecho de moedas.csv ---")
with open("moedas.csv") as f:
    for linha in f.readlines()[:4]:
        print(linha.rstrip())

print("\n--- Trecho de moedas.txt (formato YOLO) ---")
with open("moedas.txt") as f:
    for linha in f.readlines()[:4]:
        print(linha.rstrip())
```

```
--- Trecho de moedas.csv ---
id,area,perimeter,cx,cy,x,w,h,circularity,solidity,vertices
1,1376.5,140.47,359.57,268.85,338,250,45,41,0.877,0.979,11
2,1299.5,136.47,115.16,266.56,95,247,42,42,0.877,0.978,10
3,1628.0,154.37,245.55,264.02,222,243,49,46,0.859,0.98,12
```

```
--- Trecho de moedas.txt (formato YOLO) ---
0 0.938802 0.892739 0.117188 0.135314
0 0.302083 0.884488 0.109375 0.138614
0 0.641927 0.877888 0.127604 0.151815
0 0.789062 0.877888 0.130208 0.158416
```

7.8.6 Validação de Anotações por IoU: `mm.verifyBoundingBox`

Antes de utilizar anotações no treinamento ou na avaliação de modelos, é importante verificar se elas concordam com um conjunto de referência (*ground truth*).

A função `mm.verifyBoundingBox` realiza essa comparação utilizando a métrica **IoU** (*Intersection over Union*), apresentada anteriormente neste capítulo. Para cada caixa delimitadora candidata, a função calcula sua sobreposição com as caixas do gabarito e contabiliza aquelas cujo IoU é superior a um limiar especificado.

No exemplo a seguir, constrói-se um **gabarito sintético** a partir das cinco primeiras caixas delimitadoras obtidas por `mm.measure`. Como as anotações comparadas são as mesmas utilizadas para construir o gabarito, espera-se que cada objeto encontre exatamente uma correspondência com $\text{IoU} \geq 0,5$. O objetivo é apenas ilustrar o funcionamento da função `mm.verifyBoundingBox`; em aplicações reais, o gabarito deve ser obtido independentemente, por exemplo, por anotação manual.

```
# Matriz de gabaritos sintética no formato (classe, x1, y1, x2, y2), normalizado
gabaritos = np.array([
    [0, *(m["bbox"][0] + d) / largura_img for d in (0,)],
    (m["bbox"][1]) / altura_img,
    (m["bbox"][0] + m["bbox"][2]) / largura_img,
    (m["bbox"][1] + m["bbox"][3]) / altura_img]
    for m in medidas[:5]
])

acertos = 0
for m in medidas[:5]:
    correspondencias = mm.verifyBoundingBox(
        object_id=0, bbox=m["bbox"], matrix=gabaritos,
        width=largura_img, height=altura_img, threshold=0.5
    )
    acertos += int(correspondencias > 0)
    print(f"Objeto {m['id']}: {correspondencias} gabarito(s) casam com IoU >= 0.5")

print(f"\nTotal de objetos validados: {acertos}/{len(medidas[:5])}")
```

```
Objeto 1: 1 gabarito(s) casam com IoU >= 0.5
Objeto 2: 1 gabarito(s) casam com IoU >= 0.5
Objeto 3: 1 gabarito(s) casam com IoU >= 0.5
Objeto 4: 1 gabarito(s) casam com IoU >= 0.5
Objeto 5: 1 gabarito(s) casam com IoU >= 0.5

Total de objetos validados: 5/5
```

Observa-se que os cinco objetos foram corretamente associados aos respectivos gabaritos, resultando em cinco correspondências válidas. Em aplicações reais, essa mesma estratégia pode ser empregada para comparar automaticamente as caixas delimitadoras produzidas por um detector com anotações de referência, permitindo avaliar quantitativamente seu desempenho por meio da métrica IoU.

7.8.7 Visualização de Anotações Salvas: `mm.showBoundingBox`

Após exportar as anotações, é conveniente verificar se as caixas delimitadoras foram gravadas corretamente. Para isso, a biblioteca `morph` disponibiliza a função `mm.showBoundingBox`, ilustrada na Figure 7.14.

A função lê um arquivo de anotações nos formatos `yolo`, `csv` ou `txt`, reconstrói as **caixas delimitadoras** (*bounding boxes*) e as sobrepõe à imagem original, facilitando a inspeção visual do resultado. Dessa forma, torna-se possível confirmar rapidamente se as anotações estão alinhadas aos objetos, sem a necessidade de examiná-las manualmente.

Essa função complementa `mm.verifyBoundingBox`. Enquanto `mm.verifyBoundingBox` realiza uma **validação quantitativa**, comparando as anotações com um conjunto de referência por meio da métrica IoU, `mm.showBoundingBox` fornece uma **validação qualitativa**, permitindo inspecionar visualmente as caixas delimitadoras reconstruídas.

```
caminho_yolo = "moedas.txt"
img_annotada = mm.showBoundingBox(img_moedas, caminho_yolo, fmt="yolo")
```

Bounding boxes (moedas.txt)

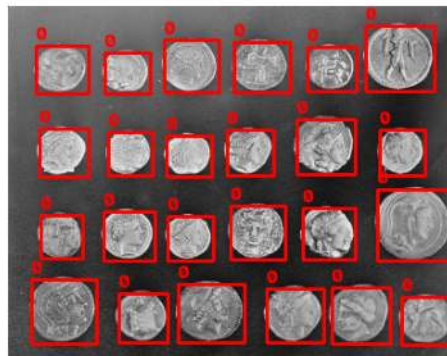


Figura 7.14: *Bounding boxes* reconstruídas a partir do arquivo de anotações YOLO exportado por `mm.saveMeasures`, sobrepostas à imagem original das moedas.

Por que funciona? — Da Segmentação às Anotações

Após segmentar uma imagem, é possível medir automaticamente cada objeto (`mm.measure`) e converter essas medidas em anotações (`mm.saveMeasures`) para diferentes formatos, incluindo o utilizado pelos detectores da família YOLO. Esse procedimento permite gerar automaticamente conjuntos iniciais de anotações em cenários controlados, reduzindo significativamente o trabalho de rotulação manual.

Antes de utilizar essas anotações no treinamento ou na avaliação de modelos, recomenda-se compará-las com um conjunto de referência (*ground truth*). A função `mm.verifyBoundingBox` automatiza essa etapa por meio da métrica **IoU**, permitindo quantificar a concordância entre as caixas delimitadoras geradas e as anotações de referência. Complementarmente, `mm.showBoundingBox` possibilita uma inspeção visual das anotações reconstruídas sobre a imagem original, facilitando a identificação de erros de normalização, posicionamento ou ordenação das coordenadas que podem não ser evidentes em uma validação apenas numérica.

Em conjunto, `mm.measure`, `mm.saveMeasures`, `mm.verifyBoundingBox` e `mm.showBoundingBox` implementam um fluxo completo para medir objetos segmentados, gerar anotações, avaliá-las e inspecioná-las visualmente. Quando a segmentação produz resultados confiáveis, esse fluxo pode reduzir significativamente a necessidade de rotulação manual na construção de conjuntos de dados.

7.9 Visão Geral dos Modelos Modernos

As técnicas apresentadas neste capítulo — como detectores em cascata, segmentação baseada em limiarização e pós-processamento por IoU e NMS — permanecem importantes para compreender os fundamentos da Visão Computacional. Atualmente, entretanto, muitas aplicações utilizam modelos de **Aprendizado Profundo**, capazes de aprender automaticamente representações discriminativas a partir de grandes conjuntos de dados, dispensando a definição manual de características.

A Table 7.2 apresenta algumas arquiteturas representativas para detecção e segmentação de imagens. O objetivo é apenas situá-las no contexto das tarefas estudadas neste capítulo. Seus princípios de funcionamento, treinamento e aplicação serão discutidos em detalhes no **Capítulo 9**.

Tabela 7.2: Arquiteturas representativas de Aprendizado Profundo para detecção e segmentação de imagens.

Modelo	Tarefa	Ideia central
YOLO (<i>You Only Look Once</i>)	Detecção de objetos	Detecta objetos em uma única passada pela rede, estimando classes e caixas delimitadoras. Versões recentes, como o YOLO26 (2026), eliminam a etapa de <i>Non-Maximum Suppression</i> (NMS), tornando a inferência totalmente <i>end-to-end</i> .
Faster R-CNN	Detecção de objetos	Gera regiões candidatas e as refina antes da classificação, priorizando precisão.
SSD (<i>Single Shot Detector</i>)	Detecção de objetos	Detecta objetos em diferentes escalas em uma única passada, buscando equilíbrio entre velocidade e precisão.
U-Net	Segmentação semântica	Produz uma máscara que classifica cada pixel da imagem segundo sua classe.
Mask R-CNN	Segmentação de instâncias	Estende o Faster R-CNN acrescentando uma máscara individual para cada objeto detectado.
Segment Anything (SAM)	Segmentação orientada por <i>prompts</i>	Segmenta objetos a partir de indicações como pontos, caixas delimitadoras ou máscaras. Evoluiu para o SAM 2 , com suporte a vídeo e rastreamento temporal de objetos, e o SAM 3 , que permite segmentação guiada diretamente por descrições em texto.

Observa-se uma evolução das tarefas abordadas neste capítulo. Modelos como **YOLO**, **Faster R-CNN** e **SSD** localizam objetos por meio de caixas delimitadoras. O **Mask R-CNN** amplia essa capacidade ao produzir também uma máscara para cada instância detectada. Já modelos mais recentes, como o **Segment Anything (SAM)**, permitem segmentar objetos a partir de diferentes tipos de *prompts*, tornando o processo mais flexível.

Essa evolução acompanha a sequência de conceitos desenvolvida ao longo do capítulo: da correspondência de características e da localização aproximada por caixas delimitadoras até a segmentação precisa dos pixels pertencentes a cada objeto. No **Capítulo 9**, essas tarefas serão revisitadas sob a perspectiva do Aprendizado Profundo, explorando como redes neurais modernas aprendem automaticamente representações capazes de superar muitas das limitações dos métodos clássicos apresentados aqui.

7.10 Limitações das Abordagens Clássicas e Motivação para o *Deep Learning*

Os exemplos desenvolvidos neste capítulo ilustram três problemas fundamentais da Visão Computacional: **correspondência de características**, **detecção de objetos** e **segmentação de imagens**. Embora as técnicas clássicas apresentadas sejam eficientes em diversos cenários, todas compartilham uma limitação importante: baseiam-se em características definidas manualmente para descrever ou identificar os objetos de interesse.

- O **ORB** utiliza descritores binários capazes de estabelecer correspondências entre pontos sob rotações e mudanças moderadas de escala, mas não foi concebido para reconhecer categorias de objetos.
- O **Haar Cascade** emprega um conjunto fixo de filtros retangulares, funcionando bem para objetos com aparência relativamente padronizada, como faces frontais, mas tornando-se mais suscetível a falsos positivos em cenas complexas.
- A **segmentação por limiarização** explora diferenças de intensidade entre objeto e fundo, sendo adequada para imagens com bom contraste, porém limitada diante de variações de iluminação, texturas ou cenas contendo múltiplas classes de objetos.

Em todos esses casos, o desempenho depende da capacidade das características escolhidas representarem adequadamente a variabilidade presente nas imagens. Em aplicações reais, fatores como mudanças de pose, iluminação, escala, oclusões e diversidade de categorias tornam essa tarefa cada vez mais difícil, reduzindo a generalização dos métodos clássicos.

O **Capítulo 9** apresenta uma abordagem diferente para esses problemas por meio das **Redes Neurais Convolucionais (CNNs)**. Em vez de utilizar características definidas manualmente, esses modelos aprendem automaticamente, a partir de grandes conjuntos de dados, representações adequadas para cada tarefa. As arquiteturas modernas apresentadas na seção anterior — como **YOLO**, **Faster R-CNN**, **U-Net**, **Mask R-CNN** e **Segment Anything (SAM)** — seguem esse princípio e representam uma evolução das técnicas clássicas estudadas neste capítulo, ampliando sua capacidade de lidar com a diversidade e a complexidade das imagens do mundo real.

7.11 Resumo

Neste capítulo foram estudados métodos clássicos de Visão Computacional para **correspondência de características**, **detecção de objetos** e **segmentação de imagens**. Ao final, foram apresentadas algumas arquiteturas modernas baseadas em Aprendizado Profundo, preparando a transição para o **Capítulo 9**. Os principais conceitos abordados foram:

- **ORB**: detecção de pontos-chave, construção de descritores binários e correspondência de características por meio da distância de Hamming.
- **Homografia e RANSAC**: estimativa robusta de transformações projetivas para registro de imagens e eliminação de correspondências inconsistentes.
- **Haar Cascade**: detecção de objetos utilizando características Haar, imagem integral e classificadores em cascata.
- **Segmentação clássica**: limiarização, operações morfológicas, rotulação de componentes conectados, transformada de distância e algoritmo *watershed* para segmentação de instâncias.
- **Medição e anotação de objetos**: extração de atributos geométricos com `mm.measure`, exportação automática de anotações (`mm.saveMeasures`) e validação por IoU (`mm.verifyBoundingBox`) e inspeção visual (`mm.showBoundingBox`).
- **Avaliação de detecções**: utilização da métrica **Intersection over Union (IoU)** e da **Supressão de Não-Máximos (NMS)**.
- **Modelos modernos**: visão geral das arquiteturas **YOLO**, **Faster R-CNN**, **SSD**, **U-Net**, **Mask R-CNN** e **Segment Anything (SAM)**, que servirão de base para o estudo dos métodos de Aprendizado Profundo no **Capítulo 9**.

Próximos Passos

Neste capítulo foram apresentados métodos clássicos para resolver problemas de **correspondência de características**, **detecção de objetos** e **segmentação de imagens**, baseados em descritores, filtros e modelos geométricos definidos manualmente.

No **Capítulo 9**, esses mesmos problemas serão revisitados sob a perspectiva do **Aprendizado Profundo**, com ênfase nas **Redes Neurais Convolucionais (CNNs)**. Em vez de utilizar características projetadas manualmente, esses modelos aprendem automaticamente representações a partir de grandes conjuntos de dados, constituindo a base dos principais sistemas modernos de Visão Computacional para detecção, segmentação e reconhecimento de objetos.

7.12 Uso do Gemini Notebook como Apoio ao Estudo

O **Gemini Notebook** pode ser utilizado como ferramenta complementar para revisar os conceitos apresentados neste capítulo. A partir do conteúdo disponibilizado como referência, ele permite responder perguntas, elaborar resumos, esclarecer dúvidas e explorar os tópicos de forma interativa.

Acesso ao Notebook do Capítulo



ACESSAR Gemini Notebook: CAPÍTULO 08

As respostas geradas pelo Gemini Notebook são produzidas automaticamente e podem conter imprecisões. Utilize-as como material complementar de estudo, confrontando as informações com o conteúdo deste livro e, quando necessário, com outras fontes acadêmicas.

7.13 Lista de Exercícios

Os exercícios a seguir consolidam os conceitos apresentados neste capítulo por meio de adaptações, experimentos e extensões dos algoritmos desenvolvidos ao longo do texto, utilizando a biblioteca didática **morph**.

1. (10%) Investigue a influência de rotações na estabilidade do detector e descritor **ORB**. Utilizando a imagem `skimage.data.astronaut()`, gere versões rotacionadas em ângulos de 0° , 45° , 90° , 135° , 180° . Para cada caso, determine o número de correspondências obtidas pelo **BFMatcher** e a fração de *inliers* identificados pelo **RANSAC**. Apresente os resultados em uma tabela e discuta a robustez do método às rotações analisadas.
2. (15%) Compare o detector de cantos **FAST** com o detector de **Harris** (Capítulo 6). Meça o tempo médio de execução e o número de pontos detectados nas imagens `skimage.data.camera()`, `skimage.data.gravel()` e `skimage.data.brick()`. Discuta as vantagens e limitações de cada detector em termos de velocidade e repetibilidade.
3. (15%) Investigue a sensibilidade do **Haar Cascade** à presença de ruído e desfoque. Adicione ruído gaussiano ($\sigma \in 5, 10, 20$) e desfoque de movimento à imagem `skimage.data.astronaut()`. Varie os parâmetros `scaleFactor` e `minNeighbors` da função `detectMultiScale()` e discuta os efeitos sobre falsos positivos, falsos negativos e número de faces detectadas.
4. (15%) Implemente manualmente a métrica **Intersection over Union (IoU)** para caixas delimitadoras e compare seus resultados com a função `mm.IoU`. Em seguida, crie uma caixa de referência e dez caixas obtidas por deslocamentos e variações de escala, construindo um gráfico que relacione o deslocamento aplicado com o valor de IoU obtido.
5. (15%) Avalie o algoritmo de **Supressão de Não-Máximos (NMS)**. Crie uma cena sintética contendo três objetos, cada um representado por pelo menos cinco caixas sobrepostas com diferentes pontuações de confiança. Execute o algoritmo para limiares de IoU iguais a 0.2, 0.4, 0.6, 0.8, visualize os resultados e discuta a influência desse parâmetro na quantidade de caixas mantidas.

6. **(15%)** Aplique o pipeline de segmentação apresentado neste capítulo (limiarização de Otsu, operações morfológicas e `mm.measure`) à imagem `skimage.data.coins()`. Exporte as medidas utilizando `mm.saveMeasures(fmt="csv")` e produza histogramas das distribuições de área, circularidade e solidez. Discuta quais desses atributos são mais adequados para caracterizar as moedas.
7. **(15%)** Construa uma imagem sintética contendo círculos, retângulos e triângulos, alguns parcialmente sobrepostos. Realize a segmentação, extraia as medidas com `mm.measure` e gere automaticamente as anotações no formato YOLO utilizando `mm.saveMeasures(fmt="yolo")`. Em seguida, modifique deliberadamente algumas *bounding boxes* e utilize `mm.verifyBoundingBox` para avaliar, em diferentes limiares de IoU, quantas anotações permanecem válidas. Discuta a influência da sobreposição entre objetos na qualidade das anotações obtidas.
8. **(Bônus – 10%)** Desenvolva um classificador simples, como **k-NN** (Capítulo 7), utilizando apenas os atributos geométricos produzidos por `mm.measure` (área, circularidade, solidez e número de vértices). Utilize um conjunto de formas sintéticas (círculos, retângulos e triângulos) com diferentes escalas e rotações, avalie a acurácia obtida e discuta a capacidade desses descritores em distinguir diferentes classes de objetos.

Referências do Capítulo

Os conceitos e algoritmos apresentados neste capítulo foram fundamentados em referências clássicas e contemporâneas da literatura de PDI-VC:

- Gonzalez; Woods (2018), para os fundamentos de segmentação de imagens, operações morfológicas, rotulação de componentes conexos e extração de atributos geométricos.
- Szeliski (2022), para os conceitos de correspondência de características, transformações geométricas, homografias, detecção de objetos e segmentação em Visão Computacional.
- (**rublee2011orb?**), para a descrição do algoritmo **ORB** (*Oriented FAST and Rotated BRIEF*), utilizado na detecção, descrição e correspondência de características locais.
- (**fischler1981random?**), para a formulação original do algoritmo **RANSAC** (*Random Sample Consensus*), empregado na estimação robusta de modelos geométricos na presença de dados inconsistentes.
- Viola; Jones (2001), Viola; Jones (2004) e Lienhart; Maydt (2002), para os fundamentos do detector **Haar Cascade**, incluindo características de Haar, imagem integral, classificadores em cascata e extensões utilizadas em implementações modernas.
- (**kirillov2019panoptic?**), para a definição da **segmentação panóptica**, que integra segmentação semântica e segmentação de instâncias em uma única representação.
- Redmon et al. (2016), Ren et al. (2015), Ronneberger; Fischer; Brox (2015), (**he2017mask?**) e Kirillov et al. (2023), para modelos modernos baseados em Aprendizado Profundo aplicados à detecção e segmentação de imagens, incluindo **YOLO**, **Faster R-CNN**, **U-Net**, **Mask R-CNN** e **Segment Anything (SAM)**.



7.14 Parte Prática com Exercícios de Programação

A presente lista de exercícios de programação (EP) consolida as formulações teóricas apresentadas ao longo do Capítulo 8 — Correspondência de Características, Detecção de Objetos e Segmentação Clássica — por meio de uma trilha prática aplicada. Assim como no capítulo anterior, os EPs isolam as **grandezas intermediárias** de cada técnica — a distância entre descritores binários, os termos de uma imagem integral, a contagem de *inliers* de um modelo candidato, a sobreposição entre caixas delimitadoras e o rótulo de cada componente conectado — permitindo validar manualmente cada etapa do raciocínio sem depender do OpenCV nem de imagens externas.

O encadeamento dos exercícios reproduz o fluxo conceitual do capítulo: inicia-se com a **distância de Hamming**, coração da correspondência de descritores binários como o ORB; avança-se para a contagem de **inliers** que sustenta o **RANSAC** na estimação robusta de uma homografia; prossegue-se com a **imagem integral**, o truque computacional que torna o **Haar Cascade** viável em tempo real; aprofunda-se em **IoU** e **Supressão de Não-Máximos**, o pós-processamento comum a praticamente todo detector de objetos; e conclui-se com a **rotulação de componentes conectados**, a abordagem clássica — e suas limitações — para segmentar instâncias individuais em uma máscara binária.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.7 | TestSuite: 1.1.2

Executando os Testes

Para avaliar os testes, execute `TestSuite("EP08_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```
codigo = """
# ... seu código aqui ...
"""

TestSuite("EP08_01").run_code(codigo)
```

Resumo dos Métodos do `morph.py` (Cap. 8)

A biblioteca `morph.py` disponibiliza funções para análise de componentes conexos, extração de contornos, métricas geométricas e anotações:

1. **Componentes e Contornos** (`connectedComponents`, `findContours`) Rotulam regiões conexas e extraem os contornos de imagens binárias.
2. **Propriedades do Contorno** (`contourArea`, `arcLength`, `convexHull`, `approxPolyDP`, `fitLine`) Calculam área, perímetro, fecho convexo, aproximação poligonal e ajuste de reta para um contorno.

3. **Geometria Envolvente** (`boundingRect`, `minAreaRect`, `boxPoints`, `minEnclosingCircle`, `fitEllipse`) Determinam retângulos envolventes (alinhados ou orientados), elipses e o menor círculo delimitador.
4. **Extração e Persistência de Medidas** (`measure`, `saveMeasures`) Extraem descritores geométricos dos objetos (área, circularidade, solidez, centroide) e exportam os dados para CSV, texto ou formato YOLO.
5. **Avaliação e Visualização** (`IoU`, `verifyBoundingBox`, `showBoundingBox`) Calculam a intersecção sobre união (*Intersection over Union*), validam bounding boxes com gabaritos e desenhavam caixas delimitadoras anotadas sobre a imagem.

7.14.1 EP08_01 Distância de Hamming e Correspondência de Descritores Binários

O ORB, usado no Projeto Prático 1 deste capítulo, descreve a vizinhança de cada ponto de interesse como uma sequência de bits — e, por isso, a comparação entre dois descritores não usa a distância euclidiana do k-NN do Capítulo 7, e sim a **distância de Hamming**: o número de posições em que os bits diferem. Antes de chamar `cv2.BFMatcher(cv2.NORM_HAMMING)`, você foi encarregado de implementar manualmente essa correspondência (*matching*) por força bruta — a mesma etapa que, executada internamente pelo OpenCV, precede a estimação robusta da homografia por RANSAC.

7.14.1.1 Diretrizes de Implementação

1. **Quantidades:** Ler os inteiros N e M — número de descritores extraídos da imagem A e da imagem B, respectivamente.
2. **Descritores de A:** Ler N linhas, cada uma contendo um descritor binário (uma *string* de caracteres 0 e 1, todos do mesmo comprimento).
3. **Descritores de B:** Ler M linhas, no mesmo formato.
4. **Limiar:** Ler o inteiro τ — distância de Hamming máxima aceitável para considerar uma correspondência válida.
5. **Distância de Hamming:** Para dois descritores binários a e b de mesmo comprimento,

$$d_H(a, b) = \sum_k \mathbb{1}[a_k \neq b_k],$$

ou seja, a contagem de posições em que os bits diferem.

6. **Correspondência por vizinho mais próximo:** Para cada descritor a_i de A (i na ordem de leitura, começando em 0), calcule sua distância de Hamming a **todos** os descritores de B e encontre o de menor distância. Em caso de empate entre dois ou mais descritores de B com a mesma distância mínima, escolha o de **menor índice**.
7. **Filtragem pelo limiar:** Se a menor distância encontrada for $\leq \tau$, a correspondência é válida; caso contrário, a_i não possui correspondência.
8. **Saída:** Para cada i de 0 a $N - 1$, na ordem de leitura, imprimir uma linha: `i j d` se houver correspondência válida (onde j é o índice do descritor de B escolhido e d sua distância), ou `i -1` caso contrário. Ao final, imprimir **Total correspondências válidas: X**.

7.14.1.2 Restrições Computacionais

- **Mesmo comprimento:** todos os descritores (de A e de B) têm exatamente o mesmo número de bits.
- **Força bruta:** compare cada descritor de A a **todos** os de B — não é necessário nenhum tipo de indexação ou estrutura de aceleração.
- **Desempate por menor índice em B, e nunca por ordem de leitura de A** (que já é natural, pois cada a_i é tratado de forma independente).

7.14.1.3 Fundamentação Teórica

Elemento	Papel na correspondência ORB
Descritor binário (BRIEF)	Cada bit é o resultado de uma comparação de intensidade entre dois pixels da vizinhança
Distância de Hamming	Métrica de dissimetria entre <i>strings</i> binárias; muito mais rápida de calcular que a distância euclidiana (operação XOR + contagem de bits)
Vizinho mais próximo	Critério de correspondência: cada ponto de A é pareado ao ponto de B com descritor mais similar
Limiar τ	Filtra correspondências pouco confiáveis antes mesmo do RANSAC — mas, como discutido no capítulo, algumas correspondências incorretas ainda passam, exigindo a robustez do RANSAC

7.14.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros N e M .
- Próximas N linhas: um descritor binário por linha (*string* de 0s e 1s).
- Próximas M linhas: um descritor binário por linha, no mesmo formato.
- Última linha: Inteiro τ .

Saída:

- N linhas, uma por descritor de A, no formato $i \ j \ d$ ou $i \ -1$.
- Última linha: Total correspondências válidas: X .

7.14.1.5 Exemplos

Entrada	Saída	Observação
3 3	0 0 1	O descritor 11110000 não encontra correspondência: seu vizinho mais próximo está a distância 4, acima do limiar $\tau = 2$.
10101010	1 -1	
11110000	2 1 1	
00001111	Total correspondências válidas: 2	
10101011		
00001110		
11111111		
2		

```
%%writefile EP08_01.py
# Código Python
```

```
Writing EP08_01.py
```

```
TestSuite("EP08_01.py").run()
```

7.14.2 EP08_02 Homografia e RANSAC: A Votação por *Inliers*

O RANSAC, apresentado na seção “Modelagem Matemática: Homografia e RANSAC”, repete um ciclo de três passos — sortear uma amostra mínima, estimar um modelo candidato, e contar quantas correspondências são consistentes com ele (os *inliers*) — mantendo ao final o modelo mais votado. A etapa de estimação do modelo a partir de 4 pontos (passo 2) envolve álgebra linear que foge ao escopo deste EP; aqui, você



Figura 7.15: Simulador: Distância de Hamming entre Dois Descritores Binários

recebe diretamente um conjunto de homografias **já candidatas** — como se cada uma tivesse sido estimada a partir de uma amostra aleatória diferente — e é encarregado de reproduzir exatamente o passo decisivo do algoritmo: **aplicar cada modelo a todas as correspondências e contar seus *inliers***, escolhendo o vencedor.

7.14.2.1 Diretrizes de Implementação

1. **Correspondências:** Ler o inteiro N e, em seguida, N linhas com quatro reais cada, $x \ y \ x' \ y'$ — um ponto da imagem A e seu correspondente (possivelmente incorreto) na imagem B, exatamente como produzido pela etapa de *matching* do EP08_01.
2. **Modelos candidatos:** Ler o inteiro K (número de homografias candidatas) e o real ε (limiar de erro de reprojeção). Em seguida, ler K linhas, cada uma com nove reais $h_{11} \ h_{12} \ h_{13} \ h_{21} \ h_{22} \ h_{23} \ h_{31} \ h_{32} \ h_{33}$ — os elementos da matriz H candidata, em ordem de leitura por linha (*row-major*).
3. **Reprojeção:** Para cada correspondência (x, y, x', y') e cada modelo candidato H_k , calcular o ponto projetado

$$\begin{bmatrix} \hat{x} \\ \hat{y} \\ \hat{w} \end{bmatrix} = H_k \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad (\hat{x}/\hat{w}, \hat{y}/\hat{w}) \text{ é o ponto projetado.}$$

4. **Erro de reprojeção:** $e = \sqrt{(\hat{x}/\hat{w} - x')^2 + (\hat{y}/\hat{w} - y')^2}$.
5. **Contagem de *inliers*:** Uma correspondência é um *inlier* do modelo H_k se $e \leq \varepsilon$.
6. **Seleção do melhor modelo:** O modelo vencedor é o de maior número de *inliers*; em caso de empate, escolha o de **menor índice** k (o primeiro encontrado durante o ciclo iterativo do RANSAC).
7. **Saída:** Para cada modelo k de 0 a $K - 1$, na ordem de leitura, imprimir **Modelo k: I inliers**. Ao final, imprimir **Melhor modelo: k_best com I_best inliers**.

7.14.2.2 Restrições Computacionais

- **Comparação inclusiva:** um erro de reprojeção **exatamente igual** a ε conta como *inlier* ($e \leq \varepsilon$).
- **Sem estimação de H :** as matrizes já são fornecidas prontas — não é necessário (nem esperado) resolver nenhum sistema linear.
- **Empate resolvido pelo menor índice**, refletindo o comportamento natural de um algoritmo iterativo que percorre os modelos em ordem e só substitui o melhor encontrado até então quando um novo modelo o **supera estritamente**.

7.14.2.3 Fundamentação Teórica

Elemento	Papel no RANSAC
Amostra mínima (4 pares)	Suficiente para determinar os 8 graus de liberdade de uma homografia
Modelo candidato H_k	Estimado a partir de uma amostra mínima; pode ser bom ou ruim, dependendo se a amostra continha <i>outliers</i>
Erro de reprojeção	Mede o quão bem o modelo “prevê” cada correspondência observada
<i>Inlier</i> vs. <i>outlier</i>	Correspondências consistentes com o modelo vencedor (<i>inliers</i>) vs. as demais, tipicamente correspondências incorretas do <i>matching</i>
Refinamento final	Na prática, após escolher o melhor modelo, o RANSAC o recalcula usando apenas seus <i>inliers</i> — passo não exigido neste EP

7.14.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N .
- Próximas N linhas: quatro reais $x \ y \ x' \ y'$.
- Próxima linha: Inteiro K e real ε .
- Próximas K linhas: nove reais (elementos de H_k , *row-major*).

Saída:

- K linhas no formato Modelo k : I *inliers*.
- Última linha: Melhor modelo: `k_best` com I_best *inliers*.

7.14.2.5 Exemplos

Entrada	Saída	Observação
5 0 0 0 0 1 1 2 2 2 0 4 0 0 2 0 4 5 5 1 1 2 0.5 2 0 0 0 2 0 0 0 1 1 0 0 0 1 0 0 0 1	Modelo 0: 4 <i>inliers</i> Modelo 1: 1 <i>inliers</i> Melhor modelo: 0 com 4 <i>inliers</i>	O Modelo 0 (escala $\times 2$) explica corretamente 4 das 5 correspondências; a 5ª, $(5, 5) \rightarrow (1, 1)$, é um <i>outlier</i> que nenhum dos dois modelos explica bem.

```
%%writefile EP08_02.py
# Código Python
```

```
Writing EP08_02.py
```

```
TestSuite("EP08_02.py").run()
```



Figura 7.16: Simulador: RANSAC — Votação por Inliers entre Modelos Candidatos

7.14.3 EP08_03 ● Imagem Integral: Somas Retangulares em Tempo Constante

Imagine uma câmera de segurança processando 30 quadros por segundo, e para cada quadro o sistema precisa varrer a imagem em dezenas de posições e escalas diferentes, testando em cada uma um conjunto de características retangulares para decidir “há um rosto aqui?”. Se calcular a soma de intensidades de cada retângulo exigisse somar pixel a pixel, o sistema não teria a menor chance de rodar em tempo real — o gargalo estaria justamente na parte mais repetida do algoritmo. É exatamente esse gargalo que a imagem integral elimina.

O Haar Cascade avalia milhares de características retangulares por janela, em múltiplas posições e escalas — algo inviável em tempo real se cada retângulo exigisse somar seus pixels um a um. A **imagem integral**, definida na seção sobre Haar Cascade, resolve esse problema: uma vez pré-computada, a soma de intensidades de **qualquer** região retangular é obtida com apenas quatro consultas e três operações aritméticas, independentemente do tamanho do retângulo.

Você foi encarregado de implementar essa estrutura do zero: primeiro, calcular a imagem integral a partir da imagem original; em seguida, respondê-la para consultas retangulares arbitrárias.

7.14.3.1 📋 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ da imagem e seus $H \times W$ valores inteiros de intensidade.
2. **Imagem integral:** Calcular, para cada posição (i,j) (indexação a partir de 0, [linha][coluna]),

$$II(i,j) = \sum_{i' \leq i, j' \leq j} I(i',j'),$$

ou seja, a soma de todos os pixels acima e à esquerda de (i,j) , incluindo a própria posição.

3. **Consultas:** Ler o inteiro Q e, em seguida, Q linhas, cada uma com quatro inteiros $x_1 \ y_1 \ x_2 \ y_2$ — os cantos superior-esquerdo e inferior-direito de um retângulo, **ambos inclusivos**, com $0 \leq x_1 \leq x_2 < W$ e $0 \leq y_1 \leq y_2 < H$.
4. **Soma retangular em $O(1)$:** Para cada consulta, calcular a soma de intensidades dentro do retângulo usando exclusivamente valores já presentes em II (sem percorrer os pixels originais):

$$S(x_1, y_1, x_2, y_2) = II(y_2, x_2) - II(y_2, x_1 - 1) - II(y_1 - 1, x_2) + II(y_1 - 1, x_1 - 1),$$

tratando qualquer termo com índice de linha ou coluna igual a -1 como 0.

5. **Saída:** Primeiro, imprimir a imagem integral completa — H linhas com W inteiros cada. Em seguida, para cada consulta, imprimir um único inteiro: a soma da região correspondente.

7.14.3.2 Restrições Computacionais

- **Não recalcule por força bruta:** a resposta a cada consulta deve usar a fórmula de quatro termos sobre II , não uma soma direta dos pixels do retângulo (ainda que o resultado numérico seja o mesmo, o objetivo do exercício é justamente essa técnica).
- **Retângulos com coordenadas inclusivas:** (x_1, y_1) e (x_2, y_2) pertencem à região somada.
- **Tratamento de borda:** ao consultar II com índice -1 (quando $x_1 = 0$ ou $y_1 = 0$), utilize o valor 0.

7.14.3.3 Fundamentação Teórica

Elemento	Papel no Haar Cascade
Imagem integral II	Pré-computada uma única vez por imagem, em tempo $O(HW)$
Consulta em $O(1)$	Cada característica Haar (diferença entre somas de regiões retangulares) é avaliada com poucas operações, independentemente da área do retângulo
Escalabilidade	É essa constância que viabiliza avaliar milhares de características, em múltiplas posições e escalas, em tempo real
Princípio de inclusão-exclusão	Os quatro termos da fórmula somam a região desejada e subtraem exatamente as áreas contadas em excesso

7.14.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros H e W .
- Próximas H linhas: W inteiros cada (imagem original).
- Próxima linha: Inteiro Q .
- Próximas Q linhas: quatro inteiros x_1 y_1 x_2 y_2 .

Saída:

- H linhas com W inteiros cada (a imagem integral).
- Q linhas, uma por consulta, com a soma da região correspondente.

7.14.3.5 Exemplos

Entrada	Saída	Observação
3 3 1 2 3 4 5 6 7 8 9 1 0 0 2 2 3 3 1 2 3 4 5 6 7 8 9 2 1 1 2 2 0 0 1 1	1 3 6 5 12 21 12 27 45 45 1 3 6 5 12 21 12 27 45 28 12	A consulta cobre a imagem inteira; a soma coincide com $II(2, 2)$ e com a soma de todos os 9 valores. A primeira consulta usa os quatro termos da fórmula; a segunda coincide diretamente com $II(1, 1)$, pois começa na origem.

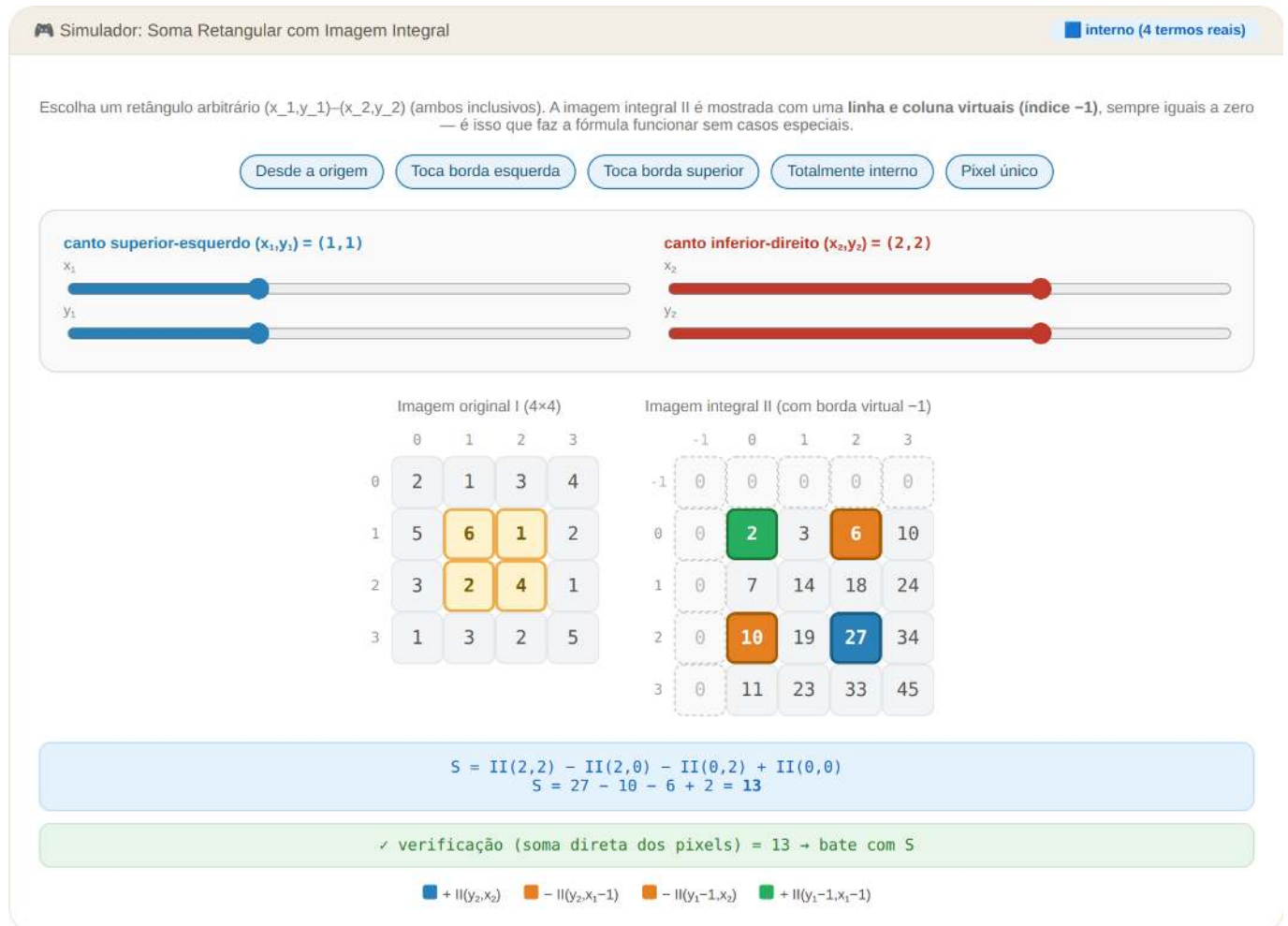


Figura 7.17: Simulador: Soma Retangular em $O(1)$ — Múltiplas Situações de Borda


```
%writefile EP08_03.py
# Código Python
```

Writing EP08_03.py

```
TestSuite("EP08_03.py").run()
```

7.14.4 EP08_04 IoU e Supressão de Não-Máximos (NMS)

A figura desta seção mostrou o efeito da Supressão de Não-Máximos sobre um conjunto de caixas produzidas por um detector do tipo *sliding window*: múltiplas detecções redundantes por objeto foram reduzidas a uma única caixa por objeto. Você foi encarregado de reimplementar, byte a byte, as duas funções que produziram aquele resultado — `calcular_iou` e `supressao_ao_maximos` — para confirmar, com suas próprias mãos, exatamente os números que o capítulo apresentou.

7.14.4.1 Diretrizes de Implementação

1. **Entrada:** Ler o inteiro N (número de caixas) e o real τ (limiar de IoU). Em seguida, ler N linhas, cada uma com cinco reais x_{min} y_{min} x_{max} y_{max} `score`.
2. **Interseção sobre União:** Para duas caixas A e B ,

$$\text{IoU}(A, B) = \frac{\text{área}(A \cap B)}{\text{área}(A \cup B)},$$

com área de interseção nula quando as caixas não se sobrepõem.

3. **Algoritmo de NMS** (exatamente como descrito no capítulo):
 - a. Ordene as caixas por `score` decrescente (empates mantêm a ordem de leitura original).
 - b. Selecione a caixa de maior pontuação entre as restantes; adicione-a à saída e remova-a da lista.
 - c. Descarte, da lista restante, **todas** as caixas cujo IoU com a caixa selecionada seja **maior ou igual** a τ — apenas as caixas com $\text{IoU} < \tau$ permanecem candidatas.
 - d. Repita (b)–(c) até que a lista de restantes esteja vazia.
4. **Saída:** Para cada caixa mantida, na ordem em que foi selecionada, imprimir seu índice original (posição de leitura, a partir de 0) e seu `score`, com 2 casas decimais. Ao final, imprimir **Total mantidas: X**.

7.14.4.2 Restrições Computacionais

- **Atenção ao sentido do limiar:** ao contrário do que se poderia supor, uma caixa é **suprimida** quando $\text{IoU} \geq \tau$ (não apenas quando $\text{IoU} > \tau$) — siga exatamente esse critério, o mesmo do código de referência do capítulo.
- **Índices originais:** a saída referencia a posição de leitura de cada caixa na entrada, não sua posição após a ordenação por `score`.
- **Área sem soma de 1 pixel:** use $\text{área} = (x_{max} - x_{min}) \times (y_{max} - y_{min})$, exatamente como no capítulo (sem o ajuste “+1” às vezes usado em outras convenções).

7.14.4.3 Fundamentação Teórica

Elemento	Papel no pós-processamento
IoU	Quantifica a sobreposição espacial entre duas caixas delimitadoras

Elemento	Papel no pós-processamento
<i>Sliding window</i> (Haar Cascade)	Produce tipicamente várias detecções sobrepostas para o mesmo objeto, em posições e escalas próximas
Limiar τ	Controla a agressividade da supressão: baixo demais funde objetos próximos; alto demais deixa passar redundâncias
Ordenação por score	Garante que, entre caixas redundantes, a de maior confiança sempre sobrevive

7.14.4.4 📦 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N e real τ .
- Próximas N linhas: cinco reais x_{min} y_{min} x_{max} y_{max} **score**.

Saída:

- Uma linha por caixa mantida, na ordem de seleção: **índice** **score** (score com 2 casas decimais).
- Última linha: **Total mantidas: X**.

7.14.4.5 📌 Exemplos

Entrada	Saída	Observação
5 0.4	3 0.95	Exatamente o exemplo da figura do capítulo: 5 caixas redundantes (2 objetos) tornam-se 2 detecções finais. O IoU entre a 1ª e a 2ª caixas é $\approx 0,775$, bem acima de $\tau = 0,4$.
50 50 150 150 0.90	0 0.90	
60 55 155 145 0.75	Total mantidas: 2	
58 60 160 150 0.60		
300 300 400 420 0.95		
310 305 395 415 0.70		



Figura 7.18: Simulador: IoU e Supressão de Não-Máximos

```
%%writefile EP08_04.py
# Código Python
```

```
Writing EP08_04.py
```

```
TestSuite("EP08_04.py").run()
```

7.14.5 EP08_05 Rotulação de Componentes Conectados: Segmentação de Instâncias

O exemplo de segmentação clássica deste capítulo separou “instâncias” de moedas simplesmente pela sua desconexão espacial na máscara binária resultante da limiarização de Otsu. Essa etapa final — rotular cada componente conectado com um identificador de instância — é exatamente o que você foi encarregado de implementar aqui, do zero, sobre uma máscara binária já pronta (0 = fundo, 1 = objeto), como se fosse uma reimplementação manual de `cv2.connectedComponents`.

Este exercício também expõe, de forma muito concreta, a limitação discutida no capítulo: o resultado depende inteiramente de como se define “vizinhança” entre pixels — e, como você verá no segundo exemplo, dois pixels em diagonal podem ser considerados a mesma instância ou instâncias diferentes, dependendo exclusivamente da **conectividade** escolhida, não de qualquer noção semântica de objeto.

7.14.5.1 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ da máscara binária e seus $H \times W$ valores (0 ou 1).
2. **Conectividade:** Ler o inteiro $c \in \{4, 8\}$. Na conectividade 4, os vizinhos de (i, j) são $(i-1, j)$, $(i+1, j)$, $(i, j-1)$ e $(i, j+1)$. Na conectividade 8, somam-se as quatro diagonais: $(i-1, j-1)$, $(i-1, j+1)$, $(i+1, j-1)$ e $(i+1, j+1)$.
3. **Descoberta de componentes:** Percorrendo a máscara em varredura linha a linha, da esquerda para a direita e de cima para baixo, sempre que um pixel de valor 1 ainda sem rótulo for encontrado, ele inicia um **novo componente**: atribua a ele o próximo rótulo disponível (o primeiro componente descoberto recebe o rótulo 1, o segundo o rótulo 2, e assim por diante) e propague esse mesmo rótulo a todos os pixels de valor 1 alcançáveis a partir dele por uma cadeia de vizinhos (de acordo com a conectividade escolhida) — por busca em largura, profundidade, ou *union-find*, à sua escolha.
4. **Pixels de fundo:** permanecem com rótulo 0 e não pertencem a nenhuma instância.
5. **Saída:** Primeiro, imprimir o mapa de rótulos completo — H linhas com W inteiros cada. Em seguida, para cada rótulo ℓ de 1 a K (na ordem de descoberta), imprimir **Instância 1: A pixels**, onde A é a quantidade de pixels com aquele rótulo. Por fim, imprimir **Total de instâncias: K**.

7.14.5.2 Restrições Computacionais

- **Ordem de descoberta = ordem de varredura:** os rótulos são numerados na ordem em que cada novo componente é encontrado pela varredura linha a linha, não por tamanho ou posição.
- **Conectividade explícita:** dois pixels de valor 1 só pertencem à mesma instância se existir uma cadeia de vizinhos **de acordo com** c ligando um ao outro — não use a conectividade oposta por engano.
- **Máscara binária pura:** todos os valores de entrada são exatamente 0 ou 1.

7.14.5.3 Fundamentação Teórica

Elemento	Papel na segmentação clássica de instâncias
Limiarização (Otsu, Cap. 4)	Etapa anterior que produz a máscara binária a partir da imagem de intensidade
Componente conectado	Cada instância é definida apenas por conectividade espacial dos pixels de objeto, sem qualquer noção de forma, classe ou aparência

Elemento	Papel na segmentação clássica de instâncias
Conectividade 4 vs. 8	Parâmetro que altera o resultado: sob conectividade 8, dois blobs unidos apenas na diagonal tornam-se uma única instância
Limitação central	A técnica funde instâncias que se tocam ou se sobrepõem (mesmo que sejam objetos claramente distintos), pois não há noção de “objeto” — apenas de “região conectada”

7.14.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros H e W .
- Próximas H linhas: W inteiros (0 ou 1) cada.
- Última linha: Inteiro c (4 ou 8).

Saída:

- H linhas com W inteiros cada (o mapa de rótulos).
- Uma linha por instância, na ordem de descoberta: **Instância 1: A pixels.**
- Última linha: **Total de instâncias: K.**

7.14.5.5 Exemplos

Entrada	Saída	Observação
6 6 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 8	0 0 0 0 0 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 2 2 0 0 0 2 2 Instância 1: 4 pixels Instância 2: 4 pixels Total de instâncias: 2	Dois blocos 2×2 claramente separados: o resultado é o mesmo sob conectividade 4 ou 8.
2 2 1 0 0 1 8	1 0 0 1 Instância 1: 2 pixels Total de instâncias: 1	Sob conectividade 8, os dois pixels em diagonal pertencem à mesma instância. Repita este exemplo com $c = 4$: o resultado passa a ser 2 instâncias de 1 pixel cada — puramente pela mudança de conectividade, sem qualquer diferença na máscara.

```
%%writefile EP08_05.py
# Código Python
```

```
Writing EP08_05.py
```

```
TestSuite("EP08_05.py").run()
```

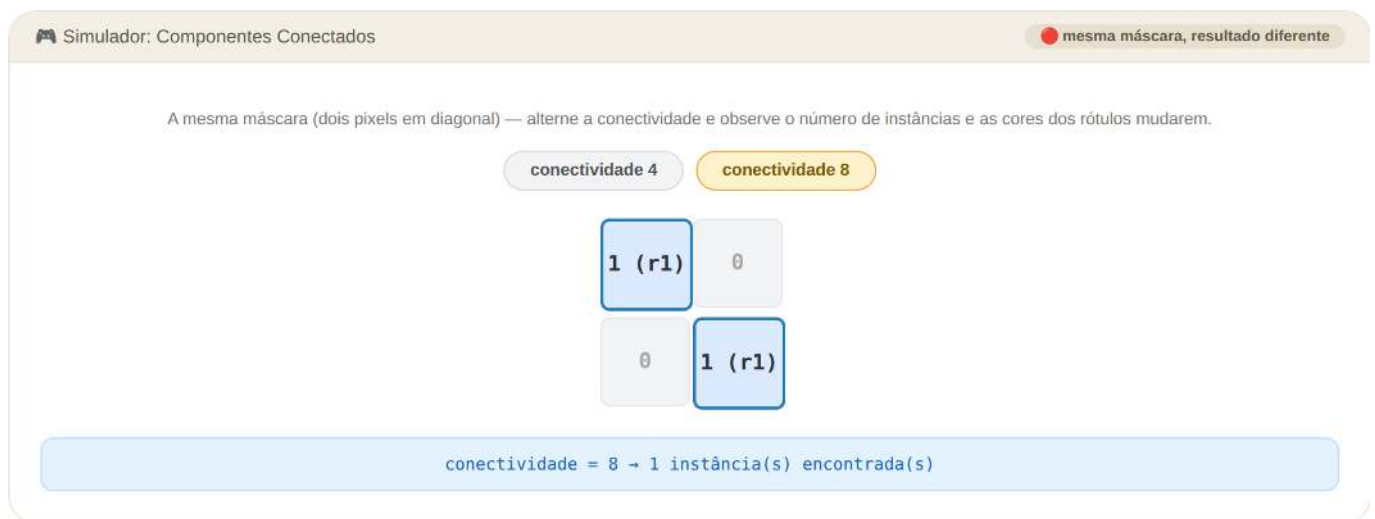


Figura 7.19: Simulador: Rotulação de Componentes Conectados — Conectividade 4 vs. 8

7.14.6 EP08_06 ● Segmentação de Instâncias em Imagem Real: Objetos Geométricos

O exemplo de segmentação clássica deste capítulo separou “instâncias” de moedas por desconexão espacial na máscara binária resultante da limiarização de Otsu. Neste exercício você vai aplicar a mesma ideia — mas agora sobre uma imagem real com objetos geométricos variados — encadeando pré-processamento, binarização, extração de contornos (`cv2.findContours`) e validação do resultado contra um gabarito de *bounding boxes*.

Diferente do exercício anterior (rotulação sobre máscara já pronta), aqui você parte da **imagem original**: a qualidade da sua segmentação depende diretamente das escolhas de pré-processamento (filtragem, limiarização, operações morfológicas) feitas antes de rotular os componentes.

7.14.6.1 📋 Diretrizes de Implementação

1. **Entrada:** utilizar a imagem `00000.jpg`.
2. **Pré-processamento e segmentação:** aplicar as etapas necessárias (filtragem, binarização e operações morfológicas) para separar automaticamente os objetos do fundo, sem recortes manuais.
3. **Rotulação e medição:** para cada objeto segmentado, determinar:
 - área;
 - centro de massa (centroide);
 - tipo, conforme o conjunto `obj2`.
4. **Anotação visual:** escrever, no interior de cada objeto, sua área e a sigla do tipo (`obj2`).
5. **Validação (IoU):** calcular o *Intersection over Union* (IoU) entre o *bounding box* detectado (`cv2.boundingRect`) e o *bounding box* de gabarito do tipo correspondente. Um objeto é considerado corretamente segmentado apenas se houver exatamente um *bounding box* do tipo correto com **IoU 0,5**.
6. **Saída:** imprimir, para cada objeto detectado, seu identificador, tipo e se foi validado com sucesso (`acertou=1`) ou não. A impressão deve seguir a ordem das classes de `obj2` (0=Tria ... 8=Cruz); dentro da mesma classe, ordenar os objetos pela coordenada vertical do centroide (`cy`) crescente. Ao final, imprimir a acurácia geral.

7.14.6.2 🚫 Restrições Computacionais

- **Sem recorte manual:** toda a segmentação deve ser feita sobre a imagem completa.
- **Conjunto de classes fixo:**

```
obj = ['Triangulo', 'Quadrado', 'Pendagono', 'Hexagono', 'Heptagono', 'Circulo',
       'Elipse', 'Estrela', 'Cruz']
obj2 = ['Tria', 'Quad', 'Pent', 'Hexa', 'Hept', 'Circ', 'Elip', 'Estr', 'Cruz']
```

- **Dimensão da imagem:** 608×608 pixels — usada para desnormalizar as coordenadas do arquivo TXT.
- **Validação por centro de massa:** um objeto só é considerado corretamente segmentado se seu centroide estiver estritamente dentro do *bounding box* do gabarito correspondente ao mesmo tipo de objeto.

7.14.6.3 🧠 Fundamentação Teórica

Elemento	Papel na segmentação de instâncias
Pré-processamento (filtragem, limiarização)	Etapa que produz a máscara binária a partir da imagem de intensidade original
<code>cv2.findContours</code>	Extraí os contornos dos componentes conectados na máscara binária
Momentos geométricos (<code>cv2.moments</code>)	Permitem calcular o centro de massa (centroide) de cada contorno
<code>approxPolyDP</code> / vértices	Auxilia na classificação do tipo de objeto (nº de lados aproximado)
Validação via <i>bounding box</i>	Confirma se a instância segmentada corresponde espacialmente a um objeto do gabarito, medindo a acurácia do método

7.14.6.4 📌 Exemplo de Saída Esperada

```
Objeto 1: tipo=Tria, validado=True
...
Acurácia: 88.89%
```

Parâmetros fixos para reprodutibilidade: para que a saída bata com o gabarito de correção automática, utilize exatamente: filtro de área mínima de 300 pixels; `cv2.approxPolyDP` com `epsilon = 0.02 * perímetro`; limiar de solidez 0.92 e contagem de vértices 9 (com 11 para diferenciar Cruz de Estrela) para formas côncavas; razão de aspecto 1.15 para diferenciar Círculo de Elipse; limiar de IoU = 0.5 na validação.

7.14.6.5 📁 Arquivos de Referência (.jpg e .txt)

Para depuração local, são disponibilizados dois arquivos de referência (incluídos nesta entrega; ao integrá-los ao repositório do capítulo, salve-os em `all/cap08/dados/EP08/`):

- 📄 **Imagem (00000.jpg):** imagem de objetos geométricos utilizada como entrada do exercício. O objetivo é segmentar automaticamente cada objeto, determinar seu tipo e calcular suas medidas.
- 📄 **Gabarito (00000.txt):** arquivo contendo as *bounding boxes* normalizadas dos objetos presentes na imagem. Cada linha possui o identificador da classe e as coordenadas normalizadas dos cantos superior esquerdo e inferior direito, sendo utilizado para validar automaticamente a segmentação.

A Figure 7.20 apresenta a imagem de entrada e a mesma imagem com as *bounding boxes* desenhadas a partir do arquivo de gabarito.

```
import os
import urllib.request
from morph import mm

def garantir_e_baixar(nome):
```



```

pasta = "dados/EP06"
caminho = os.path.join(pasta, nome)

os.makedirs(pasta, exist_ok=True)

if not os.path.exists(caminho):
    url = (
        "https://raw.githubusercontent.com/"
        "fzampirolli/pdi-vc/master/all/cap08/dados/EP06/"
        + nome
    )
    print(f"Baixando {nome}...")
    urllib.request.urlretrieve(url, caminho)

return caminho

img_arq = garantir_e_baixar("00000.jpg")
txt_arq = garantir_e_baixar("00000.txt")

img = mm.read(img_arq)
img_bb = mm.showBoundingBox(img, txt_arq, fmt="yolo", show=False)

mm.show(
    [img, img_bb],
    titles=[
        "Imagem original",
        "Bounding boxes do gabarito"
    ],
    cols=2,
    figsize=(10,5)
)

```

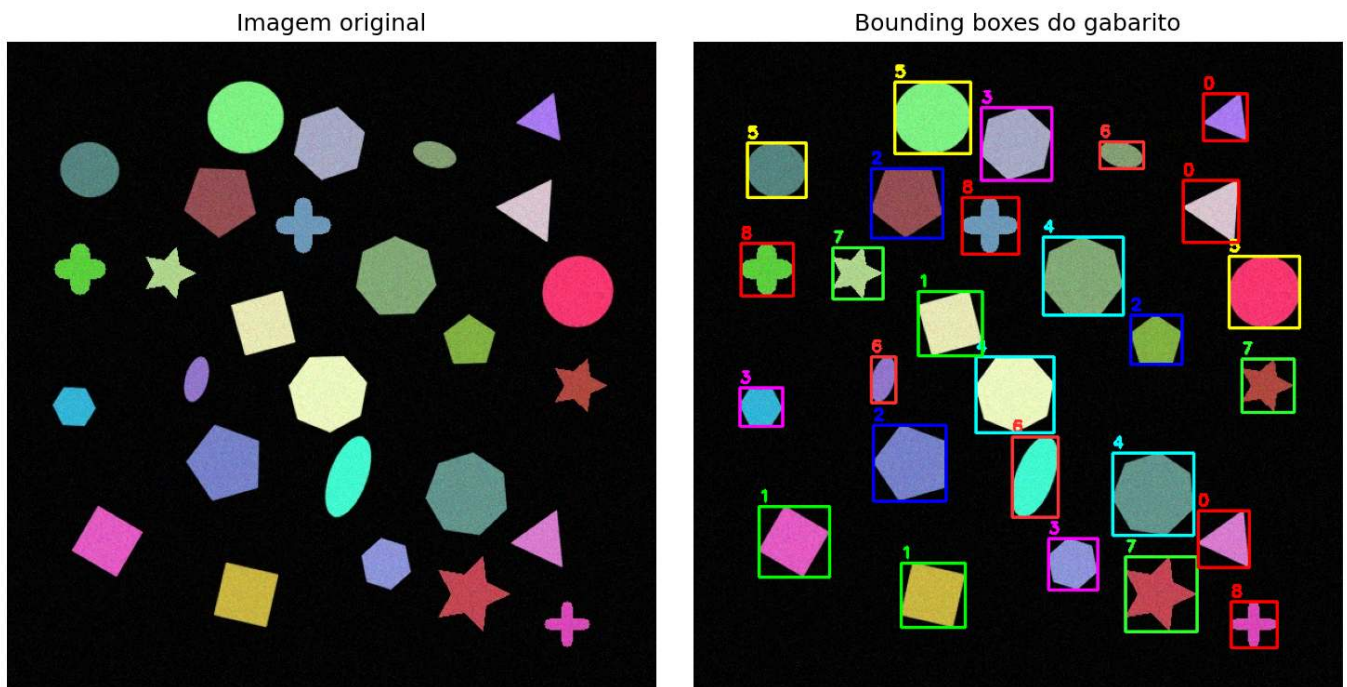


Figura 7.20: Imagem utilizada no EP08_06. À esquerda, a imagem original. À direita, a imagem com as *bounding boxes* do arquivo de gabarito.



Figura 7.21: Simulador: Acurácia de Segmentação em Múltiplos Objetos (IoU)

```
%%writefile EP08_06.py  
# Código Python
```

```
Overwriting EP08_06.py
```

```
TestSuite("EP08_06.py").run()
```

Capítulo 8

Aprendizado Profundo para Visão Computacional

 Executar  Colab  Abrir pdi-vc  GitHub

Este capítulo encontra-se em desenvolvimento.

Os capítulos anteriores estabeleceram os fundamentos da Visão Computacional por meio de métodos clássicos de extração e representação de características. No **Capítulo 7**, descritores como *Local Binary Patterns* (*LBP*) e *Histogram of Oriented Gradients* (*HOG*) mostraram como texturas e formas podem ser codificadas por descritores projetados manualmente. No **Capítulo 8**, algoritmos como *Oriented FAST and Rotated BRIEF* (*ORB*) e o detector *Haar Cascade* estenderam esse princípio a tarefas de correspondência, detecção e reconhecimento de objetos.

Essas técnicas permanecem relevantes por sua interpretabilidade e eficiência computacional, mas dependem de uma etapa prévia de definição manual de descritores, denominada **engenharia de características** (*feature engineering*). Essa dependência limita a adaptação do modelo a cenários para os quais o descritor não foi projetado.

O *Deep Learning* propõe uma alternativa: em vez de especificar manualmente as características relevantes, o modelo aprende automaticamente representações a partir dos dados durante o treinamento — processo conhecido como **aprendizado de representações** (*representation learning*) (Goodfellow; Bengio; Courville, 2016). Na Visão Computacional, essa estratégia é implementada principalmente pelas **Redes Neurais Convolucionais** (*Convolutional Neural Networks* — *CNNs*), nas quais os filtros convolucionais deixam de ter coeficientes fixos e passam a ser ajustados por algoritmos de otimização.

Ainda que representem uma mudança na construção de sistemas de reconhecimento de padrões, as *CNNs* preservam conceitos já estudados neste livro: a convolução, apresentada no **Capítulo 3**, permanece a operação responsável pela extração local de características, agora aplicada com coeficientes aprendidos em vez de projetados.

8.1 Objetivos do Capítulo

Ao final deste capítulo, o estudante deverá ser capaz de:

- Relacionar a convolução aprendida pelas *CNNs* com a convolução de *kernels* fixos apresentada no **Capítulo 3**;
- Descrever a arquitetura básica de uma *CNN* e a função de suas camadas principais;
- Implementar, treinar e avaliar modelos de *CNN* para classificação de imagens;
- Aplicar **transferência de aprendizado** (*transfer learning*) para adaptar modelos pré-treinados a novos problemas;
- Utilizar modelos pré-treinados em tarefas de classificação, detecção de objetos e segmentação;
- Implementar, treinar e avaliar uma arquitetura *U-Net* para segmentação semântica, comparando-a a abordagens clássicas;

- Preparar conjuntos de dados anotados e integrá-los a um *pipeline* de treinamento por meio de plataformas como o **Roboflow**;
- Integrar geometria computacional e *Deep Learning* em aplicações de realidade aumentada, fotogrametria e visão estereoscópica.

A Figure 8.1 sintetiza a organização dos conceitos estudados neste capítulo e as relações entre eles.

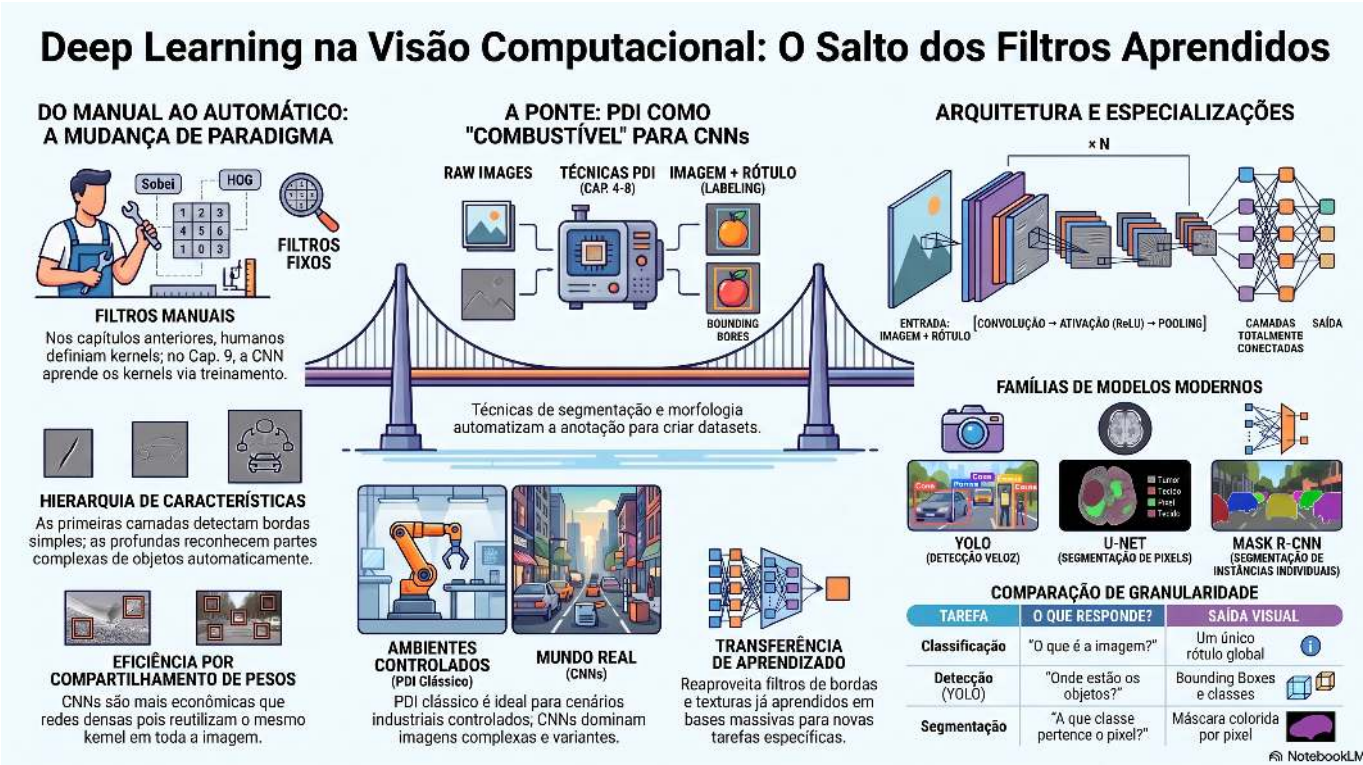


Figura 8.1: Visão geral dos principais conceitos abordados neste capítulo. **Fonte:** elaborado com auxílio do Gemini Notebook (Google, 2025).

8.2 Panorama: Classificação, Deteção e Segmentação

As tarefas de Visão Computacional diferenciam-se, sobretudo, pela informação produzida como saída. A **classificação** atribui um único rótulo à imagem inteira; a **deteção de objetos** localiza e identifica os objetos presentes na cena; a **segmentação** associa uma classe a cada pixel e, em algumas abordagens, distingue diferentes instâncias de uma mesma categoria.

A Table 8.1 resume as tarefas estudadas ao longo do livro, indicando a pergunta que cada uma responde e a granularidade da informação produzida.

Tabela 8.1: Comparativo entre as principais tarefas de Visão Computacional segundo a granularidade da informação produzida.

Tarefa	Pergunta respondida	Granularidade da saída	Capítulo
Classificação	“Qual é a classe desta imagem?”	Um único rótulo para a imagem inteira	7 e 9
Deteção de objetos	“Quais objetos existem e onde estão?”	Classe e caixa delimitadora (<i>bounding box</i>) para cada objeto	8 e 9
Segmentação semântica	“A que classe pertence cada pixel?”	Um rótulo de classe para cada pixel	8 e 9
Segmentação de instâncias	“Quais pixels pertencem a cada objeto?”	Um rótulo por pixel para cada instância	8 e 9

Tarefa	Pergunta respondida	Granularidade da saída	Capítulo
Segmentação panóptica	“Qual é a classe e a identidade de cada objeto?”	Classe e identificador de instância para cada pixel	8 e 9

Essas tarefas representam níveis crescentes de interpretação da imagem: a classificação descreve a cena de forma global, a detecção acrescenta a localização dos objetos e a segmentação produz uma representação espacial detalhada, permitindo analisar cada região individualmente. Este capítulo concentra-se, primeiro, na classificação por *CNNs*, para em seguida estender os mesmos princípios à detecção e à segmentação.

8.3 Configuração do Ambiente

Os exemplos deste capítulo utilizam o **PyTorch**, um *framework* amplamente empregado no desenvolvimento e no treinamento de modelos de *Deep Learning*. O código a seguir verifica a disponibilidade das bibliotecas necessárias e instala, automaticamente, aquelas ainda não presentes no ambiente de execução.

Caso o **PyTorch** não esteja instalado, é selecionada automaticamente uma versão compatível com o hardware disponível: a versão com suporte a **CUDA**, se houver GPU NVIDIA disponível, ou a versão para execução em CPU, caso contrário.

Em seguida, o ambiente é inicializado com a importação das bibliotecas utilizadas ao longo do capítulo, a definição de uma semente aleatória para favorecer a reprodutibilidade dos experimentos e a obtenção do arquivo `morph.py` — a biblioteca didática de processamento morfológico já utilizada em capítulos anteriores —, caso ele ainda não esteja disponível no diretório de trabalho.

```
import importlib, subprocess, sys, shutil, os, urllib.request

for mod, pkg in {
    "cv2": "opencv-python",
    "skimage": "scikit-image",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "matplotlib": "matplotlib",
    "torchviz": "torchviz",
}.items():
    try:
        importlib.import_module(mod)
    except ImportError:
        subprocess.run([sys.executable, "-m", "pip", "install", "-q", pkg], check=True)

try:
    import torch
except ImportError:
    args = (
        ["torch", "torchvision"]
        if shutil.which("nvidia-smi")
        else ["--index-url", "https://download.pytorch.org/whl/cpu",
            "torch", "torchvision"]
    )
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", *args], check=True)
import torch

# Graphviz (binário "dot") é dependência de sistema do torchviz, não pip.
# Tentativa best-effort de instalação em ambientes Linux/Debian (ex.: Colab).
if shutil.which("dot") is None:
    if shutil.which("apt-get"):
        subprocess.run(["apt-get", "install", "-y", "-qq", "graphviz"], check=False)
```



```

if shutil.which("dot") is None:
    print(
        " Executável 'dot' (Graphviz) não encontrado no PATH. "
        "Os grafos do torchviz não poderão ser renderizados. "
        "Instale manualmente: 'sudo apt-get install graphviz' (Linux) "
        "ou 'brew install graphviz' (macOS). "
    )

import cv2
import random
import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from skimage import data as skdata
from torchviz import make_dot
from PIL import Image
import torch
import torchvision.models as models

from torchvision.models.detection import (
    FasterRCNN_ResNet50_FPN_Weights,
    fasterrcnn_resnet50_fpn,
)
from torchvision.transforms.functional import to_tensor

torch.manual_seed(42)

if not os.path.exists("morph.py"):
    urllib.request.urlretrieve(
        "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph/morph.py",
        "morph.py",
    )

import morph
from morph import mm

device = "cuda" if torch.cuda.is_available() else "cpu"
gpu = f" ({torch.cuda.get_device_name(0)})" if device == "cuda" else ""

print(
    f" Ambiente pronto. morph {getattr(morph, '__version__', 'local_file')} | "
    f"PyTorch {torch.__version__} | {device}{gpu}"
)

```

Ambiente pronto. morph 1.1.8 | PyTorch 2.12.1+cpu | cpu

8.4 Da Convolução Fixa à Convolução Aprendida

O **Capítulo 3** apresentou a convolução espacial com *kernels* fixos, como os operadores de Sobel, projetados para realçar características específicas de uma imagem. Nos **Capítulos 7 e 8**, o mesmo princípio sustentou descritores como *HOG*, *LBP* e *ORB*, além do detector *Haar Cascade*: em todos esses casos, os filtros são definidos antes da execução do algoritmo e permanecem inalterados durante o processamento.

A **Figure 8.2** retoma o funcionamento da convolução espacial: o simulador permite selecionar diferentes *kernels* e acompanhar o deslocamento da janela de convolução sobre uma imagem. Em cada posição, os coeficientes do *kernel* combinam-se com a vizinhança correspondente para produzir um valor do **mapa de**

características (*feature map*), ilustrando também o **compartilhamento de pesos** (*weight sharing*) — a aplicação do mesmo *kernel* em todas as posições da imagem.

As *CNNs* preservam essa operação, mas substituem *kernels* fixos por **filtros aprendidos**: em vez de coeficientes definidos previamente, a rede ajusta esses valores durante o treinamento a partir de exemplos rotulados, buscando minimizar uma **função de perda** (*loss function*), que mede a diferença entre as previsões do modelo e as respostas esperadas.

A diferença essencial entre os métodos clássicos e as *CNNs*, portanto, não está na operação de convolução em si, mas na forma como os filtros são obtidos: enquanto os primeiros utilizam filtros projetados manualmente, as *CNNs* aprendem, a partir dos dados de treinamento, representações adequadas à tarefa.

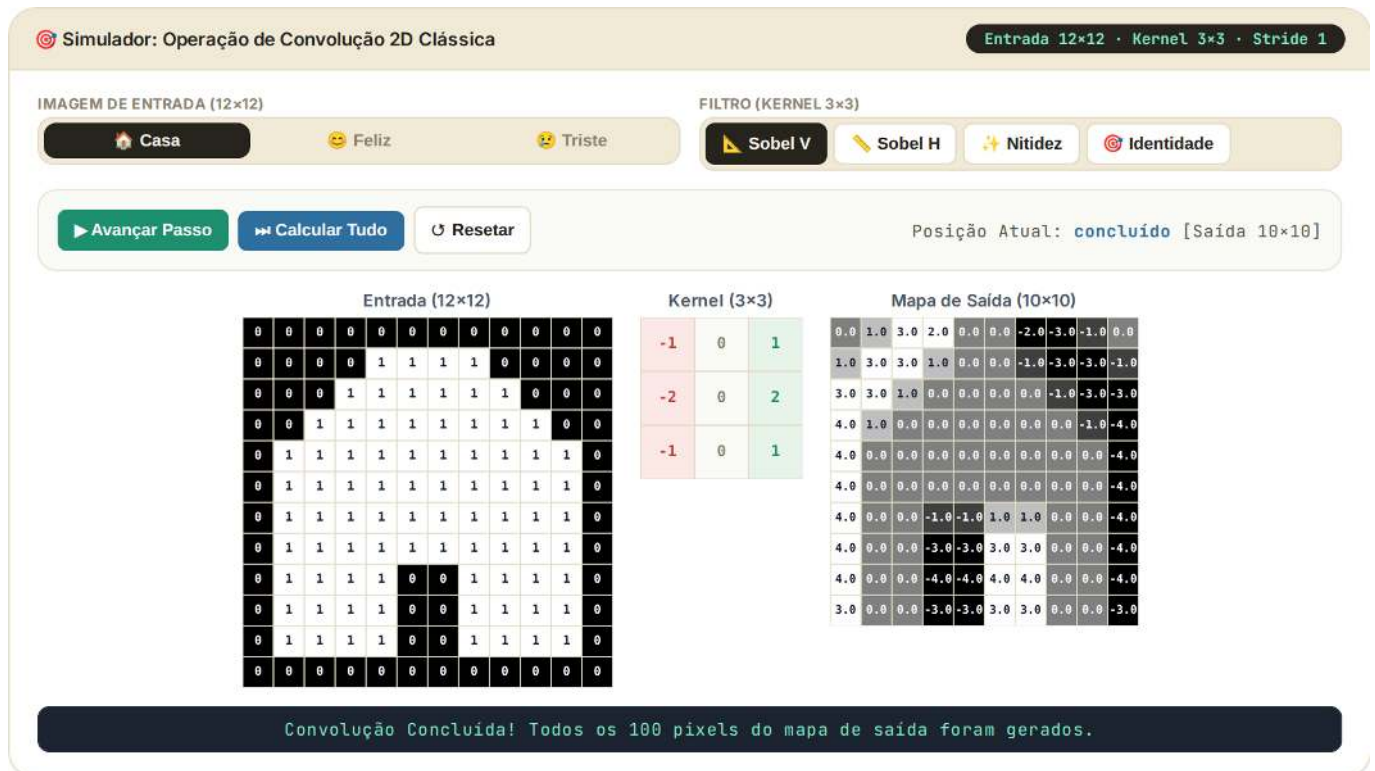


Figura 8.2: Simulador interativo de convolução 2D clássica: escolha entre as três imagens sintéticas de entrada 12×12 (casa, rosto feliz ou triste) e um filtro 3×3 (Sobel V, Sobel H, Nitidez ou Idêntidade) e avance passo a passo para observar como os produtos internos locais do campo receptivo constroem o mapa de características célula a célula.

Para compreender como esse aprendizado ocorre, é necessário estudar a unidade básica de processamento das redes neurais: o **neurônio artificial**.

8.4.1 Neurônio Artificial

O **neurônio artificial** (*artificial neuron*) é a unidade fundamental de processamento de uma rede neural. Seu primeiro modelo matemático — um conjunto de entradas combinadas e comparadas a um limiar — foi proposto por McCulloch; Pitts (1943), ainda sem qualquer mecanismo de aprendizado. O **Perceptron** (Rosenblatt, 1958) avançou sobre essa formulação ao introduzir uma regra de ajuste dos pesos a partir de exemplos, tornando-se o primeiro modelo de neurônio artificial capaz de aprender e a base das arquiteturas modernas de *Deep Learning*. O termo **aprendizado profundo** (*Deep Learning*) refere-se ao uso de redes com múltiplas camadas de processamento, capazes de aprender representações hierárquicas dos dados: as primeiras camadas aprendem características simples, como bordas e texturas, e as camadas mais profundas combinam progressivamente essas representações para identificar estruturas e objetos mais complexos.

Cada neurônio recebe um conjunto de entradas, calcula uma combinação linear desses valores e aplica uma **função de ativação** (*activation function*), produzindo um único valor de saída. Matematicamente, a

combinação linear é dada por

$$z = \sum_{i=1}^n w_i x_i + b,$$

em que x_i representam as entradas, w_i os pesos associados a cada entrada e b o **viés** (*bias*). A saída do neurônio é obtida pela aplicação da função de ativação:

$$y = f(z).$$

Nas *CNNs*, esse princípio assume formas distintas conforme a camada. Nas **camadas convolucionais** (*convolutional layers*), cada neurônio processa apenas uma pequena região da entrada, denominada **campo receptivo** (*receptive field*), preservando a organização espacial da imagem. Nas **camadas totalmente conectadas** (*fully connected layers*), cada neurônio recebe todas as saídas da camada anterior, combinando as características extraídas para produzir a saída final da rede, como a classe atribuída à imagem.

A Figure 8.3 ilustra o funcionamento de um neurônio artificial: o simulador permite modificar as entradas (x_1 e x_2), os pesos (w_1 e w_2), o viés (b) e a função de ativação, observando em tempo real o cálculo da combinação linear e da saída correspondente.

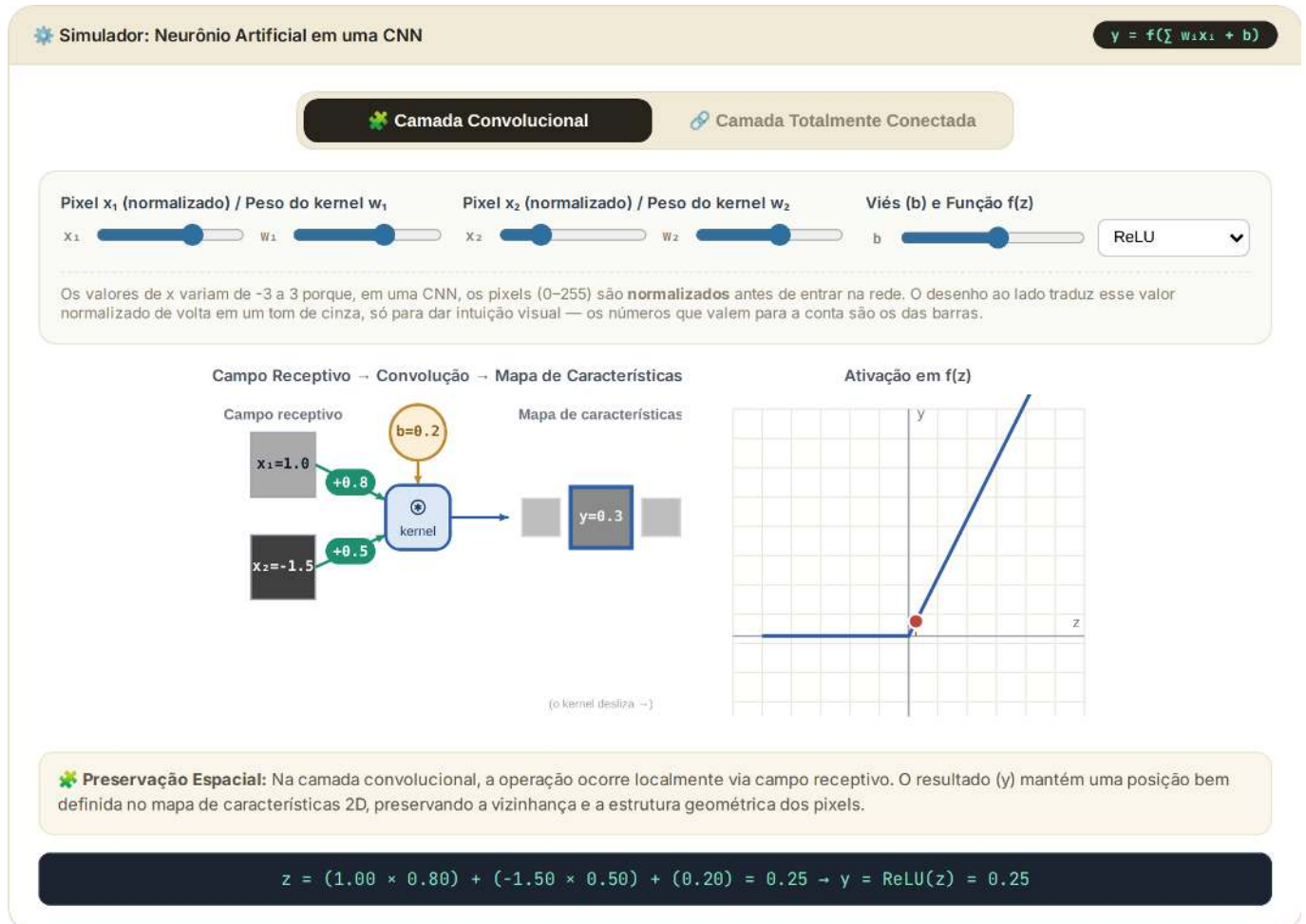


Figura 8.3: Simulador interativo do Neurônio Artificial em contexto de CNN: alterne entre um neurônio de camada convolucional (onde x_i são intensidades de pixel em um campo receptivo e w_i são pesos do *kernel*) e um neurônio de camada totalmente conectada, ajustando entradas, pesos, viés e função de ativação para visualizar o cálculo de z e da saída y em tempo real.

Em uma *CNN*, milhares de neurônios organizam-se em camadas com funções específicas: as primeiras são responsáveis pela extração de características por meio da convolução, e as últimas realizam a classificação a partir das características aprendidas.

8.4.2 Camada Convolutiva

A **camada convolutiva** (*convolutional layer*) é responsável pela extração de características da imagem. Cada filtro gera um **mapa de características** (*feature map*), cuja intensidade em cada posição indica a resposta do filtro à região correspondente da entrada.

A operação realizada segue o mesmo princípio de deslocamento e combinação local apresentado no **Capítulo 3** para a convolução espacial. Considerando um *kernel* K de dimensão $k \times k$, o valor produzido na posição (i, j) é dado por

$$F(i, j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K(u, v) I(i + u, j + v).$$

Vale registrar uma distinção terminológica: a expressão acima corresponde, formalmente, a uma **correlação cruzada** (*cross-correlation*), e não à convolução matemática estrita, que exige a reflexão do *kernel* antes da combinação. A maioria dos *frameworks* de *Deep Learning*, incluindo o **PyTorch**, implementa essa operação sem reflexão e a designa, por convenção, como convolução — convenção adotada também neste capítulo. Essa diferença não tem efeito prático sobre o treinamento, uma vez que os coeficientes do *kernel* são aprendidos e não impostos previamente.

A principal diferença em relação aos métodos clássicos está, assim, na obtenção do *kernel* K : em filtros tradicionais, seus coeficientes são definidos manualmente para destacar características específicas da imagem; nas *CNNs*, os coeficientes são inicializados automaticamente e ajustados durante o treinamento por meio da **retropropagação do erro** (*backpropagation*), o que torna cada filtro especializado em identificar padrões relevantes para a tarefa em estudo.

Dois conceitos caracterizam essa camada:

- **Compartilhamento de pesos** (*weight sharing*): o mesmo filtro é aplicado em todas as posições da imagem, reduzindo significativamente o número de parâmetros do modelo.
- **Campo receptivo** (*receptive field*): cada neurônio convolutivo processa apenas uma pequena vizinhança da imagem, preservando a estrutura espacial dos dados.

Ao empilhar várias camadas convolucionais, a rede aprende uma **hierarquia de características**: as primeiras camadas tendem a detectar padrões simples, como bordas e texturas, e as camadas mais profundas combinam essas informações para representar estruturas progressivamente mais complexas. Após a convolução, o mapa de características é submetido a uma função de ativação, introduzindo não linearidade no modelo e ampliando sua capacidade de representar relações complexas entre as variáveis de entrada.

A Figure 8.4 apresenta essa camada de forma interativa.

8.4.3 Função de Ativação

A convolução é uma operação linear. Para que a rede possa modelar relações não lineares entre entradas e saídas, aplica-se uma **função de ativação** (*activation function*) após cada camada convolutiva.

A função mais utilizada em *CNNs* é a **ReLU** (*Rectified Linear Unit*), definida por

$$\text{ReLU}(x) = \max(0, x).$$

Essa função preserva os valores positivos e substitui por zero os valores negativos, introduzindo não linearidade no modelo e favorecendo o treinamento de redes profundas com baixo custo computacional.

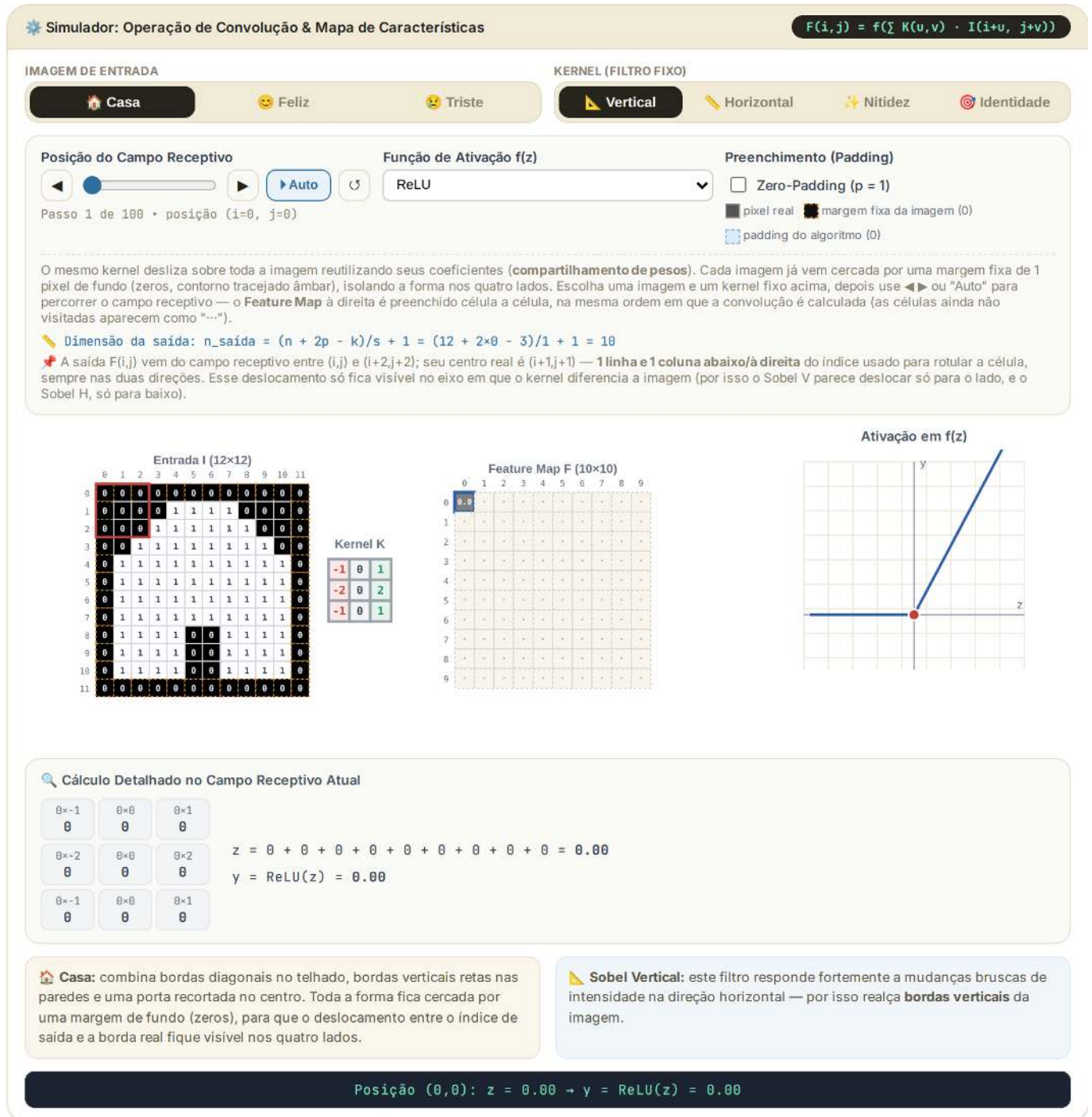


Figura 8.4: Simulador interativo da Camada Convolutiva: escolha entre três imagens de entrada 12×12 (casa, rosto feliz ou rosto triste) para observar como os mesmos *kernels* fixos reagem a diferentes bordas e formas. Navegue pelo campo receptivo com os botões ou reprodução automática, ajuste a função de ativação e alterne o zero-padding, acompanhando o mapa de características sendo revelado célula a célula, com o cálculo detalhado termo a termo e a fórmula da dimensão de saída em tempo real.

A Figure 8.5 ilustra o funcionamento da **ReLU** aplicada tanto a valores individuais quanto a um mapa de características, permitindo comparar a saída antes e depois da ativação.



Figura 8.5: Simulador interativo da função de ativação ReLU: arraste o controle para ver como valores negativos são zerados e valores positivos são preservados, tanto na curva quanto em um mapa de características real.

Os mapas de características resultantes da convolução e da ativação preservam a estrutura espacial da imagem. Em muitas arquiteturas, a etapa seguinte reduz sua resolução por meio de uma operação de *pooling*.

8.4.4 Pooling

A camada de *pooling* reduz a resolução espacial dos mapas de características, preservando as informações mais relevantes para as etapas seguintes do processamento. A operação mais utilizada é o **max-pooling**, que seleciona o maior valor em cada janela da imagem:

$$P(i, j) = \max_{(u, v) \in \text{janela}(i, j)} F(u, v).$$

Essa redução diminui o custo computacional das camadas subsequentes e torna a representação mais robusta a pequenas variações na posição dos padrões presentes na imagem.

A Figure 8.6 apresenta essa operação sobre um mapa de características de 8×8 pixels, reduzido para 4×4 por janelas de 2×2 com passo igual a 2, alternando entre **max-pooling** e **average-pooling** — que calcula, em vez do máximo, a média dos valores da janela correspondente.

Em conjunto, convolução, função de ativação e *pooling* formam o bloco básico utilizado na construção de uma CNN.

8.4.5 Treinamento de Redes Neurais: Como as CNNs Aprendem

Uma CNN aprende ajustando automaticamente seus parâmetros — os coeficientes dos filtros convolucionais, os pesos das camadas totalmente conectadas e os vieses (*biases*) — a partir de exemplos rotulados. Esse treinamento é iterativo e envolve três etapas: medir o erro produzido pela rede por meio de uma **função de perda** (*loss function*), calcular como esse erro depende de cada parâmetro por meio da **retropropagação** (*backpropagation*) e atualizar os parâmetros com um **algoritmo de otimização** (*optimizer*).



Figura 8.6: Simulador interativo de *pooling*: escolha entre *max-pooling* e *average-pooling* e avance passo a passo para observar a redução da resolução espacial do mapa de características.

8.4.5.1 Função de Perda (*Loss Function*)

A **função de perda** (*loss function*) quantifica a diferença entre a previsão da rede e a resposta correta, denominada **verdade de referência** (*ground truth*). O resultado é um escalar L : quanto menor a perda, mais próxima a previsão está da resposta esperada.

Em problemas de classificação multiclasse, a função mais utilizada é a **Entropia Cruzada** (*Cross-Entropy Loss*), aplicada às probabilidades produzidas pela camada **Softmax**:

$$L = - \sum_{c=1}^C y_c \log(\hat{y}_c),$$

em que C é o número de classes, y_c é o rótulo real em codificação *one-hot* e $\hat{y}_c$ é a probabilidade prevista para a classe c . A perda aproxima-se de zero quando a rede atribui alta probabilidade à classe correta e cresce rapidamente à medida que essa probabilidade diminui.

A Figure 8.7 ilustra esse comportamento: o simulador permite selecionar a classe correta e alterar as probabilidades produzidas pela *Softmax*, mostrando em tempo real a variação da função de perda.

8.4.5.2 Retropropagação (*Backpropagation*)

Após o cálculo da perda, é necessário determinar como cada parâmetro da rede contribui para esse resultado. Essa etapa é realizada pela **retropropagação** (*backpropagation*), que aplica a **Regra da Cadeia** do cálculo diferencial para obter o gradiente da função de perda em relação a cada parâmetro.

Para um parâmetro w , esse gradiente é dado por

$$\frac{\partial L}{\partial w}.$$



Figura 8.7: Simulador interativo da Função de Perda (*Cross-Entropy*): selecione a classe real da imagem (Casa, Feliz ou Triste) e ajuste as probabilidades estimadas pela Softmax para visualizar o cálculo da penalização escalar e o gráfico do logaritmo negativo em tempo real.

O gradiente indica como a perda varia em relação a pequenas alterações em w : um gradiente positivo indica que aumentar w aumenta a perda, e um gradiente negativo indica o efeito oposto.

A Figure 8.8 apresenta esse processo de forma visual, mostrando a propagação do gradiente da camada de saída até as primeiras camadas convolucionais.

8.4.5.3 Algoritmos de Otimização

Após o cálculo dos gradientes, um **algoritmo de otimização** (*optimizer*) atualiza os parâmetros da rede para reduzir a função de perda. Em redes profundas, essa busca ocorre em um espaço de alta dimensão e, em geral, **não convexo**, o que torna a otimização um problema desafiador.

Para facilitar a compreensão, a Figure 8.9 utiliza uma **superfície de perda simplificada**, com um mínimo global, um mínimo local e uma barreira entre essas regiões. O **mínimo global** corresponde ao menor valor da função de perda e representa o melhor conjunto de parâmetros da rede; um **mínimo local** também apresenta baixa perda, mas pode estar distante da melhor solução. Quando a otimização fica retida em um mínimo local, os ajustes de filtros, pesos e vieses tornam-se muito pequenos, e o treinamento para antes de alcançar um modelo com menor erro.

Gradiente Descendente Estocástico (*SGD*)

O **Gradiente Descendente Estocástico** (*Stochastic Gradient Descent* — *SGD*) atualiza os parâmetros na direção oposta ao gradiente:

$$w_{\text{novo}} = w_{\text{atual}} - \eta \frac{\partial L}{\partial w},$$

em que η é a **taxa de aprendizado** (*learning rate*), responsável por controlar o tamanho da atualização. O *SGD* utiliza apenas o gradiente da iteração atual; quando a busca alcança um mínimo local, os gradientes tornam-se muito pequenos e as atualizações praticamente cessam.



Figura 8.8: Simulador interativo de *Backpropagation*: avance os passos da Regra da Cadeia para acompanhar o fluxo do sinal de erro no sentido inverso da rede, observando a computação das derivadas parciais do gradiente em cada camada.

Otimizadores Adaptativos: *Adam*

O **Adam** (*Adaptive Moment Estimation*) combina estimativas adaptativas dos primeiros e segundos momentos dos gradientes (Kingma; Ba, 2015), adaptando a taxa de aprendizado de cada parâmetro individualmente. Essa adaptação favorece, em muitos casos, a superação de mínimos locais que reteriam o *SGD*.

A Figure 8.9 compara a trajetória do *SGD* e do *Adam* sobre a mesma superfície de perda não convexa.

8.4.6 Arquitetura de uma CNN

Uma *CNN* para classificação de imagens combina as camadas apresentadas nas seções anteriores. Durante o **passo à frente** (*forward pass*), a imagem percorre sucessivamente as camadas convolucionais, as funções de ativação, as operações de *pooling*, a etapa de **Flatten**, as camadas totalmente conectadas e, por fim, a camada **Softmax**, que produz as probabilidades das classes. Durante o treinamento, essa previsão é comparada ao rótulo correto para calcular a função de perda, realizar a retropropagação e atualizar os parâmetros por meio de um algoritmo de otimização.

A Figure 8.10 apresenta esse fluxo de processamento e treinamento.

Após o último bloco convolucional, a operação **Flatten** reorganiza os mapas de características em um vetor unidimensional, que alimenta as **camadas totalmente conectadas** (*fully connected layers*), responsáveis por combinar as características extraídas para produzir os escores (*logits*) de cada classe. A camada **Softmax** converte esses escores em uma distribuição de probabilidades, utilizada tanto para a classificação quanto para o cálculo da função de perda durante o treinamento.

A Figure 8.11 apresenta uma versão interativa dessa arquitetura, permitindo executar sucessivas etapas de treinamento e observar a redução da perda, a retropropagação dos gradientes e a atualização dos filtros da rede.

8.4.7 Como o Gradiente Ajusta os *Kernels* da Convolução

A compreensão do processo de aprendizado em uma Rede Neural Convolucional (CNN) requer a elucidação de um mecanismo fundamental: **como os coeficientes aleatórios de um filtro inicial se transformam em detectores precisos de bordas, texturas e padrões complexos.**

A resposta reside no princípio do **compartilhamento de pesos** (*weight sharing*). Durante a etapa de

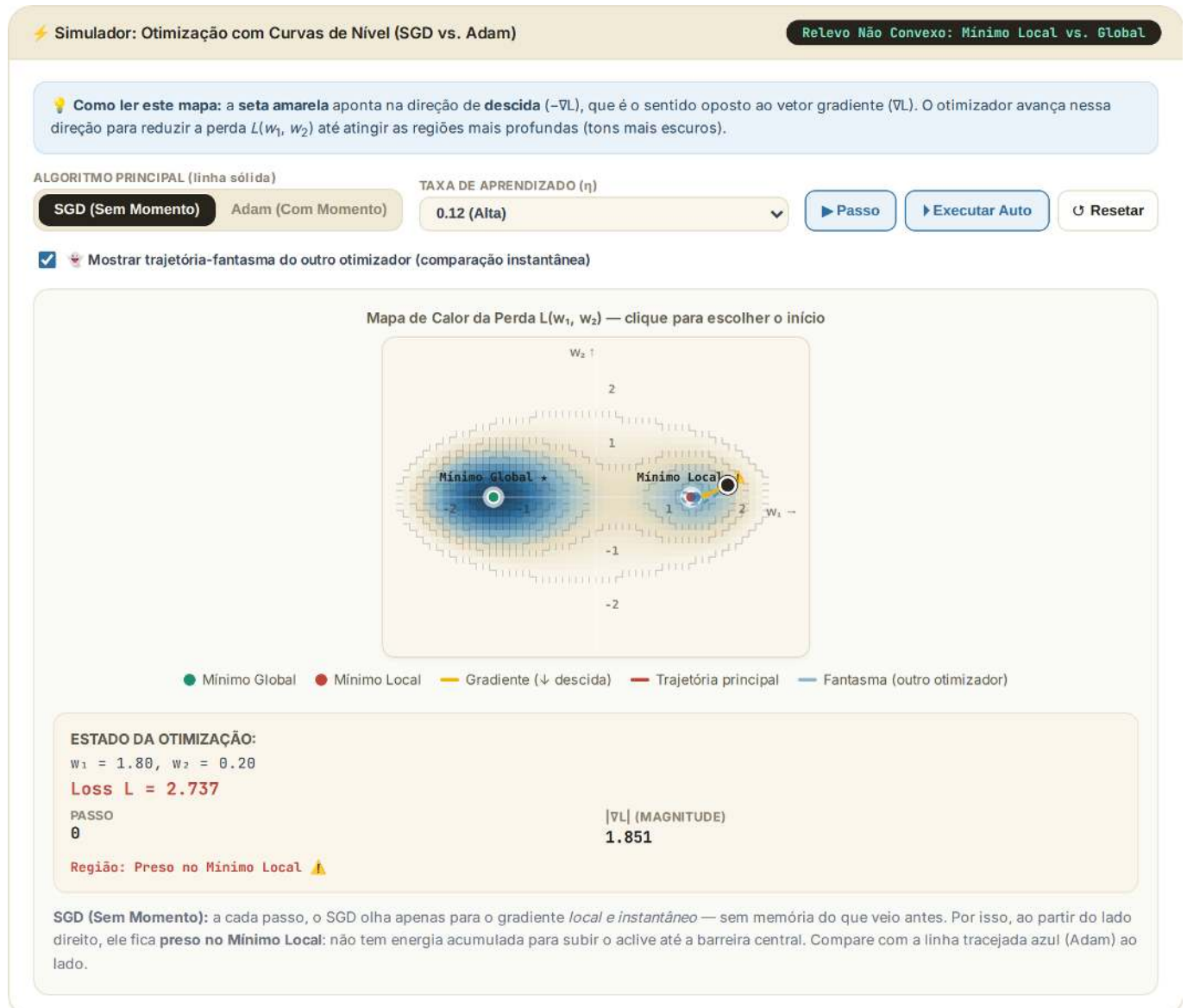


Figura 8.9: Simulador interativo dos Algoritmos de Otimização: compare a trajetória do SGD e do Adam sobre uma superfície de perda não convexa com mapa de calor e curvas de nível. A linha sólida mostra o otimizador selecionado avançando passo a passo; a linha tracejada mostra, para comparação instantânea, o caminho completo que o outro otimizador percorreria a partir do mesmo ponto inicial. Observe como o SGD fica retido no Mínimo Local à direita, enquanto o Adam pode ou não transpor a barreira central dependendo do impulso acumulado e da taxa de aprendizado. Clique em qualquer ponto do mapa para redefinir o ponto inicial dos pesos.

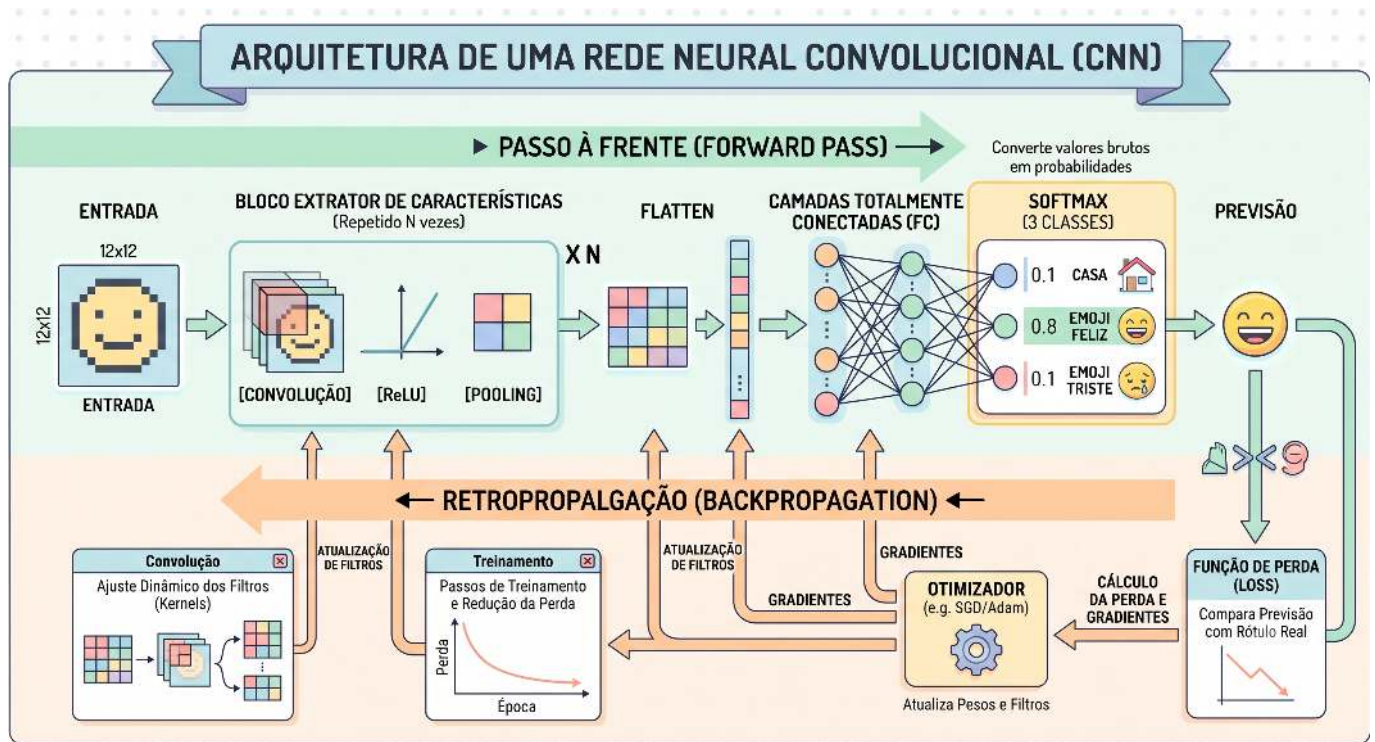


Figura 8.10: Arquitetura simplificada de uma *CNN* para classificação de imagens, destacando o *forward pass* e as etapas de treinamento por meio da função de perda, da retropropagação e do algoritmo de otimização.

propagação à frente (*forward pass*), o mesmo filtro de dimensão 3×3 desliza por toda a extensão da imagem de entrada. Consequentemente, cada peso do *kernel* — como o elemento $K[0][0]$ no canto superior esquerdo — é reutilizado múltiplas vezes ao longo das diferentes regiões espaciais do dado de entrada.

Durante a etapa de retropropagação (*backpropagation*), essa reutilização estabelece uma dinâmica direta: cada posição espacial processada pelo filtro gera uma contribuição individual (“voto”) para a atualização do respectivo peso.

8.4.7.1 A Intuição por Trás do Cálculo

Seja $Z[r][c]$ o mapa de características pré-ativação na posição (r, c) da janela deslizante, obtido pela correlação cruzada entre o *kernel* K e a entrada X :

$$Z[r][c] = \sum_{k_r} \sum_{k_c} K[k_r][k_c] \cdot X[r + k_r][c + k_c]$$

Aplicando a regra da cadeia para determinar a contribuição de um peso específico $K[k_r][k_c]$ na função de perda L , obtêm-se as seguintes etapas:

1. **Erro Local (dZ):** Em cada posição (r, c) , calcula-se a derivada parcial da função de perda em relação à pré-ativação:

$$dZ[r][c] = \frac{\partial L}{\partial Z[r][c]}$$

que quantifica a responsabilidade dessa posição específica no erro total da rede (L).

2. **Contribuição do Peso:** Como $\frac{\partial Z[r][c]}{\partial K[k_r][k_c]} = X[r + k_r][c + k_c]$, a influência de um peso específico $K[k_r][k_c]$ no erro da posição (r, c) é obtida multiplicando-se o erro local $dZ[r][c]$ pelo valor do pixel de entrada alinhado a esse peso no instante do cálculo:

$$dZ[r][c] \cdot X[r + k_r][c + k_c]$$

Simulador: Arquitetura Completa de uma CNN
Entrada (12×12) → Conv → Pool → FC → Softmax

IMAGEM DE ENTRADA DO PIPELINE (12×12)

Casa
Feliz
Triste

Entrada
12×12

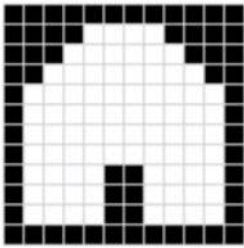
Conv1 + ReLU
10×10×4

Pooling1
5×5×4

Flatten
100 valores

Camada Densa (FC)
16 neurônios

Softmax
3 classes



Entrada — dimensão: 12×12

A **imagem de entrada** é representada como uma matriz de pixels 12×12. É a mesma estrutura (casa, rosto feliz ou triste) do simulador anterior.

Imagem em escala de cinza 12×12 com margem fixa de zeros.

Treinamento (Forward Pass + Retropropagação)

Entrada → Conv+ReLU → Pool → Flatten → FC → Softmax → Previsão
Perda ← Otimizador ← Retropropagação ← (a cada passo)

PERDA (LOSS)

perda aparecerá aqui

ACURÁCIA

acurácia aparecerá aqui

Loss: —

Acurácia: —

Passo de treinamento: 0

Passo Único
Treinar
Pesos Novos

Os *kernels* e pesos começam **aleatórios** (não são mais os filtros fixos do simulador anterior). A cada passo, a rede faz o *forward pass* nas 3 imagens, calcula a perda (*cross-entropy*) e a **acurácia** (quantas das 3 imagens são classificadas corretamente), retropropaga o erro e ajusta todos os pesos (inclusive os *kernels* da convolução) via gradiente descendente. ⚠ Como o "conjunto de treino" tem apenas 3 exemplos, isso demonstra o **mecanismo** do treinamento (perda caindo, acurácia subindo, pesos mudando) — não a capacidade de generalizar para imagens novas, que exigiria muito mais dados.

Figura 8.11: Simulador interativo da arquitetura de uma CNN: escolha uma das imagens de entrada 12×12 (casa, rosto feliz ou triste), clique em cada bloco do *pipeline* — Entrada, Conv+ReLU, Pooling, Flatten, FC e Softmax — e execute passos de treinamento reais (*forward pass* + retropropagação) para observar a perda e a acurácia evoluindo, os *kernels* sendo ajustados e o Softmax passando a apontar a classe correta.

Francisco de Assis Zampirolli

UFABC

424

3. **Acúmulo de Gradientes:** O gradiente final do peso corresponde à **soma das contribuições (“votos”) de todas as posições** percorridas pela janela deslizante:

$$\frac{\partial L}{\partial K[k_r][k_c]} = \sum_{(r,c)} dZ[r][c] \cdot X[r + k_r][c + k_c]$$

Essa formulação assegura paridade direta entre a derivação analítica e os valores computados no simulador de inspeção do gradiente (Figure 8.12).

8.4.7.2 O Papel da Função ReLU como “Filtro de Relevância”

A aplicação da função de ativação **ReLU** ($\max(0, z)$) imediatamente após a convolução introduz uma propriedade de seletividade ao gradiente:

- **Ativação Positiva** ($Z[r][c] > 0$): A derivada da ReLU é 1. O erro local é propagado integralmente ($dZ \neq 0$), permitindo que a posição contribua para a atualização dos pesos do *kernel*.
- **Ativação Inativa** ($Z[r][c] \leq 0$): A derivada da ReLU é 0. O erro local é anulado ($dZ = 0$), suprimindo a contribuição da posição para o gradiente final.

Nota didática: A ReLU assegura que apenas as regiões espaciais que produziram respostas ativas durante a propagação à frente possam capacidade de modificar os pesos do *kernel* no processo de retropropagação.

8.4.7.3 Atualização dos Pesos via Gradiente Descendente

Após a consolidação dos gradientes acumulados de todas as posições, a atualização do peso ocorre segundo o algoritmo do Gradiente Descendente Estocástico (SGD):

$$K[k_r][k_c] \leftarrow K[k_r][k_c] - \eta \cdot \frac{\partial L}{\partial K[k_r][k_c]}$$

em que η denota a **taxa de aprendizado** (*learning rate*).

- Se a soma dos gradientes for **positiva**, o valor do peso é reduzido.
- Se a soma for **negativa**, o valor do peso é incrementado.

8.4.7.4 Explorando o Simulador Interativo

Da Arquitetura Global à Inspeção do Gradiente

No simulador de arquitetura (Figure 8.11), observa-se o erro dZ derivado da retropropagação multicamada completa, originado na perda de entropia cruzada (*Softmax*) sobre as imagens de entrada 12×12 .

Para viabilizar a verificação analítica do gradiente sem a sobrecarga de 100 posições de convolução e retropropagação multicamada, o simulador de aprendizado do *kernel* (Figure 8.12) adota um modelo de inspeção reduzido (6×6). Nesse cenário, simplifica-se o problema substituindo a classificação complexa por uma **metadado de calibração escalar**: ajusta-se o filtro para produzir uma resposta acumulada predefinida (alvo = 9) ao identificar um padrão específico (como uma borda a 45 graus). O mecanismo de acúmulo de gradientes ($dZ \cdot X$) permanece rigorosamente idêntico em ambas as formulações.

Para inspecionar essa dinâmica em nível numérico, utiliza-se o simulador na Figure 8.12:

- **Imagem de Entrada** (X): Matriz 6×6 .
- **Filtro Convolutivo** (K): Matriz 3×3 (9 pesos).
- **Mapa de Saída** (Z / A): Matriz 4×4 (16 posições da janela).

- **Função de Perda (L):** Definida por $L = \frac{1}{2}(S - \text{alvo})^2$, em que $S = \sum A[r][c]$ representa a soma global das ativações pós-ReLU.

O papel do alvo = 9: O valor escalar alvo = 9 representa a “energia de ativação” ideal estipulada para a imagem com borda diagonal. Como o mapa A possui 16 posições, esse valor equivale a buscar uma resposta média de $\frac{9}{16} \approx 0,56$ por pixel ativado. Quando $S > 9$, a rede identifica que o filtro está reagindo com intensidade excessiva ao padrão, gerando um erro $dZ > 0$ que força a redução dos pesos K . Quando $S < 9$, os pesos são incrementados para amplificar o sinal.

Roteiro Sugerido de Experimentação:

1. **Seleção de Peso:** Na grade 3×3 , escolha o peso a ser analisado (ex.: $K[0][0]$).
2. **Varredura da Janela:** Utilize o botão “▷ Avançar posição” para acompanhar o deslocamento da janela pelas 16 posições espaciais. Note o destaque visual na célula do mapa de entrada que alinha o pixel X ao peso selecionado.
3. **Análise do Voto Local:** Examine o produto do erro local pelo pixel de entrada ($dZ \cdot X$) no painel de cálculo da posição.
4. **Verificação do Histórico:** Acompanhe a consolidação dos 16 resultados parciais organizados nas quatro colunas de histórico, observando o acúmulo do gradiente final.
5. **Atualização do Kernel:** Clique em “▷ Aplicar passo de gradiente descendente” para visualizar a convergência da curva de perda e a adaptação do *kernel* aleatório ao padrão de entrada selecionado.

Síntese — Da convolução ao aprendizado de representações

Os simuladores desta seção demonstram, de forma sequencial, como uma *CNN* transforma uma imagem de entrada em uma estimativa probabilística e como seus parâmetros são otimizados durante o treinamento:

- **Convolução:** aplica filtros sobre a imagem para extrair características locais, gerando mapas de características por meio do **compartilhamento de pesos**.
- **ReLU:** introduz não linearidade ao sistema, permitindo a modelagem de relações complexas entre os dados.
- **Pooling:** reduz a resolução espacial dos mapas de características, diminuindo o custo computacional e conferindo invariância a pequenas translações locais.
- **Flatten:** reorganiza os mapas multidimensionais em um vetor unidimensional para alimentação das camadas subsequentes.
- **Camada totalmente conectada:** combina as características extraídas para produzir os escores brutos (*logits*) associados a cada classe.
- **Softmax:** converte os *logits* em uma distribuição de probabilidades normalizada.
- **Função de perda:** compara a distribuição prevista com a verdade de referência (*ground truth*), quantificando escalarmente o erro da rede.
- **Retropropagação:** aplica a regra da cadeia para calcular a derivada parcial (gradiente) da função de perda em relação a cada parâmetro treinável.
- **Otimizador:** atualiza os coeficientes dos filtros, pesos e vieses na direção oposta ao gradiente, reduzindo a perda a cada iteração.

Ao longo das iterações, os filtros convolucionais convertem-se de valores estocásticos em detectores especializados: as camadas iniciais aprendem primitivas visuais de baixo nível (como bordas e texturas), enquanto as camadas mais profundas consolidam essas representações em estruturas abstratas e semânticas.

8.5 Projetos Práticos: Do Conceito à Implementação

Após a consolidação teórica dos fundamentos das CNNs e a verificação visual de cada uma de suas operações elementares por meio dos simuladores interativos, torna-se essencial observar a integração dessas etapas em *pipelines* completos de programação.

Simulador: Gradiente de um Peso do Kernel
 $\partial \text{Perda} / \partial K[kr][kc] = \sum dZ \cdot X$

PADRÃO DE ENTRADA (IMAGEM 6x6) ?

☒ Borda diagonal
 ☐ Borda vertical

Exemplo reduzido: imagem 6x6 e filtro 3x3 gerando mapas 4x4. Clique nas abas "Como é calculado?" abaixo de cada matriz para entender as contas passo a passo.

1. PESO DO KERNEL ?

K[0]	K[0]	K[0]
[0]	[1]	[2]
K[1]	K[1]	K[1]
[0]	[1]	[2]
K[2]	K[2]	K[2]
[0]	[1]	[2]

KERNEL ATUAL (K) ?

-0.49	-0.44	0.48
0.20	0.02	-0.09
-0.03	-0.26	0.05

Como é atualizado?

ENTRADA X (6x6) ?

1	0	0	0	0	0
1	1	0	0	0	0
1	1	1	0	0	0
1	1	1	1	0	0
1	1	1	1	1	0
1	1	1	1	1	1

Como funciona X?

PRÉ-ATIVACÃO Z (4x4) ?

-0.51	-0.09	-0.03	0.00
-1.04	-0.51	-0.09	-0.03
-0.56	-1.04	-0.51	-0.09
-0.56	-0.56	-1.04	-0.51

Como calcula Z?

ATIVACÃO A (4x4) ?

0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00

Como calcula A?

ERRO dZ (4x4) ?

0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00

S = 0.000 (alvo = 9)
L = $\frac{1}{2} \cdot (0.000 - 9)^2 = 40.5000$

Como calcula dZ, S e Loss?

2. CÁLCULO E SOMA DOS "VOTOS" DE CADA POSIÇÃO ?

$$\partial L / \partial K[0][0] = \sum_{(r,c)} dZ[r][c] \cdot X[r+0][c+0]$$

Voltar posição
Avançar posição
Somar tudo
Reiniciar posições

CÁLCULO DESTA POSIÇÃO ?

Clique em "Avançar posição" para ver a primeira conta.

SOMA ACUMULADA (GRADIENTE) ?

posições: 0 / 16

total = 0.000

HISTÓRICO DAS 16 POSIÇÕES (COLUNAS c=0, c=1, c=2, c=3) ?

p(0,0) 0.00x1=0.00	p(0,1) 0.00x0=0.00	p(0,2) 0.00x0=0.00	p(0,3) 0.00x0=0.00
p(1,0) 0.00x1=0.00	p(1,1) 0.00x1=0.00	p(1,2) 0.00x0=0.00	p(1,3) 0.00x0=0.00
p(2,0) 0.00x1=0.00	p(2,1) 0.00x1=0.00	p(2,2) 0.00x1=0.00	p(2,3) 0.00x0=0.00
p(3,0) 0.00x1=0.00	p(3,1) 0.00x1=0.00	p(3,2) 0.00x1=0.00	p(3,3) 0.00x1=0.00

3. USE O GRADIENTE PARA ATUALIZAR O KERNEL ?

▶ Aplicar passo de gradiente descendente

Desfazer passo

Novo kernel aleatório

perda aparecerá aqui

Atualizações: 0

PERDA AO LONGO DAS ATUALIZAÇÕES ?

Estado Inicial: Peso atual K[0][0] = -0.49. Gradiente total acumulado (soma dos 16 votos) = 0.000.
De onde vem essa soma (0.000)?

- A janela desliza pelas 16 posições da saída (4 linhas x 4 colunas).
- Em cada posição (r,c), multiplica-se o erro daquela posição (dZ[r][c]) pelo pixel de entrada correspondente (X).
- A soma de todos esses 16 produtos ("votos") resulta em: $\sum_{(r,c)} dZ[r][c] \cdot X = 0.000$.

Clique em "▶ Aplicar passo de gradiente descendente" para ver o ajuste em tempo real.

Figura 8.12: Simulador interativo do cálculo do gradiente de um peso do *kernel* convolucional com explicação detalhada da soma do gradiente inicial e alinhamento à esquerda.

Nas seções a seguir, a teoria é traduzida em código executável em **PyTorch**, explorando as três tarefas fundamentais da Visão Computacional: **classificação** (Projeto Prático 1), **detecção de objetos** (Projeto Prático 2) e **segmentação semântica** (Projeto Prático 3). Essa progressão prática permite analisar desde a construção de uma arquitetura convolucional treinada do zero até a aplicação de estratégias avançadas de **transferência de aprendizado** (*transfer learning*) em modelos pré-treinados para conjuntos de dados sintéticos e reais.

8.6 Projeto Prático 1: Uma CNN do Zero para os Dígitos

Para estabelecer uma comparação direta com as abordagens apresentadas no Capítulo 7, desenvolve-se nesta seção uma *CNN* treinada sobre o mesmo conjunto de dados de dígitos manuscritos (`load_digits`). A diferença fundamental reside na etapa de representação: enquanto os métodos clássicos dependem de *pixels* brutos ou de descritores calculados manualmente, como o *Histogram of Oriented Gradients* (*HOG*), a *CNN* aprende automaticamente os coeficientes dos filtros convolucionais durante o processo de otimização.

Os códigos a seguir (consolidados na Figure 8.15) realizam a preparação dos dados, definem uma arquitetura convolucional simples em **PyTorch**, executam o laço de treinamento por meio do algoritmo *Adam* e geram as curvas de evolução da função de perda e da acurácia.

8.6.1 Detalhamento do Bloco 1: Preparação e Estruturação dos Dados

A etapa inicial de qualquer *pipeline* de *Deep Learning* consiste na conversão e adequação dos dados de entrada para o formato exigido pelo *framework* de computação científica.

8.6.1.1 O Conceito de *Tensor*

Em *Deep Learning*, a estrutura fundamental de dados é o **tensor**. Do ponto de vista computacional, um *tensor* consiste em um arranjo multidimensional de números generalizado para n dimensões:

- Um *tensor* de ordem 0 é um escalar (um único valor).
- Um *tensor* de ordem 1 é um vetor (comprimento).
- Um *tensor* de ordem 2 é uma matriz (linhas e colunas).
- Um *tensor* de ordem 3 ou superior representa um volume ou hiper-arranjo de dados.

No contexto do **PyTorch**, a classe `torch.Tensor` estende a funcionalidade de arranjos numéricos multidimensionais (como os do **NumPy**) ao oferecer suporte a operações aceleradas em hardware por meio de *GPUs* (*Graphics Processing Units*) e suporte ao cálculo automático de derivadas (*autograd*), essencial para o algoritmo de retropropagação.

8.6.1.2 Análise do Código de Pré-processamento

1. **Carregamento e Normalização das Intensidades:** O conjunto `load_digits` possui 1.797 amostras de dígitos manuscritos de 8×8 *pixels*, cujas intensidades originais variam na escala inteira de 0 a 16. A divisão por 16.0 realiza a **normalização** dos dados para a faixa $[0.0, 1.0]$. Essa transformação em escala flutuante (`float32`) é indispensável em redes neurais para evitar a saturação das funções de ativação e estabilizar o cálculo dos gradientes no algoritmo de otimização.
2. **Divisão Estratificada (70% Treinamento / 30% Teste):** A função `train_test_split` separa 70% das amostras para o ajuste dos parâmetros da rede e reserva 30% para a avaliação do modelo em dados não vistos. O parâmetro `stratify=y` garante a amostragem estratificada, mantendo a proporção exata de cada uma das 10 classes de dígitos (0 a 9) em ambos os conjuntos, prevenindo viés de distribuição.
3. **Adequação Dimensional para Convolução 2D (`unsqueeze`):** Nas *CNNs*, camadas convolucionais bidimensionais (`nn.Conv2d`) exigem que o *tensor* de entrada possua estritamente 4 dimensões na ordenação (N, C, H, W):

- N : número de amostras (*batch size*).
- C : número de canais de cor (1 para escala de cinza, 3 para *RGB*).
- H : altura da imagem em *pixels* (8).
- W : largura da imagem em *pixels* (8).

Como o arranjo original possui formato 3D do tipo $(N, 8, 8)$, a chamada `.unsqueeze(1)` insere uma dimensão unitária especificamente no **índice 1** (a posição reservada para o canal de cor C), transformando a estrutura em um *tensor* 4D de formato $(N, 1, 8, 8)$, conforme exigido pelo PyTorch.

4. **Conversão dos Rótulos (`dtype=torch.long`):** Os rótulos das classes y são convertidos em *tensors* inteiros de 64 *bits* (`torch.long`). Essa especificação de tipo é uma exigência da função de perda de Entropia Cruzada (`nn.CrossEntropyLoss`), que utiliza inteiros não negativos como índices para associar a classe correta aos *logits* de saída da rede.

i Atenção às Dimensões: A estrutura final é representada pelo **tensor** $(N, 1, 8, 8)$, onde N é o número de amostras (*batch size*), 1 é o canal de cor (escala de cinza) e 8×8 é a resolução espacial da imagem em *pixels*.

A Figure 8.13 ilustra uma sequência de amostras do conjunto de treinamento após o pré-processamento e adequação dimensional aos *tensors* do **PyTorch**. Na etapa de exibição, a chamada `img.squeeze().numpy()` encadeia duas transformações: o método `.squeeze()` elimina a dimensão unitária redundante do canal de cor, reduzindo o *tensor* 3D de formato $(1, 8, 8)$ para uma matriz 2D de $(8, 8)$; em seguida, o método `.numpy()` converte a estrutura do **PyTorch** em uma matriz nativa do **NumPy**, formato exigido pelas ferramentas de renderização gráfica como a `mm.show()`.

```
# 1. Carregamento e pré-processamento dos dados
digits = load_digits()
X = digits.images.astype(np.float32) / 16.0 # Normalização para a faixa [0, 1]
y = digits.target

# Divisão estratificada em conjuntos de treinamento (70%) e teste (30%)
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Adequação à dimensão esperada pelo PyTorch: (N_amostras, Canais, Altura, Largura)
X_treino_t = torch.tensor(X_treino).unsqueeze(1) # Dimensão: (N, 1, 8, 8)
y_treino_t = torch.tensor(y_treino, dtype=torch.long)
X_teste_t = torch.tensor(X_teste).unsqueeze(1)
y_teste_t = torch.tensor(y_teste, dtype=torch.long)

# Exibir uma amostra
n_amostras = 8
imgs = [img.squeeze().numpy() for img in X_treino_t[:n_amostras]]
imgs_titles = [str(label.item()) for label in y_treino_t[:n_amostras]]
mm.show(imgs, titles=imgs_titles, cols=n_amostras, figsize=(12, 2.5))
```

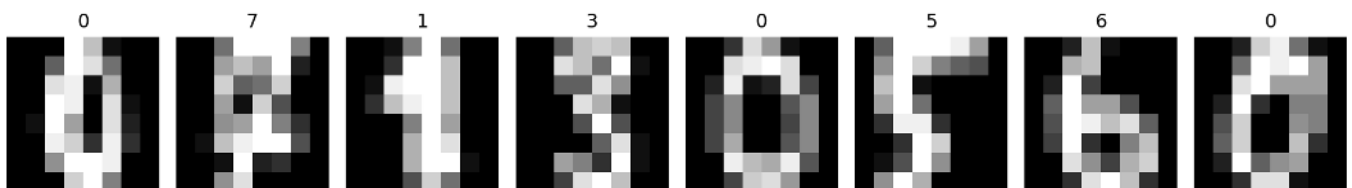


Figura 8.13: Amostras de dígitos do conjunto de treinamento após conversão para *tensores* PyTorch e normalização.

8.6.2 Detalhamento do Bloco 2: Definição da Arquitetura Convolutacional

A construção de modelos no **PyTorch** é estruturada a partir do paradigma de orientação a objetos, criando-se uma classe específica para representar a rede neural (neste exemplo, a classe **CNNDigitos**), a qual herda todas as funcionalidades da classe base **nn.Module**. O construtor **__init__** é responsável por instanciar as camadas e declarar seus parâmetros treináveis, enquanto o método **forward** estabelece a sequência numérica da propagação à frente (*forward pass*).

A Figura 8.14 sintetiza as transformações espaciais dos *tensors* e o fluxo de dados ao longo da classe **CNNDigitos**.



Figura 8.14: Representação do fluxo de transformações dimensionais dos *tensors* ao longo da arquitetura **CNNDigitos**.

1. Construtor (**__init__**) e Instanciação dos Componentes:

- **Camada Convolutacional 1 (**self.conv1**):** Aplica 8 filtros 3×3 com **padding=1** sobre a entrada em escala de cinza (1 canal), preservando a resolução espacial de 8×8 *pixels*.
- **Camada Convolutacional 2 (**self.conv2**):** Processa os 8 mapas de características recebidos da camada anterior aplicando 16 filtros 3×3 com **padding=1**.
- **Subamostragem (**self.pool**):** Instancia a operação de *Max-Pooling* com janela 2×2 e passo (*stride*) 2, reduzindo a dimensão espacial (altura e largura) pela metade a cada aplicação.
- **Camadas Totalmente Conectadas (**self.fc1** e **self.fc2**):** A primeira projeção densa recebe o *tensor* achatado de dimensão $16 \times 2 \times 2 = 64$ e produz 32 características intermediárias. A segunda projeta essas 32 características nos 10 *logits* finais de saída.

2. Propagação à Frente no Método **forward**:

- **Primeiro Bloco Convolutacional:** O *tensor* de entrada de formato $(N, 1, 8, 8)$ passa por **conv1** + ReLU e é subamostrado por **pool**, resultando no formato $(N, 8, 4, 4)$.
- **Segundo Bloco Convolutacional:** O *tensor* $(N, 8, 4, 4)$ é processado por **conv2** + ReLU e reduzido por **pool** para o formato $(N, 16, 2, 2)$.
- **Achatamento (**Flatten**):** O método **x.view(x.size(0), -1)** reconfigura a estrutura 3D em um vetor 1D de 64 elementos por amostra, preservando a dimensão do lote *N*.
- **Classificação:** O vetor de 64 elementos alimenta **fc1** com ativação ReLU (32 neurônios) e finaliza em **fc2**, produzindo os 10 *logits* não normalizados para o cálculo da função de perda.

```

# 2. Definição da Arquitetura Convolutacional
class CNNDigitos(nn.Module):
    """
    Arquitetura convolutacional compacta:
    2 camadas convolucionais com ReLU e Max-Pooling + 2 camadas densas.
    """
    def __init__(self, n_classes=10):
        super().__init__()

        # Conv1: 1 canal de entrada, 8 filtros 3x3 com padding 1 (saída: 8x8)
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3, padding=1)
        # Conv2: 8 canais de entrada, 16 filtros 3x3 com padding 1 (saída: 4x4)
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, padding=1)

        # Max-Pooling 2x2 com passo (stride) 2
        self.pool = nn.MaxPool2d(2, 2)

        # Camadas totalmente conectadas (FC)
        self.fc1 = nn.Linear(16 * 2 * 2, 32)
        self.fc2 = nn.Linear(32, n_classes)
        self.relu = nn.ReLU()
  
```



```
def forward(self, x):
    # Primeiro bloco: Conv (8x8) -> ReLU -> Pool (4x4)
    x = self.pool(self.relu(self.conv1(x)))

    # Segundo bloco: Conv (4x4) -> ReLU -> Pool (2x2)
    x = self.pool(self.relu(self.conv2(x)))

    # Achatamento (Flatten): reconfigura a matriz 3D (16, 2, 2) em vetor 1D (64)
    x = x.view(x.size(0), -1)

    # Camada densa intermediária com ReLU
    x = self.relu(self.fc1(x))

    # Camada final de classificação (logits)
    return self.fc2(x)
```

8.6.2.1 Análise das Camadas e do Fluxo da Classe CNNDigitos

1. Construtor (`__init__`) e Instanciação dos Componentes:

- **Camada Convolutiva 1 (`self.conv1`):** Aplica 8 filtros 3×3 com `padding=1` sobre a entrada em escala de cinza (1 canal), preservando a resolução de 8×8 *pixels*.
- **Camada Convolutiva 2 (`self.conv2`):** Processa os 8 mapas de características recebidos aplicando 16 filtros 3×3 com `padding=1`.
- **Subamostragem (`self.pool`):** Instancia a operação de *Max-Pooling* com janela 2×2 e passo (*stride*) 2, reduzindo as dimensões espaciais (altura e largura) pela metade a cada aplicação.
- **Camadas Totalmente Conectadas (`self.fc1` e `self.fc2`):** A primeira projeção densa recebe o *tensor* achatado de dimensão $16 \times 2 \times 2 = 64$ e produz 32 características intermediárias. A segunda projeta essas 32 características nos 10 *logits* de saída.

2. Propagação à Frente no Método `forward`:

- **Primeiro Bloco:** O *tensor* ($N, 1, 8, 8$) passa por `conv1` + ReLU e é reduzido por `pool` para ($N, 8, 4, 4$).
- **Segundo Bloco:** O *tensor* ($N, 8, 4, 4$) passa por `conv2` + ReLU e é reduzido por `pool` para ($N, 16, 2, 2$).
- **Achatamento (*Flatten*):** O método `x.view(x.size(0), -1)` converte a estrutura 3D em um vetor 1D de 64 elementos por amostra.
- **Classificação:** O vetor de 64 elementos alimenta `fc1` com ativação ReLU (32 neurônios) e finaliza em `fc2`, que produz os 10 *logits* finais para o cálculo da perda de Entropia Cruzada.

8.6.3 Detalhamento do Bloco 3: Instanciação e Parâmetros de Otimização

A etapa de configuração do aprendizado exige a instanciação da arquitetura definida e a escolha de dois componentes centrais: a **função de perda**, que quantifica o erro do modelo, e o **algoritmo de otimização**, responsável por ajustar os parâmetros em direção ao mínimo dessa função.

1. **Instanciação e Contagem de Parâmetros:** O modelo é criado a partir da instanciação do objeto `modelo_cnn` da classe `CNNDigitos`. A expressão `sum(p.numel() for p in modelo_cnn.parameters())` percorre todos os *tensors* de parâmetros treináveis da rede (pesos e vieses de cada camada) e calcula a cardinalidade total do modelo, quantificando sua capacidade de representação.
2. **Função de Perda (`nn.CrossEntropyLoss`):** A perda de Entropia Cruzada (*Cross-Entropy Loss*) é a escolha padrão para problemas de classificação multiclasse. No PyTorch, essa implementação combina internamente a aplicação da função *LogSoftmax* com a Perda de Log-Verossimilhança Negativa (*NLLLoss*). Por essa razão, a camada de saída da rede produz *logits* brutos, dispensando a aplicação explícita da função *Softmax* ao final do método `forward`.
3. **Otimizador Adaptativo (`optim.Adam`):** A atualização dos parâmetros utiliza o algoritmo *Adam*

(*Adaptive Moment Estimation*), com taxa de aprendizado inicial $\eta = 0,01$ ($\text{lr}=1\text{e-}2$). O *Adam* combina os princípios do momento com a adaptação do tamanho do passo baseada na média móvel das derivadas de primeira e segunda ordens, ajustando individualmente a taxa de aprendizado de cada parâmetro da rede.

```
# 3. Inicialização do Modelo e Parâmetros de Otimização
modelo_cnn = CNNDigitos()
num_params = sum(p.numel() for p in modelo_cnn.parameters())
print(f"Parâmetros treináveis do modelo: {num_params}")

criterio = nn.CrossEntropyLoss()
otimizador = optim.Adam(modelo_cnn.parameters(), lr=1e-2)
```

Parâmetros treináveis do modelo: 3658

8.6.4 Detalhamento do Bloco 4: Laço de Treinamento e Avaliação

O treinamento de uma *CNN* ocorre de forma iterativa por meio do algoritmo de Gradiente Descendente Estocástico por *mini-batches* (*Mini-batch SGD*).

As curvas de aprendizado resultantes desse processo são apresentadas na Figure 8.15, gerada ao final da execução.

1. **Fase de Treinamento (`modelo_cnn.train()`):** O laço principal executa o treinamento ao longo de 50 épocas. Em cada época, ocorrem as seguintes etapas:
 - **Embaralhamento Estocástico:** A função `torch.randperm(n)` gera uma permutação aleatória dos índices das amostras, garantindo que a ordenação dos *mini-batches* varie a cada época para evitar vícios de amostragem.
 - **Divisão em *Mini-batches*:** O conjunto de treinamento é fatiado em lotes de 32 amostras (`tam_lote = 32`).
 - **Zerar Gradientes (`otimizador.zero_grad()`):** Limpa os gradientes acumulados no *tensor* na iteração anterior, evitando a soma indesejada de derivadas entre lotes distintos.
 - **Passo à Frente e Perda:** O *forward pass* calcula as previsões `saida`, e a chamada `criterio(saida, y_treino_t[idx])` quantifica o erro do lote.
 - **Retropropagação (`perda.backward()`):** Aplica a regra da cadeia para calcular as derivadas parciais da perda em relação a cada parâmetro ($\frac{\partial L}{\partial w}$).
 - **Atualização dos Pesos (`otimizador.step()`):** Atualiza os parâmetros do modelo segundo as equações do otimizador *Adam*.
2. **Fase de Avaliação (`modelo_cnn.eval()`):** Ao final de cada época, o modelo é alterado para o modo de avaliação. O contexto `with torch.no_grad()` desativa temporariamente o motor de cálculo automático de derivadas (*autograd*), reduzindo o consumo de memória e acelerando a inferência sobre o conjunto de teste (`X_teste_t`). A operação `.argmax(dim=1)` extrai a classe de maior probabilidade para cada amostra, permitindo calcular a acurácia de teste.
3. **Visualização com a Biblioteca *morph*:** A função `mm.showTrainCurves` da biblioteca didática *morph* consolida o histórico de perda de treinamento e a acurácia de teste em um único painel gráfico, permitindo diagnosticar a convergência do modelo e monitorar a estabilidade do aprendizado ao longo das épocas.

```
# 4. Laço de Treinamento (Mini-batch SGD)
n = X_treino_t.size(0)
tam_lote = 32
epocas = 50
historico_perda, historico_acc = [], []

for epoca in range(epocas):
```

```
modelo_cnn.train()
perm = torch.randperm(n)
perda_epoca = 0.0
for i in range(0, n, tam_lote):
    idx = perm[i:i + tam_lote]
    otimizador.zero_grad()
    saida = modelo_cnn(X_treino_t[idx])
    perda = criterio(saida, y_treino_t[idx])
    perda.backward()
    otimizador.step()
    perda_epoca += perda.item() * len(idx)

# Avaliação do modelo no conjunto de teste ao final de cada época
modelo_cnn.eval()
with torch.no_grad():
    pred_teste = modelo_cnn(X_teste_t).argmax(dim=1)
    acc_teste = (pred_teste == y_teste_t).float().mean().item()
    historico_perda.append(perda_epoca / n)
    historico_acc.append(acc_teste)

acc_final_cnn = historico_acc[-1]
print(f"Acurácia final da CNN no conjunto de teste: {acc_final_cnn:.4f}")

final = mm.showTrainCurves(
    historico_perda, historico_acc,
    titulo="Evolução do Treinamento da CNN - Base de Dígitos",
    subtitulo=f"Acurácia final no teste: {acc_final_cnn:.4f}",
)
```

Acurácia final da CNN no conjunto de teste: 0.9759

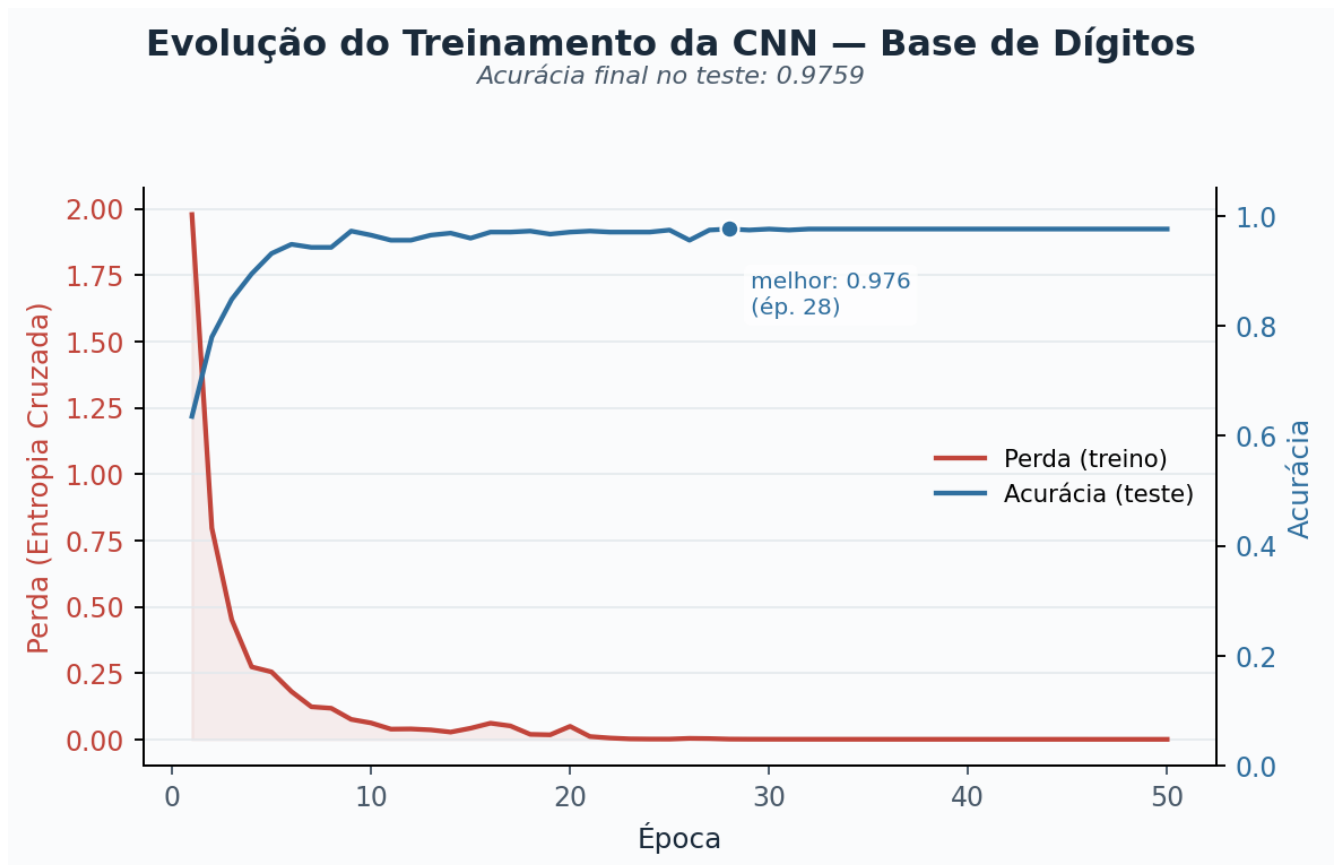


Figura 8.15: Curvas de treinamento e avaliação da CNN na base de dígitos: evolução da perda de entropia cruzada no conjunto de treinamento e da acurácia no conjunto de teste ao longo de 50 épocas.

8.6.5 Detalhamento do Bloco 5: Visualização do Fluxo de Ativações

A inspeção da rede treinada permite observar a transformação progressiva do *tensor* de entrada ao longo das camadas da arquitetura `CNNDígitos`. A Figure 8.16 ilustra as dimensões e as ativações intermediárias obtidas ao processar um exemplo real do dígito 3.

1. **Seleção e Preparação da Amostra:** A semente estocástica é fixada com `torch.manual_seed(7)` para assegurar a reprodutibilidade dos resultados. A primeira ocorrência do dígito 3 no conjunto de dados `load_digits` é isolada, normalizada para o intervalo `[0.0, 1.0]` e reconfigurada como um *tensor* `x` de dimensão `(1, 1, 8, 8)`.
2. **Inspeção Intermediária com `mm.showNet`:** A função `mm.showNet` da biblioteca `morph` executa a propagação à frente (*forward pass*) do *tensor* `x` na instância `modelo_cnn` previamente treinada. Utilizando *hooks* de *forward*, a função intercepta o estado numérico das ativações nas camadas convolucionais (`nn.Conv2d`), de agrupamento (`nn.MaxPool2d`) e totalmente conectadas (`nn.Linear`), retornando-as no dicionário `acts`. As funções de ativação não linear (`nn.ReLU`) não são registradas como estágios independentes, pois sua aplicação ocorre diretamente sobre o *tensor* de saída da camada correspondente.
3. **Verificação dos Resultados:** A instrução `list(acts.keys())` exibe a sequência de identificadores das camadas monitoradas, permitindo confirmar a redução dimensional progressiva e a geração do *logit* de valor máximo no índice correspondente à classe 3, conforme demonstrado na Figure 8.16.

```
torch.manual_seed(7)
digits = load_digits()
idx = np.where(digits.target == 3)[0][0]
img = digits.images[idx] / 16.0
x = torch.tensor(img, dtype=torch.float32).view(1, 1, 8, 8)
```

```
# Reutilização da instância do modelo previamente treinada
acts = mm.showNet(
    modelo_cnn,
    x,
    titulo="Fluxo de transformações dos tensores ao longo da arquitetura CNNDigitos",
    subtitulo=f"Exemplo real do dataset load_digits (classe verdadeira: {digits.target[idx]})",
)
print("Camadas capturadas:", list(acts.keys()))
```

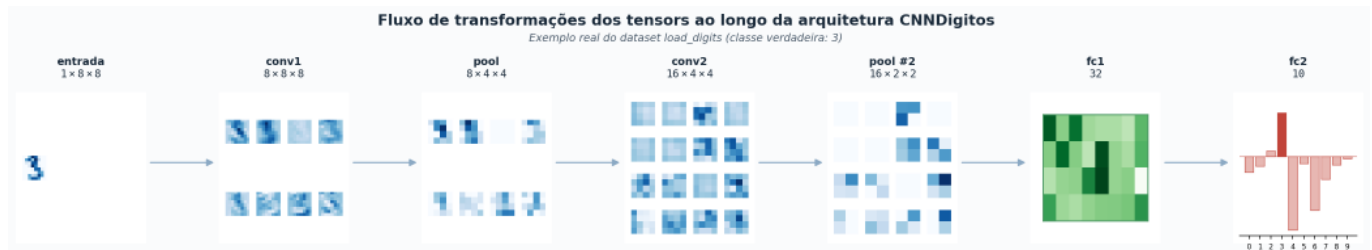


Figura 8.16: Fluxo de ativações da CNN treinada ao processar um exemplo real do dígito 3, do *dataset load_digits*: dimensões dos tensores camada a camada, da entrada ao *logit* de saída.

```
Camadas capturadas: ['conv1', 'pool', 'conv2', 'pool #2', 'fc1', 'fc2']
```

8.6.6 Inspeccionando o Grafo Computacional com torchviz

Enquanto `mm.showNet` prioriza a clareza didática — exibindo uma coluna por camada com parâmetros treináveis —, a biblioteca `torchviz` projeta o **grafo de autograd** exatamente como o PyTorch o constrói internamente para o cálculo de gradientes. A Figura 8.17 ilustra essa perspectiva ao representar a arquitetura CNNDigitos.

1. **Propagação à Frente Rastreada:** Com o modelo treinado em modo `eval()`, o *forward pass* sobre o *tensor* `x` do dígito 3 é suficiente para que o motor de *autograd* registre todas as operações executadas, incluindo aquelas sem parâmetros treináveis, como a função de ativação ReLU e a reconfiguração dimensional `view`.
2. **Geração do Grafo (`make_dot`):** A função `make_dot(saida, params=...)` constrói o grafo a partir do *tensor* de saída, percorrendo retroativamente o histórico de operações até os nós folha (os parâmetros treináveis do modelo). Cada nó do diagrama representa uma operação do *backward pass* (como `ReluBackward` ou `AddmmBackward`), e não apenas um bloco conceitual do `nn.Module`.
3. **Exportação e Renderização (`.render`):** O método `.render(..., format="png", cleanup=True)` invoca o executável `dot` do **Graphviz** para compilar a imagem em formato PNG, eliminando automaticamente os arquivos intermediários de código-fonte.

A Figura 8.17 evidencia como esse grafo computacional, mesmo para uma arquitetura compacta, apresenta maior densidade que o painel do `mm.showNet`, pois detalha cada operação atômica responsável pelo fluxo de gradientes.

```
# 1. Forward pass com rastreamento de gradiente habilitado
modelo_cnn.eval()
saida = modelo_cnn(x) # Reaproveitamento do tensor x (dígito 3)

# 2. Grafo básico: fluxo de operações até a saída
grafo_simples = make_dot(saida, params=dict(modelo_cnn.named_parameters()))
caminho_simples = grafo_simples.render("cnn_digitos_grafo_simples", format="png", cleanup=True)

# 3. Exibição direta no ambiente Quarto/Jupyter
# 0 .render() retorna o caminho do arquivo PNG gerado; precisamos abri-lo como imagem
```

```
imagem_simples = np.array(Image.open(caminho_simples).convert("RGB"))  
mm.show(imagem_simples, figsize=(8,16))
```

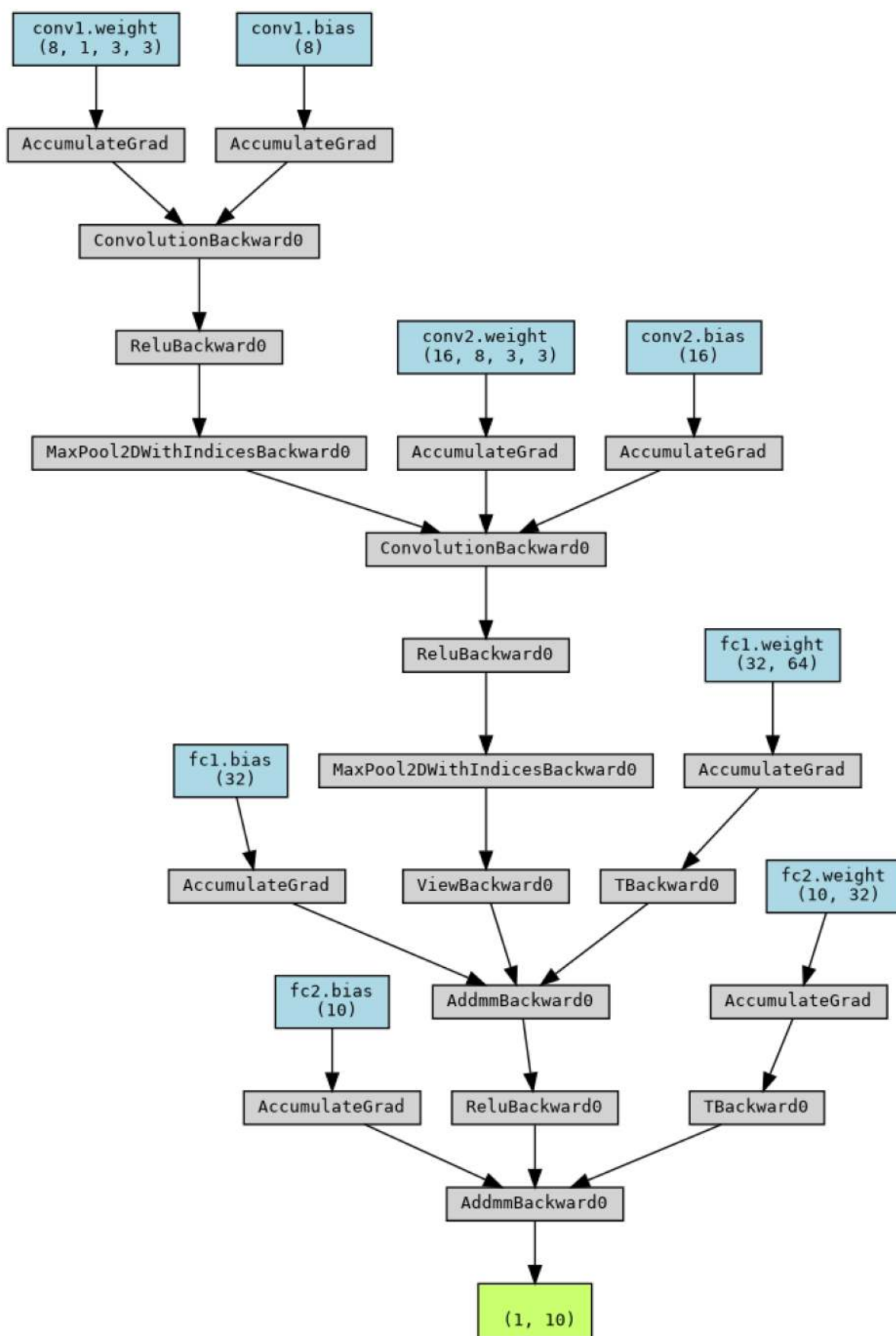



Figura 8.17: Grafo computacional da *CNNDigitos* gerado via *torchviz*, mostrando as operações de *forward* e os nós de gradiente (*backward*) associados a cada parâmetro treinável.

8.6.7 Inspeccionando o Grafo Computacional Detalhado com torchviz

Além da representação simplificada, a biblioteca `torchviz` permite expandir o grafo de *autograd* para inspecionar os detalhes internos de execução da rede `CNNDigitos`. A Figure 8.18 apresenta essa estrutura expandida para o mesmo *tensor* de entrada `x`.

1. **Rastreamento com Atributos de Operação (`show_attrs=True`):** A inclusão de atributos exibe as configurações hiperparamétricas associadas a cada nó computacional durante a propagação à frente (*forward pass*), tais como dimensões de *kernel* (`kernel_size`), passos (*stride*) e preenchimentos (*padding*) nas convoluções e subamostragens.
2. **Detecção de *Tensors* Salvos em Memória (`show_saved=True`):** O parâmetro força a exibição explícita dos *tensors* intermediários que o PyTorch retém na memória durante o *forward pass*. Esses dados são preservados porque serão estritamente necessários para o cálculo das derivadas parciais durante a etapa de retropropagação (*backward pass*).
3. **Geração e Compilação dos Grafos:** Enquanto `grafo_simples` gera uma visão direta do fluxo de gradientes, `grafo_detalhado` compila o grafo expandido no arquivo `cnn_digitos_grafo_detalhado.png` via executável `dot` do **Graphviz**.

Conforme observado na Figure 8.18, essa visualização minuciosa é útil para depurar o consumo de memória de vídeo (*VRAM*) e verificar como o motor do PyTorch aloca internamente cada nó da regra da cadeia.

```
# mesmo código anterior (make_dot + render)...

# 2. Grafo detalhado: exibição de dimensões e tensores salvos para o backward
grafo_detalhado = make_dot(
    saida,
    params=dict(modelo_cnn.named_parameters()),
    show_attrs=True,    # Exibe atributos das operações (ex.: kernel_size, stride)
    show_saved=True,    # Exibe tensores salvos na memória para a retropropagação
)

caminho_detalhado = grafo_detalhado.render("cnn_digitos_grafo_detalhado",
                                           format="png", cleanup=True)

# 3. Exibição direta no ambiente Quarto/Jupyter
imagem_detalhado = np.array(Image.open(caminho_detalhado).convert("RGB"))
mm.show(imagem_detalhado, figsize=(8, 16))
```

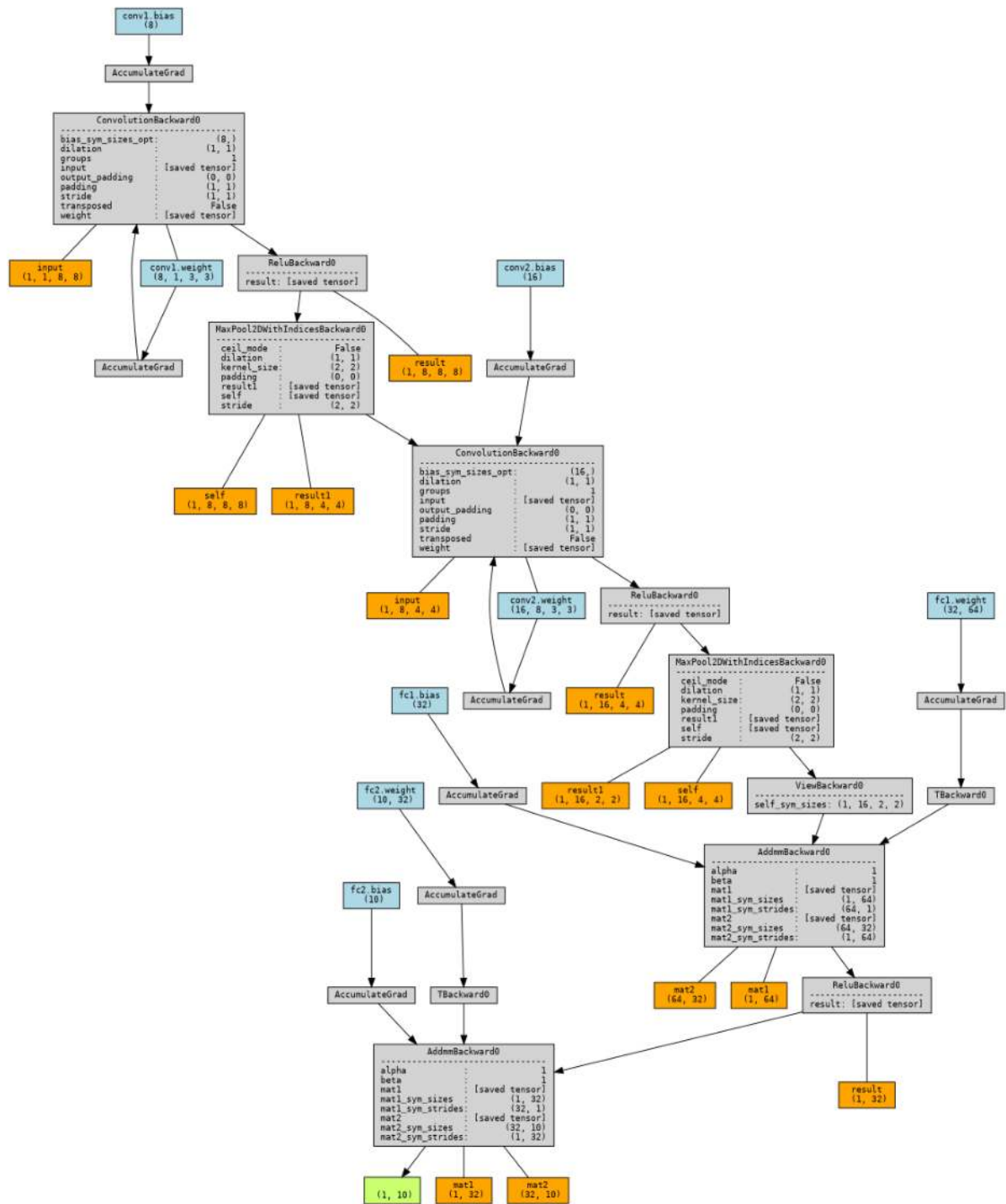


Figura 8.18: Grafo detalhado da classe CNNDigitos gerado via *torchviz*.

8.6.8 Comparando com o Capítulo 7

A Figure 8.19 reúne os resultados obtidos no mesmo conjunto de dados (*load_digits*), estabelecendo um paralelo direto entre as abordagens clássicas exploradas anteriormente e a *CNN* desenvolvida neste capítulo.

1. **Desempenho dos *Pixels* Brutos vs. Descritores Manuais:** Nos experimentos do Capítulo 7, o classificador *k*-NN ($k = 3$) atingiu uma acurácia de 98,4% quando alimentado diretamente com os *pixels* brutos das imagens. Em contrapartida, a extração prévia de características via *Histogram of Oriented Gradients* (*HOG*) resultou em um desempenho significativamente inferior (75,8%). Essa queda ocorre porque o *HOG* foi concebido para capturar gradientes de bordas em imagens de maior resolução; em matrizes de apenas 8×8 *pixels*, a resolução espacial é insuficiente para formar histogramas de

orientação informativos.

2. **Equivalência da CNN e Aprendizado *End-to-End*:** A rede convolucional **CNNDígitos** alcança um desempenho competitivo de 97,6%, aproximando-se da acurácia do k -NN com *pixels* brutos em uma base pequena e pré-alinhada. A grande vantagem conceitual reside no aprendizado de representação: em vez de depender de descritores projetados manualmente (*handcrafted features*) ou de manter todo o conjunto de dados em memória para a busca por vizinhos no momento da inferência, a *CNN* otimiza automaticamente seus próprios filtros convolucionais durante o treinamento, gerando um modelo compacto capaz de realizar a extração de características e a classificação de forma integrada (*end-to-end*).

```
import matplotlib.pyplot as plt

# Valores obtidos no Capítulo 7 (k-NN, k=3), reproduzidos para comparação direta
ACC_KNN_PIXELS_CAP7 = 0.9844
ACC_KNN_HOG_CAP7 = 0.7578

metodos = ["k-NN\n(pixels brutos)", "k-NN\n(HOG)", "CNN\n(esteste capítulo)"]
acuracias = [ACC_KNN_PIXELS_CAP7, ACC_KNN_HOG_CAP7, acc_final_cnn]

plt.figure(figsize=(5, 4))
cores = ["#6366f1", "#f97316", "#16a34a"]
plt.bar(metodos, acuracias, color=cores)
plt.ylim(0, max(acuracias) + 0.08)
plt.ylabel("Acurácia (conjunto de teste)")
plt.title("Cap. 7 vs. Cap. 9 - Base de Dígitos")

for i, v in enumerate(acuracias):
    plt.text(i, v + 0.02, f"{v:.3f}", ha="center")

plt.tight_layout()
plt.show()
```

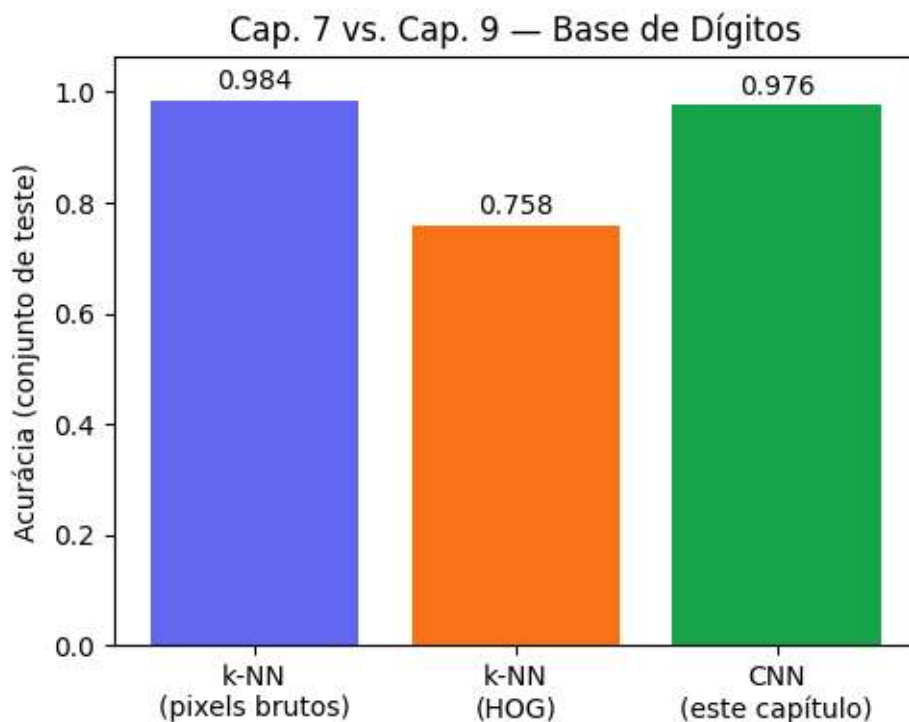


Figura 8.19: Comparação de acurácia entre os classificadores clássicos do Capítulo 7 (pixels brutos e HOG com k -NN) e a CNN treinada neste capítulo, na mesma base de dígitos.

i 🧠 Por que funciona? — E por que a CNN nem sempre “ganha”

O resultado observado aqui **repete o padrão já visto no Capítulo 7**: a *CNN*, apesar de aprender automaticamente suas características, não supera necessariamente o *k*-NN com *pixels* brutos nesta base específica. A explicação é a mesma: `load_digits` é uma base pequena (menos de 1.800 exemplos), com imagens já centralizadas, normalizadas e de baixíssima resolução (8×8) — condições em que a comparação direta de intensidades já é altamente informativa, e há poucos dados para que a rede aprenda filtros verdadeiramente superiores aos descritores simples.

O verdadeiro diferencial das *CNNs* aparece em cenários que descritores artesanais e classificadores simples não conseguem endereçar: imagens maiores e mais realistas, com milhares de categorias, variação substancial de pose, iluminação e fundo, e conjuntos de treinamento massivos — exatamente o regime em que os modelos apresentados na seção “*Aplicações em Larga Escala*”, mais adiante, foram treinados. A lição pedagógica que atravessa os Capítulos 7, 8 e 9 deste livro é consistente: **a sofisticação de um método deve ser proporcional à complexidade do problema** — usar uma *CNN* para um problema que um *k*-NN resolve igualmente bem é desperdício de recursos computacionais, não uma virtude.

Essa mesma proporcionalidade vale para as ferramentas de inspeção usadas ao longo do capítulo. O `mm.showNet` foi construído para fins **didáticos** e funciona bem em redes rasas como a `CNNDigits`, mas não escala para arquiteturas profundas: cada camada rastreada vira uma coluna na figura, e camadas convolucionais com centenas de canais geram mosaicos grandes demais para interpretação visual; além disso, os *hooks* armazenam todas as ativações em memória, e o *layout* assume um fluxo sequencial, não representando fielmente conexões residuais ou ramificações (como em *ResNets* ou módulos *Inception*). Assim, `showNet` deve ser entendido como uma lente pedagógica para redes pequenas — análoga ao papel de `mm.showBoundingBox` na depuração visual de detecções — e não como substituto de ferramentas voltadas à produção, como *TensorBoard* ou *torchviz*.

8.7 Transferência de Aprendizado

Treinar uma *CNN* do zero exige dados suficientes para ajustar todos os seus parâmetros de forma confiável — uma limitação severa quando se dispõe de poucos exemplos rotulados para a tarefa de interesse. A **transferência de aprendizado** (*transfer learning*) busca contornar essa restrição ao reaproveitar um extrator de características previamente otimizado em uma tarefa de origem:

1. Treina-se (ou obtém-se já treinado) um modelo em uma tarefa de origem com amplo volume de dados disponíveis;
2. As camadas convolucionais iniciais — que aprendem representações visuais como bordas, gradientes e texturas — têm seus parâmetros **congelados** (`requires_grad = False`);
3. Apenas as camadas finais totalmente conectadas (a “cabeça” de classificação) são substituídas e **treinadas do zero** para a nova tarefa, demandando significativamente menos tempo e esforço computacional.

Para compreender a mecânica computacional desse processo no PyTorch e avaliar seus limites práticos, analisa-se a seguir um experimento controlado sobre a base de dígitos.

8.7.1 Detalhamento do Bloco 1: Divisão dos Domínios, Escassez e Exibição das Amostras

Para demonstrar a mecânica da transferência de aprendizado, o conjunto de dados `load_digits` passa por uma reestruturação diferente da realizada no Projeto 1. Enquanto no Projeto 1 a base inteira (dígitos 0 a 9) foi utilizada para um único problema de 10 classes com 70% das amostras para treino (1.257 imagens), aqui a base é dividida em dois domínios disjuntos de 5 classes cada: o **Domínio A** (dígitos 0 a 4), que simula a tarefa de origem rica em dados, e o **Domínio B** (dígitos 5 a 9), que representa a tarefa de destino sob severa restrição de amostras.

A Figure 8.20 ilustra amostras de ambos os domínios após o pré-processamento, a separação das classes e a

conversão para *tensors* do **PyTorch**.

1. **Separação das Classes e Reindexação (Diferença para o Projeto 1):** No Projeto 1, as classes y variavam de 0 a 9. Aqui, as máscaras booleanas `mask_A` e `mask_B` filtram os exemplos de cada domínio. Para o Domínio B, subtrai-se 5 de todos os rótulos originais ($y[\text{mask_B}] - 5$), reindexando as classes para o intervalo $[0, 4]$. Isso permite que tanto o modelo de origem quanto o de destino trabalhem com um número idêntico de saídas (`n_classes=5`), alinhado à dimensão da camada final.
2. **Simulação de Escassez Extrema de Dados:** Diferente do Projeto 1, que utilizava centenas de exemplos por classe no treinamento, o gerador de números pseudoaleatórios (`np.random.default_rng(0)`) seleciona estritamente 20 amostras no total para o treino do Domínio B (`XB_tr_poucos`) — apenas 4 exemplos por dígito. Essa amostragem reduzida simula o cenário real onde o treinamento do zero tende a sofrer degradação por sobreajuste (*overfitting*).
3. **Estruturação de *Tensors* (para_tensor):** A função auxiliar `para_tensor` encapsula as operações de conversão que no Projeto 1 eram feitas manualmente linha a linha. Mantém-se o mesmo padrão dimensional $(N, 1, 8, 8)$ exigido por camadas `nn.Conv2d` e rótulos do tipo `torch.long`.
4. **Exibição Comparativa com `mm.show`:** Assim como na Figure 8.13 do Projeto 1, aplica-se `.squeeze().numpy()` para adaptar o *tensor* 3D do PyTorch em matrizes 2D do NumPy. A Figure 8.20 exibe lado a lado os dígitos do Domínio A e os dígitos do Domínio B (com seus rótulos originais e reindexados).

```
# 1. Divisão do dataset em dois domínios disjuntos
classes_A, classes_B = [0, 1, 2, 3, 4], [5, 6, 7, 8, 9]
mask_A, mask_B = np.isin(y, classes_A), np.isin(y, classes_B)

XA, yA = X[mask_A], y[mask_A]
XB, yB = X[mask_B], y[mask_B] - 5 # Reindexação dos rótulos para o intervalo [0, 4]

# Divisão em treino e teste para ambos os domínios
XA_tr, XA_te, yA_tr, yA_te = train_test_split(
    XA, yA, test_size=0.25, random_state=42, stratify=yA
)
XB_tr, XB_te, yB_tr, yB_te = train_test_split(
    XB, yB, test_size=0.25, random_state=42, stratify=yB
)

# Simulação de escassez extrema no domínio de destino: apenas 20 amostras de treino
rng = np.random.default_rng(0)
idx_poucos = rng.choice(len(XB_tr), size=20, replace=False)
XB_tr_poucos, yB_tr_poucos = XB_tr[idx_poucos], yB_tr[idx_poucos]

# Função auxiliar para conversão em tensores PyTorch
def para_tensor(Ximg, yarr):
    return torch.tensor(Ximg).unsqueeze(1), torch.tensor(yarr, dtype=torch.long)

XA_tr_t, yA_tr_t = para_tensor(XA_tr, yA_tr)
XA_te_t, yA_te_t = para_tensor(XA_te, yA_te)
XB_tr_t, yB_tr_t = para_tensor(XB_tr_poucos, yB_tr_poucos)
XB_te_t, yB_te_t = para_tensor(XB_te, yB_te)

# Exibição de amostras de ambos os domínios
n_amostras = 5
imgs_A = [img.squeeze().numpy() for img in XA_tr_t[:n_amostras]]
titles_A = [f"A: {label.item()}" for label in yA_tr_t[:n_amostras]]

imgs_B = [img.squeeze().numpy() for img in XB_tr_t[:n_amostras]]
titles_B = [f"B: {label.item()} (orig: {label.item()+5})" for label in yB_tr_t[:n_amostras]]
```



```
mm.show(
    imgs_A + imgs_B,
    titles=titles_A + titles_B,
    cols=n_amstras,
    figsize=(12, 4.5)
)
```

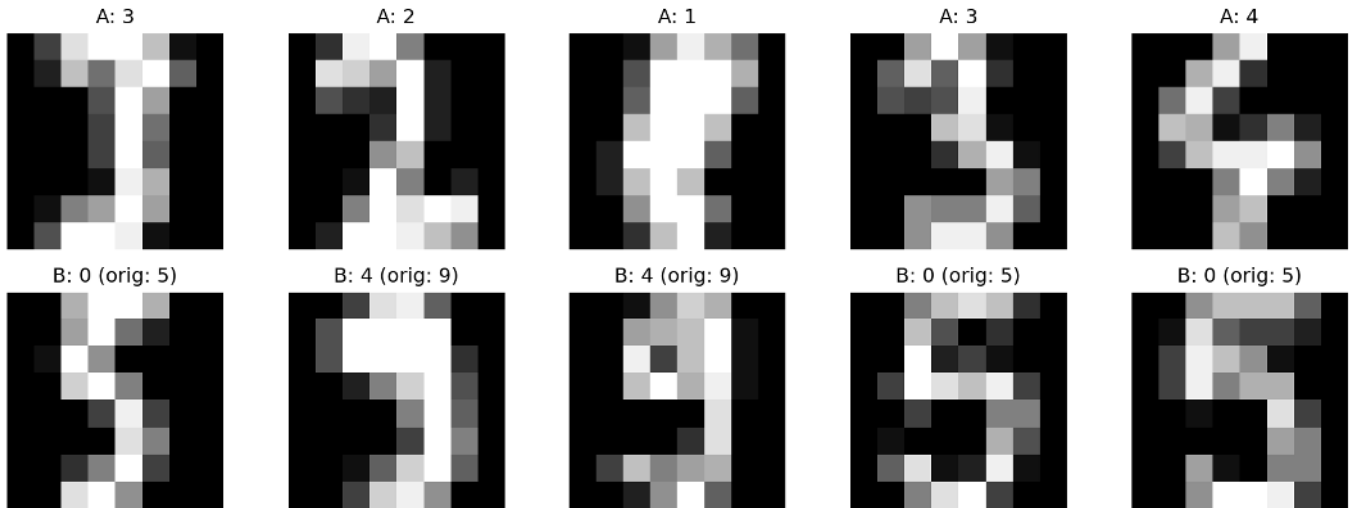


Figura 8.20: Amostras dos conjuntos de treinamento após pré-processamento e adequação dimensional aos tensores PyTorch: Domínio A (dígitos 0 a 4, tarefa de origem) e Domínio B (dígitos 5 a 9 reindexados para 0 a 4, tarefa de destino).

8.7.2 Detalhamento do Bloco 2: Arquitetura Modular e Rotinas Genéricas

Para viabilizar a transferência de aprendizado, a arquitetura convolucional e o laço de treinamento foram refatorados em relação à classe `CNNDigitos` do Projeto 1.

1. Modularização da Arquitetura (Diferença para `CNNDigitos`):

- No Projeto 1, a classe `CNNDigitos` declarava todas as camadas (`conv1`, `conv2`, `pool`, `fc1`, `fc2`) como membros diretos de uma única classe monolítica.
- Aqui, a arquitetura é separada em dois componentes: a classe `ExtratorConv` encapsula o bloco espacial convolucional (2 convoluções 3×3 , 2 *Max-Poolings* 2×2 e o achatamento para 64 elementos), enquanto a classe `CNNCompleta` instancia esse extrator em `self.extrator` e anexa a “cabeça” classificadora (`fc1` e `fc2`).
- Essa separação é o que permite copiar o estado interno do extrator (`state_dict()`) de um modelo para outro de forma isolada.

2. Ajuste no Número de Classes de Saída: Enquanto `CNNDigitos` no Projeto 1 possuía 10 *logits* na camada de saída (`self.fc2 = nn.Linear(32, 10)`), a classe `CNNCompleta` recebe `n_classes=5` no construtor para adequar-se à divisão dos domínios *A* e *B*.

3. Flexibilização do Laço de Treinamento (`treinar`):

- No Projeto 1, o laço de treinamento iterava diretamente sobre os atributos globais do modelo (`modelo_cnn.parameters()`) e calculava métricas específicas em linha.
- A função `treinar` abstrai esse processo e introduz o parâmetro opcional `parametros`. Se fornecido, o otimizador *Adam* atualiza **apenas** os parâmetros dessa lista, ignorando as camadas cujos gradientes foram desativados. Essa flexibilidade é crucial para executar o treinamento com congelamento parcial da rede.

4. Isolamento da Avaliação (`calcular_acuracia`): Assim como feito na fase de teste do Projeto 1, a função coloca o modelo em `eval()` e utiliza o contexto `torch.no_grad()` para desativar o *autograd*,

calculando a acurácia via `.argmax(dim=1)`.

```
# Definição do bloco convolucional reaproveitável (mesma extração do Projeto 1)
class ExtratorConv(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        return x.view(x.size(0), -1)

# Arquitetura modular combinando o extrator e a cabeça classificadora
class CNNCompleta(nn.Module):
    def __init__(self, n_classes=5):
        super().__init__()
        self.extrator = ExtratorConv()
        self.fc1 = nn.Linear(64, 32)
        self.fc2 = nn.Linear(32, n_classes)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.extrator(x)
        x = self.relu(self.fc1(x))
        return self.fc2(x)

# Rotina genérica de treinamento com suporte a otimização seletiva de parâmetros
def treinar(modelo, X_t, y_t, epocas, lr, tam_lote=16, parametros=None):
    params = parametros if parametros is not None else modelo.parameters()
    otim = optim.Adam(params, lr=lr)
    crit = nn.CrossEntropyLoss()
    n_amstras = X_t.size(0)
    for _ in range(epocas):
        perm = torch.randperm(n_amstras)
        for i in range(0, n_amstras, tam_lote):
            idx = perm[i:i + tam_lote]
            otim.zero_grad()
            perda = crit(modelo(X_t[idx]), y_t[idx])
            perda.backward()
            otim.step()

# Rotina de avaliação
def calcular_acuracia(modelo, X_t, y_t):
    modelo.eval()
    with torch.no_grad():
        pred = modelo(X_t).argmax(dim=1)
    return (pred == y_t).float().mean().item()
```

8.7.3 Detalhamento do Bloco 3: Pré-treinamento, Transferência e Análise Comparativa

Este bloco executa a comparação entre o ajuste do modelo a partir do zero e a aplicação da transferência com congelamento estático do extrator. Para dar total transparência ao experimento, os tamanhos dos conjuntos de treino e teste de ambos os domínios são impressos no terminal.

1. Quantificação das Amostras por Domínio:

- **Domínio A (origem, dígitos 0 a 4):** Conta com 675 amostras de treino (75%) e 226 de teste

(25%), fornecendo dados abundantes para que o `modelo_origem` aprenda o extrator convolucional até atingir 100% de acurácia.

- **Domínio B (destino, dígitos 5 a 9):** Possui 224 amostras de teste no total, mas seu conjunto de treino é intencionalmente reduzido de 672 para apenas 20 **amostras** (`XB_tr_poucos`), criando um cenário severo de escassez de dados.

2. **Etapa 1: Pré-treinamento no Domínio A (Origem):** O `modelo_origem` é treinado do zero sobre as 675 amostras dos dígitos 0 a 4. Durante 40 épocas, o extrator convolucional ajusta seus filtros para identificar os traços característicos desses cinco primeiros dígitos, atingindo 100% de acurácia no conjunto de teste (226 amostras).

3. **Etapa 2: Transferência de Pesos e Congelamento:**

- Cria-se o `modelo_transferencia` para resolver a tarefa do Domínio B (dígitos 5 a 9).
- Os pesos aprendidos no Domínio A são copiados via `load_state_dict(modelo_origem.extrator.state_dict())`.
- **Congelamento:** O laço `for p in modelo_transferencia.extrator.parameters(): p.requires_grad = False` desativa o cálculo de gradientes nas camadas convolucionais.
- **Treinamento Seletivo:** A chamada `treinar(...)` passa estritamente os parâmetros das camadas densas (`params_cabeca`), ajustando a cabeça de classificação com apenas as 20 amostras de treino.

4. **Etapa 3: Treinamento do Zero no Domínio B (Controle Experimental):** O `modelo_do_zero` possui a mesma arquitetura, mas é treinado do zero sobre as mesmas 20 amostras do Domínio B, sem qualquer reaproveitamento de pesos, pelas mesmas 40 épocas.

5. **Análise dos Resultados (Figure 8.21):**

- **Com Transferência Congelada (72,77%):** Ao reaproveitar o extrator treinado no Domínio A e congelar seus parâmetros, a rede alcança 72,77% de acurácia no teste (224 amostras) ajustando apenas as camadas densas.
- **Treinado do Zero (76,79%):** O treinamento do zero supera a transferência congelada no conjunto de teste do Domínio B.
- **Causa da Diferença:** Por se tratar de um modelo minúsculo (apenas 16 filtros convolucionais em matrizes de 8×8), o extrator treinado no Domínio A tornou-se **hiperespecializado** nas formas geométricas dos dígitos 0 a 4. Ao congelar rigidamente esses poucos filtros, o modelo de destino ficou limitado a detectores inadequados para 5 a 9. A rede treinada do zero, mesmo com apenas 20 amostras, conseguiu adaptar seus 16 filtros diretamente aos traços do Domínio B.

```
# Fixar semente para reprodutibilidade
torch.manual_seed(42)

# Exibição do tamanho dos grupos de treino e teste
print("=== Detalhamento do Tamanho das Bases ===")
print(f"Domínio A (0-4) - Treino: {len(XA_tr_t)} amostras | Teste: {len(XA_te_t)} amostras")
print(f"Domínio B (5-9) - Treino completo: {len(XB_tr)} | Treino reduzido: {len(XB_tr_poucos)}",
      f"| Teste: {len(XB_te_t)} amostras\n")

# 1. Pré-treinamento na tarefa de origem (Domínio A: dígitos 0-4)
modelo_origem = CNNCompleta(n_classes=5)
treinar(modelo_origem, XA_tr_t, yA_tr_t, epocas=40, lr=1e-2)
acc_A = calcular_acuracia(modelo_origem, XA_te_t, yA_te_t)
print(f"Acurácia no domínio de origem A (dígitos 0-4, {len(XA_te_t)} testes): {acc_A:.4f}")

# 2. Transferência de Aprendizado (Extrator Congelado)
modelo_transferencia = CNNCompleta(n_classes=5)
modelo_transferencia.extrator.load_state_dict(modelo_origem.extrator.state_dict())

for p in modelo_transferencia.extrator.parameters():
    p.requires_grad = False
```

```

params_cabeca = list(modelo_transferencia.fc1.parameters())
params_cabeca += list(modelo_transferencia.fc2.parameters())
treinar(modelo_transferencia, XB_tr_t, yB_tr_t, epocas=40, lr=1e-2, parametros=params_cabeca)
acc_transferencia = calcular_acuracia(modelo_transferencia, XB_te_t, yB_te_t)

# 3. Treinamento do Zero no Domínio B
modelo_do_zero = CNNCompleta(n_classes=5)
treinar(modelo_do_zero, XB_tr_t, yB_tr_t, epocas=40, lr=1e-2)
acc_do_zero = calcular_acuracia(modelo_do_zero, XB_te_t, yB_te_t)

print(f"Domínio de destino B (dígitos 5-9), apenas {len(XB_tr_poucos)} ",
      f"exemplos de treino ({len(XB_te_t)} testes):")
print(f"  Com transferência (extrator congelado): {acc_transferencia:.4f}")
print(f"  Treinando do zero (mesmos dados/épocas): {acc_do_zero:.4f}")

# Visualização comparativa
plt.figure(figsize=(4.5, 4))
plt.bar(["Do zero", "Transferência"], [acc_do_zero, acc_transferencia],
        color=["#dc2626", "#16a34a"])
plt.ylim(0, max([acc_do_zero, acc_transferencia]) + 0.1)
plt.ylabel("Acurácia no domínio B (teste)")
plt.title(f"Efeito da Transferência ({len(XB_tr_poucos)} exemplos de treino)")

for i, v in enumerate([acc_do_zero, acc_transferencia]):
    plt.text(i, v + 0.02, f"{v:.3f}", ha="center")

plt.tight_layout()
plt.show()

```

=== Detalhamento do Tamanho das Bases ===

Domínio A (0-4) - Treino: 675 amostras | Teste: 226 amostras

Domínio B (5-9) - Treino completo: 672 | Treino reduzido: 20 | Teste: 224 amostras

Acurácia no domínio de origem A (dígitos 0-4, 226 testes): 1.0000

Domínio de destino B (dígitos 5-9), apenas 20 exemplos de treino (224 testes):

Com transferência (extrator congelado): 0.7277

Treinando do zero (mesmos dados/épocas): 0.7679

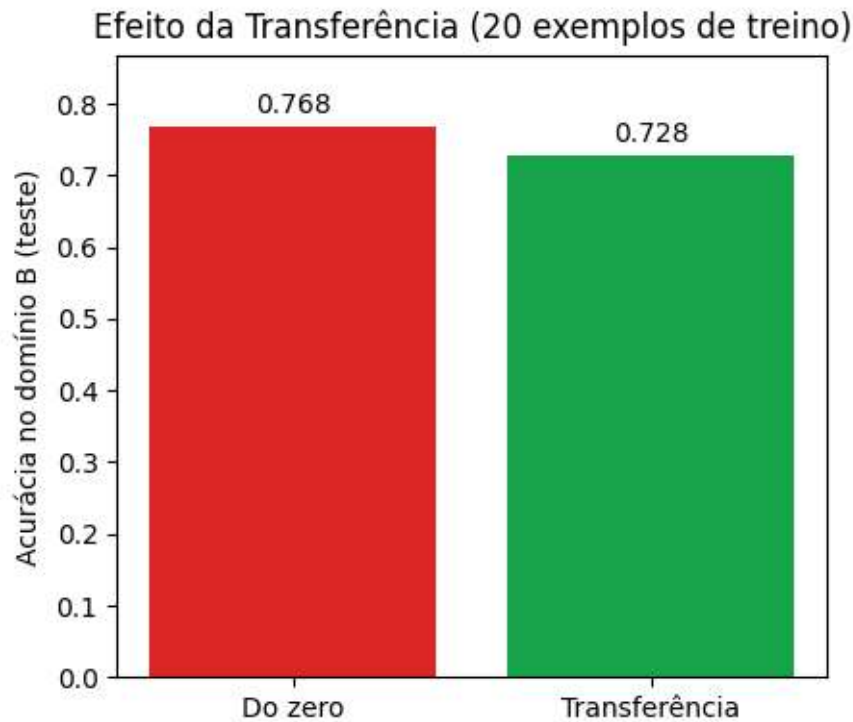


Figura 8.21: Comparação de acurácia no conjunto de teste do Domínio B (dígitos 5 a 9) sob restrição de dados (20 exemplos de treino): demonstração do impacto do congelamento rígido e da transferência negativa em redes de baixa capacidade.

8.7.4 Detalhamento do Bloco 4: Visualização do Fluxo de Ativações com `mm.showNet`

Para confirmar que a extração de características reutilizada preserva as transformações dimensionais estudadas no Projeto 1, utiliza-se novamente a função `mm.showNet` da biblioteca `morph`. A Figure 8.22 exibe o fluxo de ativações do `modelo_transferencia` ao processar uma amostra do Domínio B (dígito 7, reindexado para a classe 2).

1. **Preservação do Fluxo Convolutacional:** Como a arquitetura `ExtratorConv` replica as mesmas camadas de convolução e *pooling* da `CNNDigitos` do Projeto 1, as dimensões dos *tensors* intermediários mantêm-se em (1, 8, 4, 4) no primeiro bloco.
2. **Inspeção da Cabeça Adaptada:** A diferença em relação ao Projeto 1 surge na camada de saída (`fc2`): enquanto o modelo do Projeto 1 projetava o vetor intermediário em 10 *logits* (classes de 0 a 9), o modelo de transferência projeta o vetor em 5 *logits* (classes de 0 a 4), capturando as probabilidades relativas do Domínio B.

```
# Seleciona a primeira amostra de teste do Domínio B
x_amostra_B = XB_te_t[0:1] # Tensor de dimensão (1, 1, 8, 8)
classe_verdadeira_reindexada = yB_te_t[0].item()
classe_original = classe_verdadeira_reindexada + 5

# Inspeção do fluxo de ativações no modelo de transferência
acts_transfer = mm.showNet(
    modelo_transferencia,
    x_amostra_B,
    titulo="Fluxo de ativações no modelo de transferência (Domínio B)",
    subtitulo=f"Amostra do dígito {classe_original} (rótulo reindexado: {classe_verdadeira_reindexada})"
)

print("Camadas capturadas no modelo de transferência:\n", list(acts_transfer.keys()))
```

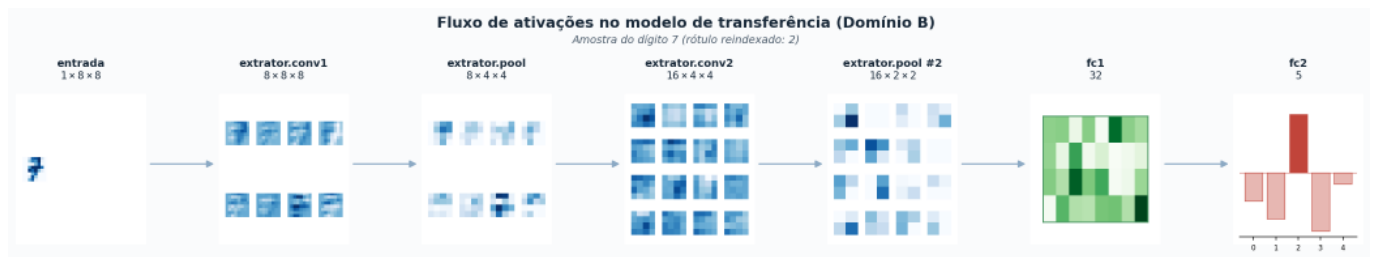


Figura 8.22: Fluxo de ativações e transformações dimensionais dos tensores no modelo de transferência de aprendizado ao processar uma amostra de teste do Domínio B (dígito 7, reindexado para classe 2).

Camadas capturadas no modelo de transferência: ['extrator.conv1', 'extrator.pool', 'extrator.conv2', 'ex

Do Brinquedo ao Mundo Real: Por que a Transferência de Aprendizado de Fato Funciona?

O experimento anterior com a base de dígitos traz uma lição valiosa e realista: a transferência de aprendizado **não é uma garantia automática de desempenho superior**. O congelamento rígido de um extrator minúsculo (16 filtros) treinado em uma tarefa muito simples pode resultar no fenômeno da **Transferência Negativa** (*Negative Transfer*), no qual pesos prévios desajustados prejudicam o aprendizado da nova tarefa.

Por que a transferência congelada foi limitada neste exemplo?

1. **Redes Pequenas Especializam Demais:** Um extrator com apenas 2.000 parâmetros em imagens de 8×8 não aprende “características visuais universais”, mas sim atalhos específicos para identificar os dígitos de 0 a 4.
2. **O Congelamento Rígido Impede a Adaptação:** Ao impedir que os filtros convolucionais se ajustem ao Domínio B, a rede fica engessada em detectores irrelevantes.

A Magia das Redes Pré-Treinadas em Larga Escala

O cenário muda drasticamente ao migrar dos modelos de brinquedo para arquiteturas profundas (*ResNets*, *ConvNeXts*, *YOLO*) pré-treinadas em bases massivas como a **ImageNet** (1,4 milhão de imagens em 1.000 categorias) ou a **COCO** (330 mil imagens rotuladas para detecção).

Nessas redes de larga escala:

- **Camadas Iniciais (1ª a 3ª):** Aprendem detectores de bordas orientadas, gradientes de cor e texturas genéricas. Essas características são **universais** — úteis tanto para detectar células microscópicas quanto para identificar veículos em rodovias.
- **Camadas Intermediárias e Finais:** Combinam essas primitivas visuais em padrões complexos de objetos.

Por isso, no **Projeto 2 com YOLO** apresentado a seguir, emprega-se um modelo que já possui um extrator visual altamente generalista. Além disso, em vez de congelar as camadas indefinidamente, adota-se o **Ajuste Fino** (*Fine-Tuning*), permitindo que toda a rede se adapte suavemente ao novo problema.

8.8 Aplicações em Larga Escala com Modelos Pré-treinados

Os experimentos anteriores utilizaram *CNNs* compactas, treinadas em matrizes de baixa resolução (8×8 pixels). Na prática profissional de visão computacional, é padrão empregar modelos **pré-treinados** em bases massivas — como a *ImageNet*, que reúne mais de 1,4 milhão de imagens distribuídas em 1.000 categorias — atuando como extratores de características universais ou como ponto de partida para ajuste fino (*fine-tuning*).

8.8.1 Classificação de Imagens com Arquiteturas Consolidadas (*ResNet-18*)

Para tarefas de classificação em novos domínios, o fluxo de transferência de aprendizado segue um protocolo padronizado: carrega-se a arquitetura com os pesos pré-treinados na *ImageNet*, congelam-se os parâmetros do extrator e substitui-se a camada linear final pela dimensão de saída da nova tarefa (por exemplo, 3 novas classes específicas do seu problema, como ["folha_saudavel", "folha_doente", "sintoma_desconhecido"]).

8.8.1.1 Detalhamento do Bloco 1: Preparação dos Dados, Visualização e Adaptação da Arquitetura

Modelos como a *ResNet-18* exigem imagens coloridas de 224×224 *pixels* normalizadas segundo as estatísticas da *ImageNet* ($\mu = [0, 485; 0, 456; 0, 406]$ e $\sigma = [0, 229; 0, 224; 0, 225]$).

1. **Transformação de Entrada (`transforms.Compose`):** Aplica-se o redimensionamento e a normalização padrão para compatibilidade com os filtros pré-treinados do extrator.
2. **Visualização com `mm.show`:** A imagem é exibida na Figure 8.23 **antes** de qualquer pré-processamento — ainda em sua forma original, tal como devolvida por `mm.read()`. A conversão para *tensor* e a normalização ocorrem apenas na etapa seguinte, imediatamente antes de alimentar a rede.
3. **Congelamento do Extrator:** O laço `for p in modelo_resnet.parameters(): p.requires_grad = False` desativa os gradientes nas camadas convolucionais pré-treinadas.
4. **Nova Camada Final:** A camada `modelo_resnet.fc` é substituída por uma nova instância `nn.Linear(modelo_resnet.fc.in_features, n_classes_destino)`, recém-inicializada e com gradientes ativos por padrão. Note que o número de características de entrada é obtido dinamicamente a partir da própria camada original (`in_features`, igual a 512 na ResNet-18), em vez de fixado manualmente no código — prática recomendada, pois torna o trecho reutilizável para outras variantes da arquitetura sem alterações.

```
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
import torchvision.models as models
import torchvision.transforms as T

# 1. Pipeline de transformações esperadas pela ResNet
transformacao_resnet = T.Compose([
    T.Resize((224, 224)),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# 2. Definição das 3 classes de destino
novas_classes = ["folha_saudavel", "folha_doente", "sintoma_desconhecido"]
n_classes_destino = len(novas_classes)

# 3. Carregamento da imagem real com mm.read (retorna numpy.ndarray)
url_imagem = "https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg"
img_dog = mm.read(url_imagem)

# 4. Exibição da imagem real com mm.show antes do pré-processamento
mm.show(
    img_dog,
    title="Imagem Real de Entrada (dog.jpg) - Base para o Novo Dataset"
)

# 5. Processamento da imagem: conversão para tensor, normalização e lote
tensor_dog = T.functional.to_tensor(img_dog) # HWC -> CHW, escala para [0, 1]
tensor_processado = transformacao_resnet(tensor_dog) # resize + normalização ImageNet

# Simulação de um mini-batch reutilizando a mesma imagem para as 3 classes
```

```
# (apenas para fins didáticos: em um projeto real, cada classe teria suas
# próprias imagens de treino - ver aviso no Bloco 2)
X_novos_dados = torch.stack([tensor_processado, tensor_processado, tensor_processado])
y_novas_classes = torch.tensor([0, 1, 2], dtype=torch.long)

dataset_novo = TensorDataset(X_novos_dados, y_novas_classes)
loader_treino = DataLoader(dataset_novo, batch_size=3, shuffle=True)

# 6. Carregamento da ResNet-18 e substituição da camada final
modelo_resnet = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

for parametro in modelo_resnet.parameters():
    parametro.requires_grad = False

modelo_resnet.fc = nn.Linear(modelo_resnet.fc.in_features, n_classes_destino)
```

Imagem Real de Entrada (dog.jpg) — Base para o Novo Dataset



Figura 8.23: Imagem real de teste carregada via *mm.read* e exibida com *mm.show* antes da conversão para *tensor* normalizado e alimentação da ResNet-18.

8.8.1.2 Detalhamento do Bloco 2: Laço de Treinamento e Otimização da Nova Cabeça

Com o extrator congelado e a nova camada de saída devidamente acoplada, executa-se o ajuste fino (*fine-tuning*) da nova cabeça de classificação.

1. **Otimização Focada:** O otimizador *Adam* recebe estritamente `modelo_resnet.fc.parameters()`, atualizando apenas a nova camada de saída — o restante da rede permanece congelado, conforme definido no Bloco 1.
2. **Execução do Laço:** O modelo itera sobre os *batches*, calcula a perda por Entropia Cruzada e ajusta os pesos da cabeça de classificação.

⚠️ Dataset Sintético: Objetivo é Ilustrar a Mecânica, Não a Convergência

O `loader_treino` construído no Bloco 1 contém **a mesma imagem repetida três vezes**, cada cópia associada a um rótulo diferente (0, 1 e 2). Como a entrada é idêntica nas três amostras e o extrator está congelado, a rede não recebe nenhuma informação visual capaz de diferenciar as classes — a acurácia

deve oscilar em torno de 33% (equivalente a um chute aleatório entre 3 classes) e **não converge** com o avanço das épocas.

Este exemplo tem propósito estritamente didático: demonstrar a *sintaxe* e a *mecânica* do laço de *fine-tuning* (otimizador restrito à cabeça nova, cálculo de perda, `backward()`, `step()`). Em um projeto real, o `DataLoader` deve ser construído a partir de um conjunto de imagens genuinamente distintas para cada classe — por exemplo, utilizando `torchvision.datasets.ImageFolder` sobre um diretório organizado por subpastas de classe.

```
import torch.optim as optim

dispositivo = torch.device("cuda" if torch.cuda.is_available() else "cpu")
modelo_resnet = modelo_resnet.to(dispositivo)

criterio = nn.CrossEntropyLoss()
otimizador = optim.Adam(modelo_resnet.fc.parameters(), lr=1e-3)

epocas = 5
modelo_resnet.train()

for epoca in range(epocas):
    perda_acumulada = 0.0
    acertos = 0
    total = 0

    for X_batch, y_batch in loader_treino:
        X_batch, y_batch = X_batch.to(dispositivo), y_batch.to(dispositivo)

        otimizador.zero_grad()
        saidas = modelo_resnet(X_batch)
        perda = criterio(saidas, y_batch)
        perda.backward()
        otimizador.step()

        perda_acumulada += perda.item() * X_batch.size(0)
        predicoes = saidas.argmax(dim=1)
        acertos += (predicoes == y_batch).sum().item()
        total += y_batch.size(0)

    print(f"Época {epoca+1}/{epocas} - Acurácia: {(acertos/total)*100:.2f}%")
```

```
Época 1/5 - Acurácia: 33.33%
Época 2/5 - Acurácia: 33.33%
Época 3/5 - Acurácia: 33.33%
Época 4/5 - Acurácia: 33.33%
Época 5/5 - Acurácia: 33.33%
```

8.8.2 Detecção de Objetos com Redes de Região (*Faster R-CNN*)

A detecção de objetos ilustra outra faceta do reaproveitamento de modelos pré-treinados. Redes complexas como a **Faster R-CNN** utilizam uma *CNN* de classificação (como a *ResNet-50*) pré-treinada na *ImageNet* como seu **extrator de características padrão** (*backbone*).

Sobre esse extrator, acoplam-se duas “cabeças” especializadas:

- **Region Proposal Network (RPN):** Identifica *onde* estão os possíveis objetos (coordenadas da caixa).
- **Cabeça Classificadora:** Determina *o que* está dentro da caixa, dentre as categorias da base *COCO*.

Ao executar o código a seguir, emprega-se a *Faster R-CNN* com os pesos do *backbone* e das cabeças já otimizados no *dataset* *COCO*, demonstrando a extração de coordenadas e nomes de classes em uma imagem

real.

 Onde está a transferência de aprendizado aqui?

Diferente do exemplo anterior com a *ResNet-18*, este trecho **não realiza *fine-tuning***: o modelo é empregado em modo `eval()` com todos os pesos — tanto do *backbone* quanto das cabeças de RPN e classificação — exatamente como foram otimizados no COCO, sem nenhuma camada nova ou ajuste de parâmetros. Trata-se de um caso de **reuso direto** de um modelo pré-treinado, não de adaptação a uma nova tarefa.

A transferência de aprendizado “genuína” (com adaptação a novas classes) neste contexto ocorreria mediante a substituição da `box_predictor` do modelo por uma nova cabeça treinável para as classes do domínio do usuário — por exemplo, via `FastRCNNPredictor(in_features, novo_n_classes)`. Em seguida, realiza-se o treinamento focado nessa nova cabeça sobre um conjunto de dados anotado, seguindo o mesmo princípio de congelamento aplicado na seção anterior. Esse cenário fica fora do escopo deste capítulo introdutório, mas representa o próximo passo natural para a detecção de objetos em domínios específicos (ex.: pragas em folhas, defeitos de fabricação, etc.).

1. **Carregamento das Categorias:** Os nomes das classes são recuperados diretamente das meta-informações dos pesos (`FasterRCNN_ResNet50_FPN_Weights.DEFAULT.meta["categories"]`). A lista contém 91 entradas por razões históricas do formato de anotação COCO — incluindo índices reservados marcados como "N/A" —, mas apenas 80 correspondem a categorias de objetos de fato utilizadas pelo modelo.
2. **Inferência e Filtragem por Limiar:** A imagem carregada com `mm.read()` é convertida em *tensor* e processada no modo de avaliação (`modelo.eval()`). Retêm-se apenas as previsões com confiança superior a 80%.
3. **Anotação Gráfica:** Para cada detecção, desenham-se a caixa delimitadora (`cv2.rectangle`) e o rótulo textual contendo a classe predita e sua pontuação de confiança (`cv2.putText`).
4. **Visualização:** A imagem anotada é exibida na Figure 8.24.

```
import cv2
import torch
from torchvision.models.detection import (
    FasterRCNN_ResNet50_FPN_Weights,
    fasterrcnn_resnet50_fpn,
)
from torchvision.transforms.functional import to_tensor

# 1. Carregamento da imagem e dos nomes das categorias do COCO
url_imagem = "https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg"
img = mm.read(url_imagem)

pesos_coco = FasterRCNN_ResNet50_FPN_Weights.DEFAULT
categorias_coco = pesos_coco.meta["categories"] # Mapeamento índice -> nome da classe

# 2. Carregamento do modelo Faster R-CNN pré-treinado
modelo_detecao = fasterrcnn_resnet50_fpn(weights=pesos_coco).eval()

# 3. Execução da inferência sem cálculo de gradientes
with torch.no_grad():
    predicao = modelo_detecao([to_tensor(img)]) [0]

# 4. Filtragem das detecções com confiança superior a 80%
limiar_confianca = 0.8
mascara_confianca = predicao["scores"] >= limiar_confianca

caixas_filtradas = predicao["boxes"][mascara_confianca].numpy()
scores_filtrados = predicao["scores"][mascara_confianca].numpy()
```

```

labels_filtrados = predicao["labels"][mascara_confianca].numpy()

img_com_caixas = img.copy()

# 5. Desenho das caixas delimitadoras e dos rótulos de classe
for box, score, label_idx in zip(caixas_filtradas, scores_filtrados, labels_filtrados):
    x1, y1, x2, y2 = box.astype(int)
    nome_classe = categorias_coco[label_idx]
    texto_rotulo = f"{nome_classe}: {score:.2f}"

    # Desenha o retângulo vermelho (RGB: 255, 0, 0) com espessura de 3 pixels
    cv2.rectangle(img_com_caixas, (x1, y1), (x2, y2), (255, 0, 0), 3)

    # Escreve a classe e a confiança acima da caixa delimitadora
    cv2.putText(
        img_com_caixas,
        texto_rotulo,
        (x1, max(y1 - 10, 20)),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (255, 0, 0),
        2,
        cv2.LINE_AA,
    )

# 6. Exibição gráfica da imagem resultante
mm.show(img_com_caixas, title="Faster R-CNN (COCO) - Detecção com Classe e Confiança")

```

Faster R-CNN (COCO) — Detecção com Classe e Confiança



Figura 8.24: Resultado da inferência com o modelo Faster R-CNN pré-treinado no *dataset* COCO: identificação da classe ‘cão’ (*dog*), caixa delimitadora e pontuação de confiança sobrepostas.

8.8.3 Segmentação Semântica com *DeepLab V3*

Em tarefas de segmentação semântica, o objetivo ultrapassa a identificação da caixa delimitadora (*bounding box*): a rede realiza a classificação **pixel a pixel** da imagem. Arquiteturas consolidadas, como a **DeepLabV3** (com *backbone* ResNet-50), estimam a classe pertencente a cada posição espacial, gerando um mapa bidimensional de rótulos.

1. **Carregamento e Inferência:** O modelo `deeplabv3_resnet50(weights="DEFAULT")` carrega os pesos pré-treinados no conjunto *Pascal VOC* (com 21 categorias de segmentação). A imagem obtida via `mm.read()` é convertida em *tensor* normalizado com `to_tensor(img)` e expandida para a dimensão de *batch* (`unsqueeze(0)`).
2. **Extração do Mapa de Classes:** A propagação à frente retorna um *tensor* de saída de dimensão $(1, 21, H, W)$, contendo os *logits* desnormalizados para cada uma das 21 classes. Aplica-se `.argmax(dim=1)` ao longo do eixo das classes para selecionar a categoria de maior probabilidade em cada *pixel*, gerando uma matriz 2D de dimensões (H, W) .
3. **Conversão e Visualização:** A matriz de classes previstas é convertida para *array* NumPy via `.byte().cpu().numpy()` e repassada diretamente à rotina `mm.show()`, que renderiza o mapa de segmentação colorido na `?@fig-09-segmentacao-pretreinada`.

```
#| label: fig-09-segmentacao-pretreinada
#| fig-cap: "Mapa de segmentação semântica gerado pelo modelo DeepLabV3 pré-treinado: classificação pixel
#| eval: false
#| echo: true
#| output: true

import torch
from torchvision.models.segmentation import deeplabv3_resnet50
from torchvision.transforms.functional import to_tensor

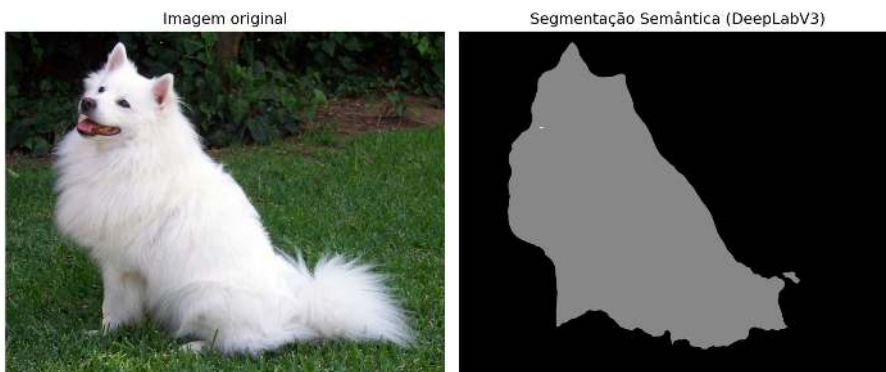
# 1. Carregamento da imagem (retorna numpy.ndarray)
url_imagem = "https://raw.githubusercontent.com/pytorch/hub/master/images/dog.jpg"
img = mm.read(url_imagem)

# 2. Carregamento do modelo DeepLabV3 pré-treinado no modo de avaliação
modelo_segmentacao = deeplabv3_resnet50(weights="DEFAULT").eval()

# 3. Execução da inferência sem cálculo de gradientes
with torch.no_grad():
    tensor_entrada = to_tensor(img).unsqueeze(0) # Formato (1, C, H, W)
    saida = modelo_segmentacao(tensor_entrada)["out"]

    # Seleção da classe com maior probabilidade por pixel (argmax no eixo dos canais)
    mapa_classes = saida.argmax(dim=1).squeeze(0).byte().cpu().numpy()

# 4. Exibição do mapa de segmentação semântica
mm.show(
    [img, mapa_classes],
    title=["Imagem original", "Segmentação Semântica (DeepLabV3)"]
)
```



8.9 Projeto Prático 2: Detecção de Objetos com YOLO e Transferência de Aprendizado

A subseção anterior demonstrou, por meio do **Faster R-CNN**, um modelo de detecção pré-treinado empregado diretamente, sem etapa de re-treinamento. O Faster R-CNN pertence à família de detectores **de dois estágios**: uma primeira rede propõe regiões candidatas a conter um objeto (*region proposals*) e uma segunda rede classifica e refina cada região.

A família **YOLO** (*You Only Look Once*) adota a filosofia **de estágio único**: a imagem é dividida conceitualmente em uma grade, e uma única rede convolucional estima, em uma única passagem direta, as caixas delimitadoras (*bounding boxes*), a confiança de presença de um objeto e a classe associada a cada região. Essa simplificação é o pilar que viabiliza o processamento **em tempo real**.

Nesta seção, o princípio de **transferência de aprendizado** é aplicado a uma **tarefa de detecção**: partindo de um modelo YOLO pré-treinado no conjunto COCO (80 categorias de objetos naturais), realiza-se o ajuste fino (*fine-tuning*) para a localização e classificação de **objetos geometricamente definidos** (triângulos, quadrados, estrelas, entre outros), variando em cor, dimensão, ângulo de rotação e sob degradação por **ruído do tipo sal e pimenta**.

8.9.1 Bloco 1: Definição das Formas, Injeção de Ruído e Exibição das Amostras do Dataset

O cenário simula um problema real de visão computacional industrial ou robótica: a partir de imagens contendo formas geométricas capturadas sob degradação de sensor, prepara-se um detector para delimitar e classificar cada objeto em nove categorias:

```
CLASSES = [
    'Triangle', 'Square', 'Pentagon', 'Hexagon',
    'Heptagon', 'Circle', 'Ellipse', 'Star', 'Cross'
]
```

1. **Geometria e Desenho:** As funções `poligono_regular`, `poligono_estrela` e `poligono_cruz` geram as coordenadas das formas rotacionadas, enquanto `desenha_objeto` renderiza a imagem e retorna os limites da *bounding box*.
2. **Injeção de Ruído Sal e Pimenta:** A função `adiciona_ruido_sal_pimenta` contamina a imagem com 5% de pixels impulsivos (pretos e brancos).
3. **Exibição Inicial de Amostras:** A Figure 8.25 apresenta 5 amostras ruidosas do conjunto sintético com as caixas delimitadoras sobrepostas no formato YOLO via `mm.showBoundingBox()` e renderizadas por `mm.show()`.

```
CLASSES = [
    'Triangle', 'Square', 'Pentagon', 'Hexagon',
    'Heptagon', 'Circle', 'Ellipse', 'Star', 'Cross'
]
N_LADOS = {'Triangle': 3, 'Square': 4, 'Pentagon': 5, 'Hexagon': 6, 'Heptagon': 7}

def poligono_regular(cx, cy, r, n_lados, rot_graus):
    ang0 = np.deg2rad(rot_graus - 90)
    angs = ang0 + 2 * np.pi * np.arange(n_lados) / n_lados
    return np.stack([cx + r * np.cos(angs), cy + r * np.sin(angs)], axis=1)

def poligono_estrela(cx, cy, r_externo, rot_graus, n_pontas=5):
    r_interno = r_externo * 0.45
    ang0 = np.deg2rad(rot_graus - 90)
    angs = ang0 + np.pi * np.arange(2 * n_pontas) / n_pontas
    raios = np.where(np.arange(2 * n_pontas) % 2 == 0, r_externo, r_interno)
```

```

    return np.stack([cx + raios * np.cos(angs), cy + raios * np.sin(angs)], axis=1)

def poligono_cruz(cx, cy, r, rot_graus, espessura_rel=0.35):
    w = r * espessura_rel
    base = np.array([
        (-w, -r), (w, -r), (w, -w), (r, -w), (r, w), (w, w),
        (w, r), (-w, r), (-w, w), (-r, w), (-r, -w), (-w, -w),
    ])
    theta = np.deg2rad(rot_graus)
    R = np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])
    return base @ R.T + np.array([cx, cy])

def desenha_objeto(img, classe_idx, cx, cy, tamanho, rotacao, cor):
    nome = CLASSES[classe_idx]
    if nome in N_LADOS:
        pts = poligono_regular(cx, cy, tamanho, N_LADOS[nome], rotacao)
        cv2.fillPoly(img, [pts.astype(np.int32)], cor)
        xs, ys = pts[:, 0], pts[:, 1]
    elif nome == 'Star':
        pts = poligono_estrela(cx, cy, tamanho, rotacao)
        cv2.fillPoly(img, [pts.astype(np.int32)], cor)
        xs, ys = pts[:, 0], pts[:, 1]
    elif nome == 'Cross':
        pts = poligono_cruz(cx, cy, tamanho, rotacao)
        cv2.fillPoly(img, [pts.astype(np.int32)], cor)
        xs, ys = pts[:, 0], pts[:, 1]
    elif nome == 'Circle':
        cv2.circle(img, (int(cx), int(cy)), int(tamanho), cor, -1)
        xs, ys = np.array([cx - tamanho, cx + tamanho]), np.array([cy - tamanho, cy + tamanho])
    else: # Ellipse
        eixo = (int(tamanho), int(tamanho * 0.6))
        cv2.ellipse(img, (int(cx), int(cy)), eixo, rotacao, 0, 360, cor, -1)
        ang = np.deg2rad(rotacao)
        dx = np.hypot(eixo[0] * np.cos(ang), eixo[1] * np.sin(ang))
        dy = np.hypot(eixo[0] * np.sin(ang), eixo[1] * np.cos(ang))
        xs, ys = np.array([cx - dx, cx + dx]), np.array([cy - dy, cy + dy])
    return xs.min(), ys.min(), xs.max(), ys.max()

def adiciona_ruido_sal_pimenta(img, quantidade=0.05):
    img_ruidosa = img.copy()
    h, w, c = img_ruidosa.shape
    num_ruido = int(quantidade * h * w)

    # Sal (255, 255, 255)
    coords_sal = [np.random.randint(0, i - 1, num_ruido) for i in (h, w)]
    img_ruidosa[coords_sal[0], coords_sal[1]] = [255, 255, 255]

    # Pimenta (0, 0, 0)
    coords_pimenta = [np.random.randint(0, i - 1, num_ruido) for i in (h, w)]
    img_ruidosa[coords_pimenta[0], coords_pimenta[1]] = [0, 0, 0]

    return img_ruidosa

def gera_imagem_ruidosa(tam_img=160, n_objetos=(1, 3), taxa_ruido=0.01, rng=None):
    rng = rng or random.Random()
    img_limpa = np.full((tam_img, tam_img, 3), 255, dtype=np.uint8)

```

```

anotacoes = []

for _ in range(rng.randint(*n_objetos)):
    classe_idx = rng.randrange(len(CLASSES))
    tamanho = rng.randint(tam_img // 10, tam_img // 5)
    cx = rng.randint(tamanho + 2, tam_img - tamanho - 2)
    cy = rng.randint(tamanho + 2, tam_img - tamanho - 2)
    rotacao = rng.uniform(0, 360)
    cor = tuple(rng.sample(range(30, 226), 3))

    x0, y0, x1, y1 = desenha_objeto(img_limpa, classe_idx, cx, cy, tamanho, rotacao, cor)
    x0, y0 = max(x0, 0), max(y0, 0)
    x1, y1 = min(x1, tam_img), min(y1, tam_img)

    xc, yc = (x0 + x1) / 2 / tam_img, (y0 + y1) / 2 / tam_img
    w, h = (x1 - x0) / tam_img, (y1 - y0) / tam_img
    anotacoes.append((classe_idx, xc, yc, w, h))

img_ruidosa = adiciona_ruido_sal_pimenta(img_limpa, quantidade=taxa_ruido)
return img_ruidosa, anotacoes

```

```

# Geração de 5 amostras para exibição inicial no topo do projeto
n_amstras_iniciais = 5
rng_demo = random.Random(42)

imgs_demo = []
titulos_demo = []

for idx in range(n_amstras_iniciais):
    img_ruid, anotacoes = gera_imagem_ruidosa(rng=rng_demo)

    # Gravação temporária da anotação para leitura nativa pelo mm.showBoundingBox
    filename_temp = f"temp_label_{idx}.txt"
    with open(filename_temp, "w") as f:
        for c, xc, yc, w, h in anotacoes:
            f.write(f"{c} {xc:.4f} {yc:.4f} {w:.4f} {h:.4f}\n")

    img_annotada = mm.showBoundingBox(img_ruid, filename=filename_temp, fmt="yolo", show=False)
    imgs_demo.append(img_annotada)
    titulos_demo.append(f"Amostra {idx+1}")

# Exibição do painel de 5 amostras
mm.show(
    imgs_demo,
    titles=titulos_demo,
    cols=n_amstras_iniciais,
    figsize=(14, 3)
)

```

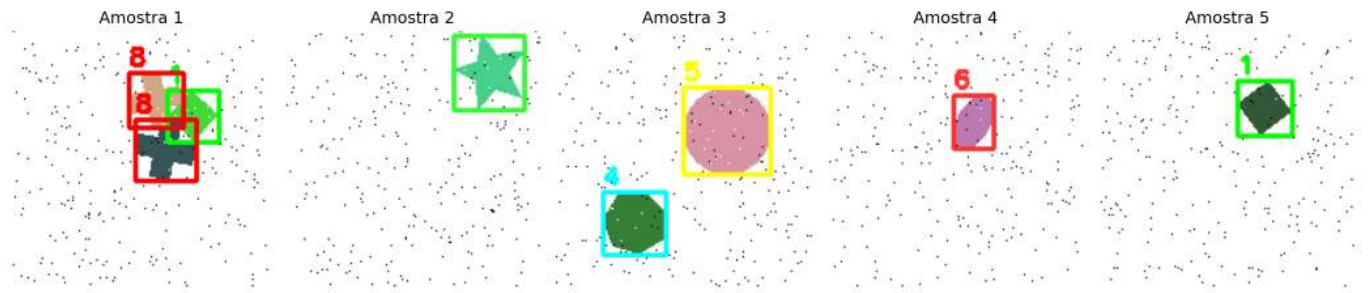


Figura 8.25: Amostras iniciais do dataset sintético ruidoso de objetos geométricos com as caixas delimitadoras do formato YOLO sobrepostas.

8.9.2 Bloco 2: Montagem da Estrutura de Pastas e do Manifesto `data.yaml`

A biblioteca de treinamento do YOLO (como a `ultralytics`) exige que as imagens e os arquivos de texto com as anotações estejam organizados em uma estrutura hierárquica específica:

```
shapes_dataset/
  data.yaml
  images/
    train/
    val/
  labels/
    train/
    val/
```

1. **Geração das Imagens e Anotações:** Grava-se o conjunto com 90 imagens ruidosas para treino (`train`) e 20 para validação (`val`), acompanhadas de seus arquivos `.txt` com coordenadas no formato YOLO (c_i, x_c, y_c, w, h).
2. **Criação do Manifesto (`data.yaml`):** Especifica os caminhos dos diretórios de treino/validação e a lista com os nomes das 9 categorias.

```
import os

base_dir = "shapes_dataset"
rng_global = random.Random(42)

for split, n_imgs in [("train", 90), ("val", 20)]:
    os.makedirs(f"{base_dir}/images/{split}", exist_ok=True)
    os.makedirs(f"{base_dir}/labels/{split}", exist_ok=True)

    for i in range(n_imgs):
        img_ruid, anotacoes = gera_imagem_ruidosa(rng=rng_global)
        cv2.imwrite(f"{base_dir}/images/{split}/{i:04d}.jpg", img_ruid)

        with open(f"{base_dir}/labels/{split}/{i:04d}.txt", "w") as f:
            for c, xc, yc, w, h in anotacoes:
                f.write(f"{c} {xc:.4f} {yc:.4f} {w:.4f} {h:.4f}\n")

    with open(f"{base_dir}/data.yaml", "w") as f:
        f.write(f"path: {os.path.abspath(base_dir)}\ntrain: images/train\nval: images/val\nnames:\n")
        for i, nome in enumerate(CLASSES):
            f.write(f" {i}: {nome}\n")

print("Dataset ruidoso gerado com sucesso: 90 imagens de treino e 20 de validação.")
```

Dataset ruidoso gerado com sucesso: 90 imagens de treino e 20 de validação.

8.9.3 Bloco 3: Pré-processamento com Filtro de Mediana para Restauração da Imagem

Embora as redes convolucionais aprendam a suavizar pequenas variações, a presença de ruído impulsivo (*sal e pimenta*) degrada a precisão do detector. O **Filtro de Mediana** (`cv2.medianBlur`) representa a solução clássica ideal: substitui cada pixel pela mediana da vizinhança, removendo completamente os ruídos pretos e brancos sem borrar as bordas dos objetos.

1. **Restauração Espacial:** Aplica-se a filtragem de mediana com janela 3×3 sobre a imagem ruidosa.
2. **Comparativo Visual:** A Figura 8.26 ilustra o resultado da restauração mantendo as caixas delimitadoras sobrepostas via `mm.showBoundingBox()`.

```
# 1. Carregamento da primeira amostra ruidosa do dataset
caminho_img = f"{base_dir}/images/train/0000.jpg"
caminho_label = f"{base_dir}/labels/train/0000.txt"

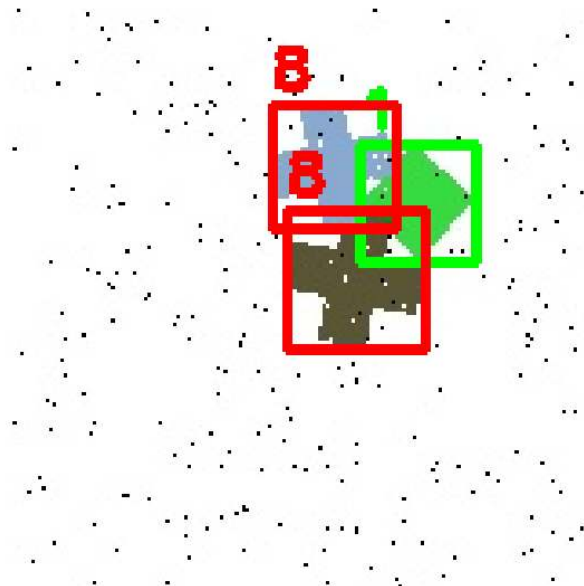
img_ruidosa = mm.read(caminho_img)

# 2. Pré-processamento com Filtro de Mediana (janela 3x3)
img_filtrada = cv2.medianBlur(img_ruidosa, ksize=3)

# 3. Sobreposição das bounding boxes com mm.showBoundingBox
img_ruid_annotada = mm.showBoundingBox(img_ruidosa, filename=caminho_label, fmt="yolo", show=False)
img_filt_annotada = mm.showBoundingBox(img_filtrada, filename=caminho_label, fmt="yolo", show=False)

# 4. Exibição comparativa com mm.show
mm.show(
    [img_ruid_annotada, img_filt_annotada],
    titles=["1. Imagem Ruidosa Original (Sal e Pimenta)", "2. Pré-processada (Filtro de Mediana 3x3)"],
    cols=2,
    figsize=(9, 4)
)
```

1. Imagem Ruidosa Original (Sal e Pimenta)



2. Pré-processada (Filtro de Mediana 3x3)

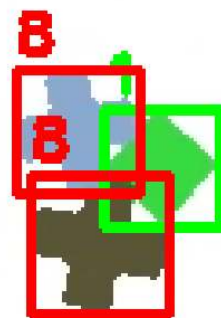


Figura 8.26: Comparativo entre a imagem ruidosa original do dataset sintético e a versão restaurada via Filtro de Mediana com as anotações YOLO sobrepostas.

8.9.4 Bloco 4: Restauração do *Dataset* e Treinamento do YOLOv8 (mAP)

Para elevar a acurácia de detecção do modelo sobre o conjunto sintético ruidoso, aplica-se o **Filtro de Mediana** (3×3) em lote sobre todas as imagens dos diretórios `train` e `val` antes do treinamento. Em seguida, realiza-se o ajuste fino (*fine-tuning*) da rede `yolov8n.pt` configurando a resolução de entrada para 320×320 *pixels* ao longo de 30 épocas sem congelamento do *backbone*, permitindo que os filtros convolucionais se adaptem às geometrias sintéticas.

1. **Restauração em Lote:** O laço varre os diretórios do conjunto de dados, aplicando `cv2.medianBlur` para eliminar os ruídos impulsivos mantendo as bordas dos polígonos intactas.
2. **Ajuste Fino:** Instancia-se a rede `YOLO("yolov8n.pt")` e executa-se a otimização via `.train()`.
3. **Métricas de Validação:** A função `.val()` extrai as métricas globais de precisão, revocação (*recall*) e o índice *mAP*@50 sobre o conjunto de validação restaurado.

```
import os
import cv2
from ultralytics import YOLO

base_dir = "shapes_dataset"

# 1. Aplicação do Filtro de Mediana em lote sobre as pastas do dataset
for split in ["train", "val"]:
    pasta_imgs = f"{base_dir}/images/{split}"
    for nome_arq in os.listdir(pasta_imgs):
        if nome_arq.endswith(".jpg"):
            caminho_completo = os.path.join(pasta_imgs, nome_arq)
            img_ruidosa = cv2.imread(caminho_completo)
            img_filtrada = cv2.medianBlur(img_ruidosa, ksize=3)
            cv2.imwrite(caminho_completo, img_filtrada)

print("Pré-processamento em lote (Filtro de Mediana) concluído em train e val.\n")

# 2. Carregamento do modelo YOLOv8n pré-treinado na COCO
modelo_yolo = YOLO("yolov8n.pt")

# 3. Treinamento (Fine-Tuning) no dataset geométrico restaurado
resultados_treino = modelo_yolo.train(
    data=f"{base_dir}/data.yaml",
    epochs=30,
    imgsz=320,
    batch=16,
    device=device,
    verbose=False,
    plots=False
)

# 4. Avaliação quantitativa no conjunto de validação
metricas = modelo_yolo.val()

precision = metricas.results_dict["metrics/precision(B)"]
recall = metricas.results_dict["metrics/recall(B)"]
map50 = metricas.results_dict["metrics/mAP50(B)"]

print(f"--- Desempenho Otimizado do Modelo YOLOv8 ---")
print(f"Precisão: {precision*100:.2f}%")
print(f"Revocação (Recall): {recall*100:.2f}%")
print(f"mAP@50 (Acurácia de Detecção): {map50*100:.2f}%")
```


Pré-processamento em lote (Filtro de Mediana) concluído em train e val.

New <https://pypi.org/project/ultralytics/8.4.112> available Update with 'pip install -U ultralytics'
 Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cpu CPU (Intel Core i7-4870HQ 2.50GHz)
 engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=1
 Overriding model.yaml nc=80 with nc=9

	from	n	params	module	arguments
0	-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	7360	ultralytics.nn.modules.block.C2f	[32, 32, 1, True]
3	-1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
4	-1	2	49664	ultralytics.nn.modules.block.C2f	[64, 64, 2, True]
5	-1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
6	-1	2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]
7	-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	460288	ultralytics.nn.modules.block.C2f	[256, 256, 1, True]
9	-1	1	164608	ultralytics.nn.modules.block.SPPF	[256, 256, 5]
10	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
11	[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
12	-1	1	148224	ultralytics.nn.modules.block.C2f	[384, 128, 1]
13	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
14	[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
15	-1	1	37248	ultralytics.nn.modules.block.C2f	[192, 64, 1]
16	-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
17	[-1, 12]	1	0	ultralytics.nn.modules.conv.Concat	[1]
18	-1	1	123648	ultralytics.nn.modules.block.C2f	[192, 128, 1]
19	-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
20	[-1, 9]	1	0	ultralytics.nn.modules.conv.Concat	[1]
21	-1	1	493056	ultralytics.nn.modules.block.C2f	[384, 256, 1]
22	[15, 18, 21]	1	753067	ultralytics.nn.modules.head.Detect	[9, 16, None, [64, 128, 160, 192, 224, 256, 288, 320]]

Model summary: 130 layers, 3,012,603 parameters, 3,012,587 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights

Freezing layer 'model.22.dfl.conv.weight'

train: Fast image access (ping: 0.0±0.0 ms, read: 192.3±66.3 MB/s, size: 3.7 KB)

train: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/train... 90 images, 0 backgrounds

train: New cache created: /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/train.cache

val: Fast image access (ping: 0.0±0.0 ms, read: 176.3±82.6 MB/s, size: 3.3 KB)

val: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/val... 20 images, 0 backgrounds

val: New cache created: /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/val.cache

optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer'

optimizer: AdamW(lr=0.000769, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0)

Image sizes 320 train, 320 val

Using 0 dataloader workers

Logging results to /home/fz/fz/VSCode/pdi-vc/runs/detect/train-19

Starting training for 30 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
1/30	OG	0.9204	4.071	1.062	49	320: 100%	6/6	1.4s/it	8
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1
	all	20	42	0.00266	0.469	0.0383	0.0155		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
2/30	OG	0.8041	3.856	1.011	45	320: 100%	6/6	1.4s/it	8
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%		1
	all	20	42	0.00179	0.175	0.0313	0.0129		
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size			
3/30	OG	0.8103	3.54	1.004	38	320: 100%	6/6	1.9s/it	1

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	
	all	20	42	0.00291	0.463	0.0649	0.0439		1.7s/it
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
4/30	OG	0.7457	3.216	0.9794	35	320:	100%	6/6	1.7s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.00584	0.819	0.126	0.0975		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
5/30	OG	0.7495	3.146	0.9868	47	320:	100%	6/6	1.6s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.00686	0.906	0.185	0.15		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
6/30	OG	0.7402	2.846	1.012	32	320:	100%	6/6	1.6s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.00695	0.984	0.241	0.205		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
7/30	OG	0.7026	2.679	0.9802	23	320:	100%	6/6	1.5s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.00693	0.984	0.235	0.207		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
8/30	OG	0.7604	2.446	1.034	42	320:	100%	6/6	1.6s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.889	0.036	0.253	0.227		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
9/30	OG	0.8199	2.261	1.031	35	320:	100%	6/6	1.6s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	1	0.0371	0.305	0.274		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
10/30	OG	0.7595	2.196	1.011	35	320:	100%	6/6	1.9s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.937	0.102	0.407	0.362		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
11/30	OG	0.7695	2.111	1.01	41	320:	100%	6/6	2.1s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.771	0.175	0.507	0.447		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
12/30	OG	0.7157	1.947	0.9876	51	320:	100%	6/6	2.5s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.786	0.336	0.535	0.469		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
13/30	OG	0.7148	1.828	0.9886	33	320:	100%	6/6	2.7s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.365	0.516	0.502	0.441		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
14/30	OG	0.7158	1.724	0.9985	26	320:	100%	6/6	2.6s/it
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	1.7s/it
	all	20	42	0.365	0.526	0.504	0.442		
Epoch	GPU_mem	box_loss	cls_loss	dfll_loss	Instances	Size			
15/30	OG	0.7018	1.674	0.9997	42	320:	100%	6/6	2.6s/it

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.407	0.634	0.567	0.501			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
16/30	OG	0.6909	1.677	0.9896	49	320:	100%	6/6	2.6s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.444	0.786	0.706	0.641			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
17/30	OG	0.6926	1.583	1.022	33	320:	100%	6/6	2.7s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.532	0.794	0.74	0.676			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
18/30	OG	0.677	1.509	0.9846	29	320:	100%	6/6	2.8s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.656	0.873	0.787	0.726			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
19/30	OG	0.6774	1.465	1.008	45	320:	100%	6/6	2.6s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.648	0.878	0.804	0.744			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
20/30	OG	0.6986	1.492	0.9979	36	320:	100%	6/6	2.6s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.606	0.825	0.791	0.732			
Closing dataloader mosaic										
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
21/30	OG	0.4995	1.289	0.8883	21	320:	100%	6/6	2.6s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.621	0.846	0.793	0.737			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
22/30	OG	0.5064	1.259	0.89	22	320:	100%	6/6	3.2s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.618	0.779	0.771	0.713			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
23/30	OG	0.4733	1.2	0.8766	22	320:	100%	6/6	2.6s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.662	0.657	0.743	0.673			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
24/30	OG	0.4578	1.198	0.88	20	320:	100%	6/6	2.4s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.501	0.751	0.801	0.746			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
25/30	OG	0.4517	1.128	0.884	22	320:	100%	6/6	2.5s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.846	0.671	0.844	0.777			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				
26/30	OG	0.4324	1.099	0.8762	19	320:	100%	6/6	2.5s/it	1/
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%		1/
	all	20	42	0.739	0.816	0.893	0.814			
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size				

```

27/30      OG      0.4487      1.056      0.8875      21      320: 100%      6/6 2.3s/it :
          Class    Images  Instances    Box(P      R      mAP50  mAP50-95): 100%
          all      20      42      0.834      0.817      0.897      0.824

Epoch    GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
28/30      OG      0.4378      1.05      0.8809      22      320: 100%      6/6 2.3s/it :
          Class    Images  Instances    Box(P      R      mAP50  mAP50-95): 100%
          all      20      42      0.902      0.808      0.897      0.819

Epoch    GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
29/30      OG      0.4156      1.029      0.8647      22      320: 100%      6/6 2.6s/it :
          Class    Images  Instances    Box(P      R      mAP50  mAP50-95): 100%
          all      20      42      0.908      0.832      0.895      0.818

Epoch    GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
30/30      OG      0.4199      0.9819      0.865      22      320: 100%      6/6 2.5s/it :
          Class    Images  Instances    Box(P      R      mAP50  mAP50-95): 100%
          all      20      42      0.907      0.843      0.896      0.839

30 epochs completed in 0.128 hours.
Optimizer stripped from /home/fz/fz/VSCode/pdi-vc/runs/detect/train-19/weights/last.pt, 6.2MB
Optimizer stripped from /home/fz/fz/VSCode/pdi-vc/runs/detect/train-19/weights/best.pt, 6.2MB

Validating /home/fz/fz/VSCode/pdi-vc/runs/detect/train-19/weights/best.pt...
Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cpu CPU (Intel Core i7-4870HQ 2.50GHz)
Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs
          Class    Images  Instances    Box(P      R      mAP50  mAP50-95): 100%
          all      20      42      0.908      0.842      0.896      0.839
Speed: 1.4ms preprocess, 75.5ms inference, 0.0ms loss, 5.8ms postprocess per image
Ultralytics 8.4.87 Python-3.13.5 torch-2.12.1+cpu CPU (Intel Core i7-4870HQ 2.50GHz)
Model summary (fused): 73 layers, 3,007,403 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 198.9±40.3 MB/s, size: 3.0 KB)
val: Scanning /home/fz/fz/VSCode/pdi-vc/all/cap09/shapes_dataset/labels/val.cache... 20 images, 0 backgr
          Class    Images  Instances    Box(P      R      mAP50  mAP50-95): 100%
          all      20      42      0.908      0.842      0.896      0.839
Triangle         2         2      0.835         1      0.995      0.945
Square          6         6      0.763         1      0.995      0.928
Pentagon         4         5         1      0.993      0.995      0.902
Hexagon          2         2         1         0      0.207      0.17
Heptagon         1         1      0.894         1      0.995      0.995
Circle           7         7         1      0.656      0.909      0.715
Ellipse          2         2      0.795         1      0.995      0.995
Star             8         9         1      0.931      0.995      0.962
Cross            7         8      0.881         1      0.982      0.934
Speed: 1.0ms preprocess, 62.7ms inference, 0.0ms loss, 5.2ms postprocess per image
Results saved to /home/fz/fz/VSCode/pdi-vc/runs/detect/val-11
--- Desempenho Otimizado do Modelo YOLOv8 ---
Precisão: 90.76%
Revocação (Recall): 84.23%
mAP@50 (Acurácia de Detecção): 89.64%

```

8.9.5 Bloco 5: Comparativo de Inferência: Imagem Ruidosa vs. Restaurada com Filtro de Mediana

Para evidenciar visualmente o impacto do ruído sal e pimenta na capacidade de localização do detector, realiza-se a inferência comparativa utilizando uma imagem de validação ruidosa sem tratamento e a mesma imagem tratada pelo **Filtro de Mediana**.

1. **Inferência com a YOLO:** A função `modelo_yolo.predict()` executa a detecção em ambas as matrizes NumPy (`img_ruidosa_teste` e `img_filtrada_teste`) com limiar de confiança fixado em

25% (conf=0.25).

2. **Visualização:** A Figure 8.27 exibe os retângulos e rótulos preditos diretamente na tela via `mm.show()`.

```
# 1. Carregamento de uma amostra de teste original (sem o filtro salvo em lote)
caminho_teste = f"{base_dir}/images/val/0011.jpg"
img_ruidosa_teste = mm.read(caminho_teste)

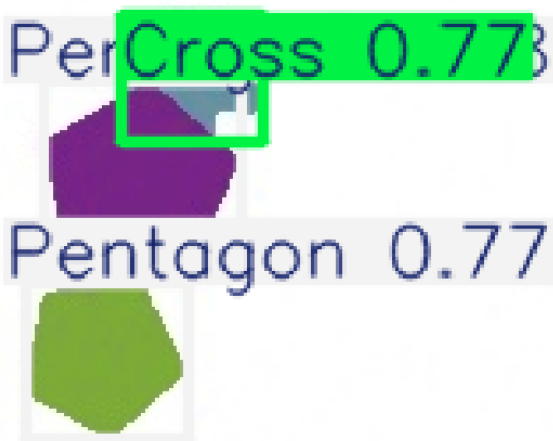
# 2. Aplicação pontual do Filtro de Mediana (3x3) para comparação
img_filtrada_teste = cv2.medianBlur(img_ruidosa_teste, ksize=3)

# 3. Inferência com o modelo YOLOv8 treinado
pred_ruidosa = modelo_yolo.predict(img_ruidosa_teste, conf=0.25, verbose=False)[0]
pred_filtrada = modelo_yolo.predict(img_filtrada_teste, conf=0.25, verbose=False)[0]

# 4. Extração das matrizes anotadas pelo gerador do YOLO (conversão BGR -> RGB)
img_pred_ruid = cv2.cvtColor(pred_ruidosa.plot(), cv2.COLOR_BGR2RGB)
img_pred_filt = cv2.cvtColor(pred_filtrada.plot(), cv2.COLOR_BGR2RGB)

# 5. Exibição comparativa padronizada via mm.show
mm.show(
    [img_pred_ruid, img_pred_filt],
    titles=[
        f"Inferência na Imagem Ruidosa ({len(pred_ruidosa.bboxes)} objetos)",
        f"Inferência na Imagem Filtrada ({len(pred_filtrada.bboxes)} objetos)"
    ],
    cols=2,
    figsize=(10, 4)
)
```

Inferência na Imagem Ruidosa (3 objetos)



Inferência na Imagem Filtrada (3 objetos)

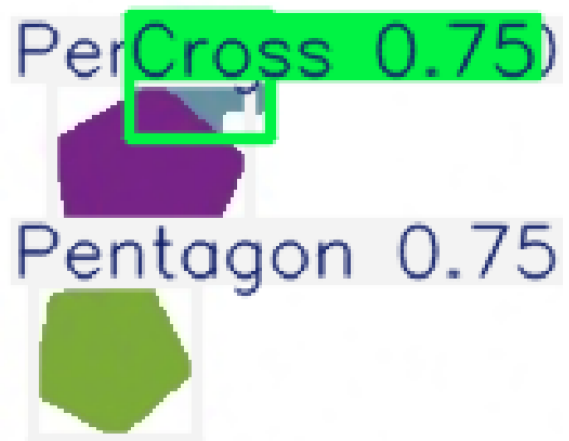


Figura 8.27: Comparativo de inferência com a YOLOv8: detecção na imagem ruidosa original e na imagem restaurada via Filtro de Mediana com as predições exibidas via `mm.show`.

i 🧠 Um domínio muito mais distante que o dos dígitos

No experimento de transferência de aprendizado entre dígitos manuscritos (domínios A e B), a tarefa de origem e a de destino compartilhavam estatísticas visuais muito próximas: ambas eram traços em tons de cinza sobre fundo uniforme. Aqui, a distância entre domínios é bem maior — o YOLO

foi pré-treinado em **fotografias naturais coloridas** da COCO (pessoas, animais, veículos, objetos do cotidiano), e a tarefa de destino consiste em **formas geométricas sintéticas, de cor sólida e contorno bem definido**, sem textura, iluminação ou fundo complexo.

Ainda assim, a transferência de aprendizado é vantajosa: as camadas iniciais de um detector treinado na COCO aprendem filtros genéricos — detectores de bordas, cantos e regiões de contraste — que continuam úteis para delimitar o contorno de um triângulo ou de uma estrela, mesmo que o conteúdo visual final seja bastante distinto. É por isso que permitir o ajuste fino (*fine-tuning*) de todas as camadas, combinado com o pré-processamento para eliminação do ruído sal e pimenta, viabiliza taxas de acurácia elevadas na detecção de objetos do novo domínio.

8.9.6 Usando um Conjunto de Dados Real

A *pipeline* acima foi construído inteiramente em torno do **formato de anotação YOLO** (classe, x_{centro} , y_{centro} , largura, altura, normalizados pela largura e altura da imagem) exatamente para que ele possa ser reaproveitado sem alterações caso o leitor tenha acesso a um conjunto de imagens reais anotado da mesma forma — por exemplo, um conjunto de imagens de objetos geométricos fotografados ou renderizados, cada um com um arquivo `.txt` correspondente no mesmo formato usado aqui. Para isso, bastaria:

1. Organizar as imagens reais em `shapes_dataset/images/train` e `shapes_dataset/images/val`, e os arquivos `.txt` de anotação correspondentes nas pastas `labels/train` e `labels/val` (um arquivo de anotação por imagem, mesmo nome-base, extensão `.txt`);
2. Ajustar o arquivo `data.yaml` caso o número ou os nomes das classes sejam diferentes;
3. Executar as mesmas células de treinamento, ajuste fino e visualização já apresentadas, sem qualquer outra modificação de código.

Essa separação entre **geração/organização dos dados** e **treinamento do modelo** é, na prática, o motivo pelo qual formatos de anotação padronizados (como o do YOLO) são tão amplamente adotados: eles permitem trocar o conjunto de dados de entrada — sintético por real, um domínio por outro — mantendo inalterado todo o restante do *pipeline* de transferência de aprendizado.

8.10 Projeto Prático 3: Segmentação Semântica com U-Net

A subseção anterior apresentou um modelo pré-treinado (*DeepLabV3*) produzindo diretamente um rótulo de classe por *pixel*. Esta seção completa a sequência de projetos práticos — classificação (Projeto Prático 1) e detecção (Projeto Prático 2) — abordando a segmentação semântica implementada e treinada do zero com a **U-Net**, a arquitetura de referência introduzida por Ronneberger; Fischer; Brox (2015).

Na classificação do Projeto Prático 1, o mapa de características final era achatado (*flatten*) em um vetor, descartando a informação espacial em favor de um único rótulo por imagem. A U-Net, por sua vez, produz uma saída com a **mesma resolução espacial da entrada**: um mapa bidimensional no qual cada *pixel* recebe sua própria classificação. Essa exigência — preservar detalhe espacial de alta resolução ao mesmo tempo em que se constrói contexto semântico em camadas profundas — motiva a arquitetura de codificador-decodificador com conexões de atalho (*skip connections*).

O cenário utilizado simula a segmentação de nódulos em exames médicos sintéticos: imagens em escala de cinza contêm uma região circular (“nódulo”) sobreposta a um fundo, ambos contaminados por ruído gaussiano e com médias de intensidade muito próximas — um desafio intencional de baixo contraste, ideal para demonstrar o ganho do aprendizado espacial em relação à limiarização pontual.

8.10.1 Bloco 1: Gerador do Conjunto Sintético de Nódulos e Exibição Inicial

O gerador a seguir produz pares (imagem, máscara): a imagem contém uma região circular de intensidade ligeiramente superior à do fundo, ambas afetadas pelo mesmo desvio-padrão de ruído gaussiano. A máscara binária delimita exatamente a região do nódulo e serve como verdade de referência (*ground truth*).

1. **Construção do Nódulo:** A função `gera_imagem_com_nodulo` sobrepõe o nódulo ao fundo sobre uma matriz 64×64 e aplica ruído gaussiano.
2. **Visualização com `mm.show`:** A Figure 8.28 ilustra as duas primeiras amostras e suas respectivas máscaras.

```
import cv2
import numpy as np

TAM_IMG = 64

def gera_imagem_com_nodulo(
    tam=TAM_IMG,
    raio_min=7,
    raio_max=15,
    media_fundo=95,
    media_nodulo=118,
    sigma_ruido=26,
    rng=None,
):
    rng = rng or np.random.default_rng()
    fundo = rng.normal(media_fundo, sigma_ruido, (tam, tam))
    nodulo = rng.normal(media_nodulo, sigma_ruido, (tam, tam))
    mascara = np.zeros((tam, tam), dtype=np.uint8)

    raio = int(rng.integers(raio_min, raio_max))
    cx = int(rng.integers(raio + 4, tam - raio - 4))
    cy = int(rng.integers(raio + 4, tam - raio - 4))

    cv2.circle(mascara, (cx, cy), raio, 255, -1)
    imagem = np.clip(np.where(mascara > 0, nodulo, fundo), 0, 255).astype(
        np.uint8
    )
    return imagem, mascara

# Geração dos conjuntos de treinamento e validação
rng_dados = np.random.default_rng(42)
N_TREINO, N_VAL = 160, 40

imgs_treino, masks_treino = zip(
    *[gera_imagem_com_nodulo(rng=rng_dados) for _ in range(N_TREINO)]
)
imgs_val, masks_val = zip(
    *[gera_imagem_com_nodulo(rng=rng_dados) for _ in range(N_VAL)]
)

print(
    f"Conjunto de treino: {N_TREINO} imagens | Conjunto de validação: {N_VAL} imagens\n"
)

# Exibição de amostras iniciais via mm.show
mm.show(
    [imgs_treino[0], masks_treino[0], imgs_treino[1], masks_treino[1]],
    titles=["Imagem 1", "Máscara 1", "Imagem 2", "Máscara 2"],
    cols=4,
    figsize=(11, 3),
)
```

Conjunto de treino: 160 imagens | Conjunto de validação: 40 imagens

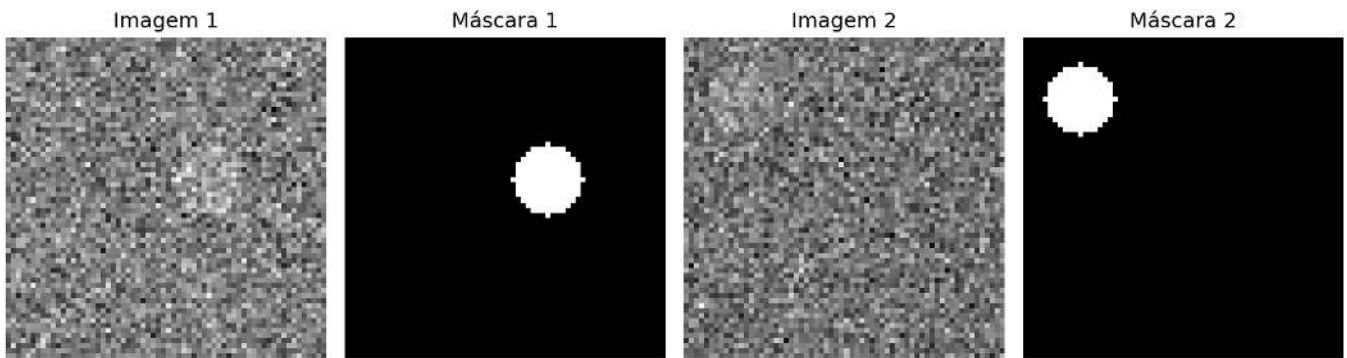


Figura 8.28: Amostras do conjunto de dados sintético de nódulos: imagem em escala de cinza sob ruído e respectiva máscara binária de referência exibidas via `mm.show`.

8.10.2 Bloco 2: Linha de Base Clássica (Filtragem, Otsu e Morfologia)

Antes de empregar a U-Net, avalia-se o desempenho de um *pipeline* morfológico clássico construído com a biblioteca `morph`: um **suavizador gaussiano** (`mm.blur`), uma **limiarização de Otsu** (`mm.threshold`) e uma **abertura morfológica** (`mm.open`) para eliminação de ruídos isolados.

1. **Métrica de Interseção sobre União (IoU):** A função `iou_mascaras` calcula o grau de sobreposição *pixel a pixel* entre a predição e a máscara real.
2. **Execução e Comparativo:** A Figure 8.29 exibe o resultado da segmentação clássica em uma imagem de teste, demonstrando as limitações do limiar global sob baixo contraste.

```
def iou_mascaras(predita, referencia):
    p, r = predita > 0, referencia > 0
    intersecao = np.logical_and(p, r).sum()
    uniao = np.logical_or(p, r).sum()
    return intersecao / uniao if uniao else 1.0

def segmenta_classico(imagem, elemento_estrutural):
    suavizada = mm.blur(imagem, 7)
    binaria = mm.threshold(suavizada)
    return mm.open(binaria, elemento_estrutural)

elemento_estrutural = mm.sedisk(5)
ious_classico = [
    iou_mascaras(segmenta_classico(img, elemento_estrutural), mask)
    for img, mask in zip(imgs_val, masks_val)
]
iou_classico_medio = float(np.mean(ious_classico))
print(
    f"IoU médio (linha de base clássica) na validação: {iou_classico_medio:.4f}\n"
)

predicao_classica_exemplo = segmenta_classico(imgs_val[0], elemento_estrutural)

mm.show(
    [imgs_val[0], masks_val[0], predicao_classica_exemplo],
    titles=["Imagem", "Máscara de Referência", "Predição Clássica"],
    cols=3,
    figsize=(9, 3.2),
)
```

IoU médio (linha de base clássica) na validação: 0.7516

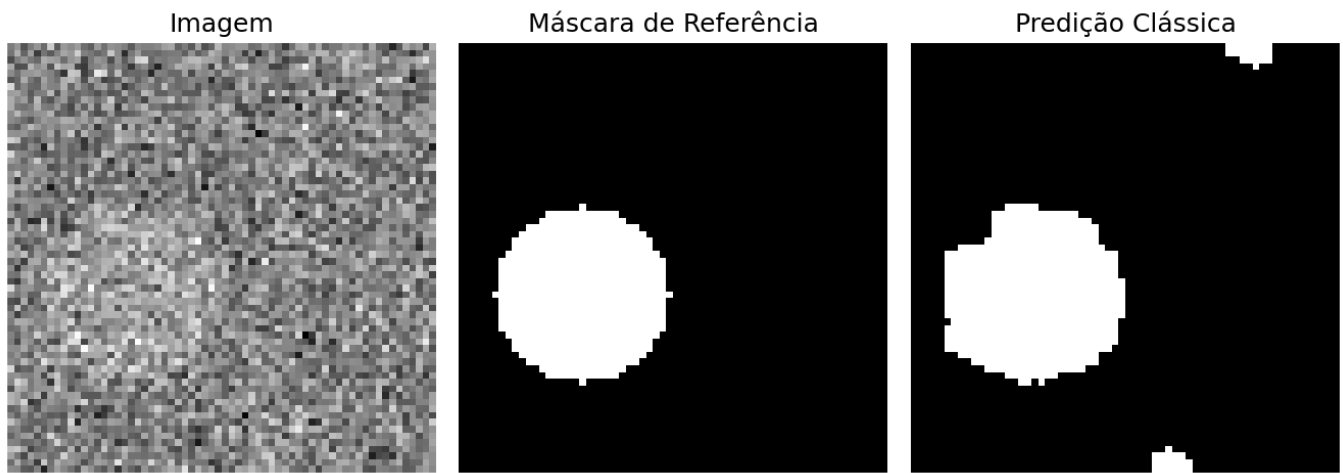


Figura 8.29: Linha de base clássica de segmentação: suavização, limiarização de Otsu e abertura morfológica exibidas.

8.10.3 Bloco 3: Construção da Arquitetura U-Net e Funções de Perda

A U-Net combina um **codificador** (redução espacial e extração de contexto) e um **decodificador** (reconstrução espacial via convoluções transpostas). O ponto central são as **conexões de atalho** (*skip connections*), que concatenam os mapas de características do codificador às etapas do decodificador na mesma resolução, preservando contornos e bordas.

Entrada → Codificador (Conv→Conv→Pool) × 3 → Base
 Conexões de atalho reinjetam mapas de alta resolução
 Base → Decodificador (Upsample→Concat→Conv→Conv) × 3 → Saída 1×1 (logits)

1. **Bloco Convolucional Base (BlocoConv)**: Duas convoluções 3×3 consecutivas com ativação ReLU e *padding* de preservação espacial.
2. **Perda Combinada (BCE + Dice)**: A função perda `perda_segmentacao` soma a Entropia Cruzada Binária ao Coeficiente de Dice para otimizar tanto o acerto pontual quanto a sobreposição da forma.

```
import torch
import torch.nn as nn

class BlocoConv(nn.Module):

    def __init__(self, canais_entrada, canais_saida):
        super().__init__()
        self.rede = nn.Sequential(
            nn.Conv2d(canais_entrada, canais_saida, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(canais_saida, canais_saida, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
        )

    def forward(self, x):
        return self.rede(x)

class UNetCompacta(nn.Module):

    def __init__(self, canais_entrada=1, base=8):
        super().__init__()
        self.enc1 = BlocoConv(canais_entrada, base)
```

```

self.enc2 = BlocoConv(base, base * 2)
self.enc3 = BlocoConv(base * 2, base * 4)
self.pool = nn.MaxPool2d(2)

self.fundo = BlocoConv(base * 4, base * 8)

self.up3 = nn.ConvTranspose2d(
    base * 8, base * 4, kernel_size=2, stride=2
)
self.dec3 = BlocoConv(base * 8, base * 4)
self.up2 = nn.ConvTranspose2d(
    base * 4, base * 2, kernel_size=2, stride=2
)
self.dec2 = BlocoConv(base * 4, base * 2)
self.up1 = nn.ConvTranspose2d(
    base * 2, base, kernel_size=2, stride=2
)
self.dec1 = BlocoConv(base * 2, base)

self.saida = nn.Conv2d(base, 1, kernel_size=1)

def forward(self, x):
    e1 = self.enc1(x)
    e2 = self.enc2(self.pool(e1))
    e3 = self.enc3(self.pool(e2))
    f = self.fundo(self.pool(e3))

    d3 = self.dec3(torch.cat([self.up3(f), e3], dim=1))
    d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))
    d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))
    return self.saida(d1)

def para_tensores(imagens, mascaras):
    X = (
        torch.tensor(np.stack(imagens), dtype=torch.float32).unsqueeze(1)
        / 255.0
    )
    Y = (
        torch.tensor(np.stack(mascaras), dtype=torch.float32).unsqueeze(1)
        / 255.0
    )
    return X, Y

def perda_dice(logits, alvo, eps=1e-6):
    probs = torch.sigmoid(logits)
    intersecao = (probs * alvo).sum(dim=(1, 2, 3))
    uniao = probs.sum(dim=(1, 2, 3)) + alvo.sum(dim=(1, 2, 3))
    dice = (2 * intersecao + eps) / (uniao + eps)
    return 1 - dice.mean()

def perda_segmentacao(logits, alvo):
    return nn.functional.binary_cross_entropy_with_logits(
        logits, alvo
    ) + perda_dice(logits, alvo)

```

8.10.4 Bloco 4: Treinamento da U-Net e Avaliação de Desempenho

O treinamento é executado por 35 épocas utilizando o otimizador *Adam*. A cada época, monitora-se a Perda de Treinamento e o índice IoU médio no conjunto de validação.

1. **Laço de Treinamento:** Atualizam-se os parâmetros com *mini-batches* de 16 amostras.
2. **Evolução Gráfica:** A Figure 8.30 exibe o gráfico com o progresso da perda e do IoU.

```
import matplotlib.pyplot as plt
import torch.optim as optim

modelo_unet = UNetCompacta()
print(
    f"Parâmetros treináveis da U-Net: {sum(p.numel() for p in modelo_unet.parameters())}"
)

X_treino_unet, Y_treino_unet = para_tensores(imgs_treino, masks_treino)
X_val_unet, Y_val_unet = para_tensores(imgs_val, masks_val)

otimizador_unet = optim.Adam(modelo_unet.parameters(), lr=1e-3)
n = X_treino_unet.size(0)
tam_lote = 16
epocas_unet = 35
historico_perda_unet, historico_iou_unet = [], []

for epoca in range(epocas_unet):
    modelo_unet.train()
    perm = torch.randperm(n)
    perda_epoca = 0.0

    for i in range(0, n, tam_lote):
        idx = perm[i : i + tam_lote]
        otimizador_unet.zero_grad()
        logits = modelo_unet(X_treino_unet[idx])
        perda = perda_segmentacao(logits, Y_treino_unet[idx])
        perda.backward()
        otimizador_unet.step()
        perda_epoca += perda.item() * len(idx)

    modelo_unet.eval()
    with torch.no_grad():
        predicacao_val = (torch.sigmoid(modelo_unet(X_val_unet)) > 0.5).float()
        intersecao = (predicacao_val * Y_val_unet).sum(dim=(1, 2, 3))
        uniao = ((predicacao_val + Y_val_unet) > 0).float().sum(dim=(1, 2, 3))
        iou_epoca = (intersecao / uniao.clamp(min=1e-6)).mean().item()

    historico_perda_unet.append(perda_epoca / n)
    historico_iou_unet.append(iou_epoca)

iou_unet_final = historico_iou_unet[-1]
print(f"IoU médio final da U-Net na validação: {iou_unet_final:.4f}")

# Gráfico da evolução do treinamento
fig, ax1 = plt.subplots(figsize=(6, 3.8))
ax1.plot(historico_perda_unet, color="#dc2626", label="Perda (treino)")
ax1.set_xlabel("Época")
ax1.set_ylabel("Perda", color="#dc2626")

ax2 = ax1.twinx()
ax2.plot(historico_iou_unet, color="#2563eb", label="IoU (validação)")
```

```
ax2.set_ylabel("IoU (validação)", color="#2563eb")

plt.title("Treinamento da U-Net - Segmentação de Nódulos Sintéticos")
fig.tight_layout()
plt.show()
```

Parâmetros treináveis da U-Net: 120681
IoU médio final da U-Net na validação: 0.8815

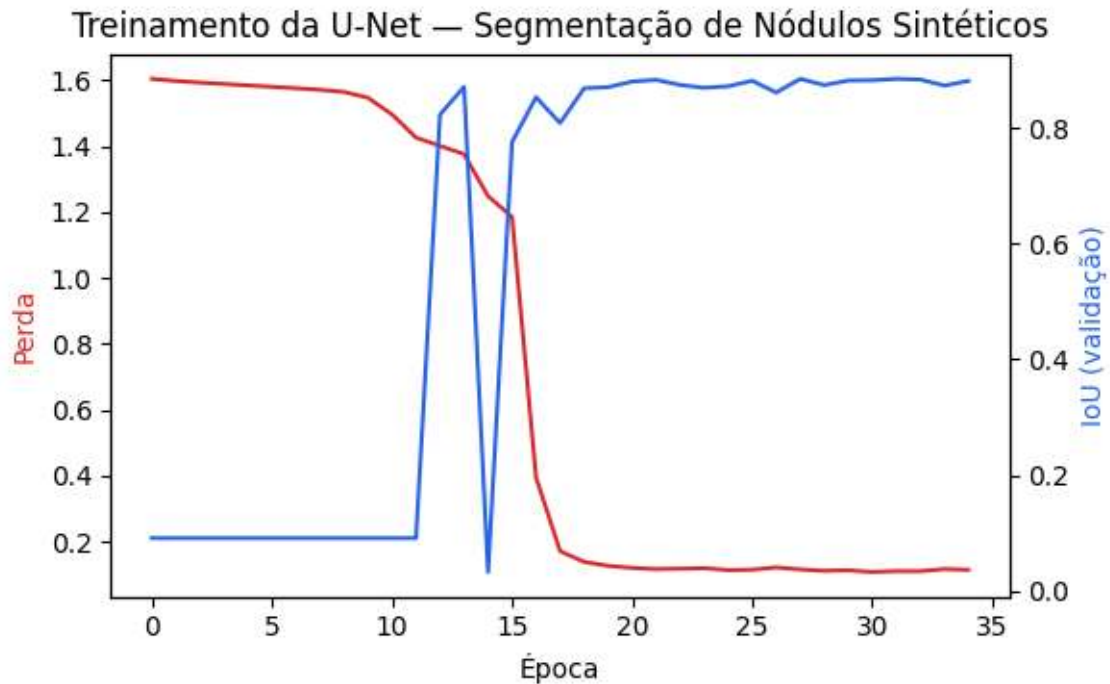


Figura 8.30: Evolução do treinamento da U-Net compacta: redução da perda e elevação do índice IoU médio no conjunto de validação ao longo de 35 épocas.

8.10.5 Bloco 5: Comparativo Quantitativo e Qualitativo (Clássico vs. U-Net)

A comparação entre a abordagem clássica e a U-Net explicita a superioridade do aprendizado de representações em cenários de baixo contraste.

1. **Painel de Métricas:** O gráfico de barras na Figure 8.31 contrasta o IoU médio de ambos os métodos na validação.
2. **Visualização Qualitativa:** A comparação visual em três amostras demonstra como as conexões de atalho recuperam o contorno do nódulo mesmo sob ruído acentuado.

```
# 1. Gráfico comparativo de barras
metodos_seg = ["Clássico\n(Suavização + Otsu)", "U-Net\n(Aprendizado)"]
ious_medios = [iou_classico_medio, iou_unet_final]

plt.figure(figsize=(4.5, 3.8))
plt.bar(metodos_seg, ious_medios, color=["#f97316", "#16a34a"])
plt.ylim(0, 1.0)
plt.ylabel("IoU Médio (Validação)")
plt.title("Segmentação Clássica vs. U-Net")

for i, v in enumerate(ious_medios):
    plt.text(i, v + 0.02, f"{v:.3f}", ha="center")

plt.tight_layout()
```



```
plt.show()

# 2. Exibição qualitativa lado a lado em 3 amostras via mm.show
imgs_comparativas = []
titulos_comparativos = []

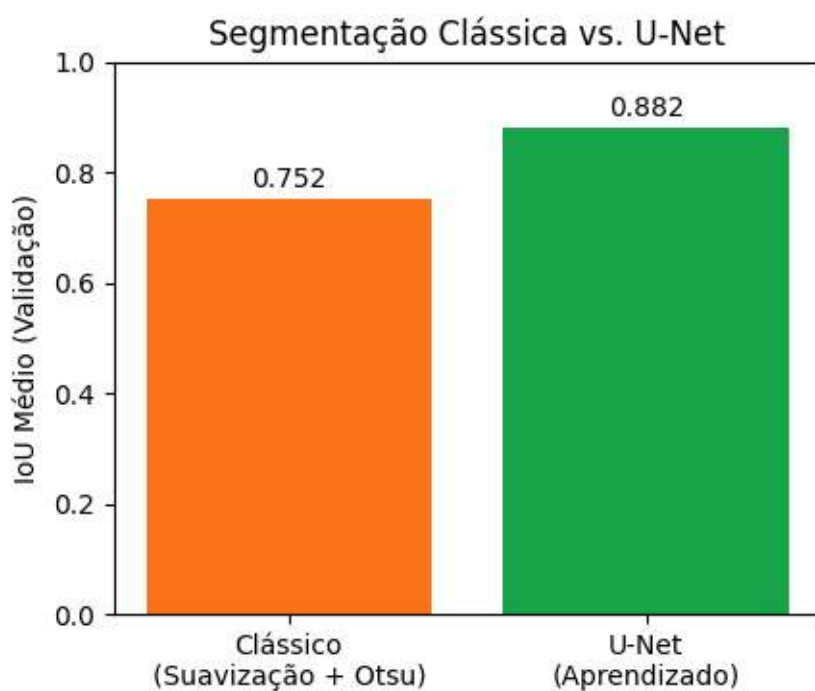
for i in range(3):
    with torch.no_grad():
        pred_unet = (
            (torch.sigmoid(modelo_unet(X_val_unet[i : i + 1]))) > 0.5)
            .float()
            .squeeze()
            .numpy()
            * 255
        )

    pred_classico = segmenta_classico(imgs_val[i], elemento_estrutural)

    imgs_comparativas.extend(
        [imgs_val[i], masks_val[i], pred_classico, pred_unet.astype(np.uint8)]
    )

    t_prefix = f"Amostra {i+1}"
    titulos_comparativos.extend(
        [
            f"{t_prefix}: Imagem",
            f"{t_prefix}: Referência",
            f"{t_prefix}: Clássico",
            f"{t_prefix}: U-Net",
        ]
    )

mm.show(
    imgs_comparativas,
    titles=titulos_comparativos,
    cols=4,
    figsize=(11, 7.5),
)
```



(a) Comparativo quantitativo (IoU médio) e qualitativo entre a abordagem clássica e a U-Net treinada em três amostras de validação.

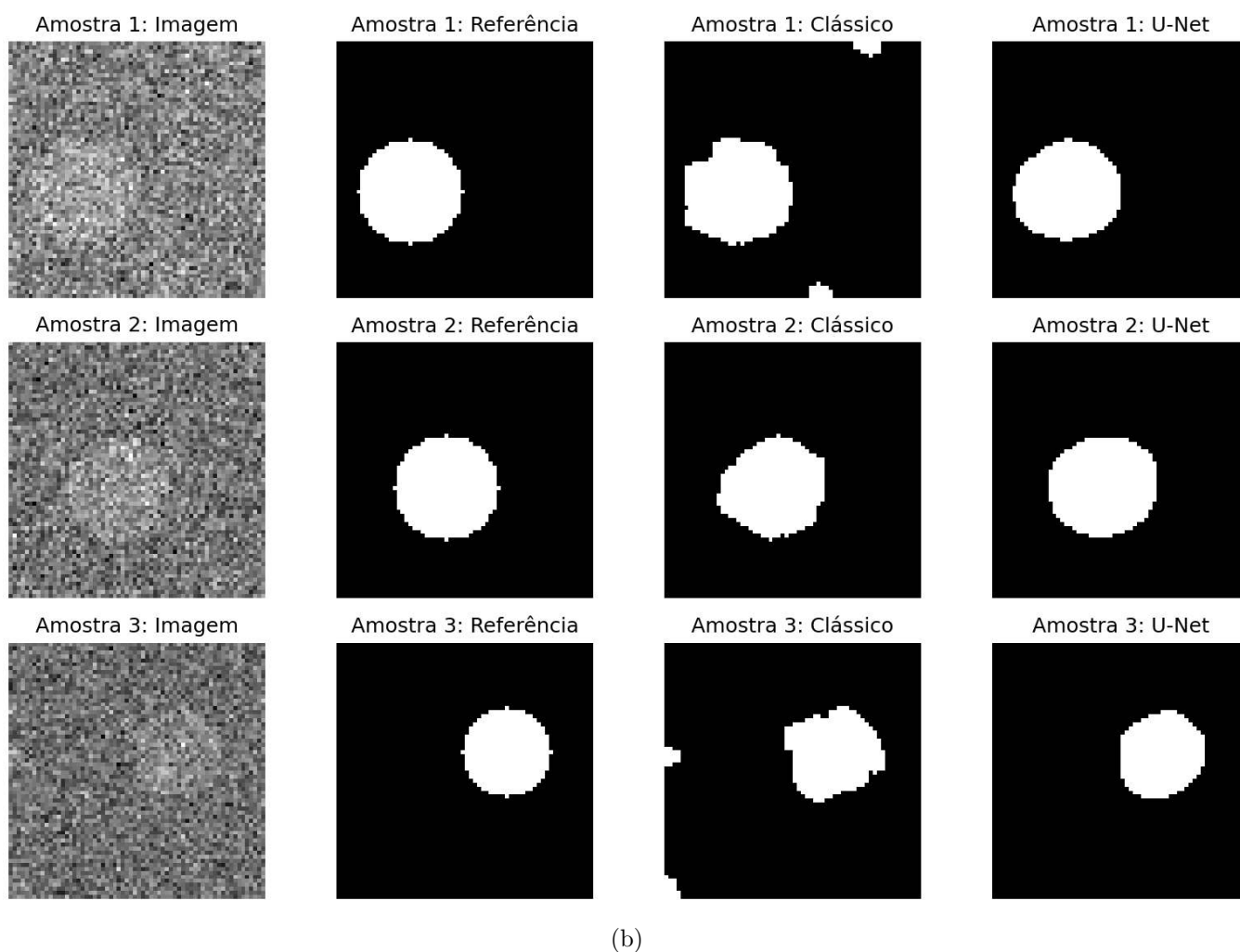


Figura 8.31

Por que a U-Net supera a limiarização fixa?

A limiarização de Otsu aplica um valor de corte global sobre a intensidade local. Quando a diferença de média entre o nódulo e o fundo é pequena diante do ruído gaussiano, essa regra comete erros sistemáticos nas bordas.

A U-Net contorna essa limitação ao combinar o contexto semântico amplo extraído pelo codificador com os detalhes espaciais finos preservados pelas conexões de atalho. Isso permite identificar a presença do nódulo e delimitar seus contornos com precisão, mesmo sob ruído intenso.

8.10.6 Bloco 6: Integração com Datasets Reais via Roboflow

O mesmo *pipeline* de treinamento da U-Net pode ser estendido a imagens reais exportadas por plataformas de anotação como o **Roboflow**. Ao selecionar a opção de exportação **png-mask-semantic**, obtêm-se arquivos de imagem e máscaras de segmentação diretamente compatíveis com a classe `UNetCompacta`.

Dependência de Conexão e Chave de API

As células a seguir exigem autenticação gratuita via API do Roboflow (app.roboflow.com). O código demonstra o carregamento e a formatação das máscaras semânticas para reutilização no laço de treinamento PyTorch.

```
import glob
import os
from roboflow import Roboflow

# resolver isso:
# # 1. Autenticação e download do dataset no formato de máscaras semânticas
# rf = Roboflow(api_key="SUA_CHAVE_DE_API")
# projeto = rf.workspace("nome-do-workspace").project("nome-do-projeto")
# dataset = projeto.version(1).download("png-mask-semantic")

# # 2. Função de carregamento e redimensionamento das amostras reais
# def carrega_split_roboflow(pasta, tam=TAM_IMG):
#     imagens, mascaras = [], []
#     for caminho_mascara in sorted(glob.glob(f"{pasta}/*_mask.png")):
#         caminho_imagem = caminho_mascara.replace("_mask.png", ".jpg")
#         if not os.path.exists(caminho_imagem):
#             continue
#
#         img = cv2.resize(
#             cv2.imread(caminho_imagem, cv2.IMREAD_GRAYSCALE), (tam, tam)
#         )
#         mask = cv2.resize(
#             cv2.imread(caminho_mascara, cv2.IMREAD_GRAYSCALE), (tam, tam)
#         )
#
#         imagens.append(img)
#         mascaras.append((mask > 127).astype(np.uint8) * 255)
#     return imagens, mascaras

# # 3. Formatação dos tensores e treinamento
# imgs_tr_real, masks_tr_real = carrega_split_roboflow(f"{dataset.location}/train")
# X_tr_real, Y_tr_real = para_tensores(imgs_tr_real, masks_tr_real)

# modelo_unet_real = UNetCompacta()
```

```
# otimizador_real = optim.Adam(modelo_unet_real.parameters(), lr=1e-3)

# for epoca in range(epocas_unet):
#     modelo_unet_real.train()
#     perm = torch.randperm(X_tr_real.size(0))
#     for i in range(0, X_tr_real.size(0), tam_lote):
#         idx = perm[i : i + tam_lote]
#         otimizador_real.zero_grad()
#         perda = perda_segmentacao(
#             modelo_unet_real(X_tr_real[idx]), Y_tr_real[idx]
#         )
#         perda.backward()
#         otimizador_real.step()

# print("Treinamento da U-Net sobre o conjunto de dados real concluído.")
```

8.11 Integrando Geometria Computacional e Aprendizado Profundo

O capítulo encerra-se integrando dois pilares da Visão Computacional: a **geometria projetiva** (estudada nos Capítulos 6 e 8) e o **aprendizado profundo**. A combinação dessas abordagens sustenta aplicações práticas no mundo real, como ilustrado a seguir.

8.11.1 Realidade Aumentada com Marcadores e Homografia

Sistemas de Realidade Aumentada (RA) projetam elementos virtuais sobre uma cena real com alinhamento geométrico. O processo utiliza a **homografia** para estimar a transformação de perspectiva entre a posição conhecida de um marcador e sua projeção na imagem.

1. **Detecção do Marcador ArUco:** Identifica-se o marcador na imagem com `cv2.aruco.ArucoDetector`.
2. **Projeção por Homografia:** A matriz de homografia H calculada por `cv2.findHomography` deforma a imagem virtual para sobrepor-la ao marcador na Figure 8.32.

```
from skimage import data as skdata

# 1. Geração do marcador ArUco sintético
dic = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
aruco = cv2.cvtColor(
    cv2.aruco.generateImageMarker(dic, 7, 200), cv2.COLOR_GRAY2BGR
)

src = np.float32([[0, 0], [200, 0], [200, 200], [0, 200]])
dst = np.float32([[150, 120], [430, 90], [460, 350], [140, 380]])

# 2. Simulação da cena com o marcador projetado
cena = np.full((480, 640, 3), 200, np.uint8)
H_cena, _ = cv2.findHomography(src, dst)

mask_cena = cv2.warpPerspective(
    np.full((200, 200), 255, np.uint8), H_cena, (640, 480)
)
cena[mask_cena > 0] = cv2.warpPerspective(aruco, H_cena, (640, 480))[
    mask_cena > 0
]

# 3. Detecção e sobreposição da imagem virtual
det = cv2.aruco.ArucoDetector(dic, cv2.aruco.DetectorParameters())
corners, ids, _ = det.detectMarkers(cena)

virtual = cv2.resize(
```

```

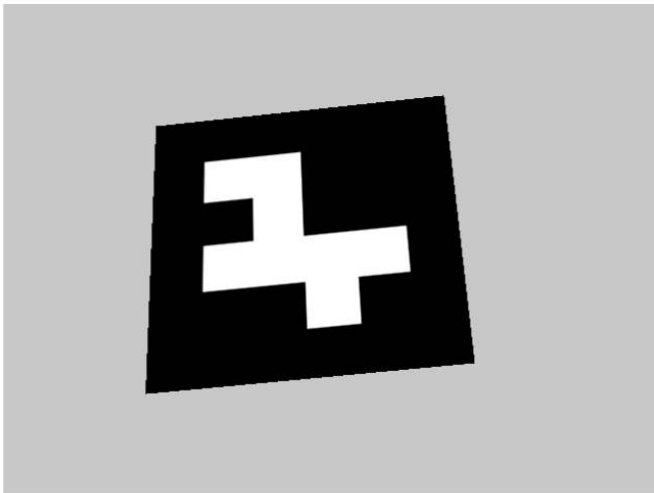
    cv2.cvtColor(skdata.astronaut(), cv2.COLOR_RGB2BGR), (200, 200)
)
H_ra, _ = cv2.findHomography(src, corners[0][0])

ra = cena.copy()
mask_ra = cv2.warpPerspective(
    np.full((200, 200), 255, np.uint8), H_ra, (640, 480)
)
ra[mask_ra > 0] = cv2.warpPerspective(virtual, H_ra, (640, 480))[mask_ra > 0]

# Exibição dos resultados com mm.show
mm.show(
    [cv2.cvtColor(cena, cv2.COLOR_BGR2RGB), cv2.cvtColor(ra, cv2.COLOR_BGR2RGB)],
    titles=["Marcador Detectado na Cena", "Sobreposição Virtual via Homografia"],
    cols=2,
    figsize=(9, 4),
)

```

Marcador Detectado na Cena



Sobreposição Virtual via Homografia

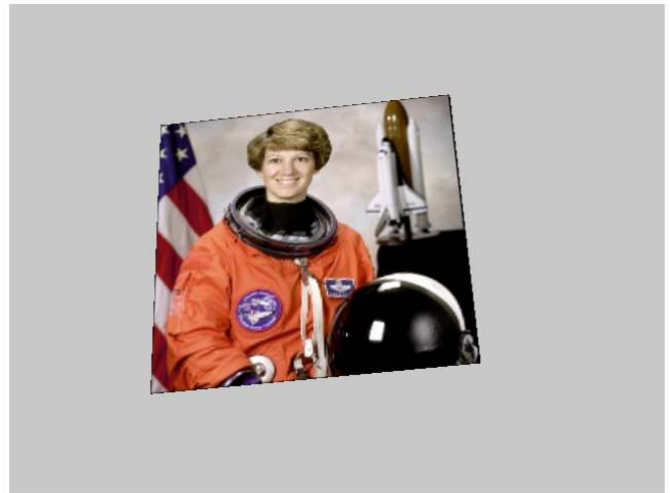


Figura 8.32: Realidade aumentada baseada em marcador ArUco: detecção de canto e sobreposição de imagem virtual por matriz de homografia exibidas via mm.show.

8.11.2 Fotogrametria e Referência de Escala

A **fotogrametria** converte dimensões em *pixels* para medidas físicas (como centímetros) a partir de um objeto de referência de dimensão conhecida presente na cena.

1. **Segmentação e Bounding Box:** Localizam-se o cartão de referência e o objeto alvo na imagem.
2. **Escalação Física:** A razão cm/pixel é calculada a partir da largura do cartão (8,56 cm, padrão ISO/IEC 7810), permitindo medir o objeto em centímetros na Figure 8.33.

```

COR_REFERENCIA = (200, 200, 200) # Cartão de referência (cinza)
COR_OBJETO = (60, 60, 220) # Objeto alvo (vermelho)

# Desenho da cena sintética
cena_medicao = np.full((300, 500, 3), 255, dtype=np.uint8)
cv2.rectangle(
    cena_medicao, (30, 200), (30 + 140, 200 + 88), COR_REFERENCIA, -1
)
cv2.rectangle(cena_medicao, (250, 100), (250 + 220, 100 + 150), COR_OBJETO, -1)

```

```

def caixa_delimitadora_por_cor(imagem_bgr, cor_bgr, tolerancia=40):
    diferenca = np.abs(imagem_bgr.astype(int) - np.array(cor_bgr)).sum(axis=2)
    mascara = (diferenca < tolerancia).astype(np.uint8) * 255
    contornos, _ = cv2.findContours(
        mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
    maior_contorno = max(contornos, key=cv2.contourArea)
    return cv2.boundingRect(maior_contorno)

x_ref, y_ref, w_ref_px, h_ref_px = caixa_delimitadora_por_cor(
    cena_medicao, COR_REFERENCIA
)
x_obj, y_obj, w_obj_px, h_obj_px = caixa_delimitadora_por_cor(
    cena_medicao, COR_OBJETO
)

# Cálculo de escala física
LARGURA_REFERENCIA_CM = 8.56
razao_cm_por_px = LARGURA_REFERENCIA_CM / w_ref_px
largura_obj_cm = w_obj_px * razao_cm_por_px
altura_obj_cm = h_obj_px * razao_cm_por_px

# Desenho das caixas e medições estimadas
resultado = cena_medicao.copy()
cv2.rectangle(
    resultado, (x_ref, y_ref), (x_ref + w_ref_px, y_ref + h_ref_px), (0, 180, 0), 2
)
cv2.rectangle(
    resultado, (x_obj, y_obj), (x_obj + w_obj_px, y_obj + h_obj_px), (0, 180, 0), 2
)

cv2.putText(
    resultado,
    f"{LARGURA_REFERENCIA_CM:.2f} cm",
    (x_ref, y_ref - 8),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.55,
    (0, 120, 0),
    2,
)

cv2.putText(
    resultado,
    f"{largura_obj_cm:.1f} x {altura_obj_cm:.1f} cm",
    (x_obj, y_obj - 8),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0, 120, 0),
    2,
)

mm.show(
    [
        cv2.cvtColor(cena_medicao, cv2.COLOR_BGR2RGB),
        cv2.cvtColor(resultado, cv2.COLOR_BGR2RGB),
    ],
    titles=[
        "Cena Original",
        "Medição por Referência de Escala (Centímetros)",
    ]
)

```



```
],
cols=2,
figsize=(9, 4),
)
```

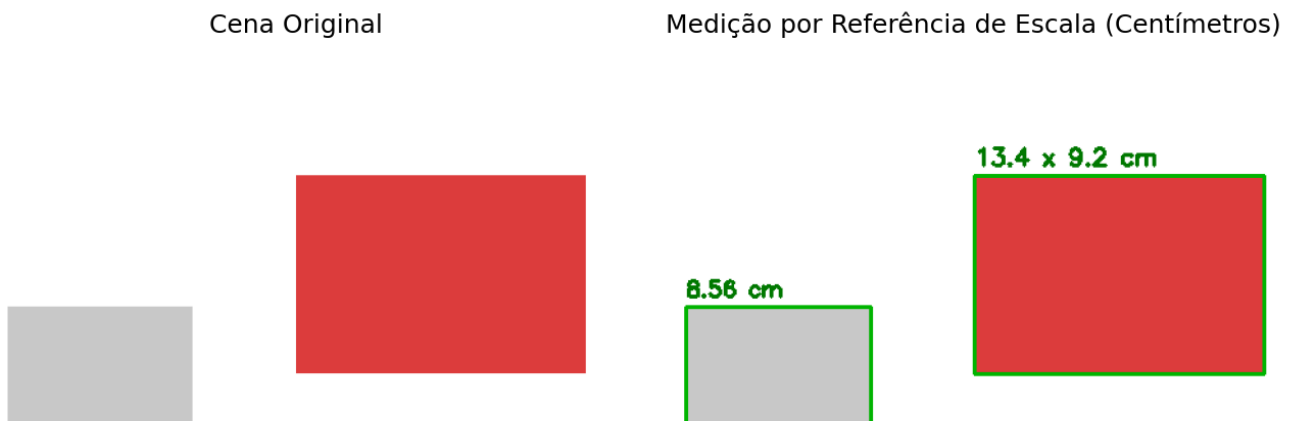


Figura 8.33: Medição de dimensões físicas reais utilizando um cartão de referência de escala conhecida (8,56 cm) renderizada via `mm.show`.

8.11.3 Visão Estereoscópica e Estimativa de Profundidade

A **visão estereoscópica** calcula a profundidade Z a partir do deslocamento horizontal (**disparidade** d) de um mesmo ponto entre duas imagens capturadas por câmeras separadas pela distância B (*baseline*):

$$Z = \frac{f \cdot B}{d}$$

1. **Par Estéreo Sintético:** Cria-se o deslocamento horizontal diferenciado para objetos próximos (`DESLOC_OBJETO`) e distantes (`DESLOC_FUNDO`).
2. **Mapa de Disparidade:** O algoritmo `cv2.StereoBM_create` calcula o mapa de profundidade na Figure 8.34, onde intensidades mais claras indicam maior disparidade (objetos mais próximos).

```
img_base = cv2.cvtColor(skdata.astronaut(), cv2.COLOR_RGB2GRAY)

DESLOC_FUNDO = 4 # Objeto distante
DESLOC_OBJETO = 22 # Objeto próximo

def deslocar(imagem, dx):
    M = np.float32([[1, 0, dx], [0, 1, 0]])
    return cv2.warpAffine(imagem, M, (imagem.shape[1], imagem.shape[0]))

img_esquerda = img_base.copy()
img_direita = deslocar(img_base, -DESLOC_FUNDO)

# Simulação de objeto em primeiro plano
y0, y1, x0, x1 = 180, 340, 200, 360
img_direita[y0:y1, x0:x1] = deslocar(img_base, -DESLOC_OBJETO)[y0:y1, x0:x1]

# Cálculo da disparidade estéreo
correspondencia_estereo = cv2.StereoBM_create(
    numDisparities=32, blockSize=15
)
mapa_disparidade = (
```

```

correspondencia_estereo.compute(img_esquerda, img_direita).astype(
    np.float32
)
/ 16.0
)

disp_fundo = mapa_disparidade[:,150, :150].mean()
disp_objeto = mapa_disparidade[200:320, 220:340].mean()

print(f"Disparidade média no fundo: {disp_fundo:.2f} px")
print(f"Disparidade média no objeto em primeiro plano: {disp_objeto:.2f} px")
print(
    "→ Disparidade maior = objeto mais próximo da câmera, conforme  $Z = f \cdot B/d$ .\n"
)

mm.show(
    [img_esquerda, img_direita, mapa_disparidade],
    titles=["Imagem Esquerda", "Imagem Direita", "Mapa de Disparidade"],
    cols=3,
    figsize=(10, 3.5),
)

```

Disparidade média no fundo: 2.58 px
 Disparidade média no objeto em primeiro plano: 21.63 px
 → Disparidade maior = objeto mais próximo da câmera, conforme $Z = f \cdot B/d$.



Figura 8.34: Par estéreo sintético e mapa de disparidade correspondente: regiões mais claras indicam maior disparidade (objetos mais próximos da câmera) exibidos via `mm.show`.

Considerações Finais

Métodos clássicos de correspondência estéreo comparam blocos de *pixels* diretamente, enfrentando dificuldades em regiões lisas ou sem textura. Redes neurais modernas de **estimativa de profundidade monocular** e **estéreo aprendida** superam essas limitações ao aprender estruturas e geometrias tridimensionais diretamente a partir dos dados, unindo os fundamentos do Aprendizado Profundo e da Geometria Projetiva apresentados ao longo deste livro.

8.12 Resumo

Este capítulo, que encerra a Parte II do livro, apresentou:

- **Convolução e pooling aprendidos:** a mesma operação matemática de convolução do Capítulo 3, mas com *kernels* tratados como parâmetros ajustados por treinamento, em vez de definidos manualmente;
- **Compartilhamento de pesos e hierarquia de características** como as propriedades que tornam as CNNs eficientes e capazes de aprender representações cada vez mais abstratas em camadas sucessivas;
- **Treinamento de uma CNN do zero**, com desempenho comparável — e não necessariamente superior — aos classificadores clássicos do Capítulo 7 em uma base pequena e simples, reforçando que a escolha do método deve ser proporcional à complexidade real do problema;
- **Transferência de aprendizado**, demonstrada experimentalmente como uma estratégia eficaz para tarefas com poucos dados rotulados, reaproveitando um extrator de características já treinado em uma tarefa relacionada — e suas limitações, na forma da transferência negativa entre domínios muito distintos;
- **Aplicações em larga escala** com modelos pré-treinados de classificação, detecção (Faster R-CNN) e segmentação (DeepLabV3), seguindo o mesmo princípio de transferência de aprendizado em escala industrial;
- **Transferência de aprendizado aplicada à detecção de objetos**, ajustando um YOLO pré-treinado na COCO para localizar e classificar objetos geométricos sintéticos, ilustrando o mesmo princípio de congelamento parcial em um domínio de origem e destino ainda mais distantes entre si;
- **Segmentação semântica com U-Net**, implementada e treinada do zero sobre um conjunto sintético de baixo contraste, superando uma linha de base clássica de limiarização graças às conexões de atalho entre codificador e decodificador — e reaproveitada, sem qualquer alteração de arquitetura, sobre um conjunto de dados real obtido via Roboflow;
- A integração de **geometria computacional** (homografia, calibração) e **aprendizado profundo** em três aplicações reais que encerram o livro: **realidade aumentada**, **fotogrametria** e **visão estereoscópica**.

8.13 Uso do Gemini Notebook como Tutor Complementar

Nesta edição, o uso do **Gemini Notebook** é incentivado como ferramenta complementar de aprendizagem. Baseado em inteligência artificial, o sistema utiliza exclusivamente os documentos fornecidos pelo autor como fonte de conhecimento, produzindo respostas alinhadas ao conteúdo e à abordagem adotada ao longo deste capítulo.

 Estude com o Tutor Inteligente

 [ACESSAR Gemini Notebook: CAPÍTULO 09](#)

 **Aviso sobre Conteúdo Gerado por IA**

Embora seja uma ferramenta valiosa de apoio aos estudos, o Gemini Notebook pode eventualmente produzir respostas incompletas, imprecisas ou incorretas. Recomenda-se validar as informações consultando o material do capítulo, livros, artigos científicos e outras fontes acadêmicas confiáveis. Sempre que possível, execute e experimente os exemplos práticos apresentados ao longo do texto para consolidar a compreensão dos conceitos.

8.14 Lista de Exercícios

Os exercícios a seguir consolidam os conceitos apresentados neste capítulo por meio de adaptações, experimentos e extensões dos algoritmos desenvolvidos ao longo do texto, utilizando as bibliotecas **PyTorch**, **ultralytics** e a biblioteca didática **morph**.

1. **(10%)** Investigue o impacto da profundidade em uma arquitetura convolucional. Partindo da rede de duas camadas do Projeto Prático 1, adicione uma terceira camada convolucional com 32 filtros antes das camadas totalmente conectadas. Treine a nova arquitetura mantendo o mesmo número de

- épocas e a mesma divisão dos dados. Compare a acurácia no conjunto de teste e o número total de parâmetros treináveis em relação à rede original, discutindo se o aumento de profundidade trouxe benefício mensurável para imagens de dimensão 8×8 .
2. (15%) Avalie o limiar de dados necessários no domínio de destino para que o treinamento de uma CNN do zero se torne competitivo com a **transferência de aprendizado**. Variando o número de amostras de treino disponíveis no domínio B para $\{5, 10, 20, 40, 80\}$, meça a acurácia de teste para ambas as estratégias. Apresente os resultados em um gráfico de linhas e determine a partir de qual volume de dados o treinamento do zero atinge desempenho equivalente ao extrator pré-treinado.
 3. (15%) Investigue a estratégia de **ajuste fino parcial** (*fine-tuning*) em comparação ao congelamento total de pesos. No cenário de transferência de aprendizado entre domínios de dígitos, descongele a segunda camada convolucional (**conv2**) do extrator para que ela seja atualizada juntamente com a cabeça de classificação durante o treinamento no domínio B. Compare a acurácia obtida com o congelamento total e com o treinamento do zero, discutindo o compromisso entre capacidade de adaptação e risco de sobreajuste (*overfitting*).
 4. (15%) Analise a relação inversa entre **disparidade e profundidade em visão estereoscópica**. Utilizando a equação $Z = (f \cdot B)/d$ com distância focal $f = 700$ pixels e linha de base $B = 6$ cm, calcule a profundidade estimada para o fundo e para o objeto em primeiro plano no experimento do capítulo. Em seguida, gere sinteticamente um terceiro objeto com um deslocamento intermediário de 12 pixels, recalcule sua profundidade e construa um gráfico mostrando a curva teórica $Z(d)$ com os três pontos experimentais destacados.
 5. (15%) Avalie a influência da profundidade do congelamento (*freeze*) no desempenho de detectores **YOLO** submetidos à transferência de aprendizado. Utilizando o conjunto de dados sintético de formas geométricas, execute o ajuste fino variando o parâmetro de congelamento do *backbone* para $\{0, 5, 10, 15\}$. Registre a métrica mAP_{50} no conjunto de validação para cada configuração, apresente os dados em uma tabela e discuta se o congelamento parcial é vantajoso quando os domínios de origem (COCO) e de destino (formas geométricas) são significativamente distantes.
 6. (15%) Estude a importância das **conexões de atalho** (*skip connections*) na arquitetura **U-Net** para segmentação semântica. Implemente uma variação **UNetSemAtalhos** que funcione como um *autoencoder* convolucional tradicional, removendo as concatenações entre os estágios do codificador e do decodificador. Treine ambos os modelos sobre a mesma base de nódulos sintéticos, compare o IoU médio no conjunto de validação e apresente visualmente a diferença na precisão das bordas segmentadas por cada método.
 7. (15%) Integre um *pipeline* completo de segmentação semântica utilizando dados do repositório público **Roboflow**. Baixe um conjunto de dados anotado no formato de máscaras semânticas, converta os pares de imagens e máscaras para os tensores do PyTorch e treine a **UNetCompacta**. Compare o IoU médio obtido pelo modelo profundo contra uma linha de base clássica baseada em limiarização de Otsu e filtros morfológicos da biblioteca **morph**, discutindo em quais cenários reais o aprendizado profundo justifica o custo computacional do treinamento.
 8. (Bônus – 10%) Desenvolva um sistema interativo que combine **detecção de objetos (YOLO)** com **medição por referência de escala (fotogrametria)**. Treine o detector para identificar duas classes em uma cena: um “Cartão de Referência” (dimensão conhecida de $8,56 \text{ cm} \times 5,39 \text{ cm}$) e um “Objeto Alvo”. Ao realizar a inferência em uma nova imagem, utilize a dimensão em pixels da *bounding box* do cartão detectado para converter as dimensões da caixa do objeto alvo em centímetros. Exiba a imagem processada com os rótulos de classe, a probabilidade de confiança e as dimensões físicas estimadas sobrepostas.

8.15 Encerramento da Parte II

Este capítulo conclui a Parte II do livro — e, com ela, a jornada que começou no Capítulo 6 com a formalização de “detectar, descrever e corresponder” características em imagens. Ao longo do caminho, um mesmo tema conceitual se repetiu sob formas cada vez mais sofisticadas: a tensão entre **características projetadas manualmente** (Sobel, LBP, HOG, ORB, Haar Cascade) e **características aprendidas automaticamente** (as CNNs deste capítulo) — e a constatação experimental, repetida em três capítulos consecutivos, de que

nenhuma das duas abordagens é universalmente superior: a escolha correta depende da complexidade real do problema, da quantidade de dados disponível e das restrições computacionais da aplicação.

Para o leitor interessado em aprofundar os tópicos aqui apresentados apenas em suas ideias centrais, algumas direções naturais de estudo incluem:

- **Arquiteturas modernas de CNN** (ResNet, EfficientNet, Vision Transformers) e as inovações que permitiram treinar redes cada vez mais profundas;
- **Deteção e segmentação em tempo real**, com foco nas famílias YOLO e nos modelos de segmentação promptable como o Segment Anything;
- **Reconstrução 3D e SLAM** (*Simultaneous Localization and Mapping*), que estendem a visão estereoscópica deste capítulo a cenários dinâmicos, com câmeras em movimento;
- **Modelos generativos** para imagens (GANs, modelos de difusão), capazes de sintetizar, e não apenas interpretar, conteúdo visual.

Os fundamentos consolidados nesta Parte II — desde a extração de características mais simples até as CNNs — são a base sobre a qual todas essas direções mais avançadas se apoiam.

Referências do Capítulo

Os conceitos e algoritmos apresentados neste capítulo foram fundamentados em referências clássicas e contemporâneas da literatura de *Deep Learning* aplicado à Visão Computacional: - McCulloch; Pitts (1943), para a proposição da primeira abstração matemática e lógica do neurônio artificial, fundamentando as bases conceituais do processamento neural computacional. - Rosenblatt (1958), para a formulação original do **Perceptron**, modelo precursor do neurônio artificial utilizado nas arquiteturas modernas de aprendizado profundo. - Goodfellow; Bengio; Courville (2016) e LeCun; Bengio; Hinton (2015), para os fundamentos de redes neurais, convolução, funções de ativação e treinamento de modelos profundos. - Bishop (2006), para os conceitos rigorosos de reconhecimento de padrões, probabilidade, estimativa de máxima verossimilhança e métodos estatísticos aplicados ao aprendizado de máquina. - Ronneberger; Fischer; Brox (2015) para a arquitetura **U-Net**, utilizada na segmentação semântica com conexões de atalho entre codificador e decodificador. - Redmon et al. (2016), para a arquitetura **YOLO** (*You Only Look Once*), utilizada nos experimentos de detecção de objetos com transferência de aprendizado. - Ren et al. (2015), para a arquitetura **Faster R-CNN**, empregada como modelo pré-treinado de detecção de objetos. - He et al. (2016), para a arquitetura **ResNet**, base de diversos extratores de características pré-treinados utilizados neste capítulo. - Chen et al. (2018), para a arquitetura **DeepLabV3**, utilizada como modelo pré-treinado de segmentação semântica. - Kirillov et al. (2023), para o modelo **Segment Anything (SAM)**, mencionado como direção de estudo para segmentação promptable. - Google (2025), referente à ferramenta **Gemini Notebook**, utilizada na elaboração do infográfico de síntese do capítulo e disponibilizada como apoio complementar ao estudo.



8.16 Parte Prática com Exercícios de Programação

Em construção!

A presente lista de exercícios de programação (EP) consolida as formulações teóricas apresentadas ao longo do Capítulo 9 — *Deep Learning* para Visão Computacional — por meio de uma trilha prática aplicada. Diferentemente do treinamento de redes neurais completas com PyTorch, que exige tempo de execução e, por vezes, GPU, os EPs deste capítulo isolam as **grandezas intermediárias** de um *pipeline* real de aprendizado profundo — a saída de uma única camada convolucional, o resultado de uma operação de *pooling*, a contagem de parâmetros treináveis de uma arquitetura, a sobreposição entre caixas delimitadoras candidatas e o filtro de supressão de não-máximos — permitindo validar manualmente cada etapa do raciocínio sem depender de bibliotecas de aprendizado de máquina nem de treinamento real.

O encadeamento dos exercícios reproduz o fluxo conceitual do capítulo: inicia-se com o cálculo manual da saída de uma **camada convolucional aprendida**, a partir de um *kernel* e um viés já treinados; avança-se para a operação de **pooling** (máximo e média), que reduz a resolução espacial entre blocos convolucionais; prossegue-se com a **contagem de parâmetros treináveis** de uma arquitetura CNN completa, evidenciando por que o compartilhamento de pesos torna essas redes tão mais econômicas que uma camada totalmente conectada equivalente; aprofunda-se no cálculo de **Interseção sobre União (IoU)** e na **Supressão de Não-Máximos (NMS)**, etapa de pós-processamento comum a detectores como o Faster R-CNN e o YOLO; e conclui-se com um **pipeline integrado**, unindo a saída de um detector de objetos (após NMS) a uma medição do mundo real por referência de escala — o mesmo princípio da fotogrametria estudada na integração final do capítulo.

! Diretrizes para a Resolução dos Exercícios de Programação

Em todos os exercícios deste capítulo, as etapas de discretização ou arredondamento numérico devem empregar o arredondamento padrão para o inteiro mais próximo (*round half away from zero*), mitigando ambiguidades em valores com fração exatamente igual a 0,5. Salvo indicação explícita em contrário: (i) a operação de “convolução” segue a convenção adotada pelos *frameworks* de aprendizado profundo — **correlação cruzada**, sem inversão espacial do *kernel*, exatamente como apresentado na Seção “Convolução como Camada Aprendida”; (ii) o preenchimento (*padding*) é feito com zeros; (iii) caixas delimitadoras são especificadas no formato canto-a-canto (x_1, y_1, x_2, y_2) , com $x_1 < x_2$ e $y_1 < y_2$; e (iv) vetores/matrizes seguem indexação a partir de 0, com a convenção [linha] [coluna] para estruturas bidimensionais.

🎯 Objetivo deste Caderno

O caderno permite desenvolver, validar, organizar e testar soluções de **Exercícios de Programação (EPs)** em ambientes interativos, como o Colab, com os mesmos casos de teste do Moodle, copiando para lá apenas na hora de registrar a nota oficial.

Download

Baixe `morph.py` e `testsuite.py` executando a célula abaixo:

```
import os, sys, importlib, inspect, urllib.request

# URLs do repositório
BASE_URL = "https://raw.githubusercontent.com/fzampirolli/pdi-vc/master/morph"
for f in ["morph.py", "testsuite.py"]:
    if not os.path.exists(f):
        urllib.request.urlretrieve(f"{BASE_URL}/{f}", f)

import morph, testsuite
importlib.reload(morph); importlib.reload(testsuite)
from morph import mm
from testsuite import TestSuite

print(f" Ambiente pronto. Morph: {morph.__version__} | TestSuite: {testsuite.__version__}")
```

Ambiente pronto. Morph: 1.1.7 | TestSuite: 1.1.2

Executando os Testes

Para avaliar os testes, execute `TestSuite("EP09_01.extensão").run()` numa nova célula, trocando a extensão pela da linguagem usada (`.py`, `.java`, `.c`, `.cpp`, `.js` ou `.r`). O sistema baixa os casos de teste do GitHub, executa o programa e calcula a nota automaticamente.

Para testar código Python diretamente, sem salvar arquivo, use `run_code(codigo)` passando o código como *string* numa variável `codigo`:

```
codigo = """
# ... seu código aqui ...
"""
TestSuite("EP09_01").run_code(codigo)
```

8.16.1 EP09_01 Convolução 2D Manual (Forward de uma Camada Aprendida)

O PyTorch, apresentado neste capítulo, executa `nn.Conv2d(x)` em uma única chamada — mas por trás dessa chamada está apenas a operação de correlação cruzada entre um *kernel* (já treinado) e uma vizinhança da entrada, seguida da soma de um viés e de uma ativação, exatamente como formalizado na Seção “Convolução como Camada Aprendida”. A diferença essencial em relação à convolução fixa do Capítulo 3 é que, aqui, os valores do *kernel* e do viés **já vêm prontos** (como se tivessem sido aprendidos por gradiente), e cabe a você reproduzir manualmente a passagem direta (*forward pass*) que o *framework* executa internamente.

Antes de treinar uma CNN de verdade, você foi encarregado de implementar essa passagem direta do zero, para uma única camada convolucional com um único canal de entrada e um único filtro de saída, incluindo suporte a *padding* e *stride* arbitrários.

8.16.1.1 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ do mapa de características de entrada e , em seguida, seus $H \times W$ valores reais.
2. **Kernel e viés:** Ler as dimensões $k_h \times k_w$ do *kernel* (já treinado), seus valores reais, e o viés b (real, escalar).
3. **Hiperparâmetros:** Ler o *padding* p (inteiro, número de zeros adicionados em cada borda) e o *stride* s (inteiro, passo do deslizamento).
4. **Preenchimento:** Adicionar p zeros em cada uma das quatro bordas do mapa de entrada antes da correlação.
5. **Correlação cruzada:** Para cada posição de saída (i, j) , calcular

$$z(i, j) = b + \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v) \cdot X_{pad}(i \cdot s + u, j \cdot s + v),$$

percorrendo a entrada **sem** inverter o *kernel* (convenção dos *frameworks* de aprendizado profundo, diferente da convolução matemática clássica).

6. **Ativação:** Aplicar ReLU a cada valor: $a(i, j) = \max(0, z(i, j))$.
7. **Dimensões de saída:** $O_h = \lfloor (H + 2p - k_h)/s \rfloor + 1$ e $O_w = \lfloor (W + 2p - k_w)/s \rfloor + 1$.
8. **Saída:** Imprimir O_h e O_w na primeira linha, seguidos de O_h linhas com O_w valores reais cada (o mapa de características de saída, já com ReLU aplicada), formatados com 4 casas decimais.

8.16.1.2 Restrições Computacionais

- **Um canal de entrada, um filtro de saída:** não é necessário lidar com múltiplos canais ou múltiplos filtros nesta versão simplificada.
- **Sem inversão do *kernel*:** implemente correlação cruzada, não a convolução matemática clássica com *kernel* invertido — é essa a operação que o PyTorch (e a maioria dos *frameworks*) chama de “convolução”.
- **Preenchimento por zeros:** os p pixels adicionados em cada borda valem sempre 0.
- **Formatação:** todos os valores de saída devem ter exatamente 4 casas decimais, mesmo quando o valor é inteiro (ex.: 2.0000).

8.16.1.3 Fundamentação Teórica

Elemento	Papel na camada convolucional
Kernel K	Parâmetros aprendidos por gradiente, análogos aos filtros de Sobel do Capítulo 3, mas ajustados a partir de dados
Viés b	Parâmetro aprendido, somado à saída de cada janela
Padding p	Controla a dimensão espacial da saída; $p = 0$ (“ <i>valid</i> ”) reduz a resolução, p maior preserva-a
Stride s	Passo do deslizamento; $s > 1$ sub-amostra a saída, reduzindo custo computacional
ReLU	Ativação não-linear aplicada após cada convolução, sem a qual camadas empilhadas colapsariam em uma única transformação linear

8.16.1.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros H e W .
- Próximas H linhas: W valores reais cada (mapa de entrada).
- Próxima linha: Inteiros k_h e k_w .
- Próximas k_h linhas: k_w valores reais cada (*kernel*).
- Próxima linha: Um valor real b (viés).
- Próxima linha: Inteiros p e s (*padding* e *stride*).

Saída:

- Linha 1: Inteiros O_h e O_w .
- Próximas O_h linhas: O_w valores reais cada, formatados com 4 casas decimais.

8.16.1.5 Exemplos

Entrada	Saída	Observação
3 3 1 2 0 0 1 2 1 0 1 2 2 1 1 1 1 -2 0 1 3 3 1 2 0 0 1 2 1 0 1 2 2 1 0 0 1 0 1 2	2 2 2.0000 3.0000 0.0000 2.0000 2 2 1.0000 0.0000 1.0000 2.0000	<i>Padding</i> 0, <i>stride</i> 1: saída 2×2 sem preenchimento. <i>Padding</i> 1, <i>stride</i> 2: entrada preenchida com zeros antes da correlação.

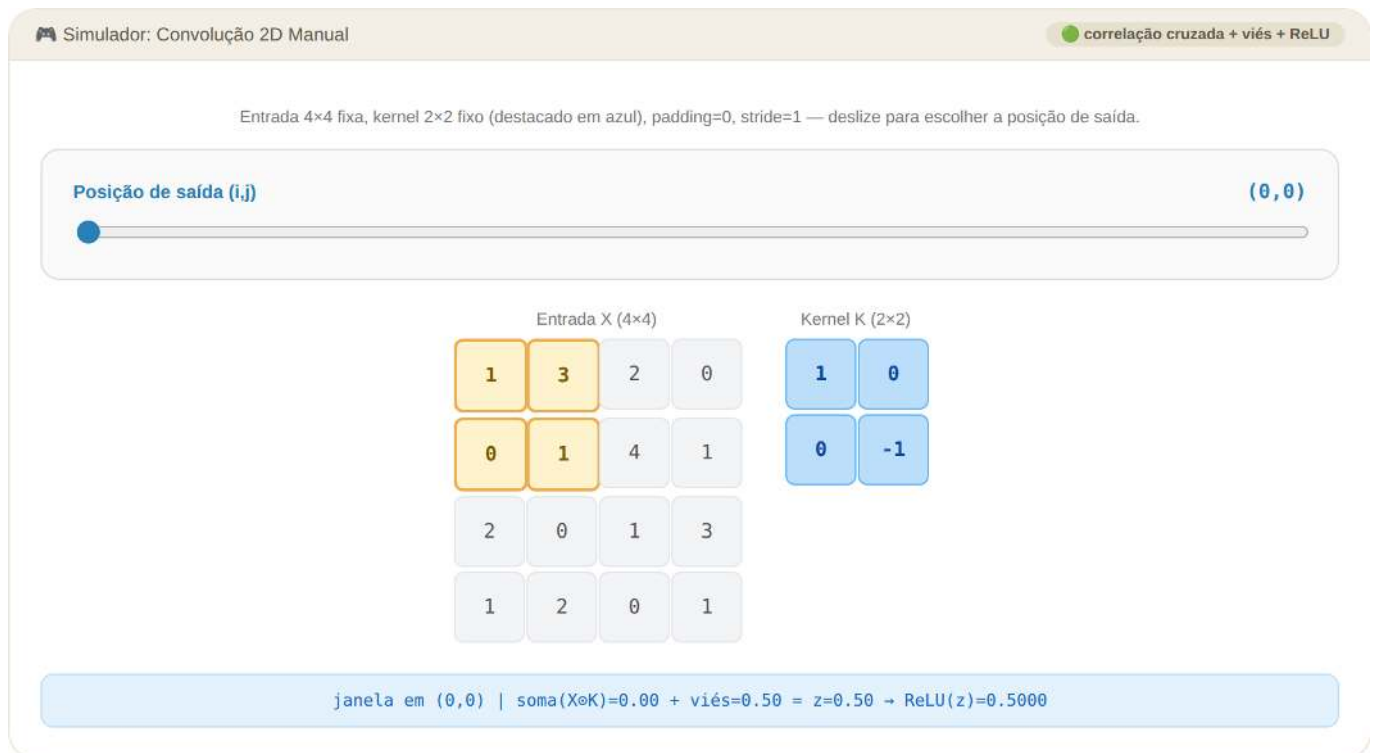


Figura 8.35: Simulador: Convolução 2D Manual (correlação cruzada + viés + ReLU)

```
%%writefile EP09_01.py
# Código Python
```

Writing EP09_01.py

```
TestSuite("EP09_01.py").run()
```

8.16.2 EP09_02 ● *Pooling* Manual (Máximo e Média)

Entre blocos convolucionais, a arquitetura típica de uma CNN intercala camadas de *pooling*, que reduzem a resolução espacial do mapa de características sem introduzir novos parâmetros treináveis — ao contrário da convolução, o *pooling* não tem pesos: ele apenas resume cada janela da entrada a um único valor, por um máximo ou por uma média.

Você foi encarregado de implementar essa operação a partir de uma janela deslizante quadrada, sem sobreposição parcial nas bordas (apenas janelas completas), suportando os dois tipos mais comuns: **max** (preserva o valor mais saliente, tipicamente usado para reter bordas e texturas fortes) e **avg** (suaviza a região, preservando informação de intensidade média).

8.16.2.1 📋 Diretrizes de Implementação

1. **Entrada:** Ler as dimensões $H \times W$ do mapa de características de entrada e seus $H \times W$ valores reais.
2. **Janela:** Ler os inteiros k (tamanho da janela quadrada $k \times k$) e s (*stride*).
3. **Tipo:** Ler uma *string*, **max** ou **avg**, indicando o tipo de *pooling*.
4. **Sem preenchimento:** Esta operação **não** utiliza *padding*; janelas que ultrapassariam a borda da entrada são descartadas.
5. **Cálculo:** Para cada posição de saída (i, j) , calcular o máximo ou a média dos $k \times k$ valores da janela correspondente, iniciando em $(i \cdot s, j \cdot s)$.
6. **Dimensões de saída:** $O_h = \lfloor (H - k) / s \rfloor + 1$ e $O_w = \lfloor (W - k) / s \rfloor + 1$.

7. **Saída:** Imprimir O_h e O_w na primeira linha, seguidos de O_h linhas com O_w valores reais cada, formatados com 4 casas decimais.

8.16.2.2 Restrições Computacionais

- **Janela quadrada:** $k \times k$, sem suporte a janelas retangulares nesta versão.
- **Sem *padding*:** apenas janelas inteiramente contidas na entrada são consideradas.
- **avg usa divisão real:** a média é sempre soma/ k^2 , mesmo quando o resultado tem muitas casas decimais — arredonde apenas na formatação final, conforme a diretriz geral do capítulo.
- **Formatação:** todos os valores de saída com exatamente 4 casas decimais.

8.16.2.3 Fundamentação Teórica

Elemento	Papel na arquitetura
<i>Pooling</i> máximo	Preserva a ativação mais forte da janela; comum após camadas convolucionais para reter bordas e texturas salientes
<i>Pooling</i> médio	Suaviza a região, preservando a intensidade média; comum em camadas finais (<i>global average pooling</i>)
Ausência de parâmetros	Diferencia o <i>pooling</i> da convolução: reduz resolução espacial sem custo adicional de treinamento
Redução de resolução	Contribui para a invariância a pequenas translações e para a redução do custo computacional das camadas seguintes

8.16.2.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiros H e W .
- Próximas H linhas: W valores reais cada.
- Próxima linha: Inteiros k e s .
- Próxima linha: **max** ou **avg**.

Saída:

- Linha 1: Inteiros O_h e O_w .
- Próximas O_h linhas: O_w valores reais cada, formatados com 4 casas decimais.

8.16.2.5 Exemplos

Entrada	Saída	Observação
4 4	2 2	<i>Pooling</i> máximo, janela 2×2 , <i>stride</i> 2.
1 3 2 4	6.0000 4.0000	
5 6 1 2	4.0000 5.0000	
2 1 0 3		
4 2 5 1		
2 2		
max		

Entrada	Saída	Observação
4 4	2 2	<i>Pooling</i> médio sobre as mesmas janelas.
1 3 2 4	3.7500 2.2500	
5 6 1 2	2.2500 2.2500	
2 1 0 3		
4 2 5 1		
2 2		
avg		

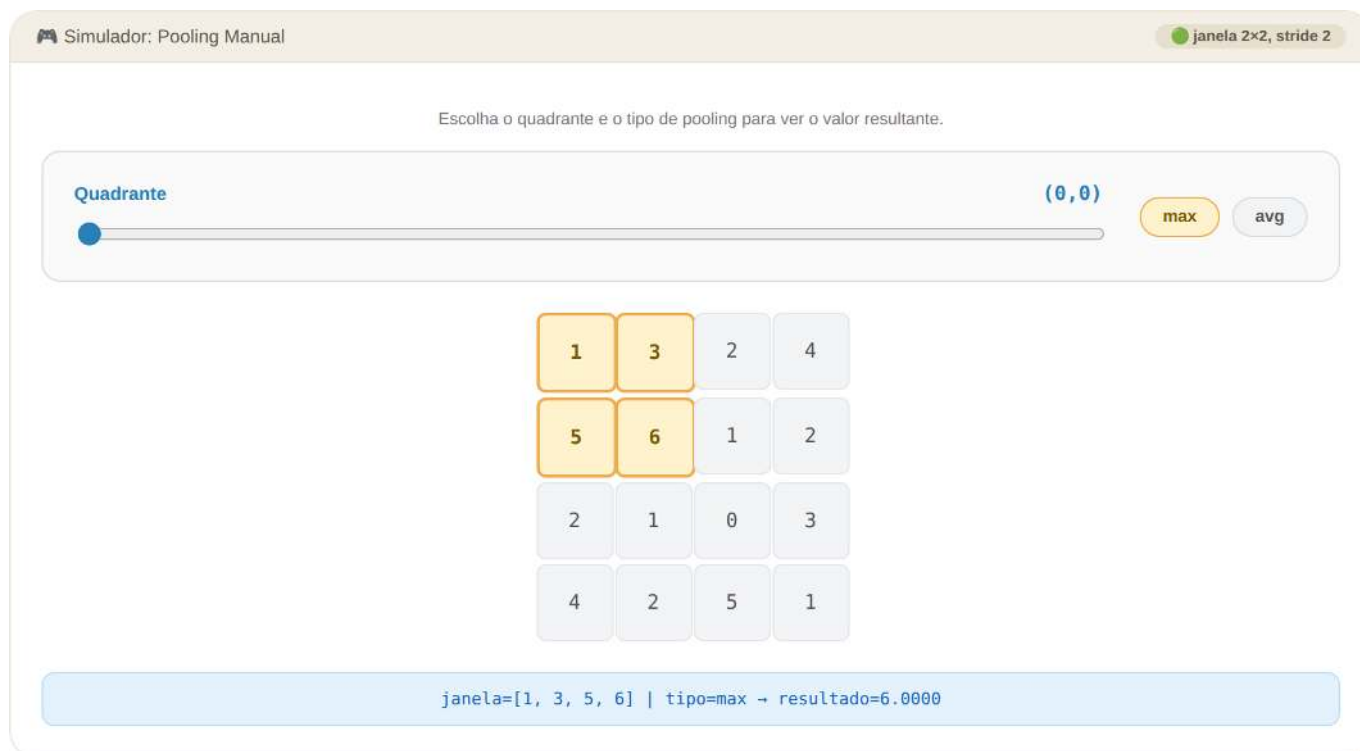


Figura 8.36: Simulador: Pooling Manual (máximo vs. média)

```
%%writefile EP09_02.py
# Código Python
```

Writing EP09_02.py

```
TestSuite("EP09_02.py").run()
```

8.16.3 EP09_03 ● Contagem de Parâmetros Treináveis de uma CNN

O Exercício 1 deste capítulo pediu para adicionar uma terceira camada convolucional à CNN do Projeto Prático 1 e comparar o número de parâmetros da rede resultante. Este EP formaliza esse cálculo: dada a descrição textual de uma arquitetura — uma sequência de camadas convolucionais, de *pooling* e totalmente conectadas — determine, para cada camada, o número de parâmetros treináveis e o total da rede.

O ponto central deste exercício é notar que a contagem de parâmetros de uma camada convolucional depende **apenas** do tamanho do *kernel* e do número de canais de entrada/saída — **nunca** das dimensões espaciais ($H \times W$) do mapa de características, graças ao **compartilhamento de pesos**: o mesmo *kernel* desliza por toda a imagem, seja ela 28×28 ou 280×280 . É exatamente esse compartilhamento que torna as CNNs muito mais econômicas em parâmetros do que uma camada totalmente conectada equivalente, na qual cada pixel de entrada tem uma conexão (e um peso) independente para cada neurônio de saída.

8.16.3.1 Diretrizes de Implementação

1. **Entrada:** Ler o inteiro L (número de camadas da arquitetura, na ordem em que são aplicadas).
2. **Camadas:** Ler L linhas, cada uma descrevendo uma camada em um dos três formatos:
 - CONV $kh\ kw\ cin\ cout\ bias$ — camada convolucional com *kernel* $k_h \times k_w$, cin canais de entrada, $cout$ canais (filtros) de saída, e $bias$ (0 ou 1) indicando se há viés por filtro;
 - POOL — camada de *pooling* (máximo ou médio; não possui parâmetros treináveis);
 - FC $in\ out\ bias$ — camada totalmente conectada com in entradas, out saídas, e $bias$ (0 ou 1) indicando se há viés por neurônio.
3. **Parâmetros de uma camada CONV:** $k_h \cdot k_w \cdot c_{in} \cdot c_{out} + c_{out} \cdot bias$.
4. **Parâmetros de uma camada FC:** $in \cdot out + out \cdot bias$.
5. **Parâmetros de uma camada POOL:** sempre 0.
6. **Saída:** Para cada camada, na ordem de leitura, imprimir **Camada i:** P, onde i começa em 1 e P é o número de parâmetros daquela camada. Ao final, imprimir **Total:** T, com T igual à soma de parâmetros de todas as camadas.

8.16.3.2 Restrições Computacionais

- **Independência da dimensão espacial:** a entrada **não** informa $H \times W$ — a contagem de uma camada CONV não depende disso, apenas de $kh\ kw\ cin\ cout$.
- **bias sempre 0 ou 1:** multiplique diretamente o termo de viés por esse valor, sem tratamento condicional especial.
- **Camadas POOL sem argumentos adicionais:** a linha contém apenas a palavra POOL.
- Todos os valores de entrada e saída são inteiros não-negativos.

8.16.3.3 Fundamentação Teórica

Elemento	Papel na contagem de parâmetros
Compartilhamento de pesos	O mesmo <i>kernel</i> $k_h \times k_w$ é reutilizado em toda a extensão espacial da entrada — por isso a contagem independe de $H \times W$
Canais de entrada/saída	Cada um dos $cout$ filtros possui um conjunto de pesos por canal de entrada, daí o fator $c_{in} \cdot c_{out}$
Viés	Um único escalar por filtro (CONV) ou por neurônio (FC), independente do tamanho do <i>kernel</i> ou da entrada
Camada FC	Cada uma das in entradas conecta-se a cada uma das out saídas — sem compartilhamento, o que explica seu custo tipicamente muito maior em número de parâmetros

8.16.3.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro L .
- Próximas L linhas: descrição de cada camada, no formato CONV $kh\ kw\ cin\ cout\ bias$, POOL, ou FC $in\ out\ bias$.

Saída:

- L linhas no formato **Camada i:** P.
- Última linha: **Total:** T.

8.16.3.5 📌 Exemplos

Entrada	Saída	Observação
3	Camada 1: 80	Rede com uma convolução, um pooling e uma camada final.
CONV 3 3 1 8 1	Camada 2: 0	
POOL	Camada 3: 13530	
FC 1352 10 1	Total: 13610	
4	Camada 1: 80	Rede com duas convoluções empilhadas, como no Projeto Prático 1; a última camada não usa viés.
CONV 3 3 1 8 1	Camada 2: 1168	
CONV 3 3 8 16 1	Camada 3: 0	
POOL	Camada 4: 2048	
FC 64 32 0	Total: 3296	

compartilhamento de pesos

Ajuste o kernel e os canais de uma camada CONV e compare com uma camada FC equivalente, para uma entrada hipotética de 32×32 pixels.

kernel (k×k)

3×3

canais de entrada

1

canais de saída (filtros)

8

☒ usar viés

CONV: $3 \times 3 \times 1 \times 8 + 8 \times 1 = 80$ parâmetros

FC equivalente (entrada 32×32×1 → 8 saídas): 8.200 parâmetros

↳ a camada FC teria 102.5× mais parâmetros que a CONV equivalente, apesar de "ver" a mesma entrada.

Figura 8.37: Simulador: Contagem de Parâmetros — Convolução vs. Camada Totalmente Conectada

```
%%writefile EP09_03.py
# Código Python
```

Writing EP09_03.py

```
TestSuite("EP09_03.py").run()
```

8.16.4 EP09_04 🟡 Interseção sobre União (IoU) e Supressão de Não-Máximos (NMS)

Tanto o Faster R-CNN quanto o YOLO, apresentados na seção “Aplicações em Larga Escala” e no Projeto Prático 2 deste capítulo, produzem internamente **muito mais** caixas delimitadoras candidatas do que objetos realmente existem na imagem — várias delas cobrindo, com pequenas variações, o mesmo objeto. A etapa de pós-processamento responsável por eliminar essas redundâncias é a **Supressão de Não-Máximos (NMS)**, e sua peça fundamental é a métrica de **Interseção sobre União (IoU)**, que mede o quanto duas caixas se sobrepõem — a mesma métrica usada, por exemplo, para definir o limiar de acerto do mAP50 calculado no Projeto Prático 2 ($\text{IoU} \geq 0,5$ com a caixa verdadeira).

Você foi encarregado de implementar o algoritmo de NMS do zero: dado um conjunto de caixas candidatas com suas pontuações de confiança, filtrar as redundantes e manter apenas as detecções mais confiáveis e suficientemente distintas entre si.

8.16.4.1 Diretrizes de Implementação

1. **Entrada:** Ler o inteiro N (número de caixas candidatas) e o limiar real τ (limiar de IoU para supressão).
2. **Caixas:** Ler N linhas, cada uma com cinco valores reais: $x_1, y_1, x_2, y_2, \text{score}$ (canto superior-esquerdo, canto inferior-direito, pontuação de confiança).
3. **Interseção sobre União:** Para duas caixas A e B ,

$$\text{IoU}(A, B) = \frac{\text{área}(A \cap B)}{\text{área}(A) + \text{área}(B) - \text{área}(A \cap B)},$$

onde a área de interseção é calculada pela sobreposição dos intervalos em x e em y (zero se não houver sobreposição).

4. **Algoritmo guloso de NMS:**
 - a. Ordene as caixas por **score** decrescente (em caso de empate, mantenha a ordem de leitura original — ordenação estável).
 - b. Selecione a caixa de maior pontuação entre as restantes; adicione-a ao conjunto de saída e remova-a da lista.
 - c. Calcule o IoU dessa caixa selecionada com **todas** as caixas ainda restantes; remova (suprima) qualquer caixa com $\text{IoU} > \tau$.
 - d. Repita os passos (b)–(c) até que não restem caixas.
5. **Saída:** Para cada caixa mantida, na ordem em que foi selecionada, imprimir seu índice original (posição de leitura, começando em 0) e seu **score**, formatado com 4 casas decimais. Ao final, imprimir **Total mantidas: X**.

8.16.4.2 Restrições Computacionais

- **Supressão estrita:** apenas caixas com $\text{IoU} > \tau$ são suprimidas; caixas com IoU exatamente igual a τ são **mantidas**.
- **Índices originais:** a saída referencia a posição em que cada caixa foi lida na entrada (começando em 0), não sua posição após a ordenação.
- **Retângulos alinhados aos eixos:** todas as caixas são especificadas por dois cantos, com $x_1 < x_2$ e $y_1 < y_2$ garantidos na entrada.

8.16.4.3 Fundamentação Teórica

Elemento	Papel no pós-processamento de detecção
IoU	Quantifica a sobreposição espacial entre duas caixas; $\text{IoU} = 1$ para caixas idênticas, $\text{IoU} = 0$ para caixas disjuntas
Limiar τ	Controla a agressividade da supressão: τ baixo elimina até detecções pouco sobrepostas; τ alto preserva quase todas
Ordenação por confiança	Garante que, entre caixas redundantes, sempre sobrevive a de maior pontuação
mAP50 (Projeto Prático 2)	Usa exatamente o mesmo limiar de IoU (0,5) para decidir se uma detecção final “acerta” a caixa verdadeira

8.16.4.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Inteiro N e real τ .
- Próximas N linhas: cinco reais $x_1 \ y_1 \ x_2 \ y_2 \ \text{score}$.

Saída:

- Uma linha por caixa mantida, na ordem de seleção: `índice score` (score com 4 casas decimais).
- Última linha: `Total mantidas: X`.

8.16.4.5 Exemplos

Entrada	Saída	Observação
3 0.5	0 0.9000	A caixa 1 é suprimida por sobrepor fortemente a caixa 0 (IoU 0,68 > 0,5).
0 0 10 10 0.9	2 0.8000	
1 1 11 11 0.75	Total mantidas: 2	
50 50 60 60 0.8		
2 0.5	1 0.9500	A caixa 0 é suprimida por sobrepor demais a vencedora (IoU=0,6 > 0,5), mesmo tendo menor score de origem que a caixa 1, aqui a de maior score.
0 0 10 10 0.9	Total mantidas: 1	
0 0 10 6 0.95		



Figura 8.38: Simulador: IoU e Supressão de Não-Máximos

```
%%writefile EP09_04.py
# Código Python

Writing EP09_04.py

TestSuite("EP09_04.py").run()
```

8.16.5 EP09_05 Pipeline Integrado: Da Detecção à Medição do Mundo Real

Este exercício final integra os dois exercícios anteriores ao princípio de **fotogrametria** apresentado na seção “Integrando Geometria e Aprendizado Profundo” — exatamente o mesmo cálculo implementado na figura de medição por referência de escala deste capítulo. O cenário reproduz uma situação realista: um detector (Faster R-CNN ou YOLO) gera **várias caixas candidatas sobrepostas** para o mesmo objeto de interesse;

após filtrá-las por NMS, a caixa sobrevivente de maior confiança é usada, junto de uma caixa de referência de largura real conhecida (como o cartão de 8,56 cm), para estimar as dimensões reais do objeto detectado.

8.16.5.1 Diretrizes de Implementação

1. **Referência conhecida:** Ler o valor real L_{ref} (largura real do objeto de referência, em cm) e, em seguida, os quatro reais $x_1 \ y_1 \ x_2 \ y_2$ de sua caixa delimitadora em pixels (já conhecida, sem necessidade de detecção).
2. **Candidatas do objeto a medir:** Ler o inteiro N (número de caixas candidatas produzidas pelo detector para o objeto de interesse) e o limiar real τ ; em seguida, ler as N linhas de caixas candidatas, cada uma com $x_1 \ y_1 \ x_2 \ y_2 \ \text{score}$.
3. **Etapa 1 — NMS:** Aplique exatamente o algoritmo de Supressão de Não-Máximos do EP09_04 às N caixas candidatas, usando o limiar τ , para eliminar detecções redundantes do mesmo objeto.
4. **Etapa 2 — Seleção da caixa final:** Após o NMS, a caixa de maior **score** entre as mantidas é a detecção final do objeto (a entrada garante que todas as caixas candidatas correspondem a um único objeto físico, portanto a primeira caixa selecionada pelo NMS já é o resultado final).
5. **Etapa 3 — Medição por referência de escala:** Calcule a razão $\text{cm/pixel} = L_{ref}/\text{largura da referência em pixels}$ e aplique-a tanto à largura quanto à altura (em pixels) da caixa final do objeto, obtendo suas dimensões reais estimadas em centímetros.
6. **Saída:** Primeiro, uma linha por caixa mantida após o NMS (mesmo formato do EP09_04): **índice score**. Em seguida, a linha **Total mantidas: X**. Por fim, a linha **Objeto: L x A cm**, onde L e A são a largura e a altura estimadas do objeto, cada uma com 2 casas decimais.

8.16.5.2 Restrições Computacionais

- **Reaproveite o NMS do EP09_04** integralmente — mesma regra de desempate, mesmo critério de supressão ($\text{IoU} > \tau$).
- **A referência não passa por NMS:** sua caixa é dada diretamente, sem candidatas concorrentes.
- **Razão única para largura e altura:** assim como na figura de fotogrametria do capítulo, a mesma razão cm/pixel (derivada da largura da referência) é aplicada tanto à largura quanto à altura do objeto — não há calibração vertical separada.

8.16.5.3 Fundamentação Teórica

Etapa	Conceito do capítulo
Múltiplas caixas candidatas	Saída bruta de um detector como o Faster R-CNN ou o YOLO, antes do pós-processamento
NMS (EP09_04)	Filtra a redundância, mantendo apenas a detecção mais confiável do objeto
Referência de escala conhecida	Mesmo princípio da fotogrametria: um objeto de dimensão real conhecida converte pixels em centímetros
Razão cm/pixel	Fator de conversão único, assumindo que a câmera está aproximadamente perpendicular à cena (sem correção de perspectiva)

8.16.5.4 Especificação de Entrada e Saída (VPL)

Entrada:

- Linha 1: Real L_{ref} .
- Linha 2: Quatro reais $x_1 \ y_1 \ x_2 \ y_2$ (caixa de referência).
- Linha 3: Inteiro N e real τ .
- Próximas N linhas: cinco reais $x_1 \ y_1 \ x_2 \ y_2 \ \text{score}$ (caixas candidatas do objeto).

Saída:

- Uma linha por caixa mantida após NMS: **índice score** (score com 4 casas decimais).
- Linha Total mantidas: **X**.
- Linha final: Objeto: L x A cm (2 casas decimais cada).

8.16.5.5 📌 Exemplos

Entrada	Saída	Observação
8.56	0 0.9200	A caixa 1 é suprimida por sobrepor fortemente a caixa 0; a detecção final do objeto é a caixa 0.
30 200 170 288	2 0.4000	
3 0.5	Total mantidas: 2	
250 100 470 250 0.92	Objeto: 13.45 x 9.17 cm	
255 105 468 245 0.88		
600 600 650 650 0.40		

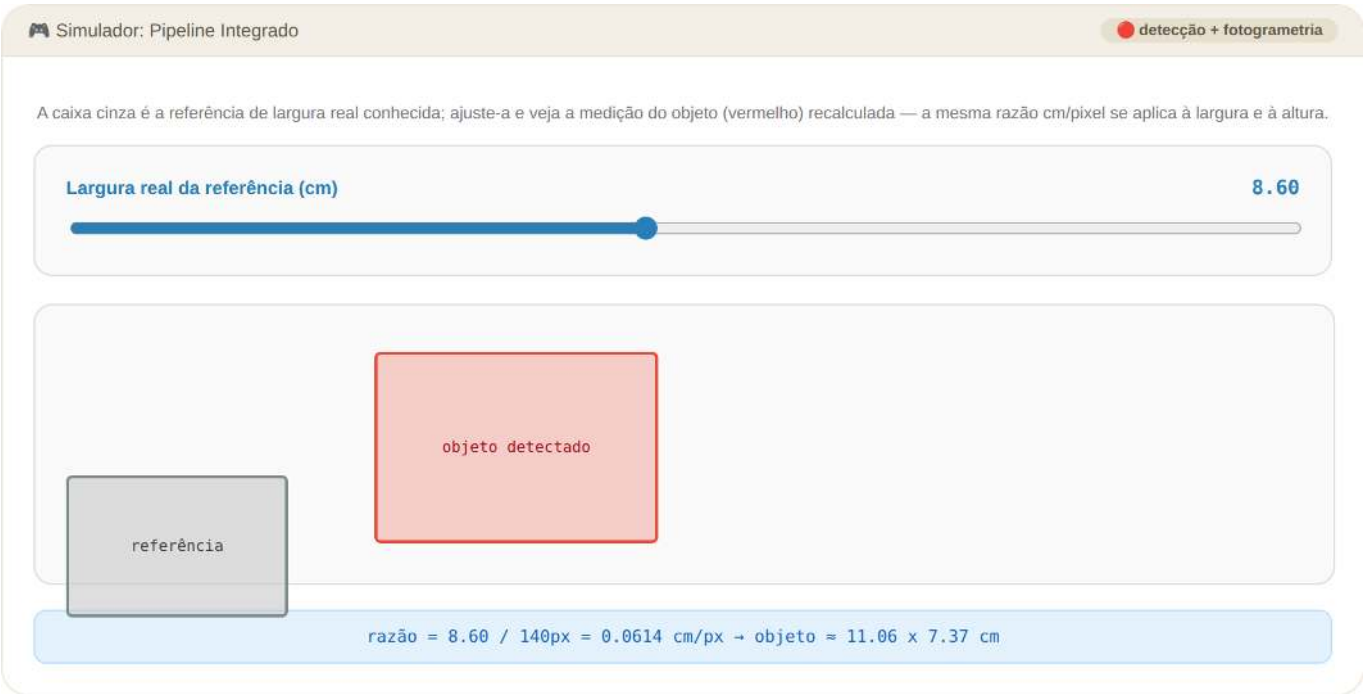


Figura 8.39: Simulador: Pipeline Integrado — NMS + Medição por Referência de Escala

```
%%writefile EP09_05.py
# Código Python

Writing EP09_05.py

TestSuite("EP09_05.py").run()
```

Referências

- BAHDANAU, Dzmitry; CHO, Kyunghyun; BENGIO, Yoshua. Neural Machine Translation by Jointly Learning to Align and Translate. *In*: 2015. Disponível em: <<https://arxiv.org/abs/1409.0473>>
- BARRERA, Junior *et al.* MMach: a mathematical morphology toolbox for the KHOROS system. **Journal of Electronic Imaging**, v. 7, n. 1, p. 174–210, 1998.
- BERGMANN, Paul *et al.* [MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection](#). *In*: 2019.
- BISHOP, Christopher M. **Pattern Recognition and Machine Learning**. New York, NY, USA: Springer, 2006.
- BRADSKI, Gary; KAEHLER, Adrian. **Learning OpenCV: Computer vision with the OpenCV library**. [S.l.]: "O'Reilly Media, Inc.", 2008.
- CHEN, Liang-Chieh *et al.* Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation. *In*: FERRARI, Vittorio *et al.* (orgs.). Cham: Springer International Publishing, 2018.
- COVER, T.; HART, P. [Nearest neighbor pattern classification](#). **IEEE Transactions on Information Theory**, v. 13, n. 1, p. 21–27, 1967.
- DALAL, N.; TRIGGS, B. [Histograms of oriented gradients for human detection](#). *In*: 2005.
- DOUGHERTY, Edward R.; LOTUFO, Roberto A. **Hands-on morphological image processing**. [S.l.]: SPIE press, 2003. v. 59
- DUDA, Richard O.; HART, Peter E.; STORK, David G. **Pattern Classification**. 2nd. ed. New York: Wiley-Interscience, 2001.
- GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing**. 4th. ed. New York: Pearson, 2018.
- GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge, MA, USA: MIT Press, 2016.
- GOOGLE. **NotebookLM**. <https://notebooklm.google.com>, 2025.
- HE, Kaiming *et al.* [Deep Residual Learning for Image Recognition](#). *In*: 2016.
- ITU-R. **Recommendation BT.601: Studio encoding parameters of digital television for standard 4:3 and wide screen 16:9 aspect ratios**. <https://www.itu.int/rec/R-REC-BT.601/>, 2011.
- KINGMA, Diederik P.; BA, Jimmy. Adam: A Method for Stochastic Optimization. *In*: San Diego, CA, USA: 2015. Disponível em: <<https://arxiv.org/abs/1412.6980>>
- KIRILLOV, Alexander *et al.* [Segment Anything](#). *In*: 2023.

- KNUTH, Donald E. [Literate Programming](#). **The Computer Journal**, v. 27, n. 2, p. 97–111, 1984.
- LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. [Deep learning](#). **Nature**, v. 521, n. 7553, p. 436–444, 2015.
- LEWIS, Patrick *et al.* Retrieval-augmented generation for knowledge-intensive nlp tasks. **Advances in neural information processing systems**, v. 33, p. 9459–9474, 2020.
- LIENHART, Rainer; MAYDT, Jochen. [An Extended Set of Haar-like Features for Rapid Object Detection](#). *In*: 2002.
- LOTUFO, R. A.; ZAMPIROLI, F. A. [Fast multidimensional parallel Euclidean distance transform based on mathematical morphology](#). *In*: 2001.
- LOTUFO, Roberto A. *et al.* MMachLib functions and MMach operators. *In*: 1997.
- MATHERON, Georges. **Random Sets and Integral Geometry**. New York: John Wiley & Sons, 1975.
- MCCULLOCH, Warren S.; PITTS, Walter. [A Logical Calculus of the Ideas Immanent in Nervous Activity](#). **The Bulletin of Mathematical Biophysics**, v. 5, n. 4, p. 115–133, 1943.
- OJALA, Timo; PIETIKÄINEN, Matti; MÄENPÄÄ, Topi. [Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns](#). **IEEE Trans. Pattern Anal. Mach. Intell.**, v. 24, n. 7, p. 971–987, jul. 2002.
- OTSU, Nobuyuki. [A Threshold Selection Method from Gray-Level Histograms](#). **IEEE Transactions on Systems, Man, and Cybernetics**, v. 9, n. 1, p. 62–66, 1979.
- PEDREGOSA, Fabian *et al.* Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.
- QUILICI-GONZALEZ, José Artur; ZAMPIROLI, Francisco de Assis. [Sistemas Inteligentes e Mineração de Dados](#). [S.l.]: Editora UFABC, 2014.
- QUILICI-GONZALEZ, José Artur; ZAMPIROLI, Francisco de Assis; SOUZA, Fábio Rezende de. **Sistemas Inteligentes e Mineração de Dados: Do Weka ao Python**. 2. ed. [S.l.]: <https://fzampiroli.github.io/simd2/>, 2026.
- REDMON, Joseph *et al.* [You Only Look Once: Unified, Real-Time Object Detection](#). *In*: 2016.
- REN, Shaoqing *et al.* Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *In*: CORTES, C. *et al.* (orgs.). Curran Associates, Inc., 2015. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf>
- RONNEBERGER, Olaf; FISCHER, Philipp; BROX, Thomas. [U-Net: Convolutional Networks for Biomedical Image Segmentation](#). *In*: : Lecture Notes em Computer Science. Cham, Switzerland: Springer, 2015.
- ROSENBLATT, Frank. [The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain](#). **Psychological Review**, v. 65, n. 6, p. 386–408, 1958.
- SERRA, Jean. **Image Analysis and Mathematical Morphology**. London: Academic Press, 1982. v. 1
- SINGH, Himanshu. **Practical machine learning and image processing**. [S.l.]: Springer, 2019.

- SINGH, Shobhna; LLOYD, Jerome; FLICKER, Felix. [Hamiltonian Cycles on Ammann-Beenker Tilings](#). **Physical Review X**, v. 14, n. 3, p. 031005, 10 jul. 2024.
- SMITH, Ray. [An Overview of the Tesseract OCR Engine](#). In: IEEE Computer Society, 2007.
- SMITH, Ray. [History of the Tesseract OCR Engine: What Worked and What Didn't](#). In: 2013.
- SONG, Kechen; YAN, Yunhui. [A Noise Robust Method Based on Completed Local Binary Patterns for Hot-Rolled Steel Strip Surface Defects](#). **Applied Surface Science**, v. 285, p. 858–864, 2013.
- SZELISKI, Richard. **Computer Vision: Algorithms and Applications**. 2. ed. [S.l.]: Springer, 2022.
- TABERNIK, Domen *et al.* [Segmentation-Based Deep-Learning Approach for Surface-Defect Detection](#). **Journal of Intelligent Manufacturing**, v. 31, n. 3, p. 759–776, 2020.
- VASWANI, Ashish *et al.* Attention Is All You Need. In: 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>
- VIOLA, Paul; JONES, Michael. [Rapid Object Detection Using a Boosted Cascade of Simple Features](#). In: IEEE Computer Society, 2001.
- VIOLA, Paul; JONES, Michael J. [Robust Real-Time Face Detection](#). **International Journal of Computer Vision**, v. 57, n. 2, p. 137–154, 2004.
- ZAMPIROLI, Francisco A. **MCTest: Como Criar e Corrigir Exames Parametrizados**. [S.l.]: Independente, 2023.
- ZAMPIROLI, Francisco de Assis *et al.* A Practical Digital Image Processing Course with morph.py. In: Porto Alegre, RS, Brasil: Sociedade Brasileira de Computação (SBC), 2024. Disponível em: <<https://sol.sbc.org.br/index.php/educomp/article/view/28199>>
- ZAMPIROLI, Francisco de Assis *et al.* [Teaching Hands-On Digital Image Processing with morph.py: Methods and Comprehensive Results](#). **Revista Brasileira de Informática na Educação**, v. 33, p. 990–1026, 2025.

Apêndice A

Sobre o MCTest

Em construção!

O **MCTest** é o sistema de código aberto (Zampirolli, 2023), utilizado para a criação e correção automática de exames parametrizados, incluindo questões de múltipla escolha, dissertativas e de programação (estas últimas integradas ao VPL do Moodle, descrito no Apêndice B). A versão do MCTest utilizada neste livro é a 5.4, disponível em código aberto em github.com/fzampirolli/mctest.

A.1 Instalação do MCTest

A instalação recomendada utiliza o VirtualBox com Ubuntu 22.04. No terminal do Ubuntu, como *root*, execute:

```
wget https://raw.githubusercontent.com/fzampirolli/mctest/master/_setup-all.sh
```

Em seguida, substitua *yourLogin* pelo nome do usuário local e execute:

```
sed -i 's/\/home\/fz\/\/home\/yourLogin\/g' _setup-all.sh
source _setup-all.sh
pip install mysqlclient
```

Após alguns minutos, o MCTest estará configurado. Para executá-lo:

```
source /home/yourLogin/PycharmProjects/runDjango.sh
```

O sistema fica então acessível em <http://127.0.0.1:8000>.

A.2 Criando um exame no MCTest

No MCTest, todo exame é configurado em uma tela de **Exame**, que reúne Turmas, Tópicos (associados a uma disciplina, como PDI-VC) e Questões — cada questão pertence a um único tópico. Além dos atributos de banco de dados, o exame possui parâmetros próprios, como o número de questões sorteadas e o número de variações geradas.

No caso das duas provas descritas na *?@sec-contexto-desta-edicao*, foi definido um conjunto de questões parametrizadas por prova, das quais um subconjunto é sorteado aleatoriamente para cada variação, totalizando **110 variações** distintas — uma por estudante matriculado nas turmas.

O botão **Create-Variations** gera um novo conjunto de variações a cada acionamento. Quando as questões estão associadas a atividades VPL, o professor recebe por e-mail um arquivo **linker.json* contendo todos os casos de teste das variações geradas. O botão **Create-PDF** gera, para cada turma, um PDF com as provas sorteadas por estudante, além de um arquivo **students_variations.csv* com nome, e-mail e variação sorteada de cada aluno.

! Atenção

Cada acionamento do botão **Create-Variations** gera um novo conjunto de variações de exames. Após imprimir o PDF, é fundamental **não alterar os atributos da questão**, sob risco de invalidar a correção automática das provas já impressas ou aplicadas.

A.3 Criando uma questão paramétrica

Uma questão paramétrica no MCTest combina três elementos:

1. **Descrição** — o enunciado da questão, escrito em LaTeX, contendo variáveis entre `[[code:variavel]]`, que são substituídas pelo valor sorteado para cada aluno;
2. **Bloco de definição** (`[[def: ... ]]`) — um trecho de código Python, embutido na própria questão, responsável por sortear os parâmetros, calcular a solução de referência e montar os casos de teste;
3. **Casos de teste para o Moodle** (bloco `moodle_cases`) — dicionário serializado em JSON contendo as entradas e saídas esperadas, consumido pela atividade VPL.

Um exemplo simplificado de definição de questão (adaptado da geração de um tabuleiro de xadrez $h \times w$) é:

```
[[def:
import json
import numpy as np

# PARÂMETROS USADOS NA DESCRIÇÃO DA QUESTÃO
height = np.random.randint(5, 15)

def chess(h, w):
    m = np.zeros((h, w), dtype='int')
    for i in range(h):
        for j in range(w):
            if (i + j) % 2:
                m[i][j] = 1
    return m

inp_list, out_list, test_cases = [], [], 4
for i in range(test_cases):
    width = int(np.random.randint(500, 1500) / 100)
    inp = str(width) + '\n'
    out = mm.drawImage(chess(height, width))
    inp_list.append(inp)
    out_list.append(out)

cases = {'input': inp_list, 'output': out_list}
moodle_cases = json.dumps(cases)

caso0_inp = cases['input'][0]
caso0_out = cases['output'][0]
]]
```

O valor de `height` é sorteado uma única vez por variação (para aparecer no enunciado), enquanto `width` é sorteado a cada caso de teste, aumentando a robustez da correção automática. Note o uso do método `mm.drawImage()`, da própria `morph.py`, para formatar a matriz de saída como texto simples — facilitando a comparação exata entre a saída do aluno e o gabarito.

Ao acionar **Create-PDF** na tela da questão, uma nova variação é gerada a cada clique, permitindo conferir o enunciado antes de publicá-lo.

Para as duas provas da disciplina, esse mecanismo foi utilizado de forma mais ampla: cada questão paramétrica define não apenas os valores numéricos, mas também, em alguns casos, pequenas variações estruturais no

enunciado (por exemplo, qual direção cardinal — Norte, Sul, Leste, Oeste — deve ser avaliada), dificultando a busca por soluções prontas na internet ou em ferramentas de IA generativa.

A.4 Exemplo real: uma questão do Simulado 4

Para ilustrar o mecanismo descrito na seção anterior com um caso concreto, apresenta-se a seguir uma questão efetivamente aplicada no **Simulado 4** (tópico *im4-Operadores Morfológicos*, dificuldade 1, taxonomia de Bloom “remember”), do tipo questão de programação parametrizada com integração Moodle+VPL.

A questão pede ao estudante que escreva um programa capaz de:

- ler uma imagem binária, corrompida por ruído sal e pimenta, contendo ao menos dois objetos geométricos isolados;
- aplicar filtragem morfológica (abertura seguida de fechamento) para eliminar o ruído;
- extrair, a partir da imagem filtrada, as medidas geométricas de cada objeto (área, perímetro, centro, bounding box, circularidade, solidez e número de vértices), usando a função auxiliar `mm.measure`;
- ordenar os objetos por posição (coordenada X do bounding box, com Y como critério de desempate) e reatribuir os identificadores sequencialmente;
- imprimir a matriz limpa e a tabela de medidas no formato esperado pelo corretor automático.

No bloco `[[def: ... ]]` correspondente, um gerador de cena (`gerarCena`) sorteia aleatoriamente a altura e a largura da imagem, o tipo, o tamanho e a posição de cada objeto (quadrado, retângulo, triângulo ou bloco), garantindo que eles não se sobreponham, e em seguida acrescenta ruído sal e pimenta com 3% de probabilidade por pixel. Dez casos de teste distintos são gerados dessa forma para cada variação da prova, e o par entrada/saída do primeiro caso é reaproveitado no próprio enunciado como exemplo para o estudante. O texto do enunciado é montado na própria variável Python `texto` e reutilizado tanto no PDF impresso quanto na atividade VPL do Moodle (via `cases['description']`), garantindo que aluno e corretor vejam exatamente a mesma redação. A biblioteca de morfologia (`mm`) é importada diretamente de `morph.py`, também disponível como módulo utilitário do próprio MCTest.

Em resumo, o enunciado pede ao estudante um programa que:

1. leia, na primeira linha, dois inteiros H e W (altura e largura da imagem);
2. leia as H linhas seguintes da matriz binária (0s e 1s separados por espaço), usando `mm.readImg(H, W)`;
3. aplique filtragem morfológica para remover o ruído sal e pimenta;
4. imprima a matriz já filtrada, em 0s e 1s separados por espaço;
5. extraia as medidas geométricas dos objetos com `mm.measure(imgLimpa)`;
6. ordene os objetos e imprima a tabela de medidas no formato esperado.

O enunciado ainda traz duas observações importantes para a correção automática: (i) a área calculada pelo OpenCV (`cv2.contourArea`) corresponde ao polígono contínuo delimitado pelos centros dos pixels de borda, sendo por isso menor do que a simples contagem de pixels 1 (`np.sum`); e (ii) a lista de objetos deve ser ordenada em ordem crescente pela coordenada X do bounding box (`bbox[0]`), usando a coordenada Y (`bbox[1]`) como critério de desempate, com os IDs reatribuídos sequencialmente de 1 a N após a ordenação.

Na questão, o texto do enunciado não é fixo: ele é montado dentro do próprio bloco `[[def: ... ]]`, na variável Python `texto`, e inserido na descrição via `[[code:texto]]`. Isso permite, por exemplo, trocar frases, valores ou até a regra de ordenação em variações futuras da questão sem tocar no restante do código. A estrutura real utilizada (com trechos de sorteio de cenário omitidos por brevidade) é:

```
[[code:texto]]

\vspace{2mm}\noindent\textbf{Exemplo de Entrada:}\vspace{-2mm}\
\begin{verbatim}
[[code:caso0_inp]]
\end{verbatim}
\vspace{-2mm}\noindent\textbf{Exemplo de Saída:}\vspace{-2mm}\
\begin{verbatim}
```

```

[[code:caso0_out]]
\end{verbatim}

\begin{comment}
[[code:moodle_cases]]
\end{comment}

[[def:
import json
import numpy as np
import cv2
from morph import mm # também disponível como módulo utilitário do MCTest

# ... geração da cena, do ruído e dos 10 casos de teste (inplist/outlist) ...

# ENUNCIADO COMPLETO, COM EXPLICAÇÃO DE ÁREA E DE ORDENAÇÃO
texto = (
    "Uma imagem binária corrompida por ruído sal e pimenta contém no mínimo dois objetos geométricos iso
    "Escreva um programa que:\n"
    "1. Leia dois inteiros ..."
)

cases = {}
cases['skills']      = ["morfologia matematica", "remocao de ruido", "mm.measure", "tabulacao"]
cases['description'] = [{"text": texto}] # aparece no Moodle, diferente para cada aluno!
cases['input']       = np.array(inplist).tolist()
cases['output']      = np.array(outlist).tolist()

moodle_cases = json.dumps(cases)
caso0_inp = cases['input'][0]
caso0_out = cases['output'][0]
]]

```

Note que `cases['description']` é o mesmo texto que alimenta tanto o PDF impresso quanto a atividade VPL no Moodle — como `texto` é gerado dentro do bloco `[[def: ...]]`, ele pode variar de aluno para aluno junto com os demais parâmetros sorteados, embora nesta questão específica apenas os dados numéricos (imagem, casos de teste) mudem entre variações.

Abaixo, a fotografia da questão efetivamente gerada e impressa para uma das 110 variações do Simulado 4, mostrando o enunciado, a imagem de entrada ruidosa e o exemplo de saída esperado (imagem filtrada e tabela de medidas):

A.5 Considerações finais

Este apêndice resumiu os passos para instalar o MCTest, criar exames e construir questões parametrizadas com integração ao VPL do Moodle. O uso combinado de variáveis sorteadas na descrição (`[[code: ...]]`) e de código Python embutido na própria questão (`[[def: ... ]]`) permite gerar centenas de variações de prova a partir de um único modelo, cada uma corrigida automaticamente e de forma consistente. O exemplo real do Simulado 4 (Seção A.4) ilustra como esse mecanismo é usado na prática para gerar questões de programação com cenários visuais distintos por aluno. O Apêndice B detalha como essas variações são publicadas e corrigidas no Moodle, e o Apêndice C descreve como restringir o acesso às provas por meio do SEB.

1. Uma imagem binária corrompida por ruído sal e pimenta contém no mínimo dois objetos geométricos isolados. Escreva um programa que: 1. Leia dois inteiros na primeira linha representando a altura (H) e largura (W) da imagem. 2. Leia as H linhas seguintes da matriz binária (valores 0 e 1 separados por espaço) usando `mm.readImg(H, W)`. 3. Aplique a filtragem morfológica para eliminar o ruído sal e pimenta. 4. Imprima a matriz limpa diretamente em valores 0 e 1 (0s e 1s separados por espaço). 5. Extraia as medições geométricas dos objetos usando `mm.measure(imgLimpa)`. 6. Ordene a lista de objetos e imprima a tabela formatada das medidas.
- Observação sobre a área: A área calculada pelo OpenCV (`cv2.contourArea`) mede o polígono contínuo delimitado pelos centros dos pixels de borda, resultando em um valor menor do que a simples contagem discreta de pixels 1 (`np.sum`).
- Regra de Ordenação: A lista de objetos deve ser ordenada em ordem crescente pela posição X do retângulo delimitador (`bbox[0]`), utilizando a posição Y (`bbox[1]`) como critério de desempate caso dois objetos comecem na mesma coluna. (`medidas.sort(key=lambda m: (m['bbox'][0], m['bbox'][1]))`). Em seguida, reatribua os IDs sequencialmente de 1 a N.

Exemplo de Entrada:

[illegible]

Exemplo de Saída:

[illegible]

Figura A.1: Questão real gerada pelo MCTest para o Simulado 4 — tópico “Operadores Morfológicos”.

Apêndice B

Sobre o Moodle (VPL)

Em construção!

Este apêndice descreve a configuração de uma atividade **VPL (Virtual Programming Lab)** no Moodle, plugin utilizado para submissão e correção automática dos Exercícios de Programação (EPs), dos simulados quinzenais e das provas descritas na **?@sec-contexto-desta-edicao**. A versão de Moodle utilizada foi a 4.1.

B.1 Configurando uma atividade VPL

A criação de um exame no MCTest com integração ao VPL (ver [Apêndice A](#)) gera dois arquivos que devem ser renomeados para `linker.json` e `students_variations.csv`, respectivamente. Esses dois arquivos, junto com o `morph.py`, devem ser incluídos nos **Arquivos de Execução** da atividade VPL, além de todos os demais arquivos disponíveis na pasta `VPL_modification` do repositório do MCTest. Todos esses arquivos devem ser marcados como **“Files to keep when running”**.

Em síntese, a atividade VPL passa a conhecer, para cada estudante, qual variação da questão foi sorteada (via `students_variations.csv`) e qual o conjunto de casos de teste correspondente (via `linker.json`), permitindo que a correção seja individualizada mesmo quando dezenas de estudantes resolvem “a mesma” atividade.

B.2 Publicando os EPs como atividades VPL

Os Exercícios de Programação (EPs) apresentados ao final de cada capítulo deste livro também podem ser publicados como atividades VPL, seguindo o mesmo mecanismo. O fluxo típico de uso em sala de aula foi:

1. Abertura dos dois *notebooks* Colab do capítulo (teórico e prático), com destaque para os simuladores interativos;
2. Execução dos blocos de código, alterando parâmetros para reforçar a compreensão dos conceitos;
3. Nas aulas em laboratório, submissão das respostas dos EPs diretamente na atividade VPL correspondente, com retorno imediato de aprovação/reprovação nos casos de teste.

Reaproveitamento dos casos de teste

Os arquivos `.cases` utilizados na validação local (classe `TestSuite`, descrita no corpo deste livro) são compatíveis com o formato esperado pelo VPL, permitindo reaproveitar o mesmo conjunto de testes tanto no desenvolvimento do EP quanto na correção automatizada no Moodle.

Nos simulados quinzenais — aplicados com o SEB (ver [Apêndice C](#)) e com bônus de 5% sobre a nota final —, foram utilizadas atividades VPL com EPs semelhantes aos das listas regulares, porém corrigidas sob restrição de acesso à internet, reforçando o caráter avaliativo da atividade.

B.3 Considerações finais

Este apêndice descreveu a configuração de atividades VPL no Moodle para a correção automática de EPs, simulados e provas parametrizadas. A combinação entre MCTest (geração de variações e casos de teste),

VPL (submissão e correção) e SEB (restrição de acesso) forma o tripé sobre o qual se apoia a avaliação automatizada e individualizada utilizada ao longo da disciplina.

Apêndice C

Uso do SEB nos simulados quinzenais

Em construção!

Além das duas provas com questões parametrizadas (ver [Apêndice A](#)), o SEB também restringiu o acesso aos **simulados quinzenais**, atividades VPL com EPs semelhantes aos das listas regulares, cada uma concedendo um bônus de 5% sobre a nota final. Nesses casos, a configuração da atividade VPL segue o mesmo procedimento descrito neste apêndice, bastando gerar uma chave de exame distinta para cada simulado.

Apêndice D

Geração de material de aula com o Gemini Notebook

Em construção!

Este apêndice descreve o uso do **Gemini Notebook (NotebookLM)** como ferramenta de apoio à preparação das aulas da disciplina, complementando o uso de IA na revisão de texto e código já mencionado no prefácio deste livro.

D.1 Vídeos conceituais e de resolução de EPs

Para cada capítulo, foi criado um projeto no Gemini Notebook tendo como fontes o PDF completo do livro e o arquivo `morph.py`. A partir dessas fontes, o Gemini Notebook gerou automaticamente um vídeo curto (em média 7 minutos), utilizado como abertura de aula. Os capítulos alternaram dois tipos de vídeo:

- **Vídeo conceitual**, apresentando os fundamentos teóricos do capítulo, geralmente exibido na primeira aula dedicada ao tema;
- **Vídeo de resolução de EPs**, apresentando a estratégia de solução dos Exercícios de Programação, exibido na aula seguinte, na qual a maior parte dos EPs era então resolvida em conjunto com a turma.

i Papel do vídeo na aula

O vídeo gerado pelo Gemini Notebook não substitui a explicação do professor: funciona como uma introdução de poucos minutos, situando o estudante no tema antes da apresentação dos slides e da abertura dos *notebooks* Colab.

D.2 Slides de apoio

Complementando o vídeo, o Gemini Notebook também gerou, a partir das mesmas fontes (PDF do livro e `morph.py`), um conjunto de aproximadamente 12 slides por capítulo, cobrindo a parte conceitual e a parte prática. A geração utilizou um *prompt* específico por capítulo, delimitando o escopo do conteúdo a ser sintetizado — evitando que o material do slide antecipasse conteúdo de capítulos posteriores ou reproduzisse trechos extensos do livro sem adaptação didática.

Após a apresentação dos slides, a aula prosseguia com a abertura dos dois *notebooks* Colab do capítulo (teórico e prático), com ênfase nos simuladores interativos e na execução dos blocos de código alterando parâmetros.

D.3 Considerações finais

A geração de vídeos e slides com o Gemini Notebook, a partir do próprio livro e da biblioteca `morph.py`, permitiu padronizar a abertura das aulas e reduzir o tempo de preparação, sem prescindir da curadoria do professor na escolha dos *prompts* e na condução da aula. Como em outros usos de IA generativa descritos neste livro, a responsabilidade pela adequação pedagógica do material permanece do professor.